软件

软件定义 · 中译本

软件定义

软件定义无线电工程师读本

软件定义无线电 · 从算法到硬件

李佳伦译 编著

译自 Software-Defined Radio for Engineers (Artech House 2018, ADI 官网免费分发) · 仅供个人学习

二〇二六年

目录

前言 . . . 1
原书信息与获取 . . . 1
翻译进度 TODO（□ 全书完成：正文 11 章 + 附录 A–D） . . . 1
译本工作流（每章通用） . . . 2
编者 . . . 2
第 1 章 软件定义无线电导论 . . . 3
1.1 简史 . . . 3
1.2 什么是软件定义无线电? . . . 3
1.3 网络与 SDR . . . 7
1.4 SDR 的射频架构 . . . 9
1.5 SDR 的处理架构 . . . 13
1.6 SDR 的软件环境 . . . 14
1.7 延伸阅读 . . . 16
1.8 参考文献 . . . 16
第 2 章 信号与系统 . . . 18
2.1 时域与频域 . . . 18
2.2 采样理论 . . . 23
2.3 信号表示 . . . 39
2.4 信号指标与可视化 . . . 44
2.5 SDR 接收技术 . . . 49
2.6 SDR 数字信号处理技术 . . . 64
2.7 SDR 发射技术 . . . 78
2.8 本章小结 . . . 91

2.9 参考文献 . . . 92
第 3 章 通信中的概率 . . . 94
3.1 通信系统中离散随机事件的建模 . . . 94
3.2 二元通信信道与条件概率 . . . 98
3.3 通信系统中连续随机事件的建模 . . . 102
3.4 通信系统中的时变随机性 . . . 109
3.5 高斯噪声信道 . . . 112
3.6 功率谱密度与 LTI 系统 . . . 116
3.7 窄带噪声 . . . 119
3.8 随机变量的应用：室内信道模型 . . . 120
3.9 本章小结 . . . 121
3.10 延伸阅读 . . . 121
3.11 参考文献 . . . 122
第 4 章 数字通信基础 . . . 123
4.1 什么是数字传输? . . . 123
4.2 数字调制 . . . 135
4.3 误比特概率 . . . 151
4.4 信号空间概念 . . . 157
4.5 Gram-Schmidt 正交化 . . . 160
4.6 最优检测 . . . 165
4.7 基本接收机实现 . . . 170
4.8 本章小结 . . . 177
4.9 延伸阅读 . . . 179
4.10 参考文献 . . . 180
第 5 章 理解 SDR 硬件 . . . 181
5.1 通信系统的组成 . . . 181
5.2 MATLAB 开发策略 . . . 191
5.3 示例：真实数据自环 . . . 197
5.4 噪声系数 . . . 198
5.5 参考文献 . . . 200
第 6 章 定时同步 . . . 202
6.1 匹配滤波 . . . 203
6.2 定时误差 . . . 206

6.3 符号定时补偿 . . . 211
6.4 替代误差检测器与系统要求 . . . 220
6.5 组装成整体 . . . 221
6.6 本章小结 . . . 224
6.7 参考文献 . . . 224
第 7 章 载波同步 . . . 226
7.1 载波偏移 . . . 226
7.2 频偏补偿 . . . 230
7.3 相位模糊 . . . 243
7.4 本章小结 . . . 245
7.5 参考文献 . . . 245
第 8 章 帧同步与信道编码 . . . 247
8.1 帧，你在哪里? . . . 247
8.2 帧同步 . . . 248
8.3 组装成整体 . . . 259
8.4 信道编码 . . . 261
8.5 本章小结 . . . 269
8.6 参考文献 . . . 269
第 9 章 信道估计与均衡 . . . 271
9.1 多径，休想！ . . . 271
9.2 信道估计 . . . 272
9.3 均衡器 . . . 279
9.4 接收机实现 . . . 285
9.5 本章小结 . . . 286
9.6 参考文献 . . . 287
第 10 章 正交频分复用 . . . 288
10.1 MCM 的动机：色散信道环境 . . . 288
10.2 OFDM 一般模型 . . . 290
10.3 常见 OFDM 波形结构 . . . 293
10.4 分组检测 . . . 296
10.5 CFO 估计 . . . 298
10.6 符号定时估计 . . . 301
10.7 均衡 . . . 303

10.8 比特与功率分配 . . . 307
10.9 组装成整体 . . . 309
10.10 本章小结 . . . 309
10.11 参考文献 . . . 309
第 11 章 SDR 的应用 . . . 312
11.1 认知无线电 . . . 312
11.2 车联网 . . . 318
11.3 本章小结 . . . 321
11.4 参考文献 . . . 321
第 12 章 附录 A 更长的通信史 . . . 325
12.1 A.1 历史总览 . . . 325
12.2 A.2 1750–1850：工业革命 . . . 326
12.3 A.3 1850–1945：技术革命 . . . 327
12.4 A.4 1946–1960：喷气时代与太空时代 . . . 330
12.5 A.5 1970–1979：信息时代 . . . 332
12.6 A.6 1980–1989：数字革命 . . . 334
12.7 A.7 1990–1999：公共互联网时代（Web 1.0） . . . 336
12.8 A.8 2000 之后：万物汇聚 . . . 339
12.9 参考文献 . . . 339
第 13 章 附录 B MATLAB 与 Simulink 入门 . . . 348
13.1 B.1 MATLAB 简介 . . . 348
13.2 B.2 有用的 MATLAB 工具 . . . 348
13.3 B.3 系统对象 . . . 351
13.4 参考文献 . . . 352
第 14 章 附录 C 均衡器推导 . . . 354
14.1 C.1 线性均衡器 . . . 354
14.2 C.2 迫零均衡器 . . . 356
14.3 C.3 判决反馈均衡器 . . . 357
第 15 章 附录 D 三角恒等式 . . . 358

前言

本书是 *Software-Defined Radio for Engineers*（Travis F. Collins, Robin Getz, Di Pu, Alexander M. Wyglinski 著，Artech House 2018，ISBN-13: 978-1-63081-457-1）的中文学习译本。原书由 Analog Devices（ADI）以永久电子书许可在其官网免费分发，配套硬件为 ADALM-Pluto SDR。

[注意] 版权与用途说明：原书为 Artech House 版权作品，本译本仅为个人学习研究用途的笔记整理，不替代原书；请勿用于商业用途或公开再分发。翻译以"通顺可读"为先，技术术语尽量遵循国内通行译法，并保留必要的英文原词。

原书信息与获取

- 原书页面：`https://www.analog.com/en/resources/technical-books/software-defined-radio-for-engineers.html`
- 原书 PDF（英文原版）已批量下载至本书目录下的 `pdf/`（已 gitignore，不入库）：整本书 `SDR4Engineers.pdf`（375 页）+ 11 章分章 PDF + 4 个附录 PDF。
- 重新下载方法见 `tools/download_pdfs.sh`。

翻译进度 TODO（□ 全书完成：正文 11 章 + 附录 A–D）

☒ 第 1 章　软件定义无线电导论（9 图）
☒ 第 2 章　信号与系统（54 图，全书最长、公式最密）
☒ 第 3 章　通信中的概率（11 图）
☒ 第 4 章　数字通信基础（26 图）
☒ 第 5 章　理解 SDR 硬件（8 图）
☒ 第 6 章　定时同步（17 图）

☒ 第 7 章　载波同步（10 图）
☒ 第 8 章　帧同步与信道编码（14 图）
☒ 第 9 章　信道估计与均衡（13 图）
☒ 第 10 章　正交频分复用（12 图）
☐ 第 11 章　SDR 的应用（Applications for Software-Defined Radio）
☒ 附录 A　更长的通信史（6 图）
☒ 附录 B　MATLAB 与 Simulink 入门（1 图）
☒ 附录 C　均衡器推导
☒ 附录 D　三角恒等式

译本工作流（每章通用）

1. `python3 tools/extract_figures.py pdf/SDR4Engineers_CH01.pdf 1 images/` —— 从原书 PDF 提取插图与英文题注（`images/captions_01.tsv`）；
2. 横躺（原书旋转 90° 排版）的插图用 `sips -r 90 images/fig_XXXX.png` 转正；
3. 按 PyMuPDF 提取的章节文本（`doc[p].get_text('text', sort=True)`）翻译成 `Chapter_XX_*.md`，插图以 `![](images/fig_XXYY.png)` 原位嵌入；
4. `python3 build.py` —— Pandoc + Tectonic 编译成 PDF 成书。

编者

李佳伦　译。基于 ADI 官网免费分发的英文原版 PDF 翻译整理，插图取自原书。

第 1 章 软件定义无线电导论

数千年间，人类的通信形式不断演进。口中说出的话语可以从一个人嘴里发出，被另一个人听到或接收。直到现代，各地的街头公告员（town crier）还会举行一年一度的比赛，看谁能在最远的距离上喊出一条听得懂的消息 [1]。然而，“最响公告员” 的世界纪录虽然有 112.8 分贝，能被人听懂的距离却不到 100 米。想比喊叫更高效地交流——这个愿望和语言本身一样古老。

随着计算技术、数字信号处理与数字通信算法、人工智能、射频（RF）硬件设计、网络拓扑以及其他众多领域的现代进展，现代通信系统已经演化为复杂、智能、高性能的平台：它们能适应运行环境，实时、无误地传递海量信息。通信系统技术的最新一步，就是软件定义无线电（software-defined radio，SDR）——它吸收各领域的最新成果，打造出终极的发射机与接收机。

1.1 简史

SDR 技术背后的这段演进史颇为精彩，图 1.1 给出了过去四个世纪中若干重要里程碑的时间线。这段历史的主旋律，是人们研究想法与概念、发表成果，再让同行与同事在他们的工作之上继续建造。其中许多人把成果变成了商业产品，名利双收；也有一些人两者皆无。与 SDR 技术相关的主要里程碑的完整清单，有兴趣的读者可以参阅附录 A。

1.2 什么是软件定义无线电?

每个专业组织都会尝试建立一套通用的术语与定义框架，让从事相似研究或产品开发的专业人士之间便于交流。无线通信与 SDR 领域也不例外。IEEE P1900.1 工作组制定了以下定义，确保领域内的每个人都使用一致的术语 [2]:

无线电（Radio） 1. 以无线方式发射或接收电磁辐射、以实现信息传递的技术。2.

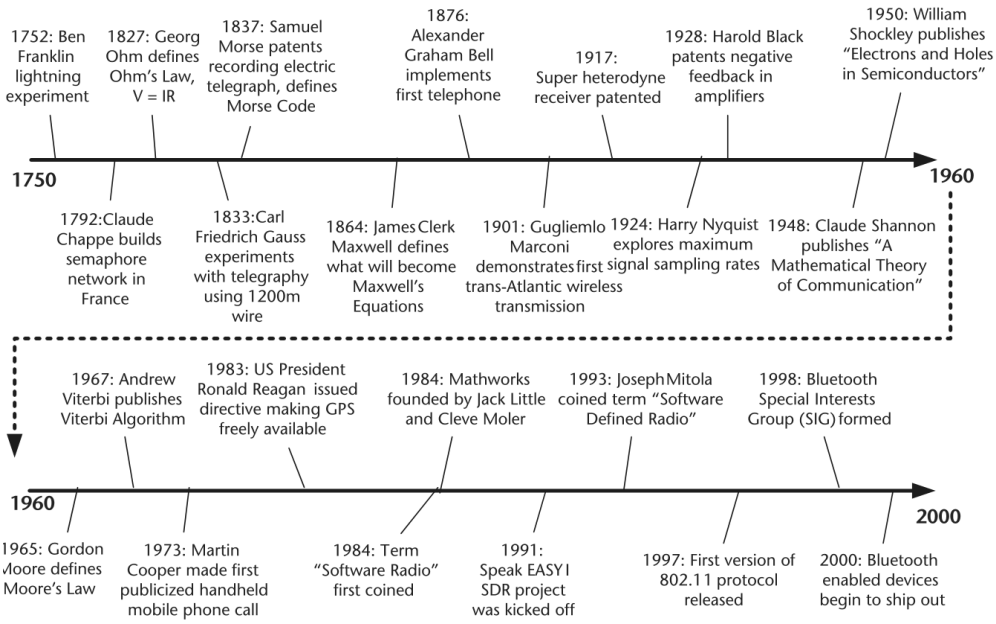


图 1.1　通信领域若干关键里程碑的时间线

包含定义（1）所述技术的系统或设备。3. 泛指无线电波的使用。

无线电节点（Radio Node）一个包含无线电发射机或接收机的无线接入点。

软件（Software）由可编程处理设备执行的可修改指令。

物理层（Physical Layer）无线协议中对射频（RF）、中频（IF）或基带信号（含信道编码）进行处理的层。它是 ISO 七层模型针对无线发射与接收改造后的最底层。

数据链路层（Data Link Layer）通过恰当的差错检测与控制流程以及介质访问控制，负责在无线链路上可靠地传输帧的协议层。

软件控制（Software Controlled）指在无线电系统或设备内使用软件处理来**选择**运行参数。

软件定义（Software Defined）指在无线电系统或设备内使用软件处理来**实现**操作功能（而非控制功能）。

软件控制无线电（Software Controlled Radio）部分或全部物理层功能由软件控制的无线电。

软件定义无线电（Software-Defined Radio, SDR）部分或全部物理层功能由软件定义的无线电。

发射机（Transmitter）为进行无线电通信而产生射频能量的装置。

接收机（Receiver）接收无线电信号并从中提取出信息的设备。

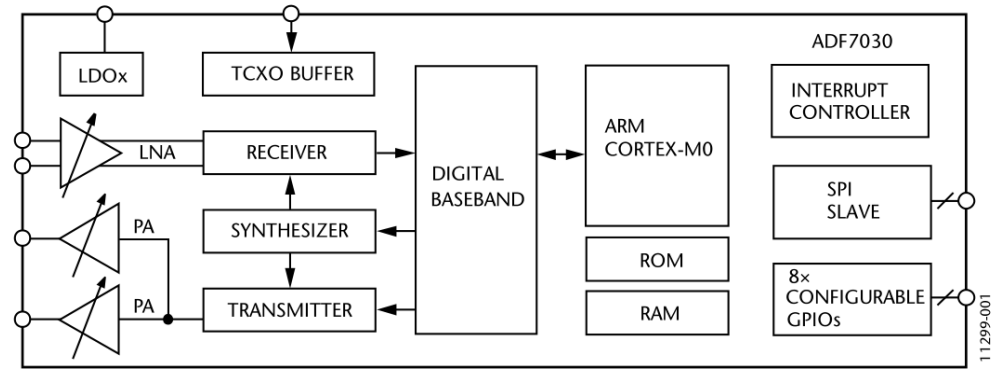


图 1.2　ADF7030 框图 [3]

空中接口（Air Interface）用于在两个无线电终端之间建立通信的波形功能子集。它是无线物理层与无线数据链路层在波形层面的对应物。

波形（Waveform） 1. 施加于待传信息的一组变换，以及将接收信号还原为其信息内容所对应的逆变换集合。2. 信号在空间中的表示。3. 已发射射频信号的表示，外加可选的直至全部网络层的其他无线电功能。

数字处理与模拟射频的组合向来是通信系统的构成基础。在现代系统中，信号处理已经发展到这样的程度：大部分基带功能都在软件中实现。射频硬件可被重新调配、重新配置的灵活性，使得一个射频前端就能应对绝大多数射频系统——通常这个射频前端是**软件控制**的，而非软件**定义**的。这种灵活射频前端与软件信号处理的现代组合，催生了软件定义无线电的诞生。

这一点可以从 Analog Devices 的 ADF7030 这类器件中看出来，如图 1.2 所示。ADF7030 是一款低功耗、高性能的集成射频收发机，支持 169.4–169.6 MHz ISM 频段的窄带工作。它使用 2GFSK 调制支持 2.4 kbps 与 4.8 kbps 的收发操作，并使用 4GFSK 调制支持 6.4 kbps 的发射操作。它包含一个片上 ARM Cortex-M0 处理器，负责无线电控制、校准以及分组管理。有了它，再配上一颗传感器，就足以支撑智能抄表或有源标签资产追踪这类应用了。而这只是摩尔定律的一个副产品——系统级集成。

一套 SDR 系统是个复杂装置，需要同时执行多项复杂任务，才能实现数据的无缝收发。一般而言，数字通信系统由一系列相互依赖的操作组成：取某种信息——无论是人声、音乐还是视频图像——将其经空中接口发射给接收机，再经处理与解码还原为原始信息信号的重建版本。如果原始信息是模拟的（如音频），必须先用量化等技术将其数字化，得到二进制表示。变成二进制之后，发射机对这些信息进行数字处理，将其转换为电磁正弦波形；该波形由其物理特性唯一确定，如信号幅度、载波频率与相位。在通信链路的另一端，接收机的任务是正确识别经可能充满噪声与失真的信道传来的已调

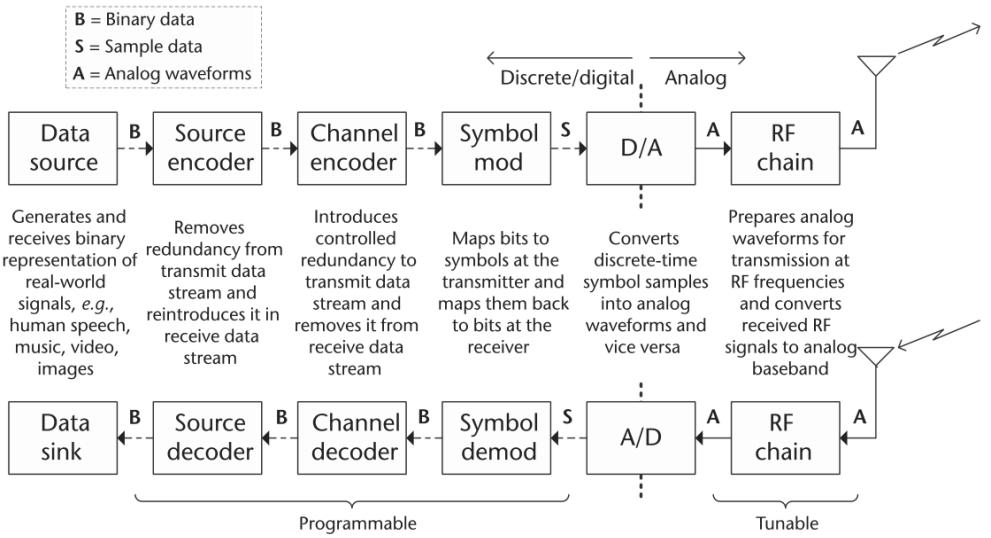


图 1.3　构成现代数字通信系统的若干重要组成部分示意图。注意对基于 SDR 的实现而言，标注为可编程的那些组件既可以用可编程逻辑实现，也可以用软件实现

波形的物理特性，并最终把截获的信号还原为正确的二进制表示。数字通信系统的基本组成框图见图 1.3。

图 1.3 显示，发射机的输入与接收机的输出分别来自数字信源（digital source）与送往数字信宿（digital sink）。这两个模块代表了待通信数字信息的来源与去处。二进制信息进入发射机后，要做的第一件事是移除信息中所有冗余/重复的二进制模式，以提高传输效率。这由**信源编码器**（source encoder）完成，其设计目标就是剥离信息中的全部冗余。注意在接收机一侧，信源译码器会把冗余重新引入，将二进制信息恢复为原始形态。发射机移除冗余之后，接着由**信道编码器**（channel encoder）向信息流中引入数量受控的冗余，以保护信息在穿越噪声信道的过程中免受潜在差错的影响。信道译码器则负责移除这部分受控冗余，把二进制信息还原回原样。发射机的下一步，是将二进制信息映射为独特的电磁波形属性，如幅度、载波频率与相位。这个映射过程称为**调制**（modulation）。相应地，接收机一侧的**解调**过程把电磁波形转换回对应的二进制表示。最后，调制模块输出的离散采样经重采样后，由数模转换器（DAC）转换为基带模拟波形，再由通信系统的射频前端（RFFE）处理并上变频到射频载波频率。接收机则执行相反的操作：截获的模拟信号先由 RFFE 下变频到基带频率，再由模数转换器（ADC）采样与处理。

考虑到 SDR 平台及其组件的复杂性，理解特定 SDR 平台的限制、以及各种设计决策会如何影响原型系统的性能，是至关重要的。例如，对于频谱感知与捷变发射这类要求高计算吞吐与低时延的操作，人们非常希望有实时基带处理。然而，如果 SDR 平台

所采用的微处理器不够强大、无法支撑数字通信系统的运算需求，就必须重新考虑整个收发信机的设计，或降低对低时延高吞吐的要求。否则，SDR 实现将无法正常工作，产生传输差错与糟糕的通信性能。

基于 SDR 平台设计数字通信系统时，需要考虑的设计要点包括：

- 通过在嵌入式处理器上实现实时协议，把物理层与网络层集成起来。注意，大多数通信系统都按逻辑分层（见 1.3 节），以便更容易地设计通信系统；但由于各层之间强烈的相互依赖，每一层都必须被正确设计。
- 确保射频前端具备足够宽的带宽、在多个子信道上的捷变性、以及可扩展的天线数量以进行空间处理。既然许多先进的通信系统设计都用到了多天线与宽带传输，弄清 SDR 硬件在这些物理属性上的能力边界是很重要的。
- 许多采用数字通信系统的网络拥有集中式架构来控制整个网络的运作（例如控制信道的实现）。了解无线网络的架构很重要，因为它决定了一台数字收发信机要与另一台通信时哪些操作是必不可少的。
- 能够在不同环境（如阴影与多径、室内与室外）中进行**受控实验**，对于证明某个 SDR 实现的可靠性很重要。换句话说，如果一个涉及 SDR 原型系统的实验在完全相同的环境、完全相同的运行参数下连续做两次，其输出与性能应当相同。因此，能做受控实验，就为 SDR 设计者提供了一种" 健全性检查" 的能力。
- 通过面向算法与协议描述的软件设计流程，实现可重配置与快速原型化。

如今许多日常使用的通信系统，已不再使用固定的模拟处理与固定电路，而是采用基于微电子的灵活中频（IF）、数字信号处理器、可编程数字逻辑、加速器以及其他类型的计算引擎来实现。为了利用计算引擎的新进展，人们转而使用 MATLAB® 和 Simulink 这样的高级语言，而非 C 或汇编。计算技术的这一转变，催生了新的通信功能与能力，例如先进的卫星通信、移动通信、数据调制解调器以及数字电视广播。

1.3 网络与 SDR

随着数字通信系统演化为高度复杂的设备，人们清楚地认识到：要让这类系统的设计与实现可行且可控，需要分而治之的策略。于是，研究者们把数字通信系统划分为一组互补的层（layer），每一层执行特定的功能，作为整个信息收发过程的一部分。得益于这种分而治之的策略，通信系统迅速演化为能力强大的平台，执行着从网页浏览、电子邮件到流媒体内容的各种业务。这个策略如此成功，以至于出现了只专注于其中一层、完全不碰其他层的研究群体——他们对上下层来的信息照单全收。

一般而言，把通信系统划分为层有两种模型：开放系统互连（OSI）七层模型与传

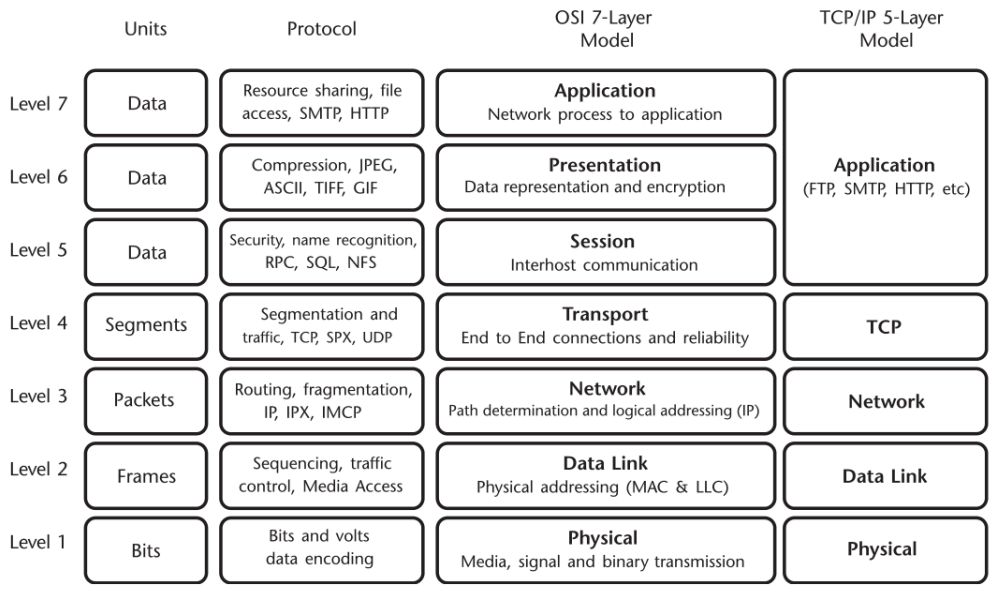


图 1.4　七层 OSI 模型与五层 TCP/IP 模型的对比

输控制协议（TCP）/网际协议（IP）五层模型，如图 1.4 所示。两个模型的功能大致相同，只是 TCP/IP 模型把最上面的几层合并成了一层。聚焦于 TCP/IP 五层模型，自顶向下依次为：

- **应用层（Application Layer）**：让用户与来自通信系统的数据交互。例如，应用层包含来自或发往网页浏览器、电子邮件客户端、流媒体接口等软件的数据。这些应用通常通过指定的套接字（socket）寻址。
- **传输层（Transport Layer）**：负责在客户端应用与服务器应用之间运送应用层报文，确保可靠的数据传输。
- **网络层（Network Layer）**：负责把网络层分组从一台主机搬到另一台主机。它定义数据报（datagram）的格式、端系统与路由器如何处理数据报，以及数据报在源与目的地之间选取的路由。
- **链路层（Link Layer）**：处理两个或多个直连设备之间的数据交换问题。可靠性方面，它包括差错检测、纠错，以及对不同通信系统的寻址。
- **物理层（Physical Layer）**：把一个个比特从一台通信系统直接送往另一台通信系统。它还涵盖数据发送设备与传输介质之间的物理接口。

从无线电系统及其实现的视角看，系统设计的重心在物理层（本书亦然），但链路层对物理层的影响也不能忘记。不过，既然基带处理全部在软件中完成，通信系统同样可以在软件中实现协议栈的更高层。许多通信标准已经采用了这种方案——整个协议栈

所有层的通信系统全部由软件实现，尽管依数据速率要求不同，这可能要求系统具备相当强的计算能力才能实时运行。全软件实现让功能升级无需更换硬件。实践中，这通常只出现在新兴标准或数据速率相对较低的场合。然而，不妨想象一下：给 Wi-Fi 路由器做一次软件升级，就能支持下一代标准，而不必换硬件。这种可软件升级的系统更复杂，成本也可能高于固定硬件系统——但消费者愿意为此多付钱吗？历史给出的答案是否定的。对于那种大批量的消费类应用，价位往往才是产品成败最关键的因素。大多数终端消费者在亚马逊上对比琳琅满目的产品时，并不会考虑长期维护与总拥有成本。哪个功能、哪一层放在固定硬件里做、哪个放在灵活的软件里做，是基于出货量、成本、功耗、复杂度等许多因素的工程决策。

近年来，将 SDR 技术与软件定义网络（SDN）相结合的兴趣日益浓厚——后者关注让更高的通信层去适配当前的运行环境。这使得诸如” 依据物理层提供的启发式信息来修改路由” 之类的做法成为可能。自愈型网格网络（self-healing mesh network）就是这类实现的一个例子。

链路层同样会影响无线通信系统的物理层（PHY），如图 1.5 所示。以 802.11（Wi-Fi）为例，PHY 层（第 1 层）实际上进一步细分为物理层收敛协议（PLCP）子层与物理介质相关（PMD）子层。PMD 子层负责通过无线介质在两个站点之间收发物理层数据单元，并将其交给 PLCP；PLCP 则向上对接 MAC 层、各管理实体以及通用管理原语，以最大化数据速率。

在 PHY 层，图 1.4 中标注的单位是比特；但穿越无线与有线链路时，这些数据会以更适合模拟传输的形式编码，称为**符号**（symbol）。图 1.5 中第 1 层标出的前导码（preamble），在接收机处多半永远不会按符号形式解调出来。这样的序列只被 PHY 层用来补偿链路中的非理想性，对上面的层几乎没有意义。然而，对用 SDR 做实现的人来说，这些看似简单的部分恰恰是主角，而帧的其余部分不过是任意数据。

1.4 SDR 的射频架构

下一代通信系统带来了新的挑战，其解决方案已超出单器件优化的范畴。向无线电注入更多软件控制与认知能力，要求射频设计在频率与带宽上更加灵活。为此，需要把静态滤波器去掉、换成可调谐滤波器。类似地，“通用平台” 的概念可以缩短开发时间、降低制造成本，并让系统之间具备更好的互操作性。通用平台要求射频系统能为那些传统上架构迥异的应用同时提供完整性能。最后，未来的平台正把尺寸与功耗推到新的极限：手持电台越来越强、越来越复杂，却同时要求更好的电池效率；小型无人机（UAV）没有大型飞机的发电能力，射频系统每消耗一毫瓦，都直接换算成载荷电池的

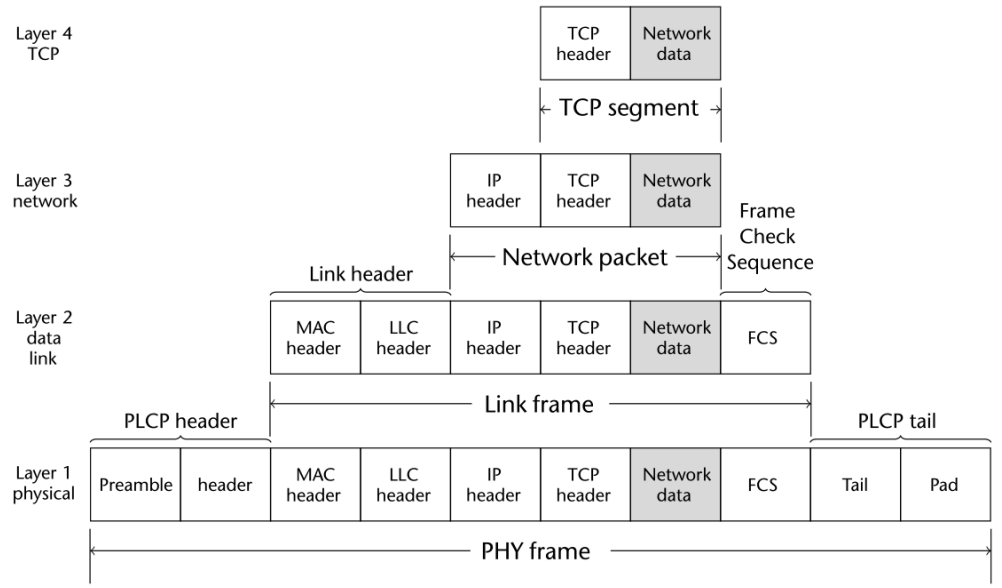


图 1.5　分组结构对 SDR 的影响，PLCP = 物理层收敛协议

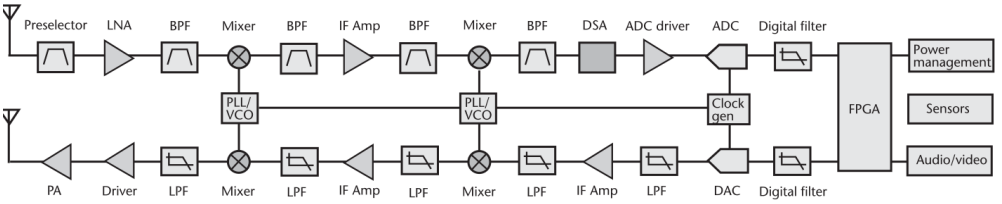


图 1.6　多级超外差接收与发射信号链 [4]

重量，进而缩短续航时间。为克服这些挑战、打造下一代航空航天与防务方案，人们正在发展新的无线电架构。

自诞生以来，超外差（superheterodyne）架构一直是无线电设计的基石。无论是手持电台、无人机数据链，还是信号情报接收机，单级或双级混频的超外差架构都是常见选择（见图 1.6）。这种设计的好处显而易见：恰当的频率规划可以把杂散发射压得很低；信道带宽与选择性由中频（IF）滤波器决定；各级之间的增益分配，则让设计者可以在噪声系数与线性度之间做折中。

在使用了一百多年之后（更多信息见附录），超外差架构在整条信号链上的性能都获得了巨大提升。微波与射频器件在降低功耗的同时提升了性能；ADC 与 DAC 的采样率、线性度和有效位数（ENOB）不断提高；FPGA 与 DSP 的处理能力跟随摩尔定律与日俱增，支撑起更高效的算法、数字校正与更深度的集成；封装技术在提高引脚密度的同时改善了散热。然而，这些器件层面的改进正开始进入收益递减的阶段。尽管射频器

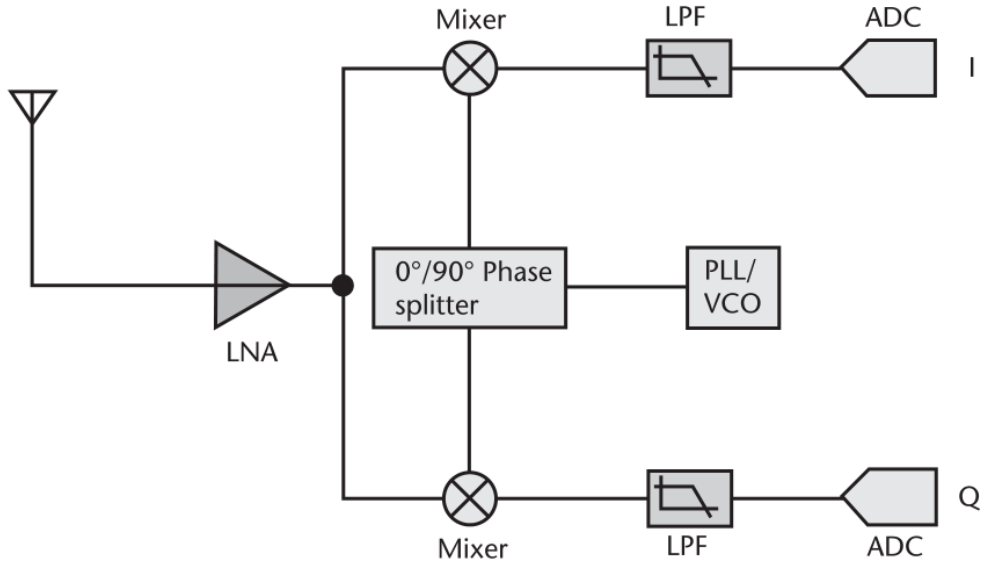


图 1.7　零中频架构 [4]

件整体上遵循着缩减尺寸、重量与功耗（SWaP）的趋势，高性能滤波器却依然体积庞大，且往往是定制设计，推高了系统总成本。此外，IF 滤波器决定了平台的模拟信道带宽，使得难以做出能跨多种系统复用的通用平台设计。就封装技术而言，大多数产线不会做到低于 0.65 mm 或 0.8 mm 的球间距，这意味着引脚数众多的复杂器件在物理尺寸上存在下限。

超外差架构的一个替代方案——零中频（zero-IF, ZIF）架构——近年来重新回到人们视野，成为一种潜在的解决方案。ZIF 接收机（见图 1.7）采用单级频率混频，将本振（LO）直接设为感兴趣的频段，把接收信号直接搬移到基带，输出同相（I）与正交（Q）信号。这种架构缓解了超外差严苛的滤波要求，因为所有模拟滤波都发生在基带——基带滤波器的设计比定制射频/中频滤波器容易得多、也便宜得多。ADC 与 DAC 此时处理的是基带 I/Q 数据，相对于被转换带宽而言采样率可以降低，显著节省功耗。在许多设计方面，得益于模拟前端复杂度与器件数量的减少，ZIF 收发信机带来了可观的 SWaP 缩减。

这种直接变频到基带的做法，可能引入载波泄漏与镜像频率分量。由于信号链中工艺偏差与温度差异等现实因素，I 与 Q 信号之间不可能维持完美的 90° 相位差，导致镜像抑制性能恶化。此外，混频级中本振隔离的不完美会引入载波泄漏分量。若不加校正，镜像与载波泄漏会恶化接收机灵敏度，并在发射时产生不合要求的频谱发射。

历史上，I/Q 失配曾把 ZIF 架构的适用范围限制得很窄。原因有二：其一，分立器件

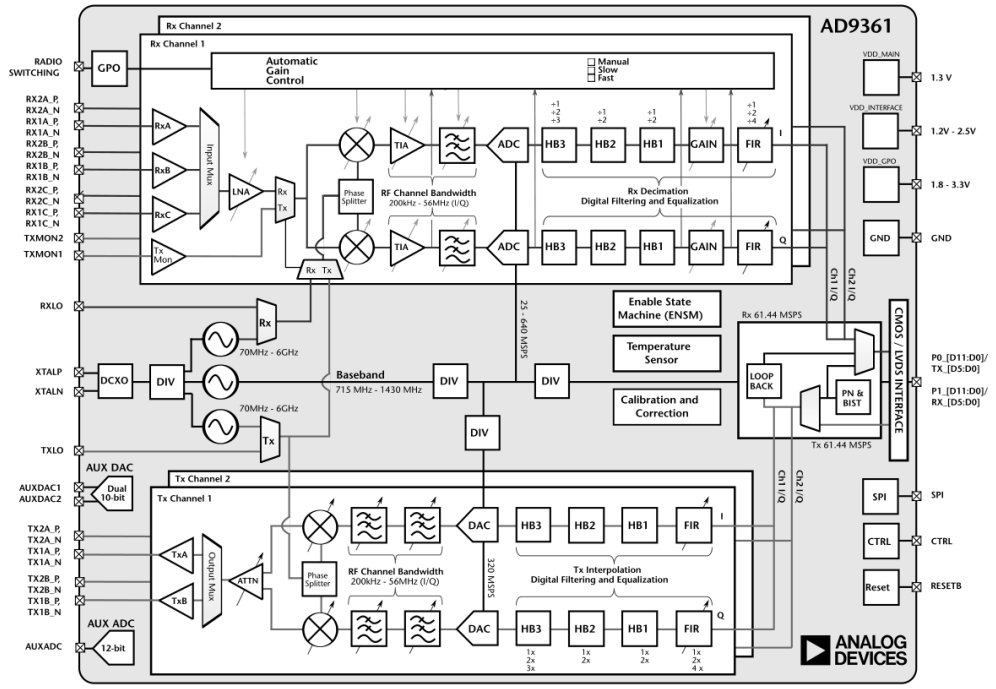


图 1.8　Pluto SDR 所用的集成 ZIF 架构

实现的 ZIF 架构中，单片器件之间以及 PCB 本身都会引入失配；而且单片器件可能来自不同批次，固有的工艺波动让精确匹配异常困难。其二，分立实现中处理器与射频器件物理上分离，使得正交校正算法很难在频率、温度与带宽的整个范围内实施。

摩尔定律——或者说把 ZIF 架构集成进单片收发信机器件——为下一代系统指明了出路。把模拟与射频信号链放在同一块硅片上，工艺波动就被控制在最小范围。数字信号处理（DSP）模块可以并入收发信机内部，消除正交校准算法与信号链之间的边界。这一途径既带来了 SWaP 上无可匹敌的改善，又能在性能指标上与超外差架构平起平坐。

图 1.8 所示的 Pluto SDR 这类器件，把完整的射频、模拟与数字信号链集成到单个 CMOS 器件上，并包含数字处理电路，在全过程的工艺、频率与温度变化下实时运行正交与载波泄漏校正。AD9361 这类器件聚焦中等性能指标与极低功耗，适用于无人机数据链、手持通信系统与小尺寸 SDR 应用。AD9371 则为高性能指标与中等功耗做了优化。此外，AD9371 还具备精细的校准控制、用于功率放大器（PA）线性化的观测接收机、以及用于空白频段检测的嗅探接收机。这为另一类应用打开了新的设计空间：使用宽带波形或占用非连续频谱的通信平台，现在可以做得小巧得多。

1.5 SDR 的处理架构

过去六十年，微电子产业飞速演进，微处理器系统的无数进步支撑起了我们如今习以为常的大量应用。这一演进的速度长期由著名的摩尔定律刻画——它定义了集成电路可容纳晶体管数量的长期趋势：单块集成电路上的晶体管数量大约每两年翻一番，并进而影响微处理器系统的处理速度与内存等性能。过去半个世纪中，微电子产业产生深远影响的一个领域就是数字通信系统：微处理器系统被越来越多地用于数字收发信机的实现，造就了更通用、更强大、更便携的通信系统平台，能够执行越来越多的先进操作与功能。随着微电子与微处理器系统的最新进展，软件定义无线电（SDR）技术应运而生——基带无线电功能可以完全用数字逻辑与软件实现，如图 1.3 所示。用于 SDR 实现的微处理器系统有好几种类型，包括：

- **通用微处理器**常用于 SDR 实现与原型开发，因为它们在可重配置性上非常灵活，实现新设计也容易。另一方面，通用微处理器并非为数学计算专门优化，功耗效率可能不佳。
- **数字信号处理器（DSP）**专为数学计算而生，新数字通信模块的实现相对容易，功耗效率也相对较好（例如手机中就在用 DSP）。但 DSP 不太擅长计算密集型任务，速度可能相当慢。
- **现场可编程门阵列（FPGA）**非常适合定制化的数字信号处理应用，因为它们可以实现定制的、全并行的算法。DSP 应用会用到大量二进制乘法器与累加器，它们可以用专用的 DSP 切片（slice）实现，如图 1.9 所示。这包括 25×18 位二补码乘法器、48 位累加器、省电预加器，以及单指令多数据（SIMD）算术单元——后者支持双 24 位或四 12 位的加/减/累加。MathWorks HDL Coder 这类工具让创建新模块、部署 FPGA 变得更容易：它可以从 MATLAB 函数、Simulink 模型与 Stateflow 图生成可移植、可综合的 Verilog 与 VHDL 代码，非常适合把信号处理算法从概念推向量产。
- **图形处理器（GPU）**计算能力极其强大。大众市场游戏对实时计算机图形的需求，把这些处理器推到了很高的性能水平与很低的价位。过去十年间，GPU 已演化为通用可编程架构及配套生态，使其能够胜任大范围的非图形应用 [6]。Nvidia 等厂商提供的 GPU 加速库，提供了比纯 CPU 方案快 2 到 10 倍的高度优化函数。面向线性代数、信号处理与图像视频处理的 GPU 加速库，为未来的软件定义无线电应用运行在这类架构上奠定了基础 [7]。
- **精简指令集机器（ARM）**近年来因其低成本、小尺寸、低功耗与不俗的计算能力而备受关注。这类处理器与一个像样的 RFFE 相组合，就构成了适合移动通信与移

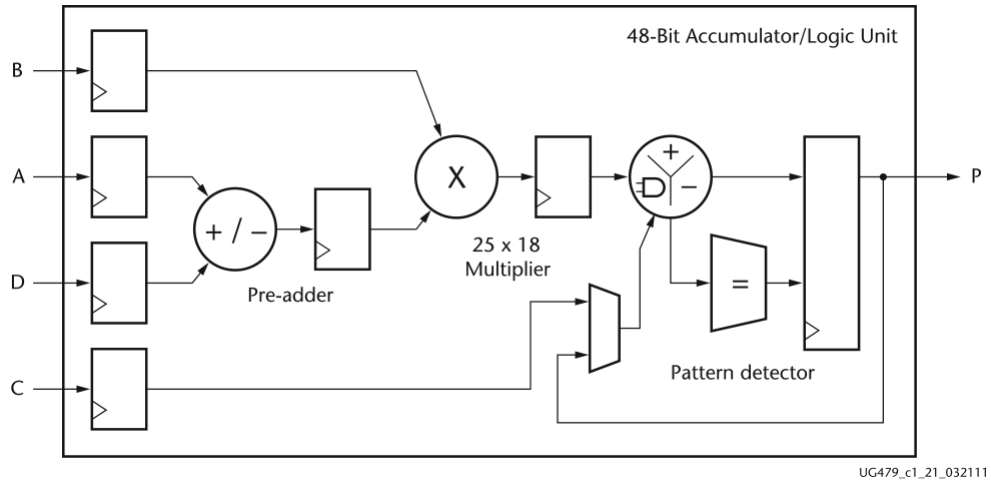


图 1.9　DSP48E1 切片的基本功能 [5]

动计算的平台。ARM 为 Cortex-A 系列与 Cortex-R52 处理器新增的 SIMD 指令集（称为 NEON [8]）可加速信号处理算法与函数，让软件定义无线电应用跑得更快。

对算法开发者而言，这是个激动人心的时代：在硬件上实现信号处理应用的新方法、高级方法层出不穷。难点在于确保无论选择哪种硬件来跑算法，该硬件与开发方法论在五年后仍有人支持。

1.6 SDR 的软件环境

如 1.2 节所述，大多数商用 SDR 平台在最基础的层面上，是把实时射频信号变换为数字基带采样，再用软件定义的调制解调机制来传递真实世界的数据。回看图 1.3：通信系统中模拟世界与数字世界的边界位于 ADC 与 DAC 处，信号信息在那里于连续信号与离散采样值之间转换。通常，电台可以配置中心频率、采样率、带宽等参数，来收发感兴趣的信号。剩下的就是调制解调技术本身了，它们通过两步的开发流程来研制：

1. **针对特定的采样率、带宽与环境，开发、调优并优化调制解调算法。**这通常在主机 PC 上完成，因为调试与可视化都容易得多。在开发的这个阶段，射频前端的调制解调都在主机上进行，为算法想法的实验与测试提供了极大的灵活性。
2. **把上述可能是用高级语言、以浮点写成的算法，移植到可量产的环境里编码实现**，并围绕产品的尺寸、重量、功耗与成本（SWaP-C）做出量产折中。当板载硬件与嵌入式处理器被编程为执行特定应用的数字通信与信号处理功能时，这些平台才是真正” 软件定义” 的。

本书只专注于 SDR 开发流程的第一步（算法步骤），但纵观整个开发流程时，第二

步（量产步骤）绝不能被排除在外。除非你的目标只是发一篇论文、从没有做出能工作的原型的打算，否则就必须把完整的开发流程记在心里。

第一步需要一种便捷的机制来采集数据，用于信号分析以及开发处理这些信号的算法。因此，拥有高效可靠的 PC 端软件，来开发与测试无线通信系统中的数据传输和数字信号处理功能，是至关重要的。

满足这一要求的一个软件环境是 MathWorks 的 MATLAB。MATLAB 是一个技术计算环境与编程语言，开发上手容易，可视化机制出色。附加产品 Communications System Toolbox（通信系统工具箱）则补充了物理层算法、信道模型、参考模型，以及连接 SDR 硬件收发实时信号的能力。MATLAB 是跨平台的（Windows、Linux、macOS），支持许多流行的商用射频前端。使用 MATLAB 可以形成一条渐进迭代的 SDR 开发工作流：

- 通过链路级仿真进行算法开发与设计验证；
- 连接商用 SDR 硬件，用实时信号对算法进行验证。

MathWorks 还提供 Simulink——一个面向真实系统仿真与软硬件自动代码生成的环境。它让无线电开发者得以延续到量产开发的第二阶段。Simulink 的这些能力提供了一条通往量产的路径：

- 开发并验证一个硬件级精确的模型；
- 用自动 HDL 与 C 代码生成，在 SDR 硬件上实现原型；
- 将原型与已验证模型进行比对验证；
- 把实现部署到量产 SDR 硬件上。

尽管本书基本上不会用到 Simulink，但对于想做出真正量产无线电的人来说，能拥有一个从概念贯穿到量产的单一环境，威力巨大，不应忽视。

另一种流行的 SDR 软件架构是开源的 GNU Radio 软件 [9]。它是一个自由（free as in freedom）软件开发工具包，提供实现软件定义无线电与信号处理系统的信号处理模块。它可以配合外部射频硬件组成软件定义无线电，也可以脱离硬件、在仿真环境中使用。它在爱好者、学术界与工业界都被广泛用于无线通信研究和真实世界的无线电系统。

在 GNU Radio 中，大量用 C++ 库实现的数字通信与数字信号处理算法，通过 Python 与 SWIG（一个把 C/C++ 程序与 Python 等多种高级语言连接起来的软件开发工具）整合在一起。这些库由开源社区产出，向所有人自由共享。

本书作者在科研、产品开发以及本科生与研究生课程教学中用过 MATLAB、Simulink 与 GNU Radio 等多种工具。每种工具各有优劣，适用于研发周期的不同阶段。作者们相信：各种软件环境既可以被用对、也可以被用错地来教授无线物理层基础，只是各工具的前置要求不同。对选择 GNU Radio 路线的人来说，需要熟练掌握 Linux、Python、C++ 与 SWIG——门槛相当高。这对计算机专业的学生很平常，对大多数通信专业的学

生却不然：要求某人在学通信理论的同时还去学工具，会很吃力。当然，GNU Radio 里可以用现成模块，绕开理解 Python 与 C++ 的要求，但这样一来，演示与实验通信基础理论的许多机会也就丢掉了——因为学生只是用了一个别人写的模块，并不明白它在做什么。Simulink 也一样：它是非常强大的工具，有大量现成的定时恢复与载波同步模块。然而，用这些模块并不能让多数学生理解模块内部发生了什么，于是学生也就难以理解如何针对自己的情形去调这些模块。

这正是本书选择 MATLAB 的原因。它是一个跨平台环境，学生可以用自己熟悉的工具；而且本书给出的所有" 模块" 都是 MATLAB 脚本，没有任何隐藏。学生若想更深入地理解某处，整个算法就写在 MATLAB 代码里，没有任何东西会遮挡通信理论本身。

1.7 延伸阅读

本章只是对不断扩张的 SDR 技术领域做了简要介绍，公开文献中还有几本书能提供更详尽的视角。例如，Reed 的书全面覆盖了 SDR 平台软件架构的诸多议题 [10]；建造 SDR 硬件原型及其射频前端的许多设计考量与方法，则见之于 Kensington 的书 [11]。关于 SDR 系统硬件实现的另一部优秀参考书来自 Grayver [12]。此外，理解模数分界的重要性、以及 SDR 系统如何跨越这道分界，是 Machado 与 Wyglinski 论文的主题 [13]。最后，IEEE Communications Magazine 上有一组出色的论文，覆盖 SDR 技术的最新进展，并对它自二十年前以来的演进给出了一些透视 [14, 15]。

1.8 参考文献

[1] American Guild Of Town Criers Website, 1997. http://www.americantowncriers.com/.

[2] IEEE Project 1900.1 — Standard Definitions and Concepts for Dynamic Spectrum Access: Terminology Relating to Emerging Wireless Networks, System Functionality, and Spectrum Management. https://standards.ieee.org/develop/project/1900.1.html.

[3] Analog Devices ADF7030. http://www.analog.com/ADF7030

[4] Hall, B., and W. Taylor, "X- and Ku-Band Small Form Factor Radio Design." http://www.analog.com/en/technical-articles/x-and-ku-band-small-form-factor-radio-design.html.

[5] Xilinx Inc., "7 Series DSP48E1 User Guide," UG479 (v1.9), September 27, 2016. https://www.xilinx.com/support/documentation/user_guides/ug479_7Series_D

SP48E1.pdf

[6] McCool, M., “Signal Processing and General-Purpose Computing on GPUs,” IEEE Signal Processing Magazine, Vol. 24, No. 3, May 2007. http://ieeexplore.ieee.org/document/4205095/.

[7] Nvidia, “GPU-accelerated Libraries for Computing.” https://developer.nvidia.com/gpu-accelerated-libraries.

[8] ARM NEON. https://developer.arm.com/technologies/neon.

[9] GNU Radio, “Welcome to GNU Radio!” http://gnuradio.org/.

[10] Reed, J. H., *Software Radio: A Modern Approach to Radio Engineering*, Upper Saddle River, NJ: Prentice Hall PTR, 2002.

[11] Kensington, P., *RF and Baseband Techniques for Software Defined Radio*, Norwood, MA: Artech House, 2005.

[12] Grayver, E., *Implementing Software Defined Radio*, New York: Springer-Verlag, 2012.

[13] Machado, R. G., and A. M. Wyglinski, “Software-Defined Radio: Bridging the Analog–Digital Divide,” *Proceedings of the IEEE*, Vol. 103, No. 3, March 2015, pp. 409–423.

[14] Mitola, J., P. Marshall, K. C. Chen, M. Mueck, and Z. Zvonar, “Software Defined Radio — 20 Years Later: Part 1 [guest editorial],” *IEEE Communications Magazine*, Vol. 53, No. 9, September 2015, pp. 22–23. http://ieeexplore.ieee.org/document/7263341/.

[15] Mitola, J., P. Marshall, K. C. Chen, M. Mueck, and Z. Zvonar, “Software Defined Radio — 20 Years Later: Part 2 [guest editorial],” *IEEE Communications Magazine*, Vol. 54, No. 1, January 2016, p. 58. http://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7378426.

第 2 章 信号与系统

SDR 是信号处理中一个面向特定应用的领域，每个参与 SDR 开发的人不仅需要扎实的信号与系统功底，还需要射频与模拟基带处理的知识。本章覆盖线性时不变（LTI）信号与系统、模拟处理与射频的诸多方面，它们构成后续章节的基础。并不要求读者逐个专题深挖，但如果哪个短专题对你来说是新的，应该去查阅这些 subjects 的更全面的专著。

2.1 时域与频域

时域和频域是同一个信号的两种替代表示。傅里叶变换——以它的最初提出者、法国数学家兼物理学家让-巴蒂斯特·约瑟夫·傅里叶命名——就是这两种表示之间的数学关系。如果信号在一个域里被修改，它在另一个域里也会随之改变，只是通常方式不同。例如，时域卷积等价于频域相乘。其他数学运算，如加法、缩放与移位，在另一个域里也有对应的运算。这些关系称为傅里叶变换的性质，描述一个域中的数学变化如何在另一个域中产生对应的数学变化。

傅里叶变换只是描述信号的另一种方式。例如，考察吉布斯现象（Gibbs phenomenon）——它说的是：把特定频率/相位/幅度组合的正弦波叠加起来，可以逼近一个方波——可以数学地表达为 (2.1)，如图 2.1 所示。

$$x(t) = \sin(t) + \frac{\sin(3t)}{3} + \frac{\sin(5t)}{5} + \cdots = \sum_{n=1}^{n=\infty} \frac{\sin(n \times t)}{n}; \quad n = \text{odd}$$

当我们沿着垂直于频率轴的时间轴观察信号时，看到的是时域。由于视线垂直于频率轴，我们不容易看出正弦波的频率。当我们变换域、沿垂直于时间轴的频率轴观察时，看到的就是频域（傅里叶域）。我们可以轻松辨认信号的幅度与频率，却失去了时间信息。两种视图代表同一个信号——只是通过变换从不同的域去观察而已。

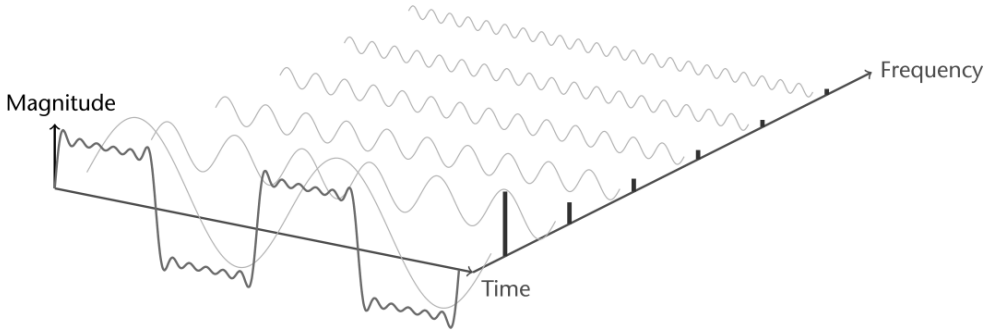


图 2.1　吉布斯现象：左为时域，右为傅里叶域

> 思考题：在 MATLAB 中用代码 2.1 研究吉布斯现象，更好地理解改变频率/相位/幅度如何影响正弦波叠加在时域与频域中的结果。

代码 2.1　吉布斯现象：gibbs.m

```
max = 9;
fs = 1000;

for i = 1:2:max
    % dsp.SineWave(amp,freq,phase,Name,Value);
    wave = dsp.SineWave(1/i, i*2*pi, 0, ...
    'SamplesPerFrame', 5000, 'SampleRate', fs);
    y = wave();
    if i == 1
        wavesum = y;
    else
        wavesum = wavesum + y;
    end

    scope(wavesum());
    pause(.5);
    % waitforbuttonpress;
end
```

2.1.1 傅里叶变换

傅里叶变换家族有四个成员：傅里叶变换（FT）、傅里叶级数、离散傅里叶变换（DFT）和离散时间傅里叶变换（DTFT）。常被提到的 FFT（快速傅里叶变换）及其逆变换 IFFT，是 DFT 的一种具体实现。

$x(t)$ 的傅里叶变换定义为 [1]：

$$X(\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt,$$

其中 t 是时域的时间变量（秒），ω 是频域的频率变量（弧度/秒）。

对 $X(\omega)$ 施加类似的变换，得到傅里叶逆变换 [1]：

$$x(t) = \int_{-\infty}^{\infty} X(\omega) e^{j2\pi\omega t} d\omega,$$

它把 $x(t)$ 写成复指数的加权和。

上面的傅里叶变换对可以记为 [2]：

$$x(t) \overset{\mathcal{F}}{\longleftrightarrow} X(\omega),$$

符号左侧是变换前的信号，右侧是变换后的信号。研究 SDR 频域时，有几个常用的傅里叶性质很有用，汇总于表 2.1 供参考。

表 2.1　傅里叶变换性质（设时间信号为 $x(t)$，其傅里叶变换为 $X(\omega)$；据 [2]）

性质	时间信号	傅里叶变换信号
定义	$x(t)$	$\int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt$
反演公式	$\int_{-\infty}^{\infty} X(\omega) e^{j2\pi\omega t} d\omega$	$X(\omega)$
线性	$\sum_{n=1}^{N} a_n x_n(t)$	$\sum_{n=1}^{N} a_n X_n(\omega)$
对称性	$x(-t)$	$X(-\omega)$
时移	$x(t - t_0)$	$X(\omega) e^{-j\omega t_0}$
频移	$x(t) e^{j\omega_0 t}$	$X(\omega - \omega_0)$
尺度变换	$x(\alpha t)$	$\frac{1}{\|\alpha\|} X(\frac{1}{\alpha}\omega)$
微分	$\frac{d^n}{dt^n} x(t)$	$(j\omega)^n X(\omega)$
积分	$\int_{-\infty}^{t} x(\tau) d\tau$	$\frac{X(\omega)}{j\omega} + \pi X(0)\delta(\omega)$
时域卷积	$x(t) * h(t)$	$X(\omega) H(\omega)$
频域卷积	$x(t) h(t)$	$\frac{1}{2\pi} X(\omega) * H(\omega)$

2.1.2 DFT 的周期性

与其他三种傅里叶变换不同，DFT 把时域和频域信号都视为周期的（永远重复）。这既让人困惑也不方便，因为许多信号处理应用中的信号并不是周期的。然而，想用 DFT（及其快速实现 FFT），就必须接受 DFT 的世界观。

图 2.2 给出了时域信号的两种解读。先看上方的信号：时域被视为 N 个点。这代表 SDR 采集信号的典型方式——一个 N 点的缓冲区。例如，这 128 个采样可能来自对某模拟信号按固定时间间隔的采样。采样 0 与采样 127 是截然不同、彼此分离的，因为它们是在不同时刻采集的。从这段信号的形成方式看，没有任何理由认为信号左边的采样与右边的采样有关联。

不幸的是，DFT 不这么看。如图 2.2 下方所示，DFT 把这 128 个点视为一个无限长周期信号的一个周期。这意味着采集信号的左侧连接着一份副本信号的右侧；同样，采集信号的右侧连接着一份相同周期的左侧。也可以理解为：采集信号的右侧” 卷绕” 回去接到它自己的左侧。在这种视图下，采样 127 紧挨着采样 0，就像采样 43 紧挨着采样 44 一样。这称为**循环**（circular）视角，与把信号视为周期信号完全等价。这正是为什么在调用 FFT 之前需要先给信号采集预乘**窗函数** [3] 的原因——窗函数乘以信号、通过把不连续处强压为零来消除它们 [4]。

2.1.3 快速傅里叶变换

计算 DFT 有多种方法，比如求解联立线性方程或相关法。FFT 是计算 DFT 的另一种方法。它与其他方法结果相同，但效率惊人地高，常把计算时间降低多个数量级。

FFT 只需要几十行代码，却是信号处理中较复杂的算法之一，其内部细节留给专门研究它的人。只要理解它的工作方式，完全可以放心使用现有的、经过验证的 FFT 例程 [4, 5]，不必彻底理解内部实现。

用图 2.3 所示的通用测试设置做一次 FFT 分析。FFT 的频谱输出是频域中的 M 个点（M 是 FFT 的规模，即缓冲存储器中保存的采样数）。点与点的间距是 $\frac{f_s}{M}$，覆盖的总频率范围是从 DC 到 $\frac{f_s}{2}$，其中 f_s 是采样率。每个频率 bin 的宽度（有时称为 FFT 的分辨率）是 $\frac{f_s}{M}$。

图 2.4 给出了用 Analog Devices 的 ADIsimADC® 程序对一个理想 12 位 ADC 做出的 FFT 输出。注意，FFT 的理论噪底等于理论信噪比（SNR）加上 $10\log_{10}(\frac{M}{2})$ 的 FFT 处理增益。要记住：SNR 计算中用到的噪声是延伸在整个奈奎斯特带宽（DC 到 $\frac{f_s}{2}$）上的噪声，而 FFT 相当于一台带宽为 $\frac{f_s}{M}$、在频谱上扫掠的窄带频谱分析仪。这会把噪底压低一个等于处理增益的量——与收窄模拟频谱分析仪带宽的效果相同。

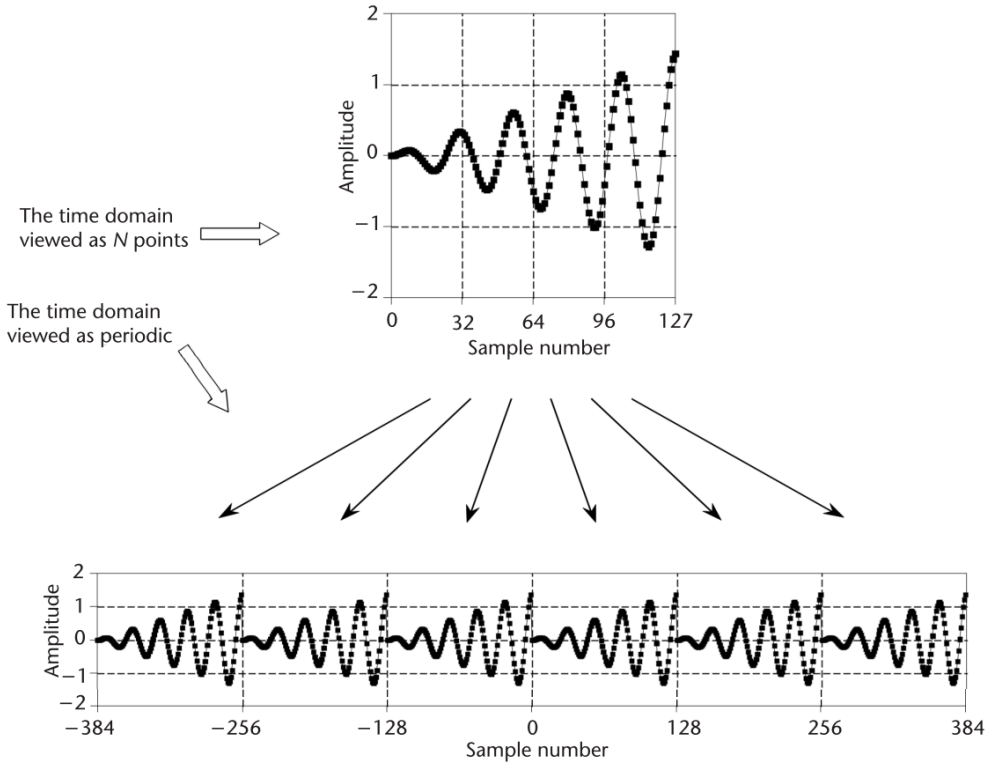


图 2.2　DFT 时域信号的周期性：上图为 N 个采样长度的视图，下图为无限长周期信号视图 [4]

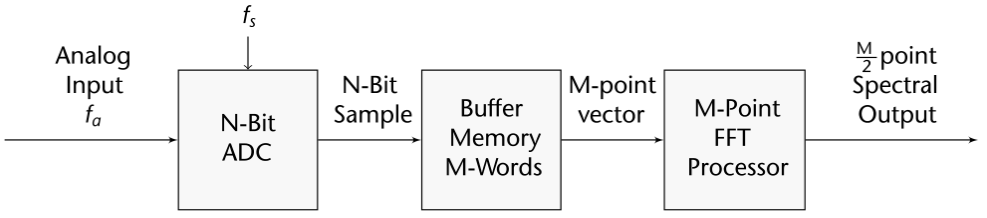


图 2.3　ADC 输出 FFT 分析的通用测试设置 [6]

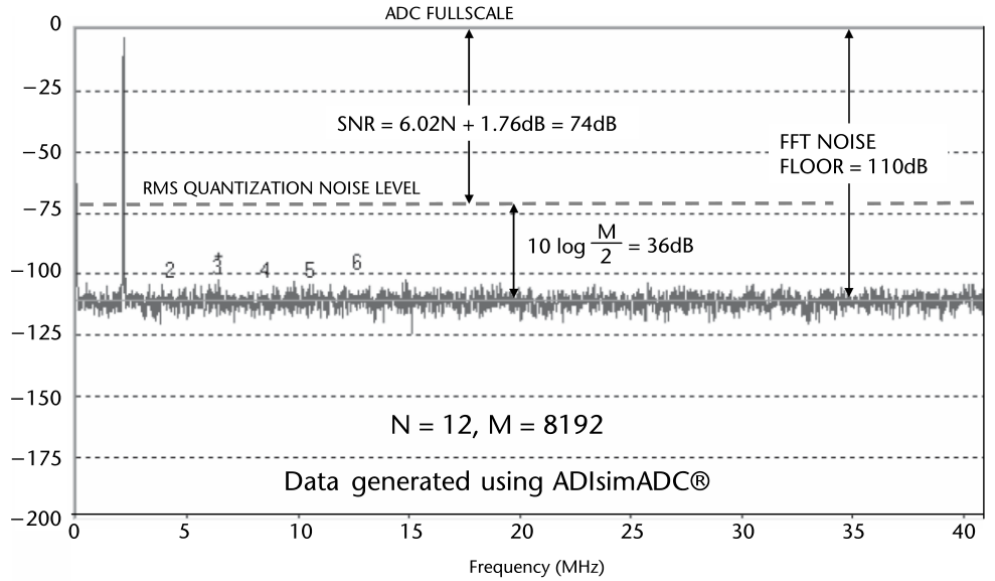


图 2.4　理想 12 位 ADC 的 FFT 输出，fa = 2.111 MHz，fs = 82 MSPS，5 次 FFT 平均，M = 8192，数据由 ADIsimADC® 生成 [6]

FFT 输出可以像模拟频谱分析仪一样，用来测量数字化信号中各次谐波与噪声分量的幅度。输入信号的谐波可以通过它们在频谱中的位置与其他失真产物区分开。

2.2 采样理论

连续时间模拟信号可以经由采样与量化转换为离散时间数字信号，如图 2.5 所示：连续模拟输入信号 $x_a(t)$ 被转换为离散数字输出信号 $x[n]$。**采样**是在离散时间点上取连续时间信号的样值，把连续时间信号变为离散时间信号 [1]。**量化**过程把采样幅度转换为数字格式。2.2.1 节将介绍一种最常用的采样方法——均匀采样。

类似地，离散时间信号也可以通过**重建**（reconstruction）转换为连续时间信号。但重建并非总能成功：有时重建出的信号与原信号不同。因为一段给定的采样信号，可以对应无穷多条穿过这些量化采样点的连续时间曲线。不过，如果采样满足特定准则，信号就可以无损重建。该准则称为奈奎斯特采样定理，将在 2.2.3 节与 2.5.1 节介绍。

2.2.1 均匀采样

把模拟信号采样成数字表示有很多方法。但若把采样间隔指定为常数 T_s，就得到了使用最广泛的采样——**均匀采样**（uniform sampling），也叫周期采样。用这种方法，我们每隔 T_s 秒对连续时间信号取一次样，可定义为

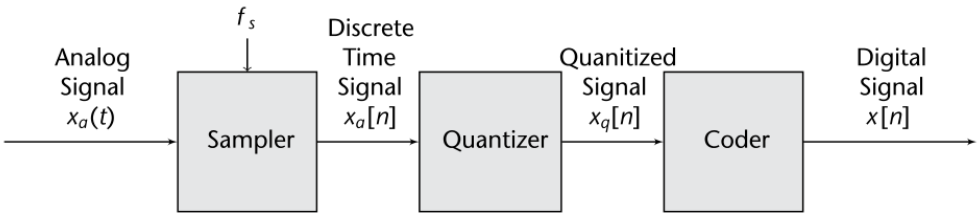


图 2.5　模数转换器（ADC）的基本组成 [1]。采样发生在采样器模块。xa(t) 为模拟连续时间信号；fs 为数字采样率；xa[n] 为离散时间连续幅度信号；xq[n] 为离散时间离散幅度数字信号（可能以格雷码输出）；x[n] 是编码器以二补码形式输出的结果 [7]

$$x[n] = x(nT_s), \quad -\infty < n < \infty,$$

其中 $x(t)$ 是输入连续时间信号，$x[n]$ 是输出离散时间信号，T_s 是采样周期，$f_s = 1/T_s$ 是采样频率。

均匀采样操作的一个等效模型如图 2.6(a) 所示：连续时间信号 $x(t)$ 与冲激串 $p(t)$ 相乘，形成采样信号 $x_s(t)$，即

$$x_s(t) = x(t)p(t),$$

其中 $p(t)$ 称为采样函数。

采样函数被假定为一串窄脉冲，取值为 0 或 1。于是 $p(t) = 1$ 时 $x_s(t) = x(t)$，$p(t) = 0$ 时 $x_s(t) = 0$。由于 $p(t)$ 只在 T_s 的整数倍时刻等于 1，得到的 $x_s(t) = x(nT_s) = x[n]$，证明这确实是均匀采样操作的等效模型。这个模型将帮助我们在 2.2.2 节得到均匀采样的频域表示。

2.2.2 均匀采样的频域表示

由于在频域推导奈奎斯特采样定理更容易，本节尝试在频域表示均匀采样过程。根据图 2.6(b)，可以把采样函数定义为

$$p(t) = \sum_{k=-\infty}^{\infty} \delta(t - kT_s), \quad k = 0, 1, 2, \ldots$$

在 kT_s 时刻 $p(t) = 1$。根据 [8]，$p(t)$ 是由狄拉克 δ 函数构成的狄拉克梳（Dirac comb）。

把 (2.7) 代入 (2.6) 得

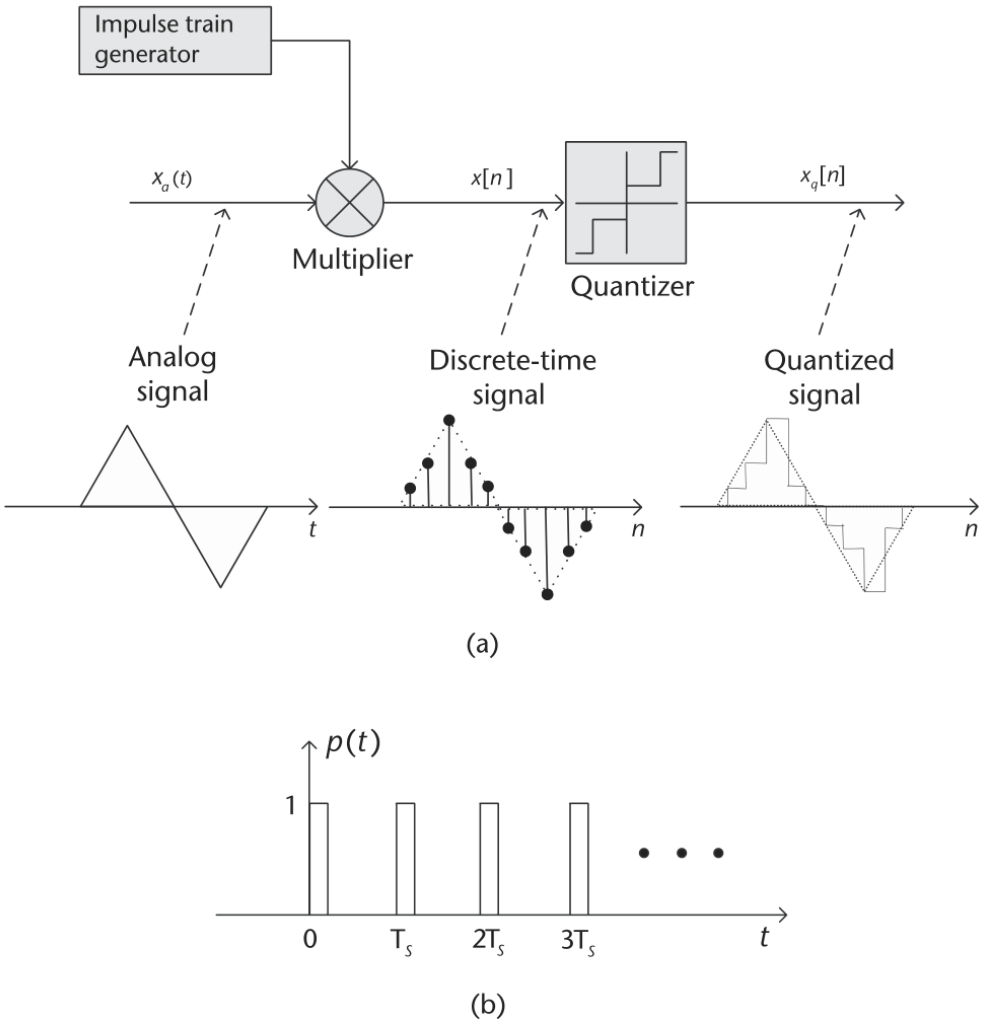


图 2.6　均匀采样操作的等效模型：(a) 连续时间信号 x(t) 与周期脉冲 p(t) 相乘形成采样信号 xs(t)；(b) 周期脉冲 p(t)

$$x_s(t) = x(t)p(t) = x(t)\sum_{k=-\infty}^{\infty}\delta(t - kT_s).$$

要在频域理解采样过程，需要对 $x_s(t)$ 取傅里叶变换。根据表 2.1 的频域卷积性质，时域相乘对应频域卷积。因此 $x(t)$ 与 $p(t)$ 的乘积将得到 $X(\omega)$ 与 $P(\omega)$ 的卷积：

$$X_s(\omega) = \frac{1}{2\pi}X(\omega) * P(\omega),$$

其中 $X(\omega)$ 是 $x(t)$ 的傅里叶变换，$P(\omega)$ 是 $p(t)$ 的傅里叶变换。

狄拉克梳的傅里叶变换还是狄拉克梳 [8]，即

$$P(\omega) = \frac{2\pi}{T_s}\sum_{k=-\infty}^{\infty}\delta\left(\omega - k\frac{2\pi}{T_s}\right) = \frac{2\pi}{T_s}\sum_{k=-\infty}^{\infty}\delta(\omega - k\omega_s),$$

其中 $\omega_s = 2\pi f_s$ 是采样频率。

与一串位于脉冲位置处的 δ 函数做卷积，得到

$$X_s(\omega) = \frac{1}{T_s}\sum_{k=-\infty}^{\infty}X(\omega - k\omega_s).$$

(2.11) 告诉我们：均匀采样使输入信号傅里叶变换产生**镜像**，镜像以采样频率 f_s 为周期重复。

2.2.3 奈奎斯特采样定理

基于 (2.11)，我们在频域画出原始信号 $x(t)$ 与采样信号 $x_s(t)$ 的频谱，如图 2.7 所示。设原始信号带宽为 $[-f_h, f_h]$，如图 2.7(a) 所示。这里暂不关心幅度，分别用 A 与 A_s 表示。设采样频率为 f_s，则采样信号的频谱将在 kf_s 处出现副本。为了从采样信号重建原始信号，我们对采样信号施加低通滤波，试图从 $X_s(f)$ 中提取 $n = 0$ 那一项，如图 2.7(b) 所示。因此，要无误差地完成重建，就要求 $X_s(f)$ 在 $f = \pm f_s$ 处的部分与 $f = 0$ 处的部分不重叠。换言之，要求 $f_s - f_h > f_h$，即 $f_s > 2f_h$——由此得到奈奎斯特采样定理。

奈奎斯特采样定理适用于**带限信号**：即没有超过 B Hz 频谱分量的信号 $x(t)$ [9]，即

$$X(\omega) = 0, \quad |\omega| > 2\pi B.$$

奈奎斯特采样定理：一个带限于 B Hz 的实信号 $x(t)$，可以用每秒 $R > 2B$ 个采样的均匀速率无误差地重建。这一最小采样频率 $F_s = 2B$ Hz 称为奈奎斯特速率或奈奎斯

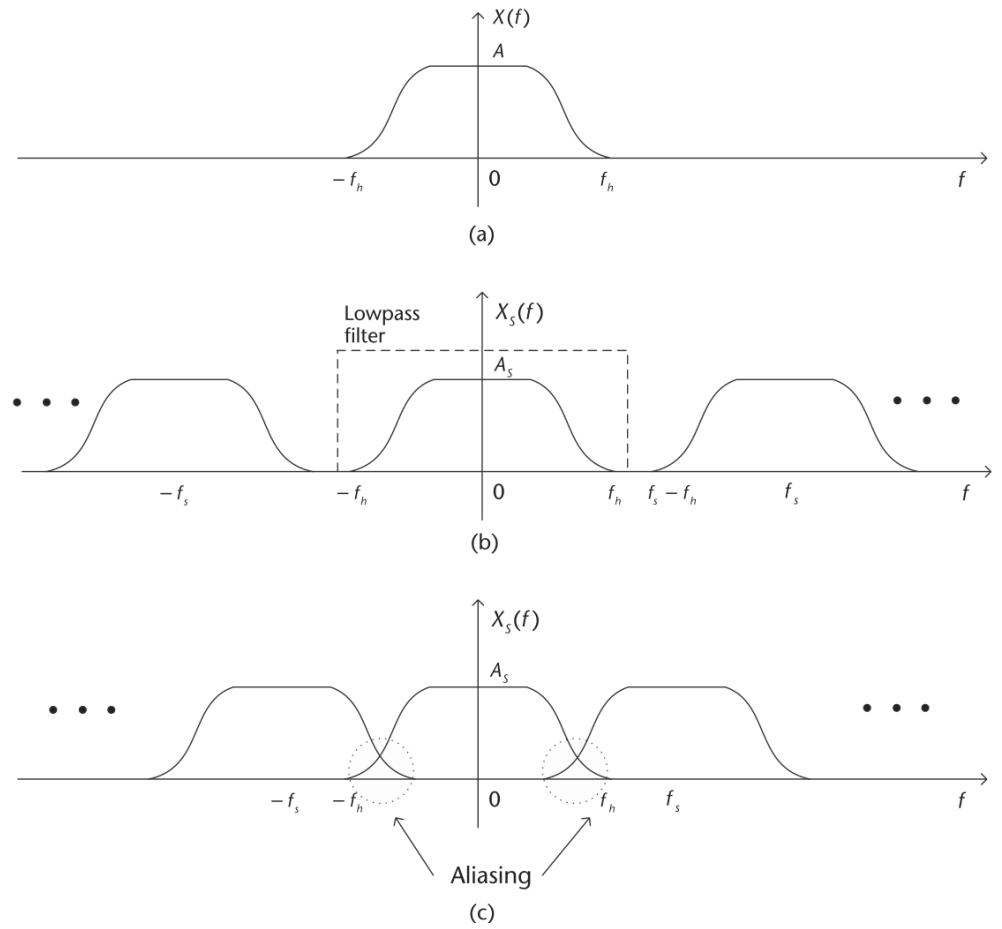


图 2.7　频域中原始信号 x(t) 与采样信号 xs(t) 的频谱：(a) 原始连续信号频谱，带宽 −fh 到 fh，幅度 A；(b) 满足奈奎斯特定理的采样信号频谱（fs > fh）；(c) 不满足定理、出现混叠的采样信号频谱（fs < fh）

特频率；对应的采样间隔 $T = \frac{1}{2B}$ 称为奈奎斯特间隔 [1]。带限于 B Hz、但以低于 $2B$ 奈奎斯特频率（即以 $T > \frac{1}{2B}$ 的间隔）采样的信号，称为**欠采样**（undersampled）。

信号欠采样时，其频谱的尾部（镜像）相互重叠，$X_s(f)$ 不再完整保留频谱信息，也就无法从采样信号恢复 $x(t)$。此时尾部频谱并不归零，而是折叠回表观频谱之上。这种尾部翻转称为**频谱折叠**或**混叠**（aliasing），如图 2.7(c) 所示 [10]。

动手 MATLAB 示例：现在用计算机仿真解释：为什么奈奎斯特准则要求采样频率至少是信号最高频率的两倍，否则信息就会丢失。而且，正如 2.5.1 节将展示的，此时会发生混叠，频率被折叠回第一奈奎斯特区。为了说明混叠的后果，可以从时域考察。

考虑图 2.8(a) 所示的单音正弦波的时域采样。这个例子里，采样频率 f_s 并未达到模拟输入频率 F_A 的两倍，而只是略高于 F_A。因此按定义就违反了奈奎斯特准则。注意，

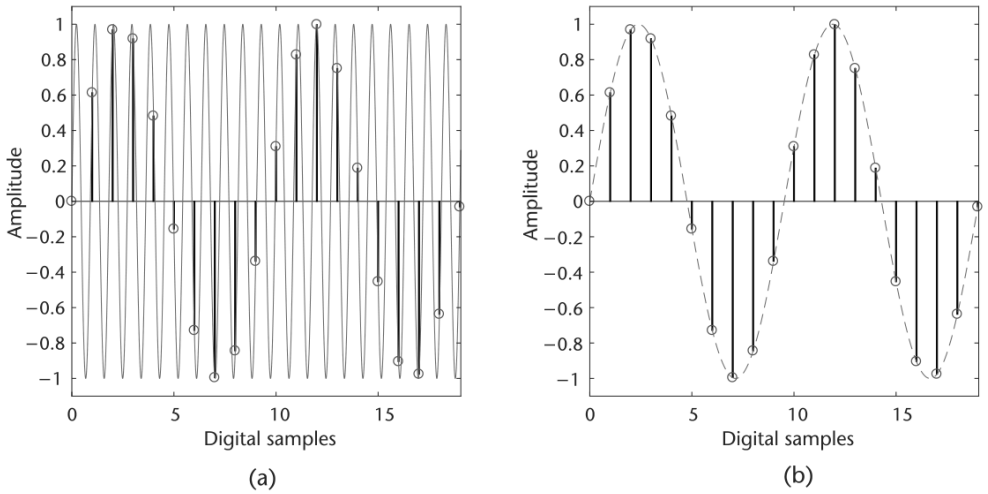


图 2.8　时域混叠：两图的数字采样相同。(a) 实线为模拟输入（FA），圆点为 fs 下的数字采样；(b) 虚线为数字重建

实际采样点的排列产生了一个频率更低的混叠正弦波，如图 2.8(b) 所示。

代码 2.2 可以生成图 2.8(a) 与 (b) 的图形，通过调整 F_s 与 F_a 更好地理解混叠的机理。图 2.7 与图 2.25 在频域演示了同样的效果。

代码 2.2　时域混叠：nyquist.m

```
Fs = 1000;      % Sample rate (Hz)
Fa = 1105;      % Input Frequency (Hz)

% Determine Nyquist zones
zone = 1 + floor(Fa / (Fs/2));
alias = mod(Fa, Fs);
if ~mod(zone,2) % 2nd, 4th, 6th, ... Nyquist Zone
    % Its not really a negative amplitude, but it is 180 degrees out
    % of phase, which makes it harder to see on the time domain side,
    % so we cheat to make the graphs look better.
    alias = -(Fs - alias)/Fs;
else            % 3rd, 5th, 7th, ... Nyquist Zone
    alias = (alias)/Fs;
end
```

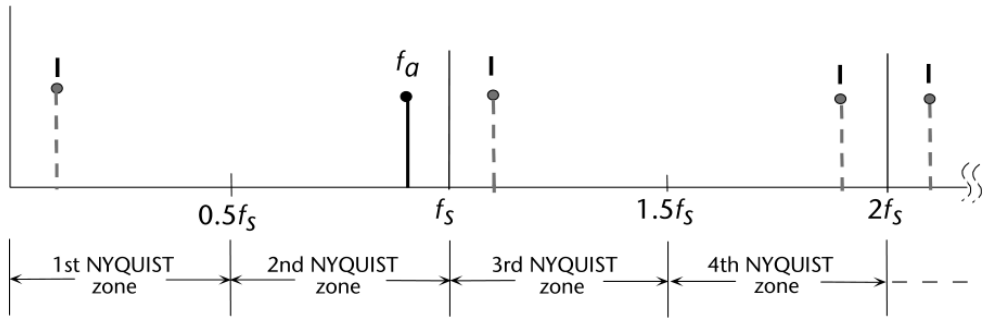


图 2.9　模拟信号 fa 以 fs 采样后，在 ±kFs ± Fa（k = 1, 2, 3, …）处出现镜像（混叠）

```
% Create the analog/time domain and digital sampling vectors
N = 2*1/abs(alias) + 1;        % Number of Digital samples
points = 256;                  % Analog points between digital samples
analogIndexes = 0:1/points:N-1;
samplingIndexes = 1:points:length(analogIndexes);
wave = sin(2*pi*Fa/Fs*analogIndexes);
```

2.2.4 奈奎斯特区

奈奎斯特带宽本身定义为从 DC 到 $\frac{f_s}{2}$ 的频谱。但频谱其实被划分为无穷多个奈奎斯特区，每个区宽度为 $0.5f_s$，如图 2.9 所示。频谱不会因为你不关心那些频率就到此为止。

这意味着采样器（或 ADC）之前需要一些滤波，来去除那些位于奈奎斯特带宽之外、但其混叠分量落入带宽之内的频率成分。滤波器的性能取决于带外信号距离 $\frac{f_s}{2}$ 有多近以及所需的衰减量。要注意：理想采样器（ADC）输入端若不加输入滤波，任何落在奈奎斯特带宽之外的任何奈奎斯特区中的频率分量（无论信号还是噪声）都会混叠回第一奈奎斯特区。正因如此，几乎所有采样 ADC 应用都会使用模拟**抗混叠滤波器**来去除这些不需要的信号。

> 思考题：当频率升入第三、第四或更高奈奎斯特区时，测量频率与绝对频率的关系会怎样变化？你可以按图 2.9 修改代码 2.2，以绝对频率为 x 轴、测量频率为 y 轴作图来验证你的假设。

2.2.5 采样率转换

真实应用中，我们常想降低采样率以减少存储与计算量；很多场景下我们又偏爱高采样率以保真。**采样率转换**是改变相邻采样间隔的过程的统称，这里的序列由连续时间函数的采样构成 [10]。

抽取（decimation）：降低采样率的过程称为抽取，通过只保留每第 D 个采样、丢弃其余来实现。时域上可定义为

$$y[n] = x[nD], \quad D = 1, 2, 3, \ldots$$

其中 $x[n]$ 是原始信号，$y[n]$ 是抽取后的信号，D 是抽取率。由 (2.13)，原始信号与抽取后信号的采样率满足

$$F_y = \frac{F_x}{D},$$

其中 F_x 为原始信号采样率，F_y 为抽取后信号采样率。

由于弧度频率变量 ω_x 与 ω_y 和采样率的关系是

$$\omega_x = 2\pi F T_x = \frac{2\pi F}{F_x}, \qquad \omega_y = 2\pi F T_y = \frac{2\pi F}{F_y},$$

由除法性质可得 ω_x 与 ω_y 的关系为

$$\omega_y = D\,\omega_x,$$

即 ω_x 的频率范围被拉伸 D 倍成为 ω_y 的频率范围。

为避免抽取序列 $y[n]$ 混叠，要求 $0 \le |\omega_y| \le \pi$。由 (2.17)，这意味着原始序列频谱应满足 $0 \le |\omega_x| \le \frac{\pi}{D}$。因此实际中抽取通常是两步过程：低通抗混叠滤波器 + 下采样器，如图 2.10 所示。低通抗混叠滤波器用来把下采样器输入 $x[n]$ 的带宽约束到 $0 \le |\omega_x| \le \frac{\pi}{D}$。

频域上，抽取后信号 $y[n]$ 的频谱可表示为 [1]

$$Y(\omega_y) = \frac{1}{D}\sum_{k=0}^{D-1} H_D\left(\frac{\omega_y - 2\pi k}{D}\right) S\left(\frac{\omega_y - 2\pi k}{D}\right),$$

其中 $S(\omega)$ 是输入信号 $s[n]$ 的频谱，$H_D(\omega)$ 是低通滤波器 $h_D[n]$ 的频率响应。滤波器 $H_D(\omega)$ 设计得当，混叠即被消除，(2.18) 中除 $k = 0$ 外的项都消失 [1]。于是 (2.18)

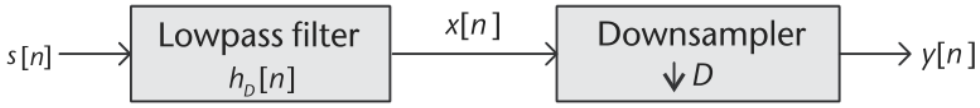


图 2.10　抽取的结构：低通抗混叠滤波器 + 下采样器

变为

$$Y(\omega_y) = \frac{1}{D} H_D\Big(\frac{\omega_y}{D}\Big)\, S\Big(\frac{\omega_y}{D}\Big), \quad 0 \le |\omega_y| \le \pi.$$

序列 $x[n]$ 与 $y[n]$ 的频谱示于图 2.11：中间信号的频率范围是 $0 \le |\omega_x| \le \frac{\pi}{D}$，抽取后信号的频率范围是 $0 \le |\omega_y| \le \pi$。

插值 (interpolation)：提高采样率的过程称为插值，通过在相邻信号值之间插入（塞零）$I-1$ 个新采样实现。时域上可定义为

$$y[n] = \begin{cases} x[n/I] & n = 0, \pm I, \pm 2I, \dots \\ 0 & \text{otherwise} \end{cases}, \quad I = 1, 2, 3, \dots$$

其中 $x[n]$ 是原始信号，$y[n]$ 是插值后信号，I 是插值率。

由 (2.20)，原始信号与插值后信号的采样率满足

$$F_y = I F_x.$$

由于 (2.15)(2.16) 在这里同样成立，可得

$$\omega_y = \frac{\omega_x}{I},$$

即 ω_x 的频率范围被压缩 I 倍成为 ω_y 的频率范围。因此插值之后，$x[n]$ 的频谱将出现 I 个副本，每个副本占据 $\frac{\pi}{I}$ 的带宽。

由于 $y[n]$ 中只有 $0 \le |\omega_y| \le \frac{\pi}{I}$ 范围内的频率分量是唯一的（其余副本都与其相同），$\frac{\pi}{I}$ 以上的镜像应通过低通滤波器去除，滤波器频率响应为

$$H_I(\omega_y) = \begin{cases} C & 0 \le |\omega_y| \le \frac{\pi}{I} \\ 0 & \text{otherwise} \end{cases},$$

其中 C 是尺度因子。

因此实际中插值也是两步过程：上采样器 + 低通滤波器，如图 2.12 所示。输出信号

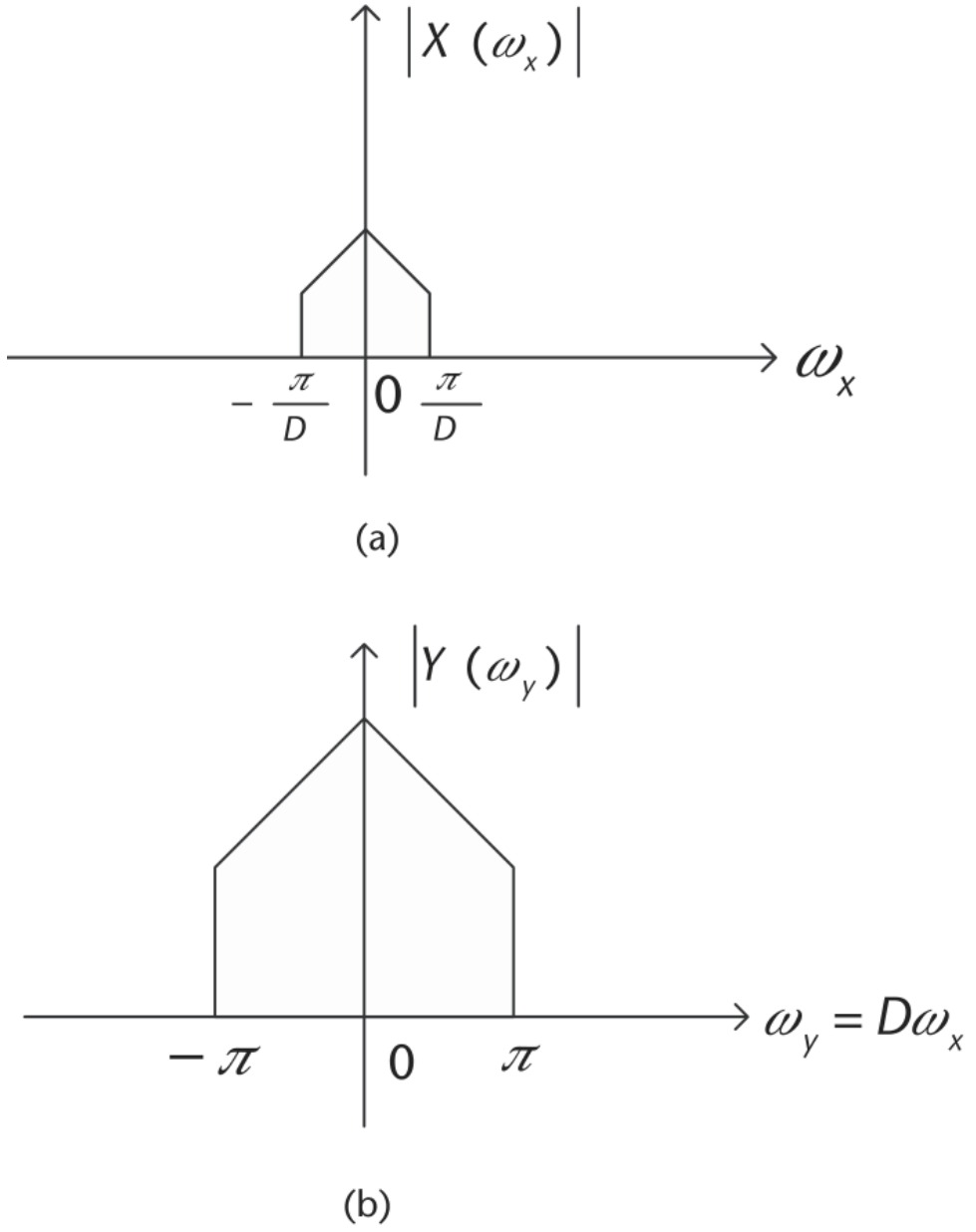


图 2.11　序列 x[n] 与 y[n] 的频谱，ωx 的频率范围被拉伸 D 倍成为 ωy 的范围：(a) 中间序列频谱；(b) 抽取后序列频谱

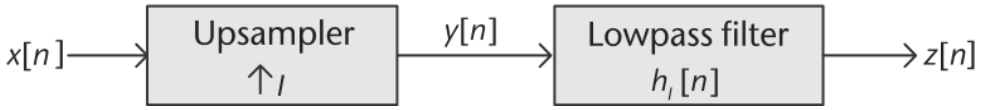


图 2.12　插值的结构：上采样器 + 低通滤波器

$z[n]$ 的频谱为

$$Z(\omega_z) = \begin{cases} CX(I\omega_z) & 0 \le |\omega_z| \le \frac{\pi}{I} \\ 0 & \text{otherwise} \end{cases},$$

其中 $X(\omega)$ 是信号 $x[n]$ 的频谱。

序列 $x[n]$、$y[n]$ 与 $z[n]$ 的频谱示于图 2.13：原始信号的频率范围是 $0 \le |\omega_x| \le \pi$，插值后信号的频率范围是 $0 \le |\omega_z| \le \frac{\pi}{I}$。

动手 MATLAB 示例：现在用 MATLAB 实验抽取与插值。下面的例子会操作两类信号：连续波（CW）信号（正弦波）与随机信号，并在时域和频域可视化上/下采样的效果。先用代码 2.3 生成信号，再用代码 2.4 的低通滤波器把它们带限。基于这些带限版本，我们将观察正确与错误的上/下采样在时域和频域的效果。

图 2.14(a) 与 (e) 中未滤波的正弦信号就是正弦函数的离散版本；图 2.14(b) 与 (f) 中，随机二进制信号是一串随机的 +1 与 −1。用最小二乘线性相位 FIR 滤波器设计（MATLAB 的 `firls` 函数），我们能快速生成截止频率约在 0.21π 的 FIR 滤波器系数。

数据经过代码 2.4 后，滤波后离散信号（正弦与随机二进制）的时域与频域示于图 2.14(c) 与 (g)。图 2.14(e) 的原始正弦与图 2.14(g) 的带限正弦几乎没有差别，因为正弦频率低于代码 2.4 中低通/带限滤波器的截止频率。图 2.14(b) 与 (d) 的随机数据差异一眼可见，频域上图 2.14(h) 相对图 2.14(f) 的带宽收缩也非常明显。时域上我们从足够大的采样处开始观察信号，以避开低通滤波器的起始暂态——这就是为什么我们不看 $t = 0$ 附近的波形。频域上，低通滤波有效限制了这些信号的带宽，让它们在本节后续练习中看得更清楚。

代码 2.3　创建数据集：up-down-sample.m

```
% Create deterministic and stochastic digital data streams
n = 0:1/Fs1:100-(1/Fs1);                % Time index vector
sin_wave = sin(5*n*2*pi);               % Generation of sinusoidal
                                         % signal
random = 2*round(rand(1,length(n)))-1;  % Random string of +1 and
                                         % -1 values
```

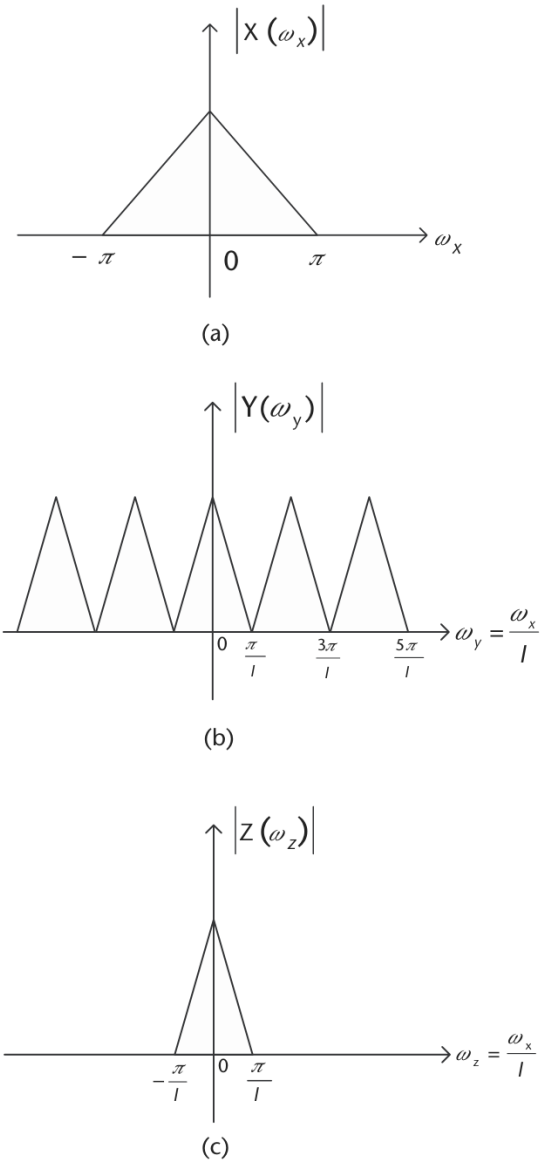


图 2.13　序列 x[n]、y[n] 与 z[n] 的频谱，ωx 的频率范围被压缩 I 倍成为 ωy 的范围：(a) 原始序列频谱；(b) 中间序列频谱；(c) 插值后序列频谱

代码 2.4　创建低通滤波器并带限信号：up-down-sample.m

```
% Create lowpass filter and apply it to both data streams
% b = firls(n,f,a),
%    n is the FIR filter order
%    f is a vector of pairs of frequency points,
%    a is a vector containing the desired amplitude at the points in f
coeffs1 = firls(taps,[0 0.2 0.22 1],[1 1 0 0]);   % FIR filter
                                                   % coefficients
sin_bwlimited = filter(coeffs1,1,sin_wave);
random_bwlimited = filter(coeffs1,1,random);
```

有了滤波后的正弦与随机二进制信号，我们通过观察频域结果来探索上采样的影响。用代码 2.5 的 upsample 函数，把滤波后信号上采样 N 倍（本例 $N = 5$）。注意 upsample 只是在相邻输入采样之间插入 $N - 1$ 个零。它与 interp 插值函数很不一样——后者把上采样与滤波操作合在一起。

上采样的影响从信号前后的频率响应可以清楚观察：我们预期频率响应被上采样因子 N 压缩，并且在原频带上出现 N 个周期副本。看图 2.15(a) 与 (b)，经代码 2.5 以 $N = 5$ 上采样后的正弦与随机二进制信号正是如此。

对比图 2.14(g) 与图 2.15(e) 的滤波正弦，或图 2.14(h) 与图 2.15(f) 的滤波随机数据：两种情形都能清楚看到频谱被上采样因子压缩、并出现周期副本。幅度显然也变了，因为插零影响了平均信号幅度。虽然文献常称之为" 压缩"，有趣的是信号频率其实并没有变——只是相对于 $-\frac{f_s}{2}$ 到 $\frac{f_s}{2}$ 的刻度出现在了不同位置。因此要记住：两幅图中的 f_s 并不相同（相差 5 倍），这里对记号略有滥用。

看过上采样如何压缩频率响应并在频谱上制造周期副本，现在探索下采样这些信号会怎样——频率扩张但不混叠，或者产生大量混叠。回忆下采样过程是周期性地丢弃 $M - 1$ 个采样，其结果是频率响应扩张 M 倍。当频谱开始与它以 2π 为周期的副本（采样信号的频谱每 2π 重复一次）重叠时，问题就来了。重叠导致混叠，信号失真，接收端很难恢复。代码 2.6 将对带限信号下采样，代码 2.5 先把信号上采样 $M = 3$ 倍。注意 MATLAB 的 downsample 函数与 decimate 不同，后者把下采样与滤波合并完成抽取。

代码 2.6 生成的图 2.15(g) 与 (h) 展示了错误情形：下采样前没有滤除周期副本，产生混叠。接着代码 2.7 做必要的滤波，在下采样前去除周期副本；这段代码还按上采样率做了幅度校正以补偿这些操作。

代码 2.5　上采样信号：up-down-sample.m

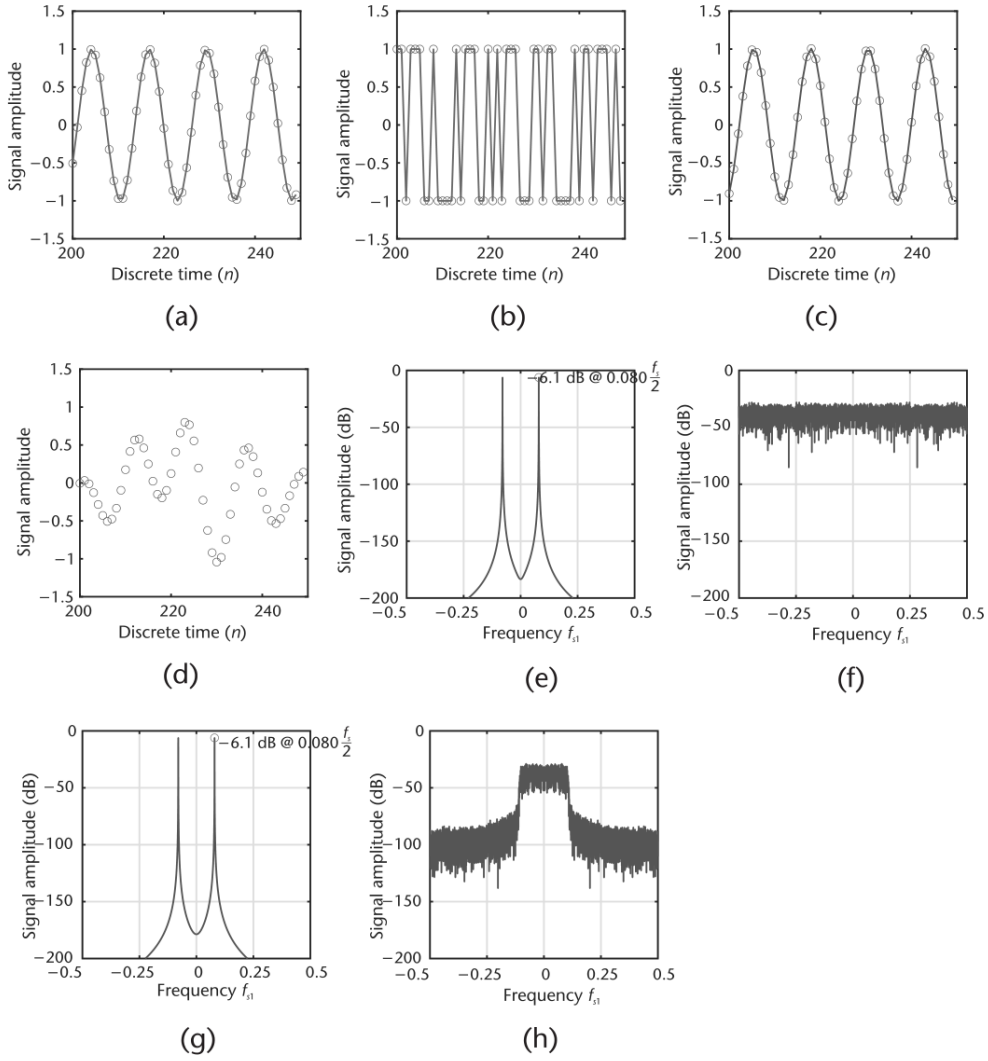


图 2.14　由代码 2.3 创建、经代码 2.4 带限的正弦与随机数据：(a) 原始正弦：时域；(b) 原始随机数据：时域；(c) 带限正弦：时域；(d) 带限随机数据：时域；(e) 原始正弦：频域；(f) 原始随机数据：频域；(g) 带限正弦：频域；(h) 带限随机数据：频域

```
% y = upsample(x,n)
%     increases the sampling rate of x by inserting (n - 1) zeros
%     between samples.
N = 5;
sin_up = upsample(sin_bwlimited,N);
random_up = upsample(random_bwlimited,N);
```

代码 2.6　下采样信号：up-down-sample.m

```
% Attempt to downsampling by M without filtering
% This is incorrect, but is instructive to show what artifacts occur
M = 3;
sin_up_down = downsample(sin_up,M);
random_up_down = downsample(random_up,M);
```

代码 2.7　先低通滤波再下采样：up-down-sample.m

```
% Lowpass filtering of baseband periodic replica followed by
% downsampling
% (correct approach)
coeffs2 = firls(taps,[0 0.15 0.25 1],[N N 0 0]); % FIR filter
                                                  % coefficients
sin_up_filtered = filter(coeffs2,1,sin_up);
sin_up_filtered_down = downsample(sin_up_filtered,M);
random_up_filtered = filter(coeffs2,1,random_up);
random_up_filtered_down = downsample(random_up_filtered,M);
```

信号被上采样后，此过程产生的周期副本铺满 $-\pi$ 到 π 弧度（即 $-\frac{f_s}{2}$ 到 $\frac{f_s}{2}$）的频谱。下采样前若不对这些副本充分滤波，它们会扩张进其他周期副本并导致混叠。图 2.15(d) 与 (h) 演示了这一现象：对上采样后的滤波正弦与随机二进制信号下采样。对上采样滤波正弦的下采样，在图 2.15(g) 中看到来自其他整数奈奎斯特带的频谱混叠。操作正确时，应当只剩一个被下采样因子扩张的副本，如图 2.16(g) 与图 2.17(b) 所示。

由于正弦信号频谱很窄、不易观察扩张，再看随机二进制信号的频响（图 2.15(h) 与图 2.17(b)）。两图中混叠清晰可见——上采样副本没有被滤除。相反，滤除上采样副本后，图 2.17(b) 中下采样信号的频响干净、无混叠。

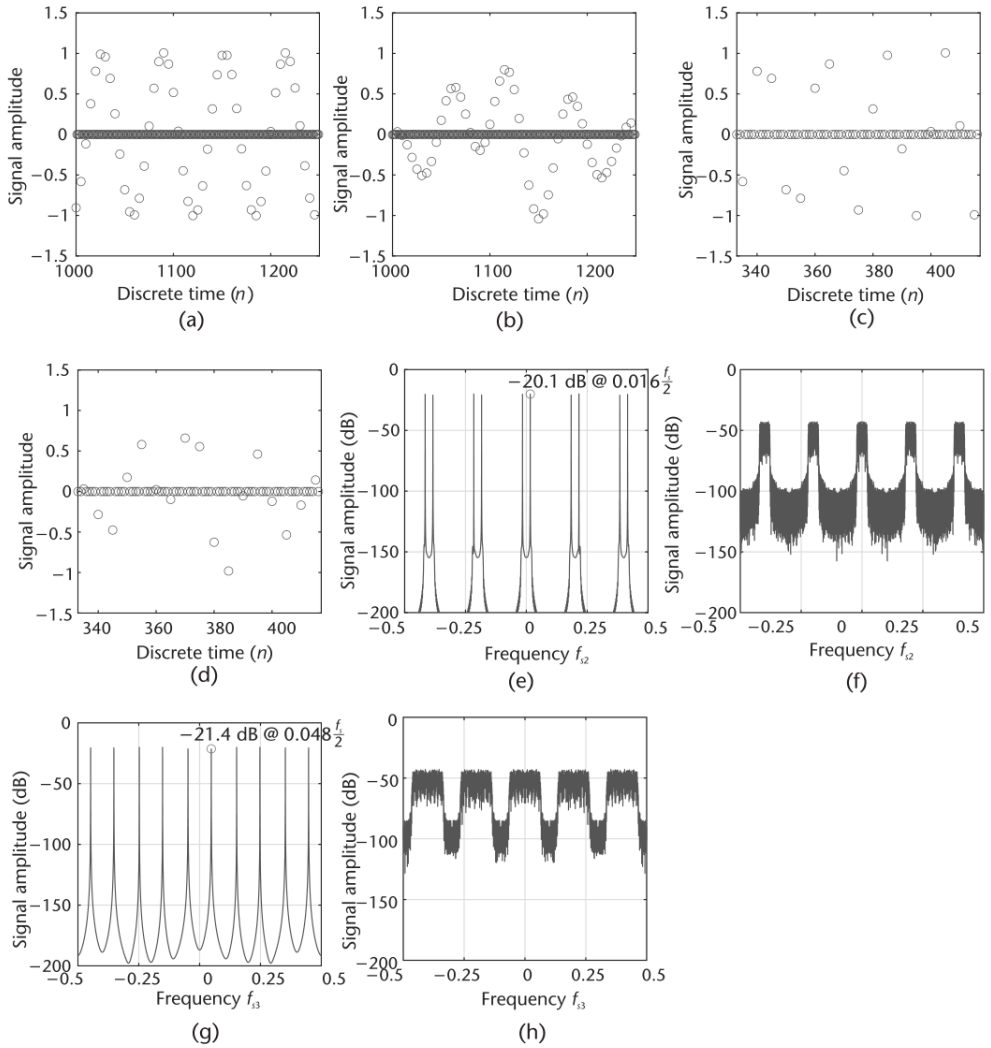


图 2.15　上采样数据（代码 2.5）再经下采样器处理（代码 2.6）：(a) 上采样带限正弦：时域；(b) 带限随机数据：频域；(c) 错误的先上后下采样带限正弦：频域；(d) 错误的先上后下采样带限随机数据：频域；(e) 上采样带限正弦：频域；(f) 带限随机数据：频域；(g) 错误的先上后下采样带限正弦：频域；(h) 错误的先上后下采样带限随机数据：频域

最后在时域比较信号：时域波形在幅度与绝对时间（秒）上几乎完全一致，只是采样率从 f_s 转换到了 $\frac{5}{3}f_s$，同一信号有了更多采样。

2.3 信号表示

理解信号如何表示，能极大提升分析与设计数字通信系统的能力。我们需要多种便利的数学框架来表示真实的射频、基带与噪声信号。通常有两种：**包络/相位**与**同相/正交**，两者都能在时域与频域中表达。

2.3.1 频率转换

要理解信号如何从基带搬到射频、又从射频搬回基带，需要细看调制器与解调器。例如图 2.18 是一个非常经典的正交调制器：ADL5375 接收两路差分基带输入与一个单端本振（LO），产生单端输出。LO 接口在内部生成两路相互正交（相差 90°）的本振信号，驱动混频器——混频器做的就是把 LO 信号与输入相乘。

数学上，这个混频过程取两路输入 I_{BB} 与 Q_{BB}（记作 $I(t)$ 与 $Q(t)$），分别与频率 ω_c 的 LO 相乘，再把结果相加得到发射信号 $r(t)$。与 $Q(t)$ 相乘的 LO 被移相 90°，使之与乘 $I(t)$ 的 LO 正交。于是有：

$$r(t) = I(t)\cos(\omega_c t) - Q(t)\sin(\omega_c t).$$

LO 频率记作 ω_c，因为它通常被称为**载波频率**——这里利用了同相分量（$I(t)\cos\omega_c t$）与正交分量（$Q(t)\sin\omega_c t$）之间的相位关系。于是发射信号同时包含两个分量，但看上去只是一条单一的正弦曲线。

接收端，我们用相似而相反的过程把 $r(t)$ 下变频回同相与正交基带信号。把同一 LO（含第二路移相分量）作用于 $r(t)$ 再低通滤波，得到

$$I_r(t) = \mathrm{LPF}\{r(t)\cos(\omega_c t)\} = \mathrm{LPF}\{(I(t)\cos(\omega_c t) - Q(t)\sin(\omega_c t))\cos(\omega_c t)\} = \frac{I(t)}{2},$$

$$Q_r(t) = \mathrm{LPF}\{-r(t)\sin(\omega_c t)\} = \mathrm{LPF}\{(-I(t)\cos(\omega_c t) + Q(t)\sin(\omega_c t))\sin(\omega_c t)\} = \frac{Q(t)}{2}.$$

实践中，发射机 LO 与接收机 LO 之间存在相差，会使 $r(t)$ 发生旋转，但 $I(t)$ 与 $Q(t)$

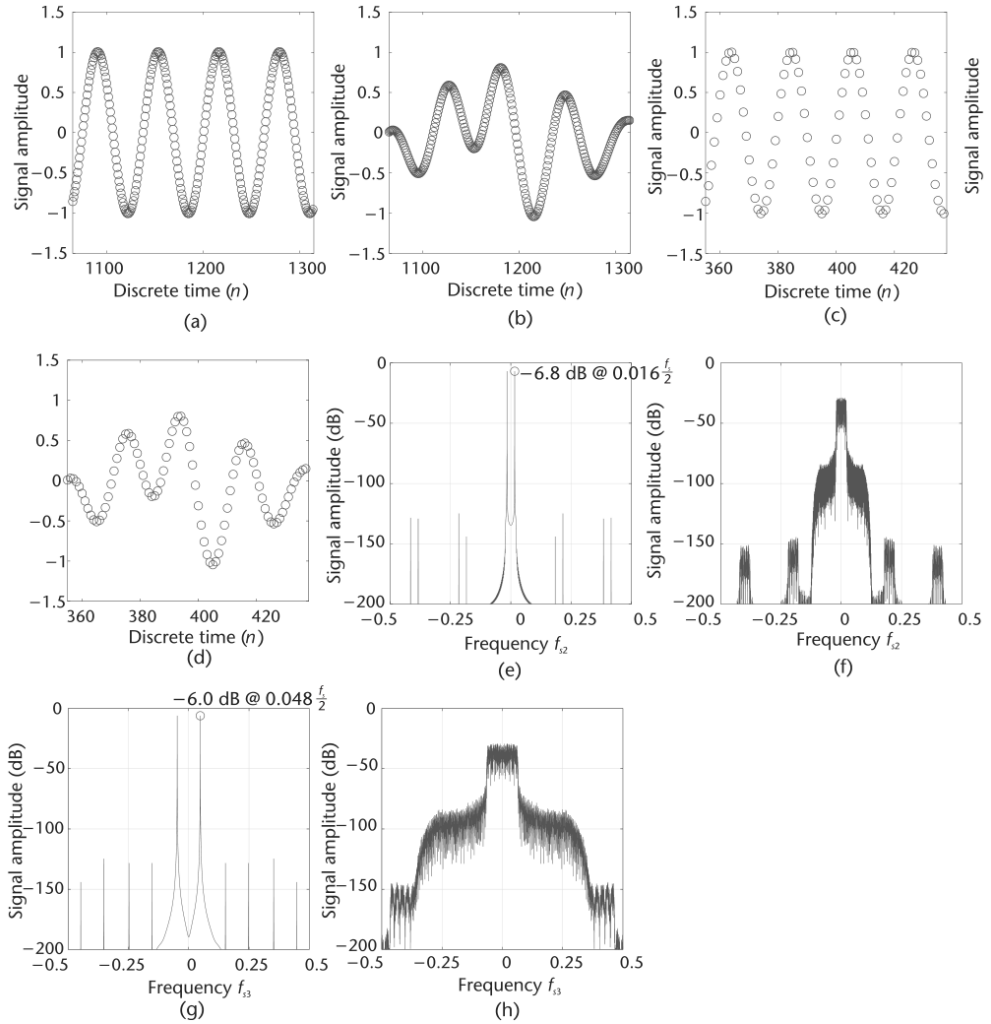


图 2.16　经代码 2.7 处理：N = 5 上采样后滤波的带限波形。仍能看到信号副本，但滤波器已将其压到很低电平；滤波也校正了上采样造成的幅度损失。(a)(c) 上采样后滤波的带限正弦：频域；(b)(d) 上采样、滤波再下采样的带限正弦：频域；(e)(f) 上采样后滤波的带限随机数据：频域；(g) 上采样后滤波的带限随机数据：频域；(h) 上采样、滤波、带限的随机数据：频域

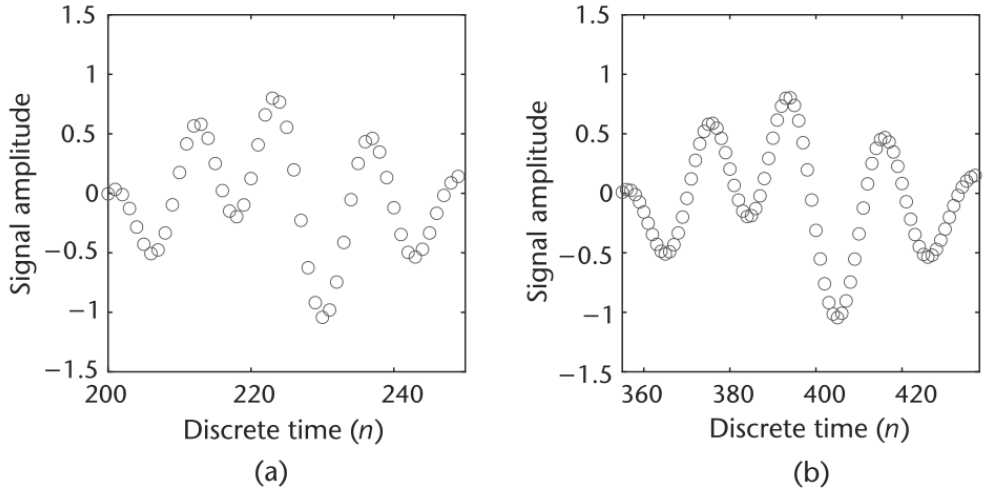


图 2.17　N = 5 上采样、滤波、M = 3 下采样后的带限随机数据。滤波引入了相移（时移）。(a) fs 下的原始带限随机数据时域；(b) 5/3 · fs 下的上采样滤波带限随机数据时域

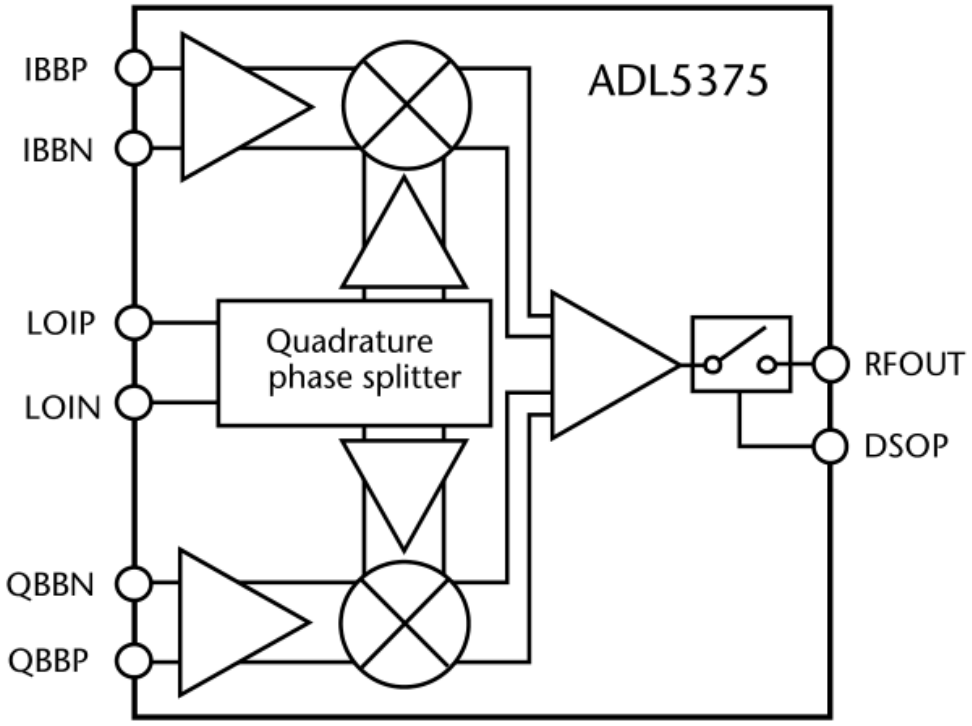


图 2.18　ADL5375 宽带正交调制器，频率范围 400 MHz 至 6 GHz

之间的相位关系总能保持。

2.3.2 虚数信号

把信号称为" 正交信号" 或" 复信号"，是在借用欧拉等人创造的数学构造来简化分析。我们尽量用同相（I）/正交（Q）来称呼信号，因为那才是严格正确的说法。如 2.3.1 节所述，**同相**（I）指与本振同相位的信号分量，**正交**（Q）指与移相 90° 后的本振同相位的分量。

把 I 看作实部、Q 看作虚部很方便，因为这能启用许多数学技巧，但归根结底只是个构造。这种便利的典型例子就是混频器完成的频率搬移。从欧拉关系式出发：

$$e^{jx} = \cos(x) + j\sin(x),$$

其中可把 x 定义为目标频率乘时间。沿用 2.3.1 节的约定但把 $I(t)$、$Q(t)$ 重新定义为实部与虚部，得到

$$y(t) = I(t) + jQ(t).$$

假设 $y(t)$ 是频率 ω_a 的连续波单音（便于说明），则

$$y(t) = \cos(\omega_a t) + j\sin(\omega_a t).$$

应用 (2.28)，可以把 $y(t)$ 搬移到期望频率 ω_c：

$$\begin{aligned} y(t)e^{j\omega_c t} &= I(t)\cos(\omega_c t) - Q(t)\sin(\omega_c t) \\ &\quad + j(Q(t)\cos(\omega_c t) + I(t)\sin(\omega_c t)) \\ &= \cos((\omega_c + \omega_a)t) + j\sin((\omega_c + \omega_a)t). \end{aligned}$$

简单套用欧拉恒等式，我们的信号就出现在了频率 $\omega_a + \omega_c$ 上。

再从硬件角度体会复数表示的用处。考虑图 2.18 的混频器：IQ 混频器可以发射任意相位与幅度的信号（在功率约束内）。这就是我们通过相位与幅度的差异把信息编码进发射数据的例子。它实际上正是通过同相与正交信号的关系实现的——可以用来产生具有任意相位与幅度的单一正弦波。数学上，用两个正交分量（正弦与余弦）可以合成具有特定包络与相位 (A, φ) 的正弦波：

$$A\sin(\omega t + \varphi) = (A\cos\varphi)\sin(\omega t) + (A\sin\varphi)\cos(\omega t).$$

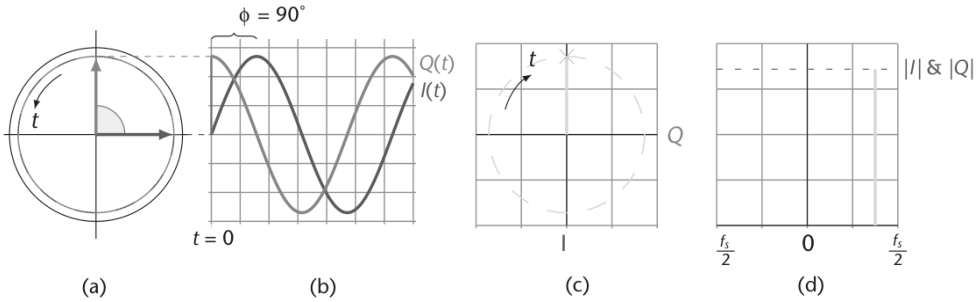


图 2.19　同一连续波信号在多个域中的表示：(a) 相量 rad(t)；(b) 时域 x(t)；(c) 笛卡尔 (I, Q)(t)；(d) 频域 X(ω)

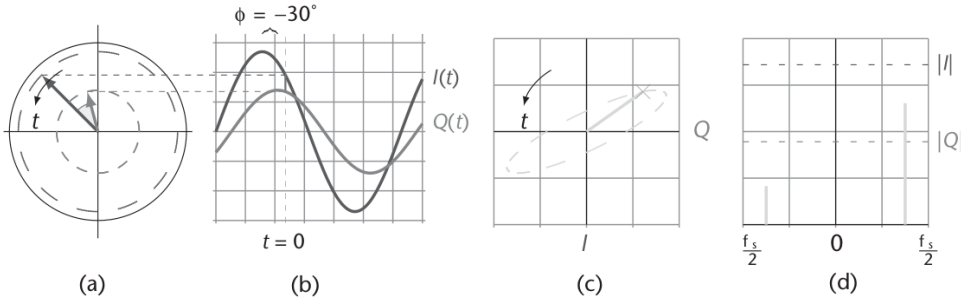


图 2.20　连续波信号的幅度与相位差异在各域中引起的变化：(a) 相量 rad(t)；(b) 时域 x(t)；(c) 笛卡尔 (I, Q)(t)；(d) 频域 X(ω)

因此，只需随时间调整正弦与余弦分量的幅度，就能用固定频率与相位的 LO 产生想要的波形。也可以换用其他坐标系来观察复数值：直角坐标形式的复数可以变换为幅值-角度（极坐标）表示，甚至画成矢量。

图 2.19 中的同相与正弦正交正弦波展示了相量图（phasor plot）随时间推移的样子——指示幅度的矢量绕轴旋转，I 与 Q 之间的相移清晰可见。时域中也能看出同相与幅度之间的差异，只是相位差稍微隐蔽些。在笛卡尔平面上，信号随时间呈现为一个旋转的圆。相量图总是随时间逆时针旋转；笛卡尔图则可顺可逆，取决于 I 与 Q 之间相位差的正负。时域图显示的是随时间的变化，而相量图与笛卡尔平面是某个时刻的快照（取时域图的 $t = 0$）。频域图给出相量的频谱，但只传达幅度、丢失相位信息——因为我们只画了频谱的实部分量。对比图 2.19 与图 2.20：I 与 Q 分量的变化（幅度与相位关系）也会体现在其他域中。

2.4 信号指标与可视化

工程师要判断一个项目是否完成，需要某种形式的验证。但通信系统收发链路之深、集成度之高，使其评估相当复杂。如 1.4 节所述，通信系统除尺寸、重量、功耗与成本（SWaP-C）外还有许多指标。误码率（BER）、数据吞吐量与通信距离等性能指标也只是**顶层**系统指标。正如整个系统有系统级指标，各子系统与 SDR 构建模块也有各自的指标与测量方法——这样我们才知道不会把特定组件设计得过剩或不足。但折中永远要在系统层面考量，因为上游的改动会影响下游的组件与实现。

系统级指标的达成，靠的是确保系统中每一块都能支撑这些指标。打造世界一流的通信系统，需要世界一流的硬件与世界一流的算法设计及实现。这正是工程学许多方面的难点所在——定量地判断某件事是否完成、是否可用、能否接入系统的其余部分。通信系统" 整体性能取决于最薄弱的环节" 这句话再正确不过：如果你把一切都拼装好，而误码率这类系统级指标不达标，你又不会测量通信系统的每一部分——从射频到 SDR 到算法——那你就会束手无策，不知道下一步怎么办。

取决于你要看什么，有无数技术与工具几乎可以测量一切。学通信的人，不适合不愿严谨的人。

2.4.1 SINAD、ENOB、SNR、THD、THD+N 与 SFDR

量化模拟动态性能的六个流行指标见表 2.2 [6]。列出并理解这些测量，会帮助你分析设计、确保设计足够稳健。尽管多数器件与系统厂商对这些指标采用了相同定义，例外仍然存在。由于它们在器件与系统比较中的重要性，不仅要准确理解每个指标的含义，还要理解指标之间的关系。

表 2.2　六个流行指标

指标	含义	MATLAB 函数
SFDR	无杂散动态范围	`sfdr`
SINAD	信纳比（信号对噪声加失真比）	`sinad`
ENOB	有效位数	—
SNR	信噪比	`snr`
THD	总谐波失真	`thd`
THD+N	总谐波失真加噪声	—

- **无杂散动态范围（SFDR）**是信号均方根（RMS）值与最差杂散信号 RMS 值之比，无论该杂散落在频谱何处。最差杂散可能是也可能不是原信号的谐波。它通常在感兴趣带宽上测量——除非另有说明，该带宽即奈奎斯特带宽：基带为 DC 到 $\frac{f_s}{2}$，

复数（射频）转换器为 $-\frac{f_s}{2}$ 到 $\frac{f_s}{2}$，Pluto SDR 这类器件用的就是后者。SFDR 在通信系统中很重要，因为它代表了能从大干扰信号（阻塞信号）中分辨出的最小信号值。SFDR 一般随信号幅度变化绘制，可以相对于信号幅度（dBc）或 ADC 满量程（dBFS）表示，如图 2.21 所示。MATLAB 的 `sfdr` 函数以 dBc 给出结果。对接近满量程的信号，峰值频谱杂散通常由基频的前几次谐波决定；但当信号低于满量程若干 dB 时，通常出现并非输入信号直接谐波的杂散——这是系统的微分非线性在较小信号时占主导所致。因此 SFDR 考虑所有来源的失真而不问出处，是评估各类通信系统的有力工具。

- **总谐波失真（THD）**是基波信号 RMS 值与其各次谐波均方根值平均之比（一般只有前五次谐波显著）。ADC 的 THD 通常也以接近满量程的输入来规定，但可以在任意电平规定。
- **总谐波失真加噪声（THD+N）**是基波信号 RMS 值与其各次谐波加全部噪声分量（不含 DC）的均方根平均之比。必须规定噪声测量的带宽：FFT 情形下为 DC 到 $\frac{f_s}{2}$（若测量带宽就是奈奎斯特带宽，THD+N 等于 SINAD）。
- **信纳比（SINAD，或 S/(N+D)）**是信号 RMS 幅度与其余所有频谱分量（含谐波、不含 DC）均方根平均之比。SINAD 是模拟系统整体动态性能的好指标，因为它涵盖了构成噪声与失真的所有分量。SINAD 常针对不同输入幅度与频率表征。给定输入频率与幅度时，只要两者噪声测量带宽相同（奈奎斯特带宽），SINAD 等于 THD+N。
- **信噪比（SNR，有时称不含谐波的 SNR）**从 FFT 数据按与 SINAD 相同的方式计算，只是把信号谐波排除在外、只留噪声项。实践中只需排除前五次谐波，因为它们占主导。SNR 在高输入频率时会恶化，但由于排除了谐波项，一般不像 SIND 那样快。

2.4.2 眼图

虽说有点废话：时域图用来观察电信号随时间的变化。幅度、频率、上升时间、时间间隔、失真、噪底等任何现象都可以被经验地测定，也可以看这些特性如何随时间变化。在电信中，**眼图**（eye diagram，又称眼型图）是一种时域显示：把接收机的数字数据信号反复采样加到垂直输入，用数据速率触发水平扫描 [9]。之所以叫眼图，是因为图案看起来像一对轨道之间的一系列” 眼睛”。

通过分析这种显示可以导出若干系统性能度量，尤其是**码间串扰**（ISI）的程度：眼闭合，ISI 增大；眼张开，ISI 减小。此外，信号太长、太短、与时钟同步不佳、太高、太低、太噪、跳变太慢、下冲或上冲过大，都能从眼图观察到。例如，图 2.22 给出了带

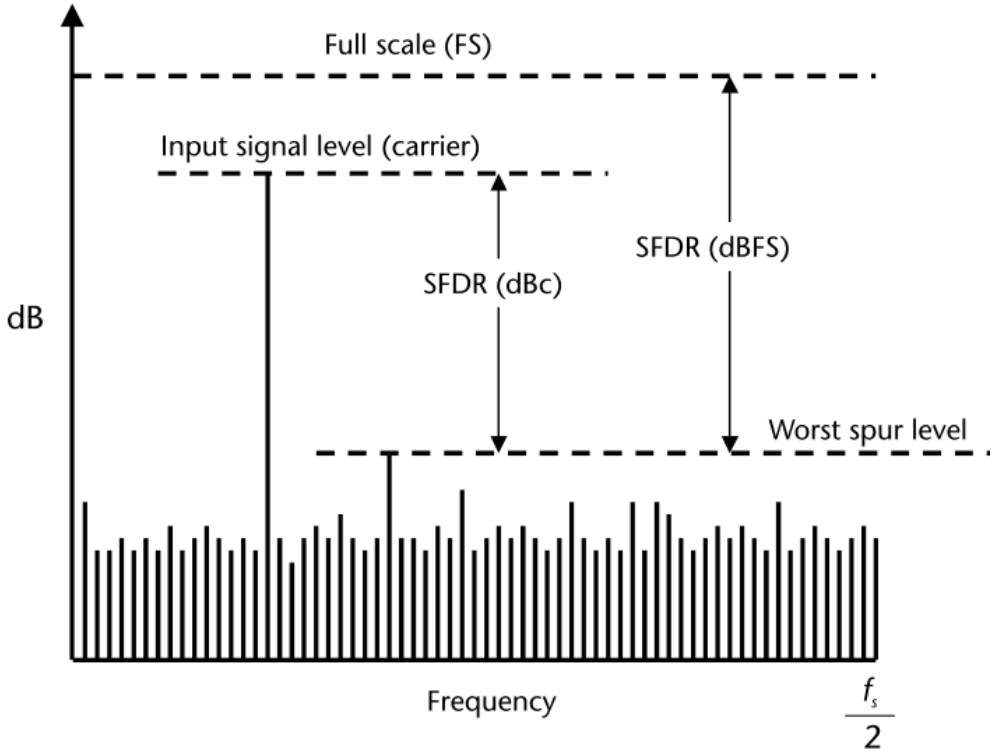


图 2.21　BW 为 DC 到 fs/2 时的无杂散动态范围（SFDR）

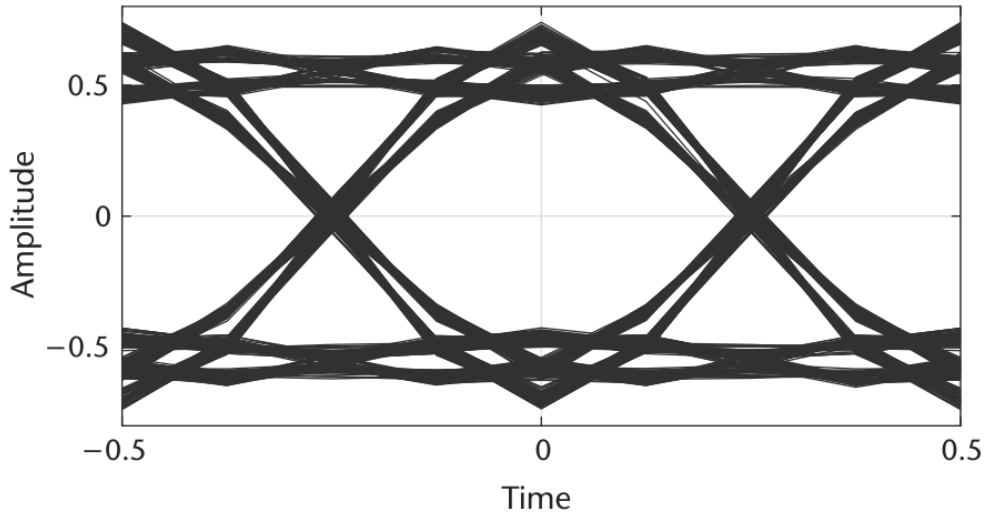


图 2.22　BPSK 信号的典型眼图。张开宽度指示可用于检测采样的时间范围；最佳采样时刻对应最大眼张开处，对噪声的防护最强

噪声的 QPSK 信号的典型眼图。

由于眼图传达并测量许多不同类型的关键数据，它有助于量化一个算法或系统工作得如何。两个关键测量是：**垂直张开度**（BER 判决门限点之间的距离）与**眼高**（眼电平之间的最小距离）。眼的垂直与水平张开度总是越大越好。

动手 MATLAB 示例：为获得眼图的动手经验，用 MATLAB 的 `eyediagram` 函数——这是可视化分析传输中噪声量与码间串扰的利器。用脉冲成形后的信号，应当能观察到传输中存在的任何失真。

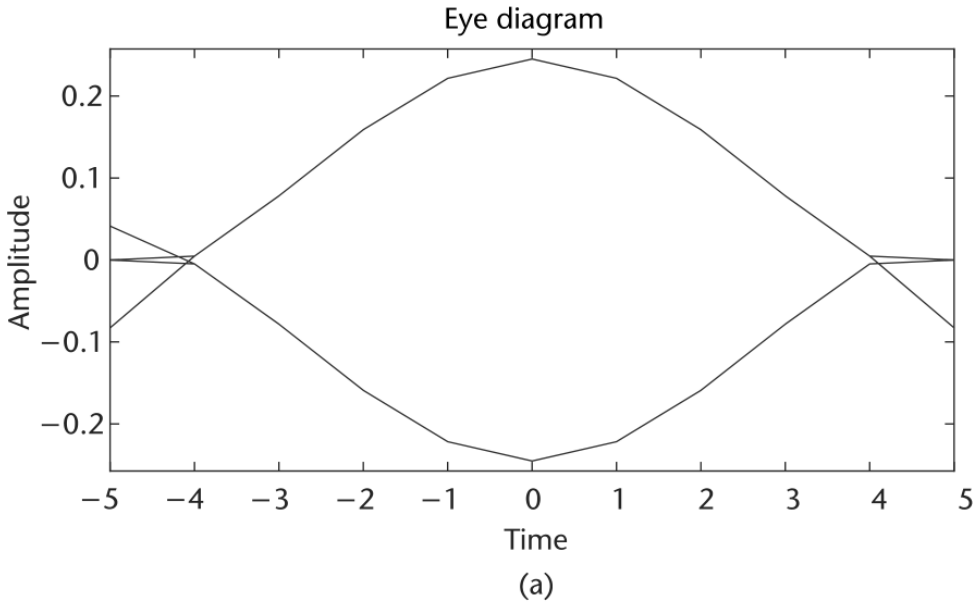
从图 2.23 可见，脉冲成形传输不含任何失真（我们预先知道，因为一开始就没往传输里加噪声与失真）。图 2.23(a) 与 (b) 这样的眼图，我们称为眼**张开**。传输中存在失真的首要标志，是时刻 0 处的孔径没有达到最大。

再来考察传输中存在失真的情形及其眼图表现。取代码 2.8 的 `y_impulse1` 与 `y_impulse2` 输出信号，往里加一些噪声——代码 2.9 用 `randn` 函数加了高斯噪声。

在图 2.24 中能清楚看到附加噪声对发射信号眼图的影响：图 2.24(a) 与 (b) 中眼图的眼壁相比图 2.23(a) 与 (b) 不再平滑。两种情形下眼仍张开——我们只加了少量噪声；大量噪声有可能把眼闭上，意味着时刻 0 处期望的采样时刻可能被破坏、转化为误码。本书后面还会探索其他形式的失真如何影响眼图的孔径。

代码 2.8　眼图示例：eye_example.m

```
% Create impulse train of period L and length len with random +/- one
% values
```



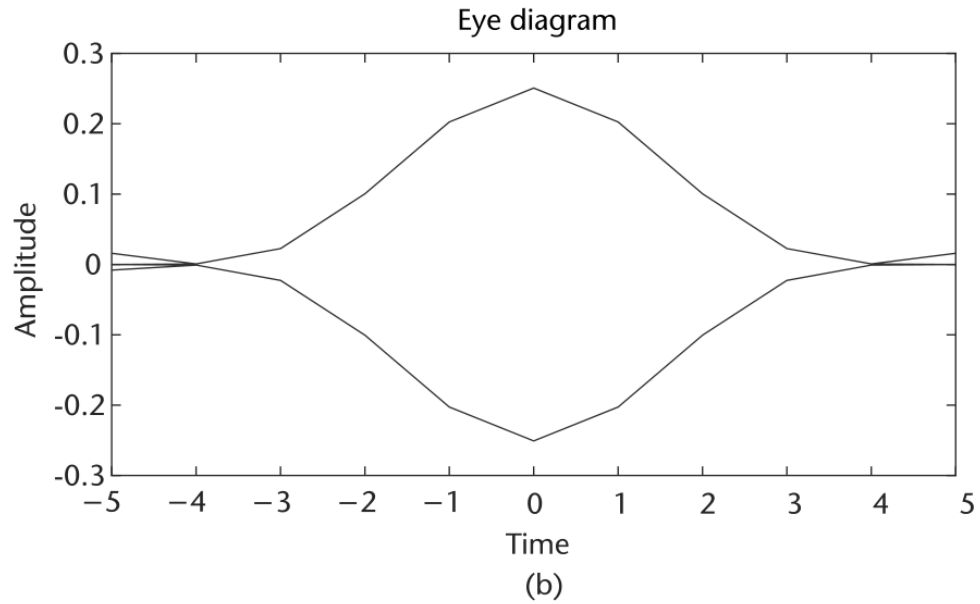


图 2.23　具有矩形频率响应与三角频率响应的系统滤波后的信号眼图：(a) 矩形频率响应脉冲滤波；(b) 三角频率响应脉冲滤波

```
L = 10;
impulse_num = 100; % Total number of impulses in impulse train
len = L*impulse_num;
temp1 = [2*round(rand(impulse_num,1))-1 zeros(impulse_num,L-1)];
x_impulse = reshape(temp1.',[1,L*impulse_num]);

% Create two transmit filter pulse shapes of order L
% Approximate rectangular frequency response --> approximate
% sinc(x) impulse response
txfilt1 = firls(L,[0 0.24 0.25 1],[4 4 0 0]);
% Approximate triangular frequency response --> approximate
% sinc(x)^2 impulse response
txfilt2 = firls(L,[0 0.5 0.52 1],[4 0 0 0]);

% Pulse shape impulse train
y_impulse1 = filter(txfilt1,1,x_impulse);
y_impulse2 = filter(txfilt2,1,x_impulse);

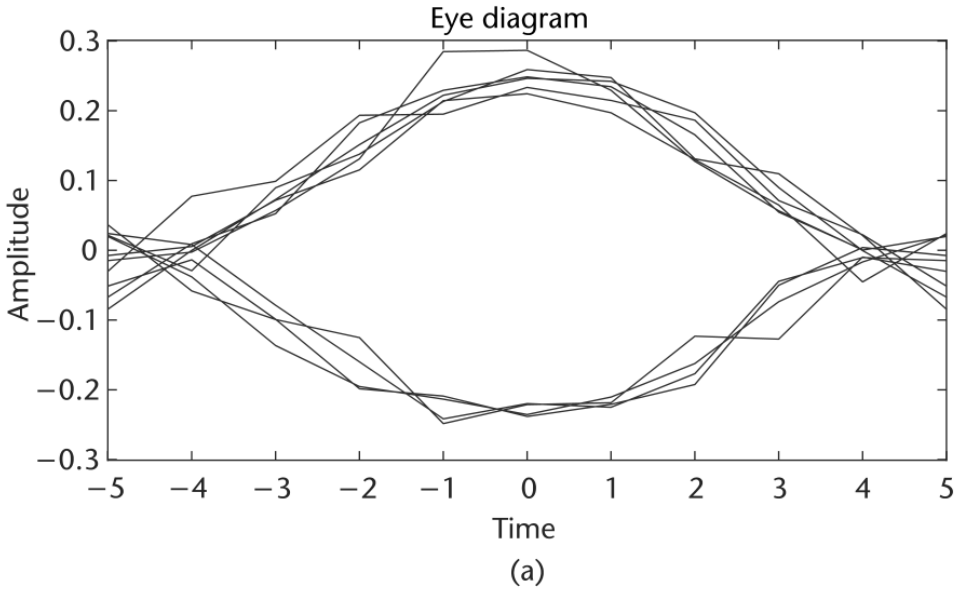
% eyediagram(x,n,period,offset)
% creates an eye diagram for the signal x, plotting n samples in each
% trace, horizontal axis range between -period/2 and period/2.
eyediagram(y_impulse1,L,L,floor(L/2));
eyediagram(y_impulse2,L,L,floor(L/2));
```

代码 2.9　眼图示例：eye_example.m

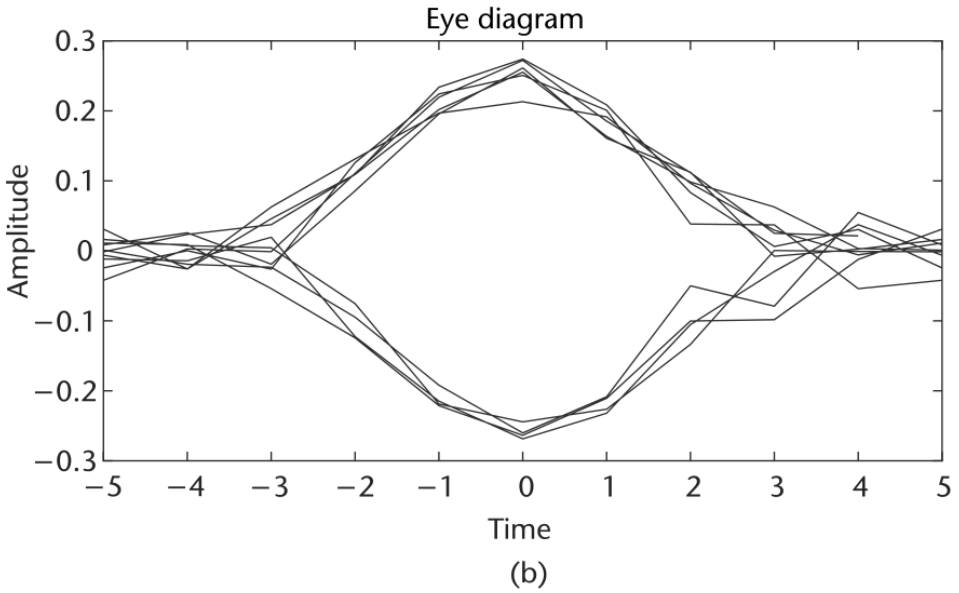
```
eyediagram((y_impulse1+0.1*randn(1,length(y_impulse1))),L,L,floor(L/2));
eyediagram((y_impulse2+0.1*randn(1,length(y_impulse2))),L,L,floor(L/2));
```

2.5 SDR 接收技术

现代通信的研究在模拟与数字两个域之间保持着极大的二元性。两个域都被高效、高速地处理。但模拟信号保持着" 无限精度" 的视角，本质上又总带有某种随机性；数字



(a)



(b)

图 2.24　含加性高斯白噪声时，矩形与三角频率响应系统滤波后的信号眼图：(a) 矩形频率响应脉冲滤波；(b) 三角频率响应脉冲滤波

信号则精确、定义明确，却受限于计算复杂度与基本原理的边界。数字通信必须同时管好这两个世界，才能在节点之间设计出稳健的链路。因此两个域高度相互依赖。

即使在当今 DSP 器件丰富而廉价的世界上，模拟信号仍然是先被处理、放大、滤波，然后才由 ADC 转换为二进制形式。ADC 的输出只是模拟信号的二进制表示，交给从 FPGA 到通用 CPU 的各种计算单元处理。处理完毕后，从数字化信号获得的信息又可以用数模转换器（DAC）变回模拟形式。SDR 物理接收的信号始于时变电场——它在接收天线上感应电流、在接收机处形成可检测的电压；而 SDR 的发射则是把时变电压加到天线上，驱动电子运动形成向外辐射的电场。

2.5.1 奈奎斯特区

2.2.3 节考虑的是基带采样的情形（即所有感兴趣的信号都在第一奈奎斯特区内）。图 2.25 展示了这样一种情形：被采样信号的频带限制在第一奈奎斯特区内，原始频带的镜像出现在其余每个奈奎斯特区。再看图 2.25 B 所示的情形：被采样的信号频带完全落在第二奈奎斯特区内。对第一奈奎斯特区之外的信号采样，常称为**欠采样**或**谐波采样**。注意落入第一奈奎斯特区的那份镜像，除原始位置外包含了原信号的全部信息。

图 2.25 还展示了被限制在第三奈奎斯特区内的采样信号。注意落入第一奈奎斯特区的镜像没有发生频率翻转。事实上，被采样信号可以位于任何一个奈奎斯特区内，落入第一奈奎斯特区的镜像仍是准确的表示。至此我们可以把奈奎斯特准则重述为：

> 信号必须以等于或大于其带宽两倍的速率采样，才能保留全部信号信息。

注意，这里完全没有提及被采样信号频带在频谱中的**绝对位置**相对于采样频率的关系。唯一的约束是：被采样信号的频带必须限于单个奈奎斯特区内（即不得跨越 $\frac{f_s}{2}$ 的任何整数倍）。事实上，这正是抗混叠滤波器的主要功能。

对第一奈奎斯特区以上的信号采样，在通信中已经流行起来，因为该过程等价于模拟解调。直接采样中频（IF）信号再用数字技术处理正成为常见做法，从而省去 IF 解调器与滤波器。但随着 IF 频率升高，对 ADC 的动态性能要求（带宽、线性度、失真等）变得更苛刻——因为性能必须在第二或第三奈奎斯特区依然够用，而不只是基带。这对许多为第一奈奎斯特区设计的 ADC 是个难题。因此适合欠采样应用的 ADC 必须在更高阶奈奎斯特区保持动态性能。这在 Pluto SDR 这类器件上尤为重要：它内置 DC 校正来去除本振泄漏。该 DC 校正可以理解为紧贴 DC（25 kHz）的高通滤波器。对 QPSK、QAM 这类并不严格在 DC 传信息的调制方案，这无关紧要；而对在 DC 传信息的调制

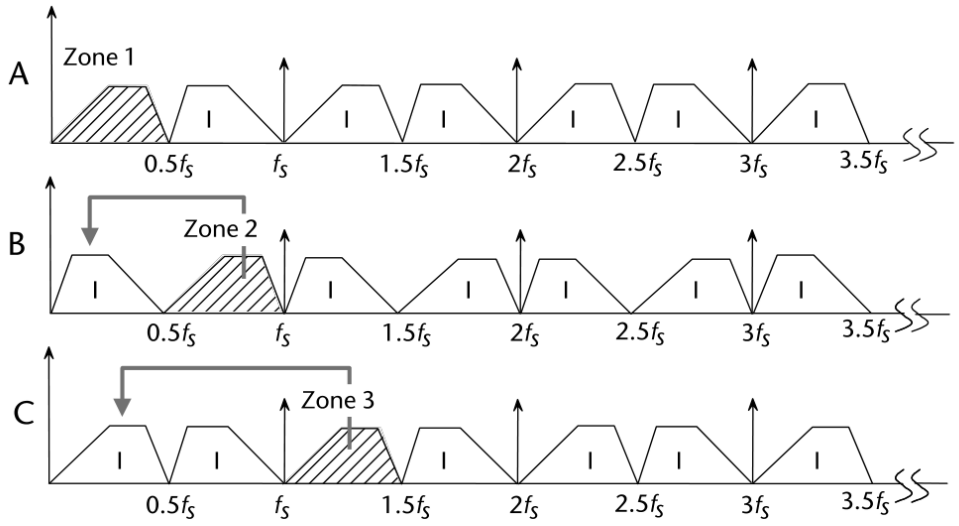


图 2.25　奈奎斯特区折叠

方案，不用上述欠采样技术就很难绕开这个问题。在 DC 以上采集数据、再用数字方式搬下来，可以改善性能。

2.5.2 定点量化

理想 N 位数据转换器仅有的误差（DC 或 AC）只与采样和量化过程有关。理想转换器数字化信号时的最大误差是 $\pm\frac{1}{2}$ LSB，正好落在两个数字值中间。这一点从图 2.26 理想 N 位 ADC 的传递函数看很明显。对任何跨越多个 LSB 的交流信号，其量化误差可以近似为一个峰峰幅度为 q（一个 LSB 的权重）的不相关锯齿波。这一分析虽不精确，但对大多数应用足够准确。

量化误差随时间的变化示于图 2.27。同样，一个简单的锯齿波就提供了足够精确的分析模型。锯齿误差的方程为

$$e(t) = st, \quad -\frac{q}{2s} < t < \frac{q}{2s},$$

其中 s 是量化噪声的斜率。$e(t)$ 的均方值可写为

$$\overline{e^2(t)} = \frac{1}{q/s}\int_{-q/2s}^{q/2s}(st)^2 dt = \frac{q^2}{12}.$$

(2.34) 的平方根即 RMS 量化噪声误差，近似为高斯分布、或多或少均匀地分布在 DC 到 $\frac{f_s}{2}$ 的奈奎斯特带宽内。

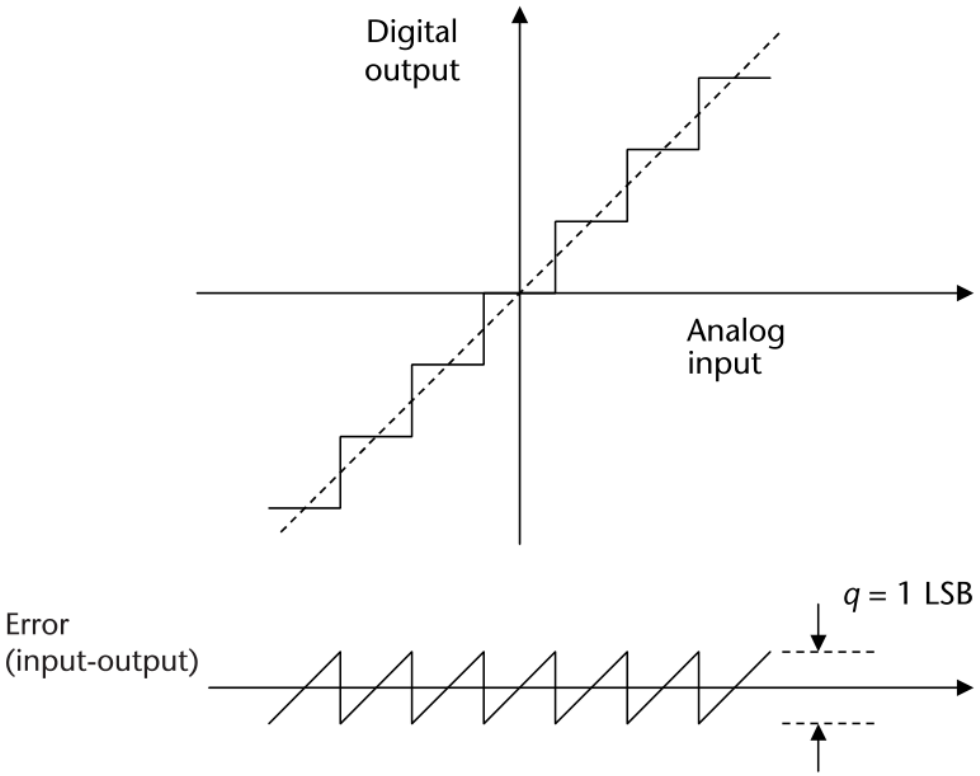


图 2.26　理想 N 位 ADC 的量化噪声

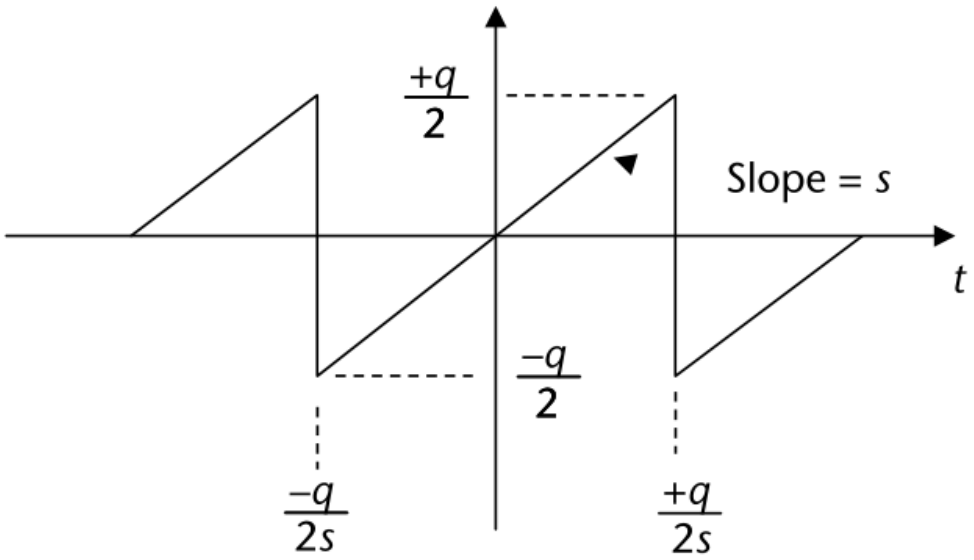


图 2.27　理想 N 位 ADC 的量化噪声随时间变化

现在可以计算满量程正弦输入 $v(t)$ 下的理论 SNR：

$$v(t) = \frac{q2^N}{2}\sin(\omega t),$$

先算输入信号的 RMS 值：

$$\overline{v(t)} = \frac{q2^N}{2\sqrt{2}}.$$

于是理想 N 位转换器的 RMS 信噪比为

$$\text{SNR} = 20\log_{10}\frac{\text{满量程输入的 RMS}}{\text{量化噪声的 RMS}} = 20\log_{10}\frac{\frac{q2^N}{2\sqrt{2}}}{\frac{q}{\sqrt{12}}}.$$

化简 (2.37) 后得到以 dB 表示的 SNR：

$$\text{SNR} = 20\log_{10}2^{2N}\sqrt{3} = 6.02N + 1.76.$$

同样，这是 DC 到 $\frac{f_s}{2}$ 带宽上的结果。许多应用中，感兴趣的信号实际只占更小的带宽（BW）。例如用数字滤波滤除 BW 之外的噪声分量后，就要在公式里加一个修正因子（称为**处理增益**）来体现 SNR 的提高。以大于信号带宽两倍的速率采样，常称为**过采样**（oversampling）。过采样配合量化噪声整形与数字滤波，正是 sigma-delta 转换器

的核心概念，将在 2.5.4 节讨论。

动手 MATLAB 示例：2.5.2 节中我们做了如下假设 [11]：

- 误差样本序列 $e(t)$ 是某平稳随机过程的样本序列；
- 误差序列与精确样本序列 $v(t)$ 不相关；
- 误差过程的随机变量互不相关，即误差是白噪声过程；
- 误差过程的概率在量化误差范围内均匀分布。

这里的根本假设是量化噪声与输入信号不相关。我们会看到，在某些常见且不起眼的例子中这并不成立。当采样时钟与信号谐波相关时，量化噪声变得相关，能量集中在信号的谐波处。在实际 ADC 应用中，由于宽带输入信号的随机性、以及系统中通常存在的少量噪声（起着抖动信号的作用、进一步随机化量化误差谱），量化误差一般表现为随机噪声。理解上面这点很重要，因为 ADC 的单音正弦 FFT 测试是普遍接受的性能评估方法之一。要准确测量 ADC 的谐波失真，必须采取措施确保测试设置测到的真是 ADC 的失真，而不是量化噪声相关性的伪影。

我们将用代码 2.10 的变体来探索 SFDR，并围绕计算所允许的精度展开这个概念。先从双精度浮点表示（MATLAB 默认）开始。为保持信号不相关，我们选了 3,959,297 Hz 的单音（一个质数）。

这给出了图 2.28 的结果：SFDR 测得 288.96 dBc，非常可观。但 Pluto SDR 这样的真实电台不能接受双精度浮点数，必须使用定点（12 位）表示。

Pluto SDR 与许多其他器件使用的定点格式是带符号格式：最高位（MSB）是符号位，其余 11 位表示幅度。MATLAB 脚本里我们用 2^{11} 作幅度，在发射前最大化信号动态范围。于是乘上该整数值、取整、再缩回 ± 1 以归一化幅度。

这得到了图 2.29 中不太拿得出手的 45.97 dBc SFDR——连 8 位性能都不到。原因是示例中我们把信号缩放到 10^{-3}，而定点数的动态范围不像双精度浮点那样等比例缩放。

为解决动态范围问题，去掉 10^{-3} 缩放、使用 12 位满量程。这给出图 2.30(a) 中体面的 86.82 dBc。要更进一步，可以利用 **FFT 处理增益**的概念：把采样数增加到 10,000（代码 2.15 只是改变了 t 的长度），得到图 2.30(b) 的 93.98 dBc。这只是增加了采样数 → 增加了 FFT bin 数 → 每个 bin 累积的能量减少。

代码 2.10、2.11、2.12 与 2.15 用的是不相关的 F_A = 3,959,297 Hz。如果相关会怎样？把示例中的 F_A 直接取整为 4 MHz（代码 2.13 与 2.14），结果见图 2.31(a)：SFDR 为 82.58 dBc，比图 2.30(b) 损失了 11.4 dB。

为在图 2.31(b) 中挽回损失，可以用**抖动**（dithering）技术：把谐波上累积的能量摊到其余噪底上。噪底更高了，但最差杂散更低——而后者才是计算 SFDR 的考量指标。结果为 91.68 dBc，与图 2.30(b) 的不相关结果只差 2.3 dB。

与其按固定模式向上或向下取整，我们在取整前给向量加上 ± 0.5 的随机偏置，实现随机地向上或向下取整。这是个非常简单的抖动算法，硬件中会有更复杂的实现。

在把数据送给硬件之前，应当理解有限位宽与数据相关性的影响。硬件只会让影响更糟，不会更好。

代码 2.10　SFDR 测试：sfdr_test.m

```
deltat = 1e-8;
fs = 1/deltat;
t = 0:deltat:1e-5-deltat;
fundamental = 3959297;
x = 10e-3*sin(2*pi*fundamental*t);
r = sfdr(x,fs)

sfdr(x,fs);
```

代码 2.11　SFDR 测试：sfdr_test.m

```
bits = 2^11;
x = round(10e-3*bits*sin(2*pi*fundamental*t))/bits;

sfdr(x,fs)
```

代码 2.12　SFDR 测试：sfdr_test.m

```
bits = 2^11;
x = round(bits*sin(2*pi*fundamental*t))/bits;
sfdr(x,fs)
```

代码 2.13　SFDR 测试：sfdr_test.m

```
t = 0:deltat:1e-4-deltat;
x = round(bits*sin(2*pi*fundamental*t))/bits;
r = sfdr(x,fs)

sfdr(x,fs);
```

代码 2.14　SFDR 测试：sfdr_test.m

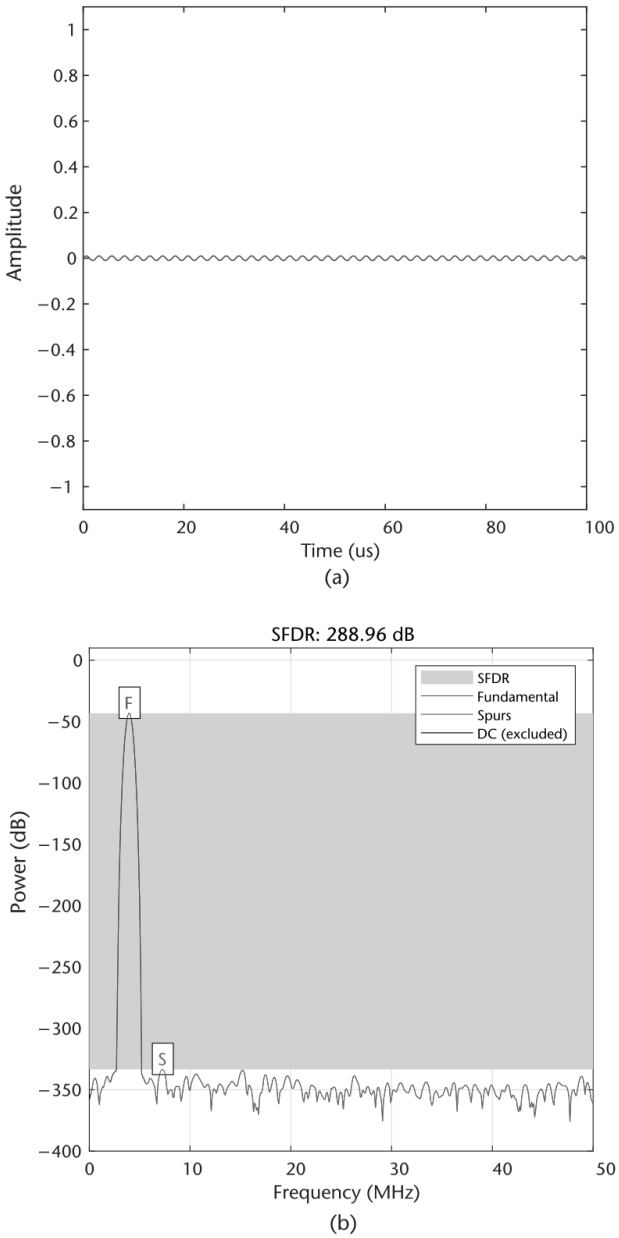


图 2.28　双精度浮点格式的 SFDR：(a) 时域浮点表示；(b) 双精度浮点的 SFDR，1k 点，FA = 3,959,297 Hz

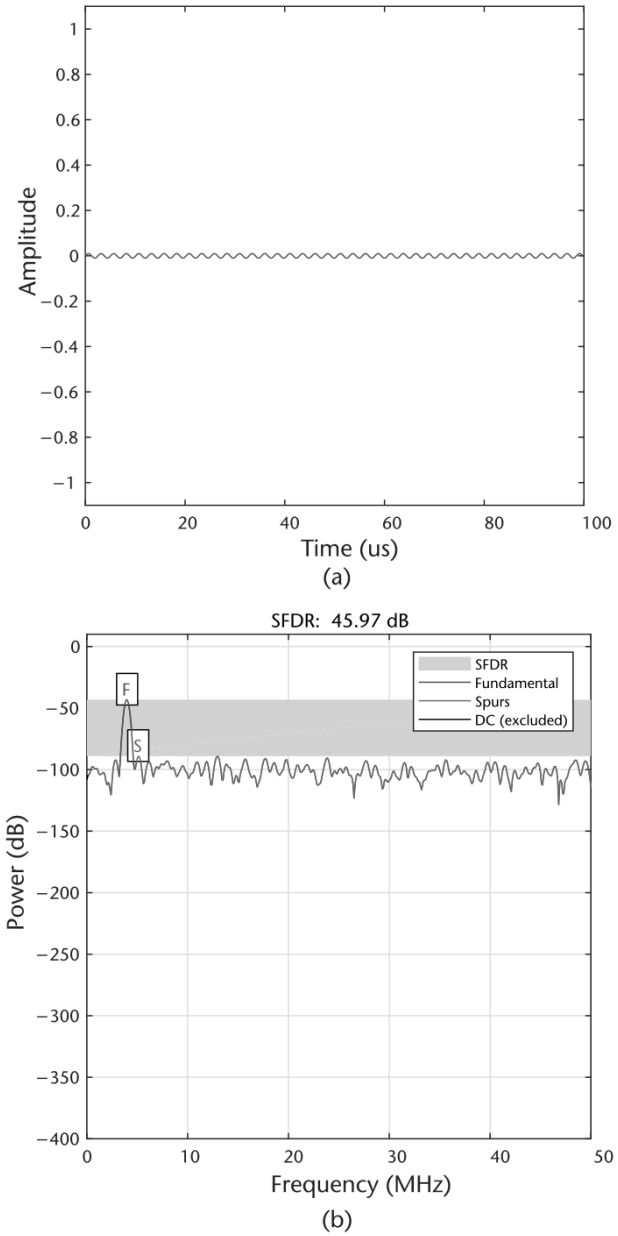


图 2.29　12 位缩放数的 SFDR：(a) 时域浮点表示；(b) 缩放的 12 位定点 SFDR，1k 点，FA = 3,959,297 Hz

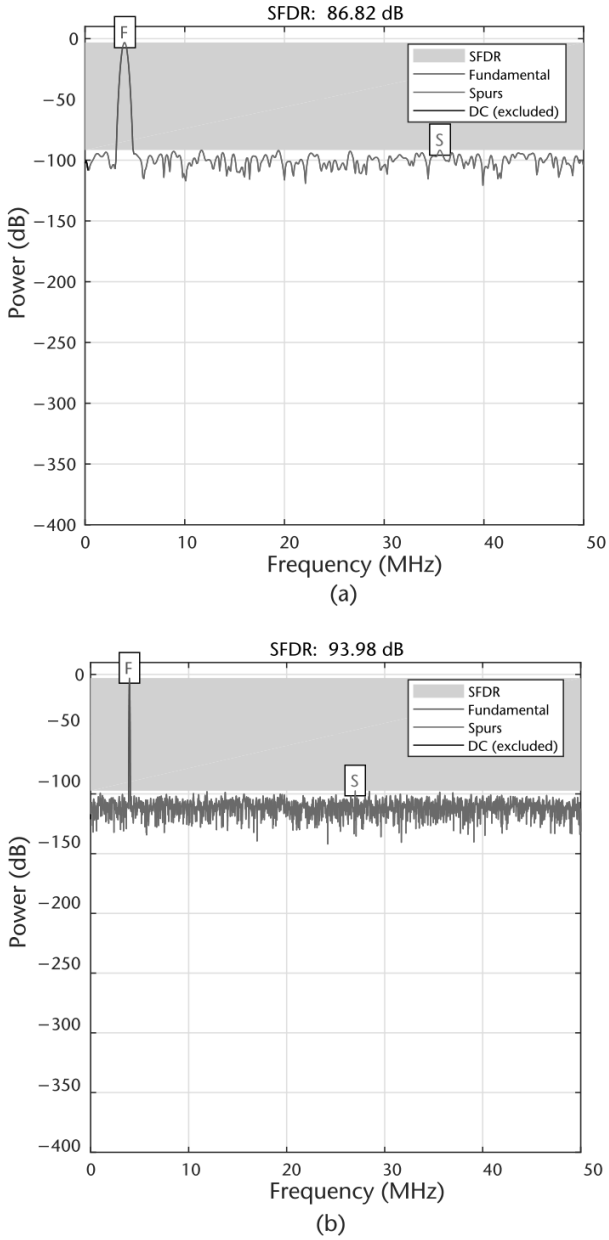


图 2.30　12 位缩放数的 SFDR：(a) 满量程 12 位定点 SFDR，1k 点，FA = 3,959,297 Hz；(b) 满量程 12 位定点 SFDR，10k 点，FA = 3,959,297 Hz

```
fundamental = 4000000;
x = round(bits*sin(2*pi*fundamental*t))/bits;
r = sfdr(x,fs)

sfdr(x,fs);
```

代码 2.15　SFDR 测试：sfdr_test.m

```
ran = rand(1,length(t)) - 0.5;
x = round(bits*sin(2*pi*fundamental*t) + ran)/bits;

sfdr(x,fs);
```

2.5.3 位数、成本、功耗等的设计折中

关于两条接收链（I/Q），最需要记住的是 ADC 本身量化的影响：一个 N 位字表示 2^N 种可能状态之一，因此一个 N 位 ADC（基准固定）只能有 2^N 种数字输出。数据转换器的分辨率有多种表达：最低有效位（LSB）的权重、满量程百万分比（ppm FS）、毫伏（mV）。不同器件——哪怕同厂——的规定方式也不同。因此转换器用户必须学会在不同指标之间换算，才能成功比较器件。

表 2.3 列出了 10 W（20 V 峰峰）输入下各种分辨率的最低有效位大小。

表 2.3　量化：最低有效位的大小（电压按 20 Vpp 计算）

分辨率 (N)	2^N	电压	ppm FS	%FS	dBFS
2 位	4	5.00 V	250,000	25	−12
4 位	16	1.25 V	62,500	6.25	−24
6 位	64	313 mV	15,625	1.56	−36
8 位	256	78.1 mV	3,906	.391	−48
10 位	1,024	19.5 mV	977	.097	−60
12 位	4,096	4.88 mV	244	.024	−72
14 位	16,384	1.22 mV	61.0	.0061	−84
16 位	65,536	305 µV	15.2	.0015	−96
18 位	262,144	76.2 µV	3.81	.00038	−108
20 位	1,048,576	19.0 µV	.953	.000095	−120
22 位	4,194,304	4.77 µV	.238	.000024	−132
24 位	16,777,216	1.19 µV	.0596	.0000060	−144
26 位	67,108,864	298 nV	.0149	.0000015	−156

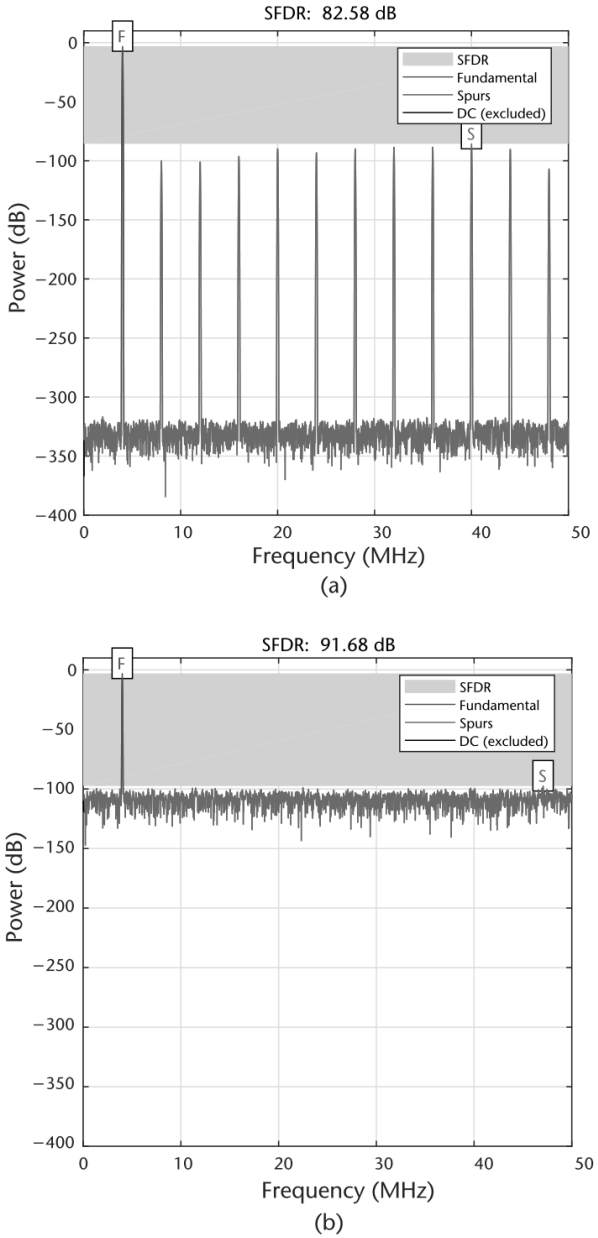


图 2.31　12 位缩放数的 SFDR：(a) 满量程 12 位定点，10k 点，无抖动，FA = 4,000,000 Hz；(b) 满量程 12 位定点，10k 点，有抖动，FA = 4,000,000 Hz

> 注：600 nV 是 2.2 kΩ 电阻在 25°C、10 kHz 带宽内的约翰逊（热）噪声。

性能 −144 dB 的 24 位转换器听起来不错，但从功耗与速度角度看并不现实。虽然 24 位 ADC 存在，但它们不是宽带的——例如 AD7177 是最先进的 32 位转换器，输出数据率限制在 5 SPS 到 10 kSPS，不适合 SDR，但非常适合温度压力测量、色谱分析或电子秤 [12]。另一方面，高速 ADC 也存在：AD9208 是双通道 14 位 3 GSPS ADC，SFDR 达 70 dBFS、9 GHz 模拟输入满功率带宽（第六奈奎斯特区）。但 AD9208 功耗超过 3 W，超过了 Pluto SDR 通过 USB 流式传输时的整机功耗 [13]。尽管如此，基于 AD9208 的系统可提供高达 12 Gbytes/秒的数据，这超出了软件可处理的范围，需要 FPGA 内的定制信号处理硬件。AD9208 这类器件用于 60 GHz 频段——单信道带宽超过 2 GHz。这些高速宽带链路使用与本书剩余部分相同的概念与技术，只是数据率更高、更耗电、也更贵。不过，学习数字通信的基础技术，用 Pluto SDR 这样 20 MHz（很多情况下远小于此带宽）的高性价比器件就够了。

就 ADC 而言，要让 12 位器件用于 Pluto SDR 这类电台应用，AD9361 的信号链包含了可编程模拟增益，如图 2.32 所示。这使 ADC 的输入尽可能被驱动到满量程——如 2.4.1 节所学，这样能提供最佳性能。

简要回顾运放理论：放大器有两种增益——信号增益与噪声增益。我们想增大信号、同时把噪声压得尽量低。做法是在用 ADC 数字化之前，先在模拟域放大信号，如图 2.32 所示。

2.5.4 Sigma-Delta 模数转换器

Sigma-delta（Σ-Δ）ADC 问世已超过 50 年，但直到最近，高密度数字 VLSI 技术才使它得以制造成廉价的单片集成电路。如今它用于许多需要低成本、中等带宽、低功耗、高分辨率 ADC 的场合。关于 Σ-Δ ADC 的架构与理论已有无数论述，但大多从深奥的数学推导开始。这不是一本 ADC 教科书，我们尽量避开数学展开，基于本章前文已覆盖的主题来理解它。

Σ-Δ ADC 没有什么特别难懂的。它包含非常简单的模拟电路（一个比较器、基准电压、一个开关、一个或多个积分器与模拟求和电路），以及数字运算电路。该电路由一个滤波器构成——通常（但不总是）是低通滤波器。不必精确知道滤波器如何工作，也能理解它做了什么。要理解 Σ-Δ ADC 的工作，需要熟悉**过采样、量化噪声整形、数字滤波与抽取**这几个概念——全部是本章前面讲过的主题。

从频域分析过采样技术。直流转换的量化误差最多 $\frac{1}{2}$ LSB，而采样数据系统则有量

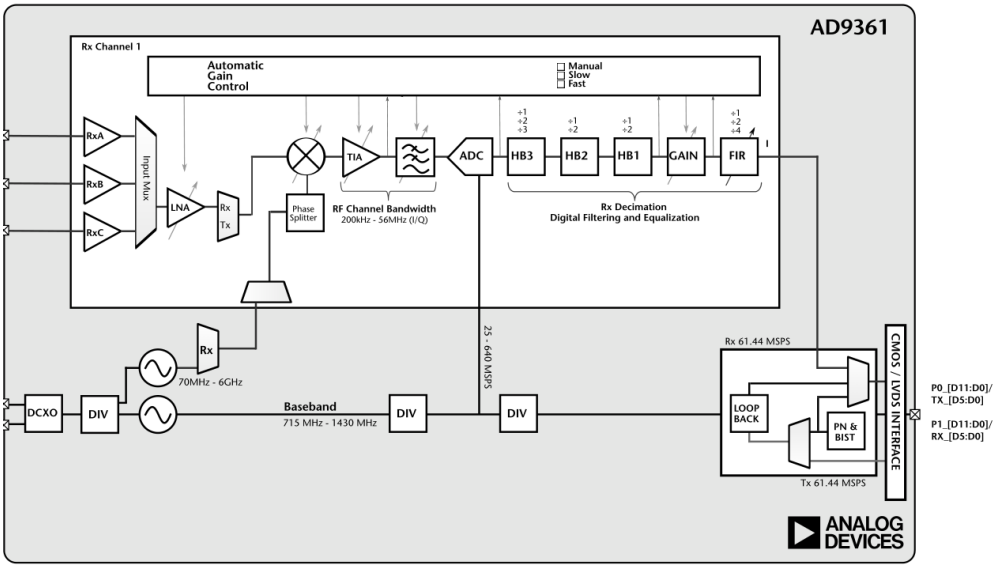


图 2.32　AD9361 简化接收链框图

化噪声。理想的经典 N 位采样 ADC 的 RMS 量化噪声为 $\frac{q}{\sqrt{12}}$，均匀分布在 DC 到 $\frac{f_s}{2}$ 的奈奎斯特带内（q 为 LSB 值），如图 2.33(a) 所示。因此满量程正弦输入下其 SNR 为 $(6.02N + 1.76)$ dB。若 ADC 不够完美、噪声大于理论最小量化噪声，其实际分辨率将低于 N 位。实际分辨率常称为**有效位数**（ENOB），定义为

$$\mathrm{ENOB} = \frac{\mathrm{SNR} - 1.76\,\mathrm{dB}}{6.02\,\mathrm{dB}}.$$

实践中，ENOB 由测量信纳比（SINAD，即 S/(N+D)——信号 RMS 幅度对其余全部频谱分量含谐波、不含 DC 的均方根平均之比）并对非满量程输入信号做修正得到 [6]。把 (2.39) 改写为计入满量程幅度 A_{FS} 与实际输入幅度 A_{IN} 的形式：

$$\mathrm{ENOB} = \frac{\mathrm{SINAD} - 1.76\,\mathrm{dB} + 20\log_{10}\frac{A_{FS}}{A_{IN}}}{6.02\,\mathrm{dB}}.$$

如果选择高得多的采样率 Kf_s（见图 2.33(b)），RMS 量化噪声仍是 $\frac{q}{\sqrt{12}}$，但噪声分布在 DC 到 $\frac{Kf_s}{2}$ 的更宽带宽上。此时对输出施加数字低通滤波（LPF），就能去除大部分量化噪声而不影响有用信号，从而改善 ENOB。于是我们用低分辨率 ADC 实现了高分辨率 A/D 转换。因子 K 通常称为**过采样比**。还要指出，过采样有个额外好处：放宽了对模拟抗混叠滤波器的要求。

由于带宽被数字输出滤波器收窄，输出数据率可以低于原始采样率（Kf_s）且仍满

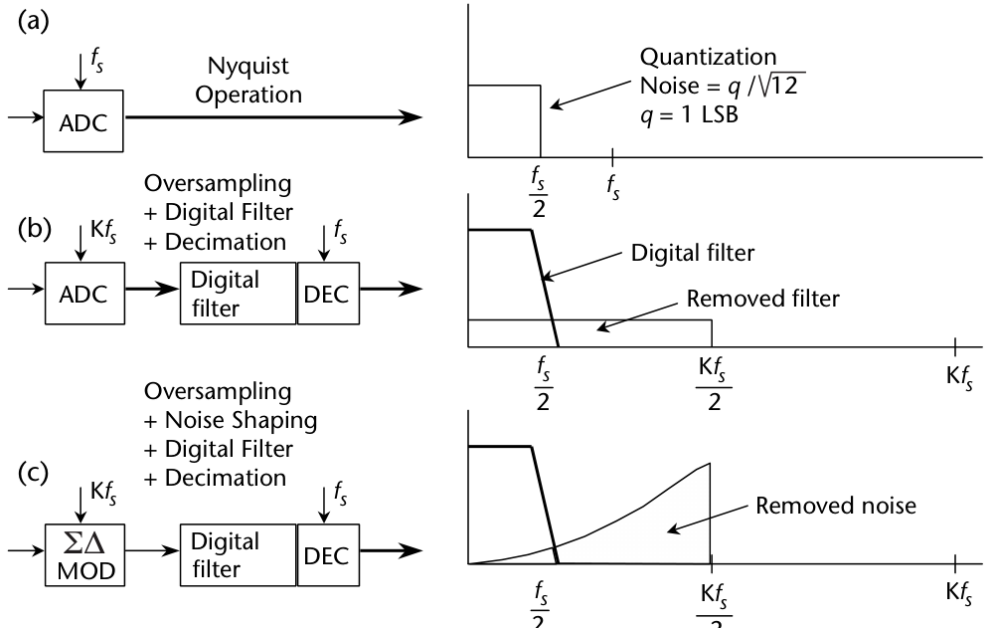


图 2.33　Σ-Δ ADC 中的过采样、数字滤波、噪声整形与抽取

足奈奎斯特准则。做法是把每第 M 个结果送到输出、丢弃其余，即**按因子 M 抽取**。只要不违反奈奎斯特准则，抽取不损失任何信息（见图 2.33(b)）。对给定输入频率，更高阶的模拟滤波器提供更大衰减。Σ-Δ 调制器也一样——前提是采取适当预防措施。在 Σ-Δ 调制器中使用多于一级积分与求和级，可以在给定过采样比下实现更高阶的量化噪声整形与更好的 ENOB，如图 2.34 所示。

Pluto SDR 所用 AD9363 中实际的 Σ-Δ ADC 是四阶的，如图 2.35 所示（Analog Devices 收发信机支持 Simulink 模型中的描述）。可见，现实总比理论或一阶近似复杂一点。

2.6 SDR 数字信号处理技术

DSP 是一个始终站在数学复杂性、计算性能与日益增长的移动性交汇点上的领域，影响着通信、医学成像、雷达、娱乐乃至科学探索。所有这些领域都依赖” 把模拟信息转换为数字表示、再以某种机制处理数据” 的范式。为此，工程师与科学家依赖通用工具与语言，包括 C 和 Verilog——它们都能高效、过程化地操纵数字信息。无论语言为何，许多重要的 DSP 软件问题是硬件相关的，例如截断误差、位模式、处理器的计算速度与效率 [14]。眼下我们基本忽略这些问题，专注于信号处理的算法问题，讨论常用算法。

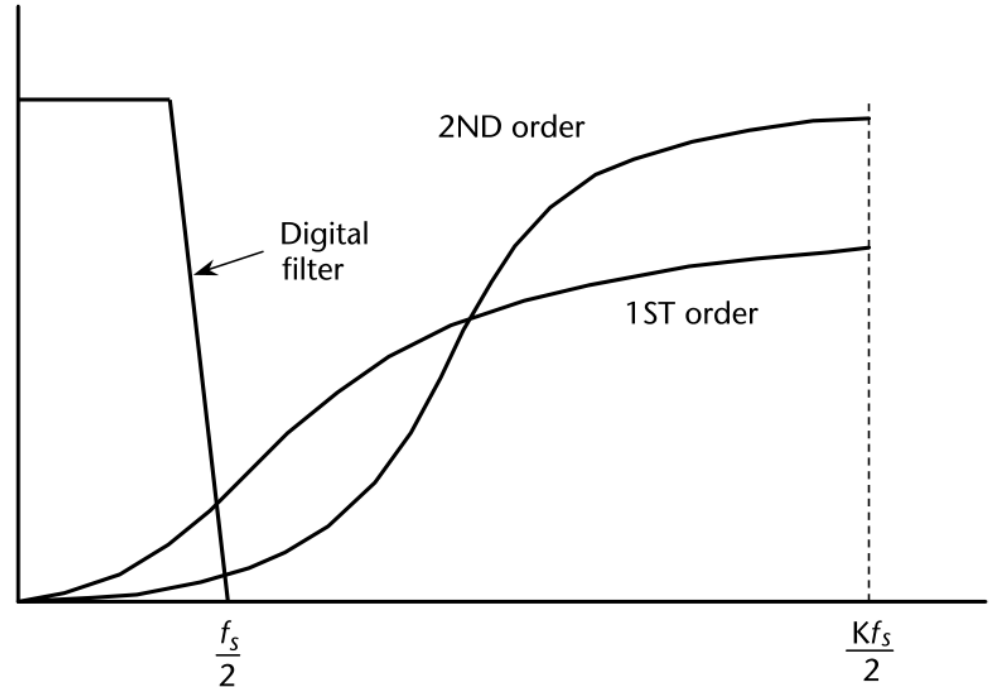


图 2.34　Σ-Δ 调制器对量化噪声整形

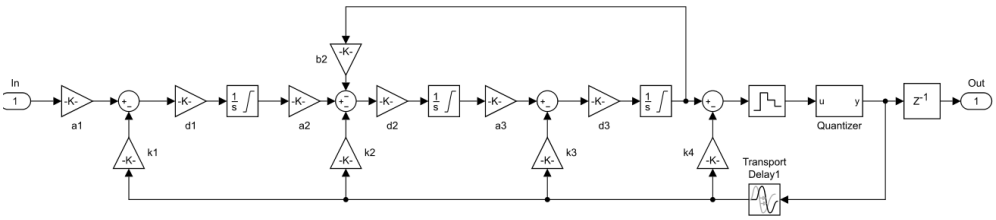


图 2.35　AD9361 Σ-Δ ADC 的 Simulink 模型

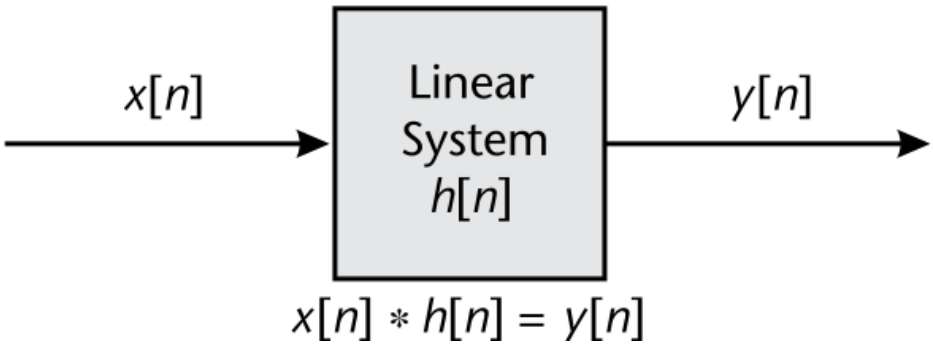


图 2.36　卷积在 DSP 中的应用：线性系统的输出信号等于输入信号与系统冲激响应的卷积

2.6.1 离散卷积

卷积是把两个信号结合形成第三个信号的数学工具，是全部 DSP 的基石。利用冲激分解策略，系统由称为**冲激响应**的信号描述。卷积之所以重要，是因为它把三个关心的信号联系在一起：输入信号、输出信号与冲激响应。

图 2.36 给出卷积在线性系统中的应用记号。离散采样输入信号 $x[n]$ 进入冲激响应为 $h[n]$ 的线性系统，产生输出信号 $y[n]$。用文字表达：输入信号与冲激响应卷积等于输出信号。如 (2.41) 所记，卷积用 $*$ 算符表示。遗憾的是多数编程语言（如 MATLAB）用星号表示乘法、用 conv 这类专用函数表示卷积。程序里的星号是乘法，这里的星号是卷积。

从根本上说，卷积的数学就是若干乘法与加法。若 $x[n]$ 是从样本 0 到 $N-1$ 的 N 点信号，$h[n]$ 是从 0 到 $M-1$ 的 M 点信号，则两者的卷积 $y[n]=x[n]*h[n]$ 是从 0 到 $N+M-2$ 的 $N+M-1$ 点信号：

$$y[i]=\sum_{j=0}^{M-1} h[j]\times x[i-j]=h[i]*x[i].$$

这个方程称为**卷积和**。它允许输出信号的每个点独立于其他所有点计算。下标 i 决定正在计算输出信号的哪个采样。不要被 $y[n]=x[n]*h[n]$ 中的 n 搞混——它只是数组下标的占位符。两个向量卷积和的 MATLAB 实现见代码 2.16。

在信号处理中，卷积可以从两个角度理解。第一从输入信号的观点：分析输入信号的每个采样如何贡献到输出信号的许多点。第二从输出信号的观点：考察输出信号的每个采样如何从输入信号的许多点获取信息。记住，这两个视角是同一数学运算的不同思考方式。第一个视角重要，因为它提供卷积与信号处理关系的概念性理解；第二个

视角描述卷积的数学。这体现了信号处理领域最困难的任务之一：把概念性理解与表达思想的一堆数学对上号。

图 2.37 展示了卷积用于低通与高通滤波（2.6.4 节详述）。示例输入信号是两个分量之和：三个周期的正弦波（代表高频）加一个缓升的斜坡（由低频构成）。在 (a) 中，低通滤波器的冲激响应是平滑的拱形，结果只有缓变的斜坡波形通过到输出；类似地，高通滤波器 (b) 只让变化更快的正弦通过。

代码 2.16　卷积：my_convolution.m

```
% Receive two vectors and return a vector resultant of
% convolution operation
function conv = simple_conv(f, g)
   % Transform the vectors f and g in new vectors with the same length
   F = [f,zeros(1,length(g))];
   G = [g,zeros(1,length(f))];

   % FOR Loop to put the result of convolution between F and G vectors
   % in a new vector C. According to the convolution operation
   % characteristics, the length of a resultant vector of convolution
   % operation between two vector is the sum of vectors length minus 1
   for i=1:length(g)+length(f)-1
       % Create a new vector C
       C(i) = 0;
       % FOR Loop to walk through the vector F and G
       for j=1:length(f)
          if(i-j+1>0)
             C(i) = C(i) + F(j) * G(i-j+1);
          end
       end
   end
   out = C;
end
```

```
 4 % Receive two vectors and return a vector resultant of
 5 % convolution operation
 6 function conv = simple_conv(f, g)
 7     % Transform the vectors f and g in new vectors with the same length
 8     F = [f,zeros(1,length(g))];
 9     G = [g,zeros(1,length(f))];
10
11     % FOR Loop to put the result of convolution between F and G vectors
12     % in a new vector C. According to the convolution operation
13     % characteristics, the length of a resultant vector of convolution
14     % operation between two vector is the sum of vectors length minus 1
15     for i=1:length(g)+length(f)-1
16         % Create a new vector C
17         C(i) = 0;
18         % FOR Loop to walk through the vector F ang G
19         for j=1:length(f)
20            if(i-j+1>0)
21                C(i) = C(i) + F(j) * G(i-j+1);
22            end
23         end
24     end
25     out = C;
26 end
```

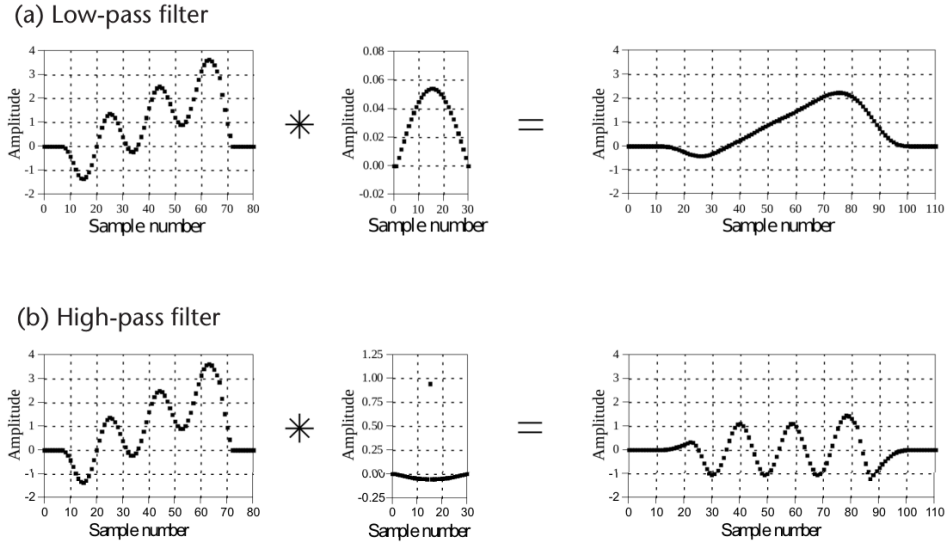


图 2.37　用卷积实现 (a) 低通与 (b) 高通滤波的示例。输入信号是几周期正弦波加缓升斜坡，通过恰当选择的冲激响应把这两个分量分开

2.6.2 相关

互相关与自相关是 SDR 中两个重要概念。**互相关**（cross-correlation）度量两个序列当其中一个相对另一个位移时的相似程度。

相关的概念最好用例子讲。图 2.38 展示了雷达系统的关键组成。特制天线在选定方向发射一短脉冲射频能量。若传播的波碰到物体（如本图中的直升机），一小部分能量被反射回位于发射机附近的接收机。发射脉冲是我们选定的特定形状（如本例的三角形）。接收信号由两部分组成：(1) 发射脉冲的移位加缩放版本；(2) 随机噪声——来自干扰无线电波、电子器件热噪声等。由于无线电波以已知速度（光速）传播，给定形状已知的信号，发射与接收脉冲之间的移位就是对物体距离的直接量度。挑战在于确定一条信号出现在另一条信号中的最佳方式。**相关**就是这一问题的解。

相关是与卷积非常相似的数学运算。和卷积一样，相关用两个信号产生第三个信号。这第三个信号称为两输入信号的互相关。若信号与自身相关，结果称为**自相关**（autocorrelation）。互相关信号每个采样的幅度，度量接收信号在该位置与目标信号的相似程度。这意味着：接收信号中每出现一个目标信号，互相关信号中就会出现一个峰。换言之，当目标信号与接收信号中的相同特征对齐时，互相关值最大。

在随机噪声中检测已知波形，相关是最优技术之一：用相关得到的峰高出噪声的幅度，比任何其他线性系统所能产生的都高。（严格说，只对随机白噪声最优。）用相关检测已知波形常称为**匹配滤波**。详见 4.7.1 节。

对离散函数 f 与 g，互相关定义为

$$(f \star g)[n] = \sum_{j=-\infty}^{\infty} f^*[m] \times g[m+n],$$

其中 h^* 表示 h 的复共轭。函数 $f(t)$ 与 $g(t)$ 的互相关等价于 $f^*(-t)$ 与 $g(t)$ 的卷积，即

$$(f \star g)[n] = f^*(-t) * g(t).$$

不要被卷积与相关的数学相似性糊弄：它们代表截然不同的信号处理概念。卷积是系统输入信号、输出信号与冲激响应之间的关系；相关是在噪声背景中检测已知波形的方法。相似的数学只是便于算法优化的巧合。

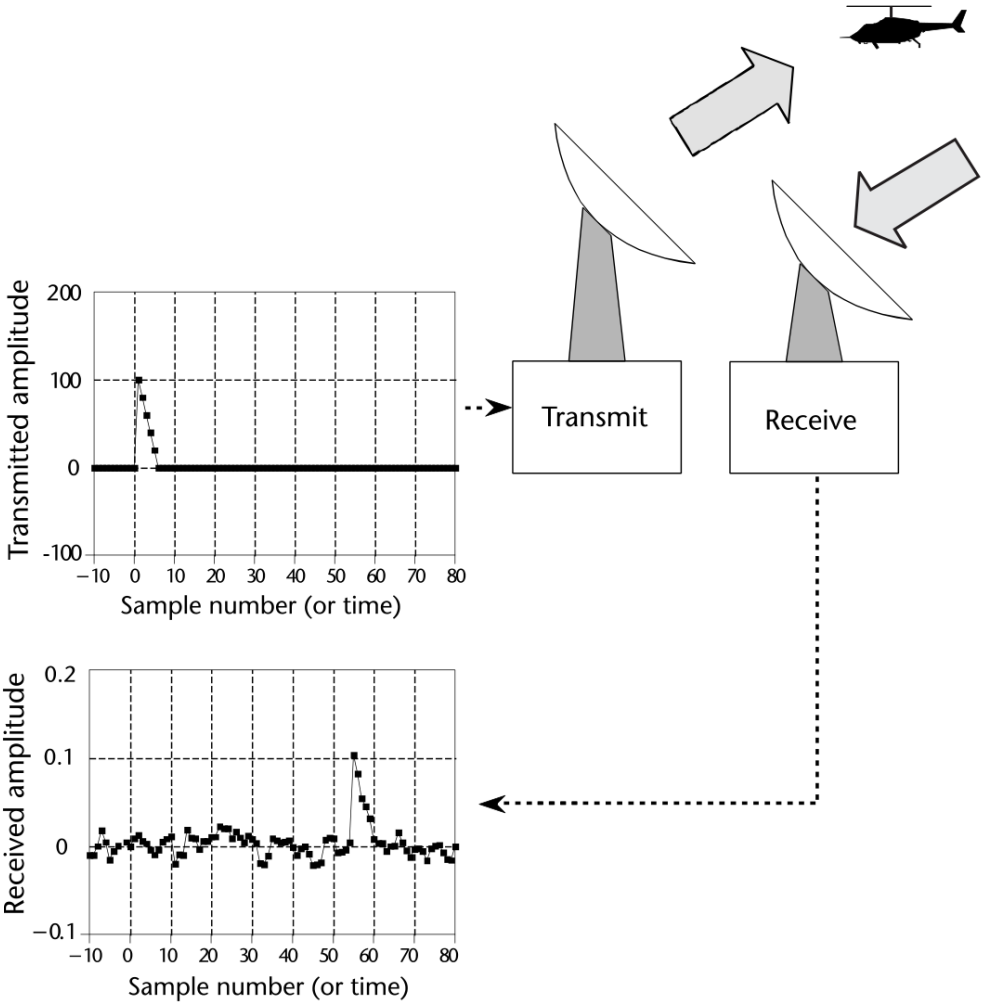


图 2.38　雷达系统的关键组成。与其他回波定位系统一样，雷达发射短脉冲能量，被考察物体反射，使接收波形成为发射波形的移位版加随机噪声。在含噪信号中检测已知波形是回波定位的基本问题，其答案就是相关

2.6.3 Z 变换

2.1.1 节介绍了处理连续时间信号频域的傅里叶变换。本节聚焦数字滤波器设计、涉及离散时间信号，需要引入新的变换——**z 变换**。

离散时间信号 $x[n]$ 的 z 变换定义为幂级数：

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n},$$

其中 z 是复变量 [1]。

z 变换用于分析离散时间系统。它的连续时间对应物是拉普拉斯变换，定义为

$$X(s) = \int_{-\infty}^{\infty} x(t) e^{-st} dt,$$

其中 t 是时域时间变量（秒），$s = \sigma + j\omega$ 是复变量。沿 $j\omega$ 轴（即 $\sigma = 0$）求值时，拉普拉斯变换退化为 (2.2) 定义的傅里叶变换。因此拉普拉斯变换把傅里叶变换从实轴（频率轴 $j\omega$）推广到了整个复平面。

根据 2.2.2 节，连续时间信号 $x(t)$ 被均匀采样后，采样信号 $x_s(t)$ 可表示为

$$x_s(t) = \sum_{n=-\infty}^{\infty} x(nT)\delta(t - nT),$$

其中 T 是采样间隔。对两边取拉普拉斯变换：

$$X_s(s) = \int_{-\infty}^{\infty} x_s(t) e^{-st} dt = \int_{-\infty}^{\infty} \sum_{n=-\infty}^{\infty} x(nT)\delta(t - nT)\, e^{-st} dt.$$

由于积分与求和都是线性算子，可以交换次序。再利用 δ 函数的采样性质：

$$X_s(s) = \sum_{n=-\infty}^{\infty} x(nT) \int_{-\infty}^{\infty} \delta(t - nT) e^{-st} dt = \sum_{n=-\infty}^{\infty} x(nT) e^{-snT}.$$

令 $z = e^{sT}$（即 $s = \frac{1}{T} \ln z$），则 (2.48) 变为

$$X(z) = \sum_{n=-\infty}^{\infty} x(nT) z^{-n}.$$

由于 T 是采样间隔，$x(nT) = x[n]$，上式进一步写成

$$X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n},$$

这正是 (2.44) 的 z 变换定义。因此 z 变换与拉普拉斯变换通过

$$z = e^{sT}, \qquad s = \frac{1}{T} \ln(z)$$

相联系。

根据 (2.44)，z 变换是 z^{-1} 的级数。实际上，级数不收敛时 z 变换没有意义。给定幅度有限的序列 $x[n]$，使其 z 变换收敛的全部 z 值的集合称为**收敛域**（ROC）。由级数理论，这些 z 值须满足

$$\sum_{n=-\infty}^{\infty} |x[n] z^{-n}| < \infty.$$

常用的 z 变换对及其收敛域列于表 2.4。

表 2.4　z 变换表：选列（据 [15]）

$x[n]$	$X(z)$	收敛域
$\delta[n]$	1	全部 z
$a^n u[n]$	$\frac{z}{z-a}$	$\|z\| > \|a\|$
$na^n u[n]$	$\frac{az}{(z-a)^2}$	$\|z\| > \|a\| > 0$
$n^2 a^n u[n]$	$\frac{az(z+a)}{(z-a)^3}$	$\|z\| > a > 0$
$\frac{1}{3}(1 + a^n + b^n)u[n]$	$\frac{1}{3}\left(\frac{z}{z-1} + \frac{z}{z-a} + \frac{z}{z-b}\right)$	$\|z\| > \max(\|a\|, \|b\|, 1)$
$a^n u[n] \sin(\omega_0 n)$	$\frac{az \sin \omega_0}{z^2 - 2az \cos \omega_0 + a^2}$	$\|z\| > a > 0$
$a^n u[n] \cos(\omega_0 n)$	$\frac{z^2 - az \cos \omega_0}{z^2 - 2az \cos \omega_0 + a^2}$	$\|z\| > a > 0$
$e^a u[n]$（$e^{-an}u[n]$ 见下两行）	$\frac{z}{z - e^{-a}}$	$\|z\| > e^{-a}$
$e^{-an} u[n] \sin(\omega_0 n)$	$\frac{ze^{-a} \sin \omega_0}{z^2 - 2ze^{-a} \cos \omega_0 + e^{-2a}}$	$\|z\| > e^{-a}$
$e^{-an} u[n] \cos(\omega_0 n)$	$\frac{z^2 - ze^{-a} \cos \omega_0}{z^2 - 2ze^{-a} \cos \omega_0 + e^{-2a}}$	$\|z\| > e^{-a}$

讨论线性时不变系统时，其系统冲激响应的 z 变换可以表示为两个多项式之比：

$$H(z) = \frac{b_m z^m + b_{m-1} z^{m-1} + \cdots + b_1 z + b_0}{a_n z^n + a_{n-1} z^{n-1} + \cdots + a_1 z + a_0} = \frac{B(z)}{A(z)},$$

其中 $A(z) = 0$ 的根称为系统的**极点**，$B(z) = 0$ 的根称为系统的**零点**。系统可以有多个极点与零点。

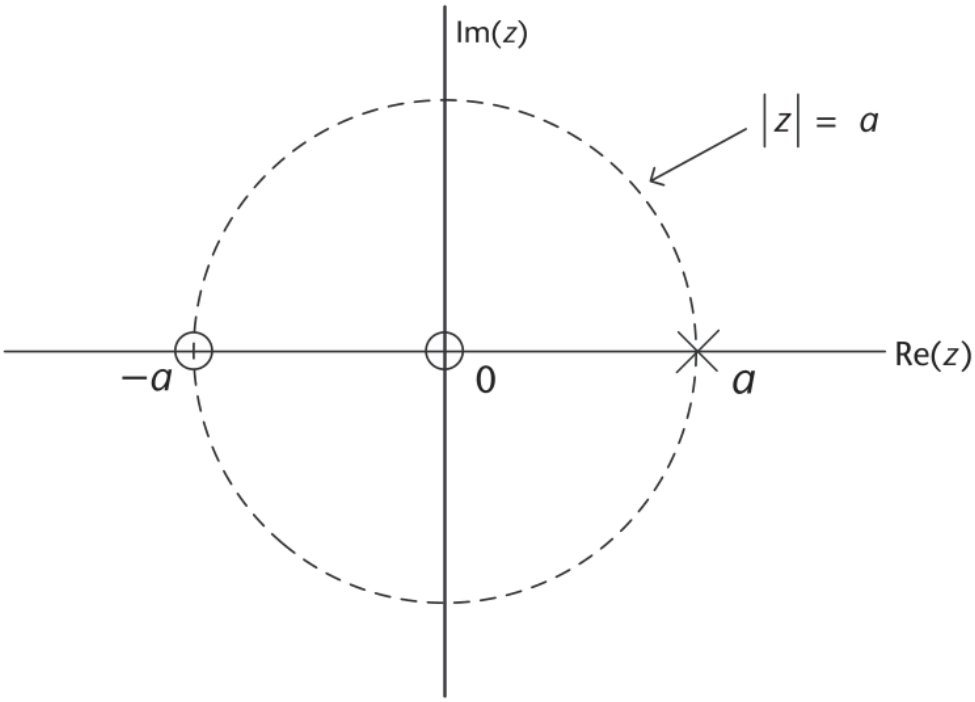


图 2.39　式 (2.57) 定义系统的零极点图。叉号表示极点、圆圈表示零点，收敛域是 |z| = |a| 圆外区域

把分子 $B(z)$ 与分母 $A(z)$ 因式分解，(2.54) 可写为

$$H(z) = C\,\frac{(z-z_1)(z-z_2)\cdots(z-z_m)}{(z-p_1)(z-p_2)\cdots(z-z_n)},$$

其中 C 是常数，$\{p_k\}$ 是全部极点，$\{z_k\}$ 是全部零点。这有助于画出 $H(z)$ 的零极点图。

设某线性时不变系统的冲激响应为

$$h[n] = n^2 a^n u[n].$$

查表 2.4，其 z 变换为

$$H(z) = \frac{az(z+a)}{(z-a)^3}.$$

把 (2.57) 与 (2.55) 对照，容易得到该系统有两个零点 $z_1 = 0$、$z_2 = -a$，三个极点 $p_1 = p_2 = p_3 = a$。其零极点图示于图 2.39。

z 变换还有若干对研究 z 域信号与系统有用的性质。由于它们与 2.1.1 节的傅里叶变换性质非常相似，直接列于表 2.5，不再展开。

表 2.5　z 变换性质（据 [15]；设时间信号为 $x[n]$，其 z 变换为 $X(z)$）

性质	时间信号	z 变换信号
线性	$\sum_{m=1}^{N} a_m x_m[n]$	$\sum_{m=1}^{N} a_m X_m(z)$
对称性	$x[-n]$	$X(z^{-1})$
移位	$x[n-m]$	$z^{-m}X(z)$
缩放	$a^n x[n]$	$X(\frac{z}{a})$
微分	$nx[n]$	$-z\frac{dX(z)}{dz}$
累加	$\sum_{m=-\infty}^{n} x[m]$	$\frac{z}{z-1}X(z)$
时域卷积	$x[n] * h[n]$	$X(z)H(z)$
频域卷积	$x[n]h[n]$	$\frac{1}{2\pi j}\oint X(v)H(\frac{z}{v})\frac{dv}{v}$

2.6.4 数字滤波

做信号处理时，我们通常需要去除噪声、提取有用信号。这个过程称为**滤波**，所用器件称为**滤波器**——它根据输入对象某种属性甄别放行什么。典型的滤波器是频率选择电路。若噪声与有用信号频率分布不同、同时出现在滤波器输入端，则施加该电路后噪声被衰减甚至消除，而有用的信号被保留。

滤波器可以从不同角度分类。例如按频率响应可分为低通、高通、带通与带阻；按处理的信号可分为模拟滤波器与数字滤波器 [1]。具体地说，模拟滤波器处理连续时间信号，数字滤波器处理离散时间信号。本节聚焦数字滤波器。这些滤波器的理想幅度响应特性示于图 2.40。由图 2.40，理想滤波器的幅度响应可以概括为：通带内幅度为常数，阻带内幅度降为零。但现实中这种理想滤波器无法实现，实际滤波器其实是理想滤波器的最佳逼近。

要进行数字滤波，数字系统的输入输出必须都是离散时间序列。若输入序列为 $x[n]$，滤波器冲激响应为 $h[n]$，则输出序列 $y[n]$ 为

$$y[n] = x[n] * h[n].$$

根据表 2.5 的时域卷积性质，频域上 (2.58) 等价于

$$Y(z) = X(z)H(z),$$

其中 $X(z)$、$Y(z)$ 是输入输出序列的 z 变换，$H(z)$ 是 $h[n]$ 的 z 变换。

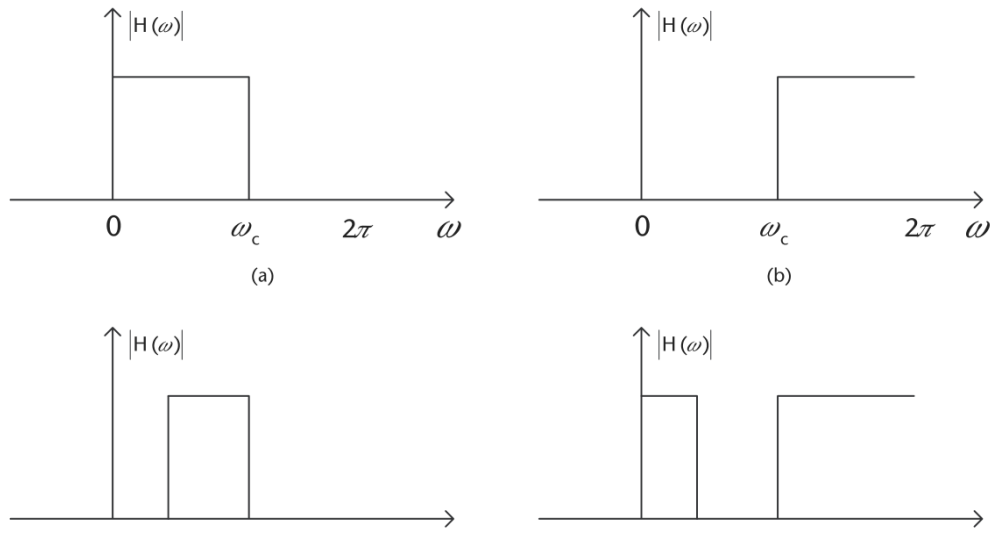


图 2.40　频率范围 [0, 2π] 上四类滤波器的理想幅度响应特性：(a) 低通；(b) 高通；(c) 带通（ωc1, ωc2 为通带）；(d) 带阻（ωc1, ωc2 为阻带）

由于理想砖墙滤波器实际做不出来，我们把注意力限制在由差分方程描述的线性时不变系统类 [1]:

$$y[n] = -\sum_{k=1}^{N} a_k y[n-k] + \sum_{k=0}^{M} b_k x[n-k],$$

其中 $y[n]$ 是当前滤波输出，$y[n-k]$ 是以前的滤波输出，$x[n-k]$ 是当前或以前的滤波输入。该系统的频率响应为

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}},$$

其中 $\{a_k\}$ 是滤波器对应极点的反馈系数，$\{b_k\}$ 是对应零点的前馈系数，N 是滤波器阶数。

数字滤波器设计的基本问题是：通过恰当选择系数 $\{a_k\}$ 与 $\{b_k\}$，使频率响应为 (2.61) 的系统逼近任一种理想频率响应特性 [1]。

数字滤波器有两大基本类型：有限冲激响应（FIR）与无限冲激响应（IIR）滤波器。当以单位样本 $\delta[n]$ 激励时，系统的冲激响应 $h[n]$ 可能只持续有限时长（图 2.41(a)），也可能永远持续（图 2.41(b)）。前者即有限冲激响应系统，后者即无限冲激响应系统。

长度为 M 的 FIR 滤波器，输入 $x[n]$、输出 $y[n]$，可用差分方程描述 [1]:

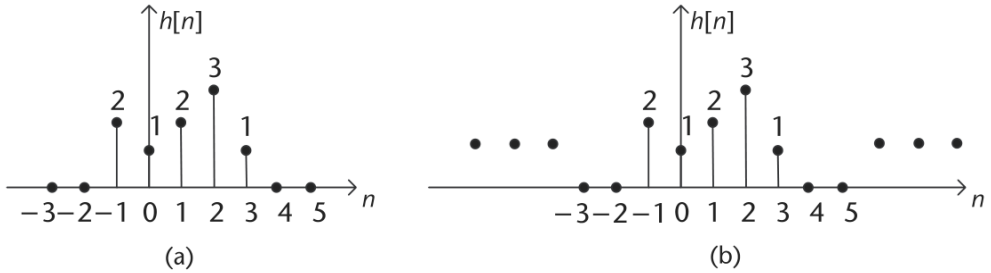


图 2.41　FIR 与 IIR 滤波器的冲激响应：(a) FIR 滤波器的冲激响应；(b) IIR 滤波器的冲激响应

$$y[n] = b_0x[n] + b_1x[n-1] + \cdots + b_{M-1}x[n-M+1] = \sum_{k=0}^{M-1} b_kx[n-k],$$

该滤波器没有反馈系数 $\{a_k\}$，因此 $H(z)$ 只有零点。

IIR 滤波器已在 (2.60) 中定义，它有一个或多个非零反馈系数 $\{a_k\}$。因此一旦被冲激激励，输出将永远持续。

2.6.4.1　案例研究：级联积分梳状滤波器

级联积分梳状滤波器（CIC 滤波器）在 SDR 硬件中扮演重要角色。它由 Eugene B. Hogenauer 发明，是用于多速率信号处理的一类 FIR 滤波器。CIC 滤波器应用于插值与抽取。但与多数 FIR 滤波器不同，它的架构内置了抽取器或插值器 [16]。

CIC 滤波器由一级或多级积分器-梳状滤波器对组成。对抽取型 CIC，输入信号先经过一级或多级级联积分器，然后是下采样器，再接数目与积分器相同的梳状节。插值型 CIC 就是该架构的镜像，只是把下采样器换成上采样器，如图 2.42 所示。

为说明梳状滤波器的简单形式，考虑由差分方程描述的滑动平均 FIR 滤波器：

$$y[n] = \frac{1}{M+1}\sum_{k=0}^{M} x[n-k].$$

该 FIR 滤波器的系统函数为

$$H(z) = \frac{1}{M+1}\sum_{k=0}^{M} z^{-k} = \frac{1}{M+1}\cdot\frac{[1-z^{-(M+1)}]}{(1-z^{-1})}.$$

假设把 z 换成 z^L（L 为正整数），得到的梳状滤波器系统函数为

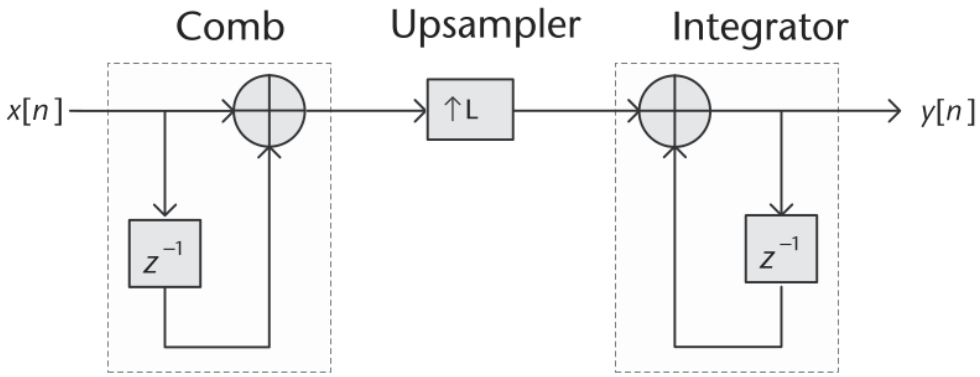


图 2.42　插值级联积分梳状滤波器的结构 [17]，输入 x[n]、输出 y[n]。由一对梳状与积分滤波器及插值比为 L 的上采样器组成

$$H_L(z) = \frac{1}{M+1} \cdot \frac{1 - z^{-L(M+1)}}{1 - z^{-L}},$$

其中 L 是抽取或插值比，M 是每级采样数（通常为 1 或 2）。

该滤波器的零点在单位圆上：

$$z_k = e^{j2\pi k/L(M+1)},$$

取除 $k = 0, L, 2L, \ldots, ML$ 之外的所有整数 k，如图 2.43 所示。

常用形式的 CIC 滤波器由若干级组成，复合 CIC 滤波器的系统函数为

$$H(z) = H_L(z)^N = \left[\frac{1}{M+1} \cdot \frac{1 - z^{-L(M+1)}}{1 - z^{-L}} \right]^N,$$

其中 N 是复合滤波器的级数。

CIC 滤波器的特点包括线性相位响应，且只用到延迟与加减法。换言之，它不需要乘法运算，因此非常适合硬件实现。

2.6.4.2　案例研究：FIR 半带滤波器

FIR 半带滤波器同样广泛用于插值/抽取的多速率信号处理。半带滤波器以多相形式高效实现，因为其系数约有一半为零。半带滤波器有两个重要特征：通带与阻带波纹必须相等；通带边缘与阻带边缘频率与半带频率 $\frac{f_s}{4}$ 等距。

例如 Pluto SDR 的收发链中就有多个可编程半带滤波器。RX HB3/DEC3 提供两种固定系数抽取滤波器可选，抽取因子为 2、3 或 1（旁路）。滤波器输入（ADC 输出）是 24

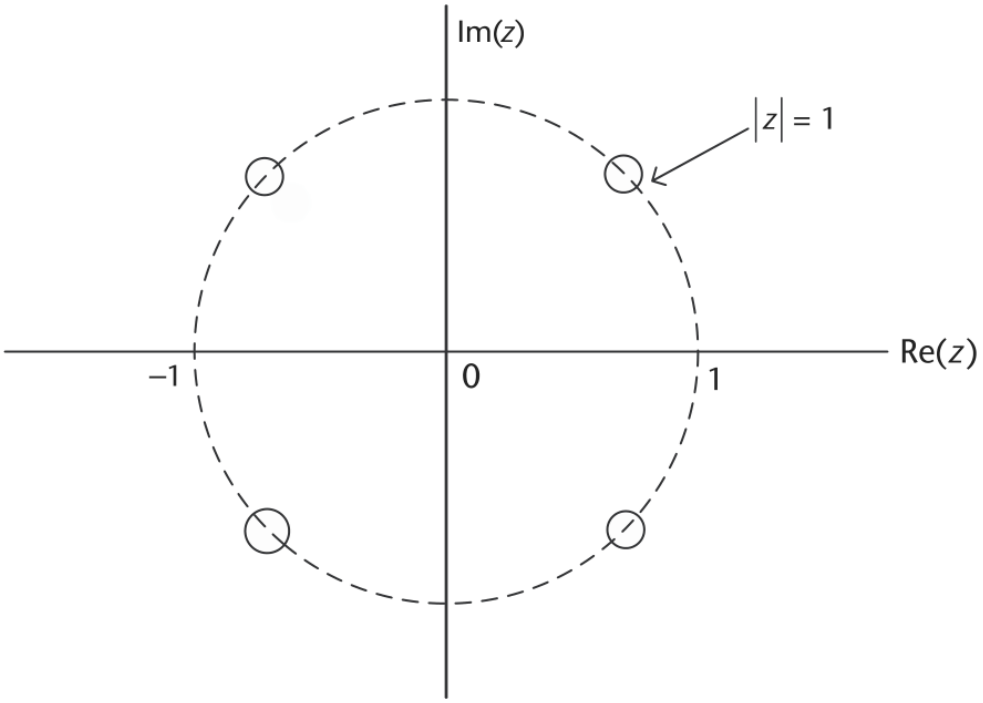


图 2.43　式 (2.65) 定义的 CIC 滤波器零点，全部零点都在单位圆上

或 16 个值。使用 RX HB3 滤波器时抽取因子设为 2，使用系数 [1, 4, 6, 4, 1]，注意 RX HB3 的满量程范围是 16（2^4）。使用 RX DEC3 滤波器时抽取因子设为 3，系数为 [55, 83, 0, −393, −580, 0, 1914, 4041, 5120, 4041, 1914, 0, −580, −393, 0, 83, 55]，满量程范围是 16384（2^{14}）。

2.7 SDR 发射技术

2.2 节已经介绍了模拟信号如何用 ADC 转换为数字信号（图 2.5）。这些由 0 和 1 构成的数字信号虽然可以用各种数字信号处理技术处理，却不能直接用于发射：必须先整形，再用 DAC 转换回模拟信号。

由此可见，额外的抽取会带来位增长并让系统获得额外的保真度。

图 2.44(a) 是一条连续模拟正弦波，以采样率 f_s 采样。这些采样经过 2.2 节描述的量化器，输出如图 2.44(b)。本例使用 4 位转换器，只有 16 个状态（便于观察）。很自然地，许多人想用平滑曲线把采样点连起来。但真实波形在采样之间确实存在多个离散台阶，如图 2.44(b) 所示。采样之间的那些值被忽略、不会从 ADC 输出。

从 ADC 出来、被数字信号处理算法处理、最终到达 DAC 的数据，并不需要波形平

滑。于是就有了图 2.45(a) 的情形：信号不连续、不平滑。只要理解 2.2.3 节的内容、不违反奈奎斯特采样定理，这就足够了。唯一出问题的时刻，是当我们真想把采样数据变回连续模拟域时：连续模拟域的带宽是无限的，并不会停在 $\frac{f_s}{2}$，如图 2.45(b) 所示。在数字界限之外考察时域数据，信号的镜像仍然可见。

这就是为什么 SDR 的发射端传统上有两个滤波器：数字的**发射（脉冲成形）滤波器**，把数字消息中的每个符号变成送入 DAC 的数字脉冲流；随后是模拟**重建滤波器**，去除 DAC 造成的混叠 [9]。该过程示于图 2.46。

2.7.1 模拟重建滤波器

2.5.4 节讨论过过采样如何放宽对抗混叠滤波器的要求，以及 sigma-delta ADC 如何天然具备这一优势。在基于 DAC 的系统中，插值的概念可以类似地配合模拟重建滤波器使用。这在数字音频 CD 播放器中很常见：CD 数据的基础更新率约 44.1 kSPS。如 2.2.5 节所述，往数据中插零、再过数字滤波器，把有效采样更新率提高到原来的 4 倍、8 倍或 16 倍。高过采样率把镜像频率推得更高，允许使用过渡带更宽、更简单的滤波器。

同一概念可以用于高速 DAC。设传统 DAC 以 30 MSPS 的输入字速率驱动（图 2.47 A），DAC 输出频率 f_s 为 10 MHz。20 MHz 处的镜像频率分量必须由模拟重建滤波器衰减，滤波器过渡带为 10 到 20 MHz。镜像须衰减 60 dB。因此滤波器要覆盖 10 MHz 通带，并在 10 到 20 MHz（一个倍频程）的过渡带内提供 60 dB 阻带衰减。模拟巴特沃斯滤波器每极点每倍频程提供 6 dB 衰减，因此至少需要 10 个极点。过渡带越窄，所需滤波器越复杂。

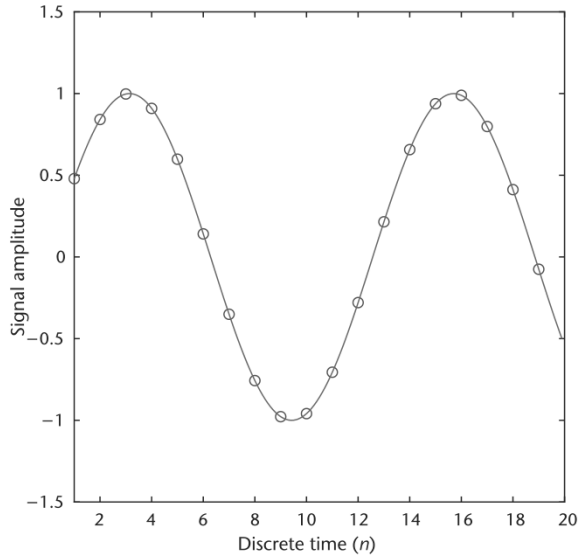
接下来，假设把 DAC 更新率提高到 60 MSPS、对原始数据采样做 2 倍插值，得到 60 MSPS 数据流。DAC 相对两倍过采样频率的响应示于图 2.47 B。模拟重建滤波器的过渡区变为 10 到 50 MHz（第一个镜像出现在 $2f_c - f_o = 60 - 10 = 50$ MHz）。该过渡区超过两个倍频程，意味着五或六极点的巴特沃斯滤波器就足够了——设计与制作都容易得多。

2.7.2 DAC

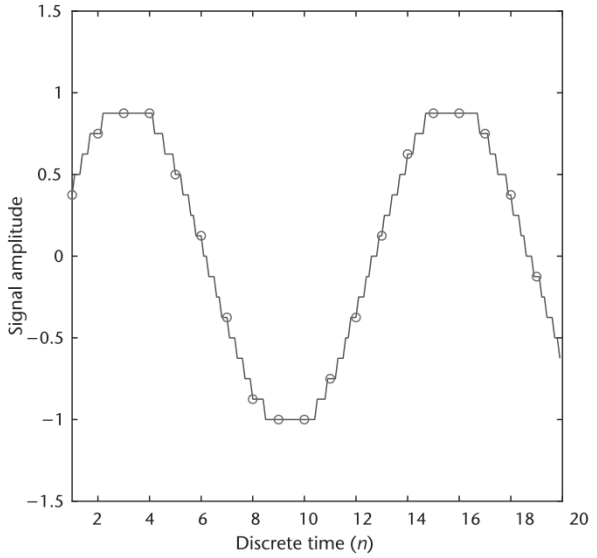
理论上，数模转换最简单的方法是把采样从存储器取出、转换成冲激串。与 2.2.2 节的采样函数类似，冲激调制器可定义为

$$p(t) = \sum_{k=-\infty}^{\infty} \delta(t - kT),$$

其中 $t = kT$ 时 $p(t) = 1$，其余时刻 $p(t) = 0$。因此冲激调制器之后的模拟脉冲串为

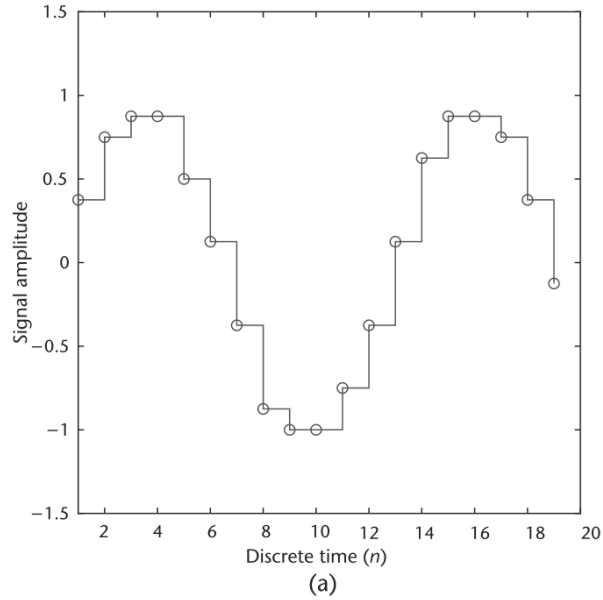


(a)



(b)

图 2.44　时域：(a) 连续模拟正弦波；(b) 量化后的模拟正弦波



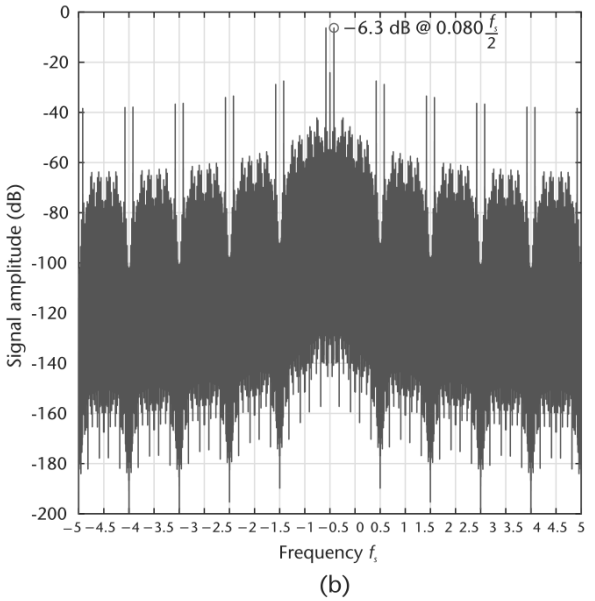


图 2.45　(a) 上采样带限正弦波：时域；(b) 带限随机数据：频域

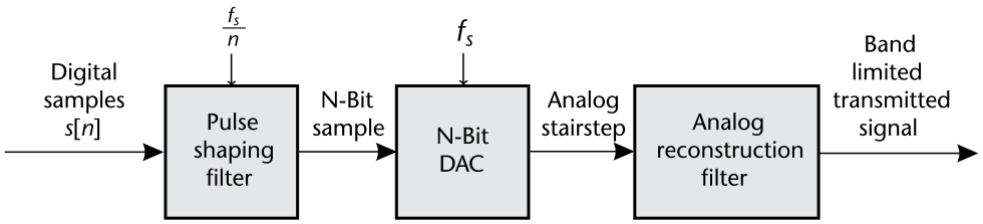


图 2.46　发射端：DAC 把数字符号转换为待发射的模拟信号

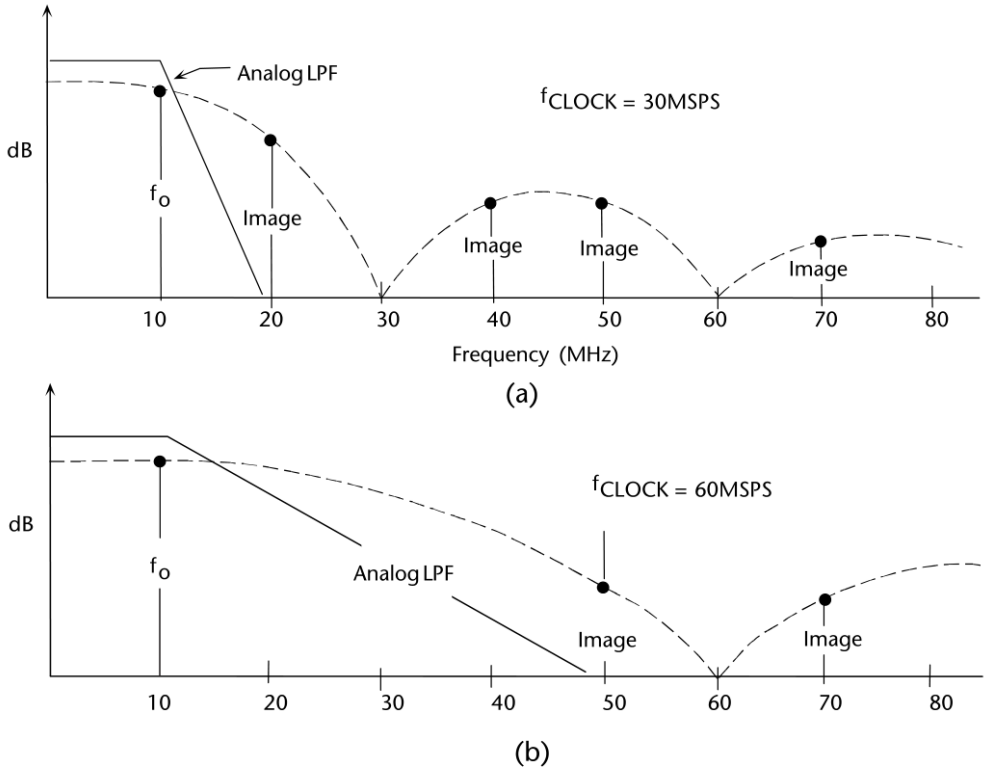


图 2.47　fo = 10 MHz、fs = 30 MSPS 与 fs = 60 MSPS 时模拟重建滤波器的要求 [18]

$$s_a(t) = s[n]p(t) = \sum_{k=-\infty}^{\infty} s(kT)\delta(t-kT) = \begin{cases} s(kT) & t = kT \\ 0 & t \neq kT \end{cases},$$

其中每个数字符号 $s[n]$ 触发一个被符号值缩放的模拟脉冲。把这个冲激串通过截止频率为采样率一半的低通滤波器，即可重建原始模拟信号。

虽然这个方法在数学上纯净，但即使在现代电子技术中，也难以产生所需的无限窄冲激脉冲。为了绕开这一点，几乎所有 DAC 都采用” 保持上一个值直到下一个采样到来” 的工作方式。这称为**零阶保持**（zeroth-order hold），相当于 ADC 采样保持的 DAC 对应物。零阶保持使时域呈现阶梯状外观，如图 2.44(b) 所示。

频域上，零阶保持使冲激串的频谱乘以 sinc 函数：

$$H(f) = \frac{\sin(\pi f/f_s)}{\pi f/f_s}.$$

如果你已有这方面背景：零阶保持可以理解为冲激串与宽度等于采样周期的矩形脉冲的卷积。这使得频域上乘以矩形脉冲的傅里叶变换，即 sinc 函数。图 2.47 中的虚线显示了 30 MHz 与 60 MHz DAC 的 sinc 函数。从图 2.47 可见，sinc 函数会衰减通带内信号。系统里需要有东西按零阶保持效应的倒数补偿这一影响，或者干脆接受这个非理想性。许多人无视这个问题，但在 Pluto SDR 上处理它非常容易——用户可以轻松地以非常平坦的通带发射。

2.7.3 数字脉冲成形滤波器

要理解为什么所有无线电设计都要包含数字脉冲成形滤波器，看一个简短的例子。相移键控（PSK）是简单而常用的数字调制方案，通过改变参考信号的相位来传输数据。图 2.48(a) 与 (d) 给出了载波的时域与频域：频率非常窄，应当容易发射。

出问题的是图 2.48(b) 与 (e) 中 1、0 交替的比特流。观察可知，方波含有无限的频率信息——在实际条件下这非常难以发射。

把图 2.48(a) 的载波与图 2.48(b) 的比特流相乘、试图把数据从天线发射出去时，得到图 2.48(c) 与 (f)——一个在连续时间模拟域中带宽无限的信号。我们不仅发射困难，相邻射频信道的近邻也不会喜欢我们。

回到冲激的数学模型：(2.70) 表明，若没有数字脉冲成形滤波器，这些脉冲 $s_a(t)$ 会直接经信道发射。理论上，只要接收端带宽无限，我们应该能收到同样的脉冲串（尽管有些延迟）。但现实中由于带宽有限与相邻符号间的干扰，我们实际上无法恢复这样的信号。

相邻符号相互干扰有两种情形：脉冲形状宽于单个符号间隔 T；信道非理想、把邻近脉冲抹开使其重叠。这两种情形都称为**码间串扰**（ISI）[9]。

为解决这些问题，引入脉冲成形滤波器来对发射波形限带。

2.7.4 奈奎斯特脉冲成形理论

通信系统中通常有两个脉冲成形滤波器，一个在发射端、一个在接收端，如图 2.49 所示，分别用 $h_T(t)$ 与 $h_R(t)$ 表示发射与接收滤波器的冲激响应。为简单起见，讨论奈奎斯特脉冲成形理论时通常用**等效信道**的概念——它不仅包含信道本身，还包含两个脉冲成形滤波器。等效信道的冲激响应为

$$h(t) = h_T(t) * h_C(t) * h_R(t).$$

现在考虑什么条件下能保证符号之间没有码间串扰。等效信道的输入 $s_a(t)$ 已在 (2.70) 定义。如前所述，每个模拟脉冲被符号值缩放，因此 $s_a(t)$ 可以写成另一种形式：

$$s_a(t) = \sum_k a_k \delta(t - kT),$$

其中 a_k 是第 k 个符号的值。它产生等效信道的输出 $y(t)$：

$$y(t) = s_a(t) * h(t) = \sum_k a_k [\delta(t - kT) * h(t)] = \sum_k a_k h(t - kT).$$

于是，给定特定时刻如 $t = mT$（m 为常数），输入 $s_a(t)$ 为

$$s_a(mT) = \sum_k a_k \delta(mT - kT) = a_m.$$

输出 $y(t)$ 则为

$$y(mT) = \sum_k a_k h(mT - kT) = a_0 h(mT) + a_1 h(mT - T) + \cdots + a_m h(mT - mT).$$

由于我们不想受其他符号干扰，希望输出只含 a_m 项：

$$y(mT) = a_m h(mT - mT).$$

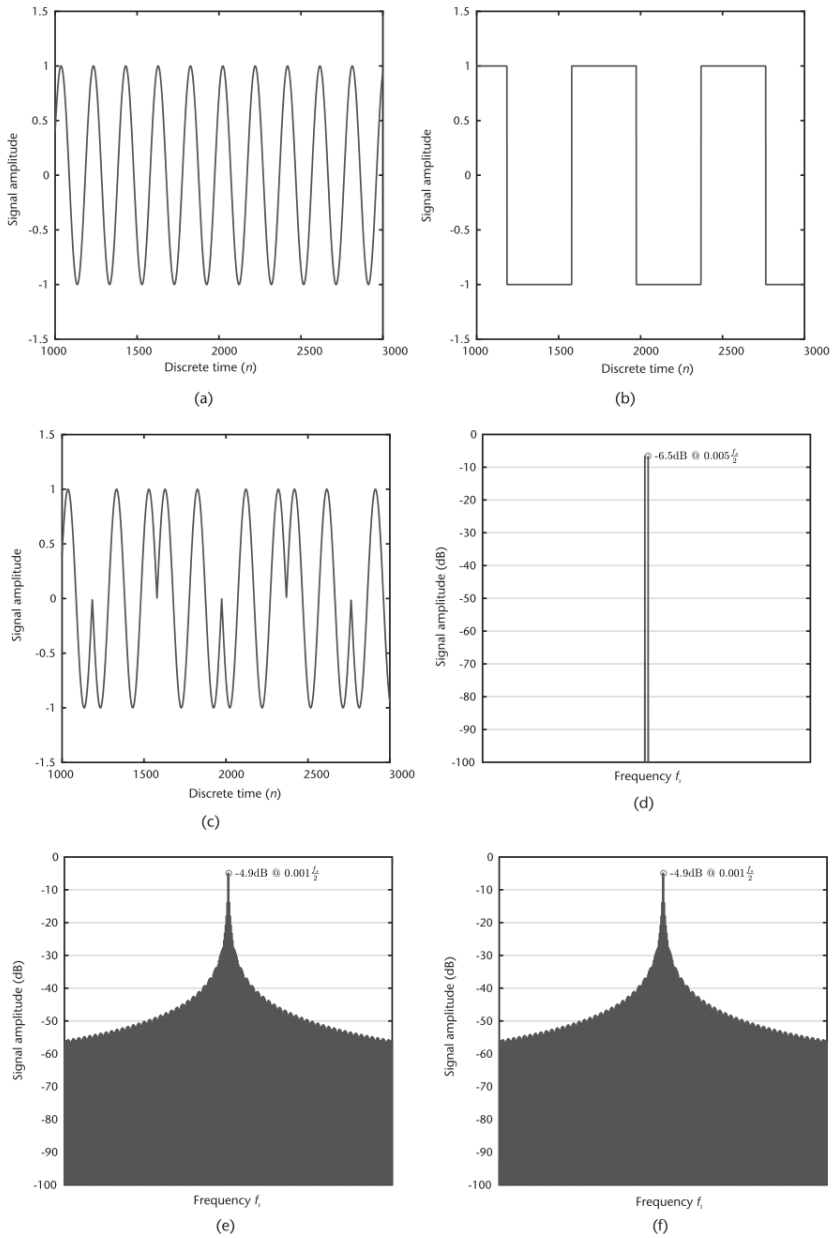


图 2.48　相移键控调制信号：载波、数据与调制后波形的时域和频域。(a) 载波：时域；(b) PSK 数据：时域；(c) PSK 调制：时域；(d) 载波：频域；(e) PSK 数据：频域；(f) PSK 调制：频域

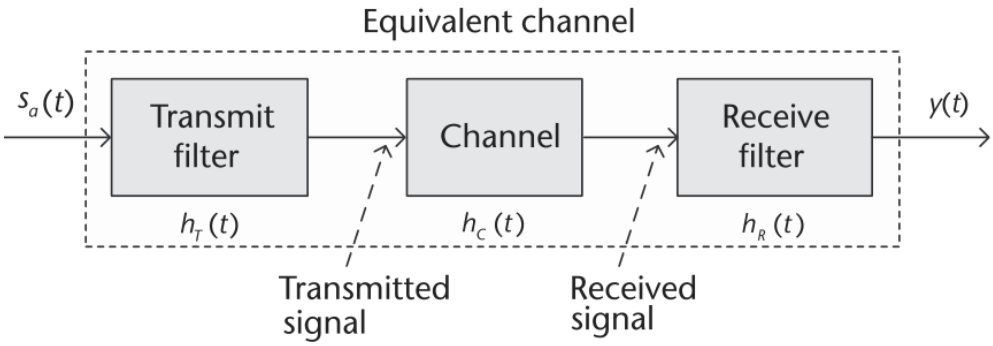


图 2.49　通信系统的等效信道：发射滤波器 + 信道 + 接收滤波器

进一步地，这意味着在时刻 $t = mT$，需要

$$h(mt - kT) = \begin{cases} C & k = m \\ 0 & k \neq m \end{cases},$$

其中 C 是某个非零常数。

把 (2.77) 推广到任意时刻，就得到奈奎斯特脉冲成形理论：一个脉冲不在后续间隔 T 的采样时刻干扰其他脉冲的条件，形式化为——$h(t)$ 若满足

$$h(t) = h(kT) = \begin{cases} C & k = 0 \\ 0 & k \neq 0 \end{cases}$$

（对所有整数 k），则称 $h(t)$ 为**奈奎斯特脉冲**。

2.7.5 两种奈奎斯特脉冲

本节介绍两种重要的奈奎斯特脉冲：sinc 脉冲与升余弦脉冲。考虑 (2.78)，最容易想到的脉冲是时间宽度小于 T 的矩形脉冲，或任何宽度小于 T 的脉冲形状。但矩形脉冲（及其他窄脉冲形状）的带宽可能太宽。窄脉冲形状不能高效利用频谱，而宽脉冲形状又可能引起 ISI——所以需要的是时间上宽（频率上窄）、又满足 (2.78) 奈奎斯特条件的信号 [9]。

数学上，sinc 函数定义为

$$\mathrm{sinc}(x) = \frac{\sin(\pi x)}{\pi x},$$

如图 2.50 所示。当变量 x 取整数 k 时，sinc 函数的值为

$$\text{sinc}(k)=\begin{cases}1 & k=0\\ 0 & k\neq 0\end{cases}.$$

换言之，归一化 sinc 函数的过零点位于非零整数处。

sinc 函数另一个重要性质：sinc 脉冲是矩形信号的傅里叶逆变换，如图 2.51(a) 所示。设矩形信号定义为 [19]：

$$H(\omega)=\begin{cases}T & |\omega|<\frac{1}{2T}\\ 0 & \text{otherwise}\end{cases},$$

对矩形信号取傅里叶逆变换得到 sinc 信号：

$$h(t)=\text{sinc}\frac{t}{T}.$$

在 (2.82) 中换变量 $t=kT$：

$$h(t)=h(kT)=\text{sinc}\frac{kT}{T}=\text{sinc}(k).$$

由于此处 k 是整数，根据 (2.80) 可以把 (2.83) 继续写为

$$h(t)=h(kT)=\text{sinc}\frac{kT}{T}=\text{sinc}(k)=\begin{cases}1 & k=0\\ 0 & k\neq 0\end{cases}.$$

把 (2.84) 与 (2.78) 对照，容易发现：只要取 $t=kT$，sinc 脉冲恰好满足 2.7.4 节的奈奎斯特脉冲成形理论。换句话说，把采样时刻选在 kT（采样频率等于 $\frac{1}{T}$），采样点就落在等间隔的过零点上（图 2.51(b)），因此不会有码间串扰。

回忆 2.2.3 节的奈奎斯特采样定理：带限 B Hz 的实信号 $x(t)$ 可以用最低 $F_s=2B$ Hz 的采样频率无误差重建。本例中最低采样频率是 $F_s=\frac{1}{T}$ Hz，因此对应的最低带宽为

$$B=\frac{F_s}{2}=\frac{1}{2T},$$

这恰好是 (2.81) 定义的矩形信号带宽。基于以上讨论，sinc 脉冲 $h(t)$ 的这一选择给出最小带宽 $B=B_{\text{min}}=\frac{1}{2T}$，因此称为**奈奎斯特-I 脉冲** [20]。

sinc 脉冲非常有吸引力：时间上宽、频率上窄，兼有频谱效率与无 ISI 的优点。但

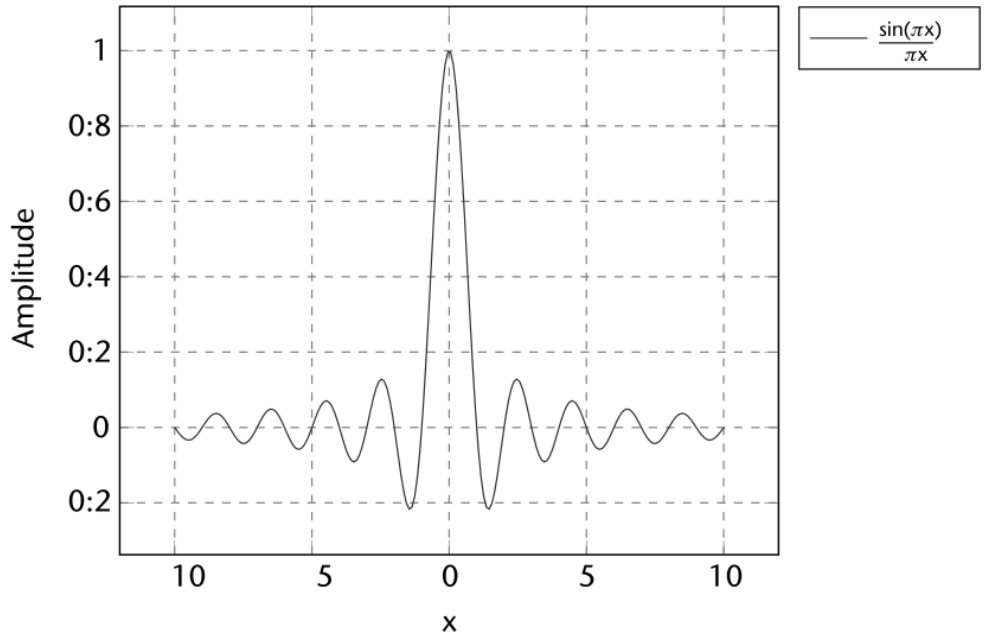


图 2.50　式 (2.79) 定义的 sinc 函数图像。x 轴为 x，y 轴为 sinc(x)

sinc 脉冲并不实用——它对定时误差引起的 ISI 很敏感。例如，对较大的 t，(2.82) 定义的 sinc 脉冲有如下近似：

$$h(t) \sim \frac{1}{t},$$

显然定时误差会造成大的 ISI。还必须注意 sinc 脉冲在时间上无限长，物理上不可实现。

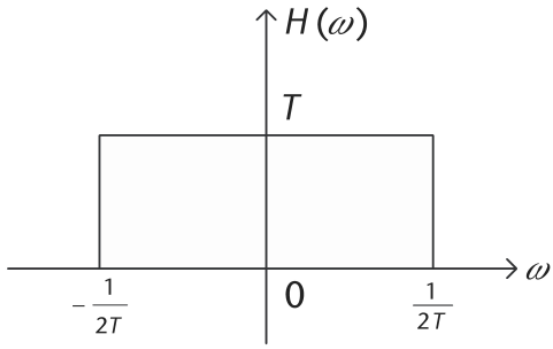
因此需要引入**奈奎斯特-II 脉冲**：带宽更大 $B > B_{\min}$，但对 ISI 敏感度更低。这类脉冲更实用，实践中用得也远为广泛。

升余弦脉冲是最重要的奈奎斯特-II 脉冲之一，其频率传递函数定义为

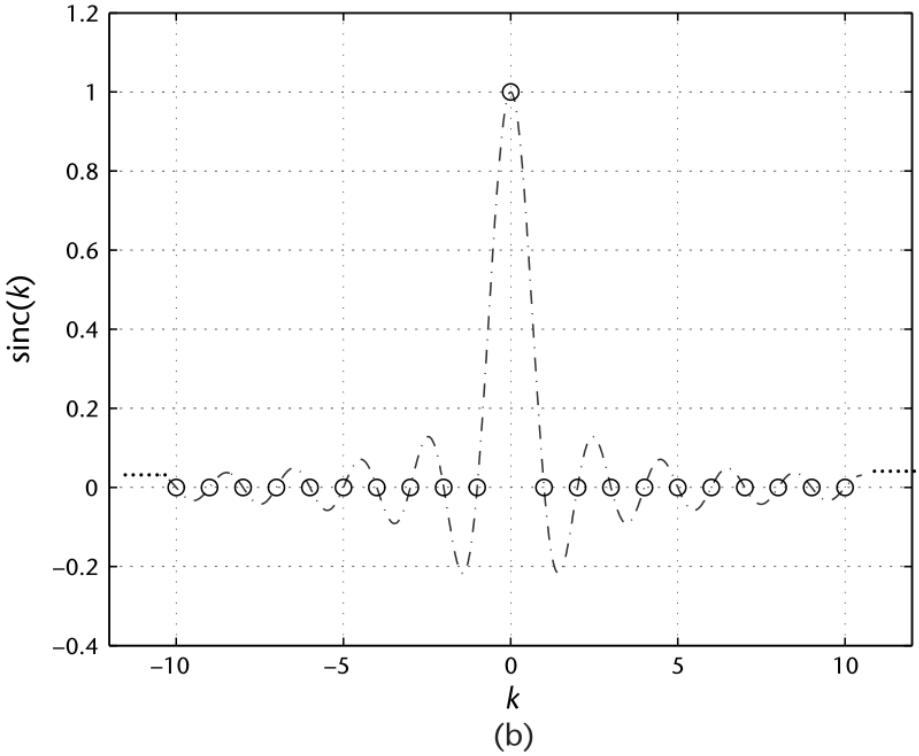
$$H_{RC}(f) = \begin{cases} T & 0 \le |f| \le \frac{1-\beta}{2T} \\ \frac{T}{2}\left[1 + \cos\frac{\pi T}{\beta}\left(|f| - \frac{1-\beta}{2T}\right)\right] & \frac{1-\beta}{2T} \le |f| \le \frac{1+\beta}{2T} \\ 0 & |f| \ge \frac{1+\beta}{2T} \end{cases},$$

其中 β 是滚降因子，取值 0 到 1；$\frac{\beta}{2T}$ 是过剩带宽。

升余弦脉冲的频谱示于图 2.52。一般地，其带宽 $B \ge \frac{1}{2T}$。当 $\beta = 0$ 时带宽 $B = \frac{1}{2T}$、没有过剩带宽，实际就是矩形脉冲；另一端，当 $\beta = 1$ 时达到最大带宽 $B = \frac{1}{T}$。



(a)



(b)

图 2.51　sinc 脉冲（时域）及其傅里叶变换矩形脉冲（频域）。(a) 频域矩形脉冲（式 2.81）；(b) sinc 脉冲（式 2.84）。x 轴为 k（k = t/T），y 轴为 sinc(k)

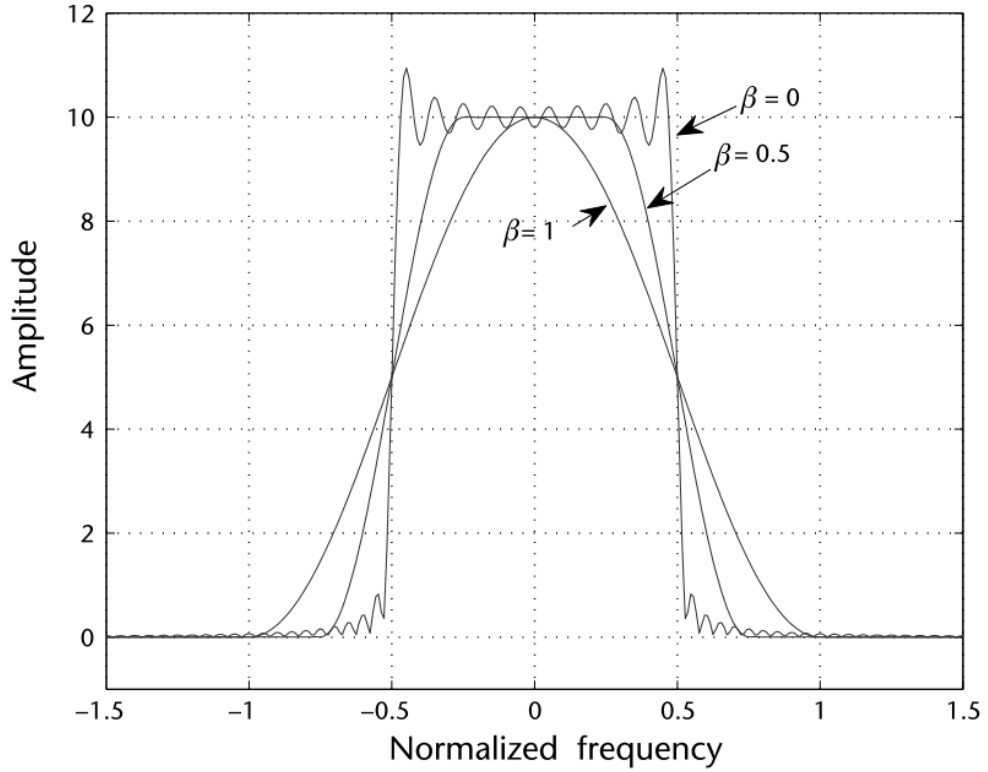


图 2.52　式 (2.87) 定义的升余弦脉冲频谱，随滚降因子 β 变化。x 轴为归一化频率 f0，实际频率为 f0/T

对 $H_{RC}(f)$ 取傅里叶逆变换，得到升余弦脉冲的冲激响应：

$$h_{RC}(t) = \frac{\cos\frac{\pi\beta t}{T}}{1-\left(\frac{2\beta t}{T}\right)^2}\,\mathrm{sinc}\frac{\pi t}{T}.$$

奈奎斯特-II 脉冲没有 ISI 敏感性问题，因为其峰值失真（$h_{RC}(t)$ 的拖尾）收敛很快，可表示为

$$D_p = \sum_{n=-\infty}^{\infty} |h_{RC}(\epsilon' + (n-k))| \sim \frac{1}{n^3}.$$

因此当定时误差发生时，失真不会像奈奎斯特-I 脉冲那样以类似方式累积到无穷 [20]。

实际上，许多实用通信系统常用的是**根升余弦滤波器** [21]。若把通信信道视为理想信道，且假设发射滤波器与接收滤波器相同，则两者都可以用根升余弦滤波器，它们的总响应必须等于 (2.87) 定义的 $H_{RC}(f)$。由于等效信道的冲激响应可表示为

$$h(t) = h_T(t) * h_C(t) * h_R(t),$$

其中 $h_C(t)$ 是通信信道的冲激响应，$h_T(t)$ 与 $h_R(t)$ 是发射与接收滤波器的冲激响应，这意味着频域上有

$$H_{RC}(f) = H_T(f)H_R(f).$$

因此根升余弦滤波器的频率响应必须满足

$$|H_T(f)| = |H_R(f)| = \sqrt{|H_{RC}(f)|}.$$

2.8 本章小结

做 SDR 实验既需要过硬的计算机技能，也需要广博的数字信号处理知识。本章列举了若干有用的主题，包括采样、脉冲成形与滤波。本章的目的是帮读者为后续实验章节（特别是 Pluto SDR 硬件）做好准备。例如在 Pluto SDR 发射路径上（图 2.53），有一个可编程 128 抽头 FIR 滤波器、若干插值半带滤波器、一个 DAC，再接两个低通模拟重建滤波器（LPF）。要理解它们如何协同工作并正确配置，就需要理解采样理论。

在 Pluto SDR 接收路径上（图 2.54），信号流经抗混叠滤波器、ADC、若干数字抽取

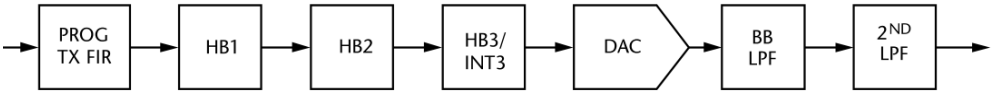


图 2.53　AD9361 内部发射信号路径

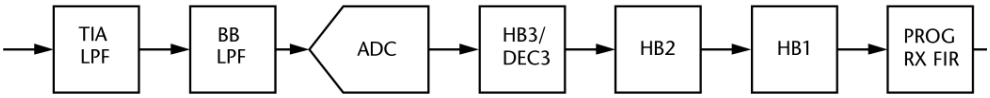


图 2.54　AD9361 内部接收信号路径

半带滤波器，最终到达 128 抽头可编程 FIR——滤波知识在这里派上用场。这个 128 抽头可编程 FIR 可用来补偿抗混叠滤波器在通带内的损失。

此外，搭建真实通信系统时还需要额外的发射与接收滤波器，这需要脉冲成形方面的专长。

2.9 参考文献

[1] Proakis, J. G., and Dimitris G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, Third Edition, Upper Saddle River, NJ: Prentice Hall, 1996.

[2] Elali, T. S., and M. A. Karim, *Continuous Signals and Systems with MATLAB*, Boca Raton, FL: CRC Press, 2001.

[3] Harris, F. J., "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform," *Proceedings of the IEEE*, Vol. 66, No. 1, January 1978.

[4] Smith, S. W., *The Scientist and Engineer's Guide to Digital Signal Processing*, Second Edition, 1999. http://www.analog.com/en/education/education-library/scientist_engineers_guide.html.

[5] Frigo, M., "A Fast Fourier Transform Compiler," *Proceedings of the 1999 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '99)*, Atlanta, GA, May 1999. http://www.fftw.org/.

[6] Kester, W., *Understand SINAD, ENOB, SNR, THD, THD + N, and SFDR so You Don't Get Lost in the Noise Floor*, Analog Devices, 2008. http://www.analog.com/MT-003.

[7] Kester, W., *Mixed Signal and DSP Design Techniques*, Analog Devices, 2002. http://www.analog.com/en/education/education-library/mixed_signal_dsp_design_book.html.

[8] James, J. F., *A Student's Guide to Fourier Transforms: With Applications in Physics and*

Engineering, Third Edition, Cambridge, UK: Cambridge University Press, 2011.

[9] Johnson, C. R., Jr., and W. A. Sethares, *Telecommunications Breakdown: Concepts of Communication Transmitted via Software-Defined Radio*, Prentice Hall, 2004.

[10] Cavicchi, T. J., *Digital Signal Processing*, John Wiley and Sons, 2000.

[11] Oppenheim, A. V., and R. W. Schafer, *Digital Signal Processing*, Prentice-Hall, 1975.

[12] Analog Devices AD7177 Product Page, 2016. http://www.analog.com/AD7177-2.

[13] Analog Devices AD9208 Product Page, 2017. http://www.analog.com/AD9208.

[14] Smith, S., *The Scientist and Engineer's Guide to Digital Signal Processing*, San Diego: California Technical Publishing, 1997.

[15] Jeffrey, A., and D. Zwillinger, *Table of Integrals, Series, and Products*, Seventh Edition, San Diego: Academic Press, 2007.

[16] Hogenauer, E. B., "An Economical Class of Digital Filters for Decimation and Interpolation," *IEEE Transactions on Acoustics, Speech and Signal Processing*, Vol. 29, No. 2, 1981, pp. 155–162.

[17] Lyons, R. G., *Streamlining Digital Signal Processing: A Tricks of the Trade Guidebook*, Piscataway, NJ: Wiley-IEEE Press, 2007.

[18] Kester, W., *The Data Conversion Handbook*, 2005. http://www.analog.com/en/education/education-library/data-conversion-handbook.html.

[19] Johnson, C. R., W. A. Sethares, and A. G. Klein, *Software Receiver Design: Build Your Own Digital Communications System in Five Easy Steps*, Cambridge, UK: Cambridge University Press, 2011.

[20] Wyglinski, A. M., M. Nekovee, and Y. T. Hou, *Cognitive Radio Communications and Networks: Principles and Practice*, Burlington, MA: Academic Press, 2009.

[21] Barry, J., E. A. Lee, and D. G. Messerschmitt, *Digital Communication*, Third Edition, Kluwer Academic Press, 2004.

第 3 章 通信中的概率

上一章从纯确定性的视角为我们提供了建模与处理信号和系统的一整套洞察、工具与算法。然而，通信系统环境中存在大量具有随机特性的元素：信息源二进制数据流的取值、传输穿越通信信道时引入的噪声、以及同一环境中多用户运行产生的干扰源。第 2 章给出的是理解与建模确定性信号和系统的工具集，本章则介绍一组建模与研究**概率性**信号与系统的技术与方法。我们从连续与离散随机变量的概念入手，接着解释随时间变化的随机现象——随机过程，最后对各种随机噪声信道进行建模。

3.1 通信系统中离散随机事件的建模

离散随机变量表示通信系统中结果并非确切已知的某类行为。例如，二进制信息源产生的下一个值有两种可能输出：二进制 1 或二进制 0。虽然我们知道两个值都有可能，但我们并没有确切知识断定信息源输出端具体会产生哪一个。因此，我们把二进制信息源产生的输出值建模为随机变量。称之为**离散**随机变量，是因为它从有限个选项中产生单个可能的输出值。

为数学地建模这个离散随机变量，设 X 存在一个它可以产生的互异实数集合 x_i，每个值都有特定的发生概率：

$$\sum_i P(X = x_i) = 1,$$

其中 $P(X = x_i)$ 是随机变量产生输出值 x_i 的概率。利用全概率定律 [1]，我们知道

$$P(X \in B) = \sum_{i:x_i \in B} P(X = x_i),$$

其中集合 B 由一组值 x_i 组成。离散随机变量的一种特定形式是整数值随机变量，其

输出值为整数 $x_i = i$，即

$$P(X \in B) = \sum_{i \in B} P(X = i).$$

由于随机变量 X 的每个输出具有不同的发生概率，我们通常把 X 产生特定离散输出 x_i 的概率定义为

$$p_X(x_i) = P(X = x_i),$$

其中 $p_X(x_i)$ 称为**概率质量函数**（PMF）。注意 PMF 的值受约束于

$$0 \le p_X(x_i) \le 1 \quad \text{且} \quad \sum_i p_X(x_i) = 1.$$

表 3.1 列出了几个常用的 PMF，包括均匀、泊松与伯努利随机变量。特别地，伯努利随机变量用于生成二进制信息源的随机输出，而泊松随机变量常用于建模计算机网络中路由分组的时延。

表 3.1: 若干常用的概率质量函数（图形表示从略，见原书）

随机变量	PMF 定义
均匀	$p_X(k) = \begin{cases} \frac{1}{n}, & k = 1, \ldots, n \\ 0, & \text{otherwise} \end{cases}$
泊松	$p_X(k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \ldots$
伯努利	$p_X(k) = \begin{cases} p, & k = 1 \\ 1 - p, & k = 0 \\ 0, & \text{otherwise} \end{cases}$

现在探索如何用这些离散随机变量建模通信系统的一个实际组件——二进制信息源。需要纳入该工具的一个关键特性是随机数发生器（RNG）产生的 1 与 0 的占比。代码 3.1 的 MATLAB 脚本实现了一个二进制 RNG，产生三个向量：向量 `b1` 含等量的 1 与 0，向量 `b2` 与 `b3` 分别按 60/40 与 20/80 的比例生成 1 与 0。注意这里用 MATLAB 的 `rand` 均匀 RNG 函数产生 0 到 1 之间的随机值，再四舍五入到最近的整数（即 1 或 0）。

代码 3.1　信息源：random_example.m

```
% Create three binary data streams of length L, each with a different
```

```
% percentage of 1 and 0 values
L = 100;
prob1 = 0.5; prob2 = 0.6; prob3 = 0.2; % Probability values for 1 outputs
b1 = round(0.5*rand(1,L)+0.5*prob1); % Even split between 1 and 0 values
b2 = round(0.5*rand(1,L)+0.5*prob2); % Have 60% 1 values and 40% 0 values
b3 = round(0.5*rand(1,L)+0.5*prob3); % Have 20% 1 values and 80% 0 values
```

通过把 rand 的输出值偏向 1 或 0 一侧再取整，可以生成 1/0 占比不同的二进制随机值，如图 3.1 所示。用 stem 命令可以可视化实际的二进制值（图 3.1(a)、(c)、(e)）。虽然这给了我们这些二进制 RNG 行为的总体观察，但要区分向量中 1 与 0 的实际占比却很困难。因此，用直方图和非常长的随机值序列可以刻画这些占比，如图 3.1(b)、(d)、(f) 所示。注意直方图如何准确显示输出向量中 1 与 0 的百分比。一个重要提醒：由于我们的刻画依赖对随机现象的观察，需要观察非常大量的数据才能准确刻画它。

3.1.1 期望

由于我们无法确切知道构成通信系统及其环境的这些随机现象的输出值，只能转而数学地刻画这些随机变量的统计行为。刻画随机变量的一种方式是它的**期望值**或**均值**，可用下式定量确定：

$$m_X = E[X] = \sum_i x_i P(X = x_i),$$

即每个值的和以其发生概率加权。于是，利用 PMF 的定义，(3.6) 可以改写为

$$E[X] = \sum_i x_i p_X(x_i).$$

设随机变量 X 与实值函数 $g(x)$，把 X 映射为新的随机变量 Z，即 $Z = g(X)$。由于 X 本身就是从样本空间 Ω 到一组实数的映射，求解随机变量 Z 时我们在做" 映射的映射"，即 $Z(\omega) = g(X[\omega])$。于是，计算随机变量 Z 的期望 $E[Z]$ 可用下式：

$$E[Z] = E[g(X)] = \sum_i g(x_i) p_X(x_i),$$

这称为随机变量的函数的期望。(3.8) 最重要的方面之一：Z 的期望可以直接用 X 的 PMF 求解，而不必先确定 Z 的 PMF。表 3.2 给出与期望算子相关的若干有用性质。

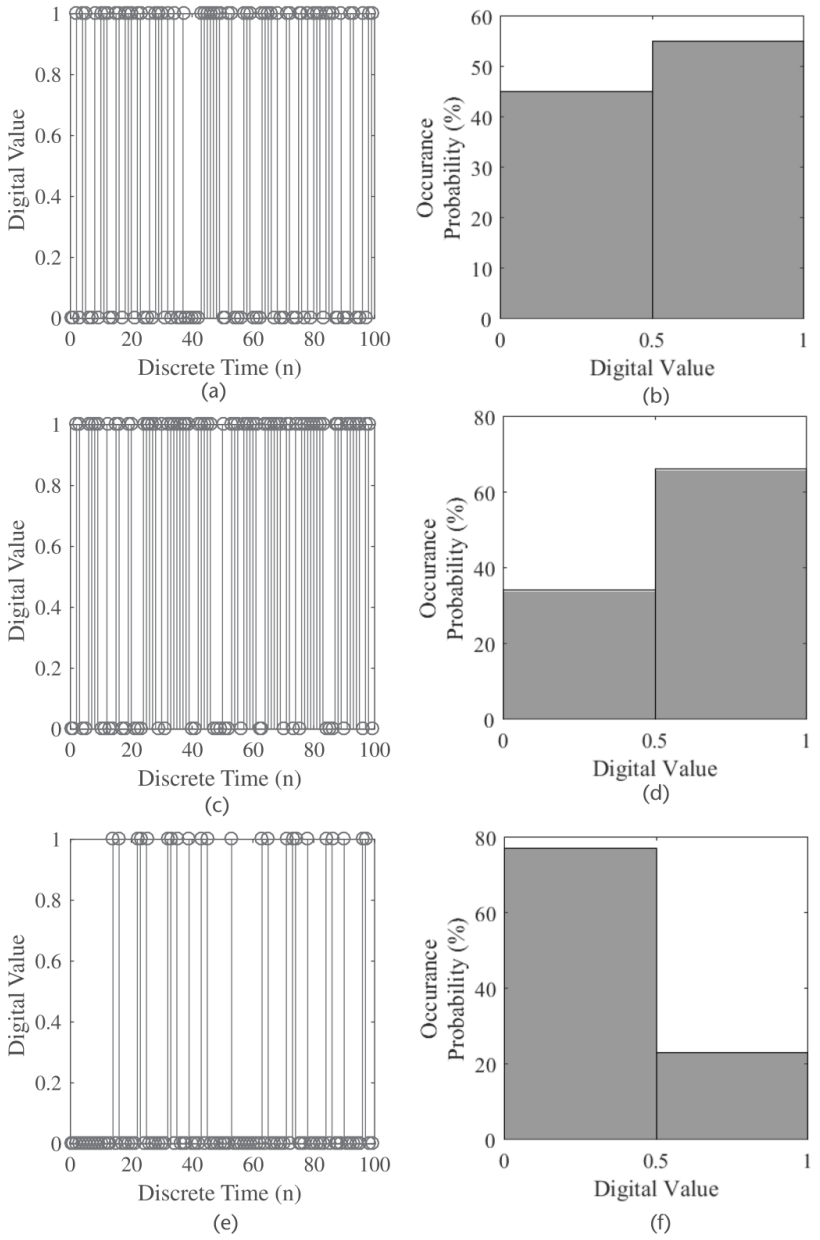


图 3.1　三条二进制传输的杆图与直方图，各具不同的 1/0 产生概率：(a) 二进制信号（50% 1，50% 0）；(b) 直方图（50% 1，50% 0）；(c) 二进制信号（60% 1，40% 0）；(d) 直方图（60% 1，40% 0）；(e) 二进制信号（20% 1，80% 0）；(f) 直方图（20% 1，80% 0）

表 3.2: 期望的若干有用性质

名称	定义
线性	若 a、b 为确定性常数，X、Y 为随机变量，则 $E[aX+bY]=E[aX]+E[bY]=aE[X]+bE[Y]$
矩	实值随机变量 X 的 n 阶矩（$n\geq 1$）定义为 X 的 n 次幂的期望，即 $\text{Moment}_n(X)=E[X^n]$
均值	均值是 X 的一阶矩，即 $\text{Moment}_1(X)=E[X]=\mu$
方差	X 减去均值后的二阶矩是它的方差，即 $\text{Moment}_2(X-\mu)=E[(X-\mu)^2]=\text{var}(X)=\sigma^2$
中心矩	方差的推广是 X 的 n 阶中心矩，即 $E[(X-\mu)^n]$。注意 X 的偏度与峰度分别是三阶与四阶中心矩
相关	两个随机变量 X 与 Y 的相关定义为 $E[XY]$
协方差	X 与 Y 的协方差定义为 $\text{cov}(X,Y)=E[(X-\mu_X)(Y-\mu_Y)]$
相关系数	X 与 Y 的相关系数为 $\rho_{XY}=E\left[\frac{X-\mu_X}{\sigma_X}\cdot\frac{Y-\mu_Y}{\sigma_Y}\right]$
马尔可夫不等式	若 X 是非负随机变量，则对任意 $a>0$ 有 $P(X\geq a)\leq\frac{E[X]}{a}$
切比雪夫不等式	对任意随机变量 Y 与任意 $a>0$，有 $P(\lvert Y\rvert\geq a)\leq\frac{E[Y^2]}{a^2}$
柯西–施瓦茨不等式	$\lvert E[XY]\rvert\leq\sqrt{E[X^2]E[Y^2]}$，当且仅当 X 与 Y 线性相关时取等
独立	随机变量独立当且仅当对所有函数 $h(x)$、$k(y)$ 有 $E[h(X)k(Y)]=E[h(X)]E[k(Y)]$

3.2 二元通信信道与条件概率

在数字通信系统中，可以把接收机的操作看作：当观测到一个传输值时，猜测某个特定发射机发送了什么。发送值可建模为随机变量 X，接收端观测到的值可建模为随机变量 Y。于是我们需要某种**判决规则**来弄清最初发射的是 X 的哪个值。图 3.2 说明了这一点：X 可以输出 0 或 1，而 Y 在接收端被观测。注意观测 Y 可能被误判为与发射值

不同的值，即 $X \neq Y$。

为刻画并理解这个随机传输环境，可以使用**条件概率**的定义，其数学表达式为

$$P(X \in B \,|\, Y \in C) = P(\{X \in B\} \,|\, \{Y \in C\}) = \frac{P(\{X \in B\} \cap \{Y \in C\})}{P(\{Y \in C\})} = \frac{P(X \in B, Y \in C)}{P(Y \in C)}.$$

此外，条件 PMF 也可以用来数学地描述这个随机现象：

$$p_{X|Y}(x_i|y_j) = P(X = x_i \,|\, Y = y_j) = \frac{P(X = x_i, Y = y_j)}{P(Y = y_j)} = \frac{p_{XY}(x_i, y_j)}{p_Y(y_j)},$$

$$p_{Y|X}(y_j|x_i) = P(Y = y_j \,|\, X = x_i) = \frac{P(X = x_i, Y = y_j)}{P(X = x_i)} = \frac{p_{XY}(x_i, y_j)}{p_X(x_i)},$$

进而可以表达为

$$p_{XY}(x_i, y_j) = p_{X|Y}(x_i|y_j)p_Y(y_j) = p_{Y|X}(y_j|x_i)p_X(x_i).$$

一个能帮助刻画条件概率和/或条件 PMF 所描述现象的有用数学工具是**全概率定律**。例如，设 Y 为任意随机变量，取 $A = \{Y \in C\}$（$C \subset \mathbb{R}$），则全概率定律定义为

$$P(Y \in C) = \sum_i P(Y \in C \,|\, X = x_i)P(X = x_i).$$

类似地，若 Y 是取互异值 y_i 的离散随机变量，取 $C = \{y_j\}$，则得到如下形式的全概率定律：

$$P(Y = y_j) = \sum_i P(Y = y_j \,|\, X = x_i)P(X = x_i) = \sum_i p_{Y|X}(y_j)p_X(x_i).$$

> 思考题：利用期望的定义，推导马尔可夫不等式 $P(X \geq a) \leq \frac{E[X^r]}{a^r}$；并推导连续随机变量情形的全概率与替换定律的相应表达式。

回到图 3.2 描述的二元通信信道模型：假设我们想基于对截获接收信号的观测来判

决发射的是什么值。此时我们希望采用**最大后验**（MAP）判决规则：给定观测 $Y=j$，若

$$P(X=1\,|\,Y=j)\geq P(X=0\,|\,Y=j),$$

MAP 规则判 $X=1$，否则判 $X=0$。换言之，给定观测 $Y=j$，若 $X=1$ 的后验概率大于 $X=0$ 的后验概率，MAP 规则就判 $X=1$。进一步，我们观察到

$$P(X=i\,|\,Y=j)=\frac{P(X=i,Y=j)}{P(Y=j)}=\frac{P(Y=j\,|\,X=i)P(X=i)}{P(Y=j)},$$

用它把 (3.14) 改写为

$$\begin{aligned}&\frac{P(Y=j\,|\,X=1)P(X=1)}{P(Y=j)}\geq\frac{P(Y=j\,|\,X=0)P(X=0)}{P(Y=j)}\\ \Longrightarrow\quad&P(Y=j\,|\,X=1)P(X=1)\geq P(Y=j\,|\,X=0)P(X=0).\end{aligned}$$

若 $X=0$ 与 $X=1$ 等概率发生，可简化得到**最大似然**（ML）规则：

$$P(Y=j\,|\,X=1)\geq P(Y=j\,|\,X=0).$$

而且在一般情形下，可以把 (3.16) 重排得到**似然比**：

$$\frac{P(Y=j\,|\,X=1)}{P(Y=j\,|\,X=0)}\geq\frac{P(X=0)}{P(X=1)},$$

其中右侧称为**门限**，因为它不依赖 j。

有了这个数学表述，现在用计算机仿真处理同一个例子。用下面的 MATLAB 脚本可以建模一个二元信道：产生 L 个二进制值，1 的概率为 `prob4`，其余为 0。此外，假设所用二元信道**不对称**——1 翻转为 0 的概率不同于 0 翻转为 1 的概率。比特翻转被视为信道产生的错误。于是定义：发射 1 被接收为 1 的概率等于 `prob11`，发射 0 被接收为 0 的概率等于 `prob00`。

代码 3.2　仿真：random_example.m

```
% Define simulation parameters
L = 100000;      % Transmission length
prob00 = 0.95;   % Probability zero received given zero transmitted
prob11 = 0.99;   % Probability one received given one transmitted
prob4 = 0.7;     % Have 40% 1 values and 60% 0 values
```

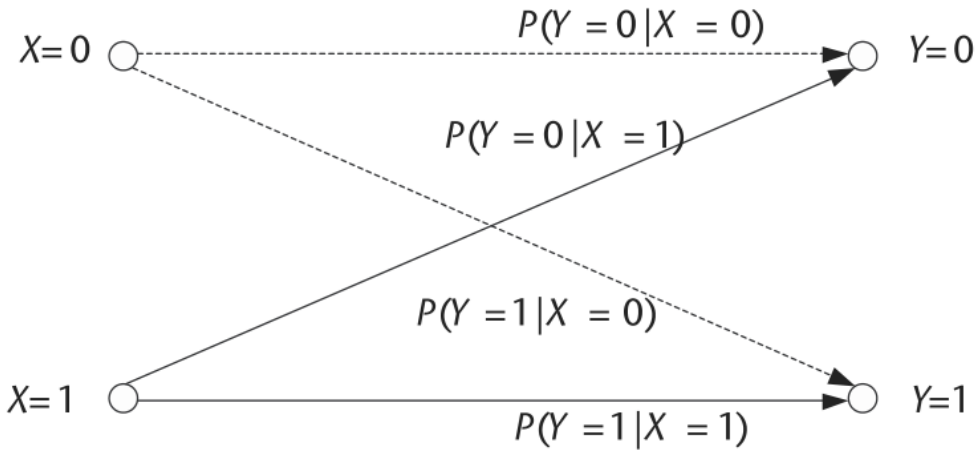


图 3.2　一个二元信道的例子：随机变量 X 生成的传输值在接收端被观测为随机变量 Y

```
% Create transmitted binary data stream
b4 = round(0.5*rand(1,L)+0.5*prob4);
                                    % Have 40% 1 values and 60% 0 values
b4hat = b4;   % Initialize receive binary data stream

% Randomly select 1 and 0 values for flipping
ind_zero = find(b4 == 0);  % Find those time instances with zero values
ind_one = find(b4 == 1);   % Find those time instances with one values
ind_flip_zero = find(round(0.5*rand(1,length(ind_zero))
                  +0.5*(1-prob00)) == 1);  % Flip zero bits to one bits
ind_flip_one = find(round(0.5*rand(1,length(ind_one))
                +0.5*(1-prob11)) == 1);    % Flip one bits to zero bits

% Corrupt received binary data stream
b4hat(ind_zero(ind_flip_zero)) = 1;  % Flip 0 to 1
b4hat(ind_one(ind_flip_one)) = 0;    % Flip 1 to 0

% Calculate bit error statistics
b4error_total = sum(abs(b4-b4hat))/L;
b4error_1 = sum(abs(b4(ind_one) - b4hat(ind_one)))/length(ind_one);
```

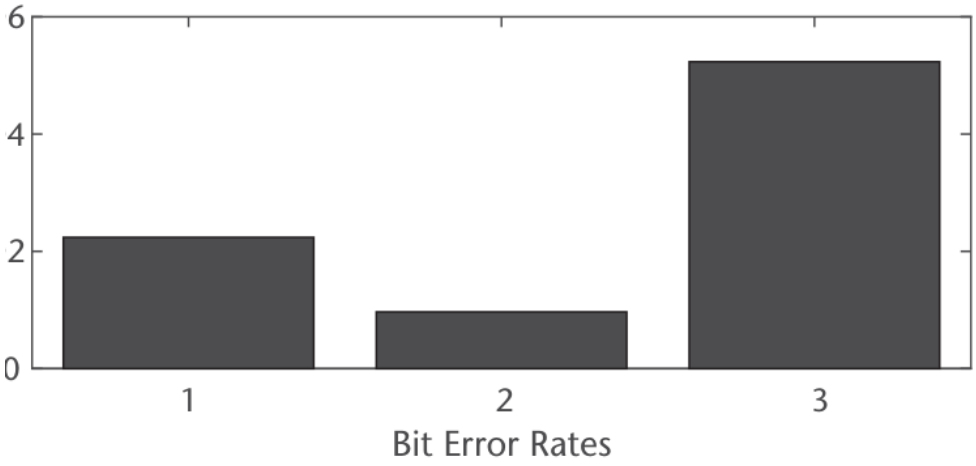


图 3.3　二元信道的误码概率：0 被无错接收的概率为 0.95、1 被无错接收的概率为 0.99，传输含 40% 的 1 与 60% 的 0。1 = 总体，2 = 仅发射 1，3 = 仅发射 0

```
b4error_0 = sum(abs(b4(ind_zero) - b4hat(ind_zero)))/length(ind_zero);
```

评估任何数字通信系统性能的基本指标之一是**误比特概率**，即误码率（BER）。BER 刻画接收到的比特错误量相对总发射比特数的比例。对不同应用，不同的 BER 值有的可接受、有的不可容忍。例如，对典型无线数据传输应用，10^{-5} 的 BER 被认为是可以接受的误码水平。

在上述二元信道的 MATLAB 例子中，我们想刻画 BER 值，看它们是否符合我们定义的参数。图 3.3 给出了整体传输的 BER，以及仅发射 1 与仅发射 0 的 BER。注意：仅 1 值的 BER 恰好对应概率 `prob11` 的补，仅 0 值的 BER 对应 `prob00` 的补。记住：要获得准确的 BER 统计评估，需要生成非常大量的二进制值。

3.3 通信系统中连续随机事件的建模

如本章前文所见，可以数学地计算由离散随机变量描述的通信系统中随机事件发生的概率：

$$P(a \leq X < b) = \sum_{i=a}^{b-1} p_X(i),$$

其中 X 是离散随机变量，a、b 是样本空间 Ω 中某子集的边界。但现在假设 X 是**连续**随机变量，可以取区间 (a, b) 内的整个连续统的值。要在此情形计算同样的概率，可以把这个场景理解为 (a, b) 中无穷多个点的求和，相邻样本间距为 Δx。

通信系统中有大量随机元素，每个都能产生连续统的可能输出值。如本章稍后将讨论的，其中一个由连续随机变量表示的元素就是传输信道中引入的噪声。设该噪声可表示为无穷多个样本，使得 Δx 最终收敛为 dx，(3.19) 的求和变成积分表达式。于是，可以用下式表达建模噪声的连续随机变量产生 a 到 b 之间输出值的概率：

$$P(a \leq X < b) = \int_a^b f(t)\, dt,$$

其中 $f(t)$ 称为**概率密度函数**（PDF）。注意 PDF 是本章前面讨论过的 PMF 的连续版本。而且，一般可以用 PDF 通过下式表达连续随机变量的概率：

$$P(X \in B) = \int_B f(t)\, dt = \int_{-\infty}^{+\infty} I_B(t) f(t)\, dt,$$

其中 $I_B(t)$ 是指示函数：当值 t 属于集合 B 时输出 1，否则输出 0。注意 $\int_{-\infty}^{+\infty} f(t)\, dt = 1$，$f(t)$ 非负，$f(t)$ 近似给出点 t 处的概率，且

$$P(a \leq X \leq b) = P(a < X \leq b) = P(a \leq X < b) = P(a < X < b),$$

即端点不影响最终的概率测度。最后，表 3.3 汇总了几个常用 PDF，包括均匀、高斯与指数随机变量。注意：高斯随机变量常用于建模通信信道引入噪声的随机性；指数随机变量则用于介质访问协议——当两个以上无线系统试图基于竞争框架经同一无线信道通信时，帮助提供随机退避时间。

表 3.3: 若干常用的概率密度函数（图形表示从略，见原书）

随机变量	PDF 定义
均匀	$f(x) = \begin{cases} \frac{1}{(b-a)}, & a \leq x \leq b \\ 0, & \text{otherwise} \end{cases}$
指数	$f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$
拉普拉斯	$f(x) = \frac{\lambda}{2} e^{-\lambda \lvert x \rvert}$
柯西	$f(x) = \frac{\lambda/\pi}{\lambda^2 + x^2}, \quad \lambda > 0$
高斯	$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-0.5((x-\mu)/\sigma)^2}$

与单个离散随机变量的期望类似，PDF 为 $f(x)$ 的连续随机变量 X 的期望可用下式计算：

$$E[g(X)] = \int_{-\infty}^{+\infty} g(x) f(x)\, dx,$$

其中 $g(\cdot)$ 是作用于随机变量 X 的某实函数。

许多实际关心的随机变量并不独立：一个事件的结果常依赖或受另一个事件结果的影响。因此有时需要计算条件概率与条件期望，尤其是处理涉及多于一个随机变量的问题时。

与离散随机变量的条件概率不同，连续随机变量的条件概率需要另行定义，因为单个精确点发生的概率为零，即 $P(X = x) = 0$。于是，若直接套用 (3.9) 的离散定义

$$P(Y \in C \,|\, X = x) = \frac{P(Y \in C,\, X = x)}{P(X = x)},$$

会出现 $P(X = x) = 0$ 导致除零的情形。因此需要确定另一个在连续随机变量框架内可行的条件概率（与条件期望）定义。

可以证明，计算条件概率必须采用**条件密度** [1]，定义为

$$f_{Y|X}(y|x) = \frac{f_{XY}(x, y)}{f_X(x)},$$

其中 $f_X(x) > 0$。借助条件密度，现在可以毫无除零之忧地计算条件概率：

$$P(Y \in C \,|\, X = x) = \int_C f_{Y|X}(y|x)\, dy.$$

进而，全概率定律定义为

$$P(Y \in C) = \int_{-\infty}^{+\infty} P(Y \in C \,|\, X = x) f_X(x)\, dx,$$

即先把所有条件概率用 X 的 PDF 加权、再积分到一起，构成事件 $Y \in C$ 的总概率。最后，与离散情形一样，处理条件概率时连续随机变量也有替换律，定义为

$$P((X, Y) \in A \,|\, X = x) = P((x, Y) \in A \,|\, X = x).$$

注意若 X 与 Y 独立，联合密度可分解，条件密度变为

$$f_{Y|X}(y|x) = \frac{f_{XY}(x, y)}{f_X(x)} = \frac{f_X(x) f_Y(y)}{f_X(x)} = f_Y(y),$$

这意味着两个随机变量独立时，不需要用一个事件给另一个事件加条件。

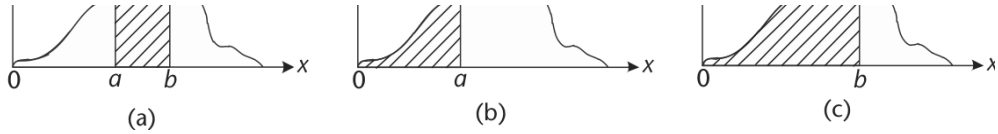


图 3.4　用 CDF 获得尾部概率 P(X < a) 与 P(X < b) 以快速计算 P(a ≤ X < b) 的例子：(a) 需要积分以得到 P(a ≤ X < b) 的 PDF 区域；(b) 得到 P(X < a) 的区域；(c) 得到 P(X < b) 的区域

类似地，连续随机变量的条件期望用条件密度定义为

$$E[g(Y) \mid X = x] = \int_{-\infty}^{+\infty} g(y) f_{Y|X}(y|x)\, dy.$$

进而，条件期望的全概率定律为

$$E[g(X,Y)] = \int_{-\infty}^{+\infty} E[g(X,Y) \mid X = x] f_X(x)\, dx,$$

替换律定义为

$$E[g(X,Y) \mid X = x] = E[g(x,Y) \mid X = x].$$

3.3.1 累积分布函数

对建模通信系统随机元素的随机变量的 PDF 与 PMF，有时可视化**累积分布函数**（CDF）很重要——它提供了随机变量概率行为的另一视角。而且 CDF 有时能用来解一些用其他定义很难下手的问题。

数学上，CDF 定义为

$$F_X(x) = P(X \le x) = \int_{-\infty}^{x} f(t)\, dt,$$

它描述由随机变量 X 刻画的实验结果小于等于哑变量 x 的概率。

举个例子：假设要用图 3.4(a) 所示的 PDF 计算概率 $P(a \le X < b)$。快速求值的一种途径是利用该分布的**尾部概率**：$P(X < a)$（图 3.4(b)）与 $P(X < b)$（图 3.4(c)）。注意尾部概率实际上就是 (3.33) 定义的 X 的 CDF：$F_X(a) = P(X < a)$、$F_X(b) = P(X < b)$。于是，既然我们只关心两个尾部概率不相交的 PDF 区域，可以计算

$$P(a \le X < b) = P(X < b) - P(X < a) = F_X(b) - F_X(a),$$

所需的只是 X 的 CDF 在 $x = a$ 与 $x = b$ 处的值。

CDF 的几个基本特性包括：$F_X(x)$ 有界于 0 与 1 之间；$F_X(x)$ 是非降函数，即若 $x_1 \leq x_2$ 则 $F_X(x_1) \leq F_X(x_2)$。此外，PDF 是 CDF 对哑变量 x 的导数：

$$f_X(x) = \frac{d}{dx} F_X(x).$$

> **Q** 函数是表达高斯随机变量右尾概率 $P(X > x)$ 的便捷方式。数学上，这等价于求 X 的补 CDF [2]：
>
> $$Q(x) = 1 - F_X(x) = 1 - P(X \leq x) = P(X > x) = \int_x^{\infty} \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\, dt,$$
>
> 其中 $F_X(x)$ 是 X 的 CDF。

CDF 的一个重要用途是精确定义通信系统中随机元素的概率本质。噪声生成、信息源的二元输出、多用户对无线信道的随机接入，都可以用 CDF 精确刻画。因此在计算机仿真中建模这些现象时，我们使用由这些 CDF 之一定义的 RNG。MATLAB 仿真环境中有多种可用于通信系统实验的 RNG，包括基于均匀、高斯（正态）与瑞利随机变量的。这些随机变量在 MATLAB 中可通过 `rand` 与 `randn` 函数生成，也可组合它们创建其他随机变量。例如，代码 3.3 的 MATLAB 代码产生三个随机值向量，其统计特性分别等效于均匀、高斯与瑞利随机变量。而且，用这些 RNG 生成的随机值可以确定概率密度，进而生成累积分布函数，并计算这些随机变量产生 0.7 到 1.0 之间值的概率。

在 MATLAB 中获得均匀与高斯分布的值，直接用 `rand` 与 `randn` 函数即可。生成非常大量的这些随机值，就能观察到均匀与高斯 PDF，如图 3.5(a) 与 (c) 所示。由于 CDF 是 PDF 从 $-\infty$ 到 $+\infty$ 的渐进累积，它们也可以从 PDF 数据直接生成，如图 3.5(b) 与 (d) 所示。至于瑞利随机变量产生的值，快速产生方法是取两个独立同分布（i.i.d.）高斯随机变量，两者平方、相加、再开方。由此操作并生成大量值，即可得到瑞利随机变量的 PDF 与 CDF，如图 3.5(e) 与 (f) 所示。注意若想求这些函数产生的随机值落在 0.7 到 1.0 之间的概率，只需把这个范围内的密度值加起来，或取 CDF 在两端点的值相减。

代码 3.3　信息源：random_example.m

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Random Variable PDFs and CDFs

% Define simulation parameters
L = 1000000;                % Length of random samples
```

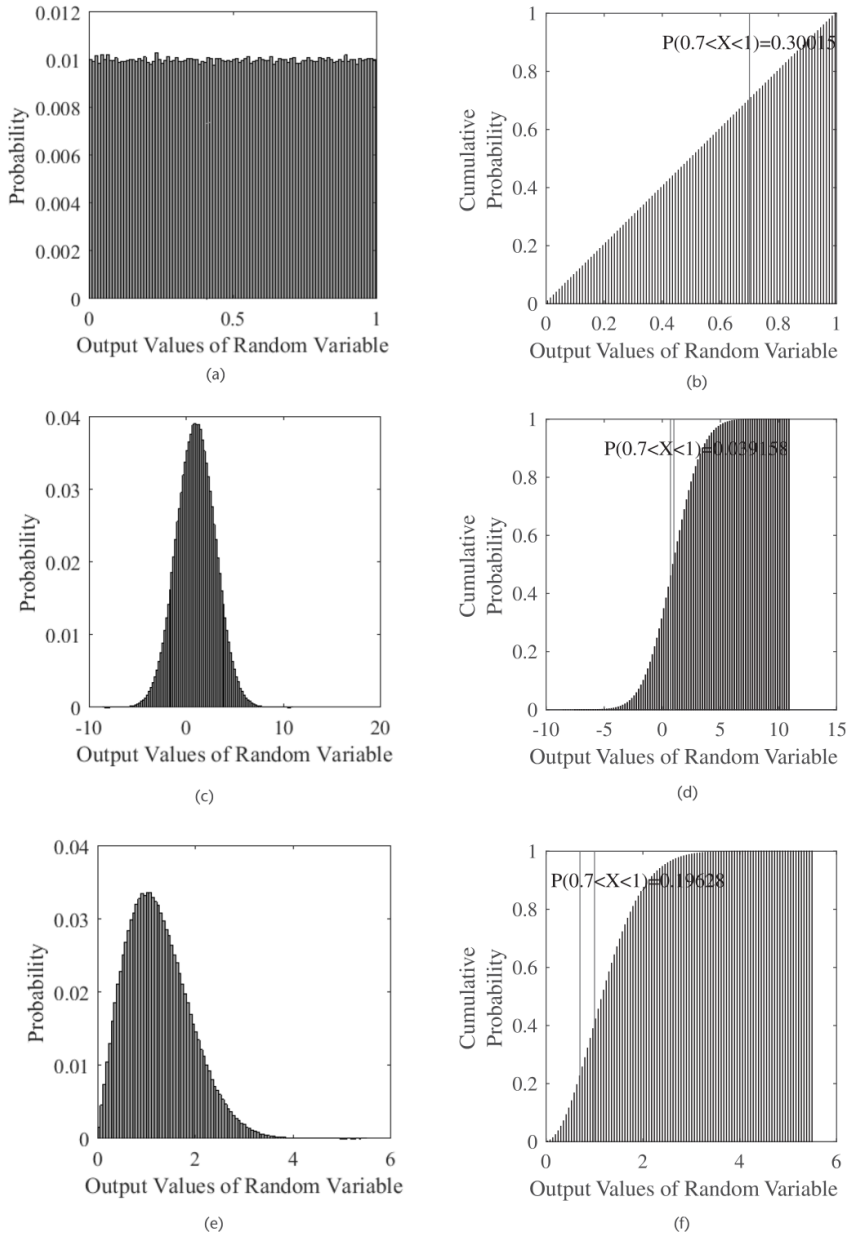


图 3.5　多种累积分布函数及相应概率密度函数：(a) 均匀 PDF；(b) 均匀 CDF；(c) 高斯 PDF；(d) 高斯 CDF；(e) 瑞利 PDF；(f) 瑞利 CDF

```
mean_normal = 1; stddev_normal = 2;
                            % Mean and standard deviation of normal RV values
res_hist = 100;             % Histogram resolution

% Generate random samples for different distributions
b_unif = rand(1,L);         % Uniform random values
b_normal = (stddev_normal*randn(1,L) + mean_normal);
                            % Normal random values
b_rayleigh = (sqrt(randn(1,L).^2 + randn(1,L).^2));
                            % Rayleigh random values

% Obtain cumulative distribution functions
[N_unif, edges_unif] = histcounts(b_unif,res_hist);
N_unif_cum = cumsum(N_unif)./L;
[N_normal, edges_normal] = histcounts(b_normal,res_hist);
N_normal_cum = cumsum(N_normal)./L;
[N_rayl, edges_rayl] = histcounts(b_rayleigh,res_hist);
N_rayl_cum = cumsum(N_rayl)./L;

% Calculate probability of values between 0.7 and 1
x_lower = 0.7;
x_upper = 1.0;
unif_ind_range = find((x_lower <= edges_unif)
                      & (edges_unif < x_upper));
normal_ind_range = find((x_lower <= edges_normal)
                        & (edges_normal < x_upper));
rayl_ind_range = find((x_lower <= edges_rayl)
                      & (edges_rayl < x_upper));
prob_unif = sum(N_unif(unif_ind_range))./L;
prob_normal = sum(N_normal(normal_ind_range))./L;
prob_rayl = sum(N_rayl(rayl_ind_range))./L;
```

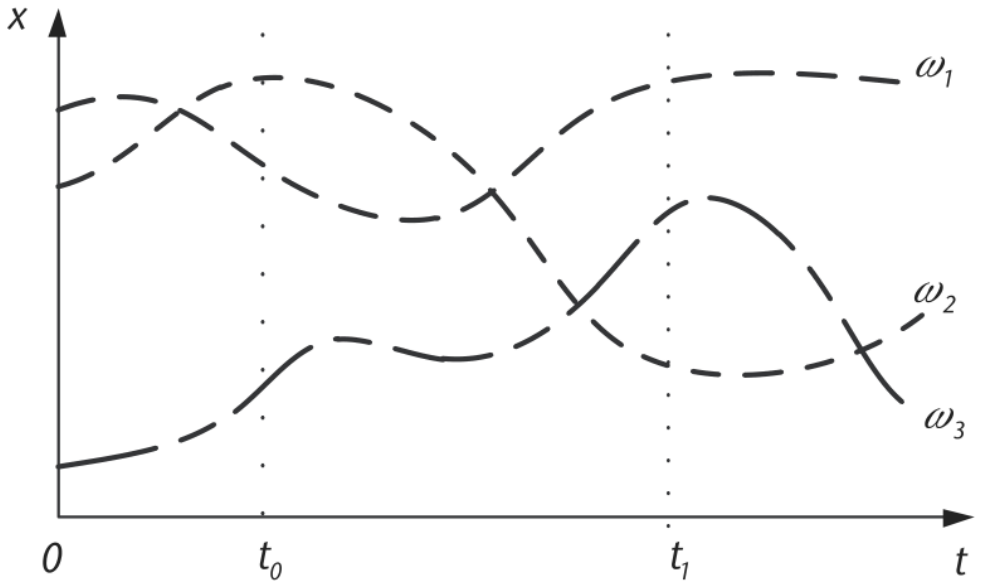


图 3.6　随机过程 X(t, ω) 的示意图

3.4 通信系统中的时变随机性

到目前为止，我们探讨的都是概率特性不随时间变化的随机现象的建模。虽然这简化了这些模型的数学推导，但未必能准确描述随机现象。例如，信息源的二元输出值可能随时间变化，取决于被编码的真实数据——比如动态城市环境的安防摄像头画面，或计算机上人类用户的互联网流量。因此，有必要为这些同样随时间变化的随机现象发展更全面的数学表示。我们称这些表示为**随机过程**（random process）或**随机过程**（stochastic process）。随机过程是依赖于参数 t 与 ω 的一族时域函数，即可以用函数

$$X(t) = X(t, \omega)$$

定义随机过程，其中左边是随机过程的简写表示，隐含假设存在从样本空间 Ω 的结果 ω 到实数值的映射。注意 ω 的定义域是 Ω，而 t 的定义域对连续时间随机过程是 $\mathbb{R}$、对离散时间随机过程是 $\mathbb{Z}$。图 3.6 描绘了随机过程如何由一族时域函数构成。

设随机过程在每个时刻 t 都是不可数无穷的。此情形下可以定义它的一阶分布 $F(x,t)$ 与一阶密度 $f(x,t)$：

$$F(x,t) = P(X(t) \leq x)$$

与

$$f(x,t) = \frac{\partial F(x,t)}{\partial x}.$$

要确定随机过程的统计性质，需要 $F(x_1,\ldots,x_n;t_1,\ldots,t_n)$ 的知识。不过在大多数通信系统应用中，实际需要的只是某些平均量。例如，随机过程最常用的统计刻画之一是**均值函数**：随机过程 $X(t,\omega)$ 的均值函数 $\mu_X(t)$ 是随机过程在特定时刻 t 的期望，数学表示为

$$\mu_X(t) = E[X(t,\omega)].$$

随机过程 $X(t,\omega)$ 的另一个有用统计刻画工具是**自相关函数** $R_{XX}(t_1,t_2)$：评估随机过程在两个不同时刻 t_1 与 t_2 的相关程度。数学定义为

$$R_{XX}(t_1,t_2) = E[X(t_1,\omega)X^*(t_2,\omega)] = \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} x_1 x_2^* f(x_1,x_2;t_1,t_2)\,dx_1 dx_2.$$

注意 $R_{XX}(t_1,t_2)$ 对角线上的值是 $X(t,\omega)$ 的平均功率：

$$E[|X(t,\omega)|^2] = R_{XX}(t,t).$$

关于自相关函数的若干其他有用性质与观察包括：

1. 由于 $R_{XX}(t_1,t_2) = E[X(t_1,\omega)X^*(t_2,\omega)]$，故 $R_{XX}(t_2,t_1) = E[X(t_2,\omega)X^*(t_1,\omega)] = R_{XX}^*(t_1,t_2)$。
2. $R_{XX}(t,t) = E[|X(t,\omega)|^2] \geq 0$。
3. 若对所有 t 有 $E[|X(t,\omega)|^2] < \infty$，则称之为二阶过程。
4. 由 $R_{XX}(t,t) = E[|X(t,\omega)|^2] \geq 0$ 及给定时刻 t_1、t_2，有不等式 $|R_{XX}(t_1,t_2)| \leq \sqrt{E[|X(t_1,\omega)|^2]E[|X(t_2,\omega)|^2]}$。
5. 当 $X(t,\omega)/\sqrt{C_{XX}(t,t)}$ 时得到归一化过程。

自相关函数定义的一个推广是 $X(t,\omega)$ 的**自协方差函数** $C_{XX}(t_1,t_2)$，即随机过程在时刻 t_1 与 t_2 的协方差。数学表示为

$$C_{XX}(t_1,t_2) = R_{XX}(t_1,t_2) - \mu_X(t_1)\mu_X^*(t_2).$$

注意 $t_1 = t_2$ 时自协方差函数给出 $X(t,\omega)$ 的方差。此外，有时可以用归一化度量表示随机过程的自协方差函数，称为**相关系数**：

$$\rho_{XX}(t_1,t_2)=\frac{C_{XX}(t_1,t_2)}{\sqrt{C_{XX}(t_1,t_1)C_{XX}(t_2,t_2)}}.$$

3.4.1 平稳性

虽然随机过程可能随时间呈现显著的变化性，但存在一类在任何两个时刻表现相同的随机过程子集——即时不变的随机过程。我们称这类随机过程为**平稳过程**。两种常见形式是严平稳（SSS）与宽平稳（WSS）随机过程。只要随机过程的统计性质对原点平移不变，它就是 SSS 的：即对任意时移 c，随机过程 $X(t,\omega)$ 与 $X(t+c,\omega)$ 具有相同的统计量。因此由定义，SSS 随机过程的 n 阶密度满足

$$f(x_1,\ldots,x_n;t_1,\ldots,t_n)=f(x_1,\ldots,x_n;t_1+c,\ldots,t_n+c)$$

对任意时移 c 成立。

由此可得 $f(x;t)=f(x;t+c)$ 对任意 c 成立，即 $X(t,\omega)$ 的一阶密度与时间 t 无关：

$$f(x;t)=f(x).$$

进而，$f(x_1,x_2;t_1,t_2)=f(x_1,x_2;t_1+c,t_2+c)$ 对任意 c 与 c 无关，于是密度变为

$$f(x_1,x_2;t_1,t_2)=f(x_1,x_2;\tau),\quad \text{其中}\tau=t_1-t_2.$$

于是随机过程在时刻 t 与 $t+\tau$ 的联合密度与 t 无关，等于 $f(x_1,x_2;\tau)$。

虽然 SSS 随机过程凭借其有用的时不变性质可以给出数学上易于处理的解，但实际通信系统中 SSS 的出现并不频繁。相反，WSS 随机过程在通信系统分析中更常见。只要以下两条性质同时成立，随机过程 $X(t,\omega)$ 就是 WSS 的：

- 均值函数 $\mu_X(t)$ 不依赖时间 t，即 $\mu_X(t)=E[X(t,\omega)]=\mu_X$；
- 自相关函数 $R_{XX}(t+\tau,t)$ 只依赖 t 与 $t+\tau$ 的相对差，即 $R_{XX}(t+\tau,t)=R_{XX}(\tau)$。

关于 WSS 随机过程的若干观察包括：

- WSS 随机过程的平均功率与时间无关，因为 $E[|X(t,\omega)|^2]=R_{XX}(0)$。
- WSS 随机过程的自协方差函数为 $C_{XX}(\tau)=R_{XX}(\tau)-|\mu_X|^2$。
- WSS 随机过程的相关系数为 $\rho_{XX}(\tau)=C_{XX}(\tau)/C_{XX}(0)$。
- 两个随机过程 $X(t,\omega)$ 与 $Y(t,\omega)$ 联合宽平稳的条件：各自 WSS 且互相关只依赖 $\tau=t_1-t_2$。
- 若随机过程 $X(t,\omega)$ 是 WSS 且不相关的，则 $C_{XX}(\tau)=q\delta(\tau)$，$q$ 为某乘性常数。

无线数据传输中还常出现另一种平稳性特征。**循环平稳**随机过程 $Y(t)$ 定义为：均值函数 $\mu_Y(t)$ 随时间 t 周期，且对固定 τ 值自相关函数 $R_{YY}(\tau+\theta,\theta)$ 随 θ 周期。于是周期为 T_0 的循环平稳随机过程可用下式描述：

$$\overline{R}_{YY}(\tau)=\frac{1}{T_0}\int_0^{T_0}R_{XX}(\tau+\theta,\theta)\,d\theta.$$

在通信系统工程领域，循环平稳随机过程常被用于噪声信道环境中的无线信号检测：假设同一时空-频域区域内存在多个信号，目标无线信号会产生独特的特征函数，使识别成为可能。

3.5 高斯噪声信道

如前所述，高斯（即正态）随机变量常用于建模通信信道内引入的噪声。事实上，许多通信系统及其性能的分析常常假设噪声信道具有高斯随机行为。因此高斯随机变量是通信系统研究中最常用的随机变量之一。

单变量高斯 PDF 定义为

$$f_X(x)=\frac{1}{\sqrt{2\pi\sigma^2}}e^{-(x-\mu)^2/2\sigma^2},$$

其中 μ 是高斯随机变量 X 的均值，σ^2 是 X 的方差。当 $\mu=0$ 且 $\sigma^2=1$ 时，称 X 为标准正态随机变量。

虽然单变量高斯分布用于众多应用，但有些情形必须采用**二维高斯分布**来刻画涉及两个彼此存在某种相关性的高斯随机变量的应用，例如具有同相与正交信号分量的复基带传输信道。图 3.7 给出了一个二维高斯分布的示例。

数学上，参数为 μ_X、μ_Y、σ_X^2、σ_Y^2 与相关系数 ρ 的二维高斯密度的一般定义为

$$f_{XY}(x,y)=\frac{\exp\left\{-\frac{1}{2(1-\rho^2)}\left[\left(\frac{x-\mu_X}{\sigma_X}\right)^2-2\rho\left(\frac{x-\mu_X}{\sigma_X}\right)\left(\frac{y-\mu_Y}{\sigma_Y}\right)+\left(\frac{y-\mu_Y}{\sigma_Y}\right)^2\right]\right\}}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho^2}},$$

其中相关系数定义为

$$\rho=E\left[\frac{x-\mu_X}{\sigma_X}\cdot\frac{y-\mu_Y}{\sigma_Y}\right].$$

假设要建模一个复基带信道，其中同相与正交噪声分量由二维高斯随机变量表示。

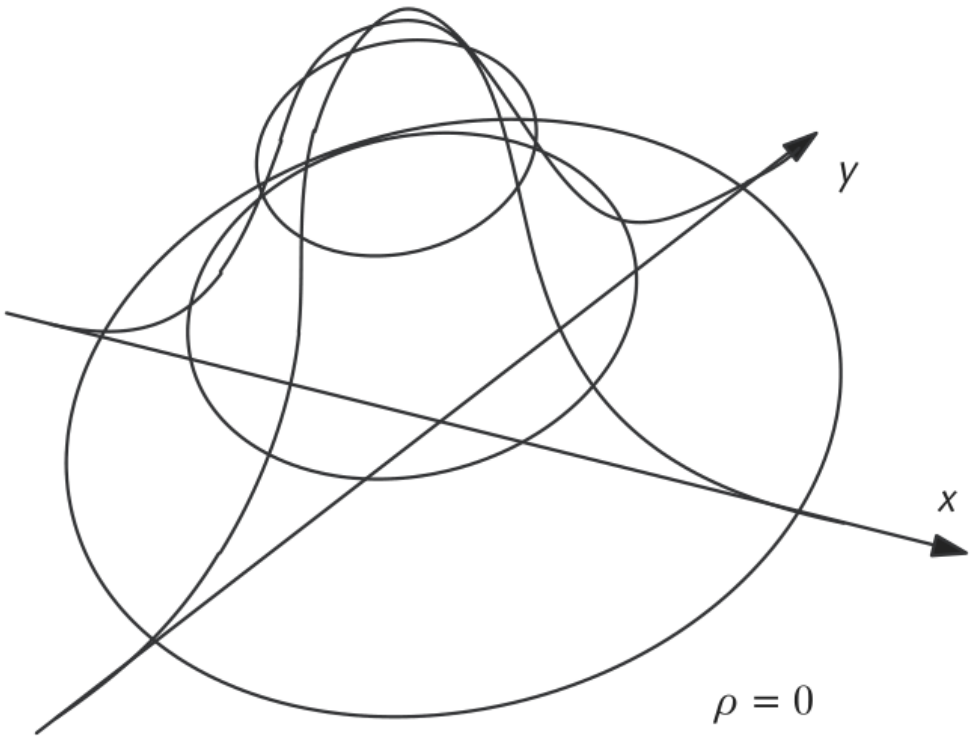


图 3.7　无相关（ρ = 0）的二维正态分布密度

有些情形我们希望噪声的同相与正交分量**不相关**，另一些情形则希望它们高度相关。因此必须确保准确建模这些二维高斯随机值。代码 3.4 的 MATLAB 脚本建模了代表信道噪声的这类二维高斯随机变量：用 randn 函数生成长度 L 的两个含高斯特性的向量。由于这两个向量是分别生成的，默认情形下它们互不相关。两个向量都是零均值、标准差均为 std_dev。由这两个向量可以方便地得到不相关的与相关的二维高斯分布数据。

代码 3.4　信息源：random_example.m

```
% Define simulation parameters
L = 1000000;        % Length of data streams
res_hist = 100;     % Histogram resolution
std_dev = 5;        % Standard deviation of input Gaussian variables

% Create uncorrelated 2D Gaussian random variable
x_normal_1 = std_dev.*randn(1,L);
y_normal_1 = std_dev.*randn(1,L);

% Create correlated 2D Gaussian random data stream
x_normal_2 = x_normal_1 + 0.1*y_normal_1;
y_normal_2 = y_normal_1 + 0.9*x_normal_1;
```

从图 3.8 的二维密度函数可以方便地观察不相关与相关二维高斯随机变量的差别。不相关情形下，二维高斯随机变量关于 x/y 平面对称，见图 3.8(a)（三维视角）与图 3.8(b)（俯视视角）。而一旦在二维高斯随机变量的同相与正交分量间引入相关性，密度形状开始扭曲：一个方向被压缩、另一方向被扩张，见图 3.8(c)（三维视角）与图 3.8(d)（俯视视角）。

3.5.1 高斯过程

如前所述，存在这样的情形：表示通信系统某元素的随机现象，其概率特性随时间变化（例如由高斯随机变量表示的噪声信道特性）。于是需要一种能刻画这些现象时间依赖随机性的数学表示，尤其是对高斯随机变量建模的事件。参照 3.4 节，可以用**高斯过程**建模时变的高斯随机变量：高斯过程是这样的随机过程——它的实现由一系列时间（或空间）范围内每一点关联的随机值组成，且每个这样的随机变量都服从正态分布。此外，这些随机变量的每个有限集合都具有多变量正态分布。

高斯过程在统计建模中很重要，因为它继承了正态分布的性质。例如，若随机过程

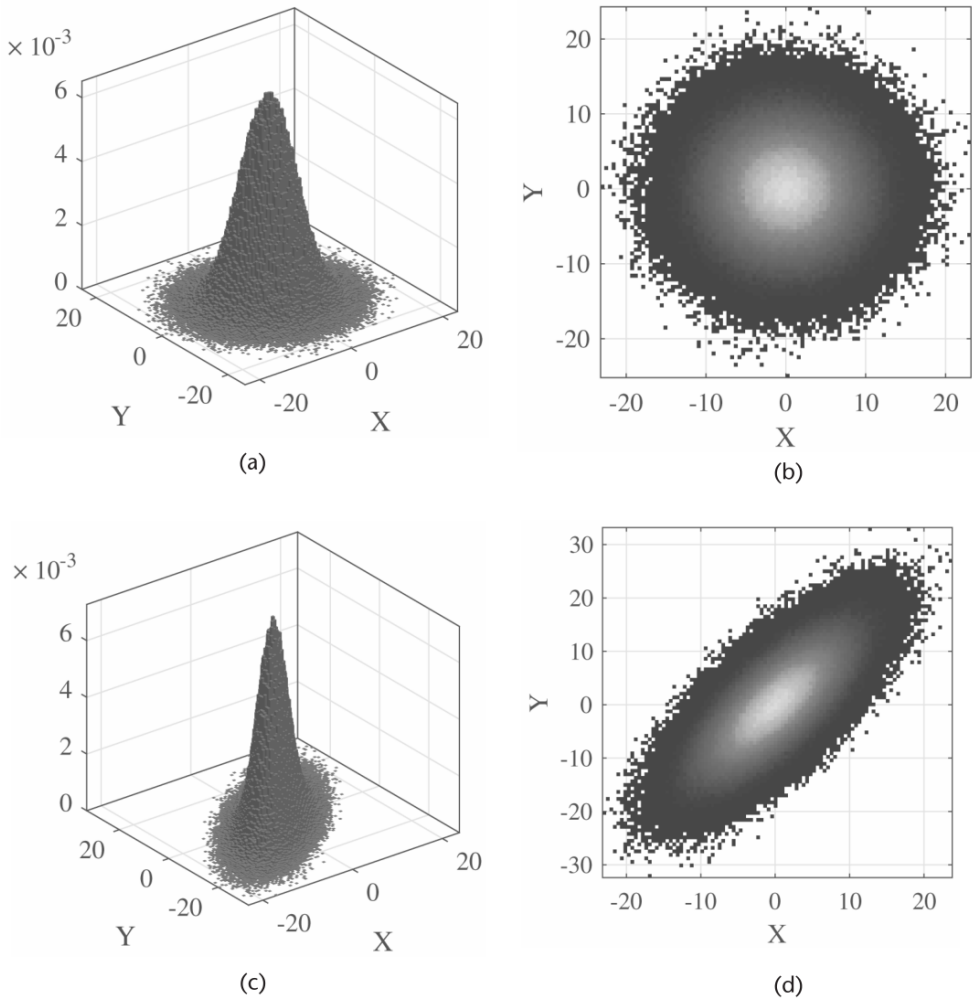


图 3.8　相关与不相关二维高斯密度的若干示例：(a) 不相关二维高斯（三维视图）；(b) 不相关二维高斯（俯视图）；(c) 相关二维高斯（三维视图）；(d) 相关二维高斯（俯视图）

被建模为高斯过程，各导出量的分布都可以显式获得。这样的量包括过程在一段时间范围内的平均值、以及用少量时刻的样本值估计该平均值的误差。

给定表达式

$$y = \int_0^T g(t)X(t)\,dt$$

若满足：

- $E(y^2)$ 有限（即不发散）；
- 对每个 $g(t)$，Y 都是高斯分布的，

则可以说 $X(t)$ 是高斯过程。注意随机变量 Y 服从高斯分布，其 PDF 为

$$f_Y(y) = \frac{1}{\sqrt{2\pi\sigma_Y^2}} e^{-(y-\mu_Y)^2/2\sigma_Y^2},$$

其中 μ_Y 是均值、σ_Y^2 是方差。这类过程很重要，因为它们与大量物理现象的行为高度吻合，例如加性高斯白噪声（AWGN）。

> 思考题：为什么一个不相关的随机过程被称为”白”的，就像加性高斯白噪声的情形那样？

3.6 功率谱密度与 LTI 系统

要在频域分析通信系统，常用**功率谱密度**（PSD）$S_{XX}(f)$ 刻画信号，它由 WSS 随机过程 $X(t)$ 的自相关 $R_{XX}(\tau)$ 取傅里叶变换得到。PSD 与自相关函数通过爱因斯坦–维纳–辛钦（EWK）关系在数学上相联系：

$$S_{XX}(f) = \int_{-\infty}^{\infty} R_{XX}(\tau)e^{-j2\pi f\tau}\,d\tau$$

$$R_{XX}(f) = \int_{-\infty}^{\infty} S_{XX}(\tau)e^{+j2\pi f\tau}\,d\tau$$

EWK 关系的一个威力巨大的推论是：当要确定”输入也是 WSS 随机过程的 LTI 系统的输出”这一 WSS 随机过程的自相关函数或 PSD 时特别有用。具体地，设 $H(f)$ 为 LTI 系统 $h(t)$ 的频率响应，则输入输出随机过程的功率谱密度由下式联系：

$$S_{YY}(f) = |H(f)|^2 S_{XX}(f),$$

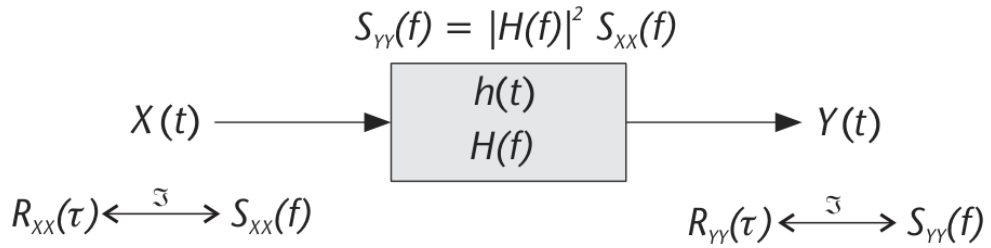


图 3.9　LTI 系统 h(t) 如何在 WSS 随机过程输入 X(t) 与 WSS 随机过程输出 Y(t) 之间变换 PSD 的示例

其中 $S_{XX}(f)$ 是输入随机过程的 PSD，$S_{YY}(f)$ 是输出随机过程的 PSD。这一非常有用的关系示于图 3.9。

要理解 LTI 系统对随机过程 PSD 的影响，可以用代码 3.5 的 MATLAB 脚本：用二元均匀随机数发生器产生同相与正交值。为突出 LTI 系统对 PSD 的影响，用 `firls` 函数设计了两个滤波器：一个通带相对较大，另一个通带相对较小。二元均匀随机值被两个系统分别滤波，结果 PSD 见图 3.10。

原始二元均匀随机值的三维与俯视视角示于图 3.10(a) 与 (b)。观察到密度几乎呈矩形块状——这正是我们对二元均匀分布的预期。用窄通带滤波器过滤该数据后观察 PSD，可以清楚看到滤波操作的效果：PSD 的大部分在外围被滤除，见图 3.10(c) 与 (d)。而用通带相对较大的滤波器时，PSD 的外围没有被明显滤除，见图 3.10(e) 与 (f)。本例的要点是：LTI 系统的滤波操作会对随机过程的 PSD 产生显著影响。

代码 3.5　信息源：random_example.m

```
% Define simulation parameters
L = 1000000;              % Length of data streams
res_hist = 100;           % Histogram resolution
cutoff_freq1 = 0.2;       % Small passband LPF
cutoff_freq2 = 0.7;       % Large passband LPF
filt_coeffs1 = firls(13,[0 cutoff_freq1 cutoff_freq1+0.02 1],[1 1 0 0]);
filt_coeffs2 = firls(13,[0 cutoff_freq2 cutoff_freq2+0.02 1],[1 1 0 0]);

% Create input 2D Gaussian random variable
x_in = rand(1,L);
y_in = rand(1,L);
```

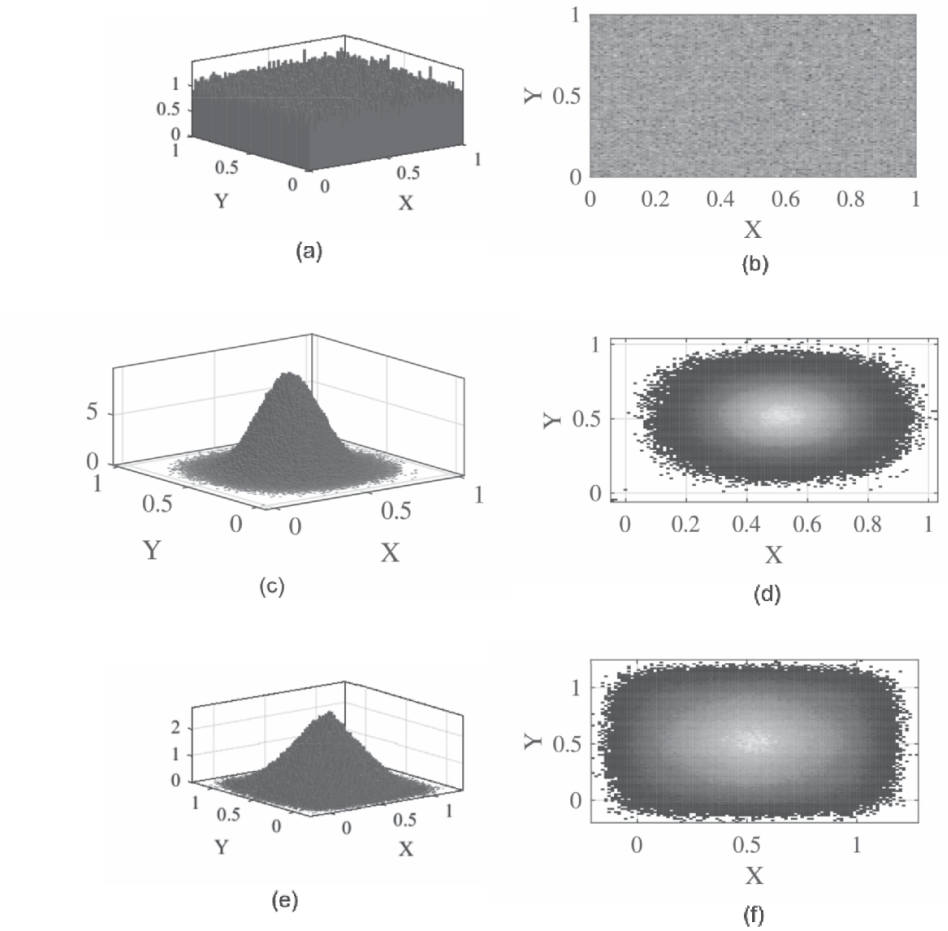


图 3.10　滤波如何影响随机过程输出功率谱密度的示例：(a) 输入信号 PSD（三维视图）；(b) 输入信号 PSD（俯视图）；(c) 滤波器系数 1 的输出 PSD（三维视图）；(d) 滤波器系数 1 的输出 PSD（俯视图）；(e) 滤波器系数 2 的输出 PSD（三维视图）；(f) 滤波器系数 2 的输出 PSD（俯视图）

```
% Filter input random data stream
filt_output1 = filter(filt_coeffs1,1,(x_in+1j.*y_in));
x_out1 = real(filt_output1);
y_out1 = imag(filt_output1);
filt_output2 = filter(filt_coeffs2,1,(x_in+1j.*y_in));
x_out2 = real(filt_output2);
y_out2 = imag(filt_output2);
```

3.7 窄带噪声

有了随机变量与随机过程的坚实基础，现在可以把这些知识应用于无线通信系统的窄带传输。一般而言，多数传输都被设计为带限的，因为任何一次传输可用的无线频谱量都有约束。这些约束是必要的：可用无线频谱有限，而寻求用这些频谱传输的无线应用与用户却在不断增长。

窄带通信系统的关键要素之一是收发两端的窄带滤波器，设计为只让已调信号通过。然而这些窄带滤波器也会让接收机截获的一部分噪声通过，因为很难把噪声与已调信号分开。如果噪声恰是白的（即不相关），那么窄带噪声将呈现余弦调制的带通滤波器响应形状。原因在于：滤波前的白噪声 PSD 是平坦的、铺满从 $-\infty$ 到 $+\infty$ 的整个频率范围；经带通滤波器处理后，得到的窄带噪声 PSD 呈现带通滤波器幅度响应平方的形状——因为时域噪声与滤波器的卷积对应为噪声 PSD 与滤波器幅度响应的乘积（见 3.6 节）。

为建立表示窄带噪声的便捷数学框架，有两条途径：**同相/正交表示**与**包络/相位表示**。两者都可以用定义 $x = Ae^{j\varphi} = a + jb$ 描述复数值 x（$x \in \mathbb{C}$），其中包络 A、相位 φ、同相分量 a 与正交分量 b 均为实数（$A, \varphi, a, b \in \mathbb{R}$）。两种表示之间的关系为：

$$A = \sqrt{a^2 + b^2}, \qquad \varphi = \tan^{-1}(b/a)$$

$$a = A\cos(\varphi), \qquad b = A\sin(\varphi)$$

于是，可以通过下式在复基带域描述窄带噪声的同相/正交表示：

$$\tilde{n}(t) = n_I(t) + jn_Q(t),$$

再表达为带通形式的窄带噪声信号：

$$n(t) = \text{Real}\{\tilde{n}(t)e^{j2\pi f_c t}\}.$$

利用欧拉关系 $e^{j\omega} = \cos(\omega) + j\sin(\omega)$，得到

$$n(t) = n_I(t)\cos(2\pi f_c t) - n_Q(t)\sin(2\pi f_c t).$$

同相/正交表示的若干重要性质见表 3.4。

表 3.4: 同相/正交表示的若干重要性质

性质
$n_I(t)$ 与 $n_Q(t)$ 都是零均值
若 $n(t)$ 是高斯的，则 $n_I(t)$ 与 $n_Q(t)$ 也是
若 $n(t)$ 平稳，则 $n_I(t)$ 与 $n_Q(t)$ 联合平稳
$n_I(t)$ 与 $n_Q(t)$ 的 PSD 相等：$S_{NI}(f) = S_{NQ}(f) = \begin{cases} S_N(f-f_c) + S_N(f+f_c), & -B \le f \le B \\ 0, & \text{otherwise} \end{cases}$
$n_I(t)$ 与 $n_Q(t)$ 的方差都等于 $n(t)$ 的方差
若 $n(t)$ 是高斯的且其 PSD 对称，则 $n_I(t)$ 与 $n_Q(t)$ 统计独立
$n_I(t)$ 与 $n_Q(t)$ 的互谱密度是纯虚的，且在 $-B \le f \le B$ 内等于 $S_{N_I N_Q}(f) = -S_{N_Q N_I}(f) = j(S_N(f+f_c) - S_N(f-f_c))$（其余为零）

类似地，窄带噪声包络/相位表示的复基带版本可写为

$$n(t) = r(t)\cos(2\pi f_c t + \varphi(t))$$

其中 $r(t) = \sqrt{n_I(t) + n_Q(t)}$ 是包络，$\varphi(t) = \tan^{-1}(n_Q(t)/n_I(t))$ 是相位。

关于两种表示的联合分布之间的关系，结果非常精彩。设 $n_I(t)$ 与 $n_Q(t)$ 的联合 PDF 为二维高斯分布：

$$f_{N_I N_Q}(n_I, n_Q) = \frac{1}{2\pi\sigma^2} e^{-(n_I^2+n_Q^2)/2\sigma^2}.$$

结果是：利用关系 $n_I = r\cos(\varphi)$、$n_Q = r\sin(\varphi)$ 以及雅可比行列式，得到如下分布：

$$f_{R\Phi}(r,\varphi) = \begin{cases} \frac{r}{2\pi\sigma^2} e^{-r^2/2\sigma^2}, & r \ge 0 \text{ 且 } 0 \le \varphi \le 2\pi \\ 0, & \text{otherwise} \end{cases}$$

它等价于瑞利 PDF 与均匀 PDF 的组合。

3.8 随机变量的应用：室内信道模型

随机变量用于建模通信系统设计与实现中随机现象的一个绝佳例子，是 Saleh 与 Valenzuela 提出的室内信道模型 [3]。该信道模型的前提是：室内环境会产生多径分量的**簇**（cluster），这些多径分量来自无线信号在周围环境上的反射。

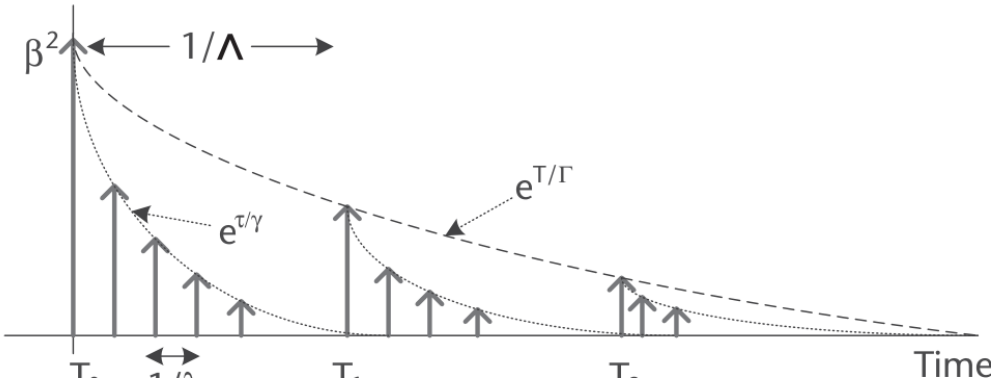


图 3.11　Saleh 与 Valenzuela 统计室内信道模型示意图

数学上，他们用下式描述接收机截获的发射信号反射：

$$h(t) = \sum_{l=0}^{\infty} \sum_{k=0}^{\infty} \beta_{kl} e^{j\theta_{kl}} \delta(t - T_l - \tau_{kl})$$

其中 β_{kl} 是第 l 个多径簇内第 k 条射线的幅度，θ_{kl} 是其相位值，T_l 是第 l 个多径簇起始的时延，τ_{kl} 是第 l 个多径簇内第 k 条射线的时延。在他们的工作中，Saleh 与 Valenzuela 用信道测量把 β_{kl} 刻画为瑞利随机变量、θ_{kl} 刻画为均匀随机变量、T_l 与 τ_{kl} 刻画为到达率分别为 Λ 与 λ 的泊松到达过程。图 3.11 直观展示了该模型。

3.9 本章小结

本章简要介绍了分析与建模通信系统随机变量和随机过程的若干关键数学工具。特别重要的是：读者应理解如何数学地操作高斯随机变量、高斯随机过程与二维正态分布——它们频繁出现在数字通信与无线数据传输应用中。此外，理解平稳性的机理、以及如何把 EWK 关系应用于"LTI 系统过滤随机过程" 的情形，至关重要——尤其在处理通信系统接收机对无线接收信号的加工时。

3.10 延伸阅读

本章虽然试图为读者介绍分析与建模众多数字通信系统中常见随机变量和随机过程所需的若干关键数学工具，但对这些数学工具的处理绝非严格或透彻。因此，鼓励有兴趣的读者查阅更详尽论述这一广阔领域的书籍。例如，Papoulis 与 Pillai 的书是概率与随机过程教科书的标准权威 [1]；而希望在通信网络语境下理解概率与随机过程理论的读

者，可能会发现 Leon-Garcia 的书高度相关 [4]。

对涉及物理层数字通信系统与数字信号处理（如维纳滤波）感兴趣的人，Gubner 的书是不错的选择——该书中许多例子与习题都与通信系统的经典问题相关 [5]。至于有大量例题解答与讲解的书，Hsu [6] 与 Krishnan [7] 的著作是合适的读物。最后，有兴趣研究随机过程高级主题与处理的读者，可参考 Grimmett 与 Stirzaker 的书 [8]。

3.11 参考文献

[1] Papoulis, A., and S. Unnikrishna Pillai, *Probability, Random Variables and Stochastic Processes*, Fourth Edition, New York: McGraw Hill Higher Education, 2002.

[2] Abramowitz, M., and I. Stegun, *Handbook of Mathematical Functions: with Formulas, Graphs, and Mathematical Tables*, New York: Dover Publications, 1965.

[3] Saleh, A. A. M., and R. A. Valenzuela, "A Statistical Model for Indoor Multipath Propagation," *IEEE Journal on Selected Areas in Communications*, Vol. 5, No. 2, 1987, pp. 128–137.

[4] Leon-Garcia, A., *Probability, Statistics, and Random Processes for Electrical Engineering*, Third Edition, Upper Saddle River, NJ: Prentice Hall, 2008.

[5] Gubner, J. A., *Probability and Random Processes for Electrical and Computer Engineers*, Cambridge, MA: Cambridge University Press, 2006.

[6] Hsu, H., *Schaum's Outline of Probability, Random Variables, and Random Processes*, New York: McGraw-Hill, 1996.

[7] Krishnan, V., *Probability and Random Processes*, Hoboken, NJ: Wiley-Interscience, 2006.

[8] Grimmett, G., and D. Stirzaker, *Probability and Random Processes*, Third Edition, Oxford, UK: Oxford University Press, 2001.

第 4 章 数字通信基础

本章概述数字数据传输中若干关键的基础概念。我们先理解二进制数据如何用来操纵电磁波形的物理特性，再考察数字通信系统的基本构成，然后集中讨论用二进制数据调制电磁波形的几种不同方法。建立数字传输的工作方式后，我们将探索评估此类系统定量性能的关键分析工具之一——误码率。最后，本章以信号矢量空间的视角介绍数字接收机的设计。

4.1 什么是数字传输?

数字收发信机是由一组数字与模拟过程组成的系统，这些过程协同工作，完成二进制信息的处理与操纵。这些过程的目的，是实现某种介质上的数据收发——无论是双绞铜线、光纤电缆还是无线环境。任何数字收发信机系统的核心是二进制数字（比特）：就本书而言，它是数字通信系统所使用的信息基本单位。

因此，数字收发信机本质上负责" 比特流" 与" 唯一表示这些比特的物理特性的电磁波形" 之间的转换。由于电磁波形通常用正弦与余弦波描述，每个时间间隔 T 内常用来表示数字数据的电磁波形物理特性包括幅度、相位与载波频率，如图 4.1 所示。注意不同的比特组合表示不同的幅度电平、不同的相位值或不同的载波频率值，每个值唯一地表示特定的二进制图案。还要注意，在某些高级映射体制中，二进制图案可能由两种或更多物理量共同表示。

然而，数字收发信机里发生的事情远不止比特与波形之间的映射，如图 4.2 所示。在这幅数字收发信机基本构成的图示中，我们看到构成通信系统的若干功能块：比特与电磁波形特性之间的映射由调制与解调块表示；此外还有负责从二进制数据中移除冗余信息的信源编码与信源译码块；为保护传输免受潜在差错而引入受控冗余的信道编码与信道译码块；以及负责把基带波形搬移到更高载频的射频前端（RFFE）块。

有人会问：为什么数字通信系统需要这么多块？注意图 4.2 中发射机与接收机之间

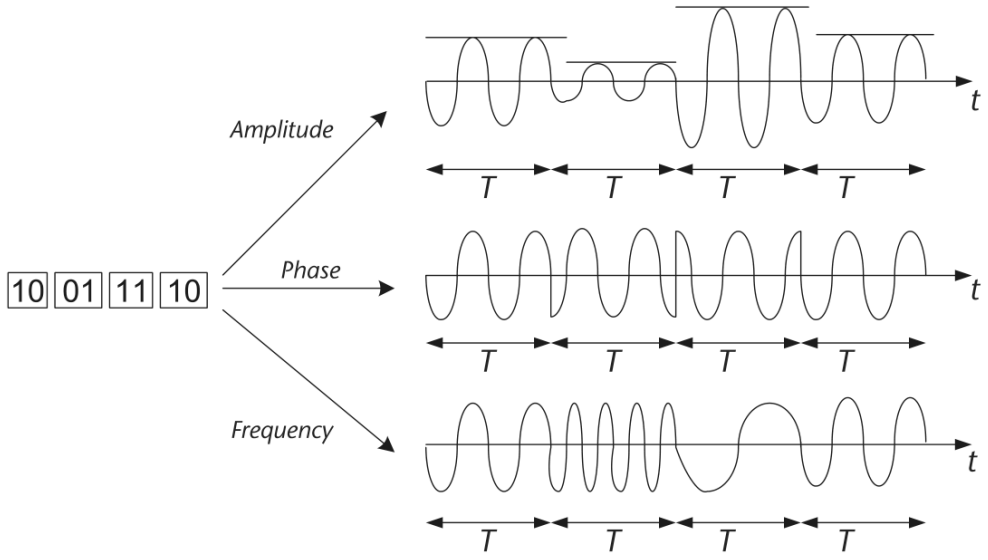


图 4.1　二进制信息到电磁波特性的可能映射

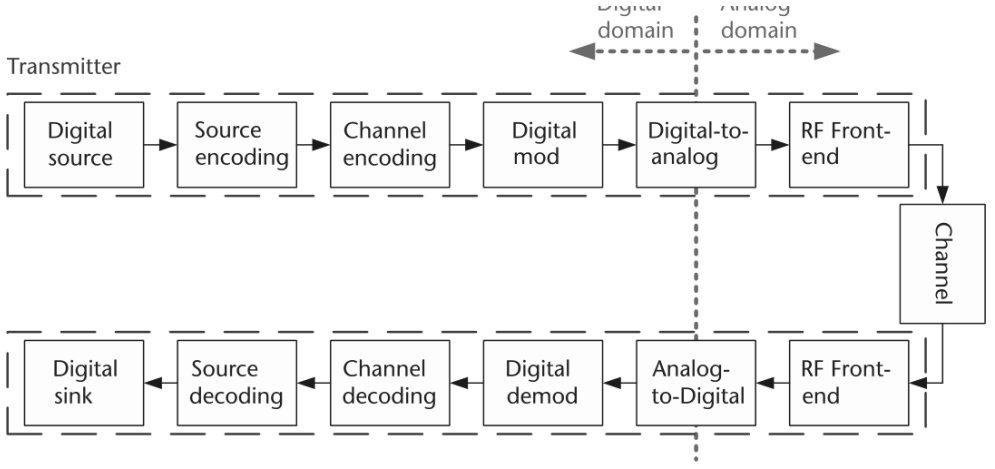


图 4.2　数字通信收发信机的通用表示

存在一个**信道**。数字通信系统设计之所以困难、实现之所以涉及这么多块，主因就是这个信道。如果信道是理想介质——发射机的电磁波形毫无失真与扰动地送达接收机——那么数字通信系统的设计将不值一提。然而现实中，信道会给数字传输引入各种随机损伤，可能影响接收机对波形的正确接收。例如，信道可能引入某种噪声，遮蔽波形的某些特性。而且许多真实场景中，信道引入的这些非理想效应还是时变的，因而难以对付——尤其当它们随时间快速变化时。

于是在真实条件下，任何数字通信系统的首要目标是：发射二进制消息 $m(t)$，使接收机输出端重建的版本 $\hat{m}(t)$ 与之相等。换言之，我们的目标是让 $P(\hat{m}(t) \neq m(t))$ 小到特定应用所需的程度。定量评估数字通信系统差错性能的指标称为差错概率或误码率（BER），定义为 $P_e = P(\hat{m}(t) \neq m(t))$。注意不同数据传输应用的 P_e 要求不同，部分取决于数据传输速率。例如：数字语音传输认为 $P_e \sim 10^{-3}$ 可接受；一般数据传输应用 $P_e \sim 10^{-5}$–10^{-6} 被认为足够；而光纤电缆这类极高速率应用需要 $P_e \sim 10^{-9}$，否则以极高的数据速率，再多差错都会淹没接收机。

为帮助缓解信道损伤可能导致的差错，在进入调制之前，我们先简要研究信源编码与信道编码的工作方式。

动手 MATLAB 示例：通信系统在把电磁信号广播到介质上之前，通过操纵其物理属性来传递信息。幅度、相位和/或频率等信号属性随时间被操纵，使接收机能够解读发射机传达的消息。这些电磁广播常用正弦信号，因而操纵起来相对直接。下面的 MATLAB 脚本生成三个基于正弦波的传输，信息分别通过幅度电平（幅移键控）、相位特性（相移键控）或频率值（频移键控）嵌入其中。脚本用 `rand` 函数生成随机二进制数据、取整到最近整数（0 或 1），再把这些二进制值映射为正弦信号所用的幅度、相位或频率值。由于正弦信号是连续波形，我们用大量离散点逼近连续。

这些随机二进制值到正弦信号物理属性的映射示于图 4.3：可以清楚看到幅度（图 4.3(b)）、相位（图 4.3(c)）与频率（图 4.3(d)）的值如何随时间变化，以表示发往接收机的二进制值（见图 4.3(a)）。三种情形用完全相同的正弦信号作信息载体，只是正弦特性随时间变化。

代码 4.1　用正弦信号操纵发送二进制数据：chapter4.m

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Sending binary data via sinusoidal signal manipulation

% Parameters
sig_len = 1000;       % Signal length (in samples)
```

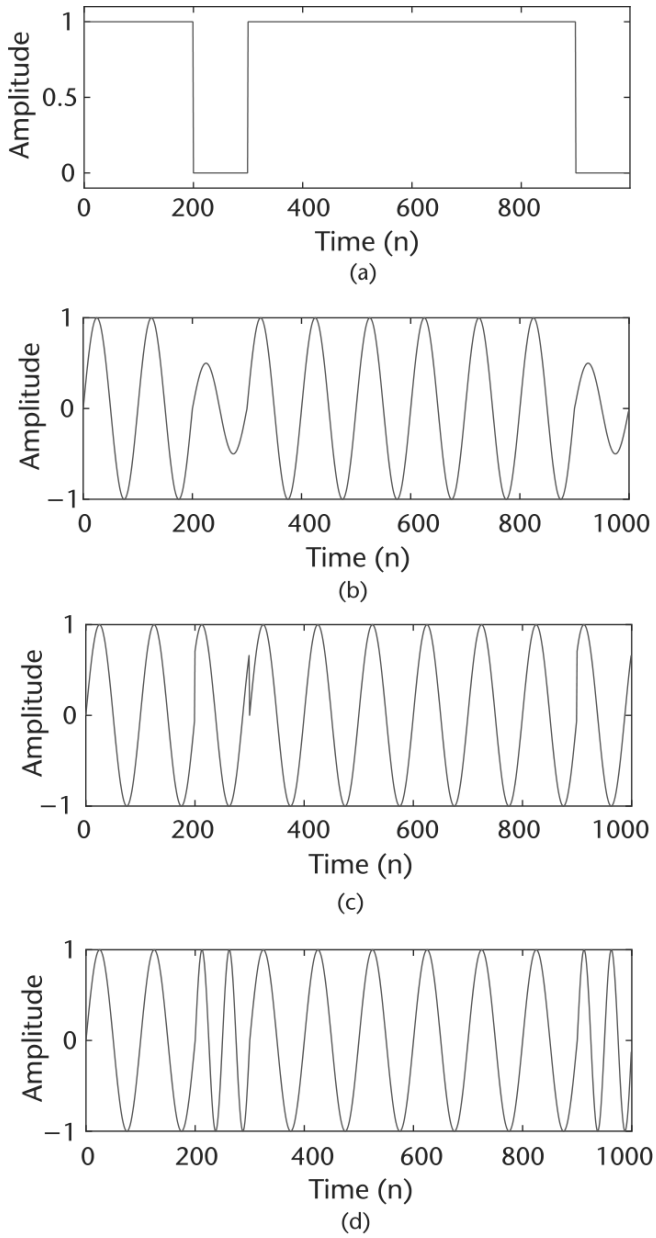


图 4.3　二进制传输的幅度、相位与频率表示示例。这些正弦波特性是数字通信系统常用调制方案的基石：(a) 二进制信号；(b) 幅移键控；(c) 相移键控；(d) 频移键控

```
sampl_per_bin = 100;  % Samples per binary representation
bin_data_len = sig_len/sampl_per_bin;
                      % Length of binary stream is a multiple of signal length
bin_data = round(rand(1,bin_data_len));

% Create sinusoidal carriers
sig_carrier_base = sin(2*pi*(0:(1/sampl_per_bin):
                      (1-(1/sampl_per_bin))));   % Baseline carrier
sig_carrier_freq = sin(2*2*pi*(0:(1/sampl_per_bin):
                      (1-(1/sampl_per_bin))));   % Double frequency
sig_carrier_phase = sin(2*pi*(0:(1/sampl_per_bin):
                      (1-(1/sampl_per_bin)))+(pi/4)); % Phase shifted 45 degrees

% Modulate sinusoidal carrier via amplitude, phase, and frequency
% manipulations
sig_bin = [];  % Binary waveform
sig_ask = [];  % Amplitude modulated
sig_psk = [];  % Phase modulated
sig_fsk = [];  % Frequency modulated
for ind = 1:1:bin_data_len,
    if (bin_data(ind)==1)
        sig_bin = [sig_bin ones(1,sampl_per_bin)];
        sig_ask = [sig_ask sig_carrier_base];
        sig_psk = [sig_psk sig_carrier_base];
        sig_fsk = [sig_fsk sig_carrier_base];
    else
        sig_bin = [sig_bin zeros(1,sampl_per_bin)];
        sig_ask = [sig_ask 0.5*sig_carrier_base];
        sig_psk = [sig_psk sig_carrier_phase];
        sig_fsk = [sig_fsk sig_carrier_freq];
    end;
end;
```

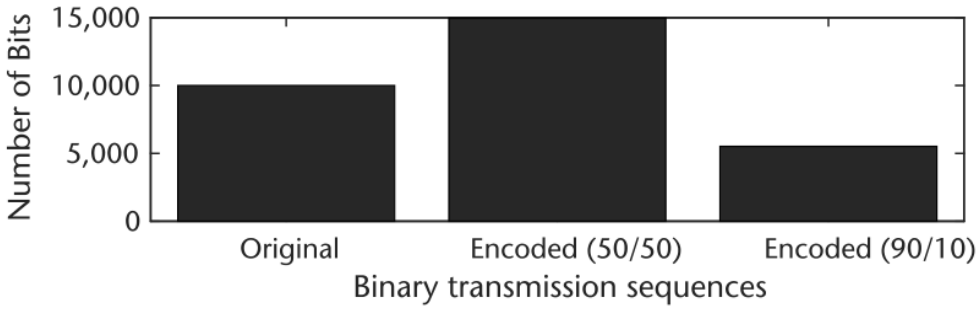


图 4.4　信源编码对二进制传输的影响

4.1.1 信源编码

任何通信系统的目标之一，是从发射机到接收机跨介质高效且可靠地传递信息。理想情况下，应把传输中所有冗余信息都去除，以最小化需要跨信道发送的信息量，最终减少传输所耗费的时间、计算资源与功率。**信源编码**就是为促成更高效通信而设计的、去除冗余信息的机制。

信源编码的运作方式：取信源符号序列 **u**，把它们映射为相应的信源编码符号序列 **v**，使 $v_i \in \mathbf{v}$ 尽可能接近随机、且 **v** 的各分量互不相关。于是，通过执行信源编码操作，我们希望在 $v_i \in \mathbf{v}$ 中实现某种程度的冗余最小化，从而限制在传输" 本可预测的符号" 上浪费的无线电资源。换言之，信源编码器从信源符号中去除冗余信息，以实现高效传输。注意执行信源编码要求信源符号是数字的。

动手 MATLAB 示例：信源编码通过去除数据集合中的冗余、用短码字取而代之，从而减少信息总量。下面的 MATLAB 脚本演示把信源编码应用于具有不同冗余度的数据。用 `rand` 与 `round` 函数的组合生成两个二进制数据向量：一个 1/0 数量均等，另一个约 90% 为 1、10% 为 0。压缩这些向量时，我们采用的编码技术是：取向量中所有连续的 1 串，用表示该串长度的十进制值替换。例如某二进制向量中两个 0 之间有 15 个 1，就用一个十进制值（或等价码字）替换这 15 个 1。

按此编码方案，直觉告诉我们：90/10 的向量相对 50/50 的向量会被压缩得更多，因为前者更可能出现长的 1 串。图 4.4 证实了直觉——压缩后的 90/10 二进制数据流尺寸显著缩减；而 50/50 的向量不仅没压缩反而变大了。原因在于：把每段 1 串替换为相应码字的开销，实际占用比原始二进制序列还多。因此做信源编码时，压缩具有明显冗余的数据流才划算，否则可能适得其反。

代码 4.2　用信源编码减少数据传输量：chapter4.m

```
% Define parameters
```

```
len = 10000;  % Length of binary data stream

% Have two binary sources, one 50/50 and the other 90/10 in terms
% of ones and zeros
bin1 = round(rand(1,len));              % 50/50 binary
bin2 = round(0.5*rand(1,len)+0.45);     % 90/10 binary

% Encode strings of ones in terms of the length of these strings
enc_bin1 = [];
enc_bin2 = [];
for ind = 1:1:len,
    if (bin1(ind) == 1)  % Encoding 50/50
        if (ind == 1)
            enc_bin1 = 1;
        else
            enc_bin1(end) = enc_bin1(end)+1;
        end;
    else
        enc_bin1 = [enc_bin1 0];
    end;
    if (bin2(ind) == 1)  % Encoding 90/10
        if (ind == 1)
            enc_bin2 = 1;
        else
            enc_bin2(end) = enc_bin2(end)+1;
        end;
    else
        enc_bin2 = [enc_bin2 0];
    end;
end;

% Find size of encoded binary streams
% (assume all one string length values possess the same number of bits)
```

```
ind1 = find(enc_bin1 ~= 0);
ind2 = find(enc_bin2 ~= 0);
[largest_ebin1,ind_largest_ebin1] = max(enc_bin1(ind1));
[largest_ebin2,ind_largest_ebin2] = max(enc_bin2(ind2));
numbits1 = length(dec2bin(largest_ebin1)-'0');
numbits2 = length(dec2bin(largest_ebin2)-'0');
total_size_ebin1 = length(ind1)*numbits1 + length(find(enc_bin1 == 0));
total_size_ebin2 = length(ind2)*numbits2 + length(find(enc_bin2 == 0));
```

4.1.2 信道编码

为保护数字传输的信息不被破坏，需要引入某种程度的受控冗余以逆转数据损坏的影响。**信道编码**通过向数据传输引入受控冗余来纠正信道传输差错。与信源编码中去除的" 本质随机" 的冗余不同，信道编码引入的冗余是专门为对抗传输比特差错而设计的（即冗余具有发射机与接收机双方都已知的特定结构）。

一般而言，信道编码这样运作：长度为 K 的信源编码输出向量（即 $\mathbf{v}_l, l = 1, 2, \ldots, 2^K$）被指派唯一的码字，如向量 $\mathbf{v}_l = (101010\ldots)$ 被指派码本 $\mathcal{C}$ 中长度为 N 的唯一码字 $\mathbf{c}_l$。此过程中信道编码器引入 $N - K = r$ 个受控比特。通信系统的**码率**等于信息比特数与码字长度之比（即码率等于 K/N）。

为信道编码器设计码本时，需要定量评估码字在数据损坏情形下的表现好坏。**汉明距离**常用来通过评估任意两个码字的相对差异，确定码本中一组码字的有效性。具体地，任意两个码字 $\mathbf{c}_i$ 与 $\mathbf{c}_j$ 之间的汉明距离 $d_H(\mathbf{c}_i, \mathbf{c}_j)$ 等于 $\mathbf{c}_i$ 与 $\mathbf{c}_j$ 不同的分量数。评估码本设计时我们常关注码字间的**最小**汉明距离：

$$d_{H,\min} = \min_{\mathbf{c}_i, \mathbf{c}_j \in \mathcal{C}, i \neq j} d_H(\mathbf{c}_i, \mathbf{c}_j),$$

因为我们的目标是最大化给定码本的最小汉明距离，确保" 意外选中正确码字之外的码字" 的概率保持最小。例如码本 $\{101, 010\}$，最小汉明距离 $d_{H,\min} = 3$，是最好可能的结果；而码本 $\{111, 101\}$ 的 $d_{H,\min} = 1$，相对就很差。

当码字在传输中损坏时，可用**译码球**（又称汉明球）对接收信息做判决，如图 4.5 所示：传输中损坏的码字被映射到最近的合法码字。注意设计码本时译码球不应重叠，以获得接收机最佳译码性能（即 $\rightarrow d_{H,\min} = 2t + 1$）。

动手 MATLAB 示例：把重复编码应用于一个实际向量，用一组重复率，观察二进制向量暴露于随机比特翻转时的影响。下面的 MATLAB 脚本取原始二进制向量 `bin_str`，

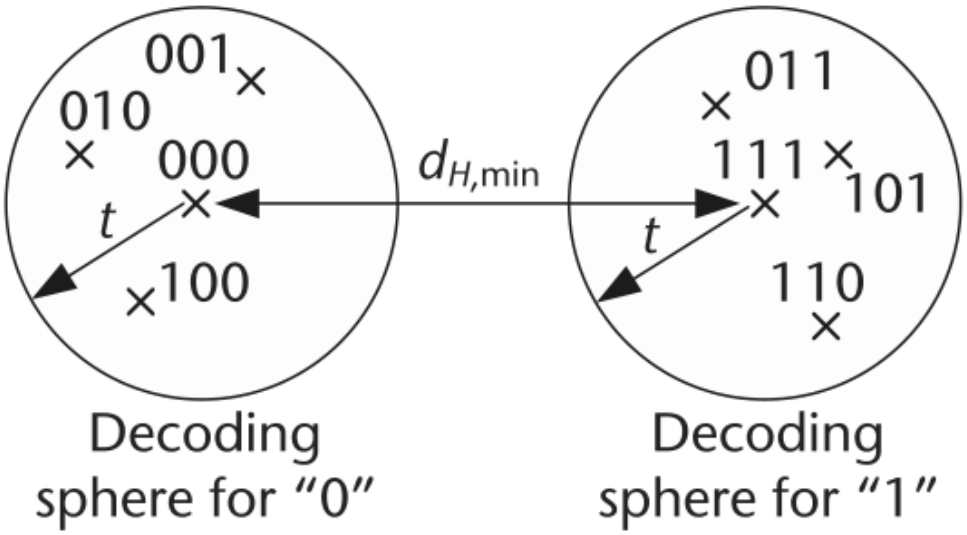


图 4.5　译码球示例

以” 把这些二进制值重复 N 次” 的形式引入受控冗余。例如不直接发 010，用重复因子 $N = 3$ 的重复编码将输出 000111000。这很重要的原因在于：当某个比特从 1 翻成 0 或从 0 翻成 1（视为一次差错）时，其余重复的比特可以在接收机处抵消这个差错——即若某个 0 翻成了 1，另外两个 0 会告诉接收机该组里有比特出错、译码时丢弃该值。直觉上我们会认为重复越多越保险；例如存在多于一个比特损坏的情形，除非有更多重复比特告知接收机，否则仍会出错。

> 思考题：一个无信源编码的码率 1/3 重复码如下：
>
> $$1 \to 111 = \mathbf{c}_1 \text{ (第 1 个码字)}, \quad 0 \to 000 = \mathbf{c}_2 \text{ (第 2 个码字)}$$
>
> $\therefore\ \mathcal{C} = \{000, 111\}$。请计算码字对的汉明距离 $d_H(111, 000)$ 与 $d_H(111, 101)$。

代码 4.3　用简单重复信道编码保护数据：chapter4.m

```
% Define parameters
len = 100000;  % Length of original binary data stream
N1 = 3;        % First repetition factor; should be odd to avoid tie
N2 = 5;        % Second repetition factor; should be odd to avoid tie
N3 = 7;        % Third repetition factor; should be odd to avoid tie
```

```
% Generate binary data stream
bin_str = round(rand(1,len));

% Employ repetition code with repetition factors N1, N2, N3
chcode1_bin_str = zeros(1,N1*len);
chcode2_bin_str = zeros(1,N2*len);
chcode3_bin_str = zeros(1,N3*len);
for ind = 1:1:max([N1 N2 N3]),
    if (ind<=N1)
        chcode1_bin_str(ind:N1:(N1*(len-1)+ind))=bin_str;
    end;
    if (ind<=N2)
        chcode2_bin_str(ind:N2:(N2*(len-1)+ind))=bin_str;
    end;
    if (ind<=N3)
        chcode3_bin_str(ind:N3:(N3*(len-1)+ind))=bin_str;
    end;
end;

% Corrupt both binary strings with zero-mean unit variance Gaussian
% noise followed by rounding (creates "bit flipping" errors)
noisy_bin_str = bin_str + randn(1,len);
rx_bin_str0 = zeros(1,len);
ind0 = find(noisy_bin_str >= 0.5);
rx_bin_str0(ind0) = 1;
noisy_chcode1_bin_str = chcode1_bin_str + randn(1,N1*len);
rx_chcode1_bin_str = zeros(1,N1*len);
ind1 = find(noisy_chcode1_bin_str >= 0.5);
rx_chcode1_bin_str(ind1) = 1;
noisy_chcode2_bin_str = chcode2_bin_str + randn(1,N2*len);
rx_chcode2_bin_str = zeros(1,N2*len);
ind2 = find(noisy_chcode2_bin_str >= 0.5);
```

```
rx_chcode2_bin_str(ind2) = 1;
noisy_chcode3_bin_str = chcode3_bin_str + randn(1,N3*len);
rx_chcode3_bin_str = zeros(1,N3*len);
ind3 = find(noisy_chcode3_bin_str >= 0.5);
rx_chcode3_bin_str(ind3) = 1;

% Decode three encoded binary sequences
dec1_bin = (vec2mat(rx_chcode1_bin_str,N1)).';
dec2_bin = (vec2mat(rx_chcode2_bin_str,N2)).';
dec3_bin = (vec2mat(rx_chcode3_bin_str,N3)).';
ind11 = find(((sum(dec1_bin,1))/N1) >= 0.5);
ind12 = find(((sum(dec2_bin,1))/N2) >= 0.5);
ind13 = find(((sum(dec3_bin,1))/N3) >= 0.5);
rx_bin_str1 = zeros(1,len);
rx_bin_str1(ind11) = 1;
rx_bin_str2 = zeros(1,len);
rx_bin_str2(ind12) = 1;
rx_bin_str3 = zeros(1,len);
rx_bin_str3(ind13) = 1;

% Calculate bit error rate
ber0 = sum(abs(bin_str - rx_bin_str0))/len;
ber1 = sum(abs(bin_str - rx_bin_str1))/len;
ber2 = sum(abs(bin_str - rx_bin_str2))/len;
ber3 = sum(abs(bin_str - rx_bin_str3))/len;
```

由 MATLAB 脚本可见重复编码对易错环境中被比特翻转破坏的二进制向量的影响（见图 4.6）：随着以更大重复率的形式引入更多受控冗余，传输中的比特差错逐渐减少。这说得通——往传输里投入更多资源使其在易错环境中更可靠，差错发生率就开始下降。但要注意其中存在成本-收益折中：投入的资源与获得的收益未必线性相关。

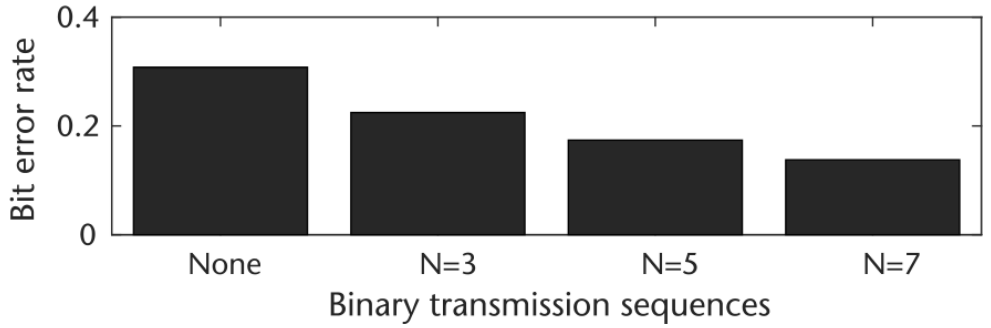


图 4.6　不同重复因子 N 下重复编码对二进制传输的影响

4.1.2.1　香农信道编码定理

数字通信中有时需要确定特定数字收发信机设计的数据速率上限。1949 年，克劳德 · 香农发表了针对这一主题的开创性论文《通信的数学理论（在噪声中的通信）》[1]。论文中他给出了描述数字收发信机数据速率极限（即容量）的定量表达式，以实现无误传输。

设信道容量为 C，以固定码率 $K/N = R_c$（常数）发送数据——于是增大 N 就必须增大 K 以保持 R_c 恒定。香农的结论是：存在这样的码，使得当 $R_c = K/N < C$ 且 $N \to \infty$ 时，差错概率 $P_e \to 0$；反之当 $R_c = K/N \geq C$ 时不存在这样的码。因此 C 是可靠通信的速率极限（即绝对极限——快过它就会出错）。

为什么这个结果如此重要？首先，数字通信的可靠性通常用接收机输出端测得的误比特概率表达。若能用数学工具而非经验测量，由传输带宽 B 与接收 SNR 得出这个容量，将非常便利。香农推导出的信道信息容量等于

$$C = B\log_2(1 + \text{SNR}) \quad \text{[b/s]},$$

该信息容量告诉我们可达的数据速率。注意香农只给出了可达容量的**理论极限**，并未告诉我们如何建造达到该极限的收发信机。其次，信息容量表达式给出了给定带宽 B 与接收 SNR 下可达数据速率的界，用于比值 $\eta = R/C$（R 为信令速率、C 为信道容量）：$\eta \to 1$ 时系统更高效。因此容量表达式为 B 与 SNR 之间的折中分析提供了基础，也可用于比较不同调制方案的噪声性能。

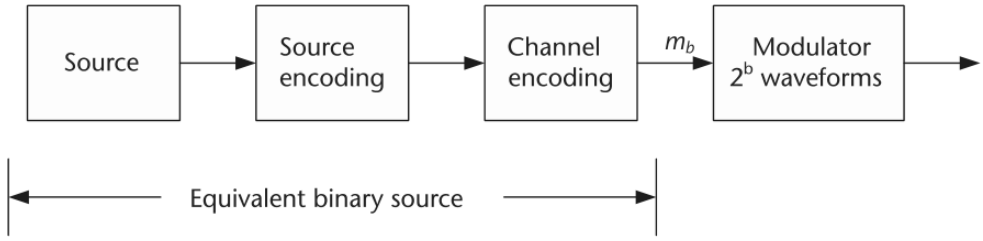


图 4.7　等效二进制数据的调制过程

4.2 数字调制

模拟调制中，模拟消息信号在跨介质传输前调制连续波；反过来，**数字调制**是用数字消息信号调制连续波形。如本章前文所见，这可以通过在每个符号周期 T 内按特定比特图案唯一地操纵信号的幅度与相位信息来实现。但多数数字调制技术还有一个中间步骤，本节将聚焦于此：把组成二进制消息 m_b 的 b 个比特的集合映射为一个**符号**，再用该符号定义连续波形在幅度与相位方面的物理特性。具体地，对 m_b 的每个可能的 2^b 个取值，我们需要唯一的信号 $s_i(t)$（$1 \leq i \leq 2^b$）去调制连续波形，如图 4.7 所示。

本节将研究把二进制数据映射为符号（再用于调制连续波形）的几大不同方法族。这些调制方案族由映射过程操纵的物理特性或特性组合来定义，以唯一表示特定比特图案。但这些族之间存在各种折中，包括把比特映射为符号在发射功率方面的效率。因此我们先研究如何评估这一折中，再学习三大数字调制方案族并比较它们。

4.2.1 功率效率

为评估" 比特到符号的映射" 在每符号耗费发射功率方面的有效性，可以采用**功率效率**度量。设符号 $s(t)$ 的能量为

$$E_s = \int_0^T s^2(t)\,dt,$$

其中 T 是符号周期。对由 M 个符号组成的调制方案，平均符号能量由加权平均定义：

$$\overline{E}_s = P(s_1(t)) \cdot \int_0^T s_1^2(t)\,dt + \cdots + P(s_M(t)) \cdot \int_0^T s_M^2(t)\,dt,$$

其中 $P(s_i(t))$ 是符号 $s_i(t)$ 出现的概率。想计算平均每比特能量，可用 $\overline{E}_s$ 除以每符号比特数 $b = \log_2(M)$：

$$\overline{E}_b = \frac{\overline{E}_s}{b} = \frac{\overline{E}_s}{\log_2(M)}.$$

为定量评估两个符号物理特性的相似度，定义**欧氏距离**为

$$d_{ij}^2 = \int_0^T (s_i(t) - s_j(t))^2 dt = E_{s_{ij}},$$

其中 $\Delta s_{ij}(t) = s_i(t) - s_j(t)$。评估调制方案性能时我们常关心最坏情形，通常计算**最小**欧氏距离：

$$d_{\min}^2 = \min_{s_i(t), s_j(t), i \neq j} \int_0^T (s_i(t) - s_j(t))^2 dt.$$

于是，用于调制的信号集的功率效率由下式给出：

$$\varepsilon_p = \frac{d_{\min}^2}{\overline{E}_b}. \tag{4.8}$$

> 思考题：设如下波形（$u(t)$ 为单位阶跃函数）：$s_1(t) = A \cdot [u(t) - u(t-T)] = s(t)$，$s_2(t) = -A \cdot [u(t) - u(t-T)] = -s(t)$。请计算：最小欧氏距离 $d_{\min}^2$；平均比特能量 $\overline{E}_b$；功率效率 ε_P。

4.2.2 脉冲幅度调制

在可用来承载信息的信号波形各物理特性中，最显而易见的选择是信号幅度电平。利用这一特性，**脉冲幅度调制**（PAM）是把消息信息编码进一串信号脉冲幅度中的数字调制方案。PAM 传输的解调通过在每个符号周期检测载波幅度电平完成。

PAM 最基本的形式是二进制 PAM（B-PAM）：单个二进制数字按下述调制规则映射到具有两个幅度电平的波形 $s(t)$：

- “1” $\rightarrow s_1(t)$
- “0” $\rightarrow s_2(t)$

其中 $s_1(t)$ 是具有某个幅度电平的波形 $s(t)$，$s_2(t)$ 也基于 $s(t)$ 但具有另一个幅度电平。波形 $s(t)$ 定义在周期 T 内、其余时间为零。由于符号持续时间等于比特持续时间，B-PAM 传输的比特率为 $R_b = 1/T$ 比特/秒。

波形 $s(t)$ 的能量定义为

$$E_s = \int_0^T s^2(t)\,dt \quad (\text{焦耳}).$$

设 $s(t)$ 为矩形波形：

$$s(t) = A \cdot [u(t) - u(t-T)],$$

其中 $u(t)$ 是单位阶跃函数、A 是信号幅度。再设比特"1" 由幅度 A 定义、比特"0" 由幅度 $-A$ 定义，则调制规则为：

- "1" $\rightarrow s(t)$
- "0" $\rightarrow -s(t)$

于是符号能量 $E_s = E_{-s} = A^2T = \frac{A^2}{R_b}$。由此，B-PAM 传输的每比特能量为

$$\overline{E}_b = P(1) \cdot \int_0^T s_1^2(t)\,dt + P(0) \cdot \int_0^T s_2^2(t)\,dt,$$

其中 $P(1)$ 是比特为"1" 的概率，$P(0)$ 是比特为"0" 的概率。定义 $s_1(t) = s(t)$、$s_2(t) = -s(t)$，则平均每比特能量等于

$$\overline{E}_b = E_s\{P(1) + P(0)\} = E_s = \int_0^T s^2(t)\,dt = A^2T.$$

计算最小欧氏距离：

$$d_{\min}^2 = \int_0^T (s(t) - (-s(t)))^2 dt = \int_0^T (2s(t))^2 dt = 4A^2T,$$

代入 (4.8) 得到 B-PAM 传输的功率效率：

$$\varepsilon_p = \frac{d_{\min}^2}{\overline{E}_b} = \frac{4A^2T}{A^2T} = 4.$$

正如本节接下来将看到的：当每个可能的二进制序列都映射到唯一符号时，功率效率 4 是任何数字调制方案能取得的最好结果。

现在把 B-PAM 的平均比特能量、最小欧氏距离与功率效率推广到把二进制序列映射到 M 个可能幅度电平的情形——**M 进制脉冲幅度调制**（M-PAM）。先把 M-PAM 波形表示为

$$s_i(t) = A_i \cdot p(t), \quad i = 1, 2, \ldots, M/2$$

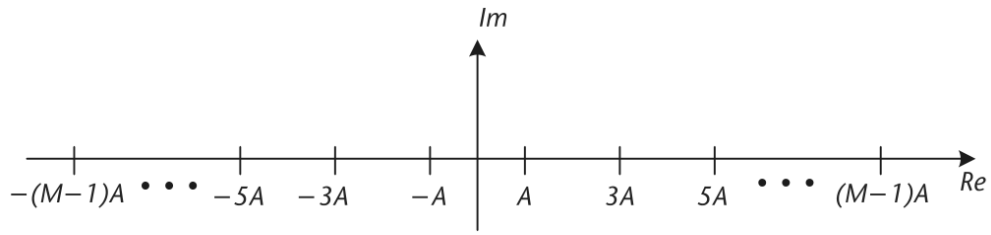


图 4.8　M-PAM 信号星座

其中 $A_i = A(2i-1)$，$p(t) = u(t) - u(t-T)$，$u(t)$ 是单位阶跃函数。图形上，M-PAM 调制方案如图 4.8 的信号星座所示。

计算 M-PAM 的功率效率 $\varepsilon_{p,M\text{-PAM}}$ 时，选 $d^2_{\min}$ 对应的一对 $s_1(t) = A \cdot p(t)$ 与 $s_2(t) = -A \cdot p(t)$（这对信号波形形状上最接近）。于是两者之差为

$$\Delta s(t) = 2A \cdot p(t),$$

得到最小欧氏距离

$$d^2_{\min} = 4A^2T.$$

计算平均符号能量 $\overline{E}_s$ 时可利用星座对称性简化数学：

$$\overline{E}_s = \frac{2}{M} A^2 T \sum_{i=1}^{M/2} (2i-1)^2 = A^2 T \frac{(M^2-1)}{3}$$

（级数求和查表可得），进而

$$\overline{E}_b = \frac{\overline{E}_s}{\log_2(M)} = \frac{A^2T(2^{2b}-1)}{3b}.$$

最后解出功率效率：

$$\varepsilon_{p,M\text{-PAM}} = \frac{12b}{2^{2b}-1}.$$

4.2.3 正交幅度调制

与 PAM 类似，**正交幅度调制**（QAM）也涉及某种幅度调制。但 QAM 是二维信号调制方案（PAM 是一维的）。QAM 调制的两个维度——同相与正交分量——相互正交，这意味着本质上可以免费把传输数据率翻倍。而且矩形 QAM 可以看作两个正交 PAM 信

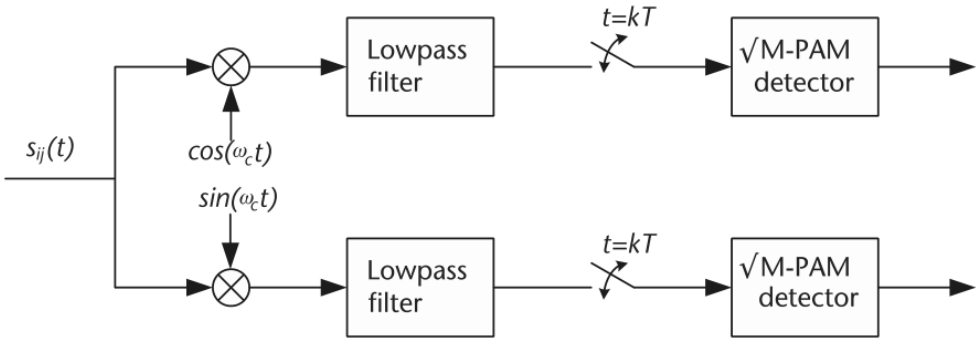


图 4.9　M-QAM 接收机结构

号同时发射。

数学上，若矩形 QAM 星座由 M 个唯一波形组成，它可以表示为 $\sqrt{M}$-PAM 在正交维度上同时传输。注意 QAM 星座也可以是嵌套圆形式（圆 QAM）或任何涉及正交调制器的几何图案。矩形 QAM 是流行方案，因为接收机结构相对简单，如图 4.9 所示，每个维度采用 $\sqrt{M}$ 进制 PAM 检测器。

为确定 M-QAM 的功率效率，先定义属于该调制形式的信号波形的数学表示：

$$s_{ij}(t) = A_i \cdot \cos(\omega_c t) + B_j \cdot \sin(\omega_c t),$$

其中 ω_c 是载波频率，A_i 与 B_j 是同相与正交幅度电平。注意余弦与正弦用于在正交维度调制这些幅度电平。把 M-QAM 画成信号星座，信号波形将落在实轴与虚轴两个方向上，如图 4.10 所示。

计算 M-QAM 的功率效率 $\varepsilon_{p,M\text{-QAM}}$，先算最小欧氏距离（不失一般性选如下信号波形）：

$$s_1(t) = A \cdot \cos(\omega_c t) + A \cdot \sin(\omega_c t)$$

$$s_2(t) = 3A \cdot \cos(\omega_c t) + A \cdot \sin(\omega_c t)$$

得到

$$d^2_{\min} = \int_0^T \Delta s^2(t)\, dt = 2A^2 T.$$

推导平均符号能量 $\overline{E}_s$ 时，可把 M-PAM 表达式中的 M 换成 $\sqrt{M}$：

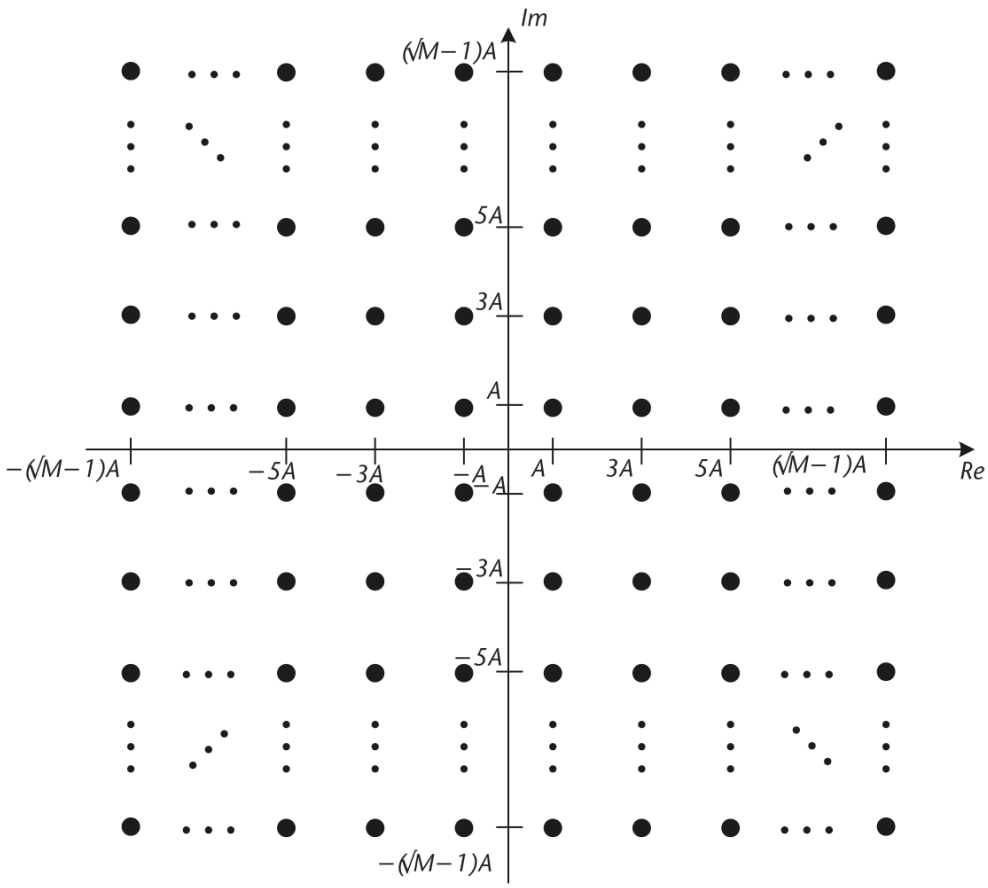


图 4.10　M-QAM 信号星座

$$\overline{E}_s = A^2 T \frac{\sqrt{M}-1}{3},$$

进而

$$\overline{E}_b = \frac{\overline{E}_s}{\log_2(M)} = A^2 T \frac{2^b - 1}{3b}.$$

于是功率效率等于

$$\varepsilon_{p,M\text{-QAM}} = \frac{6b}{2^b - 1}.$$

动手 MATLAB 示例：QAM 调度是在同一符号周期内高效发送信息的非常有用的技术。如前所述，它同时利用同相与正交两个域，把信息并行正交地沿两条信道传输。下

面的 MATLAB 脚本实现一个 QAM 发射机与接收机：取二进制向量 samp_I 与 samp_Q，调制到 QAM 传输的同相与正交信道，再用 QAM 解调提取这些二进制向量。QAM 调制的核心是正弦与余弦函数——它们数学上正交。如脚本所示，同相与正交数据在发射机调制到余弦与正弦载波上；接收机利用这一正交性与三角性质，把同相与正交信息从接收信号中分离出来。

图 4.11 展示了余弦与正弦波作为信息载体的用法：信息可以同时发送并被完好恢复。通过用两个维度传输信息，我们对每个用于广播数据的符号周期都做了更高效的利用。

代码 4.4　用 I/Q 接收机解调 QAM 波形：chapter4.m

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Decoding QAM waveform using I/Q receiver

% Define parameters
N_samp = 1000;  % Number of samples per symbol
N_symb = 10;    % Number of symbols in transmission
cfreq = 1/10;   % Carrier frequency of cosine and sine carriers

% Generate inphase and quadrature channels with 2-PAM waveforms
chI = 2*round(rand(1,N_symb))-1;
chQ = 2*round(rand(1,N_symb))-1;
samp_I = [];
samp_Q = [];
for ind = 1:1:N_symb,
    samp_I = [samp_I chI(ind)*ones(1,N_samp)];
    samp_Q = [samp_Q chQ(ind)*ones(1,N_samp)];
end;

% Apply cosine and sine carriers to inphase and quadrature components,
% sum waveforms together into composite transmission
tx_signal = samp_I.*cos(2.*pi.*cfreq.*(1:1:length(samp_I)))
              + samp_Q.*sin(2.*pi.*cfreq.*(1:1:length(samp_Q)));
```

Code 4.4 Decoding QAM Waveform Using I/Q Receiver: **chapter4.m**

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Decoding QAM waveform using I/Q receiver

% Define parameters
N_samp = 1000; % Number of samples per symbol
N_symb = 10; % Number of symbols in transmission
cfreq = 1/10; % Carrier frequency of cosine and sine carriers

% Generate inphase and quadrature channels with 2-PAM waveforms
chI = 2*round(rand(1,N_symb))-1;
chQ = 2*round(rand(1,N_symb))-1;
samp_I = [];
samp_Q = [];
for ind = 1:1:N_symb,
    samp_I = [samp_I chI(ind)*ones(1,N_samp)];
    samp_Q = [samp_Q chQ(ind)*ones(1,N_samp)];
end;

% Apply cosine and sine carriers to inphase and quadrature components,
% sum waveforms together into composite transmission
tx_signal = samp_I.*cos(2.*pi.*cfreq.*(1:1:length(samp_I)))
              + samp_Q.*sin(2.*pi.*cfreq.*(1:1:length(samp_Q)));

% Separate out inphase and quadrature components from composite
```

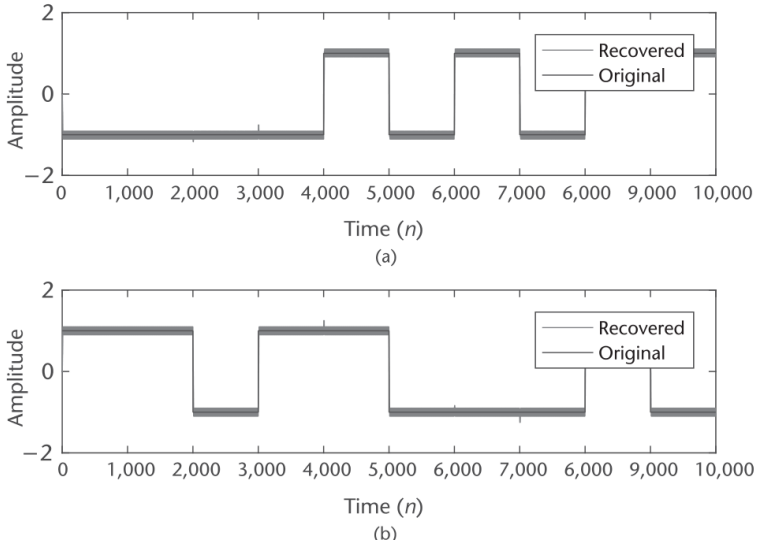


图 4.11　用同相/正交接收机解调 QAM 波形示例（原书此图为 MATLAB 会话截图，上半部与代码 4.4 内容相同）：(a) 同相信号分量；(b) 正交信号分量

```
% Separate out inphase and quadrature components from composite
```

4.2.4 相移键控

相移键控（PSK）是通过改变或调制参考信号（载波）相位来传递数据的数字调制方案。任何数字调制方案都用有限个互异信号表示数字数据；PSK 用有限个相位，每个指派唯一的二进制数字图案。通常每个相位编码相同数量的比特，每个比特图案构成由特定相位表示的符号。解调器（专为调制器使用的符号集设计）确定接收信号的相位并映射回它所代表的符号，从而恢复原始数据。这要求接收机能把接收信号相位与参考信号比较——这样的系统称为**相干**的。

PSK 用相位刻画符号。数学上，PSK 信号波形表示为

$$s_i(t) = A\cos\Big(2\pi f_c t + (2i-1)\frac{\pi}{m}\Big), \quad i = 1, \ldots, \log_2 m,$$

其中 A 是幅度，f_c 是载波频率，$(2i-1)\frac{\pi}{m}$ 是每个符号的相移。PSK 与 PAM、QAM 呈现一组有趣的折中：幅度调制方案中信道均衡是正确译码符号的重要部分，而 PSK 方案中接收信号的相位远比幅度信息重要。

按可指派给 PSK 波形的 M 个可能相位值，PSK 有几种类型。最流行最稳健之一是二进制 PSK（B-PSK），其信号星座示于图 4.12。B-PSK 的调制规则一般为：

$$\text{”1”} \to s_1(t) = A \cdot \cos(\omega_c t + \theta)$$

$$\text{”0”} \to s_2(t) = -A \cdot \cos(\omega_c t + \theta) = A \cdot \cos(\omega_c t + \theta + \pi) = -s_1(t).$$

换言之，构成 B-PSK 调制方案的两个信号波形相位相差 $\theta + \pi$。

为推导 B-PSK 的功率效率 $\varepsilon_{p,\text{BPSK}}$，先用定义计算最小欧氏距离：

$$d^2_{\text{min}} = \int_0^T (s_1(t) - s_2(t))^2 dt = 4A^2 \int_0^T \cos^2(\omega_c t + \theta)\, dt$$

$$= \frac{4A^2T}{2} + \frac{4A^2}{2}\int_0^T \cos(2\omega_c t + 2\theta)\, dt = 2A^2T. \tag{4.28}$$

> 注：注意 (4.28) 最终结果中第二项消失了——因为第二项的载频是原信号的两倍。载波是周期正弦波形，对如此高频率的信号积分，正负部分相互抵消，结果接近于零。因此称该项为倍频项。另外注意许多通信系统会对收到的信号滤波，把倍频项滤掉的概率也相当高。

计算 d^2_{min} 的另一种方式是用**相关**概念——它描述两个信号波形的相似程度。此时最小欧氏距离可表示为

$$d^2_{\text{min}} = \int_0^T (s_2(t) - s_1(t))^2 dt = E_{s_1} + E_{s_2} - 2\rho_{12}$$

其中符号 i 的能量 E_{s_i} 与符号 1、2 之间的相关 ρ_{12} 为

$$E_{s_i} = \int_0^T s_i^2(t)\,dt \quad \text{且} \quad \rho_{12} = \int_0^T s_1(t)s_2(t)\,dt.$$

用 $\overline{E}_b$ 的定义，先解两个信号波形的符号能量再平均，即得 B-PSK 的平均比特能量：

$$E_{s_1} = \int_0^T s_1^2(t)\,dt = A^2 \int_0^T \cos^2(\omega_c t + \theta)\,dt = \frac{A^2T}{2} + \frac{A^2}{2}\int_0^T \cos(2\omega_c t + 2\theta)\,dt = \frac{A^2T}{2}$$

$$E_{s_2} = \frac{A^2T}{2}, \qquad \overline{E}_b = P(0)\cdot E_{s_2} + P(1)\cdot E_{s_1} = \frac{A^2T}{2}.$$

注意由于单个符号表示的比特数为一，比特能量与符号能量等价。最后应用功率效率定义：

$$\varepsilon_{p,\text{BPSK}} = \frac{d^2_{\text{min}}}{\overline{E}_b} = 4.$$

这是使用全部可能信号表示（即 $M = 2^b$ 个波形）的调制方案所能取得的最大 ε_p。注意用相关法计算最小欧氏距离时，要得到大的 ε_p 需最大化 d^2_{min}，即希望 $\rho_{12} < 0$。要达到这一结果，需要

$$E_{s_1} = E_{s_2} = E = A^2T/2,$$

这意味着 $d^2_{\text{min}} = 2(E - \rho_{12})$，进而 $\rho_{12} = -E$。

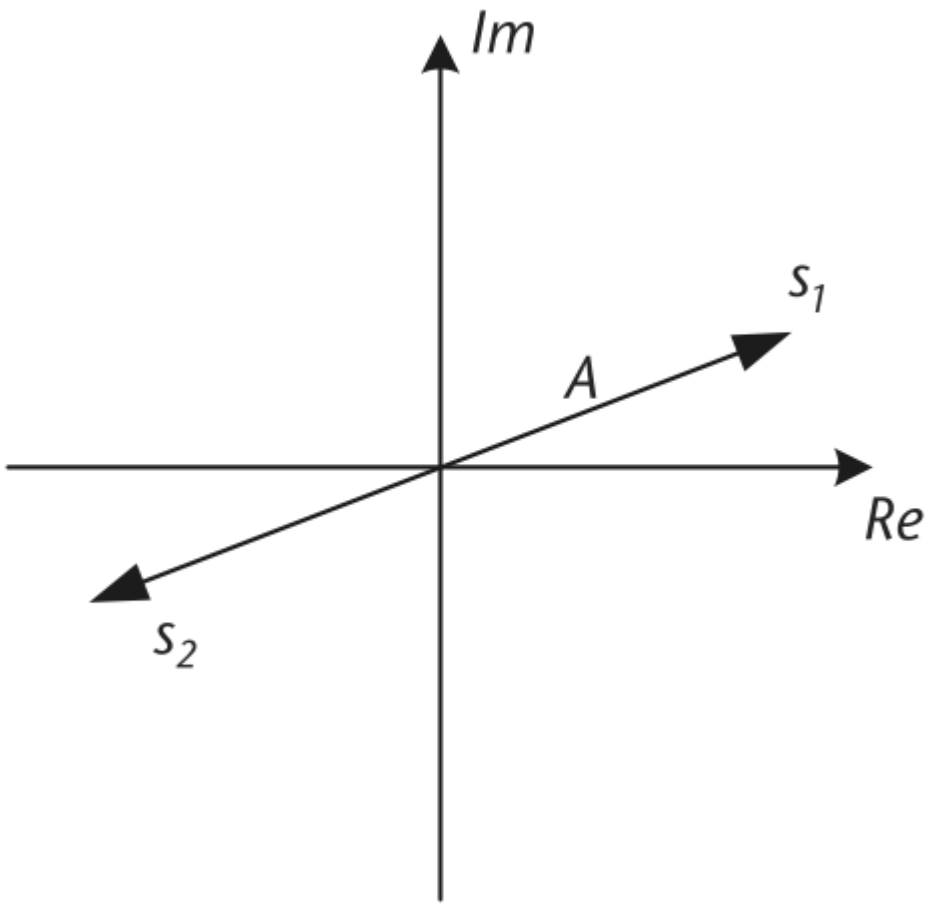


图 4.12　BPSK 信号星座

思考题：证明对波形 $s_1(t) = A \cdot \cos(\omega_c t + \theta)$、$s_2(t) = 0$，功率效率 $\varepsilon_p = 2$。

思考题：证明对波形 $s_1(t) = A \cdot \cos(\omega_c t + \theta)$、$s_2(t) = A \cdot \sin(\omega_c t + \theta)$，功率效率 $\varepsilon_p = 2$。

到目前为止我们研究的 PSK 只含两个波形。现在把 PSK 星座扩展到每方案四个互异波形。**正交 PSK**（QPSK）调制中，信号波形表示为

$$s_i(t) = \pm A \cdot \cos(\omega_c t + \theta) \pm A \cdot \sin(\omega_c t + \theta),$$

每个信号波形幅度相同但相位取四个可能值之一。这类相位调制由图 4.13 的星座图说明，每个波形位于不同相位处。

推导 QPSK 的功率效率 $\varepsilon_{p,\text{QPSK}}$，先解最小欧氏距离 $d^2_{\min}$：

$$d^2_{\min} = \int_0^T \Delta s^2(t)\, dt = 2A^2 T.$$

接着求 $\overline{E}_b$，需要对所有信号波形取平均：

$$\overline{E}_b = \frac{(E_{s_1} + E_{s_2} + E_{s_3} + E_{s_4})/4}{\log_2(M)} = \frac{A^2 T}{2},$$

其中四个符号的能量都等于 $E_{s_1} = E_{s_2} = E_{s_3} = E_{s_4} = A^2 T$。最后用 (4.8) 解功率效率：

$$\varepsilon_{p,\text{QPSK}} = \frac{d^2_{\min}}{\overline{E}_b} = 4,$$

与 BPSK 相同但每符号 2 比特——是个极佳的结果！

最后研究一般情形：PSK 调制方案有 M 个可能相位值，星座点到原点的距离恒定，星座由圆上等间隔的 M 个点组成。称为 M-PSK，信号波形数学表示为

$$s_i(t) = A \cdot \cos\left(\omega_c t + \frac{2\pi i}{M}\right), \quad i = 0, 1, 2, \ldots, M-1.$$

注意该方案有几个优缺点：如 M 增大时星座点间距缩小，导致抗差错稳健性下降；反过来，信息编码在相位中带来**恒包络**调制，对非线性功率放大器友好，且对幅度失真信道稳健。

关于 M-PSK 功率效率 $\varepsilon_{p,M\text{-PSK}}$ 的推导：设两个相邻 M-PSK 波形 $s_1(t) = A\cdot\cos(\omega_c t)$ 与 $s_2(t) = A\cdot\cos(\omega_c t + 2\pi/M)$。用

$$d^2_{\text{min}} = E_{s_1} + E_{s_2} - 2\rho_{12}$$

计算最小欧氏距离，其中符号能量为

$$E_{s_i} = \int_0^T s_i^2(t)\,dt = \frac{A^2T}{2}, \quad i = 1, 2,$$

两波形的相关为

$$\rho_{12} = \int_0^T s_1(t)s_2(t)\,dt = \frac{A^2T}{2}\cos\frac{2\pi}{M},$$

于是

$$d^2_{\text{min}} = A^2T\left(1 - \cos\frac{2\pi}{M}\right).$$

平均比特能量 $\overline{E}_b = \overline{E}_s/\log_2(M)$，其中 $\overline{E}_s = A^2T/2$。用 (4.8) 的功率效率定义得到

$$\varepsilon_{p,M\text{-PSK}} = 2b\left(1 - \cos\frac{2\pi}{M}\right) = 4b\sin^2\frac{\pi}{M}.$$

4.2.5 功率效率小结

考察过几种调制方案的功率效率性能后，评估不同方案间的折中很重要，以便日后实现数字通信系统时做出恰当的设计决策。为衡量相对最优的 $\varepsilon_{p,\text{QPSK}}$ 损失了多少功率效率，用下式：

$$\delta_{\text{SNR}} = 10\cdot\log_{10}\frac{\varepsilon_{p,\text{QPSK}}}{\varepsilon_{p,\text{other}}}.$$

用该式为本章所学的调制方案建立了 δ_{SNR} 值表，见表 4.1。从表 4.1 可见二维调制方案优于一维方案；且所学方案都是线性调制方案，接收机复杂度相近。有了这些功率效率的洞察，我们转向调制技术在噪声中的稳健性。

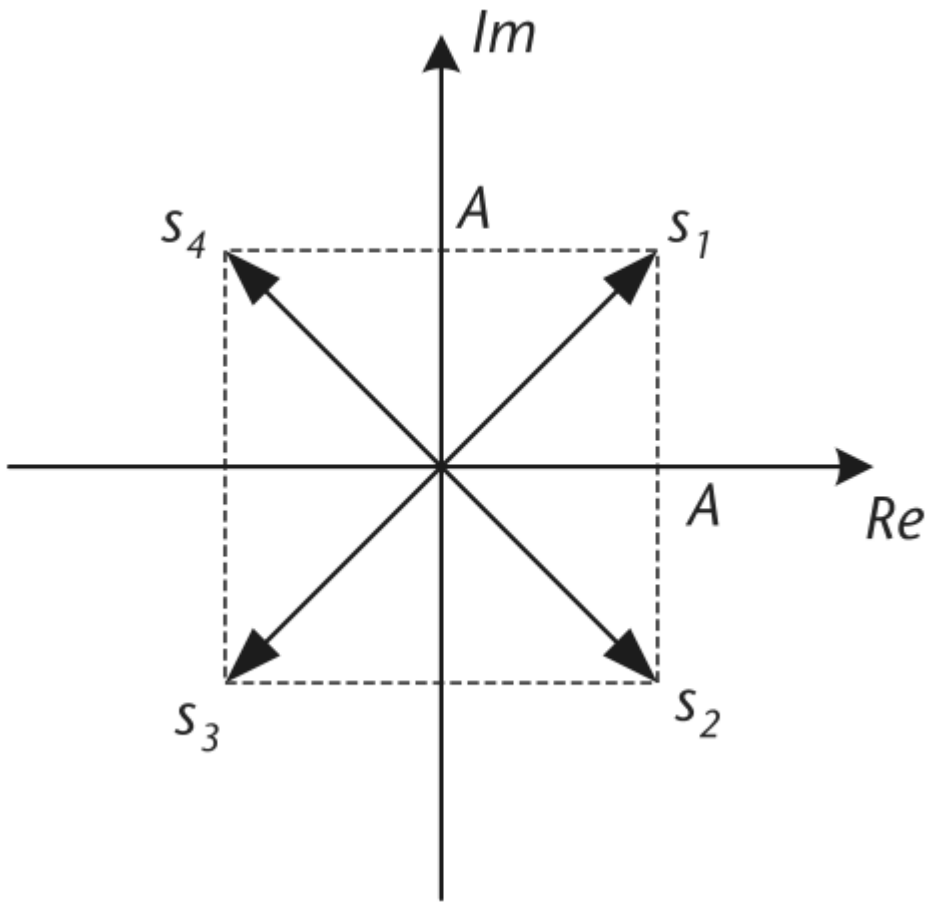


图 4.13　QPSK 信号星座

表 4.1: 各调制方案的 δSNR 值

M	b	M-ASK	M-PSK	M-QAM
2	1	0	0	0
4	2	4	0	0
8	3	8.45	3.5	—
16	4	13.27	8.17	4.0
32	5	18.34	13.41	—
64	6	24.4	18.4	8.45

动手 MATLAB 示例：传输介质引入的噪声可能导致符号被错误译码。下面的 MATLAB 脚本考察噪声的引入如何遮蔽被截获符号的真实身份。具体地，用**信号星座图**这一可视化表示比较原始发射符号与含噪接收符号——星座图把符号的位置画在以同相轴与正交轴为坐标的二维图上。本例考虑三种波形：4-PAM、4-QAM 与 QPSK。对每种波形生成其可产生的符号字母表；把代表同相与正交信息的随机二进制数据流映射为各调制方案的波形符号；再用 `randn` 函数向三个传输加高斯噪声。

4-PAM、4-QAM 与 QPSK 调制传输的前后星座图示于图 4.14：原始符号显示为红色小叉，被噪声污染的接收符号成云团状分布在红叉周围。这是因为每当传输经过含噪信道，符号就被复高斯噪声从其在同相/正交平面上的原始坐标移开。正是这种位移使接收机难以无误译码——噪声可能把符号移得离星座中另一个符号位置更近。图 4.14(a)（4-PAM）、(b)（4-QAM）、(c)（QPSK）三种调制方案都存在不可忽略的概率：符号被移近星座中的另一点，导致译码差错并转化为比特差错。

代码 4.5　生成四电平 PAM、QAM 与 QPSK 波形：chapter4.m

```
% Define parameters
len = 10000;  % Length of binary string
nvar = 0.15;  % Noise variance

% Generate the random bit streams that have already been demultiplexed
% from a single high speed data stream
bin_str1 = round(rand(1,len));  % Inphase data stream
bin_str2 = round(rand(1,len));  % Quadrature data stream

% Perform mapping of binary streams to symbols
ind_wavefm = 2.*bin_str2 + 1.*bin_str1;  % Create waveform indices
```

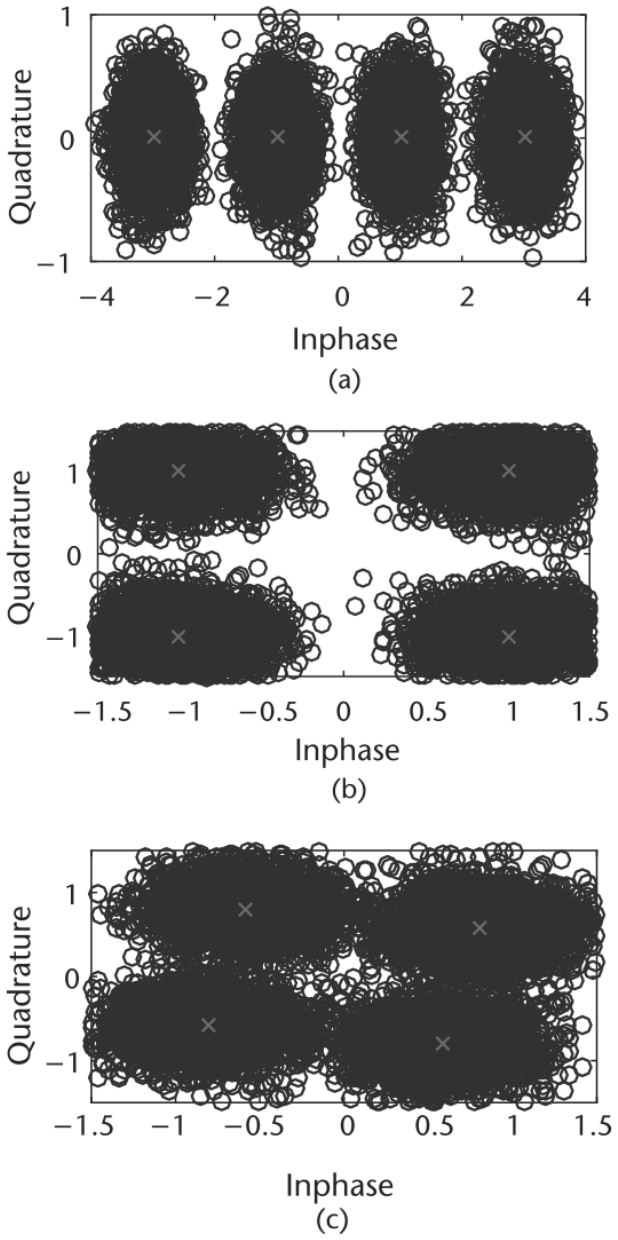


图 4.14　四电平脉冲幅度调制、正交幅度调制与正交相移键控波形示例：(a) 四电平脉冲幅度调制；(b) 四电平正交幅度调制；(c) 四电平正交相移键控

```
wavefm_4pam = zeros(1,len);  % 4-PAM
wavefm_4qam = zeros(1,len);  % 4-QAM
wavefm_qpsk = zeros(1,len);  % QPSK
symb_4pam = [-3 -1 3 1];
symb_4qam = [-1+i 1+i -1-i 1-i];
symb_qpsk = [exp(i*(pi/5+pi/2)) exp(i*(pi/5+pi)) exp(i*(pi/5+0))
             exp(i*(pi/5+3*pi/2)) ];
for ind = 1:1:4,
    wavefm_4pam(find(ind_wavefm == (ind-1))) = symb_4pam(ind);
    wavefm_4qam(find(ind_wavefm == (ind-1))) = symb_4qam(ind);
    wavefm_qpsk(find(ind_wavefm == (ind-1))) = symb_qpsk(ind);
end;

% Add complex zero-mean white Gaussian noise
noise_signal = (1/sqrt(2))*sqrt(nvar)*randn(1,len)
             + i*(1/sqrt(2))*sqrt(nvar)*randn(1,len);
rx_wavefm_4pam = wavefm_4pam + noise_signal;
rx_wavefm_4qam = wavefm_4qam + noise_signal;
rx_wavefm_qpsk = wavefm_qpsk + noise_signal;
```

4.3 误比特概率

衡量数字通信系统性能最常用的定量指标之一是误码率（BER）——发射的比特被错误译码的概率。在评估数字通信系统设计是否满足目标应用（语音、多媒体或数据）的差错稳健性要求时，该指标非常重要；在比较不同数字通信设计时也很有用。本节给出 BER 概念的数学引入。

设信号 $s_i(t)$（$i = 1, 2$）经噪声为 $n(t)$ 的 AWGN 信道传输，接收机截获信号 $r(t)$。接收机的目标是判断发射机发的是 $s_1(t)$ 还是 $s_2(t)$。鉴于发 $s_1(t)$ 还是 $s_2(t)$ 是纯随机事件，接收机关于" 发了什么" 的唯一信息就是观测到的截获信号 $r(t)$——它包含其中一个信号加上 AWGN 信道引入的噪声。

在此情形下，我们采用**假设检验** [2] 建立框架，让接收机基于截获信号 $r(t)$ 的观测判断发的是 $s_1(t)$ 还是 $s_2(t)$。采用如下假设检验框架：

$$H_1:\ r(t) = s_1(t) + n(t), \quad 0 \le t \le T$$

$$H_0:\ r(t) = s_2(t) + n(t), \quad 0 \le t \le T$$

其中 H_0 与 H_1 分别是假设 0 与假设 1。

借助该框架，下一步在接收机建立判决规则，使其能基于截获信号选择发送的是哪个波形。设假设发的是 $s_1(t)$。一般地，两信号 $x(t)$ 与 $y(t)$ 在区间 $0 \le t \le T$ 上的相关程度可用下式确定：

$$\int_0^T x(t)y(t)\,dt.$$

于是，给定观测 $r(t)$，判断发的是 $s_1(t)$ 还是 $s_2(t)$ 的判决规则定义为（设发的是 $s_1(t)$）：

$$\int_0^T r(t)s_1(t)\,dt \ \ge\ \int_0^T r(t)s_2(t)\,dt,$$

其中假设 $s_1(t)$ 被发射。回忆相关告诉我们一个波形与另一个波形的相似程度：接收机若知道 $s_1(t)$ 与 $s_2(t)$ 的样子，就能判断哪个与 $r(t)$ 更相关。既然假设发的是 $s_1(t)$，理想情况下 $r(t)$ 应与 $s_1(t)$ 比 $s_2(t)$ 更相关。

另一方面，若信道引入失真、干扰和/或噪声使发射波形损坏会怎样？当发射波形损坏到与另一个可能波形更相关时，接收机可能选错波形，产生一次差错事件。换言之，设发的是 $s_1(t)$，当

$$\int_0^T r(t)s_1(t)\,dt \ \le\ \int_0^T r(t)s_2(t)\,dt$$

时发生差错事件。把 $r(t) = s_1(t) + n(t)$ 代入，得到判决规则：

$$\int_0^T s_1^2(t)\,dt + \int_0^T n(t)s_1(t)\,dt \ \le\ \int_0^T s_1(t)s_2(t)\,dt + \int_0^T n(t)s_2(t)\,dt$$

$$\Longrightarrow\ E_{s_1} - \rho_{12} \ \le\ \int_0^T n(t)(s_2(t) - s_1(t))\,dt$$

$$\Longrightarrow\ E_{s_1} - \rho_{12} \ \le\ z.$$

从该式可见 E_{s_1} 与 ρ_{12} 都是确定量；而 z 基于信道引入的噪声，是需要刻画的随机

量。由于 $n(t)$ 是高斯随机变量，z 也是高斯随机变量——积分等价于无穷多样本的求和，高斯变量相加还是高斯变量。有了 $z \sim \mathcal{N}(0,\sigma^2)$，接着算 z 的方差 σ^2：

$$\sigma^2 = E\{z^2\} = \frac{N_0}{2}\int_0^T (s_1(t)-s_2(t))^2 dt = \frac{N_0}{2}(E_{s_1}+E_{s_2}-2\rho_{12}) \xrightarrow{E_{s_1}=E_{s_2}=E} N_0(E-\rho_{12}),$$

其中 $E = E_i = \int_0^T s_i^2(t)\,dt$、$\rho_{12} = \int_0^T s_1(t)s_2(t)\,dt$。注意我们假设信道引入零均值噪声，故这些噪声贡献之和 z 也是零均值。

确定量与随机量都刻画完毕，现在推导误比特概率。发"1" 时出错的概率 $P(e|1)$ 等于

$$P(z \geq E-\rho_{12}) = Q\left(\frac{E-\rho_{12}}{\sigma}\right) = Q\left(\sqrt{\frac{(E-\rho_{12})^2}{N_0(E-\rho_{12})}}\right) = Q\left(\sqrt{\frac{E-\rho_{12}}{N_0}}\right),$$

其中 Q 函数定义为

$$Q(x) = \int_x^\infty \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\,dt.$$

下一步通过最小化 $P(e|1)$ 来优化误比特概率，这可以通过优化相关项 ρ_{12} 达成。直觉上最佳选择是 $s_2(t) = -s_1(t)$，给出 $\rho_{12} = -E$。于是

$$P(e|1) = Q\left(\sqrt{\frac{2\overline{E}_b}{N_0}}\right).$$

注意当 $E_{s_1} \neq E_{s_2}$ 时可用 $d^2_{\min} = E_{s_1} + E_{s_2} - 2\rho_{12}$，得到误比特概率表达式：

$$P_e = Q\left(\sqrt{\frac{d^2_{\min}}{2N_0}}\right).$$

> 思考题：证明总误比特概率 $P_e = P(e|1)P(1) + P(e|0)P(0) = Q\left(\sqrt{\frac{E-\rho_{12}}{N_0}}\right)$。

处理组成调制方案的大量信号波形时，差错概率 P_e 表示为**成对差错概率**之和——即一个接收符号被判成另一个特定符号的概率。发 $s_j(t)$ 而被译码为 $s_i(t)$ 的成对差错

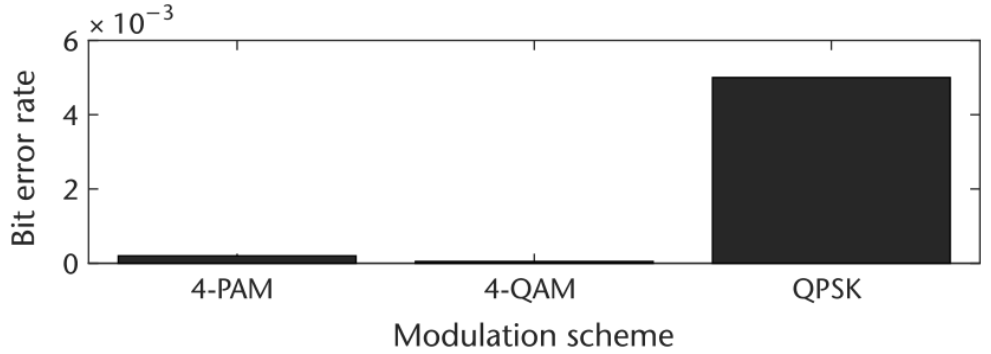


图 4.15　噪声对调制方案性能的影响

概率为

$$Q\left(\sqrt{\frac{d_{ij}^2}{2N_0}}\right),$$

其中 N_0 是噪声方差。重要提示：这里假设噪声是 AWGN，因为 Q 函数专门适用于高斯随机变量。于是 P_e 的完整表达式可表示为

$$Q\left(\sqrt{\frac{d_{\min}^2}{2N_0}}\right) \le P_e \le Q\left(\sqrt{\frac{d_{1j}^2}{2N_0}}\right) + \ldots + Q\left(\sqrt{\frac{d_{Mj}^2}{2N_0}}\right), \quad i \neq j,$$

其中右半部分是每个成对差错概率的求和。

动手 MATLAB 示例：我们已观察过含噪信道对接收星座的影响——对应特定符号的星座点可能被移到邻近星座点附近、面临被错误译码的风险。下面的 MATLAB 脚本把这些星座点的位移转化为实际 BER 值，以量化信道引入噪声的实际严重程度。代码采用**最近邻**方法在接收机译码：被噪声污染的接收星座点被映射到最近的原始发射星座点。用欧氏距离计算接收星座点到所有可能星座选项的距离、映射到最短者、再把符号译码为相应二进制字。

用该 MATLAB 脚本与欧氏距离最近邻判决，可以译码经含噪信道发来的消息。但既然存在噪声大到把符号移得离另一个星座点更近的情形，应预期会有若干符号被错误译码。图 4.15 给出 4-PAM、4-QAM 与 QPSK 最近邻译码方案的 BER 结果：前两种方案的差错量不大，QPSK 却有大量错误判决——原因在于星座点的间距：QPSK 星座点相对另两种方案更近。于是同样的噪声量下，QPSK 的表现明显更差。

代码 4.6　用欧氏距离译码上一示例的消息：chapter4.m

```
% Go through every received waveform and determine Euclidean distance
% between received waveform and the available waveforms
eucl_dist_4pam = zeros(4,len);
eucl_dist_4qam = zeros(4,len);
eucl_dist_qpsk = zeros(4,len);
for ind = 1:1:4,
    eucl_dist_4pam(ind,1:1:len) = abs(symb_4pam(ind).*ones(1,len)
                                        - rx_wavefm_4pam);
    eucl_dist_4qam(ind,1:1:len) = abs(symb_4qam(ind).*ones(1,len)
                                        - rx_wavefm_4qam);
    eucl_dist_qpsk(ind,1:1:len) = abs(symb_qpsk(ind).*ones(1,len)
                                        - rx_wavefm_qpsk);
end;

% Select shortest Euclidean distances
[mdist_4pam,min_ind_4pam] = min(eucl_dist_4pam);
[mdist_4qam,min_ind_4qam] = min(eucl_dist_4qam);
[mdist_qpsk,min_ind_qpsk] = min(eucl_dist_qpsk);

% Decode into estimated binary streams
bin_str_est_4pam = dec2bin(min_ind_4pam-ones(1,len)).';
bin_str_est_4qam = dec2bin(min_ind_4qam-ones(1,len)).';
bin_str_est_qpsk = dec2bin(min_ind_qpsk-ones(1,len)).';

% Calculate bit error rate
ber_4pam = sum([abs((bin_str_est_4pam(1,:)-'0') - bin_str2) ...
    abs((bin_str_est_4pam(2,:)-'0') - bin_str1)])/(2*len);
ber_4qam = sum([abs((bin_str_est_4qam(1,:)-'0') - bin_str2) ...
    abs((bin_str_est_4qam(2,:)-'0') - bin_str1)])/(2*len);
ber_qpsk = sum([abs((bin_str_est_qpsk(1,:)-'0') - bin_str2) ...
    abs((bin_str_est_qpsk(2,:)-'0') - bin_str1)])/(2*len);
```

4.3.1 差错界

逐个计算成对差错概率并不总是可行。可以只计算与关注点一度相邻的点的成对差错，从而建立 P_e 的上下界。考察 Q 函数 $Q(\cdot)$ 的行为：输入增大时输出趋近于零。计算更远点的成对差错概率对总 P_e 的贡献可忽略，却能节省大量时间与运算周期。于是 $P(e)$ 的准确估计可由下述界算得：

$$Q\left(\sqrt{\frac{d_{\min}^2}{2N_0}}\right) \le P(e) \le \sum_{i\in I} Q\left(\sqrt{\frac{d_{ij}^2}{2N_0}}\right),$$

其中 I 是星座中紧邻波形 j 的所有信号波形集合。要准确评估通信系统性能，必须仿真到确认一定数量的符号差错为止 [3]。多数情况下 100 个差错可给出 95% 置信区间——本书后面将用这一准则刻画被评估数字通信系统的误码率。

动手 MATLAB 示例：我们已观察过三种不同调制方案的简单通信系统在含噪环境下的性能。图 4.15 的结果虽有启发，但我们常想观察通信系统在大范围条件下的表现——尤其当噪声强度变化时。下面的 MATLAB 脚本考察简单二进制通信系统在一系列不同噪声程度的信道环境下的性能。

代码 4.7　用蒙特卡洛生成通信系统误码率瀑布曲线：chapter4.m

```
% Define parameters
len = 1000;    % Length of individual data transmission
N_snr = 9;     % Number of SNR values to evaluation
N_tx = 100;    % Number of transmissions per SNR
nvar = [(10.^((1:1:N_snr)/10)).^(-1)];  % Noise variance values

ber_data = zeros(N_snr,N_tx);
for ind = 1:1:N_snr,        % Different SNR values
  for ind1 = 1:1:N_tx,      % Different transmissions for same SNR value

     % Generate BPSK waveform (we will keep this the same for each
     % SNR value for now)
     tx_sig = 2*round(rand(1,len))-1;

     % Create additive noise
     noise_sig = sqrt(nvar(ind))*randn(1,len);
```

```
    % Create received (noisy) signal
    rx_sig = tx_sig + noise_sig;

    % Decode received signal back to binary
    decode_bin_str = zeros(1,len);
    decode_bin_str(find(rx_sig >= 0)) = 1;

    % Determine and store bit error rate
    ber_data(ind,ind1) = sum(abs(decode_bin_str - (tx_sig+1)/2))/len;
  end;
end;

% Calculate mean bit error rate and its standard deviation
mean_ber = mean(ber_data,2).';
std_ber = std(ber_data,'',2).';
```

这一分析（考察 AWGN 信道下的传输可靠性）的最终结果称为**瀑布曲线**，如图 4.16 所示。瀑布曲线被数字通信界的研究者与设计者广泛用于刻画含噪环境下通信系统的差错稳健性。之所以叫瀑布曲线，是因为无论用理论分析还是实验（计算机仿真或硬件测试）生成，曲线都呈瀑布形状。x 轴表示工作环境的 SNR（信道噪声量的度量）；y 轴表示误比特概率（损坏比特数与总发射比特数之比）。图 4.16 不仅给出平均 BER 曲线，还给出均值上下各一个标准差，以确立对曲线生成方法的置信度。由于用蒙特卡洛方法生成传输并以加性噪声损坏比特，必须确保实验做得足够长，得到的性能结果才可靠（例如跑一次实验只得到一个比特差错，对该系统 BER 的准确评估在统计上是不够的）。标准差曲线贴近平均 BER 曲线说明实验用了足够多的比特、获得了足够多的差错。注意不同 SNR 值下，同样的总比特数获得的差错数可能不同；因此常常要在高 SNR 值下发射更多比特以获得足够的误比特数。

4.4 信号空间概念

到目前为止，我们是从信号波形视角研究数字通信系统的，并由此发展了分析各调制方案功率效率与 BER 的数学工具。但有些场合波形框架会显得繁琐笨重。本节引入刻画与分析调制方案的另一种数学表示：**信号矢量**。

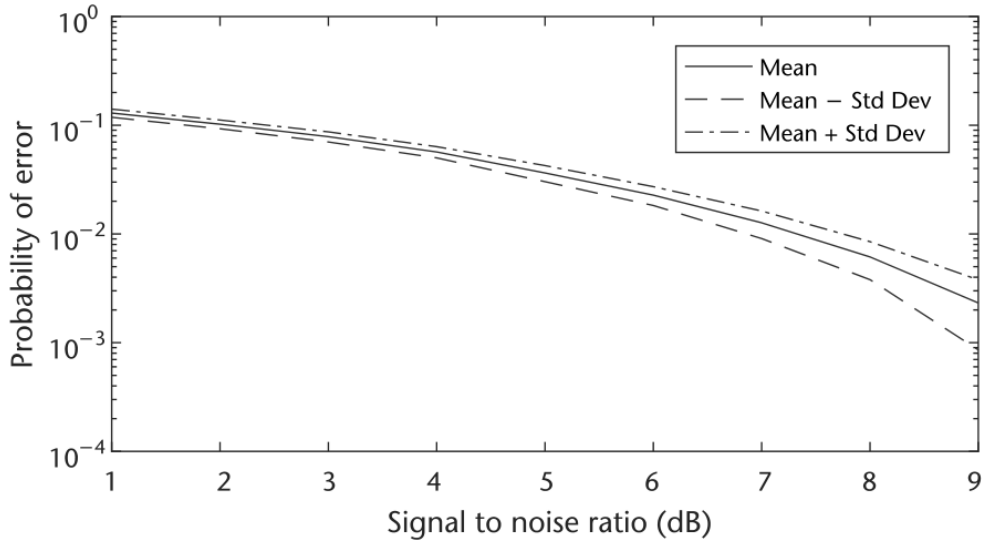


图 4.16　用蒙特卡洛仿真技术得到的 BPSK 通信系统误码率瀑布曲线示例

设 $\varphi_j(t)$ 是时间区间 $[0,T]$ 上的标准正交函数集，满足

$$\int_0^T \varphi_i(t)\varphi_j(t)\,dt = \begin{cases} 1 & i=j \\ 0 & \text{otherwise} \end{cases}$$

设 $s_i(t)$ 是第 i 个信号波形，希望把它表示为若干正交函数之和：

$$s_i(t) = \sum_{k=1}^{N} s_{ik}\varphi_k(t),$$

等价地表示为矢量

$$\mathbf{s}_i = (s_{i1}, s_{i2}, s_{i3}, \ldots s_{iN}),$$

其中矢量 $\mathbf{s}_i$ 的元素定义了用于表示波形的各正交函数的幅度缩放。图 4.17 给出了由三个正交函数表示的信号波形示例。既然信号波形可表示为正交函数的加权和，我们就能把信号波形描述为简单的矢量——接下来会看到，这带来” 可以用基于线性代数的直接数学运算” 的优势。

为求矢量元素 s_{il}，解表达式

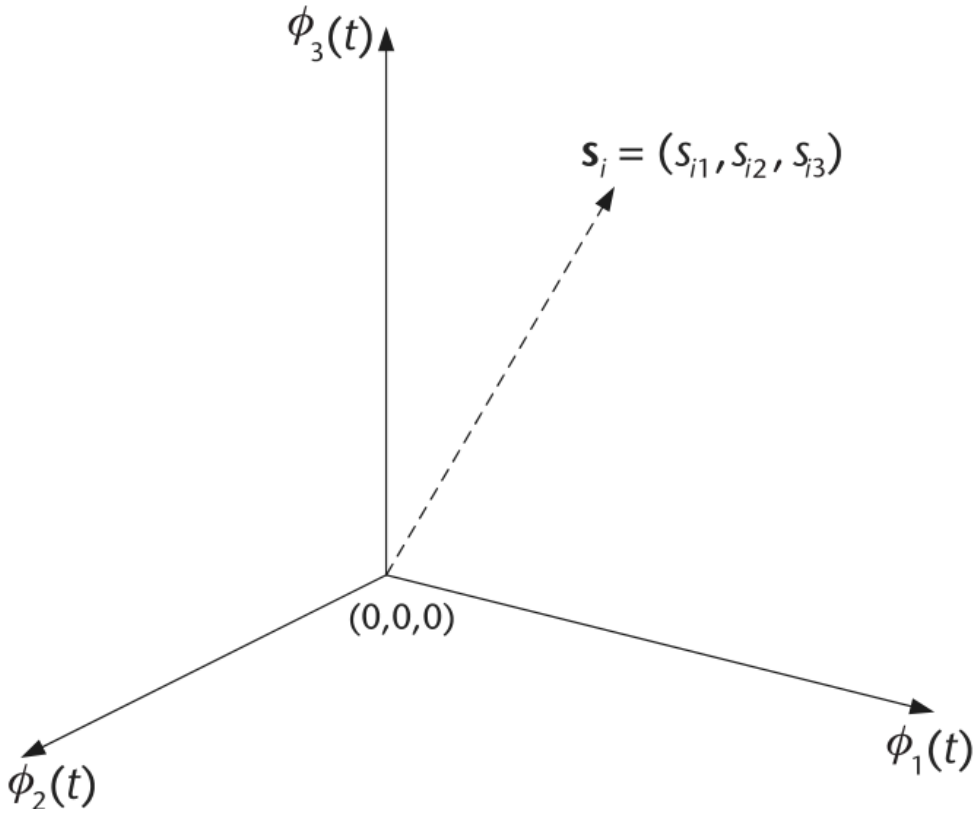


图 4.17　用基函数 φ1(t)、φ2(t)、φ3(t) 在三维空间中表示 si(t) 的矢量示例

$$\int_0^T s_i(t)\varphi_l(t)\,dt = \sum_{k=1}^{N} s_{ik}\int_0^T \varphi_k(t)\varphi_l(t)\,dt = s_{il},$$

这本质上是信号波形 $s_i(t)$ 在正交函数 $\varphi_l(t)$ 上的点积或投影。同时，对信号波形 $s_i(t)$ 与 $s_j(t)$ 做矢量点积得到相关运算：

$$\int_0^T s_i(t)s_j(t)\,dt = \mathbf{s}_i \cdot \mathbf{s}_j = \rho_{ij},$$

而信号 $s_i(t)$ 的能量等于

$$E_{s_i} = \int_0^T s_i^2(t)\,dt = \mathbf{s}_i \cdot \mathbf{s}_i = \|\mathbf{s}_i\|^2.$$

这些数学运算都将用于确定调制方案的功率效率或推导接收机的最优判决规则。

假设想用信号矢量方法（而非信号波形方法）计算调制方案的功率效率：第一步算

最小欧氏距离，可用下式求解：

$$d_{\min}^2 = \int_0^T \Delta s_{ij}^2(t)\,dt = \int_0^T (s_i(t)-s_j(t))^2 dt = \|\mathbf{s}_i-\mathbf{s}_j\|^2 = (\mathbf{s}_i-\mathbf{s}_j)\cdot(\mathbf{s}_i-\mathbf{s}_j) = E_{s_i}+E_{s_j}-2\rho_{ij},$$

其中信号波形 $s_i(t)$ 与 $s_j(t)$ 之间的相关项为

$$\rho_{ij} = \int_0^T s_i(t)s_j(t)\,dt = \mathbf{s}_i \cdot \mathbf{s}_j.$$

为解功率效率，选一组正交基函数 $\varphi_i(t)$（$i = 1, 2, \ldots, k$，k 为信号矢量空间维度）。有了这组函数即可表示矢量 $\mathbf{s}_i$（$i = 1, 2, \ldots, M$，$\mathbf{s}_i = (s_{i1}, s_{i2}, \ldots s_{ik})$），其中

$$s_{ij} = \int_0^T s_i(t)\varphi_j(t)\,dt.$$

于是，用信号与正交函数的矢量表示可计算：最小欧氏距离

$$d_{\min}^2 = \min_{i\neq j} \|\mathbf{s}_i - \mathbf{s}_j\|^2,$$

平均符号与比特能量

$$\overline{E}_s = \frac{1}{M}\sum_{i=1}^{M} \|\mathbf{s}_i\|^2, \qquad \overline{E}_b = \overline{E}_s/\log_2(M),$$

以及功率效率

$$\varepsilon_p = d_{\min}^2/\overline{E}_b.$$

4.5 Gram-Schmidt 正交化

在数学（尤其线性代数与数值分析）中，**Gram-Schmidt 正交化**是在内积空间（如欧氏空间 $\mathbb{R}^n$）中构造标准正交函数集的方法。Gram-Schmidt 过程取有限个信号波形 $\{s_1(t), \ldots, s_M(t)\}$，由它生成张成空间 $\mathbb{R}^n$ 的正交函数集 $\{\varphi_1(t), \ldots, \varphi_i(t)\}$。注意标准正交函数具有以下性质：

$$\int_0^T \varphi_i(t)\varphi_j(t)\,dt = \begin{cases} 1 & i = j \\ 0 & \text{otherwise} \end{cases}$$

而且信号波形 $s_i(t)$ 可以表示为这些正交基函数的加权和：

$$s_i(t) = \sum_{k=1}^{N} s_{ik}\varphi_k(t).$$

现在需要的是**生成**正交基函数集 $\{\varphi_j(t)\}$ 的方法。

为从信号波形集 $\{s_1(t), \ldots, s_M(t)\}$ 导出正交基函数集 $\{\varphi_1(t), \ldots, \varphi_i(t)\}$，先从 $s_1(t)$ 开始并归一化：

$$\varphi_1(t) = \frac{s_1(t)}{\sqrt{E_{s_1}}}$$

其中 E_{s_1} 是信号 $s_1(t)$ 的能量。$s_1(t)$ 的这个归一化版本将充当第一个正交基函数，其余的将在此基础上构造。换言之，我们是在用调制方案已有的信号波形” 自举” 出一组正交基。注意 $s_1(t)$ 可表示为

$$s_1(t) = \sqrt{E_{s_1}}\varphi_1(t) = s_{11}\varphi_1(t)$$

其中系数 $s_{11} = \sqrt{E_{s_1}}$，且 $\varphi_1(t)$ 满足单位能量约束。

接着构造第二个正交函数 $\varphi_2(t)$：以 $s_2(t)$ 为起点。但 $s_2(t)$ 可能含有 $\varphi_1(t)$ 的成分，须在归一化前先移除该分量。为此先取两函数点积确定 $s_2(t)$ 在 $\varphi_1(t)$ 上的投影有多大：

$$s_{21} = \int_0^T s_2(t)\varphi_1(t)\,dt.$$

为得到基函数 $\varphi_2(t)$，定义中间函数：

$$g_2(t) = s_2(t) - s_{21}\varphi_1(t),$$

它由于除去了 $\varphi_1(t)$ 分量，在 $0 \le t \le T$ 上与 $\varphi_1(t)$ 正交。最后归一化 $g_2(t)$ 得到基函数：

$$\varphi_2(t) = \frac{g_2(t)}{\sqrt{\int_0^T g_2^2(t)\,dt}} = \frac{s_2(t) - s_{21}\varphi_1(t)}{\sqrt{E_{s_2} - s_{21}^2}}$$

其中 E_{s_2} 是信号 $s_2(t)$ 的能量。快速检验可知 $\varphi_2(t)$ 满足约束

$$\int_0^T \varphi_2^2(t)\,dt = 1 \quad \text{且} \quad \int_0^T \varphi_1(t)\varphi_2(t)\,dt = 0.$$

一般地，可以用如下函数做迭代，生成一组正交基函数：

$$g_i(t) = s_i(t) - \sum_{j=1}^{i-1} s_{ij}\varphi_j(t), \qquad s_{ij} = \int_0^T s_i(t)\varphi_j(t)\,dt,\ j = 1, 2, \ldots, i-1$$

$$\varphi_i(t) = \frac{g_i(t)}{\sqrt{\int_0^T g_i^2(t)\,dt}}, \quad i = 1, 2, \ldots, N.$$

现在做一个 Gram-Schmidt 正交化的完整例子。

例：设要对图 4.18 中的信号按 $s_3(t), s_1(t), s_4(t), s_2(t)$ 的顺序执行 Gram-Schmidt 正交化，得到正交函数集 $\{\varphi_m(t)\}$。注意采用波形生成正交基的**顺序**非常重要，因为不同的顺序可能得到不同的正交基。

从 $s_3(t)$ 开始：

$$\varphi_1(t) = \frac{s_3(t)}{\sqrt{E_{s_3}}} = \frac{s_3(t)}{\sqrt{3}}.$$

再利用 $\varphi_1(t)$ 的结果，用 $s_1(t)$ 导出 $\varphi_2(t)$：

$$g_2(t) = s_1(t) - s_{12}\varphi_1(t) = s_1(t) - \frac{2}{3}s_3(t) = \begin{cases} 1/3, & 0 \le t < 2 \\ 2/3, & 2 \le t < 3 \\ 0, & t \ge 3 \end{cases}$$

$$\therefore\ \varphi_2(t) = \frac{g_2(t)}{\sqrt{\int_0^T g_2^2(t)\,dt}} = \begin{cases} 1/\sqrt{6}, & 0 \le t < 2 \\ 2/\sqrt{6}, & 2 \le t < 3 \\ 0, & t \ge 3 \end{cases}$$

随后对 $s_4(t)$ 重复该操作：

$$g_3(t) = s_4(t) - \sum_{j=1}^{2} s_{4j}\varphi_j(t) = 0, \qquad \therefore\ \varphi_3(t) = 0,$$

结果 $\varphi_3(t)$ 等于零——这意味着 $s_4(t)$ 完全可以由 $\varphi_1(t)$ 与 $\varphi_2(t)$ 刻画。最后对 $s_2(t)$：

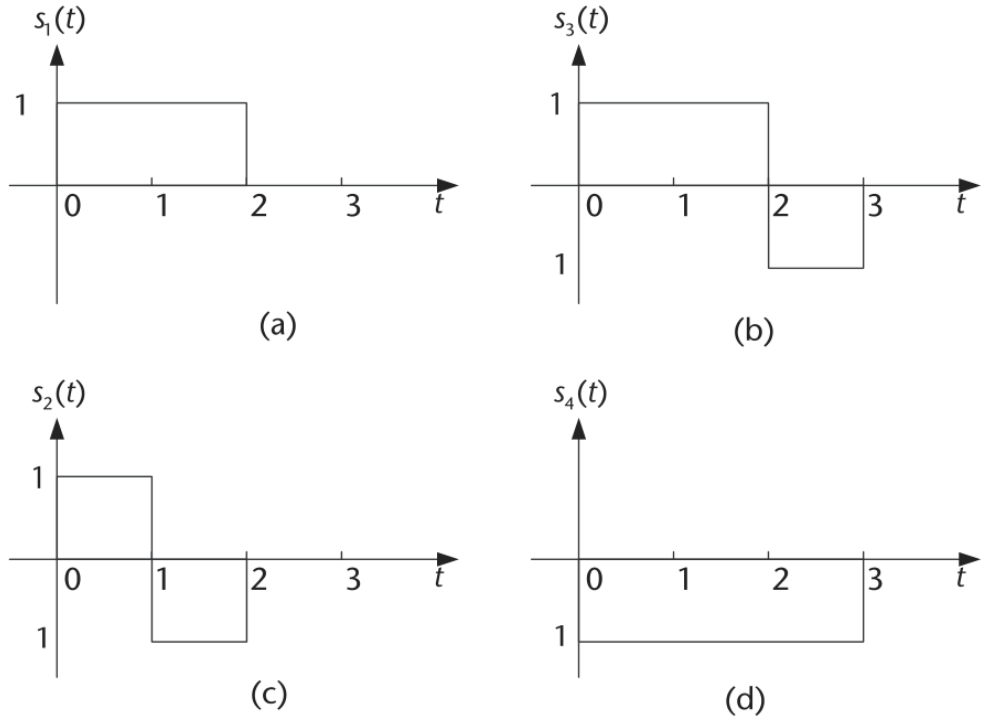


图 4.18　信号波形示例

$$g_4(t) = s_2(t) - \sum_{j=1}^{3} s_{2j}\varphi_j(t) = 0, \qquad \therefore\ \varphi_4(t) = \frac{g_4(t)}{\sqrt{\int_0^T g_4^2(t)\,dt}} = \frac{s_2(t)}{\sqrt{2}}.$$

于是，有了正交基函数 $\{\varphi_1(t), \varphi_2(t), \varphi_4(t)\}$，四个信号波形可表示为：

- $\mathbf{s}_1 = (2/\sqrt{3}, \sqrt{6}/3, 0)$,
- $\mathbf{s}_2 = (0, 0, \sqrt{2})$,
- $\mathbf{s}_3 = (\sqrt{3}, 0, 0)$,
- $\mathbf{s}_4 = (-1/\sqrt{3}, -4/\sqrt{6}, 0)$。

现在用下面的MATLAB脚本以正交基实现这些波形。脚本假设每单位时间有N_samp个采样时刻；由于每个波形持续 3 秒，MATLAB 模型中每个波形表示长 3*N_samp。

代码 4.8　Gram-Schmidt 正交化与矢量化：chapter4.m

```
% Define parameters
N_samp = 1000;  % Number of samples per time unit

% Create orthonormal basis functions
phi1 = [( 1/sqrt(3))*ones(1,N_samp) ...
    ( 1/sqrt(3))*ones(1,N_samp) ...
    (-1/sqrt(3))*ones(1,N_samp)];
phi2 = [( 1/sqrt(6))*ones(1,N_samp) ...
    ( 1/sqrt(6))*ones(1,N_samp) ...
    ( 2/sqrt(6))*ones(1,N_samp)];
phi3 = [0*ones(1,N_samp) 0*ones(1,N_samp) 0*ones(1,N_samp)];
phi4 = [( 1/sqrt(2))*ones(1,N_samp) ...
    (-1/sqrt(2))*ones(1,N_samp) ...
      0*ones(1,N_samp)];

% Based on these orthonormal basis functions, create the four symbol
% waveforms
sig_s1 = (2/sqrt(3))*phi1 + (sqrt(6)/3)*phi2 + 0*phi3 + 0*phi4;
sig_s2 = 0*phi1 + 0*phi2 + 0*phi3 + sqrt(2)*phi4;
sig_s3 = (sqrt(3))*phi1 + 0*phi2 + 0*phi3 + 0*phi4;
```

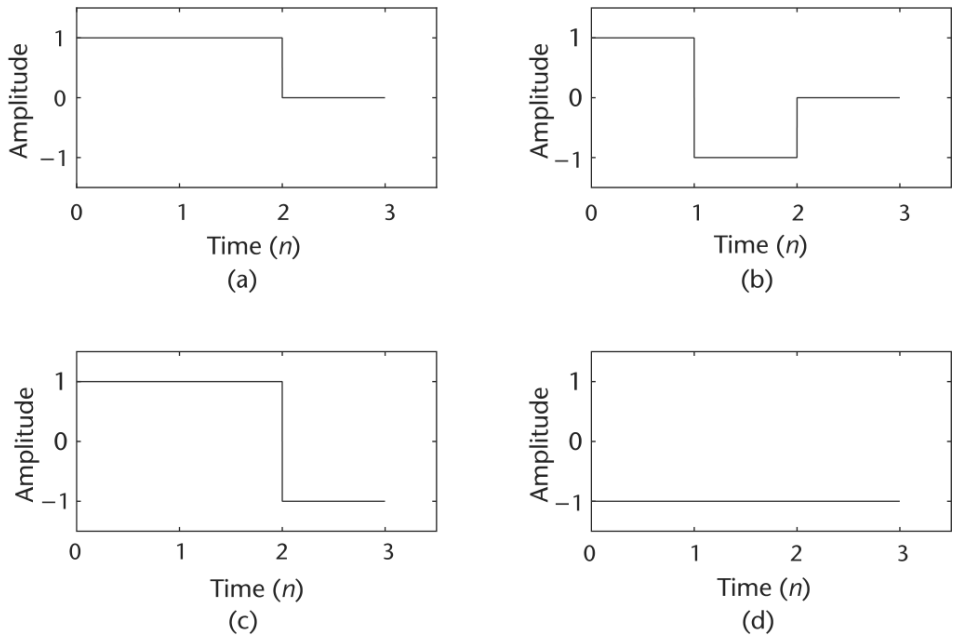


图 4.19　从一组正交基函数创建波形 (a) s1(n)、(b) s2(n)、(c) s3(n)、(d) s4(n)

```
sig_s4 = (-1/sqrt(3))*phi1 + (-4/sqrt(6))*phi2 + 0*phi3 + 0*phi4;
```

用这些正交基函数与 Gram-Schmidt 正交化的结果，该 MATLAB 脚本复现了图 4.18 所示的同样波形，如图 4.19 所示。

4.6 最优检测

检测理论（信号检测理论）用来分辨信号与噪声 [2]。用该理论可以解释改变判决门限如何影响分辨两种或多种情形的能力，常能揭示系统对其目标任务的适应程度。

4.6.1 信号矢量框架

设一个简单的数字收发信机模型如图 4.20 所示。如前所述，接收机只能观测到 $s_i(t)$ 被噪声信号 $n(t)$ 损坏后的版本 $r(t)$；$n(t)$ 通常代表所有噪声源汇聚成的单一变量。于是此时的检测问题可概括为：给定 $0 \leq t \leq T$ 的 $r(t)$，判断截获信号中出现的是哪个 $s_i(t)$ $(i = 1, 2, \ldots, M)$。

设把波形 $s_i(t)$、$n(t)$、$r(t)$ 分解为一组施加于正交基函数上的权重：

$$s_i(t)=\sum_{k=1}^{N}s_{ik}\varphi_k(t),\qquad r(t)=\sum_{k=1}^{N}r_k\varphi_k(t),\qquad n(t)=\sum_{k=1}^{N}n_k\varphi_k(t).$$

既然这些信号波形用同一组正交基函数，可把波形模型 $r(t)=s_i(t)+n(t)$ 改写为

$$\sum_{k=1}^{N}r_k\varphi_k(t)=\sum_{k=1}^{N}s_{ik}\varphi_k(t)+\sum_{k=1}^{N}n_k\varphi_k(t)\implies\mathbf{r}=\mathbf{s}_i+\mathbf{n}.$$

由于 **r** 由确定性波形 $\mathbf{s}_i$ 与概率性信号 **n** 组合而成，注意力转向数学刻画 **n**。噪声信号 $n(t)$ 假设为高斯随机变量，需要确定该随机变量的特性如何转化为信号矢量表示。噪声矢量元素 n_k 等于

$$n_k=\int_0^T n(t)\varphi_k(t)\,dt,$$

即噪声信号波形在正交基函数 $\varphi_k(t)$ 上的投影。由于 $n(t)$ 是高斯随机变量且积分是线性运算，n_k 也是高斯随机变量。于是噪声矢量 **n** 是高斯矢量。接着确定 **n** 的统计特性，用于信号波形检测。

先求矢量元素的均值。用期望的定义：

$$E\{n_k\}=E\left\{\int_0^T n(t)\varphi_k(t)\,dt\right\}=\int_0^T E\{n(t)\}\varphi_k(t)\,dt=0,$$

因为 $E\{n(t)\}=0$，最终意味着噪声矢量的均值 $E\{\mathbf{n}\}=\mathbf{0}$。

下一步求矢量元素的方差。设 $(\mathbf{nn}^T)_{kl}=n_kn_l$ 为 $\mathbf{nn}^T$ 的第 (k,l) 元。为确定 $E\{n_kn_l\}$（其中 $n_k=\int_0^T n(t)\varphi_k(t)dt$、$n_l=\int_0^T n(\rho)\varphi_l(\rho)d\rho$），应用期望定义：

$$E\{n_kn_l\}=E\left\{\int_0^T n(t)\varphi_k(t)dt\int_0^T n(\rho)\varphi_l(\rho)d\rho\right\}=\int_0^T\int_0^T E\{n(t)n(\rho)\}\varphi_k(t)\varphi_l(t)dtd\rho$$

求解得

$$E\{n_kn_l\}=\frac{N_0}{2}\int_0^T\int_0^T\delta(t-\rho)\varphi_k(t)\varphi_l(t)dtd\rho=\frac{N_0}{2}\int_0^T\varphi_k(t)\varphi_l(t)\,dt=\frac{N_0}{2}\delta(k-l),$$

其中两个正交函数乘积的积分给出 δ 函数——因为只有 $k=l$ 时两函数才相互投影。该结果的矩阵形式为

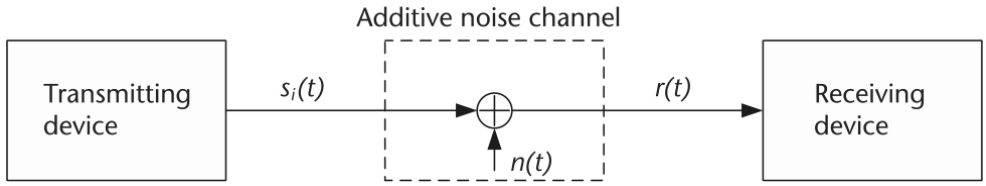


图 4.20　简单数字收发信机模型

$$E\{\mathbf{n}\mathbf{n}^T\} = \frac{N_0}{2}\mathbf{I}_{N\times N}. \tag{4.74}$$

有了 (4.74) 得到的高斯随机变量矢量表示，需要定义该表示的联合概率密度函数以刻画矢量的各元素。利用噪声元素相互独立的假设，联合 PDF 可表示为各元素 PDF 之积：

$$p(\mathbf{n}) = p(n_1, n_2, \ldots, n_N) = \frac{1}{(2\pi\sigma^2)^{N/2}} \prod_{i=1}^{N} e^{-n_i^2/2\sigma^2} = p(n_1)p(n_2)\ldots p(n_N),$$

其中 $p(n_i) = \frac{1}{\sigma\sqrt{2\pi}} e^{-n_i^2/2\sigma^2}$ 是矢量元素 n_i 的 PDF。由 $E\{n_k n_l\} = \frac{N_0}{2}\delta(k-l)$ 可解得 $E\{n_k^2\} = \frac{N_0}{2} = \sigma^2$。此外，矢量的点积可写成元素平方和：

$$\sum_{i=1}^{N} n_i^2 = \|\mathbf{n}\|^2,$$

于是联合概率密度函数可写成：

$$p(\mathbf{n}) = p(n_1, n_2, \ldots, n_N) = \frac{1}{(2\pi\sigma^2)^{N/2}} e^{-\|\mathbf{n}\|^2/2\sigma^2}. \tag{4.76}$$

4.6.2 判决规则

有了 (4.76) 的联合概率密度函数，可以定义接收机的规则，用于在信道引入噪声的情况下判断截获的是哪个信号波形。设接收机的准则为

$$\text{最小化} P(\text{error}) \to P(\hat{m}_i \neq m_i) \iff \text{最大化} P(\text{correct}) \to P(\hat{m}_i = m_i),$$

其中差错概率 $P(e) = P(\text{error})$、正确接收概率 $P(c) = P(\text{correct})$，且 $P(e) = 1 - P(c)$。

用全概率定律，总正确检测概率等于

$$P(c) = \int_V P(c|\mathbf{r}=\boldsymbol{\rho})p(\boldsymbol{\rho})\,d\boldsymbol{\rho},$$

其中 $P(c|\mathbf{r}=\boldsymbol{\rho}) \geq 0$ 且 $p(\boldsymbol{\rho}) \geq 0$。因此当 $P(c)$ 取最大值时，$P(c|\mathbf{r}=\boldsymbol{\rho})$ 也取最大值。为最大化 $P(c|\mathbf{r}=\boldsymbol{\rho})$，接收机采用如下判决规则：

$$P(\mathbf{s}_k|\boldsymbol{\rho}) \;\geq\; P(\mathbf{s}_i|\boldsymbol{\rho}), \quad i=1,2,\ldots,M \text{ 且 } i \neq k,$$

该判决规则假设 $\boldsymbol{\rho}$ 中存在 $\mathbf{s}_k$，即

$$\boldsymbol{\rho} = \mathbf{s}_k + \mathbf{n} \;\rightarrow\; \hat{m} = m_k.$$

采用由概率密度函数与概率组成的混合形式贝叶斯规则：

$$P(\mathbf{s}_i|\mathbf{r}=\boldsymbol{\rho}) = \frac{p(\boldsymbol{\rho}|\mathbf{s}_i)P(\mathbf{s}_i)}{p(\boldsymbol{\rho})},$$

希望把判决规则改写成接收机可用的形式。回忆本节前面要最大化 $P(c|\mathbf{r}=\boldsymbol{\rho})$：用混合贝叶斯规则，最优检测器可改写为

$$\max_{\mathbf{s}_i} P(\mathbf{s}_i|\mathbf{r}=\boldsymbol{\rho}) = \max_{\mathbf{s}_i} \frac{p(\boldsymbol{\rho}|\mathbf{s}_i)P(\mathbf{s}_i)}{p(\boldsymbol{\rho})}, \quad i=1,2,\ldots,M.$$

由于 $p(\boldsymbol{\rho})$ 不依赖 $\mathbf{s}_i$，可简化最优检测器表达式为

$$\max_{\mathbf{s}_i} p(\boldsymbol{\rho}|\mathbf{s}_i)P(\mathbf{s}_i), \quad i=1,2,\ldots,M.$$

基于该结果可导出两类检测器。第一类是 **MAP 检测器**：

$$P(\mathbf{s}_i|\mathbf{r}=\boldsymbol{\rho}) = \max_{\mathbf{s}_i} p(\boldsymbol{\rho}|\mathbf{s}_i)P(\mathbf{s}_i), \quad i=1,2,\ldots,M.$$

而当 $P(\mathbf{s}_i) = \frac{1}{M}$（即 $P(\mathbf{s}_i)$ 不依赖 $\mathbf{s}_i$）时，可从最优检测器表达式中省略 $P(\mathbf{s}_i)$ 项，得到第二类——**最大似然（ML）检测器**：

$$P(\mathbf{s}_i|\mathbf{r}=\boldsymbol{\rho}) = \max_{\mathbf{s}_i} p(\boldsymbol{\rho}|\mathbf{s}_i), \quad i=1,2,\ldots,M.$$

下一节将针对 AWGN 信道数据传输的最优判决规则，数学推导最大似然检测器。

4.6.3 AWGN 信道中的最大似然检测

最大似然检测是流行的统计方法，用于把统计模型拟合到数据、辨识模型参数。一般地，对固定数据与底层概率模型，最大似然方法选择" 最有可能产生观测数据" 的模型参数取值（即最大化似然函数的参数）。

设数据传输经 AWGN 信道到达接收机。回忆该场景的传输模型：

$$\mathbf{r} = \mathbf{s}_i + \mathbf{n},$$

其中 $\mathbf{s}_i$ 是发射机发的第 i 个信号波形，$\mathbf{n}$ 是 AWGN 信道引入的噪声，$\mathbf{r}$ 是接收机截获的信号波形。$\mathbf{s}_i$ 是确定量，$\mathbf{n}$ 是刚用联合 PDF 刻画过的随机实体，剩下的是刻画 $\mathbf{r}$——它可由 $\mathbf{n}$ 的刻画加上 $\mathbf{s}_i$ 的确定性导出。

设考虑接收矢量 $\mathbf{r} = \boldsymbol{\rho}$ 的单个元素（如第 k 个）在" 假设发的是 $\mathbf{s}_i$"条件下的条件概率：

$$p(\rho_k|s_{ik}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(\rho_k - s_{ik})^2/2\sigma^2},$$

其中噪声矢量的第 k 元等于 $n_k = \rho_k - s_{ik}$。由于假设 AWGN 矢量元素不相关（即独立），条件概率可改写为

$$p(\boldsymbol{\rho}|\mathbf{s}_i) = \prod_{k=1}^{N} p(\rho_k|s_{ik}), \quad i = 1, 2, \ldots, M.$$

这一多元概率密度函数之积最终给出：

$$p(\boldsymbol{\rho}|\mathbf{s}_i) = \frac{1}{(2\pi\sigma^2)^{N/2}} e^{-\|\boldsymbol{\rho} - \mathbf{s}_i\|^2/2\sigma^2}.$$

注意现在有了完全由 $\mathbf{s}_i$、$\boldsymbol{\rho}$ 及其元素表示的条件概率公式。借助该表达式即可数学地推导最大似然检测器。

为求 $\max_{\mathbf{s}_i} p(\boldsymbol{\rho}|\mathbf{s}_i)$，取 $p(\boldsymbol{\rho}|\mathbf{s}_i)$ 表达式应用于检测器，并取自然对数：

$$\ln(p(\boldsymbol{\rho}|\mathbf{s}_i)) = \ln\left(\frac{1}{(2\pi\sigma^2)^{N/2}}\right) - \frac{\|\boldsymbol{\rho} - \mathbf{s}_i\|^2}{2\sigma^2}.$$

采用自然对数是为了去掉表达式中的指数底，从而得到最优判决规则的线性表达式。而且自然对数是单调函数（若 $x_2 \geq x_1$ 则 $\ln(x_2) \geq \ln(x_1)$），使用不等式时判决规则依然有效。

解这一线性判决规则、利用自然对数的单调性，可推导：

$$\begin{aligned}\max_{\mathbf{s}_i} \ln(p(\boldsymbol{\rho}|\mathbf{s}_i)) &= \max_{\mathbf{s}_i}\left(\ln\frac{1}{(2\pi\sigma^2)^{N/2}} - \frac{\|\boldsymbol{\rho}-\mathbf{s}_i\|^2}{2\sigma^2}\right)\\ &= \max_{\mathbf{s}_i}\left(-\frac{\|\boldsymbol{\rho}-\mathbf{s}_i\|^2}{2\sigma^2}\right) = \max_{\mathbf{s}_i}\left(-\|\boldsymbol{\rho}-\mathbf{s}_i\|^2\right)\\ &= \min_{\mathbf{s}_i}\|\boldsymbol{\rho}-\mathbf{s}_i\|.\end{aligned}$$

由于我们关心的是使判决规则取最大值的 $\mathbf{s}_i$，可把判决规则改写为

$$\mathbf{s}_k = \arg\min_{\mathbf{s}_i}\|\boldsymbol{\rho}-\mathbf{s}_i\| \;\rightarrow\; \hat{m}=m.$$

注意采用矢量表示的优势之一：整个场景可以解读为**距离**。具体地，$\|\boldsymbol{\rho}-\mathbf{s}_i\|$ 实际代表两条矢量 $\boldsymbol{\rho}$ 与 $\mathbf{s}_i$（尾部都在原点）矢端之间的距离。于是，最大似然检测器等价于**最小距离检测器**。

4.7 基本接收机实现

数字通信的根本挑战是恢复" 经信道并被噪声损坏后" 的发射内容。我们将考察的第一种接收机结构基于：用使信道 SNR 最大化的静态滤波器滤波接收信号，从而最小化误码率。但匹配滤波方法的一个缺点是：需要发射机所有可能信号波形的先验知识。

> 思考题：参照图 4.21 的星座图，实现一个工作在 AWGN 信道的简单 QPSK 收发信机并实现最大似然检测器。该判决规则与图 4.21 的判决区域是否一致——即基于 ρ 落在哪个象限来充分地宣布 si 为发射信号？给定零均值噪声，不同方差值下 AWGN 信道的影响是什么？

4.7.1 匹配滤波器实现

设计接收机时，我们关心的判决规则是：接收机比任何其他可用规则更经常地给出正确结果。换言之，我们感兴趣的是检测经含噪信道传输的脉冲。

设采用如下传输模型：

$$x(t) = g(t) + w(t), \quad 0 \le t \le T,$$

其中 $g(t)$ 是脉冲信号，$w(t)$ 是均值为 $\mu=0$、功率谱密度为 $\frac{N_0}{2}$ 的白噪声过程，$x(t)$

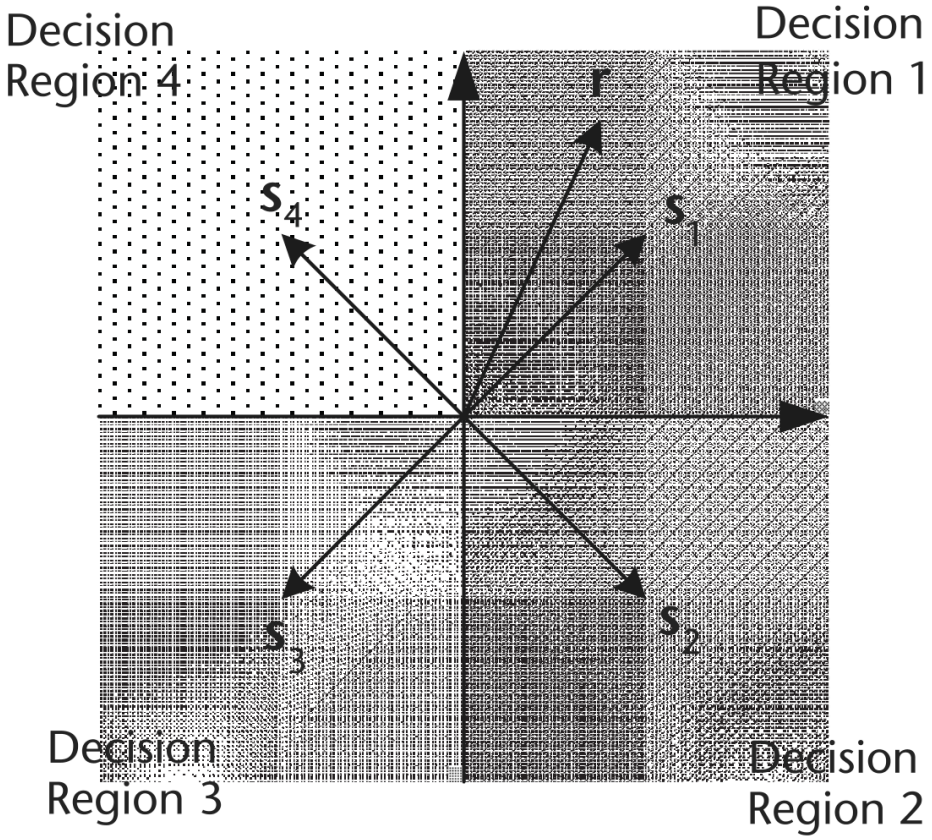


图 4.21　QPSK 信号星座的判决区域

是观测到的接收信号。假设接收机知道发射机所能产生的全部 $g(t)$ 波形，接收机的目标是基于观测 $x(t)$ 最优地检测脉冲信号 $g(t)$。注意 $g(t)$ 可以代表数字通信系统中的"1"或"0"。

为使接收机能在观测 $x(t)$ 下最优检测 $g(t)$，对 $x(t)$ 滤波，使噪声的影响在统计意义上最小化、正确检测概率增强。设用 $h(t)$ 滤波 $x(t)$，输出为

$$y(t) = g_0(t) + n(t),$$

其中 $n(t)$ 是 $w(t)$ 经 $h(t)$ 滤波的结果，$g_0(t)$ 是 $g(t)$ 经 $h(t)$ 滤波的版本。传输模型与 $h(t)$ 滤波操作示于图 4.22。

把该滤波操作改写到频域（时域卷积变频域乘积）。取 $H(f)G(f)$ 的傅里叶逆变换（等价于 $h(t)$ 与 $g(t)$ 的卷积），得到滤波后的 $g(t)$：

$$g_0(t) = \int_{-\infty}^{\infty} H(f)G(f)e^{j2\pi ft}\, df.$$

现在计算滤波信号的瞬时功率：

$$|g_0(t)|^2 = \left| \int_{-\infty}^{\infty} H(f)G(f)e^{j2\pi ft} df \right|^2.$$

为确定" 何时实现了 $g_0(t)$ 相对 $n(t)$ 最大化" 的定量指标，采用**峰值脉冲 SNR**：

$$\eta = \frac{|g_0(T)|^2}{E\{n^2(t)\}},$$

其中 $|g_0(T)|^2$ 是采样时刻 T 输出信号的瞬时功率，$E\{n^2(t)\}$ 是输出噪声的平均功率。于是匹配滤波器实现的目标是用峰值脉冲 SNR 指标最大化 $g_0(t)$ 相对 $n(t)$——通过设计能给出最大 η 的滤波器 $h(t)$ 实现。

为设计 $h(t)$，需要数学求解 $h(t)$，即计算表达式

$$|g_0(t)|^2 = \left| \int_{-\infty}^{\infty} H(f)G(f)e^{j2\pi ft} df \right|^2,$$

它是 $H(f)G(f) = \mathcal{F}\{h(t) * g(t)\}$ 傅里叶逆变换的幅度平方。由于 $w(t)$ 是功率谱密度 $\frac{N_0}{2}$ 的白高斯过程，由 EWK 定理知滤波噪声的 PSD 为 $S_N(f) = \frac{N_0}{2}|H(f)|^2$。于是应用 η 的定义并代入这些表达式得

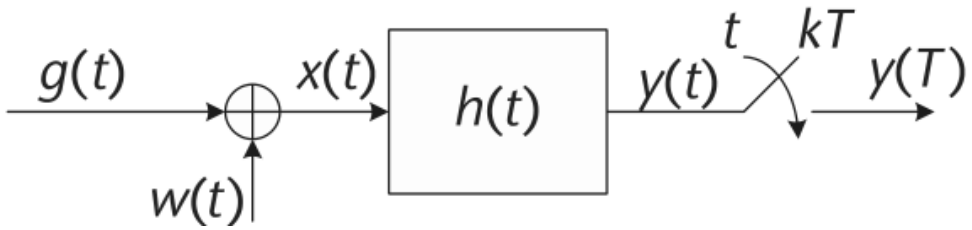


图 4.22　检测 g(t) 的滤波过程

$$\eta = \frac{\left|\int_{-\infty}^{\infty} H(f)G(f)e^{j2\pi fT}df\right|^2}{\frac{N_0}{2}\int_{-\infty}^{\infty}|H(f)|^2df}. \tag{4.99}$$

从该式可见需要求解使峰值脉冲 SNR η 最大的频率响应 $H(f)$。为得闭式解，采用施瓦茨不等式。设有两个复函数 $\varphi_1(x)$ 与 $\varphi_2(x)$ 满足：

$$\int_{-\infty}^{\infty}|\varphi_1(x)|^2dx < \infty \quad \text{且} \quad \int_{-\infty}^{\infty}|\varphi_2(x)|^2dx < \infty.$$

则由施瓦茨不等式，积分表达式可改写为不等式：

$$\left|\int_{-\infty}^{\infty}\varphi_1(x)\varphi_2(x)dx\right|^2 \le \left(\int_{-\infty}^{\infty}|\varphi_1(x)|^2dx\right)\cdot\left(\int_{-\infty}^{\infty}|\varphi_2(x)|^2dx\right),$$

当 $\varphi_1(x) = K\cdot\varphi_2^*(x)$ 时取等。于是，在峰值脉冲 SNR 表达式中利用施瓦茨不等式，(4.99) 的分子可改写为

$$\left|\int_{-\infty}^{\infty}H(f)G(f)e^{j2\pi ft}df\right|^2 \le \left(\int_{-\infty}^{\infty}|H(f)|^2df\right)\cdot\left(\int_{-\infty}^{\infty}|G(f)|^2df\right),$$

进而得到 η 的不等式：

$$\eta \le \frac{2}{N_0}\int_{-\infty}^{\infty}|G(f)|^2df.$$

于是，为使该式取等，$H(f)$ 的最优值应为

$$H_{\text{opt}}(f) = K\cdot G^*(f)e^{-j2\pi fT},$$

其时域表示可用傅里叶逆变换确定：

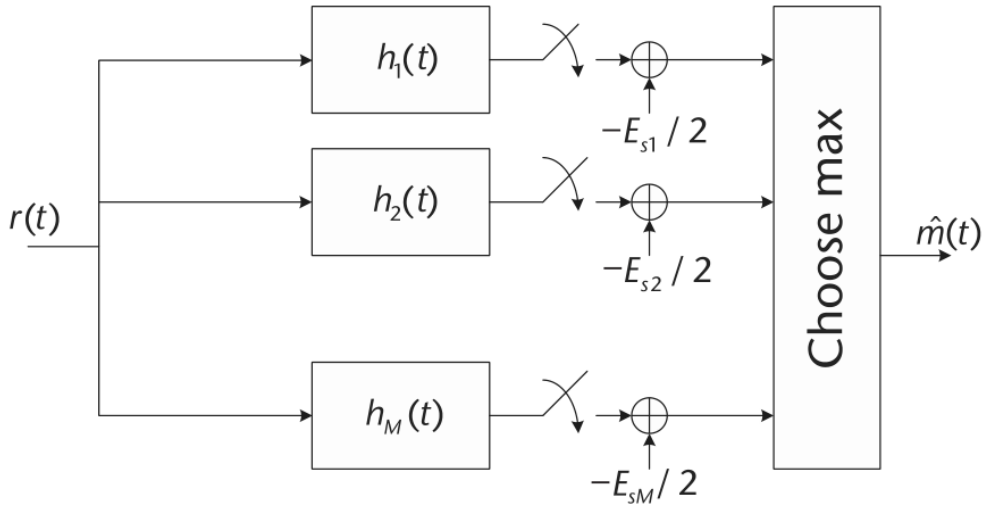


图 4.23　接收机结构的匹配滤波器实现示意图

$$h_{\text{opt}}(t) = K \cdot \int_{-\infty}^{\infty} G^*(f) e^{-j2\pi fT} e^{-j2\pi ft} df = K \cdot g(T-t).$$

注意：执行匹配滤波时，我们把发射脉冲的”时间翻转加时移”版本与发射脉冲本身卷积，以最大化 SNR。

> 注：之所以称为匹配滤波器，是因为把发射脉冲信号的时间翻转加时移版本与其自身卷积的过程是 SNR 最大化的。于是，接收机截获到某个未知的含噪信号时，可以用图 4.23 所示的实现，把截获信号与接收机已知的所有可用信号波形逐一”匹配”，方便地识别出发的是哪一个。

> 思考题：参照图 4.24，设信号 g(t) 给定。证明 h(t) 与 g0(t) 是相应的匹配滤波器与滤波输出信号。

4.7.2 相关器实现

回忆匹配滤波器实现假设对发射数据有某种先验知识。但若接收机掌握发射机及其信号特性的信息，即使存在引入损坏的含噪信道，也可以用更统计化的方法确定发送了哪些信号波形。具体地，可以采用**相关**的概念——这样只需假设对波形本身的知识（对匹配滤波器实现而言，接收机需要同时知道传输信号波形与信道引入噪声的统计特性）。

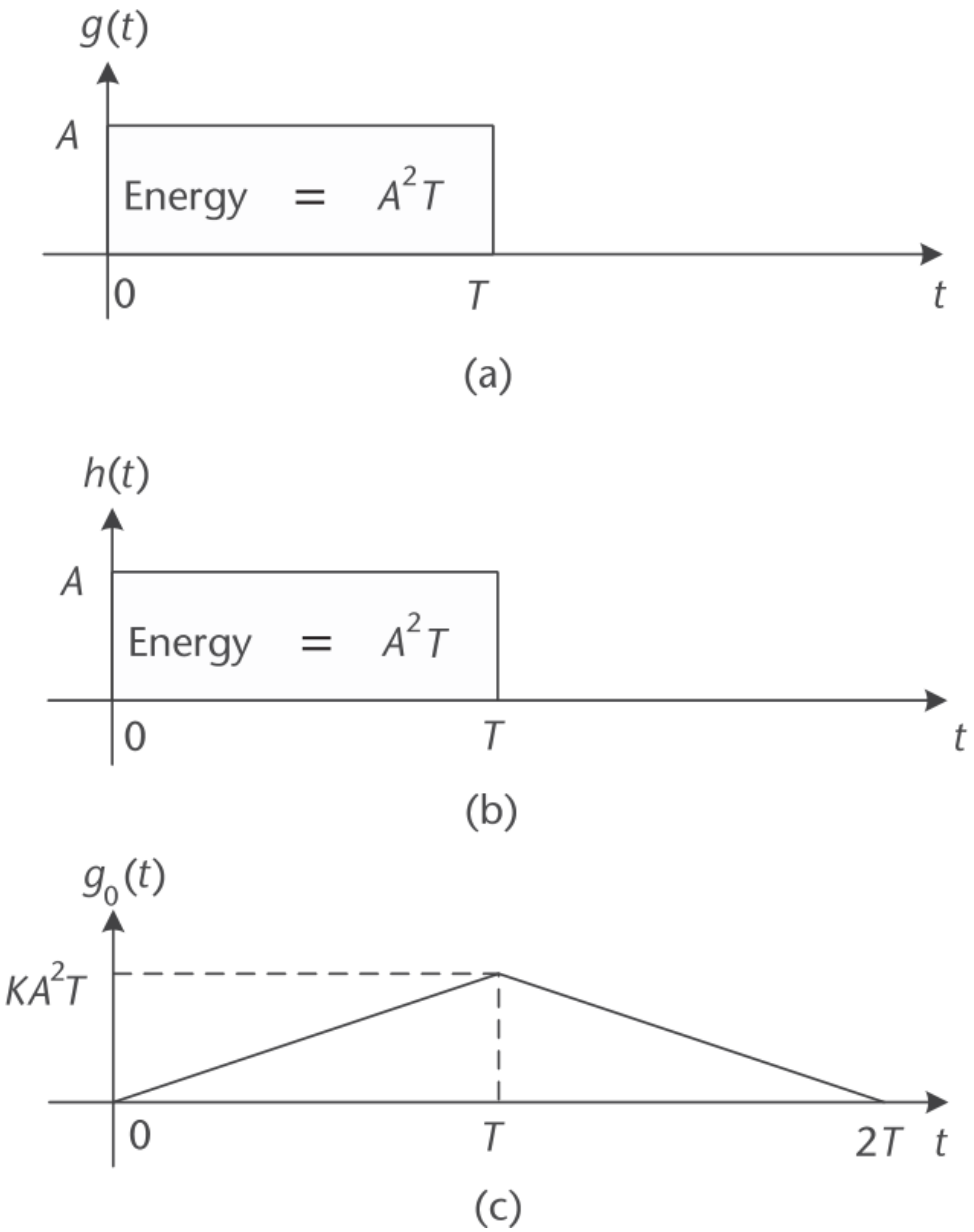


图 4.24　匹配滤波器处理时域输入输出信号波形的示例：(a) 匹配滤波器输入信号的时域表示；(b) 匹配滤波器的时域冲激响应；(c) 匹配滤波器输出信号的时域表示

从本节开头导出的判决规则出发并展开：

$$\min_{\mathbf{s}_i} \|\boldsymbol{\rho} - \mathbf{s}_i\|^2 = \min_{\mathbf{s}_i} (\boldsymbol{\rho} - \mathbf{s}_i) \cdot (\boldsymbol{\rho} - \mathbf{s}_i) = \boldsymbol{\rho} \cdot \boldsymbol{\rho} - 2\boldsymbol{\rho} \cdot \mathbf{s}_i + \mathbf{s}_i \cdot \mathbf{s}_i.$$

由于 $\boldsymbol{\rho} \cdot \boldsymbol{\rho}$ 对不同信号波形 $\mathbf{s}_i$ 的判决度量是公共项，可方便地从表达式中略去，得

$$\min_{\mathbf{s}_i} \left(-2\boldsymbol{\rho} \cdot \mathbf{s}_i + \mathbf{s}_i \cdot \mathbf{s}_i\right) = \max_{\mathbf{s}_i} \left(2\boldsymbol{\rho} \cdot \mathbf{s}_i - \mathbf{s}_i \cdot \mathbf{s}_i\right),$$

其中 $\boldsymbol{\rho} \cdot \mathbf{s}_i$ 与 $\mathbf{s}_i \cdot \mathbf{s}_i$ 定义为

$$\boldsymbol{\rho} \cdot \mathbf{s}_i = \int_0^T \rho(t) s_i(t) dt \qquad \mathbf{s}_i \cdot \mathbf{s}_i = \int_0^T s_i^2(t) dt = E_{s_i}.$$

可以看到 $\boldsymbol{\rho} \cdot \mathbf{s}_i$ 的波形表示等于 $r(t) = \rho(t)$ 相对 $s_i(t)$ 的相关。于是当 $r(t)$ 中存在 $s_k(t)$ 时，最优检测器等于

$$\mathbf{s}_k = \arg\max_{\mathbf{s}_i} \left(\int_0^T \rho(t) s_i(t) dt - \frac{E_{s_i}}{2} \right).$$

基于该结果，可以设计利用相关的接收机结构，基于接收机的观测截获信号判断发射机发的是哪个信号波形。基于相关的实现示意图见图 4.25。给定 $r(t) = s_i(t) + n(t)$ 且接收机输入端只观测到 $r(t) = \rho(t)$：先把 $r(t)$ 与所有 $s_i(t)$ 相关；再用相应信号能量 E_{s_i} 归一化相关结果以促成公平比较（注意若每个可能信号波形的能量都相同，可省去能量归一化——它不影响判决）；最后比较各支路得到的判决值、选择值最大的支路。

动手 MATLAB 示例：为突出相关器接收机结构如何译码截获波形并转换为相应二进制输出，可用下面的 MATLAB 脚本：先生成由符号 $s_1(n), s_2(n), s_3(n), s_4(n)$ 组成的随机波形流（这些波形来自 4.5 节的 MATLAB 脚本）。生成后，下一步把波形矢量化到三维信号星座空间。矢量化后用相关接收机方法求接收矢量化波形与可用信号矢量之间的欧氏距离。算出每个接收波形的距离后，执行判决过程找出接收符号矢量与可用符号矢量的最接近匹配。

该波形矢量化与基于相关器的接收机设计的结果示于图 4.26：原始发射波形与译码波形完美匹配。但本模型没有包含噪声等任何形式的失真。因此，把加性高斯白噪声引入接收信号、观察相关器接收机的表现，是一个很好的练习。

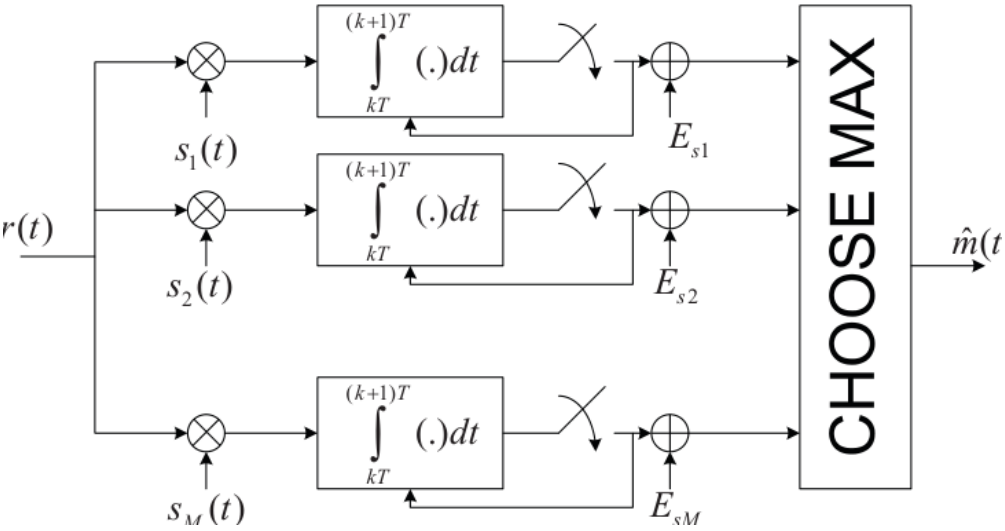


图 4.25　假设完美同步的相关器接收机结构

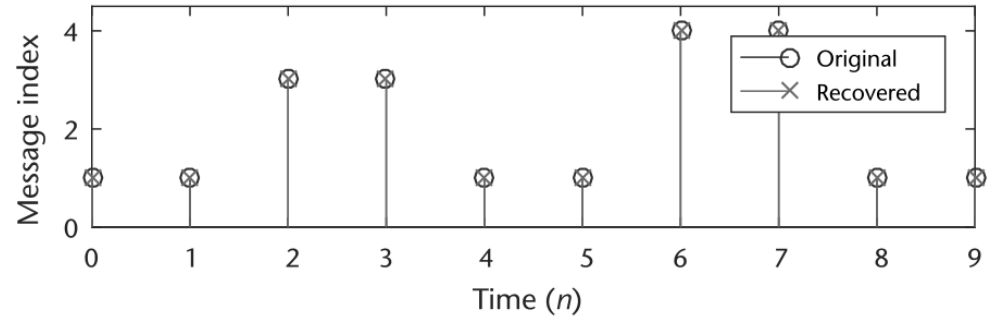


图 4.26　相关器接收机输出与原始传输的匹配

4.8 本章小结

设计与评估软件定义无线电实现时，深入透彻地掌握数字通信理论至关重要。本章概述了若干有用的主题，包括几种调制方案、差错概率推导、Gram-Schmidt 正交化、最优判决规则的建立以及两种接收机结构的呈现，为读者掌握这些既通用又复杂的系统打下基础。后续章节将大量运用本章与前两章获得的基础知识，实现基于软件定义无线电技术的数字通信系统与网络。

代码 4.9　用 Gram-Schmidt 实现基于相关器的接收机：chapter4.m

```
% Define parameters
N_symb = 10;  % Number of symbols contained within intercepted signal
```

```
% Randomly generate intercepted waveform consisting of s1(n), s2(n),
% s3(n), and s4(n)
rx_sig = [];
orig_msg = [];
for ind = 1:1:N_symb,
    rnd_val = rand(1,1);
    if (rnd_val < 0.25)  % Add s1(n) waveform
        rx_sig = [rx_sig sig_s1];
        orig_msg = [orig_msg 1];
    elseif ((rnd_val >= 0.25)&&(rnd_val < 0.5))  % Add s2(n) waveform
        rx_sig = [rx_sig sig_s2];
        orig_msg = [orig_msg 2];
    elseif ((rnd_val >= 0.5)&&(rnd_val < 0.75))  % Add s3(n) waveform
        rx_sig = [rx_sig sig_s3];
        orig_msg = [orig_msg 3];
    else  % Add s4(n) waveform
        rx_sig = [rx_sig sig_s4];
        orig_msg = [orig_msg 4];
    end;
end;

% Vectorize the intercepted signal
dim1_comp = [];
dim2_comp = [];
dim4_comp = [];
for ind = 1:1:N_symb,
    dim1_comp = [dim1_comp sum(rx_sig(((ind-1)*3*N_samp+1):1:
                           (ind*3*N_samp)).*phi1)];
    dim2_comp = [dim2_comp sum(rx_sig(((ind-1)*3*N_samp+1):1:
                           (ind*3*N_samp)).*phi2)];
    dim4_comp = [dim4_comp sum(rx_sig(((ind-1)*3*N_samp+1):1:
                           (ind*3*N_samp)).*phi4)];
end;
```

```
dim1_comp = dim1_comp/N_samp;
dim2_comp = dim2_comp/N_samp;
dim4_comp = dim4_comp/N_samp;

% Using the correlator receiver approach, we determine the closest
% symbol vector to each vectorized waveform based on Euclidean distance
s1vec = [(2/sqrt(3)) (sqrt(6)/3) 0 0];
s2vec = [0 0 0 sqrt(2)];
s3vec = [(sqrt(3)) 0 0 0];
s4vec = [(-1/sqrt(3)) (-4/sqrt(6)) 0 0];
est_msg = [];
for ind = 1:1:N_symb,
  [val,symb_ind] = min([ ...
  sum((s1vec - [dim1_comp(ind) dim2_comp(ind) 0 dim4_comp(ind)]).^2) ...
  sum((s2vec - [dim1_comp(ind) dim2_comp(ind) 0 dim4_comp(ind)]).^2) ...
  sum((s3vec - [dim1_comp(ind) dim2_comp(ind) 0 dim4_comp(ind)]).^2) ...
  sum((s4vec - [dim1_comp(ind) dim2_comp(ind) 0 dim4_comp(ind)]).^2) ...
    ]);
  est_msg = [est_msg symb_ind];
end;
```

4.9 延伸阅读

鉴于本章对数字通信主题的入门性质，非常鼓励有兴趣的读者探索对该主题有更详尽、更深入论述的大量书籍。例如 Proakis 与 Salehi 的权威数字通信书最新版 [4] 对本章许多概念给出了严格的数学处理，还有扩频技术、均衡、RAKE 接收机实现等许多未涉及的主题。为配合这本数学上简练的著作，作者还出版了从更应用的角度论述数字通信的配套书，含 MATLAB 与 Simulink 例程 [5]。

数字通信入门教材方面，Sklar 写了一本平衡数学严谨性、详细讲解与精心制作例子的优秀著作 [6]。Couch 的书与 Sklar 同类，但兼顾模拟与数字通信 [7]，适合没有通信背景的读者。Rice 的数字通信入门书从离散时间视角撰写 [8]，适合具有离散时间信号与系统背景的读者；该书还提供大量章末习题与在线 MATLAB 示例，给读者许多实践书中理论概念的机会。Barry、Messerschmitt 与 Lee 的经典数字通信书 [9] 对已有一定理

解、需要便捷查阅详细信息的读者是优秀参考。类似地，Madhow [10] 与 Pursley [11] 的书都提供了更深入的数字通信理论。最后，Hsu 的书 [12] 是一本以大量已解习题为主的优秀参考。

4.10 参考文献

[1] Shannon, C. E., “Communication in the Presence of Noise,” in *Proceedings of the Institute of Radio Engineers*, Vol. 37, No. 1, January 1949, p. 1021.

[2] Poor, H. V., *An Introduction to Signal Detection and Estimation*, New York: Springer, 2010.

[3] Wyglinski, A. M., *Physical Layer Loading Algorithms for Indoor Wireless Multicarrier Systems*, Ph.D. thesis, McGill University, Montreal, 2004.

[4] Proakis, J., and M. Salehi, *Digital Communications*, Fifth Edition, Boston: McGraw-Hill, 2007.

[5] Proakis, J. G., M. Salehi, and G. Bauch, *Contemporary Communication Systems Using MATLAB*, Second Edition, Brooks/Cole, 2003.

[6] Sklar, B., *Digital Communications: Fundamentals and Applications*, Second Edition, Prentice Hall PTR, 2001.

[7] Couch, L. W., *Digital and Analog Communication Systems*, Seventh Edition, Upper Saddle River: Prentice Hall, 2006.

[8] Rice, M., *Digital Communications: A Discrete-Time Approach*, Third Edition, Pearson-/Prentice Hall, 2009.

[9] Barry, J. R., D. G. Messerschmitt, and E. A. Lee, *Digital Communication*, Third Edition, Norwell, MA: Kluwer Academic Publishers, 2003.

[10] Madhow, U., *Fundamentals of Digital Communication*, Cambridge, UK: Cambridge University Press, 2008.

[11] Pursley, M. B., *Introduction to Digital Communications*, Prentice Hall, 2004.

[12] Hsu, H. P., *Schaum’s Outline of Theory and Problems of Analog and Digital Communications*, Second Edition, New York: McGraw-Hill, 2002.

第 5 章 理解 SDR 硬件

本章讨论使用 SDR 硬件的现实问题，以及从 MATLAB 与 Pluto SDR 交互的基础。以 Pluto SDR 为模板，我们将介绍接收与发射链路，讨论模拟波形如何在 MATLAB 中变成数字采样。掌握这一过程后，将介绍一套通用代码模板——本书余下章节在 MATLAB 中操作电台时都会用到。这套模板提供简化的工作流，能帮助缓解使用 SDR（尤其是 Pluto SDR）时的常见问题。最后以一个小示例收尾，确认 Pluto SDR 已与 MATLAB 正确配置。

5.1 通信系统的组成

5.1.1 节描述的软件定义无线电，既可充当点对点链路中的一个无线电节点，也可以是大型自组织无线电网中的一个单元。它既可以作为 MATLAB 脚本或 Simulink 模型的 RFFE，也可以被编程为一台完整的独立电台。射频前端——此处即 Pluto SDR——只是更大通信系统中的一个组件，系统通常还包括外部滤波器与频段专用天线。通信系统的描述与框图示于图 5.1(c)。其主要方面为：

- 模拟射频部分（天线、射频滤波器、输入复用器、LNA、增益、衰减、混频器）；
- 模拟基带部分（模拟滤波器、ADC 或 DAC）；
- 若干信号处理单元（收发信机内部的固定滤波器，或 FPGA/DSP 内的用户自定义处理，或通用处理器）。

虽然 Pluto SDR 为 STEM 教育与 SDR 实验提供了一个出色的低成本平台，它也代表了通信系统中使用的许多 SDR。尽管小巧廉价，Pluto SDR 的能力足以应对广泛的 SDR 应用，例如 GPS 或气象卫星接收机、自组织个人区域网。Pluto SDR 按如下方式扮演上述通信系统的角色：

- 模拟射频部分（天线、射频滤波器、输入复用器、LNA、增益、衰减、混频器）
  - 天线与射频滤波器应在 Pluto SDR 之外完成，由最终用户负责；

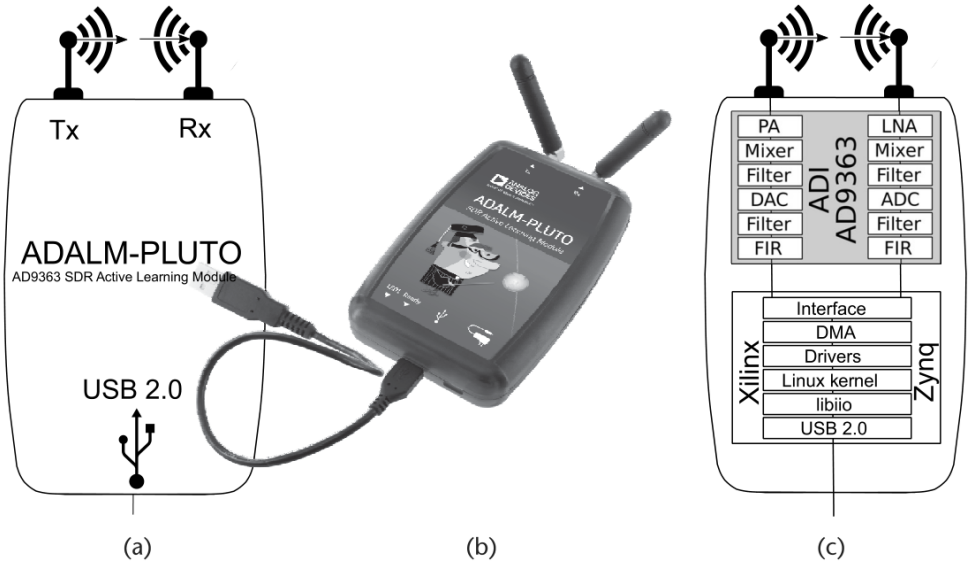


图 5.1　ADALM-PLUTO SDR 的各个视图：(a) 简化框图；(b) 实物照片 [1]；(c) I/O 接口

  - 第一条所列的其余部分（输入复用器、LNA、增益、衰减、混频器）都实现在 AD9363 集成射频敏捷收发信机中；
- 模拟基带部分（模拟滤波器、ADC/DAC）实现在 AD9363 中；
- 信号处理：分三处——
  - 部分信号处理实现在 AD9363 中，包括固定的半带抽取/插值滤波器与可编程 128 抽头 FIR 滤波器；
  - 可选的滤波与抽取可在 Xilinx Zynq 的 FPGA 逻辑中完成；
  - I/Q 数据随后经 USB 上传到主机，由 MATLAB 继续信号处理。

要理解这些部件的细节，需要” 剥开” Pluto SDR 的塑料外壳、看看组成设备的芯片。作为专门面向学生的 SDR 平台，Pluto SDR 不仅原理图随手可得，整个软件与 HDL 栈也是公开的——从硬件到软件的任何层面我们都可以细看设备的构成。

5.1.1 SDR 的组成

多数现代 SDR 器件共享相似的结构设计，构成接收和/或发射链路，把数据从模拟射频域搬到模拟基带域、变成 I/Q 采样，最终进入 MATLAB 这样的软件包。最简单地说，Pluto SDR（见图 5.1(b)）由两个组件构成，如图 5.1(a) 所示：

- 模拟射频部分（决定接收与发射能力）；
- 把 I/Q 数据送回主机处理的通信机制（以太网、USB）。

参照图 5.1(c)，ADALM-PLUTO 的接收、发射与通信规格包括：

- 发射（标注 Tx 的 SMA 连接器）
  - 300 MHz–3.8 GHz，200 kHz–20 MHz 信道带宽，65.1–61,440 kSPS
  - 2.4 Hz 本振步进、5 Hz 采样率步进
  - 调制精度（EVM）：40 dB（典型值，非逐台测量）
  - 12 位 DAC
- 接收（标注 Rx 的 SMA 连接器）
  - 300 MHz–3.8 GHz，200 kHz–20 MHz 信道带宽，65.1–61,440 kSPS
  - 2.4 Hz 本振步进、5 Hz 采样率步进
  - 调制精度（EVM）：40 dB（典型值，非逐台测量）
  - 12 位 ADC
- USB 2 OTG（480 Mbit/s），设备模式
  - libiio USB 类，用于在射频器件与主机间传输 I/Q 数据
  - 网络设备，提供对 Pluto 上 Linux 的访问
  - USB 串行设备，提供对 Pluto 上 Linux 控制台的访问
  - 大容量存储设备
- USB 2 OTG（480 Mbit/s），主机模式
  - 大容量存储设备：插上 U 盘即可采集或回放波形
  - WiFi 适配器：经 WiFi 访问 Pluto SDR
  - 有线局域网：经有线 LAN 访问 Pluto SDR
- 外部供电口：Pluto SDR 工作在主机模式时使用。

可以让 Pluto SDR 超规格运行、把频率范围扩展到 70–6,000 MHz：低端可以接收收听 FM 广播（多数地区 87.5–108.0 MHz，日本 76–95 MHz，部分东欧国家还有老的 65.8–74.0 MHz），高端覆盖全球 5.8 GHz ISM 频段的各种有趣信号。

正因为 70–6,000 MHz 的宽调谐范围跨越三个数量级以上，Pluto SDR 内没有频段专用的接收或发射滤波器。这意味着从接收侧看，70–6,000 MHz 之间一切在广播的信号都会被拾取、都可能影响你的信号。通常只有在试图接收极低幅度信号时这才构成问题。更多见 5.2.6 节。

5.1.2 AD9363 细节

Pluto SDR 的前端是 Analog Devices 的 AD9363 射频收发信机，负责射频数据的捕获与数字化。该收发信机提供放大、频率搬移（混频）、数字转换以及对发射与接收信号的滤波。图 5.2 给出了 AD9363 的详细框图。虽然对初学者看起来复杂，它无非是我们前

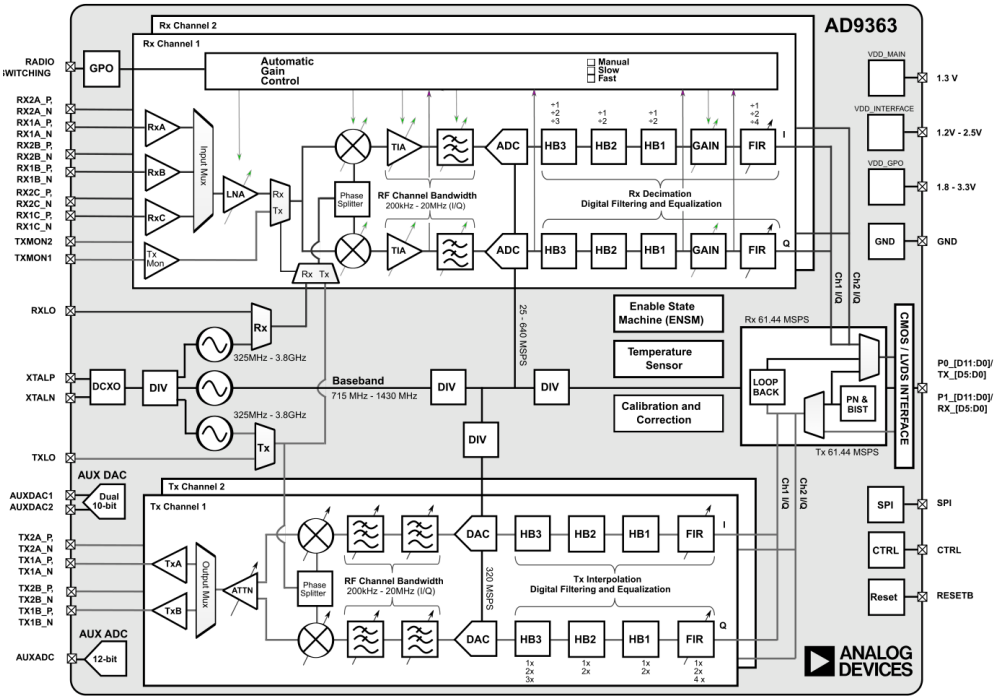


图 5.2　AD9363 框图 [3]

面提到的三部分：模拟射频部分、模拟基带部分、以及收发两用的信号处理。理解物理上模拟与数字硬件的边界很重要——这是从 MATLAB 配置器件、理解收发数据中非理想现象的必要知识。不过，有效使用 SDR 并不需要对器件钻到极致，掌握一些基础就弥足珍贵。

我们从接收机的视角讨论 AD9363；对发射机，同样的操作只是逻辑上反序。AD9363 的最前端是低噪声放大器（LNA），提供模拟增益——它是接收机自动增益控制（AGC）流水线的一环。LNA 之后是混频器，负责直接频率搬移。与许多传统超外差收发信机不同，AD9363 是直接变频（零中频，ZeroIF）设计，不使用中频（IF）级。关于超外差与直接变频接收机折中的更多细节，可阅读 Razavi [2]。

AD9363 的混频器按数据手册规格工作在 325 MHz 至 3.8 GHz [2]，但可以通过软件修改扩展该范围（见 5.2.6 节）。混频之前，信号被分成两路完全相同但独立的路径。这一过程通过简单地旋转混频器的时钟相位，产生信号的同相与正交分量。由于同相与正交信号正交（带宽从 $-\frac{f_s}{2}$ 到 $\frac{f_s}{2}$），这实际上把接收机的有效带宽翻了一倍。

混频之后，信号被滤波，以去除下混频后信号的混叠效应、降低带外干扰与噪声。跨阻放大器（TIA）与模拟滤波器组合配置以维持期望的模拟带宽（200 kHz 至 20 MHz）。TIA 充当单极点滤波器，模拟可编程滤波器是三阶巴特沃斯。

AD9363 的最后一级是数字转换与抽取级。这里 ADC 的运行速率通常远高于期望的接收带宽，但 ADC 本身并不提供规格定义的全部 12 位。额外的位数在半带滤波器（HBF）级中获得——半带级允许位增长。ADC 本身只提供约 4.5 位分辨率。这是 sigma-delta（Σ-Δ）转换器的典型设计：固有低噪声、比逐次逼近（SAR）ADC 跑得快（Σ-Δ ADC 详见 2.5.4 节）。而通过使用极高速的 ADC 加配套 HBF，接收信号可以按配置的采样率以 12 位数字化。因此，最佳信号分辨率来自对输入信号的大倍率过采样，再接若干级抽取。

5.1.3 Zynq 细节

数据数字化后交给 Xilinx Zynq 片上系统（SoC），如图 5.3 所示。Zynq-7000 家族既提供 FPGA 的灵活性与可扩展性，又具备通常与 ASIC/ASSP 相当的性能、功耗与易用性。Zynq 单芯片集成了基于 ARM Cortex-A9 的处理系统（PS）与可编程逻辑（PL），在 Pluto SDR 中充当主控制器。

“可编程逻辑 + 可编程处理子系统” 的组合带来一些独特优势。AD9363 ADC 的最低数据转换率是 25 MHz，最大抽取为 48 倍，于是最低采样率为 520.833 kSPS。FPGA 内又加了一个 8 分频抽取滤波器，把最低采样率进一步降到 65.1042 kSPS。在 Pluto SDR 内的 ARM-A9 上跑 Linux 也有独特优势：能用 Linux 的 IIO 基础设施复用 AD9363 的现成设备驱动——从采样率到 FIR 设置、LO 设置再到附加抽取滤波器，这些久经考验的软件在打造 Pluto SDR 时一行都不用改。

当 5.1.2 节所述的 AD9363 数字接收 I/Q 数据传到 FPGA 逻辑后，AXI DMAC 核把它写入 Pluto 的外部内存。这一步把” 连续唯一的采样流” 变成了” 每 n 个采样一个的缓冲区”。

5.1.4 Linux 工业输入/输出（IIO）细节

Linux 内核中的工业输入/输出（IIO）子系统，旨在为” 某种意义上的 ADC 或 DAC”、而又没有现成子系统（如音频、视频）可归的器件提供支持。这不是 Pluto 专属、也不绑定任何 SDR 实现——它是一个被众多厂商采用的开源标准，为种类繁多的器件提供统一 API，包括但不限于：ADC、加速度计、陀螺仪、IMU、电容数字转换器（CDC）、压力传感器、颜色/光/接近传感器、温度传感器、磁力计、DAC、直接数字合成（DDS）、锁相环（PLL）、可变/可编程增益放大器（VGA/PGA）、以及像 AD9363 这样的集成射频收发信机。

三个主要方面：

- Linux 内核 IIO 驱动，运行在 Linux 内核内（此处即 Pluto SDR 的 ARM 上）；

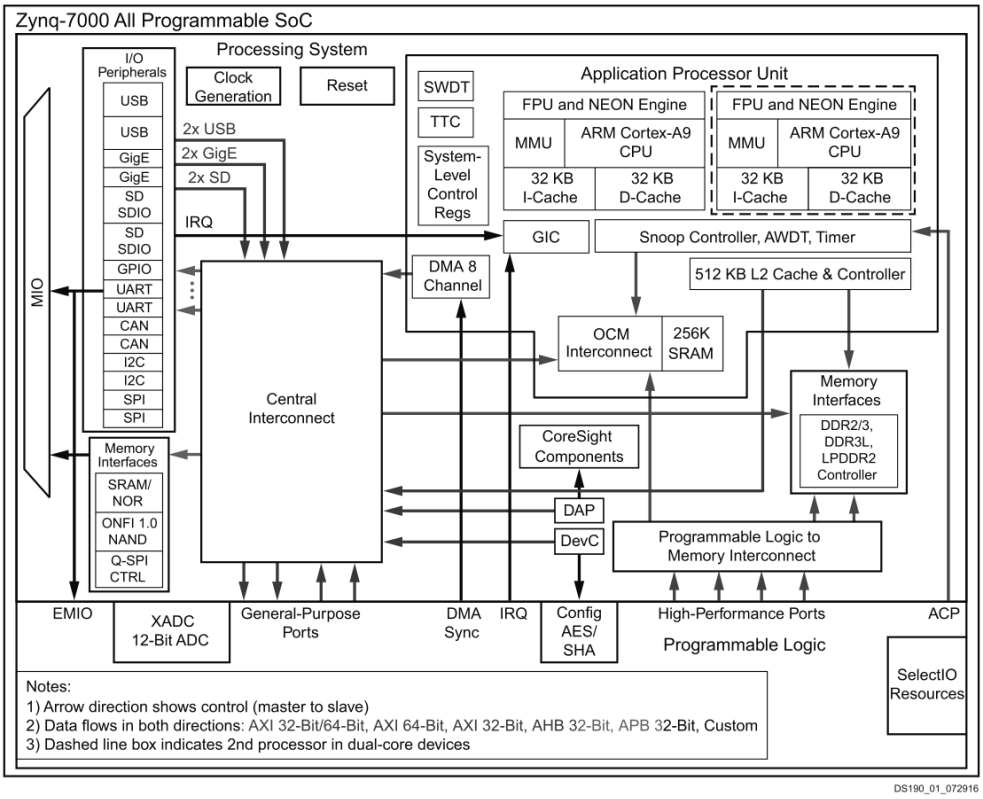

Zynq-7000 All Programmable SoC
Processing System
I/O Peripherals
USB
USB
GigE
GigE
SD SDIO
SD SDIO
GPIO
UART
UART
CAN
CAN
I2C
I2C
SPI
SPI
MIO
Clock Generation
Reset
2x USB
2x GigE
2x SD
IRQ
SWDT
TTC
System-Level Control Regs
Application Processor Unit
FPU and NEON Engine
MMU
ARM Cortex-A9 CPU
32 KB I-Cache
32 KB D-Cache
FPU and NEON Engine
MMU
ARM Cortex-A9 CPU
32 KB I-Cache
32 KB D-Cache
GIC
Snoop Controller, AWDT, Timer
512 KB L2 Cache & Controller
DMA 8 Channel
OCM Interconnect
256K SRAM
Central Interconnect
Memory Interfaces
DDR2/3, DDR3L, LPDDR2 Controller
CoreSight Components
DAP
DevC
Programmable Logic to Memory Interconnect
Memory Interfaces
SRAM/ NOR
ONFI 1.0 NAND
Q-SPI CTRL
EMIO
XADC 12-Bit ADC
General-Purpose Ports
DMA Sync
IRQ
Config AES/ SHA
High-Performance Ports
Programmable Logic
ACP
SelectIO Resources
Notes:
1) Arrow direction shows control (master to slave)
2) Data flows in both directions: AXI 32-Bit/64-Bit, AXI 64-Bit, AXI 32-Bit, AHB 32-Bit, APB 32-Bit, Custom
3) Dashed line box indicates 2nd processor in dual-core devices
DS190_01_072916

图 5.3　Zynq 框图 [4]

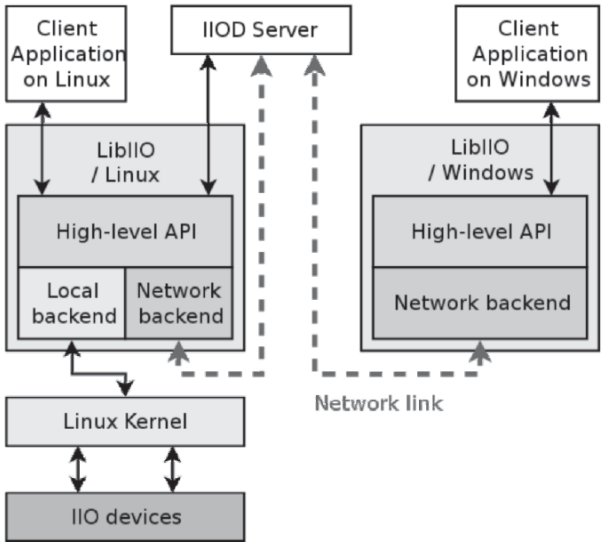


图 5.4　libiio 与 iiod 框图 [5]

- libiio——访问本地与远程 IIO 器件的用户空间库（此处既在 ARM 上也在主机上）；
- iiod——IIO 守护进程，负责允许 IIO 客户端远程连接（此处运行在 Pluto SDR 的 ARM 上）。

libiio 用来对接 Linux 的工业输入/输出（IIO）子系统。libiio 既可以在嵌入式 Linux 目标机上原生使用（本地模式），也可以从 Linux/Windows/macOS 主机经 USB、以太网或串口远程连接同一目标机。libiio 虽主要由 Analog Devices 开发，却是一个许多人为之贡献的活跃开源库。它以 GNU 宽通用公共许可证（LGPL 2.1 或更高版本）发布——这一开源许可证允许任何人在任何厂商的处理器/FPGA/SoC 上、控制任何厂商的外设（ADC、DAC 等）本地或远程使用该库，包括闭源或开源、商业或非商业应用（须遵守 LGPL 的自由、义务与限制）。

接收数据的缓冲区进入外部内存后，被交给 iiod（IIO 守护进程）。iiod 负责管理多个 iio 客户端（通常在网络或 USB 远端），并把它们的请求翻译为本地设备操作。它通过 libiio 库完成这一切——libiio 经一系列接口访问 AD9363。便利的是，无论在 ARM 上还是装了 libiio 的主机上，访问收发信机的 libiio API 完全相同。因此代码可以先在与 Pluto SDR 相连的主机上实现，再原样部署到 ARM 上。

5.1.5 MATLAB 作为 IIO 客户端

MATLAB 可以作为跨平台的 IIO 客户端与 Pluto SDR 对接。它提供 Pluto SDR 系统对象接口（MATLAB 系统对象的基础见附录 B.3）。Pluto SDR 硬件支持包（HSP）提供的两

个系统对象是：

- `comm.SDRRxPluto`：Pluto SDR 接收机系统对象
- `comm.SDRTxPluto`：Pluto SDR 发射机系统对象

这些对象通常通过代码 5.1 那样的 `sdrrx` 或 `sdrtx` 函数调用构造。

代码 5.1　实例化 Pluto SDR 系统对象：pluto1.m

```
rx = sdrrx('Pluto')


tx = sdrtx('Pluto')
```

不过这些对象也可以直接实例化。无论哪种方式，`sdrrx` 得到的对象都具有以下基本属性——不用分号时它们会直接打印到终端，见代码 5.2。

代码 5.2　实例化 Pluto SDR 系统对象：pluto1.m

```
rx = sdrrx('Pluto')
rx =
  comm.SDRRxPluto with properties:
  DeviceName: 'Pluto'
  RadioID: 'usb:0'
  CenterFrequency: 2.4000e+09
  GainSource: 'AGC Slow Attack'
  ChannelMapping: 1
  BasebandSampleRate: 1000000
  OutputDataType: 'int16'
  SamplesPerFrame: 3660
  ShowAdvancedProperties: false
```

由于 Pluto SDR 是一个更大 SDR 器件家族的成员，它共享 `DeviceName` 属性——对 Pluto SDR 而言取值为 `Pluto`。从代码 5.2 的用法可见系统对象的各项属性：

- `RadioID` 与电台使用的枚举接口相关：要么是 `usb:#`（# 是从零开始、连接到 MATLAB 的 USB 电台编号），要么是 `ip:<ipaddress>`（经以太网使用 Pluto SDR）。
- `CenterFrequency` 定义射频中心频率（Hz）。注意 Pluto SDR 的接收与发射 LO 是分开的，分别在 Rx 与 Tx 对象中管理。
- `BasebandSampleRate` 定义同相/正交接收链的采样率。注意 AD9363 的 ADC 与 DAC 只有一个时钟发生器，因此在同一器件上管理 Rx 与 Tx 时二者必须设成相同

值。

- `GainSource` 有三个选项：Manual、AGC Slow Attack、AGC Fast Attack。选择 Manual 时会出现另一个选项 `Gain`。该 `Gain` 值关联 AD9363 内部的一张增益表——`Gain` 值变化时，图 5.2 接收路径中的多级增益按内部表联动更新。必要时可使用自定义增益表，但这属于高级实现选项，本书不涉及。另两个 `GainSource` 设置启用 AD9363 内基于状态机的增益控制，依据接收信号强度（RSSI）在运行中自适应。
- Pluto SDR 的 `ChannelMapping` 属性只能设为 1。在 Analog Devices 家族的其他 SDR 上，它用于多通道（多输入多输出，MIMO）接收。
- `OutputDataType` 决定对象输出数据的格式。技术上，从 AD9363 和 libiio 来看 MATLAB 只能收到 16 位复整数，但可以让 MATLAB 默认把它们转换为其他类型。通常转成 double——精度最高，而且三角函数运算需要 double 或 single。如前所述，收发路径只提供 12 位分辨率，但若干软硬件层只能处理 8 位基类型，因此这 12 位以 16 位整数交付。参考：接收侧 16 位采样是符号位扩展的，发射侧则直接丢弃最低四个有效位。
- `SamplesPerFrame` 决定每次调用对象时 iiod 传给 MATLAB 的缓冲区/帧的采样数——即每次调用对象返回的向量长度。只要不发生溢出，这些数据总是从电台连续接收的；但对同一对象的 successive 调用不保证缓冲区之间的连续性。因此，一次采集大量数据再处理往往更有用。

发射机系统对象 `comm.SDRTxPluto` 的属性几乎相同，除了 `GainSource`、`SamplesPerFrame`、`OutputDataType`——它们在发射语境下没有意义。

5.1.6 不只是用来学习

Pluto SDR 上的”AD936x 射频收发信机 + Zynq SoC + 外部内存” 架构组合不应只被当作学习平台。许多商业系统构建在相似架构上，可以用同样的软件（HDL、Linux 内核与 IIO）与 IIO 客户端（如 MATLAB）通信。例如位于伊利诺伊州绍姆堡的 Epiq Solutions 公司就打造了图 5.5 所示的工业级商用方案 Sidekiq Z2。

Sidekiq Z2 虽然采用与 Pluto SDR 相似的架构，但做成了标准 Mini PCIe 卡外形，尺寸约 51 × 30 mm。此外，Sidekiq Z2 加入了射频滤波、高精度基准时钟、更强的双核 Zynq，并用 AD9364 把射频调谐范围扩展到 70–6,000 MHz，全部采用工业温度范围（−40 °C 至 +70 °C）额定器件。这使 Sidekiq Z2 可以作为现实世界灵活射频方案的基础——即使在恶劣工业环境中。

由于构建在同一架构、使用同一软件基础设施，算法开发者可以用 Pluto SDR 这样的器件做原型；当需要工业外形或可嵌入式时，可以轻松迁移到 Sidekiq Z2。这同样让

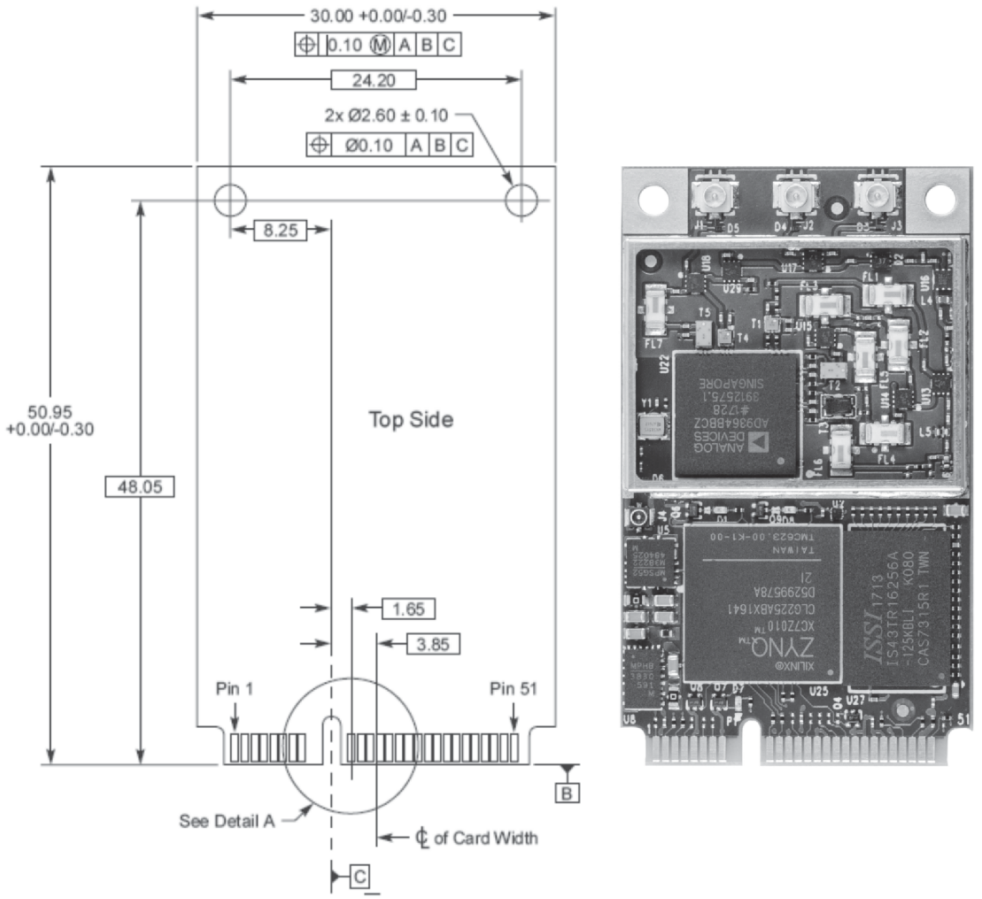


图 5.5　Epiq 的 Sidekiq Z2 [6]。非等比例

Sidekiq Z2 的开发者能直接与 MATLAB 或其他任何 IIO 客户端软件通信，在其最终产品中做测试。射频领域有许多事情是仿真不了的，除非你的最终产品能保存/回放波形或远程连到设备，否则发布量产可能非常困难甚至不可能。对工程师来说，没什么比”在实验台上好好的东西到现场就坏、还必须赶去现场排查” 更沮丧的了。使用这些开源的工业标准框架可以降低这种风险。

5.2 MATLAB 开发策略

如 5.1.5 节所述，控制数据如何进入 MATLAB 对稳定运行非常重要。本节讨论组织 MATLAB 代码、高效开发算法的若干策略。为引导开发过程，我们提供了展示如何在电台 API 周围恰当组织源码的模板。这些模板能帮助设计向” 实时” 或” 无电台离线运行” 演进。

5.2.1 电台 I/O 基础

这些模板都假设电台已实例化为对象 `rx`(如运行代码 5.1),并假设对象的 `SamplesPerFrame` 参数设为某变量 `frameSize`。代码 5.3 给出的第一个模板先采集 `framesToCollect` 帧数据(每帧 `frameSize` 个采样)。代码的目标是保证从电台连续无缺口地采集 `framesToCollect` × `frameSize` 个采样。当需要采集超过 2^{20} 个采样时——这是 Pluto SDR 系统对象 `SamplesPerFrame` 参数的最大值——这是个好技术。采集完成后做些处理,此例是用 `dsp.SpectrumAnalyzer` 做可视化。

代码 5.3　模板示例：template1.m

```
%% Template 1
% Perform data collection then offline processing
data = zeros(frameSize, framesToCollect);
% Collect all frames in continuity
for frame = 1:framesToCollect
    [d,valid,of] = rx();
    % Collect data without overflow and is valid
    if ~valid
        warning('Data invalid')
    elseif of
        warning('Overflow occurred')
    else
```

```
        data(:,frame) = d;
    end
end

% Process new live data
sa1 = dsp.SpectrumAnalyzer;
for frame = 1:framesToCollect
    sa1(data(:,frame)); % Algorithm processing
end
```

另一种做法：如果每次运行并不需要新采样，把数据存文件会很实用，免去清理工作区的烦恼。这方面一个好工具是 comm.BasebandFileWriter——它把复数接收数据连同采样率等元数据存入文件供离线处理。代码 5.4 展示了用 comm.BasebandFileWriter 保存代码 5.3 采集的数据。

代码 5.4　保存数据的模板示例：template1.m

```
% Save data for processing
bfw = comm.BasebandFileWriter('PlutoData.bb',...
    rx.BasebandSampleRate,rx.CenterFrequency);
% Save data as a column
bfw(data(:));
bfw.release();
```

调试算法时，用文件源做测试可复现性好得多；而且访问文件通常比去电台拉新数据快——尤其当还需要搭发射机时。代码 5.5 展示 comm.BasebandFileWriter 的孪生对象 comm.BasebandFileReader 的用法。comm.BasebandFileReader 可以通过 SamplesPerFrame 参数配置成每次调用返回指定数量的采样，以模拟使用 Pluto SDR 的情形——电台不在手边时这是个实用策略。

代码 5.5　读取数据的模板示例：template2.m

```
%% Template 2
% Load data and perform processing
bfr = comm.BasebandFileReader(bfw.Filename,'SamplesPerFrame',frameSize);
sa2 = dsp.SpectrumAnalyzer;
% Process each frame from the saved file
```

```
for frame = 1:framesToCollect
    sa2(bfr()); % Algorithm processing
end
```

算法调好之后，可以把处理环节放进与 Pluto SDR 系统对象交互的主循环，如代码 5.6。这类处理在 MATLAB 中称为**流处理**（stream processing）[7]——立即处理新到的数据。它限制了需要采集的数据量，而且当需要施加逻辑动作（如换信道）时很有用。只要循环内的算法跟得上电台流入的数据，就不会出现溢出告警——这就是所谓的**实时**运行。不过，由于往返电台的缓冲区有一定弹性，溢出不会总是立刻发生：例如算法只比电台数据率慢一点点时，内部缓冲区与操作系统缓冲区可能要很多次循环迭代才会填满到溢出。因此测试实时性时，应让算法连续运行数分钟，观察是否发生溢出。

代码 5.6　流处理模板示例：template3.m

```
%% Template 3
% Perform stream processing
sa3 = dsp.SpectrumAnalyzer;
% Process each frame immediately
for frame = 1:framesToCollect
    [d,valid,of] = rx();
    % Process data without overflow and is valid
    if ~valid
        warning('Data invalid')
    else
        if of
            warning('Overflow occurred')
        end
        sa3(d); % Algorithm processing
    end
end
```

5.2.2 连续发射

只要 Pluto SDR 通电，收发信机就会被激活并开始工作——哪怕用户并无此意。上电后 Pluto SDR 就会发射数据，收发信机就是这么设计的。因此，哪怕只使用接收机系统对象（`comm.SDRRxPluto`），器件实际上也在发射。通常收发信机会把 DMA 中可用的最

后一个缓冲区连续发射直到断电。若 Tx LO 恰好被调到与 Rx LO 相同的值（多电台通信或纯接收时），这种连续发射会造成显著干扰。

有两种办法减小甚至消除这种干扰。第一种是实例化发射机系统对象（comm.SDRTxPluto）并写入全零向量，如代码 5.8。这会把器件的发射能量压到最低。不过由于 Tx LO 泄漏，接收数据中仍会有少量泄漏进入。

代码 5.7　重复发射模板示例：transmitRepeat.m

```
% Transmit all zeros
tx = sdrtx('Pluto');
fs = 1e6; fc = 1e4; s = 2*pi*fs*fc*(1:2^14).';
wave = complex(cos(s),sin(s));
tx.transmitRepeat(wave);
```

代码 5.8　发射全零模板示例：transmitzeros.m

```
% Transmit all zeros
tx = sdrtx('Pluto');
tx(zeros(1024,1));
```

另一种更干净的办法：直接把发射机的 LO 挪到接收带宽之外。代码 5.9 演示了这一配置——把发射机系统对象的 CenterFrequency 偏移开。这是更好的选择，因为发射机的 LO 泄漏不会再出现在接收数据里。

代码 5.9　偏移发射模板示例：transmitoffset.m

```
% Move transmitter out of receive spectrum
tx = sdrtx('Pluto');
rx = sdrrx('Pluto');
tx.CenterFrequency = rx.CenterFrequency + 100e6;
```

5.2.3 时延与数据延迟

从主机使用 Pluto SDR，很快就会发现收发数据存在延迟——尤其是做发射到接收的自环操作时。这些延迟是主机 PC 与 Pluto SDR 各自内部缓冲区和 FIFO 的函数。而如图 5.6 所示，各层中既有确定性延迟、也有弹性/随机延迟。传输栈里之所以有非确定性延迟，是因为 ARM 与主机 PC 内核中的资源竞争。最坏情况下往返时延约为几十毫秒量级。要保证确定的或最小的延迟，就得直接在 ARM 或 FPGA 上跑。不过，在这些处理

区域开发要耗时得多。任何把数据送进 MATLAB 的电台平台都必须应对这类延迟，只是来源随电台架构而异。

看图 5.6 的一个麻烦之处：许多地方传输以字节定义，另一些地方用采样更方便。此情形下（单电台信道）一个 I/Q 采样（复数）是两个 16 位采样、即 4 字节。

具体过一遍图 5.6 的接收侧：当 MATLAB 这样的 IIO 客户端请求一个 32,768 采样的缓冲区、采样率 1 MSPS 时：

- iiod 将经 libiio 捕获 32,768 个连续采样，耗时 32.768 毫秒、消耗 131,072 字节。iiod 通过” 收齐之前不发送” 确保数据包连续，但这带来” 采样率 × 采样数” 的固定时延。iiod 的设计保证数据捕获后零内存拷贝——从 AD9363 驱动到 libiio 到 iiod 再到 USB 栈，全程只传指针。
- 数据随后交给 Pluto 上的 Linux 内核，按 USB 2.0 规范切成 512 字节的 USB 包，在主机侧重组为 131,072 字节的缓冲区。USB 栈会引入一个未知且无界的时延，取决于主机操作系统及其可用内存。
- 整个缓冲区组装完毕后交给 libiio，libiio 原封不动传给 IIO 客户端（此例即 MATLAB），后者可能做进一步处理。
- 此例中 MATLAB 可能把数据从定点转换为浮点或 ±1.0 的 double——这也需要时间。

由于这些延迟，我们必须在理解这些条件的前提下设计算法与整个系统。例如，想用同一台 Pluto SDR 发射并接收单帧数据时，会观察到帧在接收输出向量中出现之前有一大段采样空隙——正是这些延迟所致。

正因如此，许多系统希望把尽可能多的处理放到尽量靠近电台的地方——本架构中就是 FPGA 或 ARM 处理器。不过 Pluto SDR 的 FPGA 较小、ARM 又只有单核，限制了其承载能力。

5.2.4 接收频谱

我们观察到的接收信号永远包含来自环境与电台自身的噪声。为给这些噪声一个直观感受，用代码 5.10 的脚本做个简单示例。

代码 5.10　查看频谱模板示例：template_rt.m

```
% View some spectrum
rx = sdrrx('Pluto');
rx.SamplesPerFrame = 2^15;
sa = dsp.SpectrumAnalyzer;
```

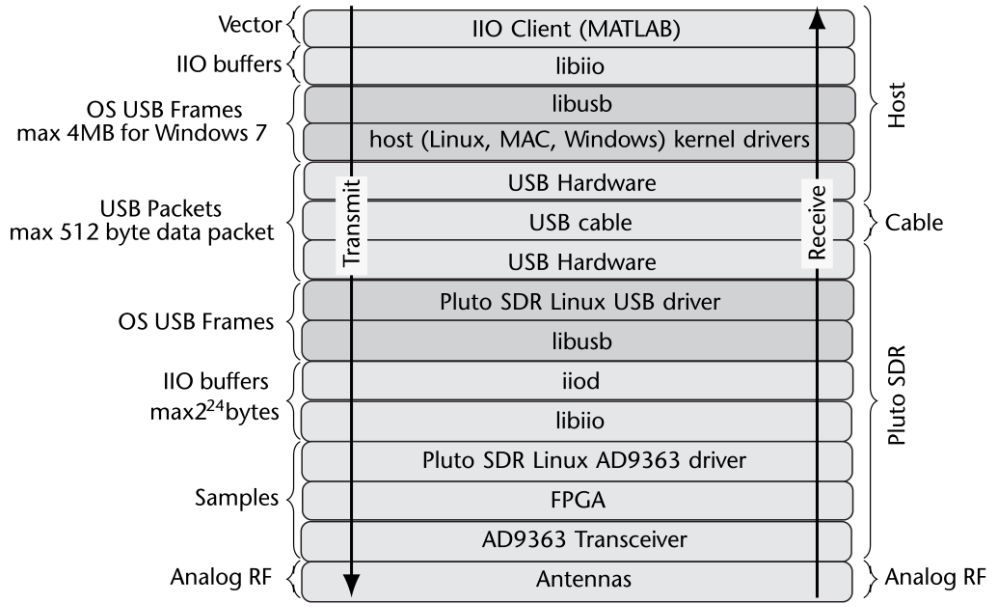


图 5.6　数据包从 MATLAB 到收发信机必须穿越的软硬件栈

```
sa.SampleRate = rx.BasebandSampleRate;
for k=1:1e3
  sa(rx());
end
```

运行代码 5.10 会注意到频谱并不完全平坦——频谱中有峰或杂散。这些其实是电台自身的伪影，源于复杂的接收链以及 AGC 各级提供的信号增益增强。依据接收链中我们选择的带宽与 AGC 配置本身，这些杂散可以改变。不过 AD9363 收发信机非常复杂——仅 AGC 就有数千个配置寄存器——调整它并不轻松。既然这些杂散相对感兴趣的信号通常较小，我们可以直接忽略，或在软件中滤波以削弱其影响。但使用 Pluto SDR 时，应始终对硬件的这些非理想性心中有数。

幸好，Pluto SDR 内置了大量校准来减少自扰问题，包括射频 DC 跟踪、正交跟踪与基带 DC 跟踪。要访问这些功能，可启用 `ShowAdvancedProperties` 参数，之后这些功能就会显示出来。由于 AD9363 是直接变频接收机，这类器件的常见问题是 DC 或零频附近出现单音或能量——它来自电台本身。两个 DC 跟踪组件（射频与基带）都在削弱这些效应。

最后一个高级功能是正交跟踪。正交跟踪削弱可能出现的同相/正交（IQ）失衡——这种失衡在频域图上不易察觉，典型表现为频域中的镜像杂散。启用正交跟踪后，镜像

杂散应显著降低。而在星座图上 IQ 失衡更明显。总会残留一些失衡，但校正在初始化时完成，不一定会随时间改善。

5.2.5 自动增益控制

AD9363 中最复杂的部件之一就是 AGC。AGC 实际上遍布接收链：在不同级施加增益、感知接收信号幅度。从 MATLAB 的简化 API 看有三个选项：Manual、AGC Slow Attack、AGC Fast Attack。**Manual** 下接收机按内部增益表把输入增益固定在特定值——手动增益本质上就是内部增益表的一个索引。当使用线缆直连、或发射机在固定已知距离时，Manual 很合适；而且 AD9363 状态保持不变，接收机算法更容易调试。

实践中接收信号幅度未知、甚至随多次传输而变化。因此可以直接用 AGC Slow Attack 与 AGC Fast Attack 模式。若接收传输很短或幅度快速变化，AGC Fast Attack 最佳；接收幅度相对平稳时 AGC Slow Attack 更合适。一个自然的问题是：为什么不总用 AGC Fast Attack？它的主要缺点是会在幅度上造成大的不连续，可能扭曲接收信号。作为示例，我们对比单台 Pluto SDR 自环发射 QPSK 信号的设置，给出接收信号的时域图，展示不同增益模式下的接收时延与幅度变化。可以看到 AGC Fast Attack 模式增益变化剧烈，AGC Slow Attack 模式增益随时间更平缓。正确模式的选择并不总是一目了然，因此算法开发期间做时序测试与观察很有用。

5.2.6 常见问题

各种信号的混频方式也可能成为问题。如 2.3.1 节所述，混频器接受单端本振（LO）。

5.3 示例：真实数据自环

掌握了控制 Pluto SDR 的系统对象与若干代码结构后，来做一个简单的自环（loopback）示例。自环指波形由同一台 Pluto SDR 既发射又接收——这是算法调试与硬件调试的常用策略。

从发射机（`tx`）看，本例首先会注意到增益设为 −30——比默认值低 20 dB。降低增益是为了防止接收端削波或饱和：收发天线相距只有约 12 mm，接收信号会相当” 响”。正弦波由 `dsp.SineWave` 系统对象生成（方便起见），它为 Pluto SDR 提供复数输出。用了一个特殊方法 `transmitRepeat`：连续不断地发射传入的向量，避免传输中出现空隙。

在图 5.7 的接收波形中可以观察信号的复数分量与实部分量随时间的变化——如预期那样二者相差 $\frac{\pi}{2}$ 弧度。信号起始处幅度比后续采样大，这是 AGC 收敛的直接结果——配置/建立时刻 AGC 默认从高增益起步。在 MATLAB 中，这个建立时刻只发生在首

次调用接收对象（rx）时（不是构造时），也就是首次调用 Step 方法时——它会调用系统对象内部的 setupImpl 方法。

代码 5.11　自环示例：loopback.m

```
% Setup Receiver
rx=sdrrx('Pluto','OutputDataType','double','SamplesPerFrame',2^15);
% Setup Transmitter
tx = sdrtx('Pluto','Gain',-30);
% Transmit sinewave
sine = dsp.SineWave('Frequency',300,...
                    'SampleRate',rx.BasebandSampleRate,...
                    'SamplesPerFrame', 2^12,...
                    'ComplexOutput', true);
tx.transmitRepeat(sine()); % Transmit continuously
% Setup Scope
samplesPerStep = rx.SamplesPerFrame/rx.BasebandSampleRate;
steps = 3;
ts = dsp.TimeScope('SampleRate', rx.BasebandSampleRate,...
                    'TimeSpan', samplesPerStep*steps,...
                    'BufferLength', rx.SamplesPerFrame*steps);
% Receive and view sine
for k=1:steps
  ts(rx());
end
```

凑近看正弦波，可以观察到由接收帧之间小空隙导致的不连续。这其实是发射机反复重发同一波形的结果：传给发射机的数据在起点与终点之间不是循环衔接的，于是出现这些不连续。要避免它们，需要保证一个周期恰好在传入帧的末尾结束、并恰好在下一帧的开头开始。

5.4 噪声系数

借助前节讨论的接收流水线，AD9363 在 800 MHz 处能做到仅 2 dB 的噪声系数（NF）。NF 是比较接收机的常用指标，度量器件内部电学元件的热噪声。NF 基于输出与输入

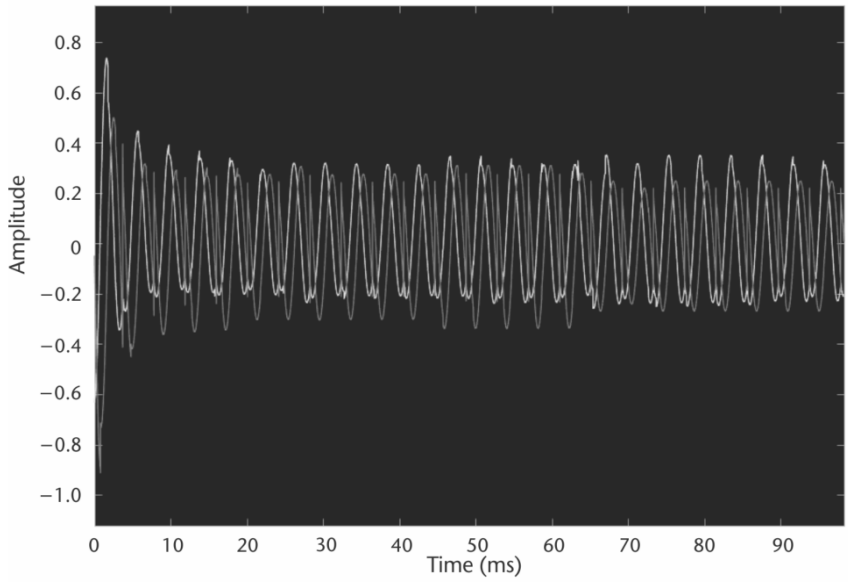


图 5.7　代码 5.11 生成的 Pluto SDR 300 Hz 自环正弦波

SNR 之比（dB）计算：

$$\mathrm{NF} = 10\log_{10}\frac{\mathrm{SNR_{INPUT}}}{\mathrm{SNR_{OUTPUT}}}, \tag{5.1}$$

其中 NF 以 dB 计，$\mathrm{SNR_{INPUT}}$ 与 $\mathrm{SNR_{OUTPUT}}$ 均为线性刻度 [8]。作为对比，另一款流行 SDR RTL-SDR 的 NF 为 4.5 dB，几乎是 Pluto SDR 的两倍。NF 之所以重要，是因为它决定接收机的最终灵敏度——越低越好。测量 NF 最简单的方法是用噪声系数分析仪：向系统接收路径注入噪声再反馈回来，测输出噪声并与输入噪声比较得出 NF。图 5.8 演示了用噪声系数分析仪评估 NF 的示例设置——被测器件（DUT）以自环模式把接收到的噪声发射出去。测 NF 时用非常稳定的 LO 驱动器件会有帮助，以限制内部噪声——毕竟要测的是 DUT 的噪声而不是驱动它的源振荡器。此外，NF 总是与频率相关，SDR 这类 DUT 的实测 NF 通常取决于器件噪声最大的部分。NF 是硬件厂商的常用指标，但与通信问题联系起来并不容易——通信工程师往往在接收链更下游、滤波级之后再测量；NF 的测量也需要专用且通常昂贵的仪器。不过，理解数据手册上 NF 的含义很重要——它给出误差矢量幅度测量下限的粗略估计。

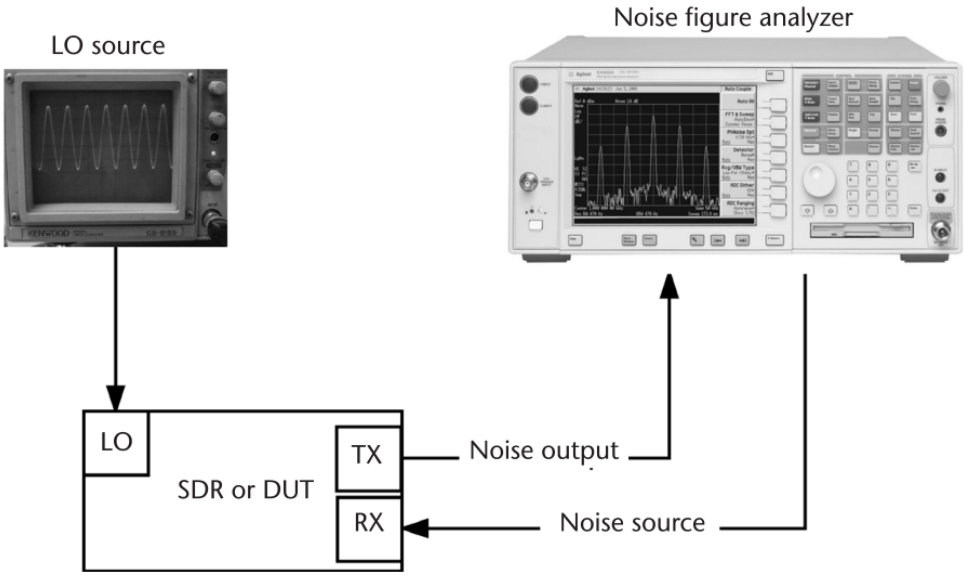


图 5.8　用噪声系数分析仪与外部稳定本振源评估 SDR 或被测器件噪声系数的示例

5.5 参考文献

[1] Analog Devices, “ADALM-PLUTO Software-Defined Radio Active Learning Module.” http://www.analog.com/plutosdr, 2017.

[2] Razavi, B., “Design Considerations for Direct-Conversion Receivers,” *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, Vol. 44, No. 6, June 1997, pp. 428–435.

[3] Analog Devices, “AD9363 Datasheet.” http://www.analog.com/AD9363, 2015.

[4] Xilinx, “Zynq-7000 All Programmable SoC Overview,” [Online], 2017. https://www.xilinx.com/support/documentation/data_sheets/ds190-Zynq-7000-Overview.pdf.

[5] Analog Devices, “What is libiio?” [Online], 2018. https://wiki.analog.com/resources/tools-software/linux-software/libiio.

[6] Epiq Solutions, “Sidekiq Embeddable Software Defined Radio 70MHz–6GHz,” [Online]. https://epiqsolutions.com/sidekiq/.

[7] The MathWorks, Inc., “Stream Processing in MATLAB: Process Streaming Signals and Large Data with System Objects.” https://www.mathworks.com/discovery/stream-processing.html.

[8] Friis, H. T., “Noise Figures of Radio Receivers,” *Proceedings of the IRE*, Vol. 32, No. 7, July 1944.

第 6 章 定时同步

在接下来的一系列章节中，我们将介绍接收机同步与信号恢复的概念——多设备间无线通信的必要基础。本章介绍定时恢复的概念，并把它放在最前面有两个主要原因。第一，本书用于相干调制的下游恢复方法对定时偏移敏感，必须最先补偿。第二是为了用 Pluto SDR 做原型：我们将大量依赖电台的自环功能，在一定程度上控制非理想性。然而，信号从发射 DAC 到接收 ADC 之间总要走过一段距离，两条链路之间会存在一个固定但随机的时延偏移——定时恢复正是用来校正这一偏移的。话说回来，接收机可以用很多方式设计，本书各章的特定顺序对应算法的逐次应用：先定时恢复，再载波相位校正，最后帧同步。这三件大事处理完，我们将进入均衡与编码等更高级的主题。图 6.1 中的功能块会在每个相关章节的开头被高亮，以勾勒整个接收机设计的进程、展示它们如何互相配合。本章高亮的是匹配滤波与定时恢复。

本章把定时恢复拆成五个主要部分：先概述发射滤波器——这是理解” 如何用算法执行定时恢复” 的必要基础；然后给出一个演示定时误差的简单模型，其中也包括用 Pluto SDR 做的真实数据；最后讨论若干自适应校正定时问题的恢复技术，并提供调试方法，帮助你把这些技术调校到自己的数据上。对于所讨论的算法，本书不显式给出解析分析或推导，而是把它们当作逐组件搭建接收机的工具——只需顶层理解即可。更深入的分析可参考所引文献；本书聚焦实现，尤其是 SDR 上的实现。我们的目标是先从现有成果出发 motivate 接收机的构建，希望从实现中榨取更多性能的读者可以自行探索更多文献。

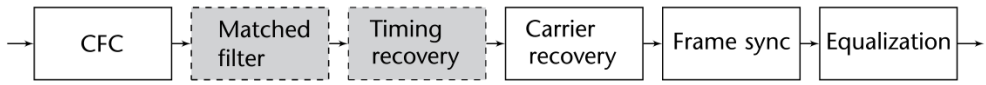


图 6.1　接收机框图

6.1 匹配滤波

数字通信理论中讨论匹配滤波时，通常把发射端的叫**脉冲成形**、接收端的叫**匹配滤波**。这些技术的目标有三：一、通过限制有效带宽使信号适合在通信信道中传输；二、提高接收波形的 SNR；三、减少多径信道与非线性引起的码间串扰（ISI）。脉冲成形在 2.7.5 节讨论过，这里换个视角重新审视，聚焦这些滤波器更实用的方面。

对符号滤波后，尖锐的相位与频率跳变减少，信号更紧凑、频谱效率更高。图 6.2 给出了 DBPSK 信号经发射滤波器滤波前后频谱表示的简单示例。可见信号的有效带宽减小了——主要来自发射机施加的上采样/插值。由于时域与频域成反比关系，带宽随之缩减。这些滤波级的实现通常会对信号上/下采样，从而降低有效带宽。但上采样固有地增大了符号的" 表面积"，使其更易被判决——因为接收端会得到它的多份副本。因此我们是在用带宽换可恢复性：发射机仍以相同速率产出数据，却不占用全部可用带宽。速率转换（上/下采样）操作在匹配滤波级完成——用单个滤波器同时做两件事是高效的。

生成该图所用的是平方根升余弦（SRRC）滤波器——通信系统的常用滤波器。升余弦（RC）滤波器已在 2.7.5 节给出，而更常用的 SRRC 的冲激响应为：

$$h(t)=\begin{cases}\frac{1}{\sqrt{T_s}}\left(1-\beta+\frac{4\beta}{\pi}\right), & t=0\\ \frac{\beta}{\sqrt{2T_s}}\left[\left(1+\frac{2}{\pi}\right)\sin\frac{\pi}{4\beta}+\left(1-\frac{2}{\pi}\right)\cos\frac{\pi}{4\beta}\right], & t=\pm\frac{T_s}{4\beta}\\ \frac{1}{\sqrt{T_s}}\frac{\sin\left[\frac{\pi(1-\beta)t}{T_s}\right]+4\beta\frac{t}{T_s}\cos\left[\frac{\pi(1+\beta)t}{T_s}\right]}{\frac{\pi t}{T_s}\left[1-\left(\frac{4\beta t}{T_s}\right)^2\right]}, & \text{otherwise}\end{cases}$$

其中 T_s 是符号周期，$\beta\in(0,1]$ 是滚降因子。实践中这些滤波器按图 6.3 的两种方式布置：要么在发射机放一个 RC 滤波器，要么在发射机与接收机各放一个 SRRC。两种方案都产生奈奎斯特滤波布置，可减少或消除 ISI。如何选择 β 与 T_s 的细节将在 8.2 节讨论。这里我们通过图 6.4 的眼图考察 β，可以清楚看到 $\beta=[0.3,0.99]$ 两个差异很大的滚降在时域的效果。这些结果中，信号先经发射机 SRRC 滤波、再经 SNR 27 dB 的 AWGN 信道、最后经接收机 SRRC 滤波。用高 SNR 是为了让眼保持张开、把噪声从最终判决的限制因素中排除。在判决时刻 200 与 400 处，两种情形符号都清晰，但 $\beta=0.99$ 的过渡更平滑。不过 $\beta=0.99$ 时频域外带含有更多能量（见图 2.52），这可能并不理想。

也可以考察冲激响应随 β 的变化，如图 6.5 对比了 RC 与 SRRC 响应。注意与 RC 滤波器不同，SRRC 的冲激响应在 T_s 整数倍处不为零；但发射与接收两个 SRRC 的复合响应会像 RC 一样在 T_s 整数倍处过零。之所以用 SRRC，是因为它是奈奎斯特型滤波

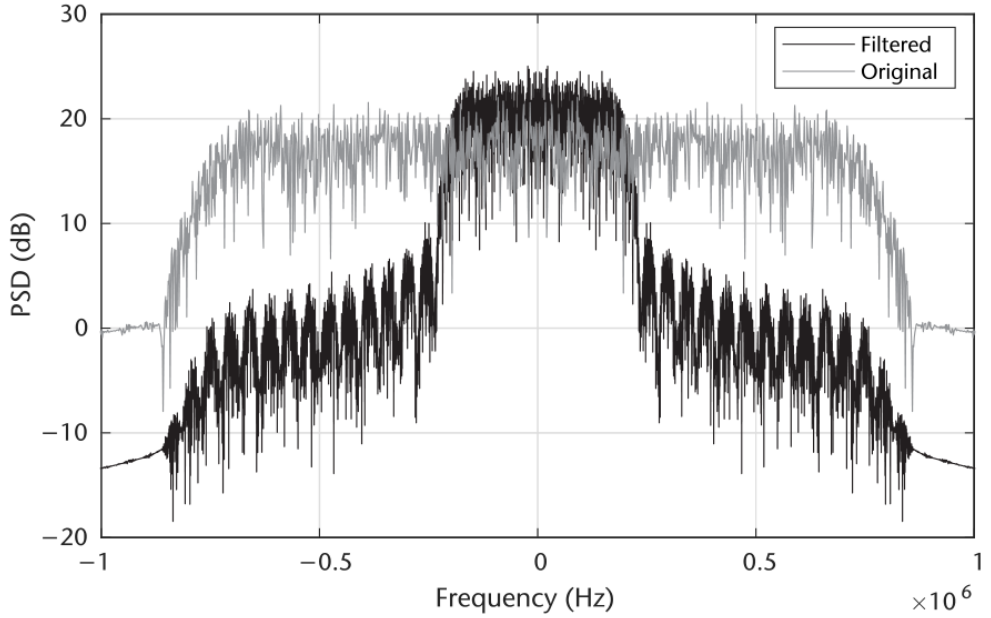


图 6.2　PSK 信号脉冲成形滤波前后的频谱

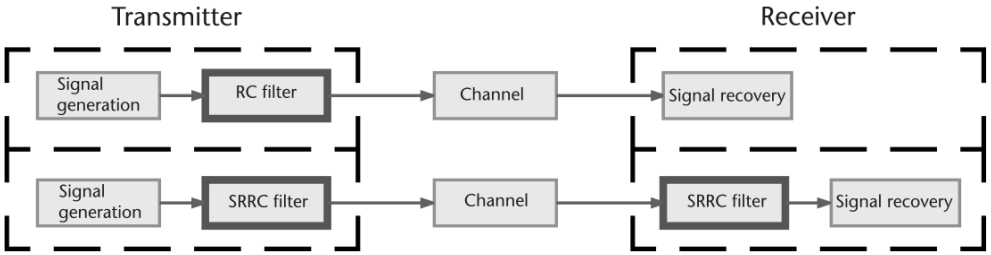


图 6.3　升余弦与根升余弦滤波器在收发两端的布置方式

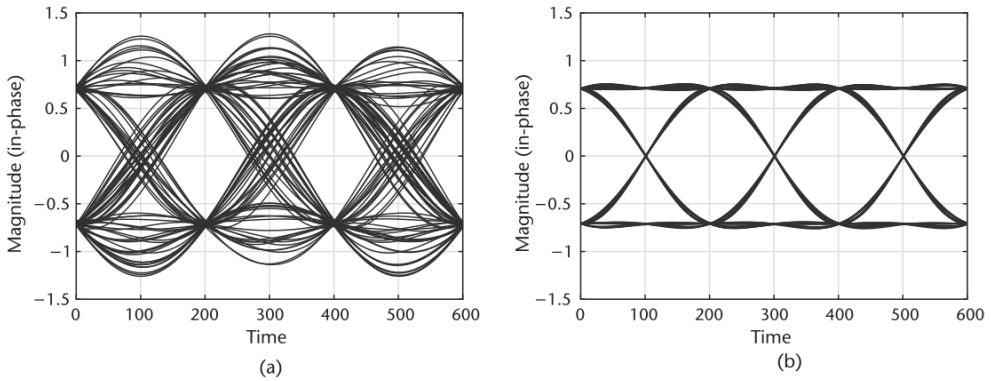


图 6.4　经不同 β 值 SRRC 滤波后 QPSK 信号同相部分的眼图：(a) β = 0.3；(b) β = 0.99

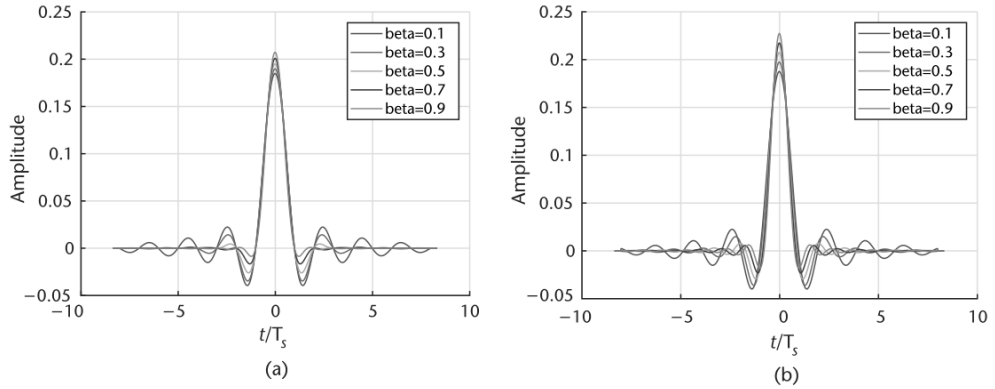


图 6.5　升余弦与平方根升余弦滤波器的冲激响应对比：(a) RC 冲激响应；(b) SRRC 冲激响应

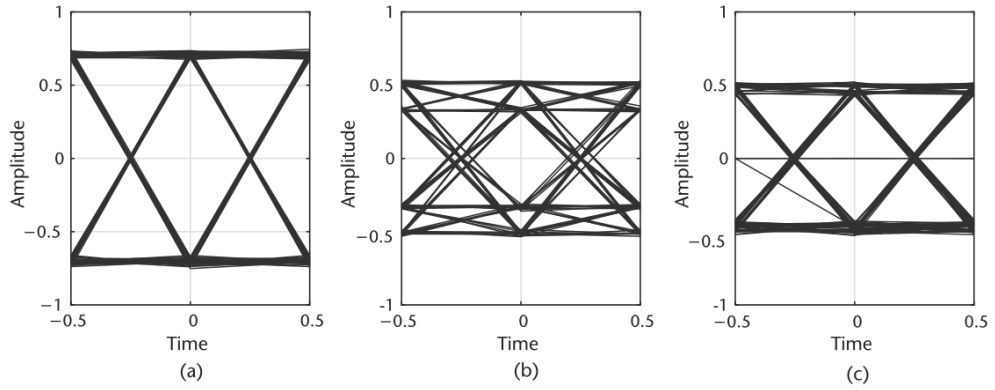


图 6.6　受引起 ISI 的非线性影响的 QPSK 信号眼图，SRRC 匹配滤波削弱了该影响：(a) 发射机原始信号；(b) 无脉冲成形经非线性；(c) 收发均用 SRRC 滤波并经非线性

器——如第 2 章所述，正确采样时产生零 ISI [1]。

我们可以通过向信道引入一个简单非线性来演示 ISI 的效果，并查看 2.4.1 节介绍的眼图。非线性造成幅度与相位失真——当我们削波或使发射放大器工作在极限时就会发生。所用模型的更多细节见 [2]，其他模型如 [3] 也存在。图 6.6 中我们观察到 ISI 的影响：眼被压缩，正确采样时刻变得难以确定。第 9 章讨论均衡时我们将再次回到 ISI 效应。

如前所述，速率转换通常发生在这些发射或接收滤波器中。因此可以用**多相滤波器**——SRRC 的抽头分布在多相滤波器的各支路中。这是非常高效的实现：抽头在多相设计中插值之前或抽取之后以较低速率施加，减少了所需乘法数。

我们要讨论的匹配滤波器的最后一个方面是 **SNR 最大化**。这个论断从相关概念自然得出：脉冲成形/滤波后的信号与脉冲成形滤波器相关，而噪声不相关，于是匹配滤波起到对信号 SNR 最大化的作用，在接收脉冲的中心位置产生峰。图 6.7 演示了这一

效果：展示 AWGN 下有/无脉冲成形的传输数据。图 6.7(b) 的中间图里，即使在高噪声下也能观察到与原始发送序列密切相关的信号；而无脉冲成形时（图 6.7(c) 第三图），即便没有引入任何定时偏移，仅凭视觉判断发射脉冲就已困难，甚至能观察到解调差错。

6.2 定时误差

发射机与接收机之间的定时误差概念上简单，但在通信系统中可能难以可视化与调试。最基本地说，符号定时同步的目的是对齐两台独立通信设备的时钟信号或采样时刻。图 6.8(a) 考虑一个简单示例：把发射与接收节点的时钟信号及待采样输入信号重叠起来。采样发生在时钟上升沿，最优定时是发射机的时钟。接收端引入了一个小于单个时钟周期的小时延 τ。由于不足一个采样，这称为**分数时延**。由于该时延，信号在错误位置被采样，最终解调的信号不正确。图 6.8(b) 比较了正确符号与接收端解调符号，在第二次跳变处出现明显差错。

数学上，可以把带偏移的接收信号 r 建模为

$$r(t) = \sum_n x(n) h(t - \tau(t) - nT_s) + v(t),$$

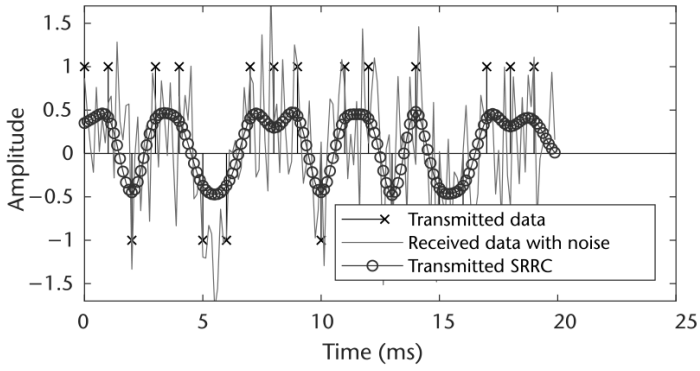
其中 x 是发送符号，h 是发射滤波器的脉冲形状，τ 是分数偏移，T_s 是采样周期，n 是采样索引，v 是加性信道噪声。接收后，r 经过接收匹配滤波器，源符号的关系为

$$y(t) = \sum_n x(n) h_A(t - \tau(t) - nT_s) + v_h(t),$$

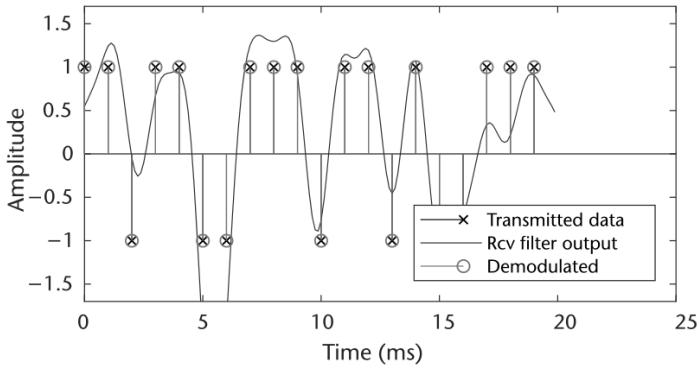
其中 $h_A = h(t) * \overline{h}(-t)$ 是发射滤波器与其共轭的自相关（作用于源数据 x），v_h 是成形后的噪声，y 是输出符号。这印证了我们对匹配滤波与相关的理解。注意时延 τ 也带索引——因为收发两端的振荡器频率不会完全相同，该时延会随时间漂移。不过在实际射频通信系统中，τ 逐符号的变化相对器件采样率应当很小。

如 6.1 节所述，我们会在发送前于发射滤波级对信号插值。虽然这降低了系统吞吐量，但给接收端提供了更多数据做判决，而无需自己再过采样。MATLAB 中我们用 `comm.RaisedCosineTransmitFilter`——它先用多相插值器上采样信号，再施加所需的 RC 或 SRRC 抽头。

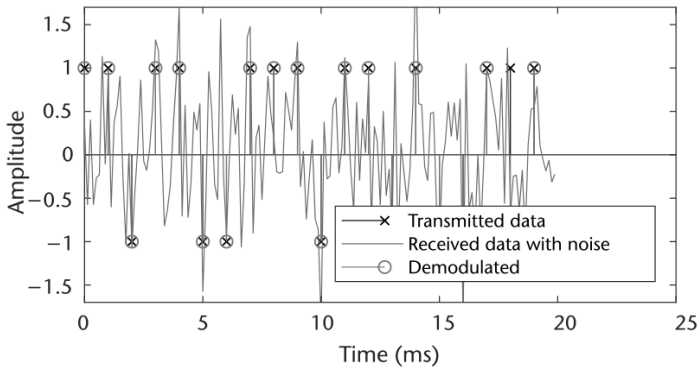
上采样因子 N（也称每符号采样数）将依据所用恢复算法与系统期望数据率选择。一般地，增大 N 能在一定程度上改善接收端的恢复过程，但这同样会压缩有效带宽，迫使硬件以更高速率运行来维持吞吐量。



(a)



(b)



(c)

图 6.7　AWGN 信道后有/无脉冲成形的接收信号对比，匹配滤波的优势一目了然：(a) 发射的 SRRC 滤波信号；(b) 接收的 SRRC 滤波信号；(c) 接收端无 SRRC 滤波的信号

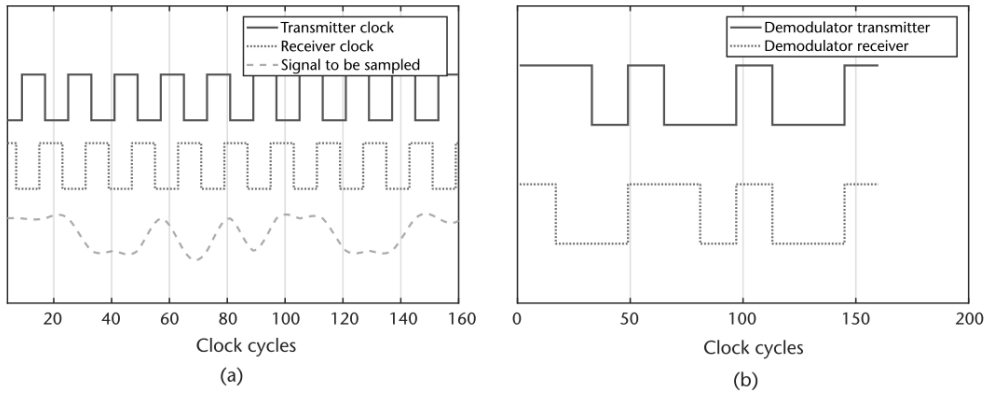


图 6.8　不同时钟定时与模拟波形及采样解调输出的对比：(a) 收发时钟信号与待采样模拟波形；(b) 按各自采样时刻的收发解调器输出

接下来从星座图的视角看定时误差，会观察到符号的聚簇或散布。图 6.9(a) 给出仿真定时偏移 τ 为 0.2N 与 0.5N 的情形（N 为每符号采样数）。0.5N 偏移是最坏情形——恰好落在两个符号正中间。图 6.9(b) 给出经单台 Pluto SDR 自环的 QPSK 信号：可以清楚看到类似的聚簇，以及收发链路缺少相位同步带来的旋转。聚簇的发生是因为接收端实际采样在采样之间的过渡区上。例如符号 $y(n)$ 与 $y(n+1)$ 分别为 $[1+i]$ 与 $[-1-i]$，若在时刻 $n+0.5$ 采样，结果点将接近零。

> 思考题：图 6.10(b) 的 Pluto SDR 星座图为什么出现了旋转？提示：可参考第 5 章。

接收端必须估计未知时延 τ 才能为下游提供正确解调。演示偏移校正的一种粗糙但简单的方法：用多相滤波器对信号做分数重采样。下面的脚本用 `dsp.VariableFractionalDelay`——它对给定延迟实现多相滤波器。我们可以配合 Pluto SDR 演示应提供怎样的延迟来校正偏移。当取到正确值 $\hat{\tau}$（满足 $\hat{\tau}+\tau = kT_s$、$k \in \mathbb{Z}_{\geq 0}$）时，星座将呈现四个清晰的点。

图 6.10 用四个示例延迟作为估计，校正单台 Pluto SDR 自环中的定时失配——它们是上述 MATLAB 脚本的四个运行实例。

代码 6.1　Pluto 自环示例：plutoLoopback.m

```
% User tunable (samplesPerSymbol>=decimation)
samplesPerSymbol = 12; decimation = 4;
% Set up radio
tx = sdrtx('Pluto','Gain',-20);
rx = sdrrx('Pluto','SamplesPerFrame',1e6,'OutputDataType','double');
```

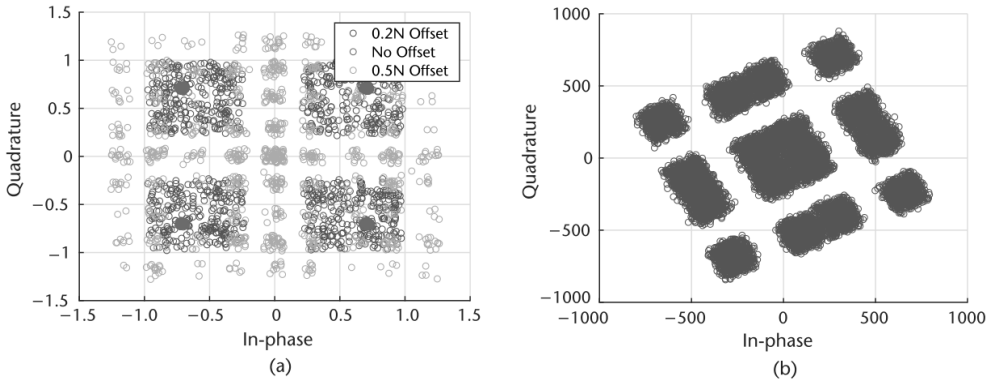


图 6.9　仿真与硬件定时偏移对比：(a) 仅有仿真的多个定时偏移下 QPSK 接收信号示例；(b) 用 Pluto SDR 自环线缆发送 QPSK 信号得到的硬件示例

```
% Create binary data
data = randi([0 1],2^15,1);
% Create a QPSK modulator System object and modulate data
qpskMod = comm.QPSKModulator('BitInput',true); modData = qpskMod(data);
% Set up filters
rctFilt = comm.RaisedCosineTransmitFilter( ...
    'OutputSamplesPerSymbol', samplesPerSymbol);
rcrFilt = comm.RaisedCosineReceiveFilter( ...
    'InputSamplesPerSymbol',  samplesPerSymbol, ...
    'DecimationFactor',       decimation);
% Pass data through radio
tx.transmitRepeat(rctFilt(modData)); data = rcrFilt(rx());
% Set up visualization and delay objects
VFD = dsp.VariableFractionalDelay; cd = comm.ConstellationDiagram;
% Process received data for timing offset
remainingSPS = samplesPerSymbol/decimation;
% Grab end of data where AGC has converged
data = data(end-remainingSPS*1000+1:end);
for index = 0:300
    % Delay signal
    tau_hat = index/50;delayedsig = VFD(data, tau_hat);
    % Linear interpolation
```

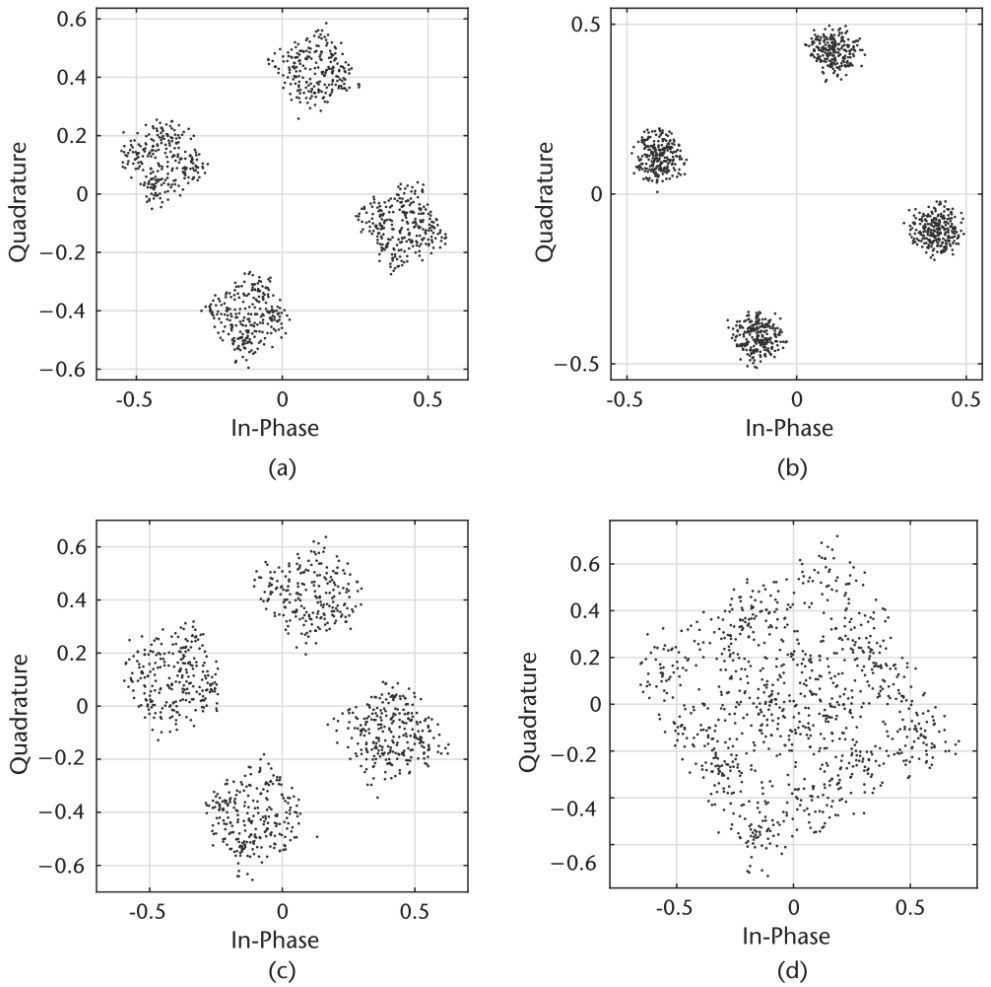


图 6.10　不同分数延迟 $\hat{\tau}$ 下 Pluto SDR 自环数据得到的星座：(a) $\hat{\tau}=0.1$；(b) $\hat{\tau}=0.5$；(c) $\hat{\tau}=1$；(d) $\hat{\tau}=1.5$

```
    o = sum(reshape(delayedsig,remainingSPS,...
      length(delayedsig)/remainingSPS).',2)./remainingSPS;
    % Visualize constellation
    cd(o); pause(0.1);
end
```

> 思考题：用上述 MATLAB 代码验证观测到的定时偏移。它是固定偏移吗？把发射与接收频率都改为 900 MHz，解释观察结果。再把收发采样率都改为 2 MHz，解释你的观察。

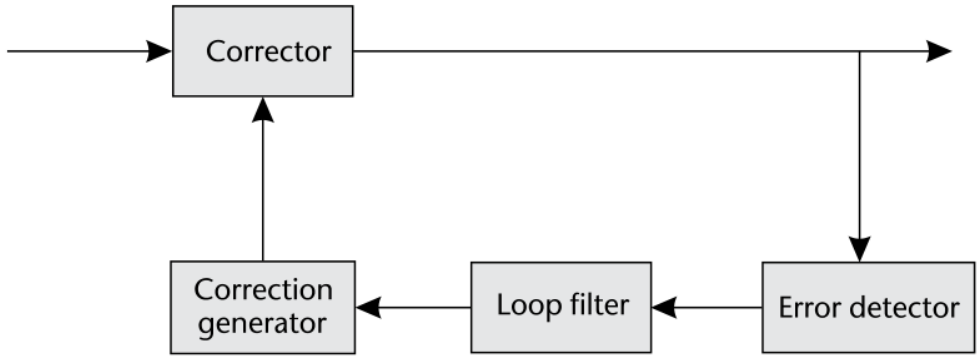


图 6.11　基本 PLL 结构的四个主要组成块

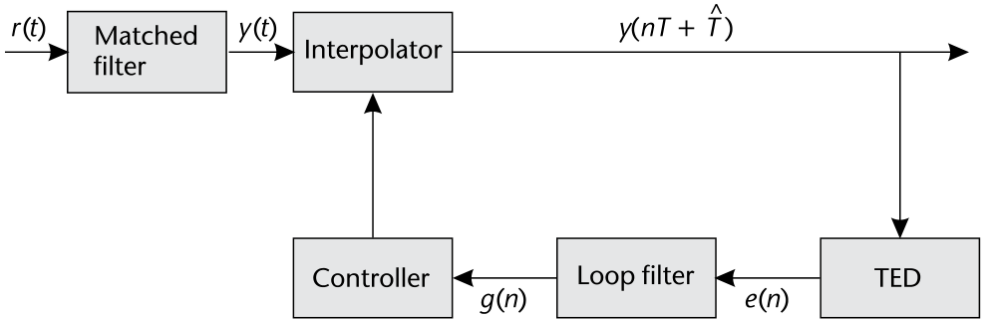


图 6.12　用于定时恢复的 PLL 基本结构（判决引导与盲定时恢复通用）。五个主要块测量、控制并校正接收信号 y 的定时误差

6.3 符号定时补偿

校正收发定时失配有很多方法。本章考察三种数字 PLL 策略，它们与第 7 章载波恢复实现共享同一套方法论。选择这类定时恢复，是因为它能与后续恢复方案集成、稳健、且算法上不过于复杂。先介绍基本 PLL 结构，用于推导反馈定时校正设计。讨论将围绕图 6.11 展开——我们会逐块替换，最终得到图 6.12 的设计。过程中先概念性地概述定时如何同步，再逐个模块讲解设计。所讨论的检测器有：过零（Zero-Crossing）、Müller/Mueller 与 Gardner。更多方法可参考文献中 Mengali [4]、Oerder [5] 等。Rice [6] 对这些技术提供了更深入的分析，覆盖纯模拟设计以及模数混合设计。这里我们聚焦 MATLAB 实现与算法结构特征。

6.3.1 锁相环

我们将使用的定时校正是基于 PLL 理论的反馈（闭环）方法。算法结构见图 6.11（取自 [6] 第 7 章），本质上当误差信号被压到零时环即锁定。PLL 有许多设计与实现策略，这

里勾勒四个基本组件——它们既用于本章的定时校正，也用于下一章的载波恢复，可互相替换。图示的全数字 PLL 算法的工作方式：先在误差检测器（ED）中测量接收采样的某个偏移（如定时误差或相位误差），称为误差信号 e。ED 依据期望接收星座/符号的结构或序列本身的性质设计。接着，环滤波器掌管整个 PLL 的动态——它可以决定工作范围、锁定时间与 PLL 的阻尼/响应速度。再往下是校正生成器，负责为输入生成校正信号并反馈回系统。最后是校正器本身，依据校正生成器的输入修改输入信号。这些块协同工作，最终应使 e 随时间最小化，并持续适应环境或系统自身的后续变化。

校正生成器、误差检测器与校正器随 PLL 的用途（定时恢复或载波恢复）而不同；但环滤波器只需调整数值配置即可在各设计间共享。环滤波器是所有 PLL 设计中最具挑战的部分，却也是对系统自适应控制力最强的部分。这里用比例-积分（PI）滤波器作环滤波器，其传递函数很简单：

$$F(s) = g_1 + \frac{g_2}{s}, \tag{6.4}$$

其中 g_1、g_2 是可选增益。PI 滤波器构成二阶 PLL，传递函数只含一个极点，因此分析起来相对容易。既然处理的是离散时间信号，z 域表示更合适：

$$F(z) = G_1 + \frac{G_2}{1 - z^{-1}}, \tag{6.5}$$

其中 $G_1 \neq g_1$、$G_2 \neq g_2$。[1] (6.5) 的分数部分可以用双二阶（biquad）滤波器优雅地实现。[2] 增益值 (G_1, G_2) 的计算基于期望的阻尼因子 ζ 与环路带宽 B_{Loop}：

$$\theta = \frac{B_{\text{Loop}}}{M(\zeta + 0.25/\zeta)}, \qquad \Delta = 1 + 2\zeta\theta + \theta^2$$

$$G_1 = \frac{4\zeta\theta}{\Delta M} \qquad G_2 = \frac{4\theta^2}{\Delta M}$$

其中 M 是输入信号的每符号采样数。注意 B_{Loop} 是归一化频率，取值范围 $B_{\text{Loop}} \in [0, 1]$。对推导感兴趣的读者见 [6] 附录 C。$\zeta$ 的选择：

$$\zeta = \begin{cases} < 1, & \text{欠阻尼} \\ = 1, & \text{临界阻尼} \\ > 1, & \text{过阻尼} \end{cases}$$

它决定 PLL 的响应速度与稳定性。

[1] 在 (6.4) 与 (6.5) 之间转换的一个简单办法是使用双线性变换。[2] 双二阶滤波器的简单实现见 `dsp.BiquadFilter`。

6.3.2 反馈定时校正

三种算法共用的定时同步 PLL 由四个主块组成：插值器、定时误差检测器（TED）、环滤波器与插值控制器。它们的交互与数据流见图 6.14。这些操作先估计未知偏移误差、按比例缩放误差，再施加更新供后续符号校正。为了给定时误差一个直观视角，看图 6.13 的眼图。该眼图上采样了二十倍，以便细看过渡——注意它们不像图 6.6 那样粗糙。最优情形下，我们选择在眼最张开处的深灰时刻采样输入信号；但会存在未知分数时延 τ 使采样周期移位——浅灰标记即移位后的采样位置。为绕开这个问题，接收信号通常不做完全抽取（并不总是如此），给接收端留每符号多个采样。于是，若像黑色标记那样" 跨坐" 在最优采样位置两侧，只需在这些点之间插值即可得到期望周期。该插值对采样产生分数时延的效果，本质上是移到眼图中的新位置。既然 τ 未知，必须正确加权这次插值，才不会过冲或欠冲期望的校正量——这与 6.2 节结尾的思路一致。最后，控制" 何时从输入数据产出/采样输出" 的时刻是插值控制块的功能，它工作在符号率。这个校正环实现得当时，能让带时钟失配的输入信号的眼图张开；而星座图（如图 6.4 与 6.9）也可作为评估定时校正的工具。

我们先从**过零**（ZC）方法的视角讨论图 6.12 中各块的细节（它最易理解），再推广到其他方法。顾名思义，ZC 在某个采样位置恰好位于过零点时产生零误差信号 $e(n)$。ZC 要求每符号两个或更多采样，使另一采样位置落在或接近最优位置。ZC 的 TED [4] 计算为

$$
\begin{aligned}
e(n) = {} & \mathrm{Re}\left(y\left((n-\tfrac{1}{2})T_s+\tau\right)\right)\left[\mathrm{sgn}\{\mathrm{Re}(y((n-1)T_s+\tau))\}-\mathrm{sgn}\{\mathrm{Re}(y(nT_s+\tau))\}\right] \\
& + \mathrm{Im}\left(y\left((n-\tfrac{1}{2})T_s+\tau\right)\right)\left[\mathrm{sgn}\{\mathrm{Im}(y((n-1)T_s+\tau))\}-\mathrm{sgn}\{\mathrm{Im}(y(nT_s+\tau))\}\right], \quad (6.9)
\end{aligned}
$$

其中 Re 与 Im 提取采样的实部与虚部，sgn 取采样的符号（−1 或 1）。注意 (6.9) 中的索引是按**采样**而非符号；具体地说，$y(nT_s+\tau)$ 就是插值器滤波器的最新输出。看 (6.9)：sgn 操作先给出误差的方向，而补偿所需的位移由中点确定。同相与正交部分独立运作——这很理想。

算出误差后交给环滤波器——完全可以照搬 6.3.1 节的。原则相同，公式形式几乎一

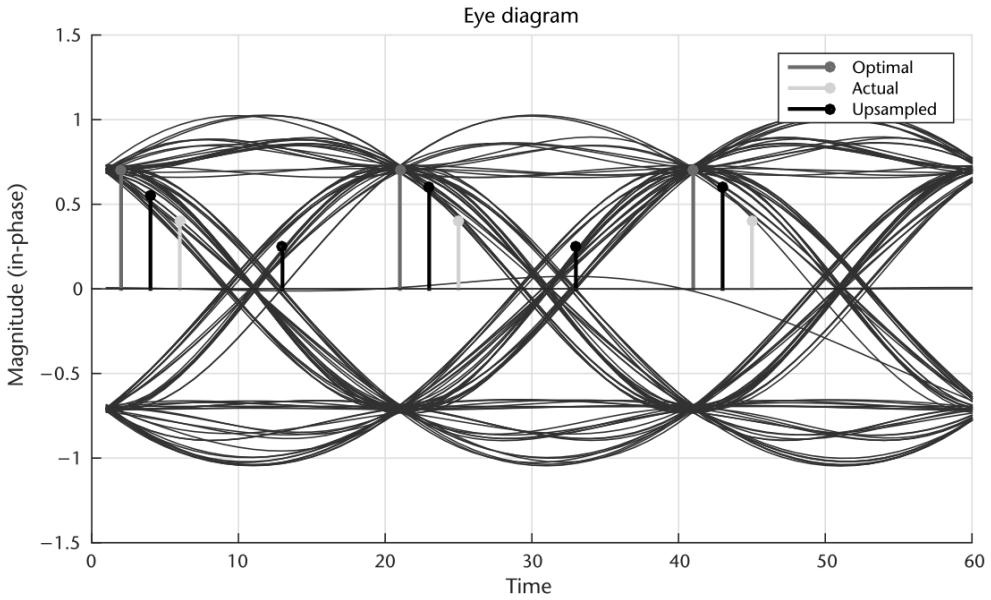


图 6.13　接收信号的眼图，标记接收采样可能处的位置。图为展示多个位置而高度过采样，实际接收采样可落在眼上任意处

致，只是写得更紧凑：

$$G_1 = \frac{-4\zeta\theta}{G_D N} \qquad G_2 = \frac{-4\theta^2}{G_D N}$$

这里 B_{Loop} 是归一化环路带宽，ζ 是阻尼因子，N 是每符号采样数，G_D 是检测器增益。新变量 G_D 为校正提供额外的步长缩放。环滤波器的目的同样是维持校正速率的稳定。该滤波器可用简单线性方程实现：

$$y(t) = G_1 x(t) + G_2 \sum_{n=0} y(n),$$

或用 biquad 滤波器。

下一个要考虑的块是**插值控制器**，负责向插值器提供必要的信令。相对图 6.11 的原始 PLL 结构，插值控制器取代了校正生成器的位置。既然插值器负责对信号做分数延迟，控制器必须提供该信息以及通常的起始插值样本。所谓起始插值样本，指" 跨坐" 左侧的那个采样——如图 6.13 中左起第二个黑色采样位置。这里实现的插值控制器利用基于计数器的机制，在恰当的符号位置有效触发。在这些触发位置，插值器收到信令并更新，同时系统产出一个输出符号。

计数器控制器的核心思想：维持插值器更新之间特定的触发间隔，其平均更新周期

等于输入流的符号率 N。图 6.14 给出了插值控制器的逻辑流程图，便于理解这个复杂的流程。考虑定时最优、环滤波器输出 $g(n)$ 为零的情形：我们希望每 N 个采样产生一次触发。因此计数器的加权/递减量取

$$d(n) = g(n) + \frac{1}{N}$$

在对计数器 $c(n)$ 做" 模 1 减法" 下最大值为 1，每 N 次减法发生一次模回绕。该模计数器更新定义为

$$c(n+1) = (c(n) - d(n)) \bmod 1.$$

我们在更新计数器之前判定触发条件，依据模回绕是否将发生：

$$\text{Trigger} = \begin{cases} \text{True}, & c(n) < d(n) \\ \text{False}, & \text{Otherwise} \end{cases}$$

该触发信号定义了新符号的起点，因此也可用来确保我们在正确的采样上估计误差。触发发生时更新 $\mu(n)$——插值点与最优采样位置之间的估计间隔。该更新是新计数器步长 $d(n)$ 与当前计数 $c(n)$ 的函数：

$$\mu(k) = c(n)/d(n).$$

这个 μ 将传给插值器以更新其施加的延迟。

我们要避免跨越多个符号做定时估计——那会产生错误误差信号与错误更新。可以在 TED 块中加入条件来避免。图 6.15 给出了 TED 块的附加结构及辅助逻辑，说明如何有效利用触发信号。基于该 TED 结构，只有触发发生时输出误差 e 才可能非零。

往图 6.14 的 TED 下游看：由于我们只用 PI 环滤波器，只有非零输入才能更新输出、进而修改与 d 关联的触发周期。系统进入稳态（PLL 锁定）后，TED 输出可每 N 个采样非零一次。

定时恢复还没讨论的最后一块是插值器本身。相对图 6.11 的原始 PLL 结构，插值器取代了校正器的位置。这里的插值只是当前与历史输入 y 的线性组合——本质上可看作一个滤波器。然而，要用 FIR 滤波器实现任意延迟 $\tau \in [0, \ldots, T_s]$ 是做不到的 [7]。理想插值 IIR 滤波器的实现确实存在，但其抽头的计算在实际系统中不可行 [8]。因此我们采用 FIR 低通滤波器的自适应实现——**分段多项式滤波器**（PPF）[6]。PPF 只能提供多项式阶数以内的偏移估计。替代实现有多相滤波器组设计，但依所需分辨率相位

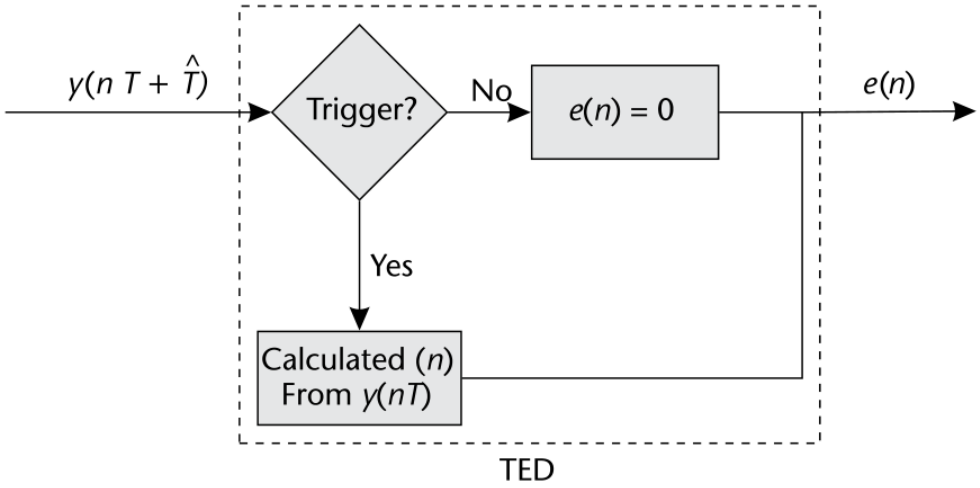


图 6.14　定时误差检测器的内部视图，概述误差与触发控制信号同图 6.14 中其他模块操作的关系

数可能变得很大；不过它们实现起来可以很直接 [9]。

PPF 的实用之处：可以通过选择滤波器阶数轻松控制插值形式——最高等效于用于估计底层接收信号的多项式阶数。这里用二阶（二次）插值，需要四抽头滤波器。插值器输出的一般形式为

$$y(kT_s + \mu(k)T_s) = \sum_{n=1}^{2} h(n)y((k-n)T_s),$$

其中 h_k 是时刻 k 的滤波器系数，由 [10] 给出：

$$\mathbf{h} = [\alpha\mu(k)(\mu(k)-1),\ -\alpha\mu(k)^2-(1-\alpha)\mu(k)+1,\ -\alpha\mu(k)^2+(1+\alpha)\mu(k),\ \alpha\mu(k)(\mu(k)-1)],$$

其中 $\alpha = 0.5$。$\mu(k)$ 与分数延迟相关，由插值控制块提供——它把符号周期 T_s 与估计偏移关联起来。于是可以把真实延迟估计为

$$\hat{\tau} \approx \mu(k)T_s.$$

无偏移（$\mu = 0$）时，插值器表现为两采样延迟——即 ZC 实现下的单符号延迟。可以把 PPF 扩展到更多采样实现三次及更高次插值，但实现随之更复杂。确定插值器实现与所需自由度时，应考虑底层波形能否被准确刻画。

这个用二次形式、四个采样的设计可认为是不规则的——抽头数达不到三。但奇数

长度实现（用奇数个采样）并不可取，因为我们要找的是所给采样**之间**的值；我们也不想要两采样实现——考虑到图 6.13 中眼的曲率。

MATLAB 中几行代码即可实现该插值器，依赖输入数据 y 与插值控制器的最近输出 mu，见代码 6.2。

代码 6.2　插值器：interpFilter.m

```
% Define interpolator coefficients
alpha = 0.5;
InterpFilterCoeff = ...
    [ 0,       0,         1,       0;    % Constant
     -alpha, 1+alpha, -(1-alpha), -alpha; % Linear
      alpha,  -alpha,    -alpha,   alpha]; % Quadratic
% Filter input data
ySeq = [y(i); InterpFilterState]; % Update delay line
 % Produce filter output
filtOut = sum((InterpFilterCoeff * ySeq) .* [1; mu; mu^2]);
InterpFilterState = ySeq(1:3); % Save filter input data
```

由输出 filtOut 可以用 ZC 方程 (6.9) 驱动 TED，为环滤波器与插值控制器生成误差信号。依据图 6.14，该 TED 计算基于插值控制器来的触发信号。还会检查历史触发，防止定时环的输出驱动快于符号率。这段逻辑与 TED 测量见代码 6.3。

代码 6.3　ZC TED：zcTED.m

```
% ZC-TED calculation occurs on a strobe
if Trigger && all(~TriggerHistory(2:end))
    % Calculate the midsample point for odd or even samples per symbol
    t1 = TEDBuffer(end/2 + 1 - rem(N,2));
    t2 = TEDBuffer(end/2 + 1);
    midSample = (t1+t2)/2;
    e = real(midSample)*(sign(real(TEDBuffer(1)))-sign(real(filtOut))) ...
        imag(midSample)*(sign(imag(TEDBuffer(1)))-sign(imag(filtOut)));
else
    e = 0;
end
```

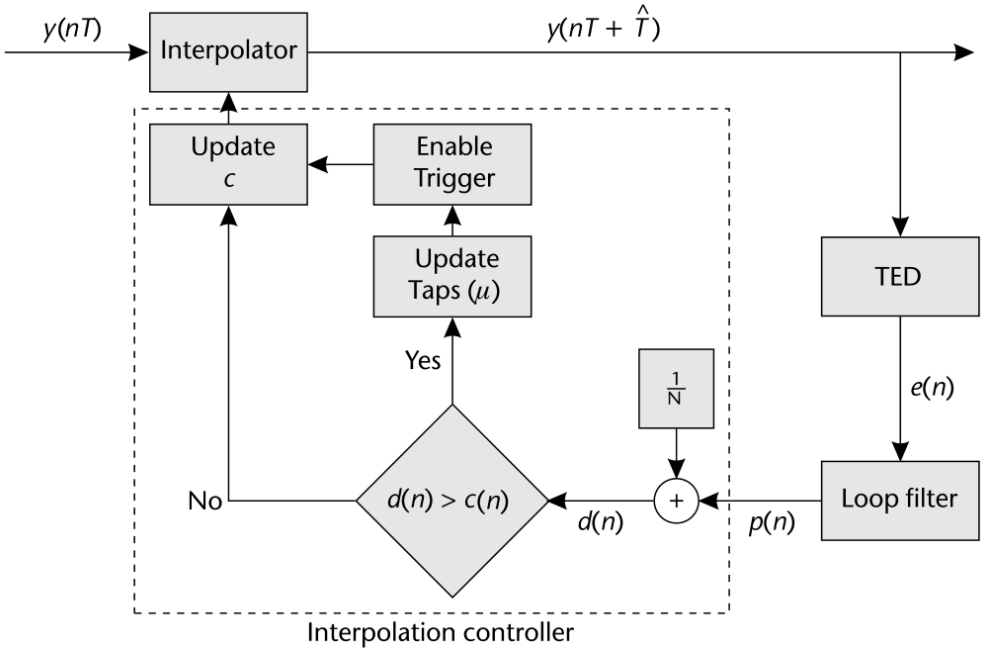


图 6.15　定时恢复触发逻辑，用于维持对输入信号的精确插值

```
% Update TED buffer to manage symbol stuffs
switch sum([TriggerHistory(2:end), Trigger])
    case 0
      % No update required
    case 1
      % Shift TED buffer regularly if ONE trigger across N samples
      TEDBuffer = [TEDBuffer(2:end), filtOut];
    otherwise % > 1
      % Stuff a missing sample if TWO triggers across N samples
      TEDBuffer = [TEDBuffer(3:end), 0, filtOut];
end
```

TED 的第 13–22 行加入了管理**符号填充**（symbol stuffing）的附加逻辑。符号填充基本上是强制同步例程产生一个额外触发。当时钟偏差迫使插值器最小程度地”跨坐”在目标符号上时就需要它；作为补偿必须插入一个额外的输出符号。注意系统输出端的采样率将等于符号率——本质上在 $N > 1$ 时对信号做了下采样。

TED 之后是环滤波器，6.3.1 节已讨论过。该滤波器很简单，不用滤波对象也能直接

实现；不过用 biquad 滤波器对象代码更紧凑，见代码 6.4。

代码 6.4　环滤波器：loopFilter.m

```
% Loop filter
loopFiltOut = LoopPreviousInput + LoopFilterState;
g = e*ProportionalGain + loopFiltOut; % Filter error signal
LoopFilterState = loopFiltOut;
LoopPreviousInput = e*IntegratorGain;
% Loop filter (alternative with filter objects)
lf = dsp.BiquadFilter('SOSMatrix',tf2sos([1 0],[1 -1])); % Create filter
g = lf(IntegratorGain*e) + ProportionalGain*e; % Filter error signal
```

最后，在插值控制块处评估滤波后的误差。稳态时该块应每 N 个输入采样产生一次触发。该触发信号可视为有效的输出信号，与整个算法的输出数据一致。在本文的代码语境下：时刻 n 的 Trigger 为真时，输入 $n+1$ 处插值滤波器的输出应被下游处理。插值控制器自身利用滤波后误差信号 g，随数据处理更新内部计数器，见代码 6.5。

代码 6.5　插值控制逻辑：interpControl.m

```
% Interpolation Controller with modulo-1 counter
d = g + 1/N;
TriggerHistory = [TriggerHistory(2:end), Trigger];
Trigger = (Counter < d); % Check if a trigger condition
if Trigger % Update mu if a trigger
    mu = Counter / d;
end
Counter = mod(Counter - d, 1); % Update counter
```

定时同步器的整体设计可能复杂，各实现工作在不同的相对速率上。因此提供表 6.1 作为推荐实现的指引。这些速率与图 6.14 概述的触发实现一致。当插值器的输出采样与触发对齐时，系统将输出每符号一个采样（N）。

表 6.1: 定时恢复各模块的工作速率

模块	工作速率
插值器	采样率

模块	工作速率
TED	符号率
环滤波器	符号率
插值控制器	采样率

> 思考题：从建模定时偏移的脚本 `timingError.m` 出发，用 ZC 实现定时校正。

6.4 替代误差检测器与系统要求

在上述 PLL 框架内，若应用或系统配置不同，可使用替代 TED。例如，前面讨论的 ZC 方法无法在载波相位或频偏下工作——这样的非理想性必须先补偿才能用 ZC，其他方法则不然。除载波偏移外，ZC 还要求上采样因子 N 至少为 2，受带宽与数据率约束的某些系统可能做不到。

6.4.1 Gardner

第二种 TED 叫 Gardner [11]，与 ZC 非常相似。误差信号由下式确定：

$$e(n) = \mathrm{Re}\left(y\left((n-\tfrac{1}{2})T_s+\tau\right)\right)\left[\mathrm{Re}(y((n-1)T_s+\tau)) - \mathrm{Re}(y(nT_s+\tau))\right]$$

$$+ \mathrm{Im}\left(y\left((n-\tfrac{1}{2})T_s+\tau\right)\right)\left[\mathrm{Im}(y((n-1)T_s+\tau)) - \mathrm{Im}(y(nT_s+\tau))\right].$$

该方法同样要求每符号两个采样，与 ZC 的差别仅在对误差方向的量化上。Gardner 的一个有用特性是不需要载波相位校正，对 BPSK 与 QPSK 信号尤其有效。但由于 Gardner 不是判决引导方法，为获得最佳性能，发射滤波器的过剩带宽应取 $\beta \in (0.4, 1]$。

> 思考题：在你现有的定时误差检测器中实现 Gardner TED。向接收信号引入 π/4 的小相移，比较 ZC 与 Gardner 在这些情形下的表现。

6.4.2 Müller 与 Mueller

接下来是以 Kurt Müller 与 Markus Müller 命名的 Müller-Mueller（MM）方法 [12]。它可以视为最高效的方法——不需要对源数据上采样，工作在每符号一个采样。误差信号由 [6] 给出：

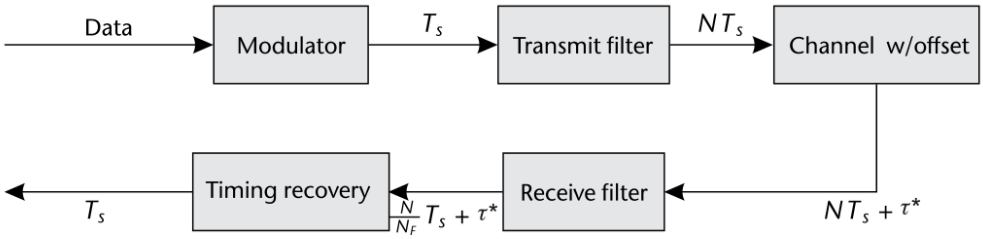


图 6.16　收发链路各阶段相对采样率的速率关系。τ* 表示定时移位而非数据率提升，此处对记号略有滥用

$$e(k)=\mathrm{Re}(y(kT_s+\tau))\times\mathrm{sgn}\{\mathrm{Re}(y((k-1)T_s+\tau))\}-\mathrm{Re}(y((k-1)T_s+\tau))\times\mathrm{sgn}\{\mathrm{Re}(y(kT_s+\tau))\}$$

$$+\mathrm{Im}(y(kT_s+\tau))\times\mathrm{sgn}\{\mathrm{Im}(y((k-1)T_s+\tau))\}-\mathrm{Im}(y((k-1)T_s+\tau))\times\mathrm{sgn}\{\mathrm{Im}(y(kT_s+\tau))\}.$$

MM 同样在匹配滤波尽量减小过剩带宽（即 β 小）时工作最佳。注意当收/发滤波器的过剩带宽高时，sgn 操作产生的判决可能无效——实现时必须考虑这个折中。不过，尽管 MM 技术上最高效，$N=1$ 时由于每符号可用信息少，性能可能存疑。

> 思考题：向输入信号加相位与频偏，比较 ZC、Gardner 与 MM 估计方法的性能。对固定的分数延迟 $\frac{T_s}{2}$、$\frac{T_s}{4}$、$\frac{T_s}{5}$ 做此事，并画出 Gardner 与 ZC 的 TED 误差输出。

6.5 组装成整体

本章我们勾勒了基于 PLL 的定时恢复算法的结构与逻辑及配套 MATLAB 代码。本节讨论把算法组件组装起来，并对评估期间发生的事情给出直觉，同时讨论参数化及其与系统动态的关系。

系统级脚本贯穿着一个不变的主题：数据调制 → 发射滤波 → 经带定时偏移的信道 → 再次滤波 → 定时恢复。这一串步骤中可能发生多次速率变化。为帮助理解这些关系，可以像图 6.16 那样把它们画出来。调制器产出的符号等于采样率；经过发射滤波器获得上采样因子 N，每符号采样数升到 N；接收端可在接收滤波器中按因子 $N_F \le N$ 抽取；最后跨剩余采样执行定时恢复、移除分数偏移 τ，回到每符号一采样的原始速率。图 6.16 概述的速率模式与代码 6.1 的第一个 MATLAB 脚本一致。该脚本可修改为产生略微动态的定时偏移，如下：

代码 6.6　发射滤波数据：srrcFilterData.m

```
% User tunable (samplesPerSymbol>=decimation)
samplesPerSymbol = 4; decimation = 2;
% Create a QPSK modulator System object and modulate data
qpskMod = comm.QPSKModulator('BitInput',true);
% Set up filters
rctFilt = comm.RaisedCosineTransmitFilter( ...
    'OutputSamplesPerSymbol', samplesPerSymbol);
rcrFilt = comm.RaisedCosineReceiveFilter( ...
    'InputSamplesPerSymbol',  samplesPerSymbol, ...
    'DecimationFactor',       decimation);
% Set up delay object
VFD = dsp.VariableFractionalDelay;
% Delay data with slowly changing delay
rxFilt = [];
for index = 1:1e3
    % Generate, modulate, and tx filter data
    data = randi([0 1],100,1);
    modFiltData = rctFilt(qpskMod(data));
    % Delay signal
    tau_hat = index/30;
    delayedsig = VFD(modFiltData, tau_hat);
    rxSig = awgn(delayedsig,25); % Add noise
    rxFilt = [rxFilt;rcrFilt(rxSig)]; % Rx filter
end
```

由代码 6.6 可以评估带时变偏移的接收滤波信号。图 6.17(a) 给出 `samplesPerSymbol` 等于 `decimation` 时 `rxFilt` 的直接输出：观察到信号星座随时间塌缩成多个环——类似图 6.9，本质上就是没有做定时恢复。接下来，运用本章的方法与所提定时恢复实现，可以适应这个变化的分数延迟。图 6.17(b) 演示 $N = 2$ 时 ZC 技术的恢复：信号实部分量之间出现清晰的分层，说明输出星座正在收敛为正确的 QPSK 信号。图 6.17(c) 把 B_{Loop} 从 0.001 提到 0.01，系统反应更快；但收敛后，$B_{\text{Loop}} = 0.001$ 的残余符号噪声比 0.01 时更小。

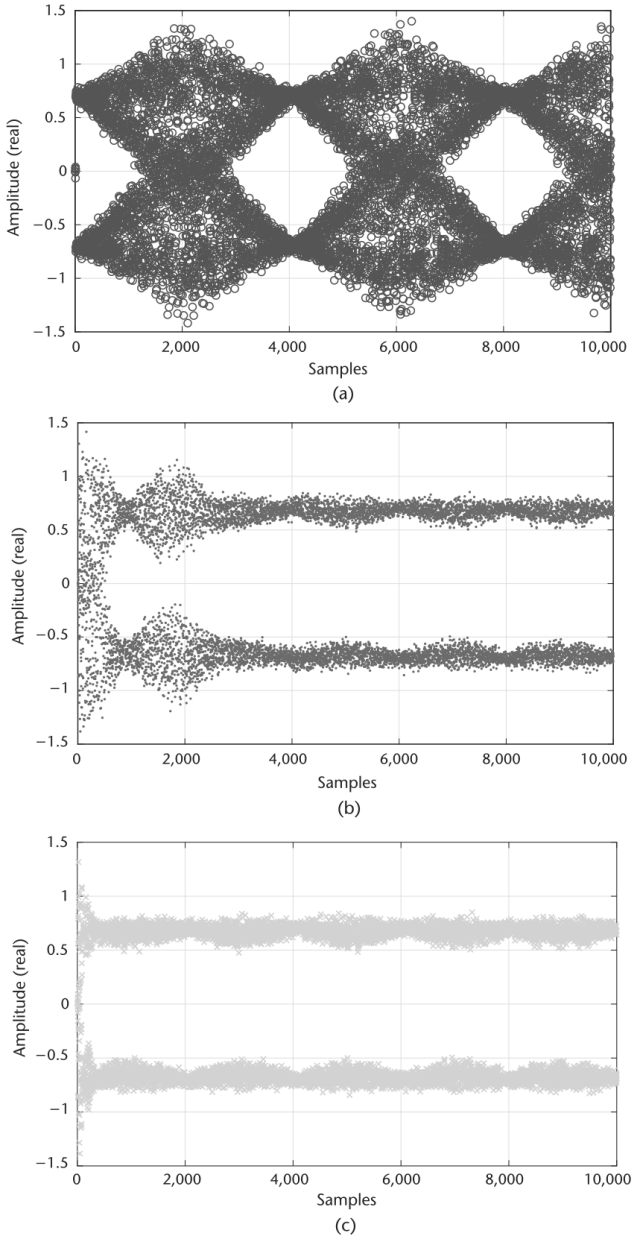


图 6.17　需要定时恢复的信号与应用两种参数化 ZC 定时恢复后输出的对比：(a) 无定时恢复的接收信号；(b) ZC 定时恢复参数 {N, ζ, BLoop, GD} = {2, 1, 0.001, 2.7}；(c) 参数 {2, 1, 0.01, 2.7}

> 思考题：重新生成图 6.17 并使用替代恢复算法 $\zeta = \{0.5, \sqrt{2}, 2\}$。评论恢复算法的动态特性。
>
> 用 Pluto SDR 自环作为信道重新生成图 6.17。调校恢复算法（N, ζ, B_{Loop}, G_D），测出你能达到的最好收敛。

6.6 本章小结

定时恢复是振荡器各自独立的节点间成功传输的基础工具。本章从数学、仿真与 Pluto SDR 演示三个层面引入了定时偏移模型。为对抗该偏移，介绍了基于 PLL 的定时恢复方法论，包含多种定时误差检测器，并对第 2 章引入的匹配滤波做了回顾与延伸。给出了定时恢复算法各组件的 MATLAB 代码，并花了相当篇幅考察它们的配置与交互。最后，在全部组件研究完毕后，探索了设计参数化的部分内容。后续章节将利用这里开发的实现，构建能够恢复两台 Pluto SDR 设备间传输信号的完整接收机。

6.7 参考文献

[1] Proakis, J., and M. Salehi, *Digital Communications*, Fifth Edition, Boston: McGraw-Hill, 2007.

[2] Saleh, A. A. M., "Frequency-Independent and Frequency-Dependent Nonlinear Models of TWT Amplifiers," *IEEE Transactions on Communications*, Vol. 29, No. 11, November 1981, pp. 1715–1720.

[3] Boumaiza, S., T. Liu, and F. M. Ghannouchi, "On the Wireless Transmitters Linear and Nonlinear Distortions Detection and Pre-correction," in *2006 Canadian Conference on Electrical and Computer Engineering*, May 2006, pp. 1510–1513.

[4] Mengali, U., *Synchronization Techniques for Digital Receivers, Applications of Communications Theory*, New York: Springer, 2013.

[5] Oerder, M., and H. Meyr, "Digital Filter and Square Timing Recovery," *IEEE Transactions on Communications*, Vol. 36, No. 5, May 1988, pp. 605–612.

[6] Rice, M., *Digital Communications: A Discrete-Time Approach*, Third Edition, Pearson-/Prentice Hall, 2009.

[7] Laakso, T. I., V. Valimaki, M. Karjalainen, and U. K. Laine, "Splitting the Unit Delay [FIR/All Pass Filters Design]," *IEEE Signal Processing Magazine*, Vol. 13, No. 1, January 1996, pp. 30–60.

[8] Thiran, J. P., “Recursive Digital Filters with Maximally Flat Group Delay,” *IEEE Transactions on Circuit Theory*, Vol. 18, No. 6, November 1971, pp. 659–664.

[9] Rice, M., and F. Harris, “Polyphase Filterbanks for Symbol Timing Synchronization in Sampled Data Receivers,” in *MILCOM 2002, Proceedings*, Vol. 2, October 2002, pp. 982–986.

[10] Erup, L., F. M. Gardner, and R. A. Harris, “Interpolation in Digital Modems. II. Implementation and Performance,” *IEEE Transactions on Communications*, Vol. 41, No. 6, June 1993, pp. 998–1008.

[11] Gardner, F., “A BPSK/QPSK Timing-Error Detector for Sampled Receivers,” *IEEE Transactions on Communications*, Vol. 34, No. 5, May 1986, pp. 423–429.

[12] Mueller, K., and M. Muller, “Timing Recovery in Digital Synchronous Data Receivers,” *IEEE Transactions on Communications*, Vol. 24, No. 5, May 1976, pp. 516–531.

第 7 章 载波同步

本章介绍发射与接收节点之间的**载波频偏**概念。具体地，先讨论一个简化的误差模型，再给出两种恢复方法——依据实现方式它们可以联合或独立工作。载波恢复与上一章（第 6 章）实现的定时恢复互为补充，是维持振荡器各自独立的电台间无线链路所必需的。

本章通篇假设收发电台之间的定时失配已经校正。不过这并非所有情形下的必要条件——尤其本章最初提供的实现中并不是——但对最终实现的最佳性能而言是必要条件。为简单起见，除讨论 Pluto SDR 本身外，我们的仿真将忽略定时效应（那种情形下定时校正显然不可忽视）。对照图 7.1 的完整接收机框图，本章考虑的是载波恢复与 CFO 功能块。

7.1 载波偏移

接收与发射节点通常是两个物理上分离的独立单元。因此，由于杂质、电噪声、温度差异等天然效应，它们的 LO 之间会存在相对频偏。这些差异还可能相对动态，两个 LO 会彼此漂移。这些偏移可包含随机相位噪声、频偏、频率漂移与初始相位失配。但为简单起见，我们只把该偏移建模为固定值——在射频通信的时间尺度上，这是个合理的假设。

考察商用振荡器时，频偏以百万分率（PPM）给出，可以换算成给定频率下的最大载波偏移。Pluto SDR 的内部 LO 额定 25 PPM [1]（校准后 2 PPM），可用 (7.1) 把最大载波偏移 f_o 与工作载频 f_c 联系起来：

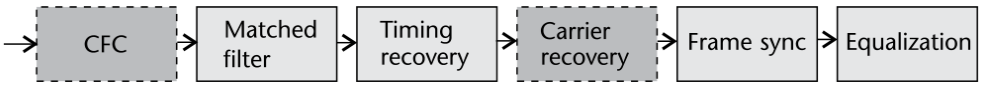


图 7.1　接收机框图

$$f_{o,\max} = \frac{f_c \times \text{PPM}}{10^6}. \tag{7.1}$$

确定 $f_{o,\max}$ 很重要，因为它给出载波恢复的设计准则：把资源浪费在" 校正超出工作范围的频率" 上毫无意义。当然这类情形可以用扫描技术，但超出本书范围。

数学上，可以把基带处带载波频偏 f_o（或 ω_o）的受损源信号 $s(k)$ 建模为

$$r(k) = s(k)e^{j(2\pi f_o kT+\theta)} + n(k) = s(k)e^{j(\omega_o kT+\theta)} + n(k), \tag{7.2}$$

其中 $n(k)$ 是零均值高斯随机过程，T 是符号周期，θ 是载波相位，ω_o 是角频率。

文献中，载波恢复有时被定义为载波**相位**恢复或载波**频率**恢复。它们的目标大体一致：在同步器输出端提供稳定的星座。但理解频率与相位的关系很重要——这能让这些命名惯例清晰起来。角频率 ω（或等效的频率 $2\pi f$）纯粹是相位 θ 随时间变化率的度量：

$$\omega = \frac{d\theta}{dt} = 2\pi f. \tag{7.3}$$

因此，恢复信号的相位本质上就是恢复该信号的频率。通过这一关系得到了估计信号频率的通用方法——频率不像相位那样可以直接测量。代码 7.1 的简单 MATLAB 脚本演示了该技术：先生成给定频率的连续波（CW）单音，测量信号的瞬时相位，再取相位差作为频率估计。任意复信号 $x(k)$ 的瞬时相位 θ 可测得为

$$\theta = \tan^{-1}\frac{\text{Im}(x(k))}{\text{Re}(x(k))},$$

其中 Re 与 Im 分别取信号的实部与虚部。代码 7.1 还用 `hilbert` 函数经希尔伯特变换从实信号生成复正弦。希尔伯特变换非常适合生成实信号的解析（复）表示。想深入了解希尔伯特变换，推荐阅读基于信号处理理论的 Oppenheim [2]。

图 7.2 给出了代码 7.1 的输出。图 7.2(a) 首先可见希尔伯特变换的输出与由正弦（虚部）/余弦（实部）生成的 CW 单音相等。图 7.2(b) 清楚可见基于相位差的估计技术正确估出了信号的频率。脚本还用了 `unwrap` 函数防止相位估计被限制在 $\pm\pi$ 之间。该估计是 (7.3) 关系的直接应用。另外，考察频偏（通常只是较大偏移）时直接看频域也有用——因为仅凭时域信号（尤其含已调数据与噪声时）很难解读这类偏移。图 7.3 给出了原始信号与偏移信号的 PSD，清楚展示了这一视角。信号相对原始信号保持 10 kHz 偏移——完全在 200 MHz 以上通信的 Pluto SDR 的 25-PPM 规格之内。

在频率上搬移复信号是 (7.2) 的简单应用，图 7.3(b) 正是这样生成的。代码 7.2 的

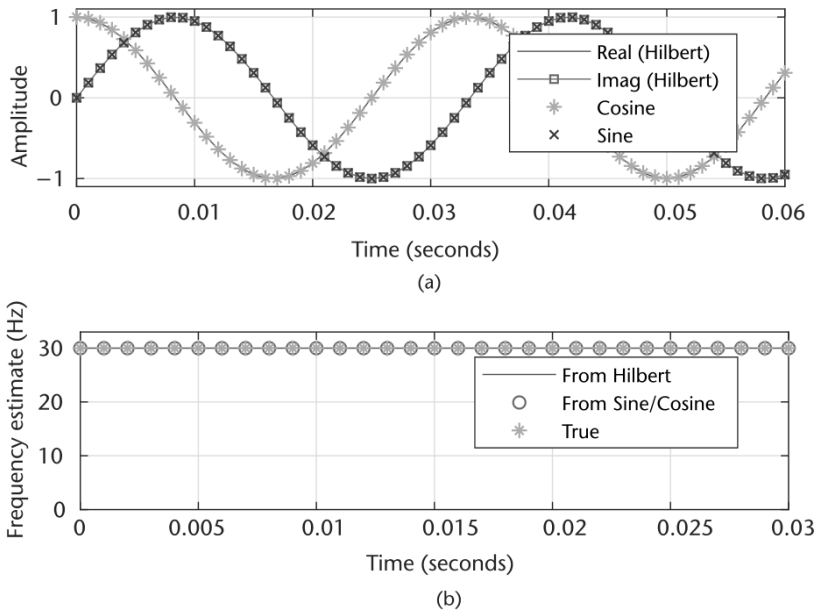


图 7.2　MATLAB 脚本对简单频率估计技术与真实偏移的输出对比：(a) 由正弦/余弦与希尔伯特变换生成的 CW 单音；(b) CW 单音的频率估计

MATLAB 示例脚本演示了如何用指数函数搬移复信号；愿意的话也可以直接用正弦。所给脚本中，先用 SRRC 滤波器（代码 7.2）对信号上采样/过采样很重要——这让频移一目了然，因为主要信号能量只占带宽的一小部分。

代码 7.1　freqEstimate.m

```
% Sinusoid parameters
fs = 1e3; fc = 30; N = 1e3;
t = 0:1/fs:(N-1)/fs;
% Create CW Tone
r = cos(2*pi*fc*t); i = sin(2*pi*fc*t);
% Alternatively we can use a hilbert transform from our real signal
y = hilbert(r);
% Estimate frequency from phase
phaseEstHib = unwrap(angle(y))*fs/(2*pi); freqEstHib = diff(phaseEstHib);
phaseEstCW = unwrap(atan2(i,r))*fs/(2*pi); freqEstCW = diff(phaseEstCW);
tDiff = t(1:end-1);
```

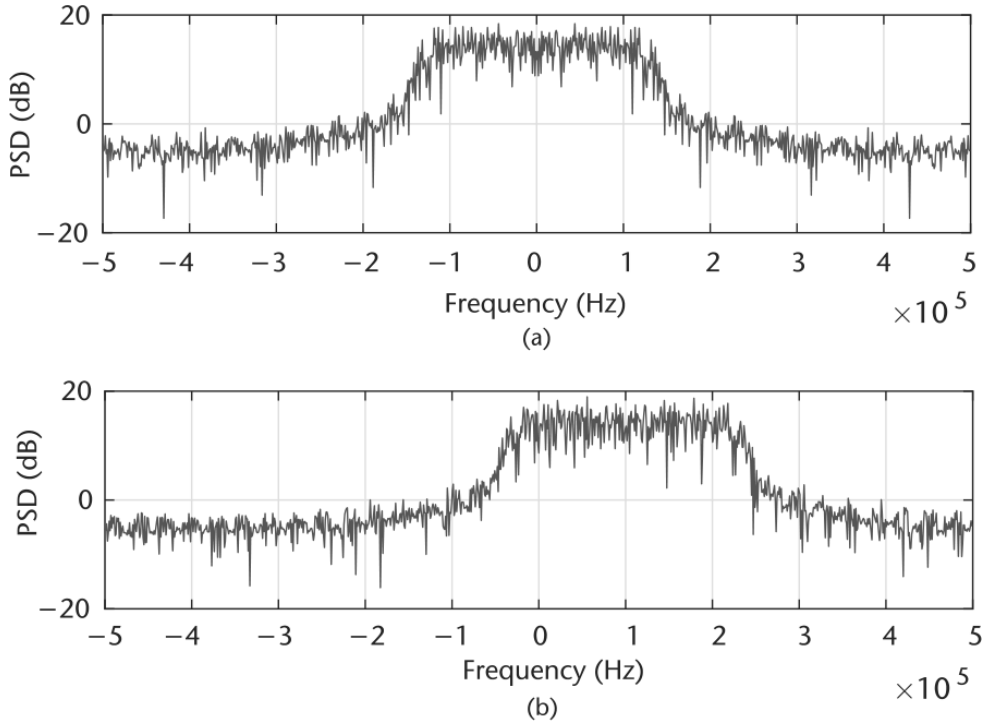


图 7.3　有/无频偏的频域信号对比：(a) 无频偏 BPSK 信号的 PSD；(b) 10 kHz 偏移 BPSK 信号的 PSD

> 思考题：从 MATLAB 代码 7.1 出发，考察该估计技术相对采样率 fs 与单音频率 fc 的频率范围。(忽略希尔伯特变换的输出。) 大致能被正确估计的最大频率是多少？当频率偏移超过这个点会发生什么？

代码 7.2　freqShiftFFT.m

```
% General system details
fs = 1e6; samplesPerSymbol = 1; frameSize = 2^8;
modulationOrder = 2; filterOversample = 4; filterSymbolSpan = 8;
% Impairments
frequencyOffsetHz = 1e5;
% Generate symbols
data = randi([0 samplesPerSymbol], frameSize, 1);
mod = comm.BPSKModulator(); modulatedData = mod(data);
% Add TX Filter
```

```
TxFlt = comm.RaisedCosineTransmitFilter('OutputSamplesPerSymbol',...
    filterOversample, 'FilterSpanInSymbols', filterSymbolSpan);
filteredData = TxFlt(modulatedData);
% Shift signal in frequency
t = 0:1/fs:(frameSize*filterOversample-1)/fs;
freqShift = exp(1i.*2*pi*frequencyOffsetHz*t.');
offsetData = filteredData.*freqShift;
```

> 思考题：修改代码 7.2 中的 filterOversample 并观察频谱，解释你的观察。接着用原始脚本把频率偏移以 0.1Fs 为步长从 0.1Fs 增加到 1.0Fs（Fs 为采样率），解释观察到的效应。

7.2 频偏补偿

设计无线接收机有很多方式：恢复技术与算法编排各异。本节采用" 先频偏、后其余同步任务" 的思路。如 10.3 节所述，Pluto SDR 的振荡器额定 25 PPM。在 2.4 GHz 这样的免许可频段发射信号，两台电台之间可能产生最大 120 kHz 的偏移。这个范围相当大，因此我们将发展**两级**频率补偿技术：粗频偏校正与细频偏校正。这一设计有利——它能缩短相对载波估计的收敛（锁定）时间。

7.2.1 粗频偏校正

文献中粗频偏校正有两大类：**数据辅助**（DA）与**盲校正**。DA 技术利用相关型结构，借助对接收信号的知识（通常是前导码形式）估计载波偏移 f_o。DA 方法虽能给出精确估计，其性能通常受前导码长度限制 [3]——前导码越长，系统吞吐量越低。

另一条路是**盲**或非数据辅助（NDA）方法，可以在信号的整个持续时间上工作。因此可以论证：在现实系统中 NDA 能胜过 DA 算法。这些粗校正技术通常以开环方式实现，便于使用。这里我们概述并实现一种**基于 FFT 的 NDA 技术**做粗校正。思路很直接：基于图 7.3 的初步观察，可以对符号偏移给出粗略估计。但直接取 FFT 的峰值不会很准——尤其当信号在频率上不对称时。为补偿这一点，我们把信号自乘其调制阶数 M 次以去除信号自身的调制分量。由 (7.2) 的模型（忽略噪声）可观察：

$$r^M(k) = s^M(k)e^{j(2\pi f_o kT+\theta)M}.$$

这会把偏移搬到原位置的 M 倍处，并使 $s(t)$ 变为纯实或纯复。于是 $s^M(t)$ 项可以忽

略，只剩下指数（单音）。为估计该单音的位置，对 $r^M(t)$ 取 FFT，把能量最大的 bin 对应到单音位置。图 7.4 是 MATLAB 代码 7.2 生成的 BPSK 信号（偏移 10 kHz）的 $r^M(t)$ 频谱示例——峰值清晰地出现在两倍频偏处，符合预期。该频率估计可以形式化地写成一条方程 [4]:

$$\hat{f}_o = \frac{1}{MTK} \arg\max_k \left| \sum_{i=0}^{K-1} r^M(i) e^{-j2\pi ik/K} \right| \tag{7.6}$$

其中 K 是 FFT 长度。(7.6) 的估计之所以称为" 粗"，是因为 $\hat{f}_o$ 只能取 FFT 产生的 K 个值之一。若有需要，可以通过对多组估计的固定 FFT bin 集合做插值来扩展精度。信号每个 FFT bin 的频率分辨率就是

$$f_r = \frac{1}{MTK}.$$

因此，增大 FFT 尺寸或降低系统采样率都能提高估计器性能。但不要把采样率降到低于感兴趣信号的带宽。

> 思考题：该方法的局限是什么？（M 变大时会怎样？）最后向接收信号加不同 SNR 的 AWGN，考察峰值何时变得难以辨别。给出峰值估计 MSE 对 SNR 的曲线用于分析。

在 MATLAB 中实现该方法很直接；为效率起见，K 应始终取 2 的幂以发挥 FFT 的效率。代码 7.3 中我们对每 K 个采样产生一个估计，并补偿 `fft` 函数的频率排布。使用该技术时还应考虑系统的其他方面或该操作的影响：从下游算法的视角看，它们每 K 个采样会观察到一个频率校正。理想情况 $\hat{f}_o$ 保持不变，但若偏移恰好靠近 FFT bin 边界则不太可能——结果是信号相对先前出现 $\pm f_r$ 的频率跳变。不幸的是，这类突变会扰乱下游的反馈算法——它们不擅长应对所估计/校正信号频率或相位的突然漂移。对策有两条主线：其一，随时间用滤波器对估计取平均，平滑变化；其二，只在帧首施加校正——只要帧尺寸合理，偏移在其间应相对平稳，一次测量在该时段内就够准。该校正也被认为是" 粗" 的——精度只能到 f_r 以内，这本身就限定了这类校正的施加节奏。话虽如此，FFT 技术的弱点是需要相当多的数据才能给出像样的估计；且当发射机用发射滤波器过采样时，产生的也非纯单音。其他技术（如 Luise [5]）专为突发型应用设计、所需数据少于上述 FFT 方法，但遗憾的是 Luise 方法是有偏估计（FFT 方法无偏）。

代码 7.3　fftFreqEst.m

```
%% Estimation of error
```

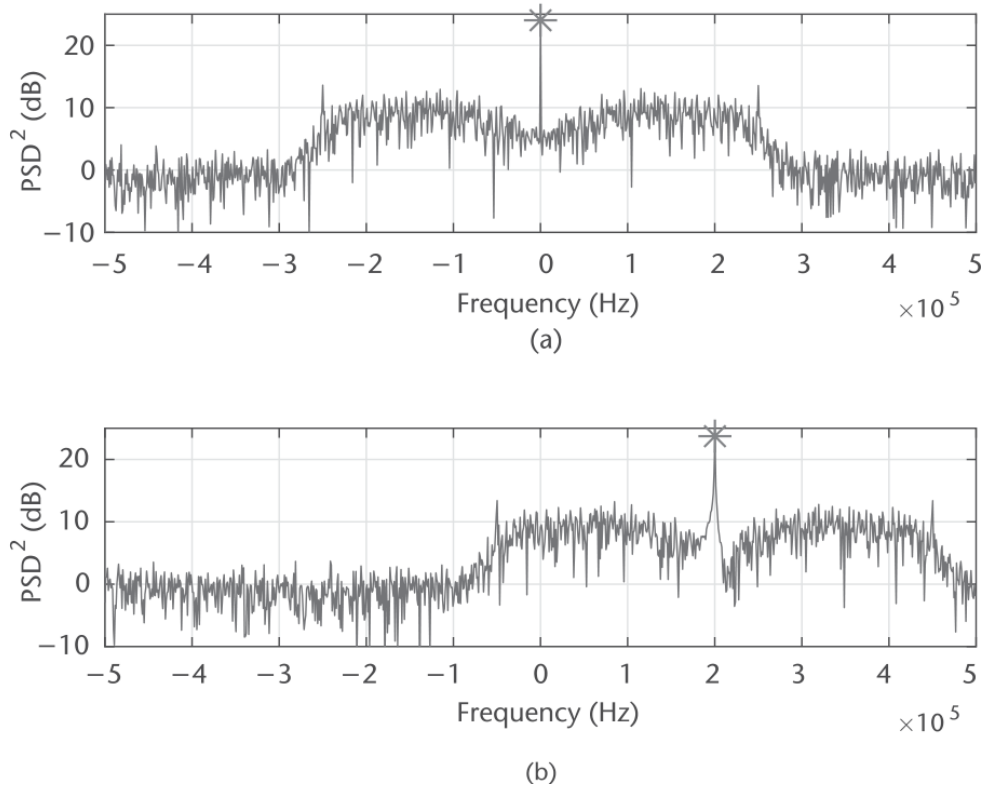


图 7.4　有/无频偏的频域信号对比（平方后）：(a) 无频偏平方 BPSK 信号的 PSD；(b) 10 kHz 偏移平方 BPSK 信号的 PSD

```
fftOrder = 2^10; k = 1;
frequencyRange = linspace(-sampleRateHz/2,sampleRateHz/2,fftOrder);
% Precalculate constants
offsetEstimates = zeros(floor(length(noisyData)/fftOrder),1);
indexToHz = sampleRateHz/(modulationOrder*fftOrder);
for est=1:length(offsetEstimates)
    % Increment indexes
    timeIndex = (k:k+fftOrder-1).';
    k = k + fftOrder;
    % Remove modulation effects
    sigNoMod = offsetData(timeIndex).^modulationOrder;
    % Take FFT and ABS
    freqHist = abs(fft(sigNoMod));
    % Determine most likely offset
    [~,maxInd] = max(freqHist);
    offsetInd = maxInd - 1;
    if maxInd>=fftOrder/2 % Compensate for spectrum shift
       offsetInd = offsetInd - fftOrder;
    end
    % Convert to Hz from normalized frequency index
    offsetEstimates(est) = offsetInd * indexToHz;
end
```

> 思考题：用 Pluto SDR 自环与代码 7.3，测量频率估计的均方误差随收发中心频率差（fo,max）变化的曲线。用 BPSK 信号，在 1 MHz 基带采样率下考察 Δf 从 0 到 100 kHz。再固定发射机增益为 −30，接收机手动增益模式取 10、30、50 重复实验。

7.2.2 细频偏校正

粗频偏校正（CFC）之后，基于所选分辨率 f_r 仍会残留偏移。细频偏校正（FFC）——也称载波相位校正——应为最终解调产生稳定的星座：本质上是把接收信号的残余频偏压到零。之所以说” 产生稳定星座”，是因为细频偏的影响通常就用星座图考察。数字已调信号若有频偏，星座图上会表现为随时间旋转。图 7.5 演示了这一效果：每个数字

代表采样的相对时间次序，给出旋转的视角。信号是 BPSK——随源符号不同跳过原点两侧。若施加正频偏，旋转为逆时针；负频偏则顺时针。旋转速率等于频偏——这正是 ω（角频率）概念的来源，如 (7.3) 所定义。

用 Pluto SDR 也能以类似方式观察该偏移。图 7.6 中我们以自环方式发射 BPSK 信号，收发 LO 相差 1 kHz。图 7.6(b) 中观察到与图 7.5 类似的旋转。为了正确可视化该效果，需要先做定时校正（借自第 6 章）——没有定时校正时星座图难以解读，如图 7.6(a) 所示。由于此情形下定时校正在频率校正之前进行，必须用 6.4.1 节详述的 Gardner 技术。

与采用前馈技术的 CFC 不同，FFC 我们将使用基于 PLL 理论的反馈（闭环）方法（如第 4 章所考察）。算法结构见图 7.7（取自 [6] 第 7 章），并可回溯到图 6.11 的原始轮廓。

该全数字 PLL 算法的工作方式：先在**相位误差检测器**（PED）中测量接收采样的相位偏移，称为误差信号 $e(n)$。PED 依据期望接收星座/符号的结构设计。接着，环滤波器掌管整个 PLL 的动态——它决定工作频率范围（有时称捕获带）、锁定时间与响应速度，并把误差信号变平滑。最后是**直接数字合成器**（DDS）——这个名字是模拟 PLL 压控振荡器（VCO）设计的遗迹。DDS 负责为输入生成校正信号并反馈回系统。就 FFC 设计而言，该 PLL 最终应产出偏移为零的期望输出信号。

从 PED 开始。该块的目标很简单：测量输入复数据相对期望参考星座的相位（径向）偏移。所谓参考（以及 $e(n)$），实际指到星座基的距离。对 QAM、PSK、PAM 而言，基就是实轴与虚轴；但面对 FSK 等其他调制可能需要扩展该视角。这一思路的根本理由是：它去除了特定维度的缩放因素，转而考察特定基下能量或幅度之比。为更好理解，考虑 QPSK，其 PED 方程为

$$e(n) = \text{sign}(\text{Re}(y(n))) \times \text{Im}(y(n)) - \text{sign}(\text{Im}(y(n))) \times \text{Re}(y(n)). \tag{7.8}$$

(7.8) 中 $e(n)$ 本质上测量 $y(n)$ 实部与虚部之差，仅当 $\text{Re}(y(n)) = \text{Im}(y(n))$ 时为零。注意这只会把输出星座拉到**特定取向**，而非特定范数值。若 $y(n)$ 需要别的取向，可在同步器之后乘以期望相移 φ_{POST} 完成：

$$y_{\text{SHIFT}}(n) = y(n)e^{j\varphi_{\text{POST}}}.$$

注意 $y_{\text{SHIFT}}(n)$ 不应送入 PED。

其他调制方案的 PED 误差估计随期望信号排布而变。例如 BPSK 或 PAM 的误差估计为

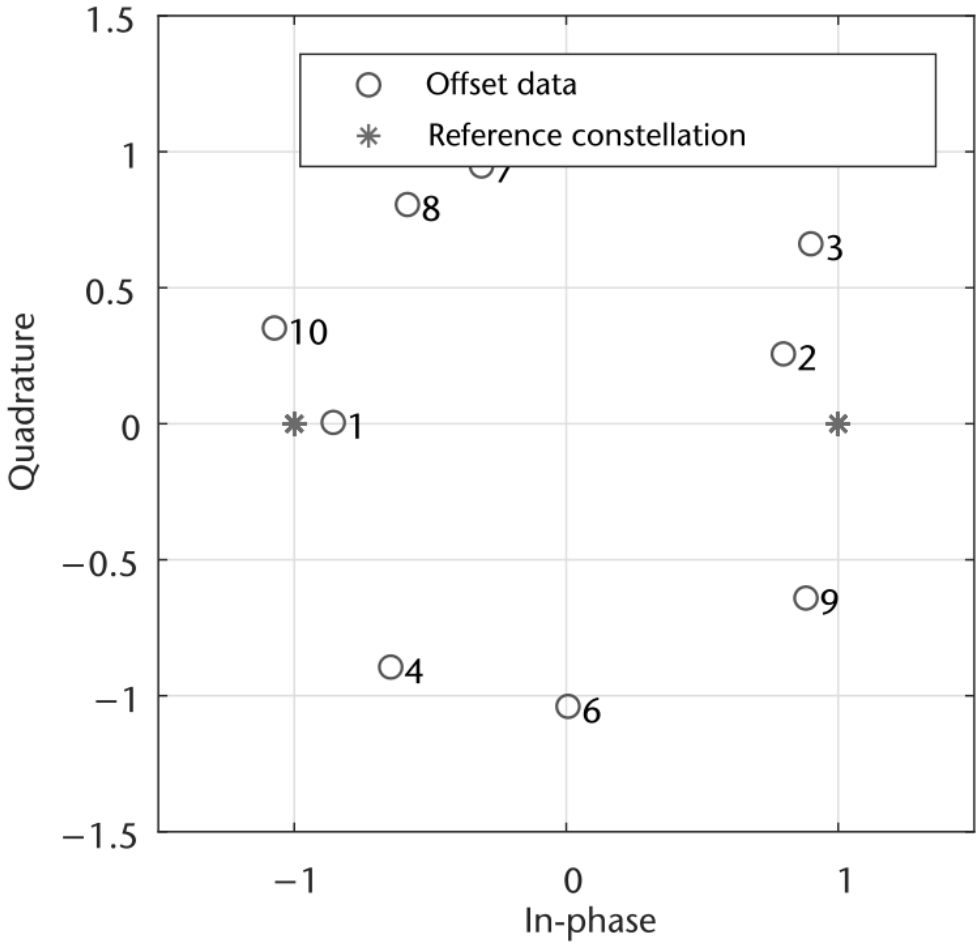


图 7.5　带频偏的 BPSK 源信号的旋转星座

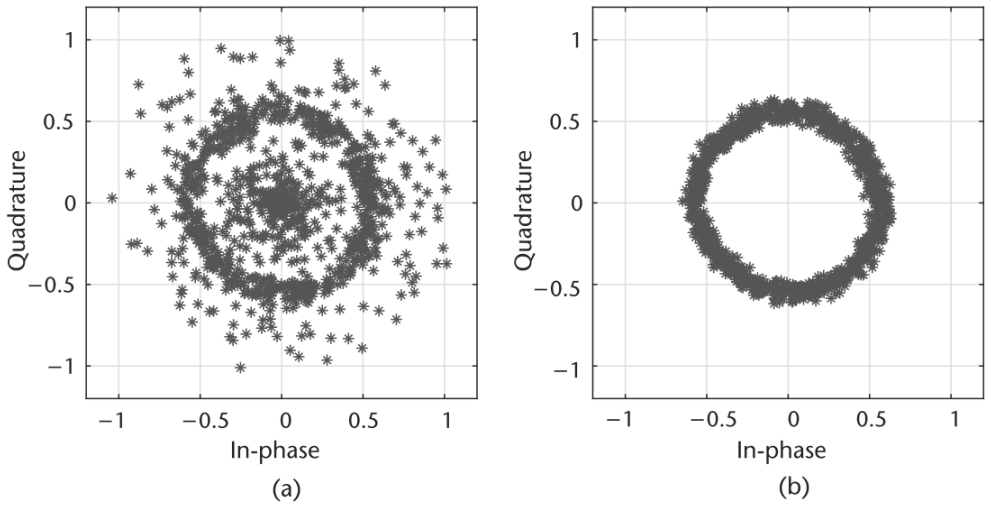


图 7.6　BPSK 信号经 Pluto SDR 自环发射，1 MHz 下 1 kHz 偏移：(a) 定时校正前；(b) 定时校正后

$$e(n) = \text{sign}(\text{Re}(y(n))) \times \text{Im}(y(n)). \tag{7.10}$$

该 PED 误差函数的目标同样只是提供 $y(n)$ 的取向误差：(7.10) 中仅当 $y(n)$ 纯实时 $e(n)$ 为零。

(7.8) 与 (7.10) 的道理很直接。另一方面，环滤波器仍是所有 PLL 设计中最难的部分，但也是对系统自适应控制力最强的部分。这里同样用 6.3.1 节详述的 PI 滤波器作环滤波器。FFC 同步器的最后一块是 DDS——它就是个积分器。环滤波器产出控制信号（等效于输入信号的频率），因此需要改为提取该信号的相位。这里积分器所用传递函数为

$$D(s) = \frac{G_3}{s} \;\rightarrow\; D(z) = G_3 \frac{z^{-1}}{1 - z^{-1}}.$$

注意我们在离散域多加了单采样延迟；由于产出的是校正信号，取 $G_3 = -1$。该积分器同样可用 biquad 滤波器实现。

图 7.7 所示的 PLL 布置中，系统应产出相位与频偏都最小的输出 $y(n)$。沿图 7.7 再绕环走一圈：PED 先产出等于观测到的已校正符号[1] $y(n)$ 相位偏移的误差；环滤波器把该观测误差与全部历史误差加权关联；最后 DDS 把加权后的误差/控制信号 $f(n)$ 转换为相位 $\varphi(n)$，用于校正下一个输入采样 $x(n+1)$。对频偏而言，φ 会持续变化——因为它是相位值而非频率值。但如果输入信号太动态、或滤波器增益设置不当，PLL 将跟不

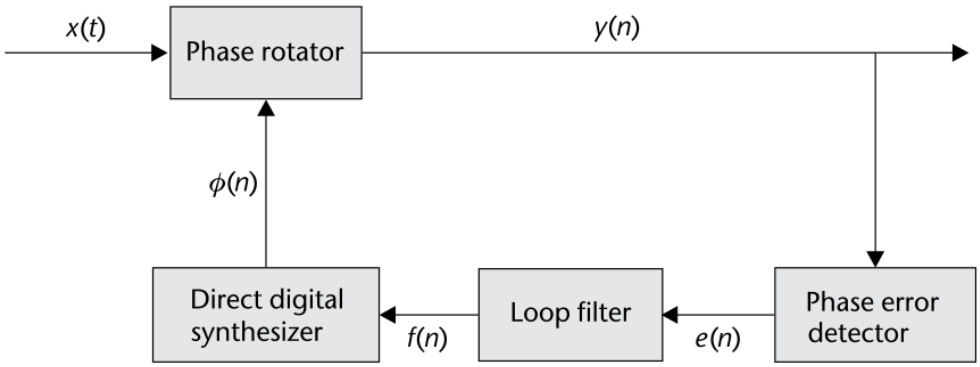


图 7.7　基于 PLL 设计的 FFC 结构，用于反馈校正

上 x 变化的相位（频率）。

[1] 我们称之为”已校正符号”，因为它已经过旋转器、且不会再对其施加额外相移。它也是 PLL 的输出。

环滤波器增益值 (G_1, G_2) 的计算基于期望的阻尼因子 ζ 与环路带宽 B_{Loop}：

$$\theta = \frac{B_{\text{Loop}}}{M(\zeta + 0.25/\zeta)}, \qquad \Delta_1 = 1 + 2\zeta\theta + \theta^2$$

$$G_1 = \frac{4\zeta\theta}{\Delta_1 MK} \qquad G_2 = \frac{(4/M)\theta^2}{\Delta_1 MK}$$

其中 M 是每符号采样数，K 是检测器增益。QPSK 与矩形 QAM 的 $K = 2$，PAM 与 PSK 的 $K = 1$。注意 B_{Loop} 是归一化频率。推导细节可查 [6] 附录 C 的完整分析。ζ 的选择回看 6.3.1 节（定义相同）。B_{Loop} 的选择应联系期望的最大归一化频率锁定范围 Δf_{lock}：

$$\Delta f_{\text{pull}} \approx 2\pi\sqrt{2\zeta B_{\text{Loop}}}. \tag{7.14}$$

注意该值基于 PLL 线性化模型的估计，仿真版本中可能存在出入。但配置得当时，该 PLL 设计即使强噪声下也应表现良好。与 CFC 不同，FFC 一般没有同样大的工作范围。设计中不妨从阻尼因子 $\zeta = 1$、环路带宽 $B_{\text{Loop}} = 0.01$ 起步。经验上此处更倾向过阻尼系统——它直接增大捕获带，代价是环收敛更慢。

> 思考题：从代码 7.4 出发，为 BPSK 实现载波恢复算法。针对归一化频偏 0.001 与 0.004 调校该实现。在一组 SNR 范围上评估两种实现的频率估计 MSE。

现在万事俱备，可以实现 FFC 同步器了——代码 7.4 给出完整参考实现。但有必要讨论几个设计考量。首先我们说过，FFC 同步器的输出可以以星座的特定取向为目标——这完全由 PED 决定。但同步器未必总能达到该目标取向：星座可能略带旋转，或出现在期望位置的倍数角度处。这源于载波偏移超出 FFC 配置的处理能力，或最典型地——系统被配置成欠阻尼。另外，接收信号 SNR 差时也会削弱同步器的有效捕获带、造成不希望的锁定位置。代码 7.4 演示了这些：用了两组不同的 ζ 配置，并把系统驱动到接近该配置估计的最大偏移。一般而言这些估计偏保守，需要针对具体调制方案、SNR 与环滤波器配置做经验测试。本例中考察图 7.8 的收敛信号会注意到一组有趣的结果：图 7.8(b) 的星座实际收敛到了**假极小值**——这是 PLL 处于欠阻尼状态的动力学结果。让系统更" 刚" 一些就会得到图 7.8(a) 的正确结果；但 ζ 太大时同步器不收敛或收敛极慢。

> 思考题：向代码 7.4 的模型引入定时偏移。把你在第 6 章的定时恢复实现接入系统完成恢复。在一组 SNR 范围上评估这些实现的频率估计 MSE。

实现与测试自己的系统时，实际测量同步器自身的频率估计很有用。既然 DDS 的输出 φ 是下一符号所需的瞬时相位校正，直接套用 (7.3)（类似代码 7.1 的计算）即可。由角频率估计可换算成以赫兹计的频率估计（同 (7.3)）。检查代码 7.4 中 Phase(φ) 的导数：对 $f_s = 1000$ Hz 下 20 Hz 的偏移考察估计的收敛。图 7.9 画出了 f_{est}——明显收敛到正确值附近。但由于信号含噪声、PLL 也有固有噪声，估计不会静止。这对真实系统反而有用：收发 LO 之间的偏移永远彼此动态变化。

> 思考题：基于你现有的" 定时 + 载波偏移" 双恢复仿真方案，引入 Pluto SDR 作为信道机制（从代码 6.1 与 7.4 出发可能有帮助）。自环下逐渐增大收发 LO 频差，评估你的实现。

7.2.3 性能分析

评估同步性能可以考虑多个变量：锁定时间、有效捕获带、收敛后的误差矢量幅度（EVM）等等。这些指标应按特定设计的需求做平衡——它们彼此冲突。例如设计一个锁定快的系统不难，但捕获带多半有限。这与 (7.14) 直接相关；另一个度量来自 [6] 附录 C，定

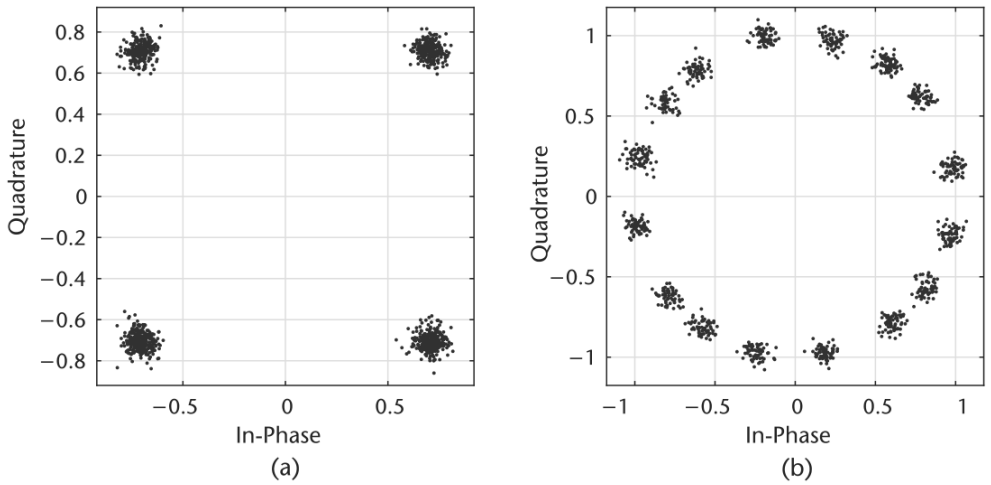


图 7.8　载波恢复后不同阻尼因子收敛的 QPSK 信号：(a) 过阻尼（ζ = 1.3）；(b) 欠阻尼（ζ = 0.9）

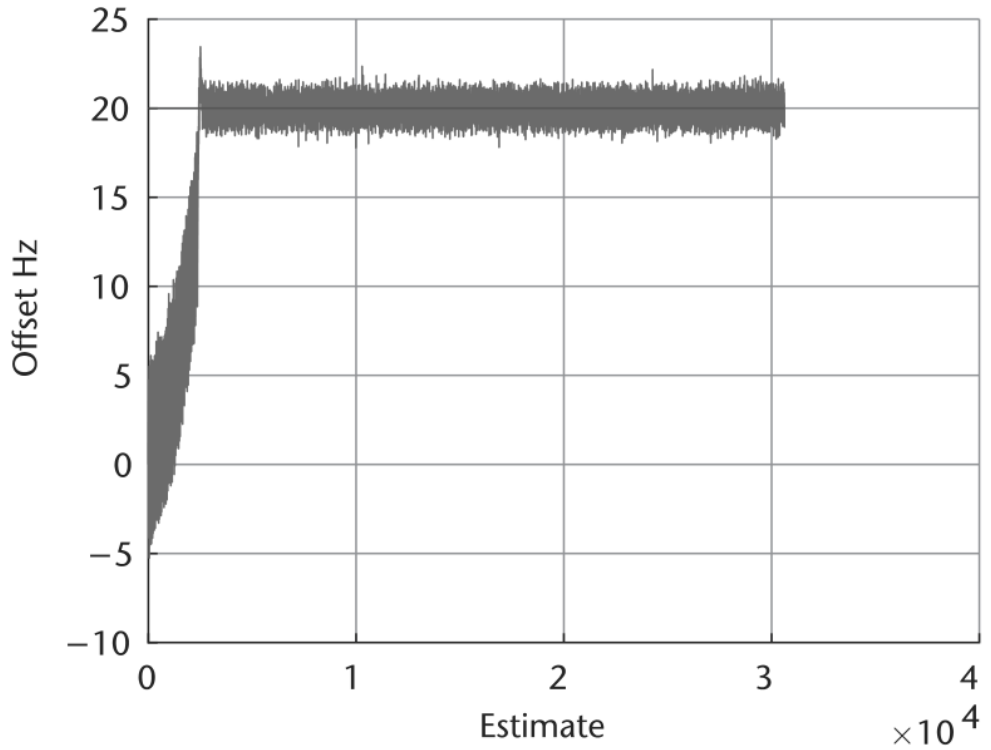


图 7.9　已实现 FFC 对 20 Hz 偏移的估计随时间变化及最终收敛

义了归一化频率锁定时延：

$$t_{\text{lock,Max}} \approx \frac{32\zeta^2}{B_{\text{Loop}}}.$$

聚焦 PED 的误差信号可以直接演示 ζ 与 B_{Loop} 的折中。把代码 7.4 改为固定归一化载波偏移 0.01、$\zeta = 1.3$、取两个不同的 B_{Loop}，可观察图 7.10 中的 $e(n)$。两种配置下归一化偏移都小于 Δf_{pull}：$B_{\text{Loop}} = 0.24$ 时系统几十个采样内收敛；$B_{\text{Loop}} = 0.03$ 时慢一个数量级——但 $B_{\text{Loop}} = 0.03$ 情形收敛后误差信号方差 σ_e^2 却只有前者（0.24 情形）的三分之一。该误差会表现为 $y(n)$ 上的相位噪声，影响信号的解调正确性。

7.2.4 误差矢量幅度测量

评估 $y(n)$ 的 EVM 可以度量恢复信号中该相位噪声的大小。EVM 是理解系统算法性能的极有用的测量：它度量星座相对参考位置的残余误差。按百分比 RMS 计算的 EVM 可用下式：

$$\text{EVM}_{\text{RMS}} = 100 \times \sqrt{\frac{\sum_{k=0}^{N-1} e_{\text{const}}(k)}{\sum_{k=0}^{N-1} \left(\text{Re}(\bar{y}(k))^2 + \text{Im}(\bar{y}(k))^2\right)}}, \tag{7.16}$$

其中

$$e_{\text{const}}(k) = \left(\text{Re}(y(k)) - \text{Re}(\bar{y}(k))\right)^2 + \left(\text{Im}(y(k)) - \text{Im}(\bar{y}(k))\right)^2$$

$\bar{y}(k)$ 是 $y(k)$ 的参考符号。EVM 度量接收信号的散布程度，因此越低越好。有些场合用分贝计算 EVM 更方便，可从 (7.16) 换算：

$$\text{EVM}_{\text{dB}} = 20\log_{10}\frac{\text{EVM}_{\text{RMS}}}{100}.$$

分贝 EVM 在 OFDM 标准中很常见——高阶星座的发送需要显著的 EVM 裕量才能恢复。为方便起见，Communications System Toolbox 提供了 `comm.EVM` 系统对象代劳这些计算。

> 思考题：从代码 7.4 出发，评估收敛信号相对 ζ 与 B_{Loop} 的 EVM。ζ 分别取欠阻尼、过阻尼与临界阻尼配置。

代码 7.4　badlock.m

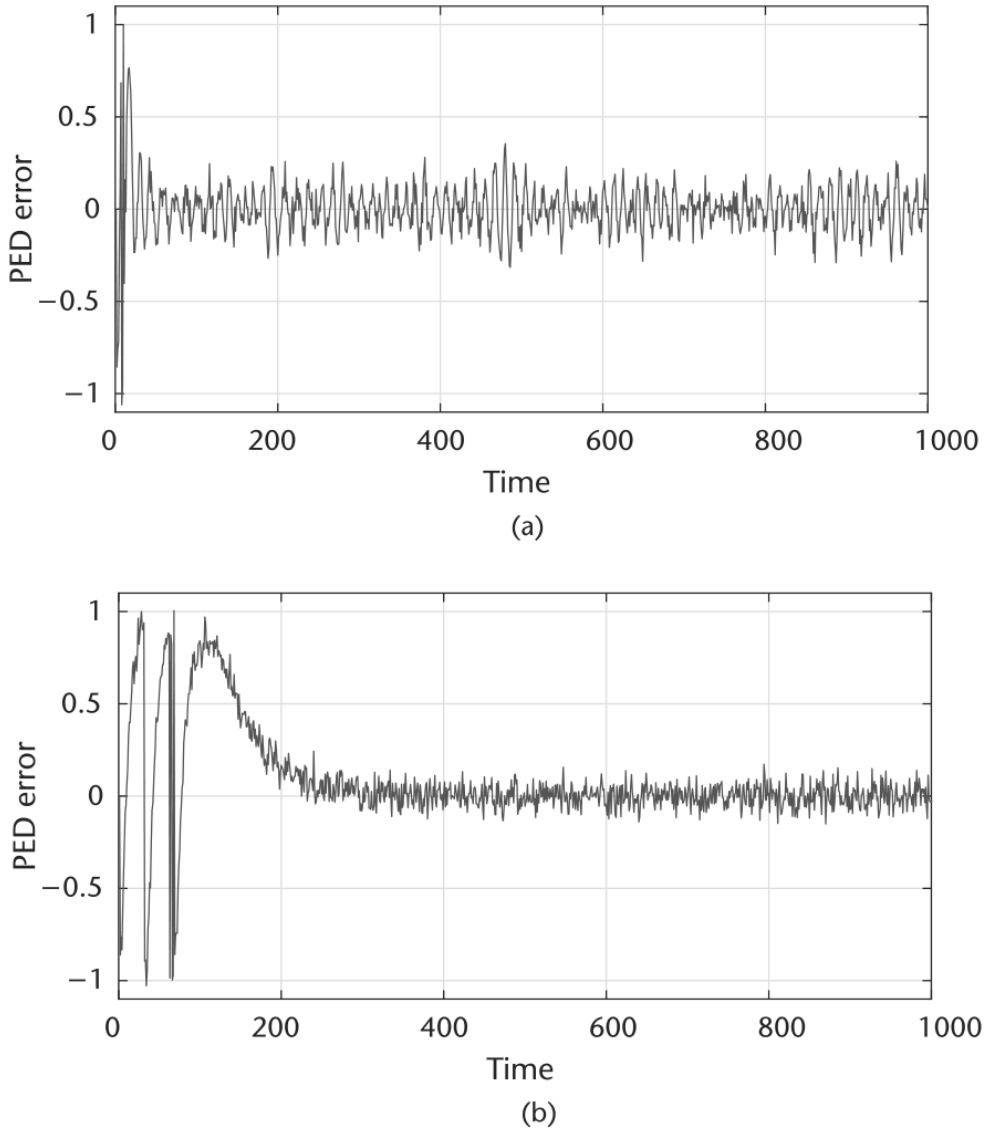


图 7.10　不同环路带宽下 QPSK PED 的误差信号随时间变化：(a) BLoop = 0.24，收敛后 σe^2 = 0.0103；(b) BLoop = 0.03，收敛后 σe^2 = 0.0031

```
%% General system details
sampleRateHz = 1e6; samplesPerSymbol = 1; frameSize = 2^10;
numFrames = 10; nSamples = numFrames*frameSize;
DampingFactors = [0.9,1.3]; NormalizedLoopBandwidth = 0.09;
%% Generate symbols
order = 4; data = pskmod(randi([0 order-1], nSamples, 1),order,0); % QPSK
%% Configure LF and PI
LoopFilter = dsp.IIRFilter('Structure', 'Direct form II transposed', ...
    'Numerator', [1 0], 'Denominator', [1 -1]);
Integrator = dsp.IIRFilter('Structure', 'Direct form II transposed', ...
    'Numerator', [0 1], 'Denominator', [1 -1]);
for DampingFactor = DampingFactors
    %% Calculate range estimates
    NormalizedPullInRange = min(1, 2*pi*sqrt(2)*DampingFactor*...
        NormalizedLoopBandwidth);
    MaxFrequencyLockDelay = (4*NormalizedPullInRange^2)/...
        (NormalizedLoopBandwidth)^3;
    MaxPhaseLockDelay = 1.3/(NormalizedLoopBandwidth);
    %% Impairments
    frequencyOffsetHz = sampleRateHz*(NormalizedPullInRange);
    snr = 25; noisyData = awgn(data,snr);% Add noise
    % Add frequency offset to baseband signal
    freqShift=exp(1i.*2*pi*frequencyOffsetHz./sampleRateHz*(1:nSamples)).';
    offsetData = noisyData.*freqShift;
    %% Calculate coefficients for FFC
    PhaseRecoveryLoopBandwidth = NormalizedLoopBandwidth*samplesPerSymbol;
    PhaseRecoveryGain = samplesPerSymbol;
    PhaseErrorDetectorGain = log2(order); DigitalSynthesizerGain = -1;
    theta = PhaseRecoveryLoopBandwidth/...
        ((DampingFactor + 0.25/DampingFactor)*samplesPerSymbol);
    delta = 1 + 2*DampingFactor*theta + theta*theta;
    % G1
    ProportionalGain = (4*DampingFactor*theta/delta)/...
```

```
        (PhaseErrorDetectorGain*PhaseRecoveryGain);
    % G3
    IntegratorGain = (4/samplesPerSymbol*theta*theta/delta)/...
        (PhaseErrorDetectorGain*PhaseRecoveryGain);
    %% Correct carrier offset
    output = zeros(size(offsetData));
    Phase = 0; previousSample = complex(0);
    LoopFilter.release();Integrator.release();
    for k = 1:length(offsetData)-1
        % Complex phase shift
        output(k) = offsetData(k+1)*exp(1i*Phase);
        % PED
        phErr = sign(real(previousSample)).*imag(previousSample)...
            - sign(imag(previousSample)).*real(previousSample);
        % Loop Filter
        loopFiltOut = step(LoopFilter,phErr*IntegratorGain);
        % Direct Digital Synthesizer
        DDSOut = step(Integrator,phErr*ProportionalGain + loopFiltOut);
        Phase =  DigitalSynthesizerGain * DDSOut;
        previousSample = output(k);
    end
    scatterplot(output(end-1024:end-10));title('');
end
```

7.3 相位模糊

载波同步要考虑的最后一个主题是**相位模糊**。相位模糊源于：这里的 FFC 同步器对发射信号的真实取向是” 盲” 的。对给定的对称调制方案，可能存在多个收敛取向——数量与调制阶数相关。例如 PAM 有两种可能取向，QPSK 与矩形 QAM 有四种，MPSK 有 M 种。不过该问题有多种对策：码字、带训练数据的均衡器、差分编码。各有各的适用场景。

7.3.1 码字

用**码字**消解相位模糊是常见做法，它依赖接收数据中的一段已知序列——通常就是前导码本身（存在于每帧、接收端已知）。该策略可在解调前或解调后使用。若在解调后进行，输出比特必须重映射回真实位置。这个过程用例子说明最好。考虑源字 **w** 与相应 QPSK 符号 **s**：

$$\mathbf{w} = [1, 0, 3] \qquad \mathbf{s} = [(-1,\ 1i)\ (1,\ 1i)\ (-1,\ -1i)].$$

可能收到的符号为

$$\mathbf{s}_1 = [(-1,\ 1i)\ (1,\ 1i)\ (-1,\ -1i)]$$

$$\mathbf{s}_2 = [(-1,\ -1i)\ (-1,\ 1i)\ (1,\ -1i)]$$

$$\mathbf{s}_3 = [(1,\ -1i)\ (-1,\ -1i)\ (1,\ 1i)]$$

$$\mathbf{s}_4 = [(1,\ 1i)\ (1,\ -1i)\ (-1,\ 1i)].$$

对每个码字符号解调并与期望结果比较，即可得到正确解调其余数据符号所需的映射。实现时，最好对全部前导码符号解调并取最常见的取向映射——依赖单个符号容易出错。

另一种做法：直接测量相对正确取向的相位偏移 θ_p——设接收前导码符号为 p、参考（真实）前导码符号为 p_r，所需校正就是

$$\theta_p = \tan^{-1} \frac{\mathrm{Im}(p(n)^* \times p_r(n))}{\mathrm{Re}(p(n)^* \times p_r(n))},$$

假设 $p(n)$ 具有合适取向。于是其余信号 y 校正为

$$y_c = y\, e^{-j\theta_p}.$$

7.3.2 差分编码

对付相位模糊的第二种选择是对源比特本身做**差分编码**。目标是让真实数据依赖于相继比特之差，而非接收比特本身。编码源数据时，发射机施加：

$$b_t(n) = b_t(n-1) \oplus b(n), \tag{7.23}$$

其中 b_t 是发射的已编码比特，b 是未编码比特，$\oplus$ 是模二加。解码基本上是 (7.23) 的逆操作：

$$b(n) = b_t(n) \oplus b_t(n-1).$$

该方案中第一个比特通常被忽略——它只基于自身而非相邻两比特之差。工程师可能觉得这浪费，但它免去了" 用符号测偏移" 的复数运算，只需要比特级操作；还能因比特差错的传播特性降低接收信号的误码率。

7.3.3 均衡器

第三种流行选择是依靠**均衡器**为系统校正该模糊。利用训练数据，均衡器可以学习并校正该相移——本质上只是一次复数乘法。均衡器将在第 9 章详细讨论；不过，如果信道校正或同步不也要由均衡器承担的话，这对均衡器实现来说只是个小任务。

7.4 本章小结

本章讨论并给出了载波偏移的模型及其与接收机操作的关系。由该模型提供了两种载波偏移补偿方案：粗校正与细校正算法。当然也存在可联合执行定时与载波恢复的其他实现 [6]。我们考察了这些算法在系统级的用法以及单项性能的评估方法，包括其参数化的刻画与恢复数据上的度量。总之，在两个拥有独立 LO 的分离节点间传输数据时，载波偏移补偿是必需的同步技术。

7.5 参考文献

[1] Analog Devices, Inc., "ADALM-PLUTO SDR Active Learning Module." http://www.analog.com/media/en/news-marketing-collateral/product-highlight/ADALM-PLUTO-Product-Highlight.pdf.

[2] Oppenheim, A. V., and R. W. Schafer, *Discrete-Time Signal Processing*, Prentice Hall, 1989.

[3] Morelli, M., and U. Mengali, "Feedforward Frequency Estimation for PSK: A Tutorial Review," *European Transactions on Telecommunications*, Vol. 9, No. 2, 1998, pp. 103–116.

[4] Wang, Y., K. Shi, and E. Serpedin, "Non-Data-Aided Feedforward Carrier Frequency Offset Estimators for QAM Constellations: A Nonlinear Least-Squares Approach," *EURASIP Journal on Advances in Signal Processing*, (2004) 2004: 856139. https://doi.org/10.1

155/S1110865704403175.

[5] Luise, M., and R. Reggiannini, “Carrier Frequency Recovery in All-Digital Modems for Burst-Mode Transmissions,” *IEEE Transactions on Communications*, Vol. 43, No. 2, 1995, pp. 1169–1178.

[6] Rice, M., *Digital Communications: A Discrete-Time Approach*, Third Edition, Pearson-/Prentice Hall, 2009.

第 8 章 帧同步与信道编码

本章覆盖帧同步与信道编码两个主题。先讨论帧同步，完成从发射机完整接收数据的最后一块拼图。与第 7 章（需要先做定时恢复才能实现）类似，帧同步要求信号已经过定时与频率校正。帧同步完成之后，我们就能在无线链路上完整译码数据。随后进入信道编码，从系统集成者的视角讨论流行的编码技术与部分实现细节。

对照图 8.1 的接收机框图，本章处理倒数第二个功能块——帧同步（已高亮）。

8.1 帧，你在哪里?

前几章讨论了频率校正、定时补偿与匹配滤波。同步的最后一方面就是帧同步。此时假设可用采样已表示单个符号、并已完成定时/频率/相位偏移校正。但现实系统中帧的起点仍然未知，需要做一次额外的校正或估计。图 8.2 直观展示了这个问题：一个已同步的示例帧带有未知偏移 p 个采样。数学上，这就是信号 y 中的未知时延：

$$u[n] = y[n - p],$$

其中 $p \in \mathbb{Z}$。有了估计 $\hat{p}$，就能从期望帧中提取数据、解调成比特，并执行最初施加于信号的任何信道译码或信源译码。估计方法多种多样，本章实现的思路围绕**互相关**展开。

依据接收机结构与波形，帧同步有可能放到解调之后进行——用一个特定比特序列标记帧首。但若下游均衡器需要符号，或前导码携带下游调制的配置参数，就不行。

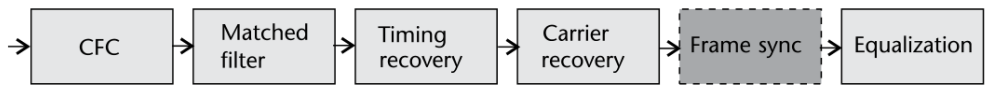


图 8.1　接收机框图

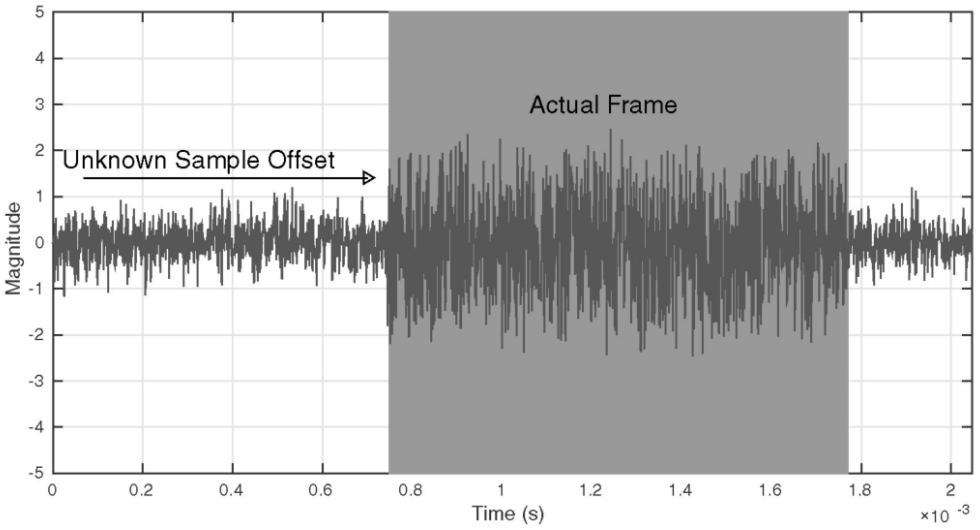


图 8.2　AWGN 中带未知采样偏移的示例接收帧



图 8.3　无线分组的常见帧结构：前导码在后是帧头与载荷数据

IEEE 802.11 [1] 就是后者的例子：前导码可以采用与载荷不同的调制。另外，若系统是分组制、不连续发射数据，仅凭比特很难把噪声与实际信号区分开——只看比特会丢掉信号的相对增益信息，而判断” 信号是否存在” 时这些信息很有用。

8.2 帧同步

确定给定帧起点的通用方法是使用**标记**（marker）——即使有线网络也是如此。但无线信号让这个问题变得更难，图 8.2 可见一斑（它其实用了标记）。由于信号噪声含量高，需要在调制前给帧附加专门设计的**前导码**序列。这类序列在接收端通常精确已知，并具有让帧估计准确的若干特性。图 8.3 概述了典型的帧排布：前导码、帧头与载荷数据。帧头与载荷在接收端未知，但会保持某种结构以便正确译码。

在讨论常用序列之前，先介绍一种” 在未知采样时刻估计已知序列起点” 的技术。设 N 条不同的二进制序列 b_n（$n \in \{1, \ldots, N\}$），每条长 L。给定另一条二进制序列 d，我们想知道 d 与这 N 条序列各自的相似程度。互相关可以给出恰当的估计：

$$C_{d,b}(k) = \sum_{m} d^{*}(m) b_n(m+k), \tag{8.2}$$

它与卷积的差别只在第二项不做时间反转。当 $d = b_n$（某个 n）时，$C_{d,b}$ 相对其余 $n-1$ 条序列达到最大，且至少在第 L 个索引处产生峰。利用这一概念即可构建帧起点估计器——如前所述，其中会包含称为前导码的已知序列。

窄带通信前导码的常用序列是 **Barker 码** [2]。Barker 码之所以被采用，是因为其自相关特性独特：非峰值相关极小乃至理想。具体地，这类码/序列 $a(i)$ 的自相关函数定义为

$$c(k) = \sum_{i=1}^{N-k} a(i)a(i+k),$$

满足

$$|c(v)| \le 1, \quad 1 \le v < N.$$

但已知的 Barker 码只有 $N \in \{1, 2, 3, 4, 5, 7, 11, 13\}$ 的九条序列，见表 8.1。图 8.5 给出了其中若干长度的自相关可视化：序列越长，中心峰越突出。通信系统中通常把多条这样的码拼接起来产生更长的序列以获得更好性能，也用于其他识别用途。

表 8.1: comm.BarkerCode 的 Barker 码

N	码
2	−1, +1
3	−1, −1, +1
4	−1, −1, +1, −1
5	−1, −1, −1, +1, −1
7	−1, −1, −1, +1, +1, −1, +1
11	−1, −1, −1, +1, +1, +1, −1, +1, +1, −1, +1
13	−1, −1, −1, −1, −1, +1, +1, −1, −1, +1, −1, +1, −1

用这些码我们实现了一个小例子（代码 8.1），展示 Barker 序列 $a(k)$ 如何在大数据集 $r(k)$ 中定位序列：把 Barker 码嵌入更大的随机序列中的未知位置 p（类似 8.1 节的误差模型），如图 8.4(a) 所示；再用 MATLAB 的 `xcorr` 函数做互相关，如图 8.4(b) 所示。互相关长度为 $2L_r - 1$（L_r 为 r 的长度）。由于 `xcorr` 会给 a 补零使其长度等于 L_r [3]，相关结果中会出现至少 $L_r - L_a$ 个零（L_a 为 a 原长）。由图 8.5 可知峰出现在序列起点

后 L_a 个采样处。据此可直接确定期望序列的偏移位置：

$$\hat{p} = \arg\max_k C_{ra}(k) - L_r,$$

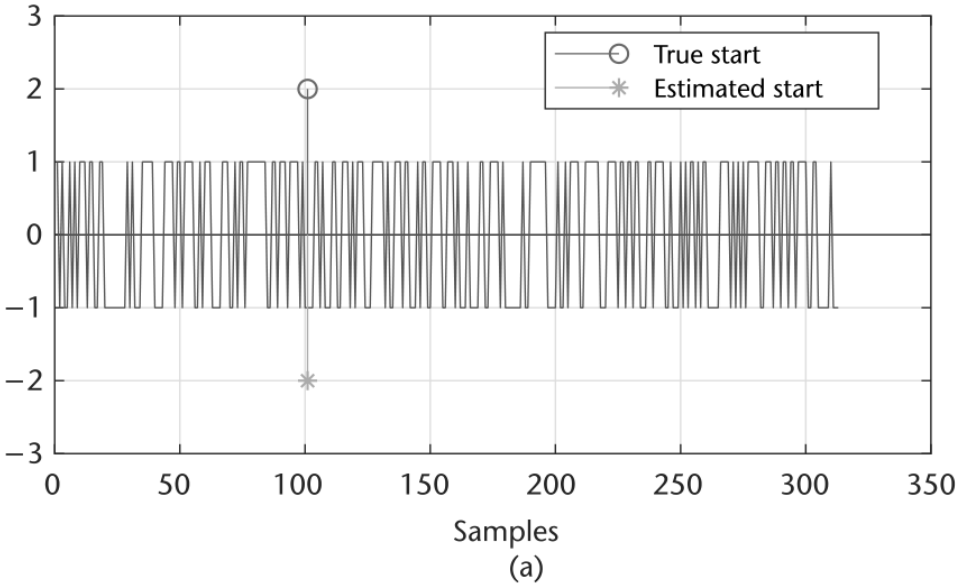
这正是图 8.4(a) 中估计所观察到的。`xcorr` 是 MATLAB 的有用工具，大序列时实际会用 `fft` 提速。既然第 2 章已知卷积是频域相乘、上面又有相关与卷积的关系，`xcorr` 采用该策略显而易见。但该过程实际会膨胀数据量——因为相关要求两序列等长。从图 8.4(b) 里的零就能看出这种膨胀。更高效的实现是用**滤波器**。抽头为 b_i 的 FIR 滤波器输出 y 可写为

$$y[n] = \sum_{i=0}^{N} b_i u[n-i], \tag{8.6}$$

其中 u 是含感兴趣序列的接收信号。(8.6) 与 (8.2) 除时间反转外几乎相同。因此，要把 FIR 滤波器当互相关器用，只需把 b_i 换成感兴趣序列的**倒序**。该实现不需要给感兴趣序列 d 补零，且可在硬件中高效实现。

代码 8.1　Barker 序列示例：barkerBits13.m

```
% Show Barker Autocorrelations search example
sequenceLength = 13;
hBCode = comm.BarkerCode('Length',7,'SamplesPerFrame', sequenceLength);
seq = hBCode(); gapLen = 100; gapLenEnd = 200;
gen = @(Len) 2*randi([0 1],Len,1)-1;
y = [gen(gapLen); seq; gen(gapLenEnd)];
corr = xcorr(y,seq);
L = length(corr);
[v,i] = max(corr);
% Estimation of peak position
% The correlation sequence should be 2*L-1, where L is the length of the
% longest of the two sequences
%
% The first N-M will be zeros, where N is the length of the long sequence
% and N is the length of the shorter sequence
%
% The peak itself will occur at zero lag, or when they are directly
```



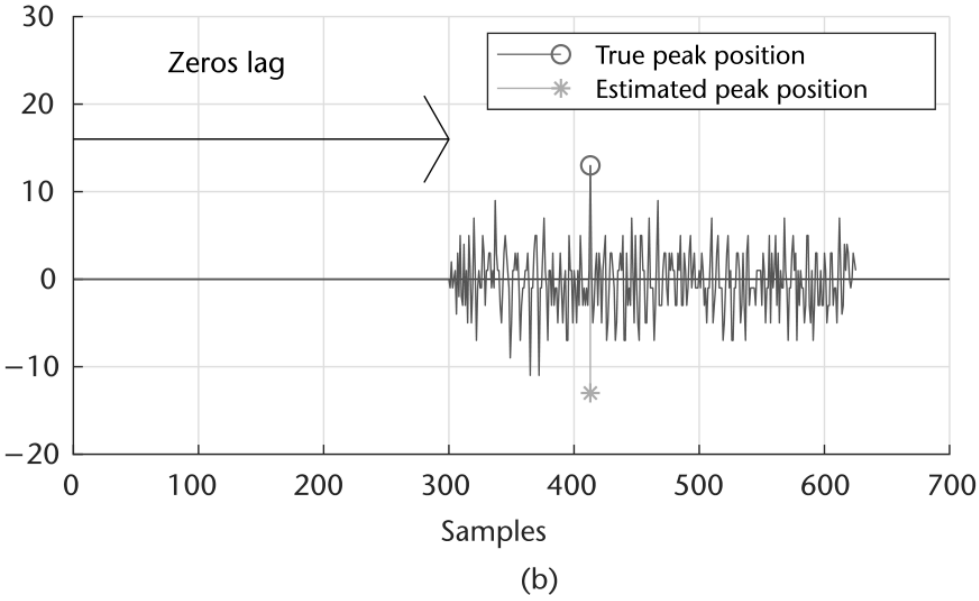


图 8.4　用互相关在更大序列中寻找序列的示例：(a) Barker 码嵌入延迟 p 处的随机比特序列；(b) Barker 码与含码随机序列的互相关

```
% aligned. (Truncated in the original book.)
```

> 思考题：基于代码 8.1，用 FIR 滤波器重新实现序列搜索。

8.2.1 信号检测

有了估计帧起点的方法，再考虑一个稍简单的问题：能否从相关结果判断”帧存在”？当希望按小块处理数据而非整帧工作时、或需要判断信道是否被占用时，这很有用。谈信号检测时，通常把它定义为**功率灵敏度**——接收机可检测的最小接收功率。但该灵敏度基于某个源波形，多数情况下无法泛化。因此这个值绝不应单独给出，除非相对某个标准传输。即使是奈曼–皮尔逊或贝叶斯这类正式检测理论方法，也必须对源信号有所了解或参照 [4]。IEEE 802.11ac 的接收机灵敏度要求具体定义为：在给定调制与编码方案下维持 10% 分组差错率的最小接收信号功率 [1]。

正式数学语境下，该过程退化为简单的二元假设检验：

$$H_0：\text{无信号}, \qquad H_1：\text{有信号},$$

其中 H_0 通常称为原假设，H_1 称为备择假设。

对原假设而言，由于没有主信号存在，接收信号只是射频环境中的噪声；对备择假设，接收信号是噪声与主信号的叠加。于是两个假设可表示为

$$H_0：r[n] = n[n], \qquad H_1：r[n] = x[n] + n[n], \tag{8.8}$$

其中 $r[n]$ 是接收信号，$n[k]$ 是射频环境噪声，$x[n]$ 是我们要检测的信号。基于观测 r，需要在描述该观测的两种统计情形之间做决断，可表达为

$$\delta(x) = \begin{cases} 1, & x \in \aleph_1 \\ 0, & x \in \aleph_1^c \end{cases}$$

当观测 x 落在区域 $\aleph_1$ 内时选 H_1，落在区域外时选 H_0。因此上式称为**判决规则**——把观测映射到相应假设的函数 [5]。在分组检测的语境下，**门限化**其实就是判决规则的实现。

无论信号模型或检测器多么精确，由于加性噪声、有限观测与观测数据固有的随机性，感知差错不可避免 [6]。在检验 H_0 对 H_1 时可能犯两类错误：H_0 被误拒或 H_1 被误拒 [5]。前者：信道中实际无信号，检测却报信道占用——称**虚警**（Type I 错误）。后

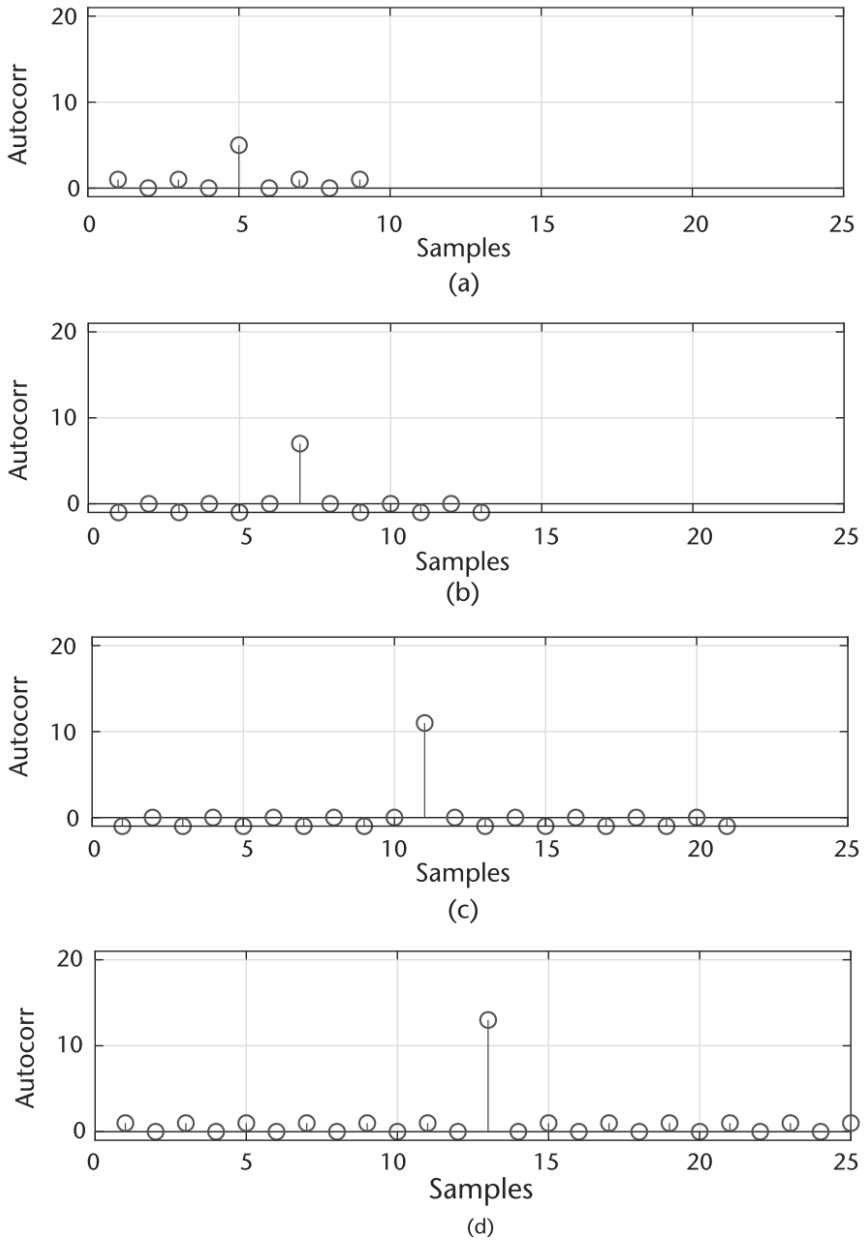


图 8.5　不同长度 Barker 序列自相关的对比：(a) N = 5；(b) N = 7；(c) N = 11；(d) N = 13

者：信道中实际有信号，检测却只报信道空闲——称**漏检**（Type II 错误）。虚警可能导致糟糕的数据恢复，而漏检会丢掉整帧数据、需要重传 [6]。

给定这两类错误，检测器的性能可由两个参数刻画：虚警概率 P_F 与漏检概率 P_M [7]，分别对应 Type I 与 Type II 错误，定义为

$$P_F = P\{\text{判}H_1 \,|\, H_0\},$$

与

$$P_M = P\{\text{判}H_0 \,|\, H_1\}.$$

注意由 P_M 可导出常用参数——检测概率：

$$P_D = 1 - P_M = P\{\text{判}H_1 \,|\, H_1\},$$

它刻画检测器识别信道中主信号的能力，故 P_D 常称为检测器的**功效**。我们希望检测器虚警概率尽量低、同时检测概率尽量高。但现实中做不到——两参数相互制约。展示它们关系常用的图是**接收机工作特性**（ROC）[8]，如图 8.6 所示：x 轴为虚警概率、y 轴为检测概率。从图中可见 P_D 增大时 P_F 也增大，“最高 P_D 且最低 P_F” 的最优点不存在。因此检测问题也是一个折中——取决于如何平衡 Type I 与 Type II 错误。

考虑检测信号在实现上的影响后，设计比代码 8.1 复杂得多。最基本的，检测成为相关器的门限化问题。目标变为确定验证峰值的参考或判据——它随信道噪声与 Pluto SDR 的自动增益控制可能剧烈变化。而且即使在仿真里，恰当的门限化也不好做，图 8.7(a) 与 (b) 可证：**无帧**时峰相对相关其余部分反而更大，**有帧**时更小。因此表现良好的实现应能处理这些情形、不依赖输入缩放。辅助门限化的常用技术是对接收信号做**自归一化**：回看图 8.7，幅度可能差异很大，使门限化难上加难；自归一化可把信号压到接近 $[0, 1]$ 的范围。实现它的简单办法是用输入信号 x 的平均能量缩放互相关度量 $C_{y,x}$。高效做法又是滤波——实现一个滑动平均滤波器。数学上该滑动平均建模为另一个求和：

$$u_{ma}[n] = \sum_{i=0}^{N} u[n-i], \tag{8.13}$$

其中 N 是前导码（感兴趣序列）的长度。(8.6) 与 (8.13) 的好处在于都是简单 FIR 滤波器实现、便于硬件化。实际上 (8.13) 与 2.6.4 节讨论的 CIC 滤波器一样不需要乘法。

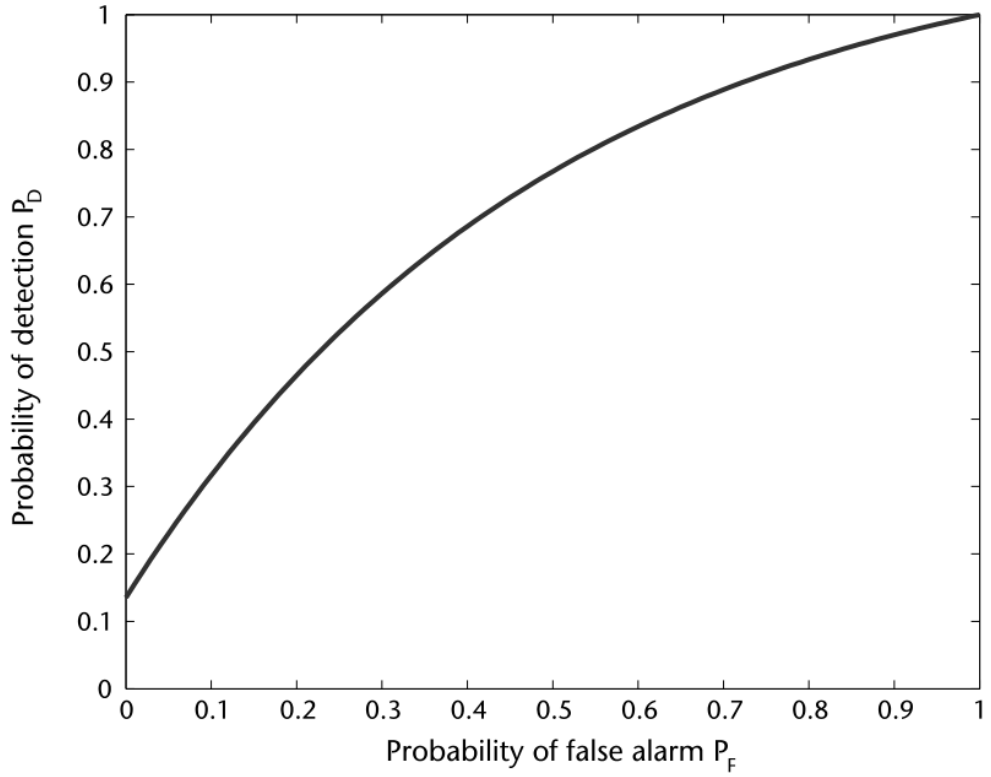


图 8.6　典型接收机工作特性曲线：x 轴为虚警概率 PF，y 轴为检测概率 PD

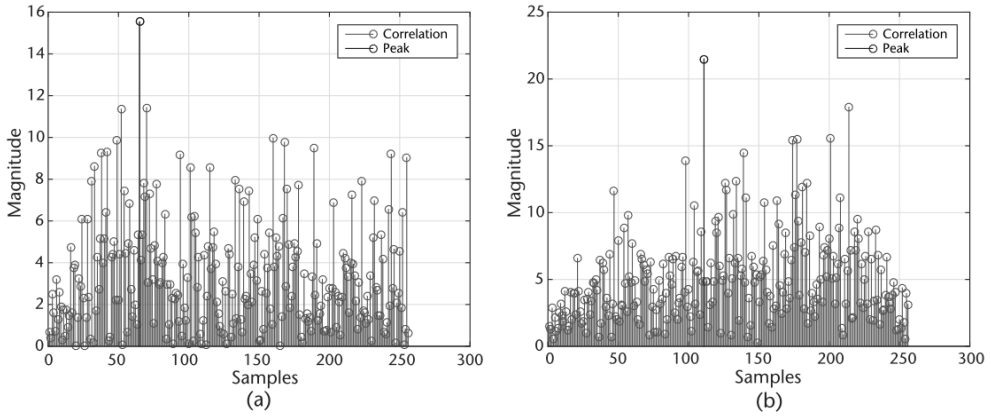


图 8.7　互相关序列搜索中假峰的示例：(a) 无信号时的相关；(b) 有信号时的相关

这很重要：许多系统中帧同步器还兼任接收机最前端的分组检测机制，要求它以输入数据最快的速率运行、不能抽取。把 (8.6) 的相关器与 (8.13) 的缩放器组合，检测器可写为

$$H_0:\ \frac{y[n]}{u_{ma}[n]} < T \text{ 无信号}, \qquad H_1:\ \frac{y[n]}{u_{ma}[n]} \geq T \text{ 有信号}, \tag{8.14}$$

其中 T 是门限值。

MATLAB 代码 `findSignalStartTemplate` 提供了一个模板，它漂亮地补偿了系统中的发射滤波器时延，给出给定分组在接收机的真实时延。

> 思考题：从代码 8.2 出发，用 `xcorr` 与 `filter` 函数实现前导码起点估计器。在 SNR $\in$ [0, 12] dB（步长 1 dB）上评估估计精度。

代码 8.2　Pluto 自环示例：findSignalStartTemplate.m

```
%% General system details
sampleRateHz = 1e6; samplesPerSymbol = 8; numFrames = 1e2;
modulationOrder = 2;  filterSymbolSpan = 4;
barkerLength = 26; % Must be even
%% Impairments
snr = 15;
%% Generate symbols and Preamble
bits = randi([0 3], modulationOrder*1e3,1);
```

```
hBCode = comm.BarkerCode('Length',7,'SamplesPerFrame', barkerLength/2);
barker = hBCode()>0; frame=[barker;barker;bits];frameSize = length(frame);
% Modulate
modD = comm.DBPSKModulator(); bMod = clone(modD);
modulatedData = modD(frame);
%% Add TX/RX Filters
TxFlt = comm.RaisedCosineTransmitFilter(...
    'OutputSamplesPerSymbol', samplesPerSymbol,...
    'FilterSpanInSymbols', filterSymbolSpan);
RxFlt = comm.RaisedCosineReceiveFilter(...
    'InputSamplesPerSymbol', samplesPerSymbol,...
    'FilterSpanInSymbols', filterSymbolSpan,...
    'DecimationFactor', samplesPerSymbol);
RxFltRef = clone(RxFlt);
%% Setup visualization object(s)
hts1 = dsp.TimeScope('SampleRate', sampleRateHz,'TimeSpan', ...
    frameSize*2/sampleRateHz);
hAP = dsp.ArrayPlot;hAP.YLimits = [-3 35];
%% Demodulator
demod = comm.DBPSKDemodulator;
%% Model of error
BER = zeros(numFrames,1);PER = zeros(numFrames,1);
for k=1:numFrames
    % Insert random delay and append zeros
    delay = randi([0 frameSize-1-TxFlt.FilterSpanInSymbols]);
    delayedSignal = [zeros(delay,1); modulatedData;...
        zeros(frameSize-delay,1)];
    % Filter signal
    filteredTXDataDelayed = TxFlt(delayedSignal);
    % Pass through channel
    noisyData = awgn(filteredTXDataDelayed,snr,'measured')
    % Filter signal
    filteredData = RxFlt(noisyData);
```

```
    % Visualize Correlation
    hts1(filteredData);pause(0.1);
    % Remove offset and filter delay
    frameStart = delay + RxFlt.FilterSpanInSymbols + 1;
    frameHatNoPreamble = filteredData(frameStart:frameStart+frameSize-1);
    % Demodulate and check
    dataHat = demod(frameHatNoPreamble);
    demod.release(); % Reset reference
    BER(k) = mean(dataHat-frame);PER(k) = BER(k)>0;
end
% Result
fprintf('PER %2.2fn',mean(PER));
```

8.2.2 替代序列

除 Barker 序列外，还有其他"除特定时刻外互相关极小"的序列。两个流行选择是**Zadoff-Chu 序列**与 **Golay 互补序列**——它们都已是现行无线标准的组成部分。

Zadoff-Chu 序列以作者 Solomon Zadoff 与 David Chu 命名 [9]，用于 LTE 的同步与信道探测。它们的用处在于：幅度恒定、循环自相关为零、不同序列间相关极低。"序列间相关有限"这一性质在多用户同时发射的多址环境特别有用。数学上，序列号生成为

$$s_n = \exp\left[-j\pi\frac{kn(n+1+2q)}{L}\right],$$

其中 L 是序列长度、n 是序列索引、q 是整数、k 与 L 互素。与纯整数 ±1 的 Barker 序列不同，Zadoff-Chu 序列是复数值的。

第二个感兴趣的序列是 **Golay 互补序列**，目前用于 IEEE 802.11ad，同样用于 802.11ad 分组前导码内的信道估计与同步。Golay 互补序列是具有极小自相关特性的双极性符号序列，概念上与 Barker 码非常相似。但顾名思义，这些序列以**互补对**的形式出现，通常记为 G_a^n 与 G_b^n（n 为序列长度）。IEEE 802.11ad 使用 $G_{a32}, G_{a64}, G_{b64}$ 等对。这些序列配 BPSK 非常出色——即使在严重相位旋转下自相关性能依然很高。Golay（确切说 Ga/Gb 对）的另一个重要方面是自相关可以在硬件中**并行**执行——这对瞄准 7 Gbit/s 传输速率的 802.11ad 这类标准非常有用 [10]。基于"极小自相关对 + 并行处理"的概念，802.11ad 的前导码可以仅凭自相关特性就向接收机传递信令信息：依据分组类型，可用一组相关器库识别特定结构，把接收机的处理引导到该类分组的特定译码路径。

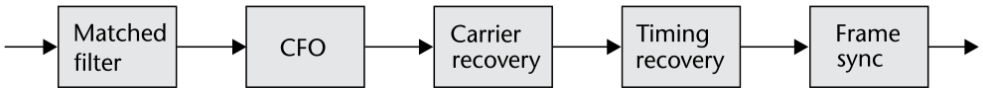


图 8.8　以频率恢复为先的示例接收机处理流程

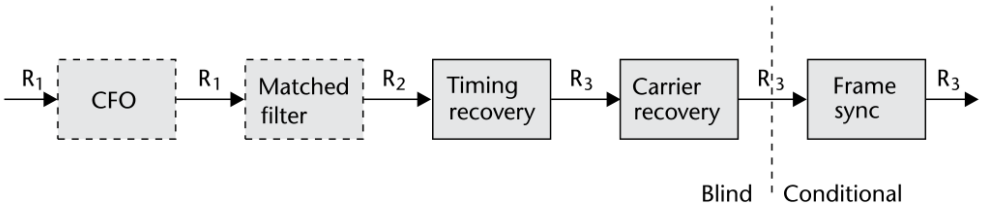


图 8.9　恢复发射帧的完整接收机处理流程，相对采样率由 Rn 定义

8.3 组装成整体

至此我们已集齐构建无线接收机的全部要件——它能应对载波偏移、定时失配与随机分组时延。趁工具齐备，正好把系统当作整体来谈，依据系统需求确定组件的排布。关于算法排布的讨论将基于前几章所学。

从接收机前端开始，要先完成两个目标：载波偏移去除与接收信号的定时同步。在目前已提出的系统中，我们先在接收链实现定时恢复，但这要求使用对相位旋转不敏感的 TED。按第 6 章的选项，这需要 Gardner 这类技术或 Harris 的多相类实现 [11]。也可以在第 7 章所述 FFC 之后再做定时恢复，但系统中可能残留相位噪声。接收机排布类似图 8.8。不过，把 CFO 放在所有处理之前（甚至匹配滤波之前）确实有用——可减轻接收链中其他恢复算法的负担。话又说回来，把 CFO 放在匹配滤波**之后**会让 CFO 估计更准——因为那时接收信号已 SNR 最大化。这些情形都得权衡。

图 8.9 概述了一种可行的接收机流程，标注了各恢复级之间的相对采样率 R_n。虚线框的块（匹配滤波与 CFO）是可选的——发射机用了 RRC 滤波或载波偏移不严重时可省。我们把 CFO 特意放在匹配滤波之前，因为匹配滤波会压缩 CFO 可利用的带宽。关于速率：$R_m \geq R_k$（$m < k$），即系统绝不会在下游上采样或以更快速率运行。总的说，下采样只可能发生在两个阶段：匹配滤波与定时恢复。两处都不是必须的，但各有好处。例如在匹配滤波级抽取能提供有用的平均，使符号更易分辨；从硬件角度，抽取降低采样率与下游处理的约束，降低下游环路中密集运算的时钟率下界。考虑定时恢复算法时，第 6 章已知所用 TED 对 R_n 有特定要求；给定时恢复环更多每符号采样通常性能也更好。

图 8.9 中，我们在载波恢复与帧同步块之间把系统划分为两个域：**盲**与**有条件**。这是为了区分哪些处理块依赖实际发送的数据、哪些块对此基本” 失明”。可以利用这一

设计特点向系统引入**训练数据**——在实际数据需要恢复之前，帮助系统中的恢复环收敛到锁定状态。这类配置中，可以在每帧前附加（或连续地在数据帧之间发送）按实际数据同样方式调制滤波的随机比特，使接收机保持锁定，消除系统中的收敛时延。硬件上可以直接把线性反馈移位寄存器（LFSR）接到调制器——这是生成随机比特的便利机制；MATLAB 中对应 `comm.PNSequence` 系统对象。收敛之后或许不必连续发训练数据，系统吞吐量随之提高。

用 Pluto SDR 实现系统（或仿真）时，向系统引入训练信息很有帮助。该配置中，图 8.9 的帧同步块充当下游处理的网关——只有检测到前导码或标记才放行数据。

8.3.1 Pluto SDR 完整恢复

过去三章我们介绍了代码模板并指导了 Pluto SDR 场景的实现。但涉及完整帧恢复与数据实际解调时，需要再强调一些实现细节。接收机是需要大量处理组件的复杂系统，实时运行不应作为最初目标；等接收算法跑通后再扩展到实时（若有需要）。因此实现时应聚焦代码模板 5.3、5.4 与 5.5。在两次调用 Pluto SDR 之间做处理（类似代码 5.6）很可能导致溢出与丢数据，完整帧恢复会很困难。

第二个重要工具是 Pluto SDR 的 `transmitRepeat` 方法（如示例代码 5.7）——它确定性地连续发送数据。接收端由于发射机来的时延是随机的，应始终至少采集 $2L$ 个采样（L 为期望帧长）。把 Pluto SDR 的 `SamplesPerFrame` 属性设为 $2L$ 即可保证发射机处于 `transmitRepeat` 模式时至少收到一个完整分组。第 4、7 章不需要这样做——溢出丢点对测试影响甚微；但检查完整帧时这是头等大事。代码 8.3 是个简单的范例：例中我们实际多次 step 接收机，清空其 IIO 缓冲中可能的陈旧数据。

代码 8.3　捕获重复帧：captureExample.m

```
% Transmit frame repeatedly
tx = sdrtx('Pluto');
tx = sdrtx('Pluto','SamplesPerFrame',length(frame)*2);
tx.transmitRepeat(frame);
for k=1:4,rx();end; % Remove stale data from buffers
rxBuffer = rx();
```

> 思考题：用代码 8.3 的模板与第 6、7 章及本章开发的同步功能块，开始估计分组的起点。先用 transmitRepeat 采集 L×N 个采样（N = 1, 2, 3, 4, 5）。你的分组检测器应能对每个 N 至少定位 L×(N−1) 的数据。评估系统随分组数增长的表现。采集 1,000 个分组并确定你的检测概率（试着每次采 10 个分组并重复该过程）。

评估完整恢复帧的一个有用工具是**循环冗余校验**（CRC）——它给出分组是否含错的二元判定。MATLAB 的 Communication Systems Toolbox 方便地提供了 `comm.CRCGenerator` 与镜像检测器 `comm.CRCDetector` 两个系统对象。代码 8.4 展示了调用方式，两个对象都用多项式 $z^3 + 1$。CRC 的工作方式是在数据末尾附加长度 L_c 的序列（校验和）。L_c 等于多项式的阶数——代码 8.4 中是 3。该多项式决定比特如何组合（异或）生成校验和。L_c 越大，8.2.1 节所述 Type I/II 错误的概率越低——但也依赖于校验和所覆盖的数据长度。实践中，一个校验和覆盖的数据量比 L_c 大若干数量级。对我们的帧同步测试，可在发射帧里用 CRC 轻松检查是否恢复了全部发射比特。

代码 8.4　Pluto 自环示例：crcExample.m

```
x = logical([1 0 1 1 0 1 0 1 1 1 0 1]');
crcGen = comm.CRCGenerator('z^3 + 1');
crcDet = comm.CRCDetector('z^3 + 1');
codeword = crcGen(x);
codewordWithError = codeword; codewordWithError(1) = ~codewordWithError(1);
[tx, err]   = crcDet(codeword);
[tx1, err1] = crcDet(codewordWithError);
```

> 思考题：再用代码 8.3 的模板与第 6、7 章及本章的同步功能块估计分组起点。这次发射前往每帧附加 CRC。接收端解调恢复的符号，对 1,000 个分组检查 CRC 值。重复该过程，但改为计算每个恢复帧的误码率，跳过丢失的帧。

8.4 信道编码

现在我们已能跨无线链路成功恢复数据，可以讨论让这一过程更稳健的技术了。信道编码是显然的选项，在任何数字通信标准中无处不在。

8.4.1 重复编码

任何通信系统的关键构件之一是**前向纠错**（FEC）：向发射流添加冗余数据，使其对信道差错更稳健。FEC 技术有很多种，例如重复编码——每个发射比特重复多次。本节我们一起实现一个简单重复编码器（重复因子 $R = 4$），探索对抗数据传输差错的一种技术。重复因子 $R = 4$ 的重复编码器是什么意思？简单定义：若要发射符号”0”，该”0”会被重复编码器重复四次，输出”0000”。

先从双击重复编码器块开始——会打开 MATLAB 函数块编辑器，可以在其中编写自定义 MATLAB 代码。如前所述，设断点是理解与调试 M 文件的好办法。断点及其调试用法详见附录 B。

> 注：MATLAB 的 `repmat` 函数可用于实现简单的重复编码方案。例如要把向量 u 重复 4 次，可用表达式 `y=repmat(u,4,1);`。

> 思考题：选择高重复因子与低重复因子时要考虑哪些折中？

8.4.2 交织

重复编码是通信系统工程师增强数据传输稳健性的若干有用工具之一，但有时并不够——它没有解决” 大块连续数据被损坏” 的问题。例如发射机发送数据流”101101”，重复因子 4 的重复编码器输出

$$111100001111111100001111,$$

每个输入比特重复四次。该编码看似稳健，但传输中仍可能发生跨多个连续比特的显著噪声突发，损坏大量二进制数字，得到如下结果：

$$1111\,00\ ------1100001111,$$

其中部分原始数据彻底无法恢复。

> 思考题：为什么即使有重复编码，数据传输仍可能受严重影响？还能做什么让它更稳健？

交织（interleaving）是把二进制数据重新排序、显著降低序列内各比特间相关性的方法。由于差错通常发生在连续的一串比特上，发射前交织比特序列、接收端解交织截获序列，可把比特差错分散到整个序列上，从而最小化其对传输消息的影响。简单交织器会把重复的比特打乱，使数据中的冗余对差错更稳健：它把重复的比特彼此穿插重排，确保即使连续丢一串比特、每个比特至少有一份冗余副本到达。例如用交织步长 4，即按索引 $[1, 5, 9, \ldots, 2, 6, 10, \ldots]$ 重排向量。于是"111100001111111100001111" 经此交织器输出：

$$101101101101101101101101.$$

交织步长可取数据长度的任何因子。不同的混合算法会改变交织器的有效性。

> 注：MATLAB 的 `reshape` 函数可用于实现交织。

实现交织器后，把重复编码器与交织器组合成单个 FEC 子系统。上文介绍的简单交织技术对我们的实现已足够，但交织还有其他多种形式，接下来两节考察。

8.4.2.1　块交织

第一种交织方法是**块交织器**，如图 8.10 所示。块交织是重排比特序列的一种方法：$N \times M$ 个比特按列填入 N 列 M 行的矩阵，再把得到的各行串行拼接从交织块输出。发射端，块交织器逐列装入 N 个码字（每个长 M 比特），然后逐行发出直至交织器排空，再重新装入、循环往复。块交织器的主要缺点是逐列填充带来的时延 [12]。

8.4.2.2　卷积交织

另一种方法是**卷积交织器** [13]，如图 8.11 所示。发射端，比特序列移入一组 N 个寄存器，每个的缓冲存储量递增；寄存器组中的比特经换向器重新合并并发送过信道。接收端执行逆过程恢复原序列。与块交织器相比，卷积交织器把存储需求降低约一半 [14]；但初始填充相关的时延问题依然存在。

8.4.3 编码

除了交织多份数据，还可以把数据**编码**成引入冗余的替代表示。许多编码方案的独特性质是：引入冗余而数据量不按整数倍增长。例如重复因子 $R = 2$ 的重复编码（FEC 讨论中通常取倒数称为码率 $\frac{1}{2}$），而卷积编码的码率可以更接近 1——效率更高、有效

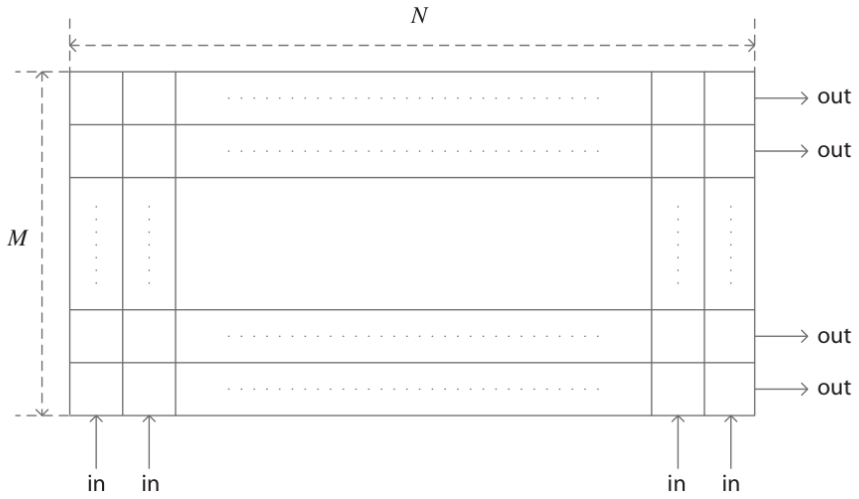


图 8.10　块交织器示意图

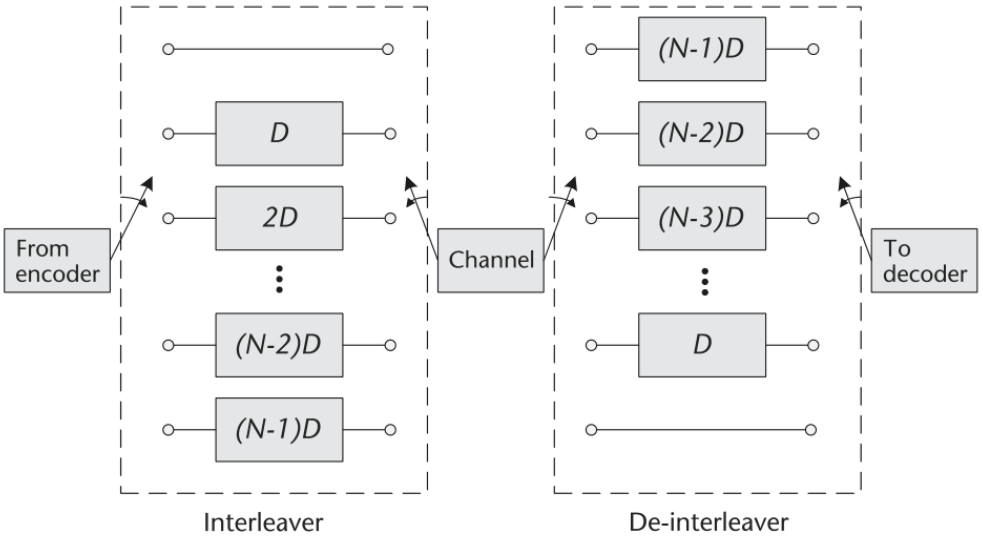


图 8.11　卷积交织器示意图（据 [13]）

吞吐量更大。本节讨论几种常见信道编码方案的基本工作原理。总体而言，信道编码是信息论中数学上复杂的领域；我们不深入这些方案的理论设计，而是比较其相对性能与部分实现细节及缺点。

与交织器类似，编码器通常分两类：**块编码器**与**卷积类编码器**。块编码器处理特定预定义的比特组/块；卷积编码器处理大小不定的数据流，必要时也可按块处理。

第一种要讨论的是 **Reed-Solomon（RS）码**——20 世纪 60 年代发展的线性块码。RS 码通过向给定帧/数据块中插入符号来纠正发生的符号差错。定义 M 为给定帧的长度（有时称消息长度）、E 为编码后的帧长，则可纠正最多 $\lfloor \frac{E-M}{2} \rfloor$ 个符号。RS 的有趣之处还在于它甚至能告知实现中发现并纠正了多少错误。不过 RS 编码器对可处理的输入比较挑剔——这就是我们到目前为止只谈符号不谈比特的原因。RS 能编码的符号是 $[0, 2^N - 1]$ 之间的整数，N 是有限伽罗瓦域 $GF(2^N)$ 的指数。伽罗瓦域是定义了某些数学运算、含有限个对象的域（集合）。由该定义，E 与 N 会有一些限制，但超出本书范围。译码时 RS 正是利用这个集合——依据接收数据与 M, E, N 的选择收缩解空间。

实现角度可以论证：RS 码适合**突发**差错场景——相邻一片符号被损坏的情形。考虑发射比特时，每个符号代表 B 个比特；RS 按符号操作，可一次纠正一组内 B 个比特。因此一组内损坏 $B + 1$ 个比特的突发差错最多损坏 2 个符号。

与 RS 类似的是 **Bose-Chaudhuri-Hocquenghem（BCH）码**，同样依赖伽罗瓦域概念。与 RS 不同，BCH 码更擅长纠正**不成群**的差错。为了降低数据成群的概率，可以在收发前后对比特做打乱/扰码以降低差错局部性——这对差错稀疏时的 BCH 更有利，但对 RS 则在一定程度上是坏事。相同校验比特数下 BCH 能纠正更多错误，但一般而言 BCH 译码比 RS 更耗算力。

最后一种要考虑的主流块码是**低密度奇偶校验（LDPC）码**——它甚至已经开始取代长期占主导的 Turbo 码（马上讨论）。LDPC 码早在 1960 年代就存在，但因其复杂度近十年才被硬件实现考虑。由 Robert Gallager 发展的 LDPC 码在特定冗余率下可逼近理论上的香农极限 [15]——RS 与 BCH 做不到。但使用 LDPC 所需的计算量高得多。它仍存在于 802.11n 与 DVB-S2 标准的某些模式中。使用 LDPC 时，实现者必须选定编码器与译码器使用的**奇偶校验矩阵**，矩阵特性决定码的性能。性能强的码通常块长很大，如 IEEE 802.11n/ac 的 648、1296、1944——这意味着许多场合下需要比其他码编码多得多的比特才能高效使用 LDPC。

除块码外，替代的**流式**编码实现是**卷积码**。这类码对数据做卷积式编码——冗余经由” 信息接连通过编码器/译码器” 引入，本质上是让相继的符号或比特相互依赖。卷积编码器用例子最好理解。考虑 $R = 2$、带三个寄存器的递归编码方案。图 8.12 给出了这种编码器的可能结构：每推入一个比特输出两个。注意当前输出至少依赖最近三个

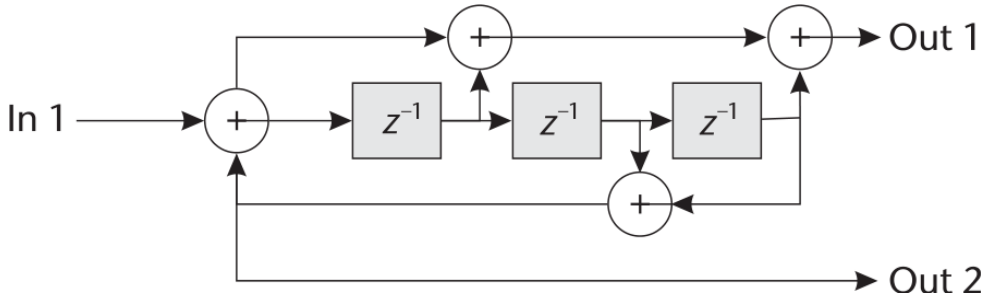


图 8.12　3GPP LTE 使用的示例 R = 2 卷积编码器

输入——类似 FIR/IIR 滤波器的记忆。图 8.12 可以解读为两条方程，见 (8.16)：

$$y_{n,1} = (x_n + x_{n-2} + x_{n-3}) + x_{n-1} + x_{n-3}$$
$$y_{n,2} = x_{n-2} + x_{n-3} \tag{8.16}$$

译码器正是利用数据中的这种依赖来去除可能发生的差错。完成该任务最流行的算法是 **Viterbi 算法** [15]，有时称网格（trellis）算法或译码器。Viterbi/网格译码器的思想是回溯先前做过的判决、利用它们最好地确定当前最可能的比特或采样。这自然引出图 8.13，也正是" 网格" 之名的由来。图 8.13 中最左侧的位置代表最近收到的符号或比特；连接圆点的线代表可能的先前符号，其中粗灰线代表依据先前判决更可能的符号。该网格的深度称为**回溯长度**——回溯越深、比特恢复越好。但该过程在回溯约 34 个符号附近趋于平台，且回溯越深、译码单个符号耗时越多。回溯限制可见图 8.14：对 16-QAM 信号考察了多种回溯长度随 E_b/N_0 的表现。

1990 年代早期引入的 **Turbo 码**属于卷积码家族 [16]，曾被第三、四代蜂窝标准大量用作主 FEC 方案。与 LDPC 一样，Turbo 码可工作在香农极限附近，但计算量比 LDPC 低、纠错性能也差一些。Turbo 内部本质上使用 Viterbi 算法译码，外加交织、以及一组译码器并在其间做似然估计。这是对 Turbo 译码器实际工作方式的极度简化——分析其操作是非常困难的领域。但只要硬件资源能在必要速度上实现译码器，它就是非常强大的编码技术。

使用 FEC 时应始终权衡：计算复杂度、所需性能、链路允许的编码开销。这很重要——清晰的传输信道可能不需要重编码，更接近 1 的码率能换来额外吞吐量。因此 LTE 与 IEEE 802.11 等现代标准采用**自适应调制与编码方案**（MCS）：降低编码冗余、提高调制阶数，使链路吞吐量大增。IEEE 802.11 本身有 32 个 MCS 状态/索引，表 8.2 给出前四项，供体会码率与调制如何折中冗余与数据率。

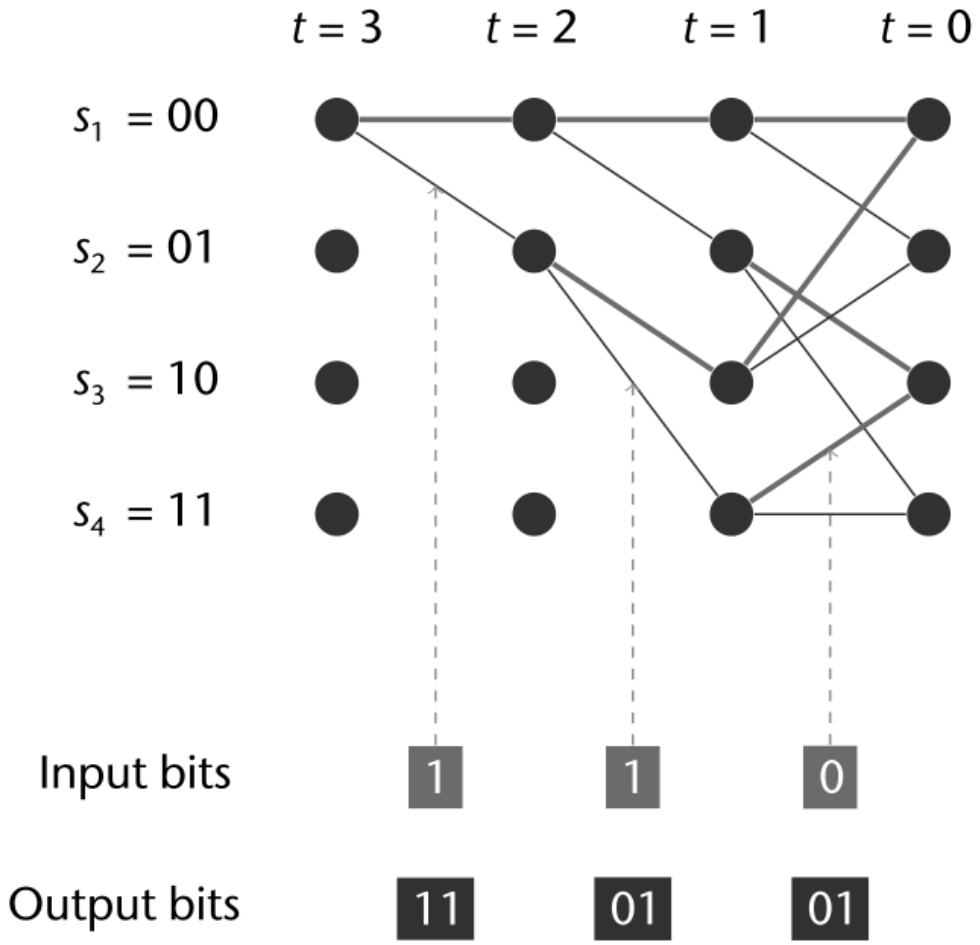


图 8.13　Viterbi/网格译码器格状图

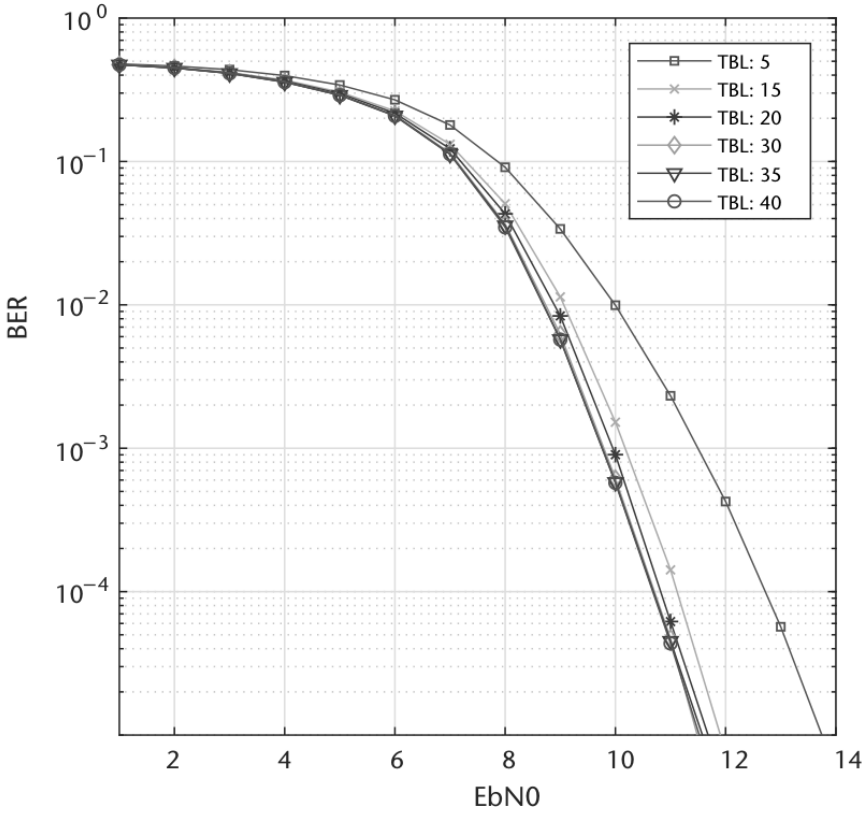


图 8.14　16-QAM 下 Viterbi 译码器随回溯长度增加的 BER 结果

表 8.2: IEEE 802.11 的调制编码方案简表（据 [1]）

MCS 索引	流数	调制	R	数据率 (Mbits/s)
0	1	BPSK	1/2	6.5
1	1	QPSK	1/2	13
2	1	16-QAM	3/4	19.5
3	1	16-QAM	1/2	26

MATLAB 提供了到目前为止讲过的全部编码技术。但某些高级码用起来更复杂，尤其在不同模式下。例如 Turbo 译码器因其工作方式需要信道估计来正确确定信道噪声方差——接收机设计中必须提供该信息才能有效译码。LDPC 如前所述需要奇偶校验矩阵：MATLAB 默认提供 DVB 的矩阵，每块要求 32,000 比特——对许多应用相当不合理；设计更小的矩阵可能很复杂，超出 MATLAB 所提供工具的范围。尽管如此，RS、BCH 与通用 Viterbi 译码开箱即用极其方便、参数化简单。

8.4.4 BER 计算器

考察过若干保护数据传输免受干扰与噪声的技术后，现在需要定量测量这些技术表现的方法。回顾第 4 章：BER 是评估与比较数字通信系统的常用指标。计算 BER 最直接的办法是数一数收到的信息比特、再确定其中哪些错了——即误比特数与总接收比特数之比给出近似的 BER。注意接收比特越多，该链路级指标越准。

8.5 本章小结

本章通过相关技术考察了帧同步的概念，并通过冗余插入覆盖了一些常见信道编码技术。随着同步的最后一块拼图就位，可以实现完整帧恢复的完整接收机。本书至此讨论过的接收机算法的各种可行排布也已考察，聚焦设计层面的要求。完整同步讲完之后，我们转向通过编码实现让链路更稳健的技术，包括它们在实现与使用上的优缺点讨论。

8.6 参考文献

[1] IEEE, *Standard for Information Technology–Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications–Amendment 4: Enhancements for Very High Throughput for Operation in Bands below 6 GHz*, IEEE Std 802.11ac-2013, December 2013, pp. 1–425.

[2] Barker, R. H., "Group Synchronizing of Binary Digital Sequences," in *Communication Theory*, London: Butterworth, 1953, pp. 273–287.

[3] The MathWorks Inc., "xcorr" [online], 2017. https://www.mathworks.com/help/signal/ref/xcorr.html.

[4] Kay, S. M., *Fundamentals of Statistical Signal Processing, Volume II: Detection Theory*, Upper Saddle River, NJ: Prentice Hall, 1998.

[5] Poor, H. V., *An Introduction to Signal Detection and Estimation*, New York: Springer, 2010.

[6] Zhao, Q., and A. Swami, "Spectrum Sensing and Identification," in *Cognitive Radio Communications and Networks: Principles and Practice*, Burlington, MA: Academic Press, 2009.

[7] Kay, S. M., "Statistical Decision Theory I," in *Fundamentals of Statistical Signal Processing, Volume II: Detection Theory*, Upper Saddle River, NJ: Prentice Hall, 1998.

[8] Shanmugan, K. S., and A. M. Breipohl, “Signal Detection,” in *Random Signals: Detection, Estimation and Data Analysis*, Wiley, 1988.

[9] Finger, A., *Pseudo Random Signal Processing: Theory and Application*, Hoboken, NJ: Wiley, 2013.

[10] IEEE, *Standard for Information Technology–Telecommunications and Information Exchange between Systems–Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 3: Enhancements for Very High Throughput in the 60 GHz Band*, IEEE Std 802.11ad-2012, December 2012, pp. 1–628.

[11] Harris, F. J., and M. Rice, “Multirate Digital Filters for Symbol Timing Synchronization in Software Defined Radios,” *IEEE Journal on Select Areas in Communications*, Vol. 19, October 2001, pp. 2346–2357.

[12] Jacobsmeyer, J. M., *Introduction to Error-Control Coding*. http://www.pericle.com/papers/Error_Control_Tutorial.pdf.

[13] Forney, G. D., “Burst-Correcting Codes for the Classic Bursty Channel,” *IEEE Transactions on Communications*, COM-19, 1971, pp. 772–781.

[14] Sklar, B., *Digital Communications Fundamentals and Applications*, Upper Saddle River, NJ: Prentice Hall, 1988.

[15] Anderson, J., and S. Mohan, *Source and Channel Coding: An Algorithmic Approach*, New York: Springer, 1991.

[16] Berrou, C., *Error-Correction Coding Method with at Least Two Systematic Convolutional Codings in Parallel, Corresponding Iterative Decoding Method, Decoding Module and Decoder*, US Patent No. 5,446,747, 1995.

第 9 章 信道估计与均衡

本章介绍信道估计与信道均衡的概念。先讨论一个简化的误差模型，再给出向通信系统添加均衡器的若干实用策略，具体涵盖判决引导与基于训练两种均衡设计。均衡器是件趁手的工具——环境中的众多失真源以及节点自身之间的失配，都可以交给它处理。

对照图 9.1 的接收机框图，本章处理最后一个功能块——均衡（已高亮）。

9.1 多径，休想！

前几章聚焦发射与接收节点间的同步。把前几章组合起来，帧恢复已经可能，我们已站在成功译码帧的门槛上。但某些场景下这些实现还不够。假设接收信号 SNR 充足，环境中剩下的挑战就是**多径**与其他加性干扰。多径由信道的色散特性引入：发射信号沿不同路径到达接收机的、经过缩放的反射。由于这些反射走过不同路径，从接收机的视角看它们会经历不同的衰减与时延。这些损伤自然可看作发射信号的" 回波"，用 FIR 滤波器即可轻松建模。

首个接收信号与最后一个回波之间的时间差定义为信道的**时延扩展** [1]。图 9.2 与图 9.3 给出了多径的物理表示，以及视距（LOS）信号加两个散射体产生的时域表示。这是多径的射线追踪表示；现实中多径是信号辐射特性造成的连续反射，射线追踪是该连续统的离散化——因为远比连续模型容易理解与数学建模，所以常用。

当时延扩展相对信号带宽很长时，多径效应会对接收信号造成显著失真，产生第 6 章讨论过的码间串扰（ISI）。时延扩展是环境的函数，设计系统时必须考虑。例如室外环境多径距离大，会产生大的时延扩展。数学上，散射体多走的距离 D 与对应的采样

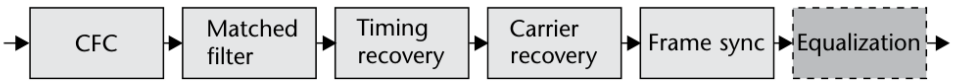


图 9.1　接收机框图

时延 t_s 的关系为

$$t_s = \frac{B \times D}{c},$$

其中 c 是光速。20 MHz 的 Wi-Fi 信号要多走约 15 米才产生一个采样的时延。因此常用于室内的 Wi-Fi 时延扩展小；不过信道距离短，反而可能存在大量高功率散射体。由于路径损耗，干扰信号的功率与时延扩展通常成反比。

数学上，接收多径信号 r 可建模为：发射信号 x 在随机时延偏移 Δ_n 上、带相关增益 α_n 的冲激串：

$$r(t) = \mu(t) + \sum_{n=1}^{N} \alpha_n x(t - \Delta_n), \tag{9.2}$$

其中有 $N-1$ 个散射体、μ 是附加干扰或噪声。只要 μ 与 x 不相关且是周期或自回归的，就能把它从信号空间有效滤除 [2]。存在多径时，接收端会经历 ISI。图 9.4 演示了这一情形——观察 QPSK 信号 r：符号随时间拖尾（smearing），在星座图上表现为符号漂移。基于 (9.2)，用 FIR 滤波器即可简单建模多径效应。

9.2 信道估计

在考虑校正给定信道的效应之前，可以先换个角度：估计这个未知信道。信道估计（相对信道均衡）是个理想的起点——仿真中有已知解可供比对；而均衡依据信道与噪声条件未必给出唯一解。做信道估计，我们将使用 Widrow 与 Hoff 提出的**最小均方**（LMS）算法 [3]。LMS 是梯度下降/牛顿法类算法，可视为信号处理的标准自适应滤波算法。LMS 利用发射序列中已知的信息或符号来估计接收数据的损坏——这里把损坏建模为 FIR 滤波器。图 9.5 给出了信道估计的常用框图，用于辅助推导系统实现。替代的自适应滤波算法确实存在，如递归最小二乘（RLS）——许多情形下优于 LMS。但 RLS 设计不当时有稳定性隐患，且因需要矩阵求逆而计算更复杂。RLS 超出本章范围，Haykin [4] 是不错的参考。

图 9.5 展示了信道估计系统的各部件：未知静态信道 h 作用于训练数据 $x(t)$。系统的目标是让 h 与 $\hat{h}$ 匹配、把误差压到零。为此，自适应算法需要误差信号 $e(n)$ 与原始信号 $x(t)$。我们用 LMS 估计未知信道滤波器 $h \in \{L \times 1\}$，其估计定义为 $\hat{h} \in \{M \times 1\}$（$M \geq L$）。对经过信道滤波器 h 的发射信号 $x(t)$，h 可用如下递归算法估计：

$$y(n) = \hat{h}^H(n)x(n) \tag{9.3}$$

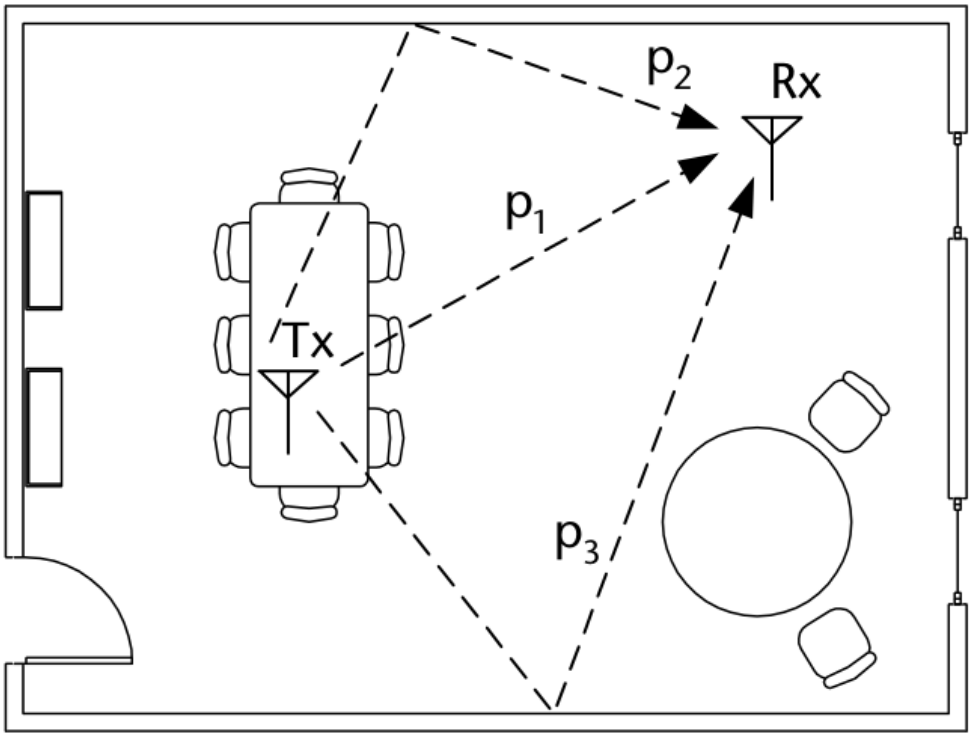


图 9.2　室内环境中多径的示例

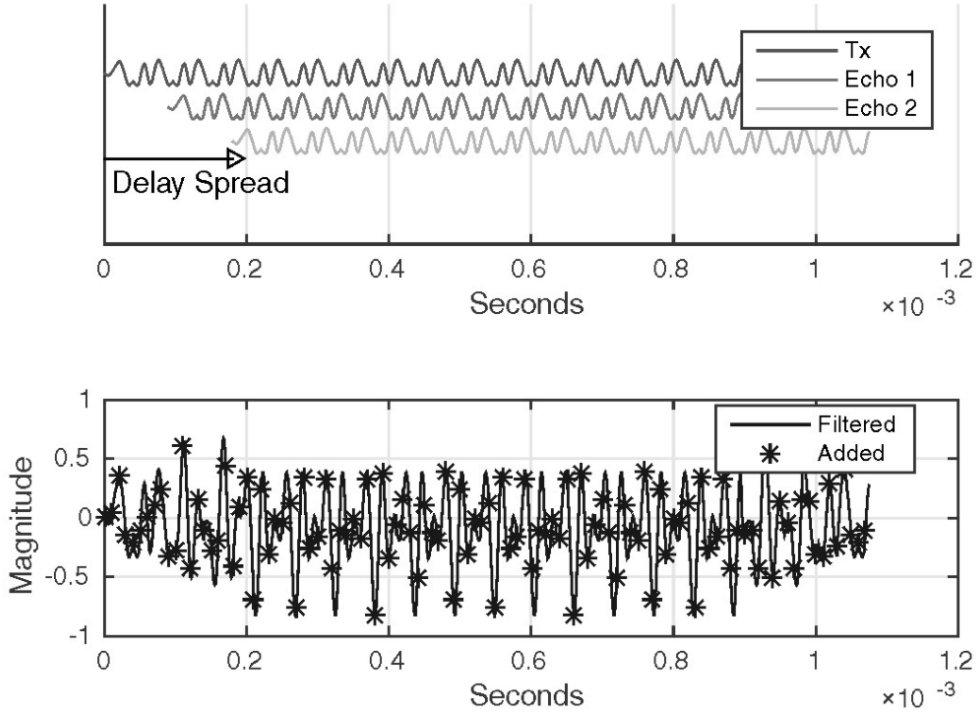


图 9.3　多径传播环境的物理特性

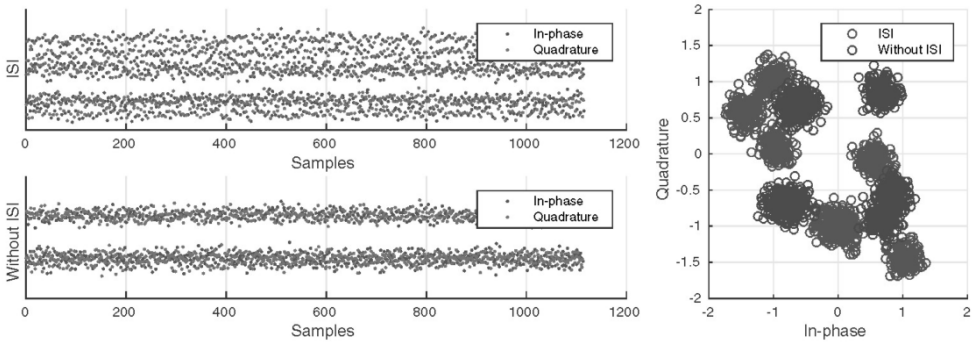


图 9.4　符号随时间拖尾的效果

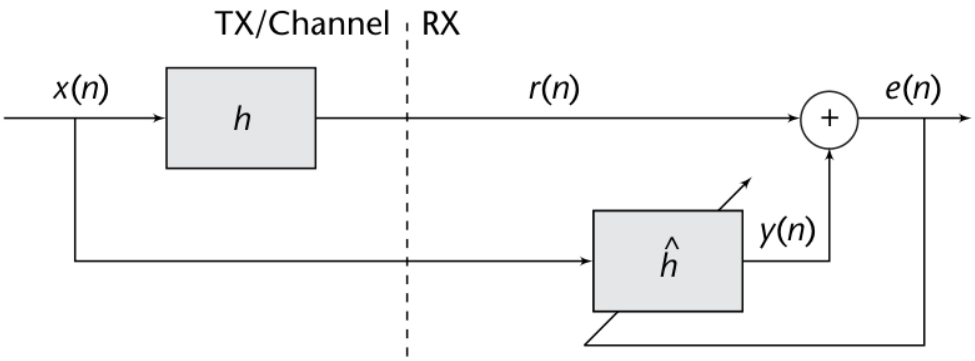


图 9.5　用训练数据对 FIR 信道 h 的自适应 FIR 估计

$$e(n) = r(n) - y(n) \tag{9.4}$$

$$\hat{h}(n+1) = \hat{h}(n) + \mu x(n) e^*(n) \tag{9.5}$$

其中

$$x(n) = [x(n), x(n-1), ..., x(n-M-1)]^T. \tag{9.6}$$

μ 是控制收敛速率与稳定性的步长。步长的选择与实现相关，但应落在 $0 < \mu < \frac{2}{\lambda_{\max}}$ 范围内，其中 $\lambda_{\max}$ 是真实信道自相关 $\mathbf{R} = E[\mathbf{h}\mathbf{h}^H]$ 的最大特征值。由于 h 未知，一个更宽松也更安全的界是 $M\sigma^2$（σ^2 为 $r(n)$ 的方差）。LMS 稳定性分析的更多信息可查 Haykin [4] 第 9 章。

信道长度 L 对具体环境是未知量，但通过测量与信道环境的物理特性，可以为合理长度做出估计。这关联到前面关于时延扩展的讨论——时延扩展就是信道长度的另一种描述。标准委员会常做研究为接收机设计者提供指引。

(9.3)–(9.6) 可以用一个简单递归实现（代码 9.1 第 8–17 行），需要 $2L+1$ 次乘法与 $L+1$ 次加法。对 $L = M = 2$，考察 $\hat{h}$ 的均方误差（MSE）随时间在解空间等高线图（图 9.6）上的轨迹，可观察 LMS 下降到真解的路径。图 9.6 演示了估计 $\hat{h}$ 从四个随机起始位置的下降。等高线图是自适应滤波理论中刻画估计演化的常用分析工具。但当 $M > 2$ 时无法在等高线上轻松可视化所有误差维度，就改用类似图 9.7 的 MSE 图。

代码 9.1　LMS 信道估计：chanEst.m

```
h = [0.5; 1; -0.6]; % Channel to estimate
```

```
mu = 0.01; % Stepsize
trainingSamples = 1000;
x = sign(randn(trainingSamples,1)); % Generate BPSK data
r = filter(h,1,x); % Apply channel
L = length(h); h_hat = zeros(L,1);
%% Estimate channel
for n = L:trainingSamples
    % Select part of training input
    in = x(n:-1:n-L+1);
    % Apply channel estimate to training data
    y = h_hat'*in;
    % Compute error
    e = r(n)-y;
    % Update taps
    h_hat = h_hat + mu*conj(e)*in;
end
```

还可以进一步扩展代码 9.1，把信道估计的形状也可视化，更好地理解设计的精度。考察响应（尤其对相当恶劣的信道）对确定均衡器设计的参数化非常有用。代码 9.2 给出了可视化信道响应与估计的示例，用了 `freqz` 函数——它用 `fft` 计算数字滤波器响应。

代码 9.2　绘制响应：chanEst.m

```
%% Plot responses
Fs = 1e6; N = 64;
htrue=freqz(h,1,N,Fs,'whole');
[hb,we]=freqz(h_hat,1,N,Fs,'whole');
semilogy(we,abs(hb),'b')
hold on;semilogy(we,abs(htrue),'bo');hold off
grid on;xlabel('Frequency (Hz)');ylabel('Magnitude');
legend('Channel Est','Channel Actual','Location','Best');
```

图 9.8 给出的响应与估计非常准确。但随着系统引入噪声、信道变得更复杂，估计会变差。不过，只要 LMS 算法已收敛，误差应只受所选步长 μ 与信道噪声的限制。

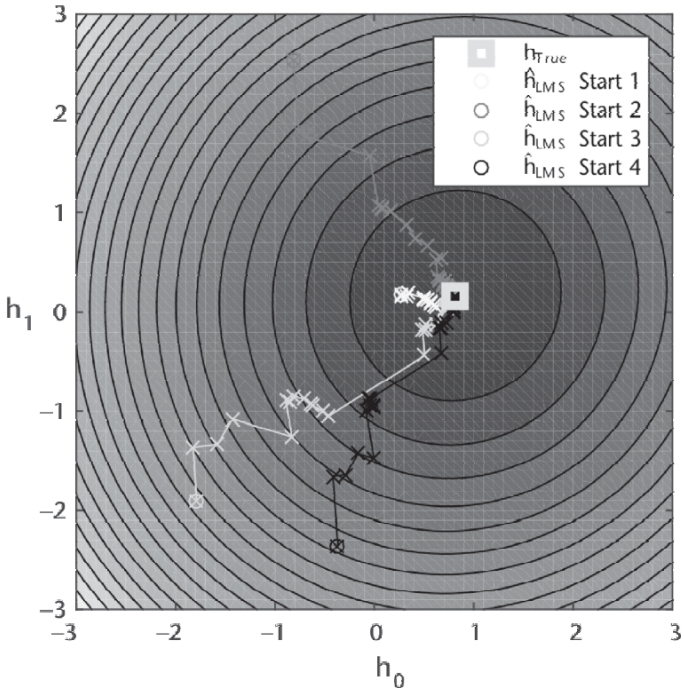


图 9.6　M = 2 时信道估计解空间的等高线

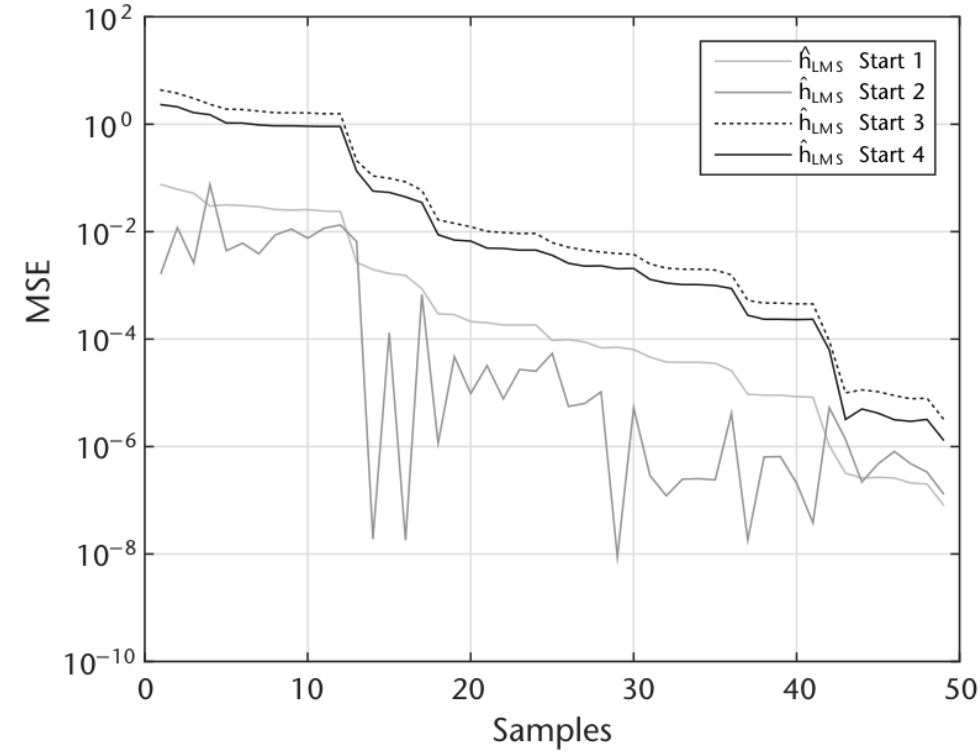


图 9.7　信道估计 MSE 随接收采样数的变化（不同起始位置）

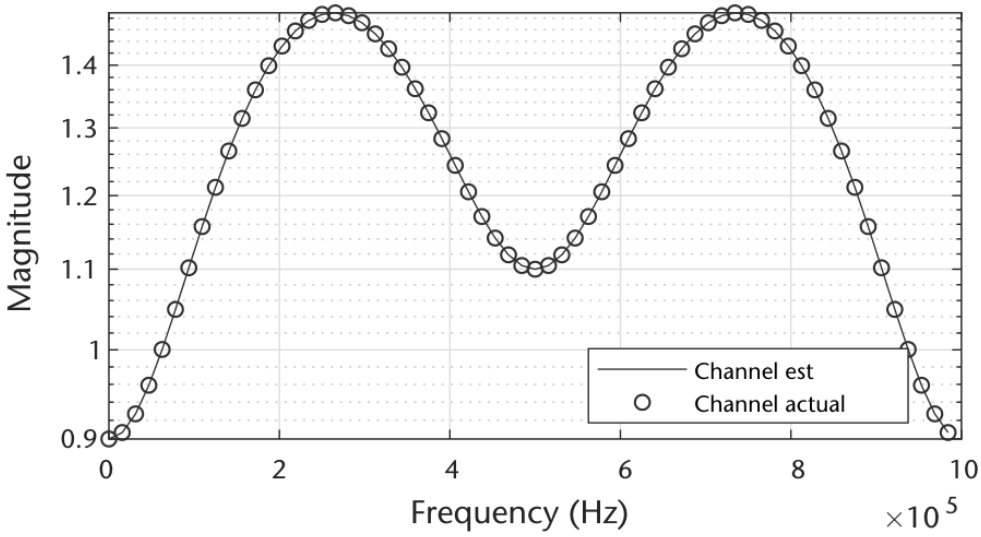


图 9.8　用 LMS 算法的示例信道及其估计

9.3 均衡器

与信道估计不同，**均衡器**试图撤销信道的效应、并在可能时去除干扰。与信道估计类似，接收端训练均衡器需要一些关于源数据的知识以削弱信道效应。这些信息通常在帧的前导码或帧头里——逻辑上我们总要发些未知数据（称为载荷）。本节讨论几种自适应均衡器实现，文献中还有大量变体与替代 [5]。

均衡器从不同于 9.2 节的视角处理滤波器演化问题：不是把滤波器演化成匹配信道 h，而是演化出一个滤波器 f 来**补偿**信道、在滤波器输出端准确重现训练数据。该布置见图 9.9——均衡器滤波器（前向滤波器）放在接收数据路径上，再用已知发射数据自适应该滤波器。为用 LMS 求解均衡给定信道 h 的滤波器 $f \in \{K \times 1\}$，需要把 (9.3)–(9.6) 的递归修改为

$$\hat{x}(n) = f^H(n) r(n)$$

$$e(n) = x(n) - \hat{x}(n)$$

$$f(n+1) = f(n) + \mu e^*(n) r(n)$$

其中：

$$r(n) = [r(n), r(n-1), ..., r(n-K-1)]^T.$$

此情形中，我们驱动接收采样 $r(n)$ 去匹配发射数据 $x(n)$。若算法收敛，组合滤波器的输出应只是把信号延迟：

$$x(n) * h * f = \hat{x}(n - \delta),$$

其中 $*$ 表示卷积、δ 是级联信道滤波器与均衡器的群时延。因此均衡器与信道的组合响应应当平坦。更新后的代码见代码 9.3。

时延 δ 把训练序列进一步推入滤波器内部——当最早的滤波器抽头并不产生最大增益时这很有用；可以论证这更高效地利用了均衡器相对信道长度的额外抽头优势。由于均衡器必须因果，$\delta > 0$，否则就需要未来采样（非因果）。该时延还会影响输出时延，也必须补偿。要正确评估信号中的误比特，用代码 9.4。

代码 9.3　信道均衡：chanEQ.m

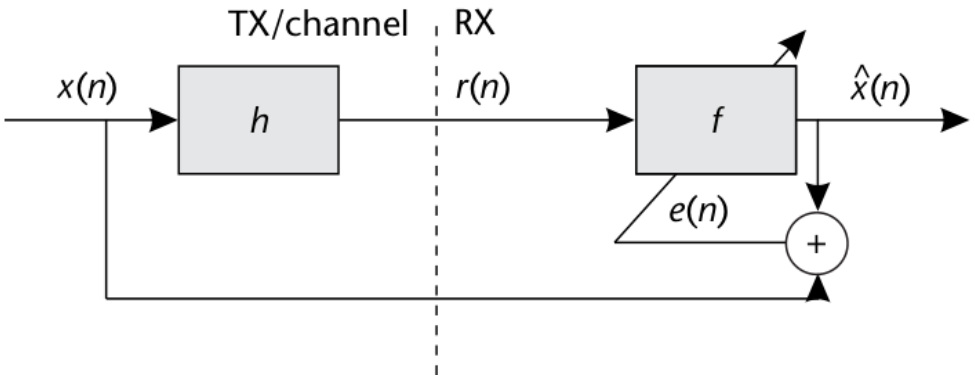


图 9.9　用已知训练数据对 FIR 信道 h 的自适应 FIR 均衡

```
h = [0.2; 1; 0.1; 0.02]; % Channel taps
mu = 0.001; % Stepsize
trainingSamples = 1e4;
x = sign(randn(trainingSamples,1)); % Generate BPSK data
r = filter(h,1,x); % Apply channel
K = length(h)+1; f = zeros(K,1);
delta = 3; % Reference tap
%% Equalize channel
index = 1;
[e,x_hat]=deal(zeros(trainingSamples-K+1,1));
for n = K:trainingSamples
    % Select part of training input
    in = r(n:-1:n-K+1);
    % Apply channel estimate to training data
    x_hat(index) = f'*in;
    % Compute error
    e = x(n-delta)-x_hat(index);
    % Update taps
    f = f + mu*conj(e)*in;
    index = index + 1;
end
```

代码 9.4　确定误比特：chanEQ.m

```
% Slice
x_bar = sign(x_hat);
% Calculate bit errors
eqDelay = K-delta;
fprintf('BER %2.4f n',mean(x(eqDelay:end-eqDelay-1) ~= x_bar));
```

当我们确切知道源符号时，LMS 是到达给定信道维纳解的有效算法 [4]。但许多场合用不了训练信息，需要**盲均衡**技术。当接收信号不含已知前导码、或想在帧的载荷部分也更新均衡器时，盲均衡是必需的。信道滤波器长度长、或所需数据多于前导码所能提供的场合就需要这种实现。另外，对高度动态的信道——信道相干时间短于一帧——可能需要持续更新均衡器以维持期望的 BER。这里我们考虑**判决引导**（DD）LMS 均衡器，其他实现还有恒模（CM）均衡器（有时称色散最小化 DM 算法）[6] 等。

DD 均衡器对收到的数据做**硬判决**以估计最可能的符号，再基于该估计更新均衡器。更新后的均衡器结构见图 9.10——在均衡器输出后插入判决块。于是除调制方案外，该设计的接收机对 $x(n)$ 一无所知。这使 DD 情形的误差项更新为

$$e(n) = \hat{x}(n) - \bar{x}(n) \tag{9.12}$$

其中 $\bar{x}(n)$ 是均衡后接收信号 $x(n)$ 的最大似然估计。BPSK 与 QPSK 的估计器为：

$$\bar{x}(n) = \begin{cases} \text{sign}(y(n)), & \text{BPSK} \\ \text{sign}(\text{real}(x(n))) + i \times \text{sign}(\text{imag}(x(n))), & \text{QPSK} \end{cases}$$

当 $x(n)$ 与 $\hat{x}(n)$ 相对接近时 DD 均衡很有效；差异大时 DD 表现会很差。换个视角（借第 6 章的说法）：眼图的眼必须初始就有一定程度的张开，DD 均衡才能有效、或在合理符号数内反转信道。图 9.11(a) 给出含 ISI 的接收信号，图 9.11(b) 是 LMS 均衡后的结果，图 9.11(c) 是 DD 均衡后的结果。LMS 实现在几百个采样内收敛，而 DD 均衡器要约 2,000 个采样才睁开眼、且仍有可观噪声。LMS 优势明显，但需要确知源符号（DD 不用）。注意两个均衡器用同一 μ，初始化均为

$$f_{\text{LMS}} = f_{\text{DD}} = [1, 0, \ldots, 0] \in \{K \times 1\},$$

f_{DD} 不能全零初始化。

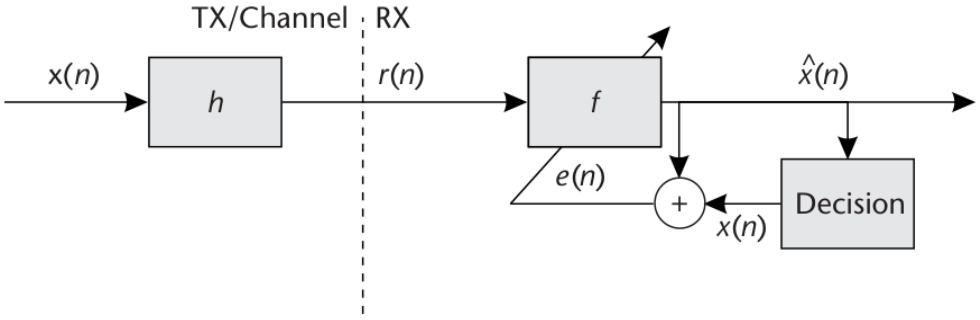


图 9.10　用已知判决引导数据对 FIR 信道 h 的自适应 FIR 均衡

9.3.1 非线性均衡器

到目前为止只考虑了带前向滤波器的均衡器（线性均衡器）。也可以实现前向 + 反馈双滤波器——称为**判决反馈均衡器**（DFE）。DFE 的反馈滤波器产出从前向滤波器输出中减去。见图 9.12（展示两个滤波器及各自更新路径）。由于反馈滤波器只能估计后标符号，必须与前向滤波器配合使用。收敛后，反馈滤波器包含" 信道与前向滤波器卷积" 的冲激响应估计。由于反馈滤波器利用该复合输出，DFE 可以补偿严重的幅度失真而不在高度失真的信道中放大噪声。

加入新滤波器 $d \in \{P \times 1\}$，更新先前的均衡器方程。为用 LMS 求解均衡给定信道 h 的滤波器 d，把 (9.3)–(9.6) 的递归修改为

$$\hat{x}(n) = f^H(n)r(n) - d^H(n)\bar{x}(n)$$

$$e(n) = \bar{x}(n) - \hat{x}(n)$$

$$f(n+1) = f(n) + \mu e^*(n)r(n)$$

$$d(n+1) = d(n) + \mu e^*(n)\bar{x}(n)$$

其中

$$r(n) = [r(n), r(n-1), ..., r(n-K-1)]^T,$$

与

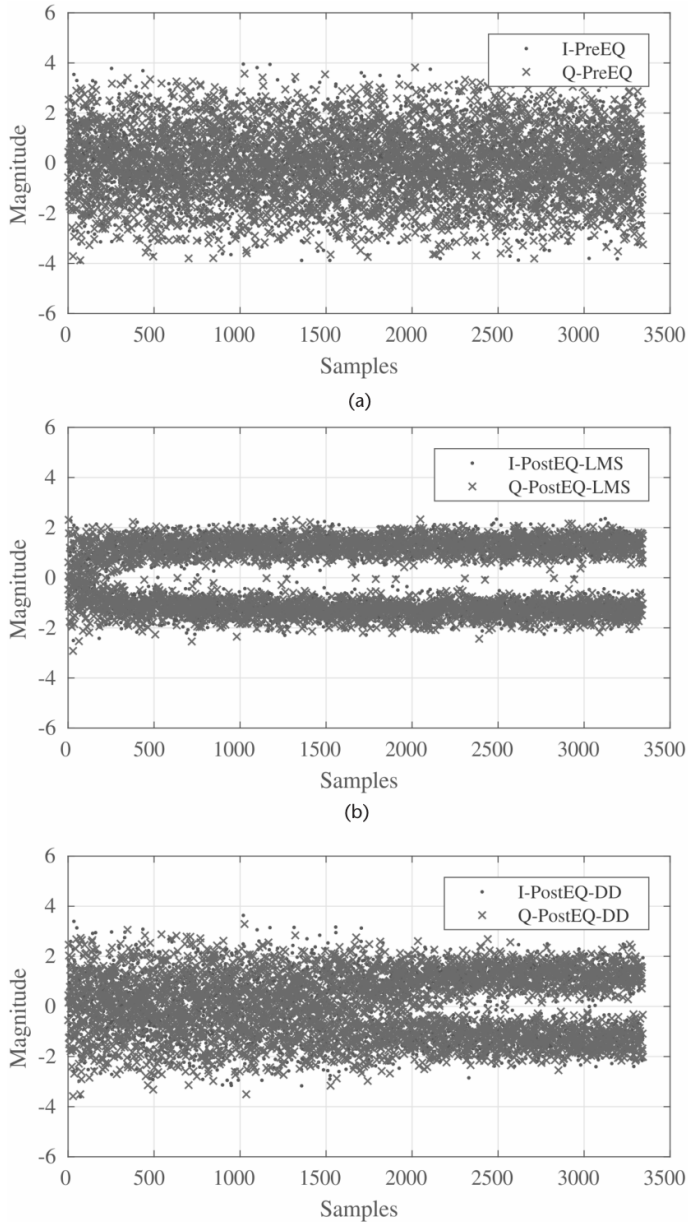


图 9.11　均衡前后的 ISI 对比：(a) 均衡前含 ISI 的 QPSK 信号；(b) LMS 均衡后；(c) DD 均衡后（原书此图题注误抄为互相关示例，已按图实际内容修正）

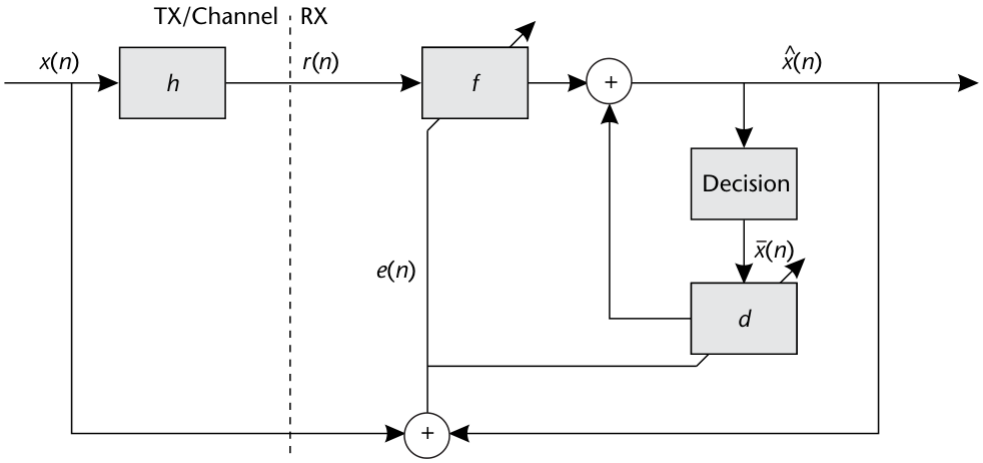


图 9.12　用判决反馈均衡器对 FIR 信道 h 的自适应 FIR 均衡

$$\bar{x}(n) = [\bar{x}(n), \bar{x}(n-1), ..., \bar{x}(n-P-1)]^T.$$

这里 $\bar{x}(n)$ 在用训练数据时取 $x(t)$ 本身，否则取 DD 版本。同样可更新 MATLAB 代码使用新的反馈滤波器，见代码 9.5。注意 DFE 实现因含两个滤波器更难调校；DFE 还可以对每个滤波器更新用不同步长，使调校更可控。

代码 9.5　确定误比特：chanEQDFE.m

```
h = [0.2; 1; 0.1; 0.02]; % Channel taps
mu = 0.001; % Stepsize
trainingSamples = 1e4;
x = sign(randn(trainingSamples,1)); % Generate BPSK data
r = filter(h,1,x); % Apply channel
K = length(h)+2; f = zeros(K,1);
P = length(h)-1; d = zeros(P,1); x_bar_vec = zeros(P,1);
delta = 4; % Reference tap
%% Equalize channel
index = 1;
[e,x_hat]=deal(zeros(trainingSamples-K+1,1));
for n = K:trainingSamples
    % Select part of training input
    in = r(n:-1:n-K+1);
```

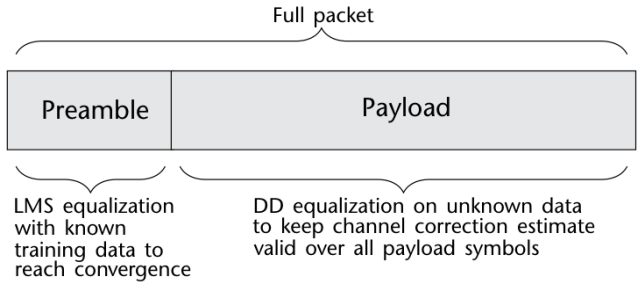


图 9.13　示例分组结构与不同均衡器算法的应用

```
    % Apply channel estimate to training data
    x_hat(index) = f'*in - d'*x_bar_vec;
    % Compute error
    e = x(n-delta)-x_hat(index);
    % Update taps
    f = f + mu*conj(e)*in;
    d = d - mu*conj(e)*x_bar_vec;
    % Update feedback filter
    x_bar_vec = [x(n-delta);x_bar_vec(1:end-1)];
    index = index + 1;
end
```

9.4 接收机实现

要把均衡器真正实现进接收机结构，有若干依系统需求而定的设计策略。一个值得考虑的合理设计视角：均衡给定信道环境所需的训练数据量。这里有三个方面：信道长度、均衡器的收敛、信道的动态性。训练数据通常限于前导码序列，所选长度按 L 的最大值选取——它也决定 M 的取值。

信道动态时（相对帧长在短时间内就会变化），需要使用多个均衡器。例如：用 LMS 均衡器以前导码序列初始睁开眼，再在数据段用 DD 均衡器保持眼睛张开（见图 9.13）。若不用第二个均衡器，就得缩短帧长才能满足同样的 BER 要求。但前导码必须足够长以允许均衡器恰当自适应——这取决于 M 与 L：均衡器越大、收敛所需数据越多、前导码越长。把代码 9.3 更新为 BPSK 下 LMS+DD 并用的版本只需几处简单修改：代码 9.6 里简单地加一个条件，在不同模式间切换。

代码 9.6　带 DD 与 LMS 模式的均衡器：chanEQLMSDD.m

```
for n = K+1:FrameLength
    % Apply equalizer to received data
    x_hat = f'*r(n:-1:n-K+1);
    % Estimate error
    if n<(PreambleLength-delta)
      e = x(i-delta) - x_hat;
    else
      e = sign(y) - x_hat;
    end
    % Update equalizer
    f = f + mu*conj(e)*r(n:-1:n-K+1);
end
```

均衡器在系统中的用处远不止补偿多径：还能补偿频偏、定时失配、甚至帧同步错误——前提是均衡器配置得当。例如应对载波偏移时（相对第 7 章的情形只能是比较小的偏移），应增大 μ。均衡器本身就是滤波器，补偿频偏只是让抽头形成等效于载波瞬时相位的复增益；但必须持续更新才能补偿该状况，迫使采用 DD+ 训练数据的双重实现。

考虑定时补偿时，实现**分数均衡器**很有用——前端前向滤波器消费多个采样。这使均衡器成为多速率的，也类似第 6 章的定时补偿设计。该配置下均衡器能跨符号加权/插值，准确校正接收信号的延迟。除分数定时偏移外，利用时延变量 δ 还能补偿帧同步期间的采样错误：只要期望采样（帧起始采样）落在前向滤波器范围内，就能补偿错误的初始估计。对长度 L 的前向滤波器与 $\delta = \lfloor \frac{L}{2} \rfloor$，应能补偿 $\pm\delta$ 的采样偏移。

9.5 本章小结

本章介绍了信道估计与均衡的概念。我们围绕梯度下降算法 LMS（该领域的主导代价函数）构建了几种均衡器设计；给出了训练数据可用与需要盲操作两种情形下的均衡器实现策略。最后讨论了组合多种均衡器设计的接收机实现策略，以及应对环境非理想性的不同用例。有兴趣的读者可在附录 C 找到基本线性均衡器、迫零均衡器与判决反馈均衡器的简要数学推导。

9.6 参考文献

[1] Goldsmith, A., *Wireless Communications*, Cambridge, UK: Cambridge University Press, 2005.

[2] Johnson, C. R., Jr., W. A. Sethares, and A. G. Klein, *Software Receiver Design: Build Your Own Digital Communications System in Five Easy Steps*, Cambridge, UK: Cambridge University Press, 2011.

[3] Widrow, B., and M. E. Hoff, *Neurocomputing: Foundations of Research, Adaptive Switching Circuits*, Cambridge, MA: MIT Press, 1988, pp. 123–134.

[4] Haykin, S., and S. S. Haykin, *Adaptive Filter Theory*, Pearson Education, 2014.

[5] Farhang-Boroujeny, B., *Adaptive Filters: Theory and Applications*, Wiley, 1999.

[6] Johnson, R., P. Schniter, T. J. Endres, J. D. Behm, D. R. Brown, and R. A. Casas, "Blind Equalization Using the Constant Modulus Criterion: A Review," *Proceedings of the IEEE*, Vol. 86, No. 10, 1998, pp. 1927–1950.

第 10 章 正交频分复用

到目前为止，我们学习的都是单载波调制方案：输入二进制比特由中心频率为 f_c 的载波信号调制。但还有其他跨信道传递数据的方法，包括称为**多载波调制**的技术。多载波调制（MCM）不只用一个中心频率，而是把串行输入数据复用成若干并行流、在相互独立的**子载波**上传输。这些子载波可以单独调制与操纵，从而针对信道进行优化。MCM 依据信道调整各子载波传输参数的能力，在对抗**频率选择性衰落**信道环境时尤其有用。因此，大多数高速率通信系统——包括所有数字用户线（xDSL）调制解调器 [1–3]——以及大多数商用通信标准——包括 Wi-Fi [4, 5]、WiMAX [6, 7] 与 LTE [8, 9]——都在其实现核心使用某种形式的 MCM。本章将探索最流行的 MCM 形式之一：正交频分复用（OFDM）。

10.1 MCM 的动机：色散信道环境

高中物理课上我们学过波的传播基础与相长/相消叠加（如涟漪槽实验）。完全相同的原理适用于高速数据传输。例如无线通信系统中，发射机向所有方向辐射能量（除非天线有方向性、能量聚焦在某方位角）。在开阔环境（如空旷的农田），能量会持续传播，其中一部分到达接收天线；其余能量则继续传播直到耗散。

在室内环境（如图 10.1(c)），情形不同。视距分量（若存在）p_1 最先到达接收天线——与开阔田野一样。但其余能量不会简单耗散：能量被墙壁与室内物体反射。其中一些反射（如 p_2 与 p_3）会到达接收天线，只是相位与幅度不同。这些接收分量是若干参数的函数，包括收发天线的总距离以及反射次数。

在接收端，这些分量只是同一发射信号的副本——到达时间、幅度与相位不同。因此可把信道视为一个冲激响应与发射信号卷积。开阔田野情形下，信道冲激响应（CIR）就是冲激——接收天线不会收到其他副本。而室内环境会在接收天线截获多个副本，其 CIR 类似图 10.1(a) 的示例；对应频率响应示于图 10.1(b)。

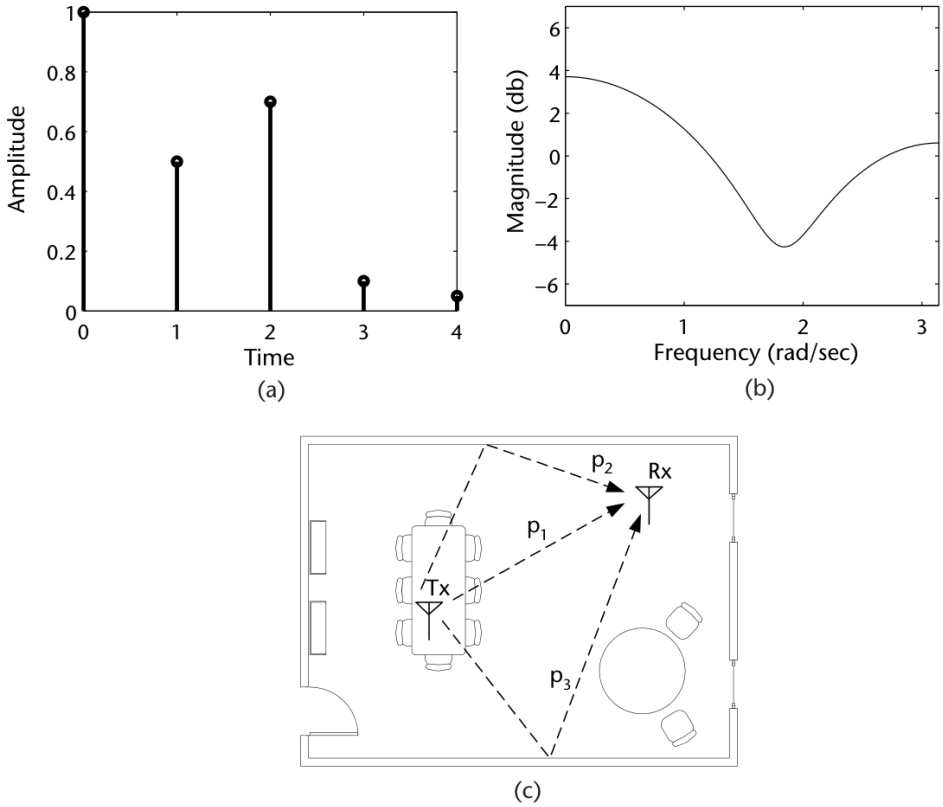


Figure 10.1 Example of a channel response due to dispersive propagation. Notice the three distinctive propagate paths, p_1, p_2, and p_3, that start at the transmitter Tx and are intercepted at the receiver Rx. (a) Impulse response, (b) frequency response, and (c) the process by which dispersive propagation arises.

图 10.1　色散传播引起信道响应的示例。注意从发射机 Tx 出发、在接收机 Rx 处被截获的三条独特传播路径 p1、p2、p3：(a) 冲激响应；(b) 频率响应；(c) 色散传播的产生过程

对具有显著多径特性的工作环境细看图 10.1(b)：CIR 的频谱在频域上起伏变化。这对任何经此类信道传输的信号都很麻烦——信号会经历随频率变化的非均匀衰减，在单载波通信系统的语境下相对难以缓解（需要设计与实现复杂的均衡滤波器）。而 MCM 通信系统恰好擅长处理这类失真：多个子载波被设计成对信道" 分而治之"、逐个处理频率相关的衰减，从而实现相对低复杂度的方案。

10.2 OFDM 一般模型

OFDM 是 MCM 的高效形式：用 DFT 与逆 DFT（IDFT）调制解调多个并行数据流。如图 10.2 所示，OFDM 通信系统的一般结构由发射端的 $2N$ 点 IDFT 与接收端的 $2N$ 点 DFT 组成。图 10.2 的 OFDM 收发信机这样工作：高速数字输入 $d[m]$ 经换向器分用到 N 个子载波；每个子载波上的数据被调制成 M-QAM 符号（一次映射一组 $\log_2(M)$ 比特）。对子载波 k，把 $a_k[\ell]$ 与 $b_k[\ell]$ 排列成实部与虚部，使调制器块输出 $p_k[\ell] = a_k[\ell] + jb_k[\ell]$。为使 IDFT 块的输出为实数，给定 N 个子载波须用 $2N$ 点 IDFT，其中端子 $k = 0$ 与 $k = N$ 是无关项输入。子载波 $1 \le k \le N-1$ 的输入为 $p_k[\ell] = a_k[\ell] + jb_k[\ell]$；子载波 $N+1 \le k \le 2N-1$ 的输入为 $p_k[\ell] = a_{2N-k}[\ell] + jb_{2N-k}[\ell]$。然后执行 IDFT，得到

$$s[2\ell N + n] = \frac{1}{2N} \sum_{k=0}^{2N-1} p_k[\ell] e^{j(2\pi nk/2N)},$$

其中此时 $s[n]$ 的 $2N$ 个连续采样构成一个 OFDM 符号——N 个不同 QAM 符号之和。由此数据被调制到若干子信道上，通过每个数据流乘以 $\sin(Nx)/\sin(x)$ 实现，其中若干示于图 10.3。

子载波经换向器复用成信号 $s[n]$ 并发往接收机。接收端用换向器与 $2N$ 点 DFT 把信号分用为 $2N$ 路子载波数据 $\hat{s}[n]$，DFT 定义为

$$\bar{p}_k[\ell] = \sum_{n=0}^{2N-1} \hat{s}[2\ell N + n] e^{-j(2\pi nk/2N)},$$

作用于输入，得到 $p_k[\ell]$ 的估计 $\bar{p}_k[\ell]$。均衡器的输出 $\hat{p}_k[\ell]$ 再经解调器，结果用换向器复用，得到重建的高速比特流 $\hat{d}[m]$。

10.2.1 循环扩展

CIR 建模为与发射信号采样卷积的有限冲激响应滤波器时，会使过去的采样拖尾到当前采样、当前又拖尾到未来。这种拖尾造成发射信号失真，增大系统总 BER、损失性

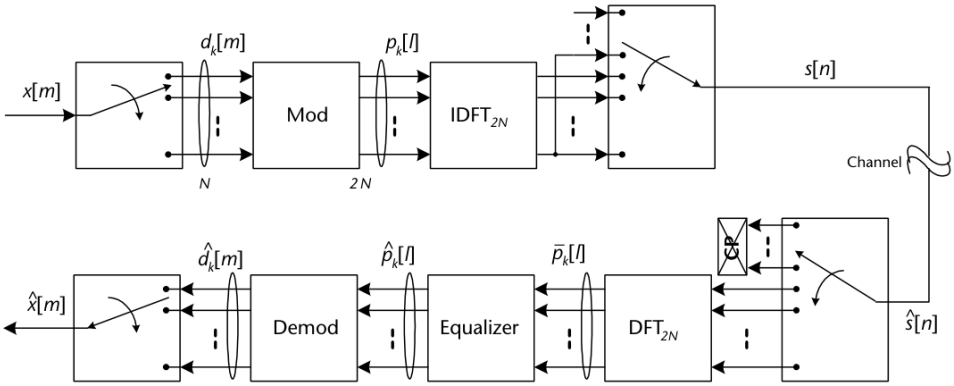


图 10.2　OFDM 系统总示意图，DFT 与 IDFT 用于调制解调数据流

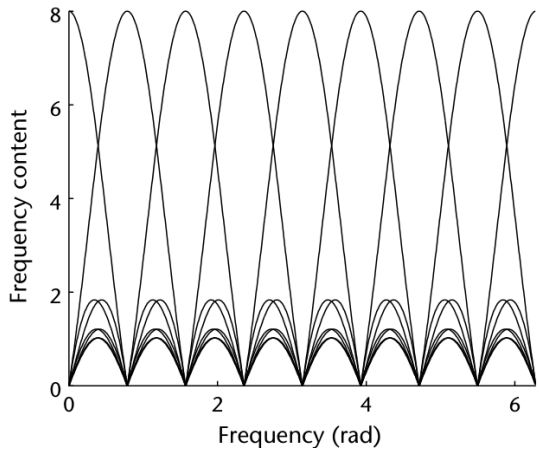


图 10.3　OFDM 子载波的频率响应（正交特性）

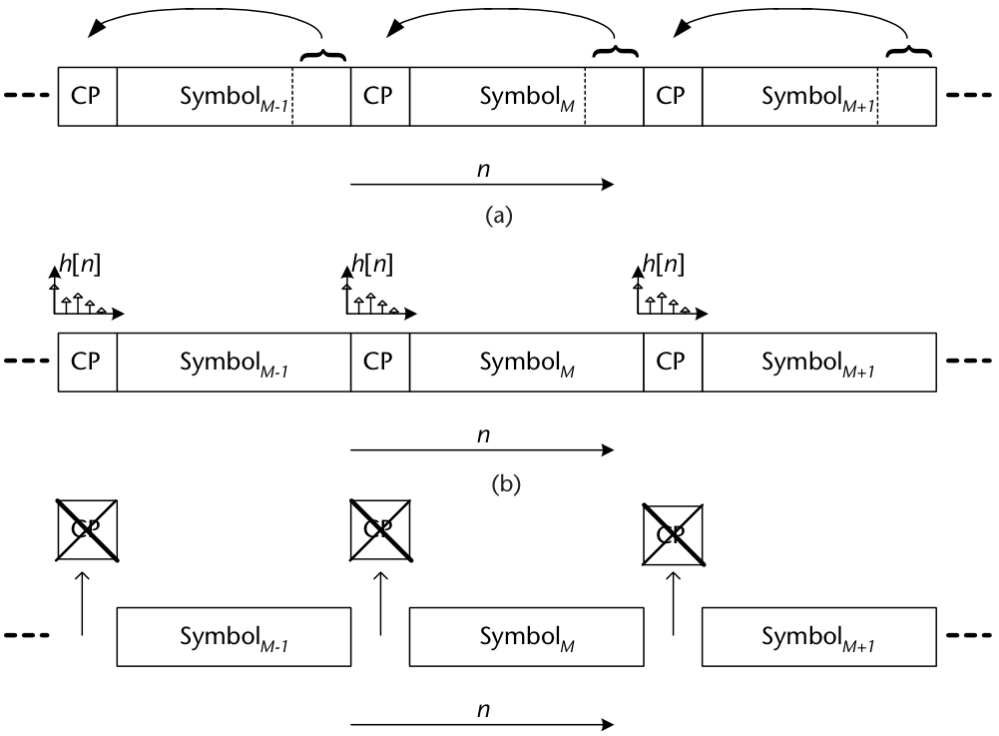


图 10.4　循环前缀的添加、拖尾捕获与移除过程：(a) 向 OFDM 符号添加循环前缀；(b) 信道 h(n) 从前一个符号向循环前缀的拖尾；(c) 移除循环前缀

能。

虽然可以设计均衡器来撤销信道效应，但均衡器的选择存在复杂度与失真最小化的折中。尤其是" 上一个 OFDM 符号拖尾到下一个符号" 的失真是难题。一个简单的解决方案：在符号之间放几个**哑采样**来捕获符号间拖尾效应。这 K 个哑采样最流行的选择是当前 OFDM 符号的**最后 K 个采样**——这种哑采样称为**循环前缀**（CP），如图 10.4(a) 所示。

于是，带循环前缀的 OFDM 符号经过信道时，来自先前符号的拖尾被循环前缀捕获（图 10.4(b)）。结果各符号只经历自己符号内部采样的拖尾。接收端移除循环前缀（图 10.4(c)），OFDM 符号继续解调与均衡。

尽管循环前缀有用，也有几个缺点。其一，循环前缀长度必须足以捕获 CIR 的影响，否则无法防止其他符号引入的失真。其二，循环前缀引入开销：给符号加缓冲采样就得向接收端发更多信息——要达到无 CP 系统的吞吐量，必须以更高速率发送。

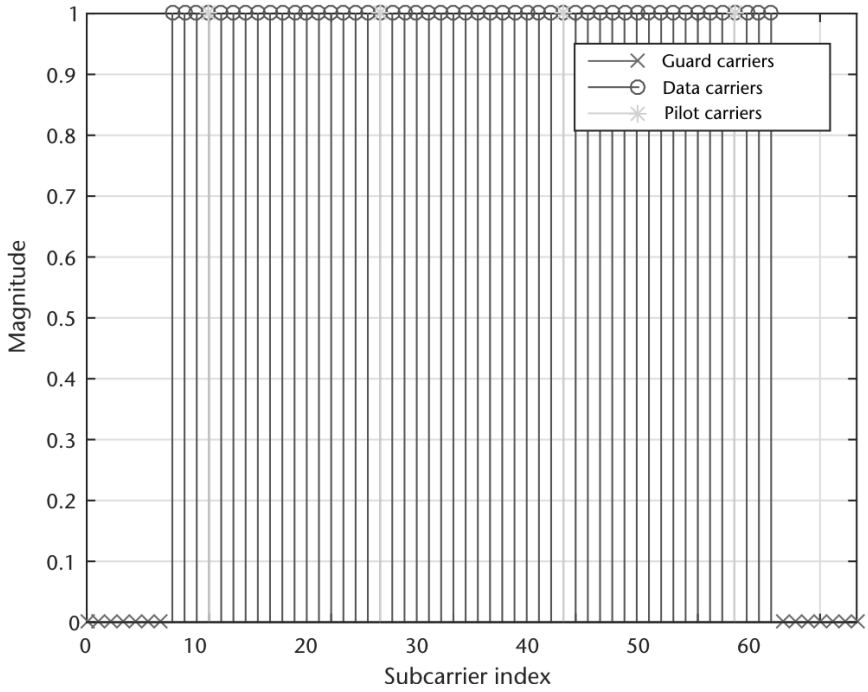


图 10.5　含数据与保护载波的 OFDM 符号子载波布局

10.3 常见 OFDM 波形结构

10.2 节讨论了一般 OFDM 传输模型与限制 ISI 的 CP。接下来考察许多标准（如 IEEE 802.11）使用的 OFDM 符号内部常见结构。首先，OFDM 符号的子载波数 N_s 通常取 2 的幂——用于调制解调的 DFT/IDFT 高效实现（FFT/IFFT）在这些长度上最高效。但典型实现会使用 N_g 个**保护载波**占据 OFDM 符号的外侧频率位置。例如图 10.5 中 $N_s = 64$、$N_g = 14$：前七个与后七个子载波空置，用于限制影响邻信道信号的带外干扰。代价是保护载波降低总体数据率。注意这是**加 CP 之前**的 OFDM 符号（CP 长度为 N_{cp}）。此外，会选择部分子载波作为训练或**导频音**，由接收端均衡器用于校正相位与频率偏移以及信道效应。类似图 10.5，所选子载波通常在符号内均匀选取，以覆盖整个 OFDM 符号。这些导频音在接收端已知。

OFDM 调制与 CP 应用于已调符号后，模拟发射前还可以向帧添加额外结构，依据传输类型而定。例如连续发射系统（LTE 或广播信号）会以周期速率向各帧做小插入，使接收机能够同步。这些训练/同步符号可以很精简——接收机可以多次尝试与发射机同步。而突发式系统（如无线局域网 WLAN）会有大的前导码结构，给接收机更多同步时间——这额外的时间允许高概率恢复帧、无需重传。对采用载波侦听多路访问（CSMA）

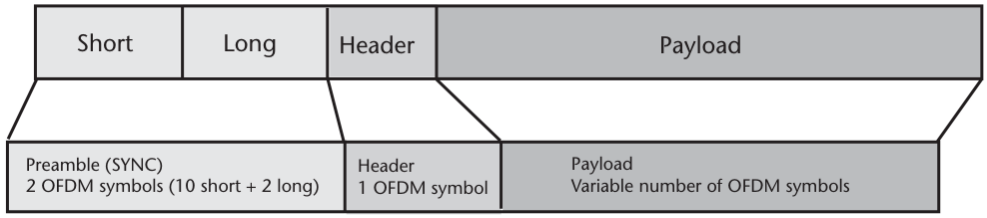


图 10.6　WLAN 802.11a 帧的前导码、帧头与载荷部分概览

/冲突避免（CA）MAC 方案的无线系统，重传的性能代价可能很高。

我们先考察 WLAN 的帧结构——它是 OFDM 系统的常见实现。数字视频地面广播（DVB-T2）使用与 WLAN 类似的结构，但前导码数据的调制方式不同 [10]。我们选择从标准视角考察 OFDM 收发有两个原因：第一，OFDM 实现因系统设计而异，但大体遵循此处讨论的两种模式；第二，本节讨论的标准在工业界应用广泛，理解其工作原理弥足珍贵——工程师职业生涯中多半会与 WLAN 或 LTE 打交道。

由图 10.6 可见，WLAN 帧的前导码由四个完整 OFDM 符号组成，无论模式或 MCS 链路设置如何都保持不变。第一段是前导码的短训练部分——IEEE 802.11 标准称为 Legacy Short Training Field（LSTF）[11]——含十个重复的短序列副本。该序列的目的是让接收机的自动增益控制（AGC）在给定的 8 μs 内稳定，并执行载波频偏（CFO）估计。这一般被视为接收机的**分组检测**阶段。

分组检测与 CFO 校正后，接收机利用长训练部分——IEEE 802.11 标准称为 Legacy Long Training Field（LLTF）[11]——含两个重复序列与一个 CP。前导码的这一部分设计用于信道估计。总体而言，OFDM 接收机先识别分组起点、校正频偏、校正信道效应、再去除残余相位失真。相比前几章考察单载波传输的顺序，这一处理可谓” 倒着来”。

前导码用于校正接收帧后，帧头符号将用于确定剩余载荷的长度与 MCS。最后，载荷中的导频音将用于校正任何残余偏移与信道失真。类似的帧可由代码 10.1 生成。

为提供使用 OFDM 的标准的另一视角，LTE 是绝佳例子。不同于把同步全部集中在前导码几个符号里的 WLAN，LTE 使用主同步信号（PSS）与辅同步信号（SSS），如图 10.7 所示。接收机经互相关检测，同时利用帧边界与散布在资源块（RB）中的导频音在接收端校正。由于下行信号在小间隔内恒定，漏掉一个 PSS/SSS 可以直接在下一个出现的序列中找到。一旦同步，接收机将持续解码来自基站的信息——不同于更偏突发式的 WLAN。

代码 10.1　生成 OFDM 分组：genOFDMPacket.m

```
NumFrames = 1;
%% Build OFDM Modulator
```

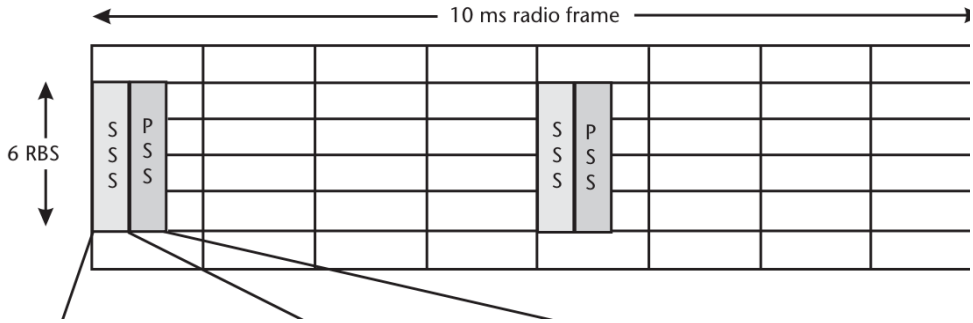


图 10.7　LTE 下行帧的简化概览，聚焦主辅同步信号

```
FFTLength            = 64;
NumGuardBandCarriers = [6; 5];
NumDataCarriers      = 48;
CyclicPrefixLength   = 16;
PilotCarrierIndices  = [12;26;40;54];
NumOFDMSymInPreamble = 5;
NumBitsPerCharacter  = 7;
% Convert message to bits
msgInBits = repmat(randi([0 1], NumDataCarriers, 1),10, 1);
PayloadBits = msgInBits(:);
% Calculate number of OFDM symbols per frame
NumOFDMSymbols = ceil(length(PayloadBits)/NumDataCarriers);
% Calculate number of bits padded in each frame
NumPadBits = NumDataCarriers * NumOFDMSymbols - length(PayloadBits);
% Get preamble for each frame
Preamble = double(getOFDMPreambleAndPilot('Preamble', ...
    FFTLength, NumGuardBandCarriers));
% Get pilot for each frame
Pilots = double(getOFDMPreambleAndPilot('Pilot', NumOFDMSymbols));
% BPSK modulator
BPSKMod = comm.BPSKModulator;
% OFDM modulator
DataOFDMMod = comm.OFDMModulator(...
    'FFTLength' ,           FFTLength, ...
```

```
    'NumGuardBandCarriers', NumGuardBandCarriers, ...
    'InsertDCNull',         true, ...
    'PilotInputPort',       true, ...
    'PilotCarrierIndices',  PilotCarrierIndices, ...
    'CyclicPrefixLength',   CyclicPrefixLength, ...
    'NumSymbols',           NumOFDMSymbols);

%% Modulate Data
symPostBPSK = BPSKMod.step([PayloadBits; randi([0 1], NumPadBits, 1)]);
% OFDM modulation for one frame
symPostOFDM = DataOFDMMod.step(reshape(symPostBPSK, ...
    NumDataCarriers, NumOFDMSymbols), Pilots);
% Repeat the frame
y = repmat([Preamble; symPostOFDM], NumFrames, 1);
```

10.4 分组检测

前导码中 LSTF 结构背后的原理基于 Schmidl 与 Cox 的工作 [12]。他们在论文中给出了一种符号定时恢复策略：寻找时域中**两半完全相同**的训练符号——经过信道后两半仍相同，只是存在由载波频偏引起的相位差。让训练符号两半相同（IFFT 之后）的方法是：在**偶数频率**上发伪噪声（PN）序列、奇数频率置零。这意味着每个偶数频率上发送源星座（WLAN 为旋转 BPSK）的一个点。给定这一结构可以完成两件事：其一，利用时域中已知的重复序列经相关估计前导码起点；其二，依据训练符号两半之间的相位差估计频率。[12] 有很多扩展（如 Minn [13]、[14]），但都保持" 频率与时间上对称的前导码" 这一原则。

数学上，[12] 的分组检测定时度量为

$$M(k)=\frac{\left|\sum_{m=0}^{L-1} r^{*}(k+m)r(k+m+L)\right|^{2}}{\left(\sum_{m=0}^{L-1}|r(k+m+L)|^{2}\right)^{2}}, \tag{10.3}$$

其中 r 是接收信号、L 是半个训练符号的长度。WLAN 中该长度为 16 个采样（0.8 μs）。(10.3) 利用前导码中长度 L 的重复序列。

代码 10.2 提供了执行 (10.3) 的简单示例——它依赖前面的代码 10.1 按 WLAN 802.11a

规格生成 OFDM 分组。结果度量绘于图 10.8：可以观察到从前导码起点的真实位置开始出现明显的平台。低 SNR 时 LSTF 序列的平台更难观察但仍可辨认。不过，度量 M 的目的只是二元地判断" 是否发了分组"——并不需要前导码的精确起点，这一接收阶段只需 LSTF 大致位于数据何处的粗略估计。

实践中该方法成为门限检测问题，必须把接收机建成能应对不同条件。(10.3) 是自相关实现——只用了接收信号。该实现有用之处在于自归一化输入、尽量把输出压到 0 与 1 之间。如图 10.8 所示这并非总能做到，但即使在低 SNR 下前导码的输出 M 也高于载荷数据。因此，对度量 M 选择恰当的门限值对判断信号空间中是否存在分组很重要。该参数在某些 Cisco 商用硬件中确实可调，定义为接收机分组起始检测门限（Rx SOP）[15]。概念上，改变该参数会改变无线小区尺寸、并反向影响接收分组差错率。

代码 10.2　检测分组起点：packetDetect.m

```
%% Generate OFDM Waveform
genOFDMPacket;
% Add random offset
offset = randi([0 1e2]);
y = [zeros(offset,1);y];

%% Schmidl and Cox: Coarse Packet Detection
L = 16;  % Short sync field length
m = L;   % Distance between fields
N = 300; % Autocorrelation samples
M = zeros(N,1);
SNR = [0,5,10];
for SNRindx = 1:length(SNR)
    r = awgn(y,SNR(SNRindx),'measured');
    % Determine timing metric
    for k=1:N
        P = r(k:k+m)' *  r(k+L:k+m+L);
        a = abs(y(k+L:k+m+L));
        R = a'*a;
        M(k) = abs(P)^2/(R^2);
    end
```

```
    % Plot
    subplot(length(SNR),1,SNRindx);stem(M);
    hold on; stem(offset+1,M(offset+1),'r*'); hold off;
    grid on;xlabel('k');ylabel('M');
    legend('Autocorrelation','True Start');
    title(['SNR: ',num2str(SNR(SNRindx)),'dB']);
end
```

10.5 CFO 估计

OFDM 的一个问题与频偏有关。当接收符号经历的偏移超过子载波带宽的一半时，信号失去正交性——破坏了与信号相关的恢复假设。因此，OFDM 信号解调（即 FFT 处理）之前必须先考虑频率校正。如 10.4 节所述，前导码序列两半的相位差与接收波形经历的频偏直接相关。这很容易证明：给定频偏 Δf 与发射波形 $s(t)$：

$$r(t) = s(t)e^{j2\pi\Delta f t/f_s} \tag{10.4}$$

利用 LSTF 中长度各为 L 的符号两半，可以写出

$$p(k) = \sum_{m=0}^{L-1} r^*(k+m)r(k+m+L) = \sum_{m=0}^{L-1} s(k+m)^* e^{-j2\pi\Delta f(k+m)/f_s} s(k+m+L)e^{j2\pi\Delta f(k+m+L)/f_s}$$

$$= e^{j2\pi\Delta f(L)/f_s} \sum_{m=0}^{L-1} s(k+m)^* s(k+m+L) = e^{j2\pi\Delta f(L)/f_s} \sum_{m=0}^{L-1} |s(k+m)|^2. \tag{10.5}$$

因此，频偏是前导码两半相位差的直接结果。对频偏 Δf，前导码符号两半之间的相位差 φ 直接相关：

$$\varphi = \frac{2\pi L\Delta f}{f_s}, \tag{10.6}$$

其中 f_s 是采样率。于是 φ 可估计为

$$\hat{\varphi} = \tan^{-1}\frac{\Im(P(k))}{\Re(P(k))}, \tag{10.7}$$

其中

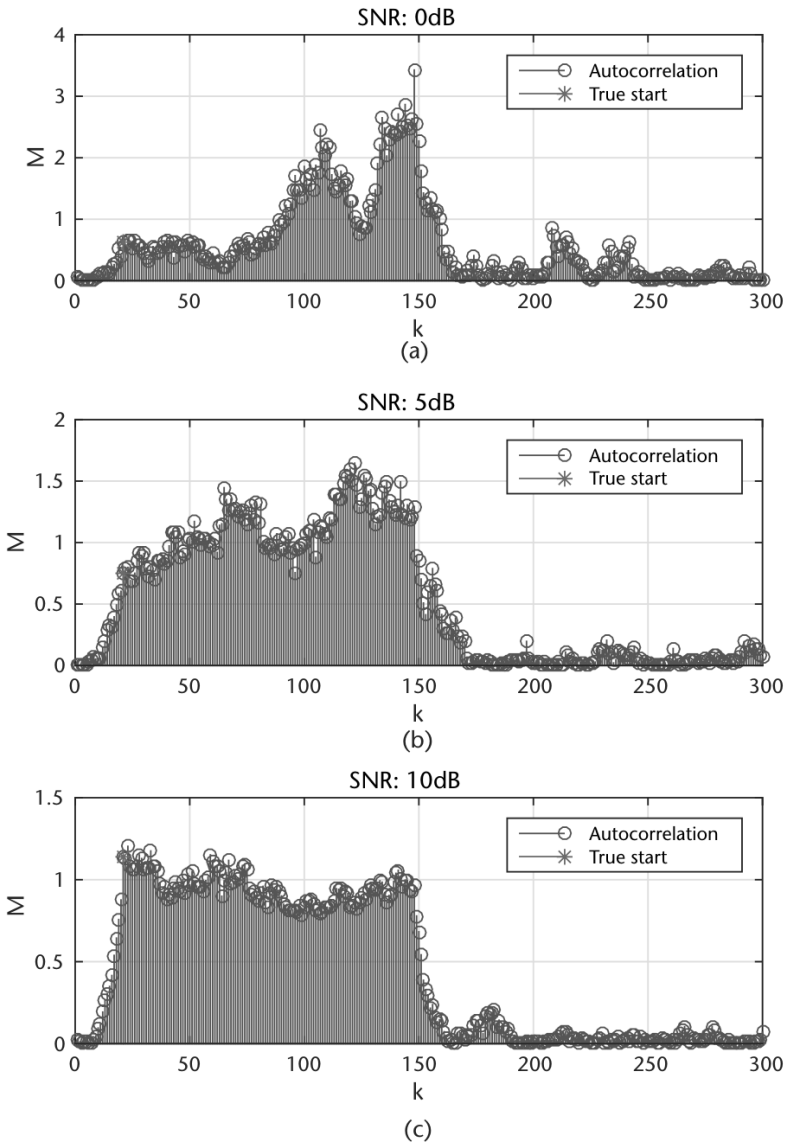


图 10.8　Schmidl-Cox 算法的定时度量 M（SNR 0/5/10 dB），给出 AWGN 下接收帧起点的粗估计

$$P(k) = \sum_{m=0}^{L-1} r^*(k+m)r(k+m+L), \tag{10.8}$$

且 (10.7) 中的 k 位于短前导码序列的前九个符号内。由于 φ 至多为 π，可直接确定最大偏移：

$$\Delta f_{\max} = \frac{f_s}{2LN}. \tag{10.9}$$

WLAN 802.11a 的情形下 $\Delta f_{\max} = 625$ kHz。但 IEEE 标准要求接收机维持 20 PPM 振荡器——5.3 GHz 处收发差至多 212 kHz [11]。

代码 10.3 在多个频偏上实现了 (10.5)–(10.8) 的技术。结果以归一化偏移分析——这很常见，因为频偏依赖系统采样率，可用于补偿大偏移：简单地把电台跑在更快速率就能降低恢复算法看到的归一化偏移。

代码 10.3　确定 CFO：freqEstOFDM.m

```
%% Generate OFDM Waveform
genOFDMPacket;
% Add random offset
offset = randi([0 1e2]); y = [zeros(offset,1);y];
SNR = 20;
%% CFO Estimation
L = 16;  % Short sync field length
m = L;   % Distance between fields
N = 300; % Autocorrelation samples
Fs = 1e6; % Sample rate
% CFO estimation over multiple frequencies
subchannelSpacing = Fs/FFTLength;
CFOs = subchannelSpacing.*(0.01:0.01:0.5);
cfoError = zeros(size(CFOs));
for cfoIndx = 1:length(CFOs)
    % Add noise
    r = awgn(y,SNR,'measured');
    % Frequency offset data
    pfOffset = comm.PhaseFrequencyOffset('SampleRate',Fs,...
```

```
        'FrequencyOffset',CFOs(cfoIndx));
    r = pfOffset(r);
    % Determine frequency offsets
    freqEst = zeros(N,1);
    for k=1:N
        P = r(k:k+m)' *  r(k+L:k+m+L);
        freqEst(k) = Fs/L*(angle(P)/(2*pi));
    end
    % Select estimate at offset estimated position
    freqEstimate = freqEst(offset);
    % Calculate CFO error
    cfoError(cfoIndx) = abs(freqEstimate-CFOs(cfoIndx))/subchannelSpacing;
end
```

由于前导码的短训练部分含同一序列的十个副本，至多可做九次估计。但受 AGC 收敛影响，短前导码的早期符号不太可能被检测到或可用——这正是提供额外短前导码副本的原因。若短前导码恢复得早（检测到大量符号），则可做更多频率估计并最终平均。

前导码的长训练部分保持与短训练部分相同的对称性，但只有两半可比较。仅用 LLTF 会因 L 增大而缩小估计范围。不过，若 AGC 来不及锁定，LLTF 可能只能用于频率估计。在理想情形（长短前导码序列都能用）下，可以实现**分级补偿**设计：类似第 7 章讨论的单载波实现——短序列提供粗偏移估计，长前导码用于精细频率估计。之所以这样分级，是因为估计时可用的数据量不同：长前导码对给定的一次 P 计算通常能用更多数据，给出更可靠的估计。该分析直接来自 [12]，其估计 Δf 估计值的方差为

$$\sigma^2 = \frac{1}{L \times \mathrm{SNR}}. \tag{10.10}$$

10.6 符号定时估计

分组检测与频率校正之后，可以进行符号定时。符号定时与分组检测类似，但提供时域中前导码序列的**符号级**精度估计。为提供这一额外精度，用**已知的发射前导码序列**做互相关。互相关提供更好的相关 SNR，但若不加额外缩放则不直接归一化到任何范围——取值范围不定，使互相关不适合初始分组检测。

OFDM 符号定时估计中的互相关技术用于确定短训练序列与长训练序列的边界（采

样级精度）——此时 LSTF 的后续使用已无必要。示例实现：用 LLTF 序列（记为 $d_{\mathrm{LLTF}}(t)$）的前 80 个采样。d_{LLTF} 与接收信号 r 的度量 c 为

$$c(k) = \left| \sum_{m=0}^{L-1} d^*_{\mathrm{LLTF}}(m) r(m+k) \right|^2 . \tag{10.11}$$

代码 10.4 给出计算 c 的两种实现，见图 10.9。两者结果相同，但滤波器实现在计算上更高效。绘制代码 10.4 任一实现的 c 应得到类似图 10.9 的图——清晰显示相距 64 个采样的两个峰。这些峰代表 LLTF 两半的强相关，即使低 SNR 下也很突出。由这些采样位置可直接确定 OFDM 符号边界、用 FFT 执行 OFDM 解调。

实际偏移可按代码 10.5 轻松算出：由于相关操作，第一个峰的位置会偏移 LLTF 长度（1 个 OFDM 符号）的一半。

代码 10.4　确定精细符号偏移度量：fineSymEst.m

```
genOFDMPacket;
% Add random offset with noise
offset = randi([0 1e2]); y = [zeros(offset,1);y];
SNR = 0; r = awgn(y,SNR,'measured');
%% Estimate fine sample offset
LSTF = Preamble(1:160);
LLTF = Preamble(161:161+160-1);
symLen = 80; % FFTLength + CPLen
known = LLTF(1:symLen,1);
% Filter
coeff = conj(flipud(known));
c_filt = abs(filter(coeff, 1, r)).^2;
% Correlation
m = abs(xcorr(r,known)).^2;
padding = length(r)-symLen+1;% Remove padding added to known sequence
c_cor = m(padding:end);
```

代码 10.5　计算偏移位置：fineSymEst.m

```
[v1,peak1] = max(c_cor);
c_cor(peak1) = 0;
```

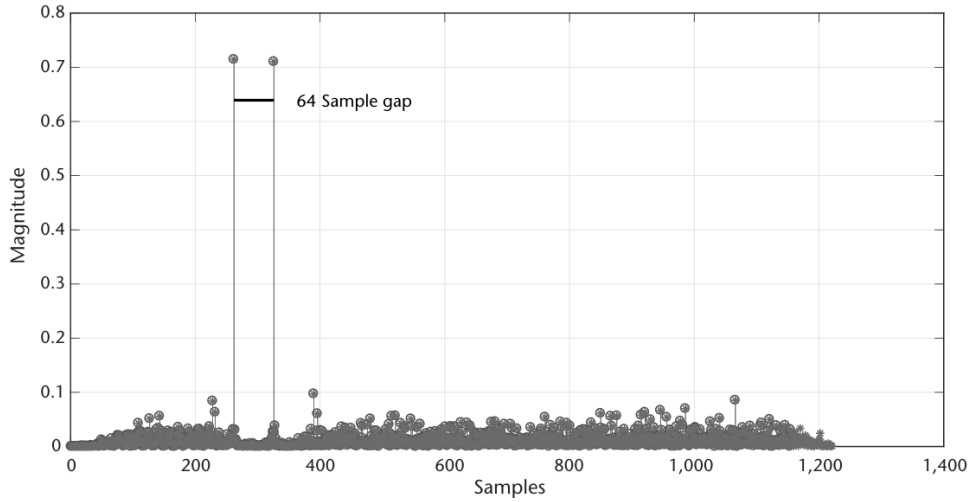


图 10.9　10 dB SNR 下用互相关得到的符号定时度量 c。LLTF 序列位置处的峰清晰高于其余相关采样

```
[v2,peak2] = max(c_cor);
% Get numerical offset
if abs(peak2-peak1)==FFTLength
    % Adjust position from start of LLTF
    p = min([peak1 peak2]);
    LLTF_Start_est = p-symLen;
    LSTF_Start_est = LLTF_Start_est - length(LSTF);
    % Display
    disp([offset LSTF_Start_est]);
end
```

10.7 均衡

接收机的最后一块是均衡器——负责削弱信道效应、去除接收信号中残余的相位或频率偏移。这里讨论的技术在 OFDM 解调**之后**执行，但也存在可在 FFT 之前实现的其他方法。

循环前缀的主要优点之一：它把发射信号 $s[n]$ 与信道冲激响应 $h[n]$ 之间的线性卷积变换成逐符号的**循环卷积**。为看清这一点，细看一个带循环前缀的 OFDM 符号（符号从时刻 $n = 0$ 开始）。用 $s[0], \ldots, s[2N-1]$ 表示发射机 IDFT 对第一个 OFDM 符号的 $2N$ 个输出采样，添加循环前缀得到新信号：

$$\tilde{s}[n]=\begin{cases}s[n+2N-K], & 0\le n\le K-1\\ s[n-K], & K\le n\le 2N-1\end{cases}$$

记 $\tilde{r}[n]$ 为 $\tilde{s}[n]$ 与长度 L 的信道冲激响应 $h[n]$ 卷积的结果：

$$\begin{aligned}\tilde{r}[n]&=\sum_{k=0}^{L-1}h[k]\tilde{s}[n-k]\\ &=\begin{cases}\displaystyle\sum_{k=0}^{n-K}h[k]s[n-K-k]+\sum_{k=n-K+1}^{L-1}s[n-k+2N-K], & K\le n\le K+L-1\\ \displaystyle\sum_{k=0}^{L-1}h[k]s[n-K-k], & K+L\le n\le 2N-1\end{cases}\end{aligned}$$

由上式容易看出：移除循环前缀后，接收序列 $r[n]=\tilde{r}[n+K]$ 等于

$$r[n]=\sum_{k=0}^{2N-1}h[k]s[((n-k))_{2N}]=h[n]\circledast_{2N}s[n]. \tag{10.12}$$

于是，移除循环前缀得到的接收采样由发送信号（每符号 $2N$ 个采样）与信道冲激响应 $h[n]$ 的**循环卷积**构成。在频域观察 (10.12)：

$$R[k]=H[k]\cdot S[k],$$

其中大写字母表示对应序列的 $2N$ 点 DFT。回看图 10.2：接收采样的 $2N$ 点 DFT $R[k]$ 已算出，记为 $\bar{p}_k[\ell]$。

参照图 10.3 与图 10.1(b)：考虑对应频率采样相乘，可见每个子载波经历不同的信道增益 $H[k]$。因此要做的，是给每个子载波乘以作用于该子载波的信道频率响应的**逆**增益——这就是**逐音均衡**的原理。

知道不同子载波处的信道频率增益后，可用它们除各子载波、反转信道造成的失真。例如系统有 64 个子载波、中心频率 $\omega_k=2\pi k/64$（$k=0,\ldots,63$），则取 CIR $h[n]$ 做 64 点 FFT 得频率响应 $H[k]$（$k=0,\ldots,63$）。然后为反转信道对每个子载波的影响，只需取该子载波对应信道频率响应点的倒数：

$$W[k]=\frac{1}{H[k]}, \tag{10.13}$$

与该子载波相乘。

回看 WLAN 的帧结构：LLTF 最适合信道估计——它接收端永远已知、且子载波覆盖整个 OFDM 符号。LLTF 有两个完整 OFDM 符号都用于信道估计，对其估计取平均得到更可靠的结果。OFDM 解调用 `comm.OFDMDemodulator`——它可以方便地从匹配必要设置的调制器对象生成（见代码 10.6，原书此处误标为 10.8）。

代码 10.6　OFDM 调制器/解调器：exampleMod.m

```
% Modulator
Mod = comm.OFDMModulator(...
        'FFTLength' ,           FFTLength,...
        'NumGuardBandCarriers', [6;5],...
        'CyclicPrefixLength',   0,...
        'NumSymbols', 2,...
        'InsertDCNull', true);
% Demodulator
Demod = comm.OFDMDemodulator(Mod);
```

用该解调器对象可解调 LLTF 并用所述最小二乘（迫零）法估计信道。LLTF 的解调、信道估计与数据符号的均衡见代码 10.7。但即使做了这一步的校正，数据中仍可能存在频偏。幸运的是，数据符号内仍嵌有导频用于纠正这些问题——代码 10.7 也给出了用插入导频的均衡。

导频的用法很简单：校正数据中逐符号可能经历的加性相位旋转。该旋转同样是复增益，估计方式与 LLTF 类似 [16]。假设数据中的导频也已被 LLTF 估计校正，则把数据 OFDM 符号中的 N_p 个导频 $\hat{p}(k)$ 与已知导频 $p(k)$ 对照产生单一增益：

$$e(k) = \frac{1}{N_p} \sum_{n=0}^{N_p-1} p_n(k)\hat{p}_n^*(k). \tag{10.14}$$

该复增益 $e(k)$ 随后与第 k 个 OFDM 符号中的每个采样相乘。说明：$p_n(k)$ 是第 k 个 OFDM 符号的第 n 个已知导频。

代码 10.7　OFDM 均衡：eqOFDM.m

```
genOFDMPacket;
% Add random offset
offset = randi([0 1e2]); y = [zeros(offset,1);y];
```

```
%% Equalization Example
r = awgn(y(offset+1:end),15,'measured'); Fs = 1e6;
pfOffset = comm.PhaseFrequencyOffset('SampleRate',Fs,...
    'FrequencyOffset',Fs*0.001);
r = pfOffset(r);
%% Channel estimation
preambleOFDMMod = comm.OFDMModulator(...
    'FFTLength' ,           FFTLength,...
    'NumGuardBandCarriers', NumGuardBandCarriers,...
    'CyclicPrefixLength',   0,'NumSymbols', 2,'InsertDCNull', true);
od = comm.OFDMDemodulator(preambleOFDMMod);
od.PilotOutputPort = true;
% OFDM Demodulate LLTF
LLTF = Preamble(161:161+160-1); rLLTF = r(161+32:161+160-1);
[rLLTFFreq,rp] = od(rLLTF); [LLTFFreq,p] = od(LLTF(33:end));% remove CP
% Estimate channel
ls = rLLTFFreq./LLTFFreq; % Least-square estimate
chanEst = mean(ls,2); % Average over both symbols
CSI = real(chanEst.*conj(chanEst));
ls = rp./p; % Least-square estimate
chanEstPilots = mean(ls,2); % Average over both symbols
CSIPilots = real(chanEstPilots.*conj(chanEstPilots));
%% Perform Equalization
data = r(2*length(LLTF)+1:end);
odd = comm.OFDMDemodulator(DataOFDMMod);
[dataFreq,pilots] = odd(data);
% Apply LLTF's estimate to data symbols and data pilots
postLLTFEqData = bsxfun(@times, dataFreq, conj(chanEst(:))./CSI(:));
postLLTFEqPilots = ...
    bsxfun(@times, pilots, conj(chanEstPilots(:))./CSIPilots(:));
% Visualization objects
tt1 = comm.ConstellationDiagram;tt2 = comm.ConstellationDiagram;
tt2.Position = tt2.Position + [500 0 0 0];
```

```
% Estimate remaining offsets with pilots
correctedSymbols = zeros(size(postLLTFEqData));
for symbol = 1:size(postLLTFEqData,2)
    % Estimate rotation across pilots
    p = postLLTFEqPilots(:,symbol);
    e = conj(mean(p.*conj(Pilots(:,symbol))));
    % Equalize
    sig = postLLTFEqData(:,symbol).*e;
    correctedSymbols(:,symbol) = sig;
    % Visualize
    tt1(sig);tt2(postLLTFEqData(:,symbol));pause(0.1);
end
```

10.8 比特与功率分配

资源分配是 OFDM 设计的重要方面——它决定 OFDM 系统的 BER 性能。给定固定比特率与发射功率约束，通过在子载波间恰当分配比特与功率电平可以最小化 BER。本节介绍比特与功率分配的理论与技术。

大多数 OFDM 系统在全部子载波上使用同一信号星座（图 10.10(a)）——换向器给每个子载波分配相同大小的比特组。但其总差错概率受**性能最差的子载波**支配。为改善性能，可采用**自适应比特分配**：各子载波上信号星座尺寸的分布依据实测 SNR 值变化（图 10.10(b)）——换向器分配不同大小的比特组。极端情况下，子载波 SNR 太差时可以直接关断或置零。

假设收发双方都掌握子载波 SNR 电平，就能确定各子载波的 BER 值。由于子载波 BER 依赖于给定 SNR 下所选的调制方式，可以改变各子载波的调制方案来改变其 BER。更大的星座（如 64-QAM）需要更大 SNR 才能达到小星座（如 4-QAM）在更小 SNR 下的相同 BER。

我们直接按

$$b_i = \log_2\left(1 + \frac{\gamma_i}{\Gamma}\right), \tag{10.15}$$

给子载波 i 分配比特数 b_i，其中 γ_i 是子载波 i 的 SNR（非 dB）。当然，由此得到的 b_i 附近会出现非整数比特，需适当取整。

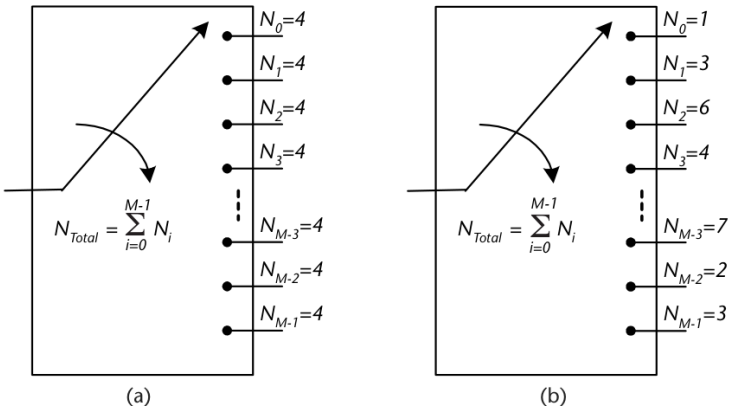


图 10.10　等速率与变速率换向器的对比（总速率相同）

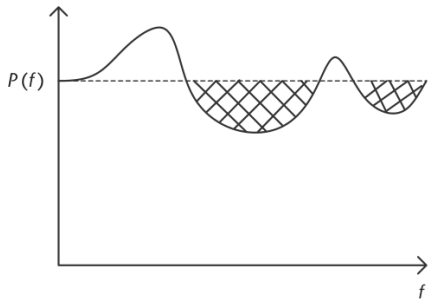


图 10.11　注水原理示意。注意功率如何分配给各子载波，使结果功率为常数 K

信道响应不总是平坦的：某些频率可能远比其他频率衰减得多。可以证明，子载波的最优功率分布应为常数 K。确定该电平通过**注水**完成——选择某频率处的功率 $P(f)$，使所有 $P(f_1) = P(f_n)$。$P(f)$ 由下式给出：

$$P(f) = \begin{cases} \lambda - \frac{N(f)}{|H(F)|^2}, & f \in F \\ 0, & f \in F' \end{cases}$$

其中 $N(f)$ 是高斯噪声的 PDF、$H(f)$ 是表示线性信道的传递函数、λ 是满足

$$\int_F P(f)df = P$$

的值（即图 10.11 中曲线下的面积）。这样注水是为了让所有子载波的误比特概率相同。

10.9 组装成整体

至此，恢复经空中发射的 OFDM 符号所需的全部接收机算法都已给出。显然整体设计还可以用信道编码与扰码器进一步降低接收 BER，但基本构件已齐。作为回顾，图 10.12 给出接收机流程图，注释了前导码与导频的用法。目标是最大限度利用前导码符号提供主要的同步校正——这在很多方面不同于主要依赖调制方案本身的单载波实现。

10.10 本章小结

本章聚焦多载波调制的原理与实现，基于 802.11a WLAN 标准。详述了 OFDM 的优点（包括简洁的接收机设计）。OFDM 是现代高速收发机制——从有线互联网到某些卫星链路。尽管接收机的实现基本与前几章的顺序相反，它却可以被认为比单载波实现简单得多。

10.11 参考文献

[1] Cioffi, J. M., Chapter 34, “Asymmetric Digital Subscriber Lines,” in *Communications Handbook*, CRC Press in Cooperation with IEEE Press, 1997.

[2] Golden, P., H. Dedieu, and K. Jacobsen, *Fundamentals of DSL Technology*, Boca Raton, FL: Auerbach Publications, 2004.

[3] Golden, P., H. Dedieu, and K. Jacobsen, *Implementation and Applications of DSL Technology*, Boca Raton, FL: Auerbach Publications, 2007.

[4] Hanzo, L. L., Y. Akhtman, L. Wang, and M. Jiang, *MIMO-OFDM for LTE, WiFi and WiMAX: Coherent versus Non-coherent and Cooperative Turbo Transceivers*, Chichester, UK: Wiley-IEEE Press, 2010.

[5] Lee, B. G., and S. Choi, *Broadband Wireless Access and Local Networks: Mobile WiMax and WiFi*, Norwood, MA: Artech House, 2008.

[6] Pareek, D., *WiMAX: Taking Wireless to the Max*, Auerbach Publications, 2006.

[7] Nuaymi, L., *WiMAX: Technology for Broadband Wireless Access*, Chichester, UK: Wiley, 2007.

[8] Sesia, S., I. Toufik, and M. Baker, *LTE—The UMTS Long Term Evolution: From Theory to Practice*, Second Edition, Chichester, UK: Wiley, 2011.

[9] Cox, C., *An Introduction to LTE: LTE, LTE-Advanced, SAE and 4G Mobile Communications*, Second Edition, Chichester, UK: Wiley, 2012.

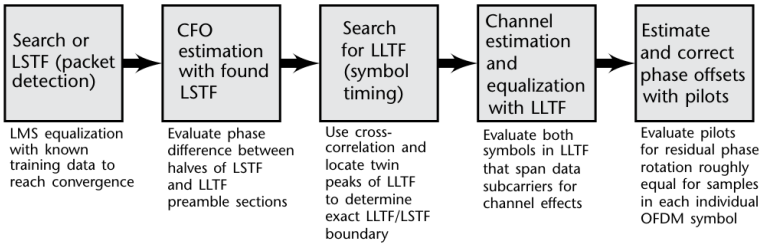


Figure 10.12 System overview for OFDM receiver for full frame recovery.

Chapter Summary

This chapter focused on the principle and implementation of multicarrier modulation, based on the 802.11a WLAN standard. Details were provided on the benefits of OFDM, including the simplistic receiver design. OFDM is the modern mechanism for high-speed transmission and reception, ranging from wired internet to even some satellite links. Although the implementation of the receiver is basically reversed from the previous succession of chapters, it can be considered much simpler than our single-carrier implementations.

ɛnces

[1] Cioffi, J. M., Chapter 34, "Asymmetric Digital Subscriber Lines," in *Communications Handbook*, CRC Press in Cooperation with IEEE Press, 1997.

[2] Golden, P., H. Dedieu, and K. Jacobsen, *Fundamentals of DSL Technology*, Boca Raton, FL: Auerbach Publications, 2004.

[3] Golden, P., and H. Dedieu, and K. Jacobsen, *Implementation and Applications of DSL Technology*, Boca Raton, FL: Auerbach Publications, 2007.

[4] Hanzo, L. L., Y. Akhtman, L. Wang, and M. Jiang, *MIMO-OFDM for LTE, WiFi and WiMAX: Coherent versus Non-coherent and Cooperative Turbo Transceivers*, Chichester, UK: Wiley-IEEE Press, 2010.

[5] Lee, B. G., and S. Choi, *Broadband Wireless Access and Local Networks: Mobile WiMaxand WiFi*, Norwood, MA: Artech House, 2008.

[6] Pareek, D., *WiMAX: Taking Wireless to the Max*, Auerbach Publications, 2006.

[7] Nuaymi, L., WiMAX: *Technology for Broadband Wireless Access*, Chichester, UK: Wiley, 2007.

[8] Sesia, S., I. Toufik, and M. Baker, *LTE—The UMTS Long Term Evolution: From Theory to Practice*, Second Edition, Chichester, UK: Wiley, 2011.

[9] Cox, C., *An Introduction to LTE: LTE, LTE-Advanced, SAE and 4G Mobile Communications*, Second Edition, Chichester, UK: Wiley, 2012.

[10] Digital Video Broadcasting (DVB), Implementation Guidelines for a Second Generation Digital Terrestrial Television Broadcasting System (DVB-T2), ETSI TS 102 831 V1.2.1, ETSI, 2012.

图 10.12　OFDM 接收机完整帧恢复的系统概览

[10] Digital Video Broadcasting (DVB), *Implementation Guidelines for a Second Generation Digital Terrestrial Television Broadcasting System (DVB-T2)*, ETSI TS 102 831 V1.2.1, ETSI, 2012.

[11] IEEE, *Standard for Information Technology–Telecommunications and Information Exchange between Systems, Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications–Amendment 4: Enhancements for Very High Throughput for Operation in Bands below 6 GHz*, IEEE Std 802.11ac-2013, December 2013, pp. 1–425.

[12] Schmidl, T. M., and D. C. Cox, "Robust Frequency and Timing Synchronization for OFDM," *IEEE Transactions on Communications*, Vol. 45, No. 12, December 1997, pp. 1613–1621.

[13] Minn, H., M. Zeng, and V. K. Bhargava, "On Timing Offset Estimation for OFDM Systems," *IEEE Communications Letters*, Vol. 4, No. 7, 2000, pp. 242–244.

[14] Kang, K. W., J. Ann, and H. S. Lee, "Decision-Directed Maximum-Likelihood Estimation of OFDM Frame Synchronisation Offset," *IEEE Electronics Letters*, Vol. 30, No. 25, December 1994, pp. 2153–2154.

[15] Cisco Inc., "Configuring Receiver Start of Packet Detection Threshold," *Cisco Wireless Controller Guide*, Release 8, April 2015, pp. 517–523.

[16] van Zelst, A., and T. C. W. Schenk, "Implementation of a MIMO OFDM-Based Wireless LAN System," *IEEE Transactions on Signal Processing*, Vol. 52, No. 2, February 2004, pp. 483–494.

第 11 章 SDR 的应用

到目前为止，本书聚焦于理解与掌握用软件定义无线电技术构建成功通信系统的工具。通信系统各构建块的理论概念与实现它们的实用洞察都已广泛覆盖。此刻你可能会问：**我能用 SDR 做什么？** SDD 确实是设计、探索与实验通信系统的强大工具，但如何挥舞它去创新与创造？本章讨论两个显著受益于 SDR 多功能性与性能的应用：**认知无线电**与**车联网**。特别地，将讨论认知无线电中实现智能与学习的两种途径——熊蜂行为建模与强化学习；车联网方面则聚焦定义车载自组织网络（VANET）中车对车（V2V）与车对基础设施（V2I）通信的 IEEE 802.11p 与 IEEE 1609 标准。本章的目标是为读者提供 SDR 如何应用于这些前沿应用的洞察。

11.1 认知无线电

认知无线电的概念由 Joseph Mitola 于 2000 年提出 [1]，是一种强大的通信方法：网络中的每台电台都能感知环境、适应其运行行为、并在遇到新情况时即时学习（见图 11.1）。由于认知无线电具备感知、适应与学习的能力，它要求其运行的通信系统高度多功能——因此 SDR 技术非常适合实现基于认知无线电的通信系统。

虽然 SDR 看似非常适合认知无线电应用，仍需解决若干设计与实现考量。看图 11.2：认知无线电引擎在 SDR 平台的基带处理部分之上形成一层。基带处理可以是若干计算技术之一——通用微处理器、FPGA、DSP、GPU、ARM 及其他嵌入式计算技术；甚至可能多种基带处理技术共存于同一系统。给定具体 SDR 系统的计算技术后，需要留意：并非所有 SDR 都一样，每种计算技术各有优劣。例如 FPGA 不易重编程——不适合频繁变更的通信系统操作；但非常胜任要求原始算力的应用。选对计算硬件会显著影响认知无线电的性能。

细看图 11.2，认知无线电引擎由若干输入（感知的环境信息、期望的性能指标、可用的电台配置）、一个输出（待实施的电台配置）与一条反馈回路组成。引擎的核心是

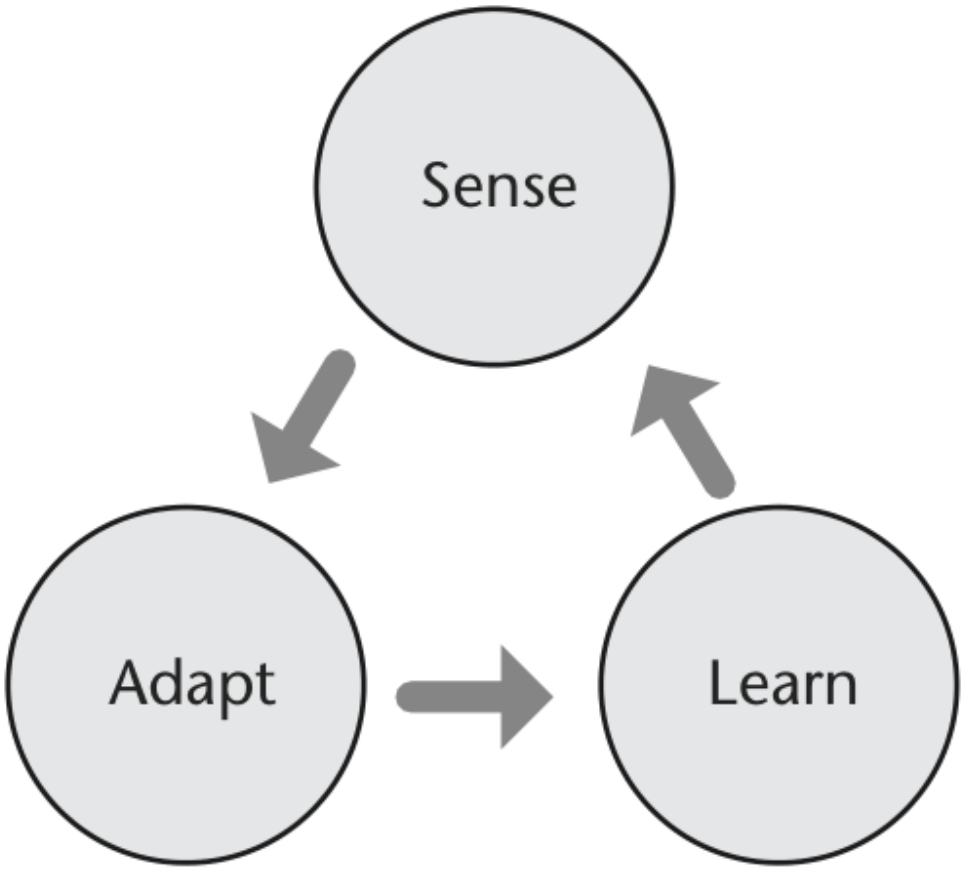


图 11.1　认知无线电的感知-适应-学习循环。正是学习环节的存在，把认知无线电与纯自动化或纯自适应的电台区分开来

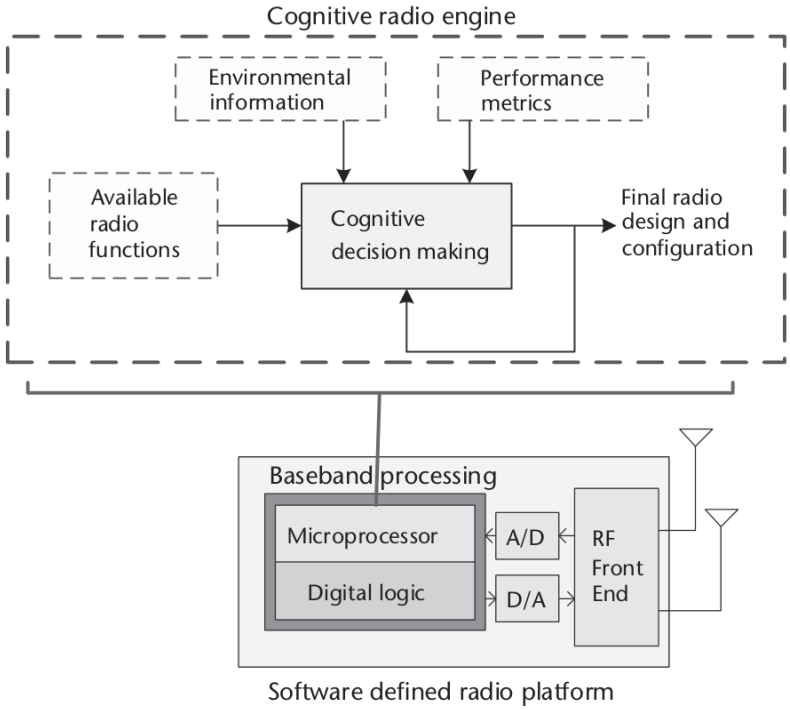


图 11.2　运行于软件定义无线电平台上的认知无线电引擎概念图

决策过程——依据过往经验、可用电台配置、感知的环境参数与期望性能指标，确定最佳可实施的电台配置。决策过程常用机器学习实现，可选项极多。为理解认知无线电引擎的设计考量，逐一细看这些要素。

机器学习技术已被广泛研究用于部分或完全自动化（重）配置过程（见 [2–6] 及其文献）。但这些技术产生的解通常需要对无线设备与目标网络行为有所了解（如高速率、高纠错、低时延 [7]）。不过，机器学习技术非常适合处理设备配置解空间极大的场景 [4–6]。

影响认知无线电系统的一个主要问题是其决策的准确性——取决于认知引擎输入信息的质量与数量。信息越多，引擎决策越好，越精确地达成期望的用户体验。回看图 11.2，认知无线电系统使用三类参数：

1. **设备配置**：可改变设备当前运行状态的一组参数。注意若干潜在配置可能无法实施，会被适应算法禁止。
2. **环境参数**：表示设备当前状态的信息，以及用外部传感器感知的无线环境信息。
3. **目标网络体验**：这些指标近似描述普通人类用户操作无线设备时的体验。认知无线电的目标是为给定指标取得尽可能好的值。

由于所有应用运行在不同环境、需求各异，认知引擎为某一应用产出优异性能的解，用于另一应用时可能不可接受。

最优决策的定义：达成目标网络体验优于任何其他可用组合的” 设备配置 + 环境参数” 组合。为认知无线电系统定义恰当的设备配置参数清单至关重要。构造良好的清单由一组各自对目标网络行为影响巨大的常用无线参数组成。表 11.1 给出无线网络设备常用的九个传输参数。

表 11.1: 常用无线网络设备配置参数（据 [5, 8]）

参数	说明
发射功率	原始发射功率
调制类型	调制的种类
调制阶数	给定调制方案的符号数
载波频率	载波的中心频率
带宽	发射信号带宽
信道编码率	具体编码率
帧大小	发射帧的尺寸
时分双工	发射时间占比
符号率	每秒符号数

环境参数告知系统周围环境特性，包括设备运行状态的内部信息与代表无线信道环境的外部信息，两者都可辅助认知无线电决策。这些变量连同目标网络体验一起作为算法输入。表 11.2 给出可影响认知无线电设备运行状态的六个环境参数。

表 11.2: 常用无线网络环境参数（据 [5, 8]）

参数	说明
信号功率	接收端看到的信号功率
噪声功率	给定信道的噪声功率密度
时延扩展	信道路径时延及其幅度的方差
电池寿命	电池剩余能量的估计
功耗	当前配置的功耗
频谱信息	频谱占用信息

基于机器学习的认知无线电系统的目的，是为人类操作者自主改善无线网络体验。但” 改善体验” 可以指多种事情：大量研究聚焦同一网络内容纳更多无线用户；其他重要方面包括无差错通信、高速率、限制用户间干扰、乃至无线设备的实际功耗（移动应

用中极其重要）。表 11.3 定义了引导认知无线电走向特定最优解的五个常用目标网络体验。

表 11.3: 常用无线网络目标体验（据 [5, 8]）

目标	说明
最小化误码率	改善传输环境的总体 BER
最大化数据吞吐量	提高电台发射的总数据吞吐量
最小化功耗	降低系统消耗的功率
最小化干扰	减少无线干扰贡献
最大化频谱效率	最大化频谱的高效利用

表 11.3 的目标体验是引导认知无线电无线系统状态的手段。认知无线电通过" 描述如何修改设备参数达成这些目标" 的关系来使用这些体验。为便于决策，每个目标网络体验必须用数学关系表示——把设备配置与环境参数映射到描述该组合达成特定目标程度的标量 [5, 8]。这些函数为认知无线电提供了对" 配置 + 环境参数组合" 排序的方式，最终导向决策。

注意可以同时指定多个目标网络体验，总分用数值表示：各目标体验的分数按其在具体应用中的重要性加权求和，形成最终总分 [5]。

11.1.1 熊蜂行为模型

到目前为止聚焦于认知引擎决策过程的框架，还未真正探索在电台上做决策的不同途径。来看两种潜在的决策执行方案。第一种是基于熊蜂行为的仿生方法 [9]。

谈到认知无线电时，人们听到" 认知" 就会联想到某种驱动决策的人类智能。但就我们追求的性能与计算复杂度的显著成本而言，这可能大材小用。因此，若干研究者转而聚焦认知能力更简单的生命形式，作为认知引擎及其他应用中决策过程的基础。过去几十年，研究者考察了蚁群与蜜蜂的行为作为智能高效决策的基础。这些生命形式是社会性动物——相互交换信息、执行产生最佳回报的行动。但蚁群与蜜蜂是**社会依赖型**生命形式：一个个体的行动完全依赖于集体。把这些仿生决策过程移植到认知无线电与 SDR 时，会产生极具挑战的运行环境。假设每台电台运行一个认知引擎，收集自身环境信息以及邻近电台的信息——有了这些丰富信息，我们期望电台对自己的配置做出出色决策。但若使用这些社会依赖模型，也约束了逐电台的决策方式。

熊蜂也是在群体中生活的社会性生命形式，但与蜜蜂和蚁群不同，熊蜂本质上是**社会独立型**——它们从环境收集信息、彼此分享信息，但**自己做决定**、不受集体控制。正是这种灵活性使熊蜂非常适合可能随时间快速变化的运行环境。把熊蜂行为模型用于

认知无线电，很适合这样的环境：网络拓扑频繁变化、信道条件与频谱可用性随时间变化、网络中电台数量变化。让每台电台运行基于熊蜂行为模型的认知引擎非常棒——它们收集环境信息、然后行动增强自身性能。

为在认知无线电引擎中实现熊蜂行为模型，需要借助**觅食理论**——熊蜂与许多其他生命形式收集资源时所采用的理论 [9]。本质上，觅食理论是生命形式用来采集食物、猎捕猎物、寻找庇护所等操作的资源优化形式。就熊蜂而言，可以把它们的活动与对周围世界的解读映射到采用 SDR 的无线通信环境。例如表 11.4 给出了熊蜂行为与感知到动态频谱接入的车联网之间的映射。可以看到熊蜂的许多行为与基于认知无线电的车辆网络有某种程度的相似。

表 11.4: 车联网通信中的若干定义及其在熊蜂中的对应定义（据 [9]）

车辆	熊蜂
带内干扰	蜂群在同一花卉物种上觅食
带外干扰	蜂群在替代花卉物种上觅食
最小信道能量级	每个花卉物种的最大花蜜级
计算/处理时间	处理/搜索时间
时延 vs 可靠性	采样频率 vs 选择准确性
信道间切换成本/时间	花卉物种间切换成本/时间
信道活动随时间变化	花卉物种占用随时间变化
信道-用户分布	蜂群在花卉物种间的分布

11.1.2 强化学习

近来备受关注的另一类决策过程是**强化学习**——机器学习的一种形式。如图 11.3 所示，强化学习采用一个**智能体**：输入上一次行动的回报及相关状态，决定新的行动。行动可以是任何东西，但就构建认知引擎而言，行动主要是具体的电台配置——使通信系统在当前运行条件（如色散无线信道）下性能可接受。接收端计算与该行动相关的**回报**——定义行动选得好不好。它经开销信道送回智能体闭合反馈回路，使智能体调整未来的行动（回忆图 11.2 认知引擎中的反馈回路）。

虽然图 11.3 的结构看似直接，但一个顾虑是闭合回路所需的开销信道。若能最小化开销信道在该框架中的影响，强化学习方法可以更稳健。最小化开销信道影响同时保持不错性能的一种方案是使用**神经网络**。神经网络本质上是个黑箱：可以用大量数据训练，形成复杂的输入-输出关系来代替闭式数学表达式。看图 11.4：神经网络在强化学习框架内辅助智能体依据过往回报与状态决定行动的示例。事实证明，若神经网络训练充分、能数学地建模通信信道的相关回报，就可以用它运行强化学习智能体——直

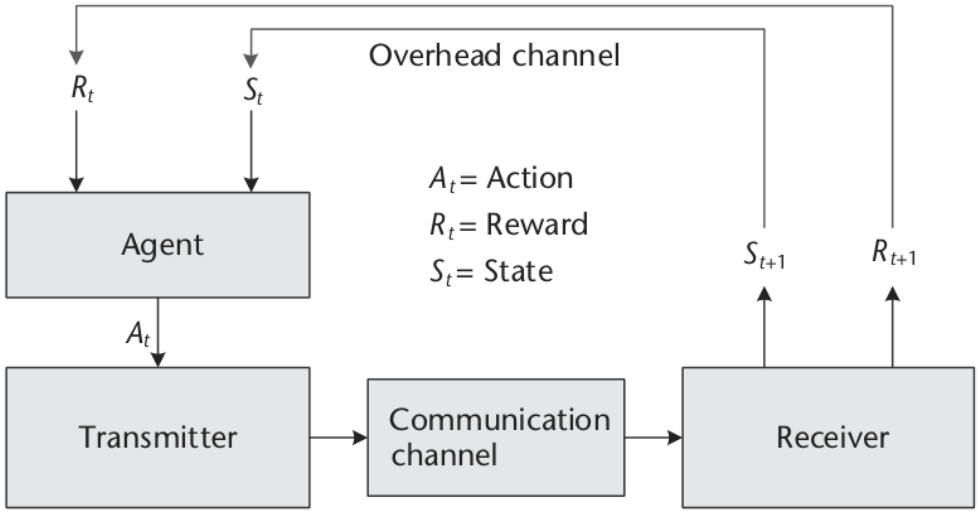


图 11.3　用强化学习方法智能适配通信系统到其运行环境的概念图 [10]

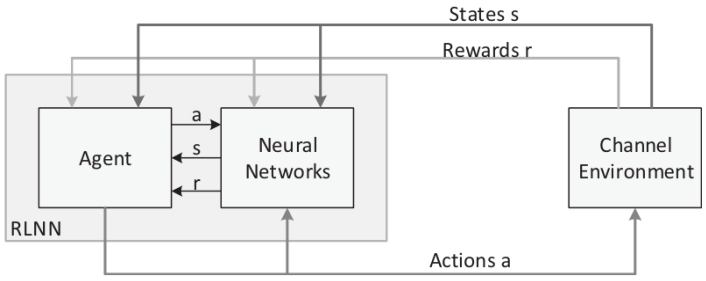


图 11.4　通信系统的混合强化学习框架 [11]

到信道条件显著变化、需要重新训练网络为止。

11.2 车联网

有了认知无线电的洞察，接下来探索一个” 认知无线电 +SDR 技术” 真能改变格局的应用：**车联网**。

车联网在过去几十年被广泛研究 [12]，尤其是车对车（V2V）与车对基础设施（V2I）通信 [13–16]。鉴于运行环境的复杂本质——快速变化的网络拓扑 [17]、时变的传播介质物理特性 [18, 19]、对稳健介质访问控制（MAC）协议的需求 [20]——车联网是学术界与工业界共同攻关的挑战性研究领域。

IEEE 802.11p（专用短程通信 DSRC）与 IEEE 1609（车载环境无线接入 WAVE）是实现 V2V 与 V2I 网络架构的已批准标准 [13, 16, 20–23]。由于这些标准是流行 IEEE 802.11

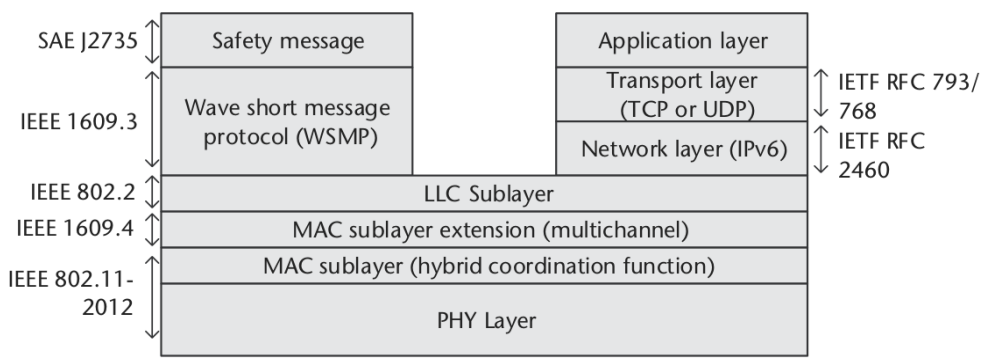


图 11.5　车载通信系统的协议栈

无线网络架构族的相对简单扩展，部署合规无线设备的成本相当低。但与使用 Wi-Fi 的室内环境不同，车联网环境复杂得多，带来了 Wi-Fi 社区此前未曾经历过的问题。

VANET 是专门针对移动地面车辆场景的移动自组织网络（MANET）的一种。三类 VANET 应用包括 [16]：

- **道路安全应用**：告警应用与紧急车辆告警应用。这些应用的消息拥有最高优先级。
- **交通管理应用**：本地与地图信息。
- **信息娱乐**：基于传统 IPv6 互联网的多媒体内容。

VANET 架构中可以同时存在 V2V 与 V2I 链路。V2V 中每辆车装备车载单元（OBU），V2V 通信在各车的 OBU 之间进行，主要用于道路安全与交通管理应用 [24]（V2V DSRC 的测量见 [15]）。V2I 应用中，路边基础设施可装备路侧单元（RSU）。为支持这些通信，VANET 有两个标准化协议：IEEE 802.11p 与 IEEE 1609。图 11.5 给出了采用 IEEE 802.11p 与 IEEE 1609 的车载电台单元协议栈的图形表示。

参照图 11.5：IEEE 802.11p [13] 规定 PHY 与 MAC 层，上层由 IEEE 1609.x 协议定义。IEEE 802.11p 具有许多与 IEEE 802.11-2012 标准 [23] 相似的特性。但为降低高度动态车辆通信环境中的通信时延，MAC 层需要能支持网络拓扑的快速变化与低时延通信。因此 IEEE 802.11p 与 IEEE 1609.4 都为 MAC 层定义了新特性。例如 IEEE 802.11 中无线节点可组成服务集（SS）——节点拥有相同 SSID 并共享通信。带接入点（AP）的网络称为基本服务集（BSS），无 AP 的网络（自组织网络）称为独立 BSS（IBSS）；多个 BSS 可连成扩展服务集（ESS），共享同一扩展 SSID（ESSID）。对 VANET 而言问题在于：任何通信开始前先组建 SS 可能相当耗时——不适合快速变化的车联网环境。为处理该问题，IEEE 802.11p 标准提出了**上下文外 BSS**（OCB）：OCB 模式适用于单个无线电链路覆盖区内的多台设备。OCB 模式下，车辆可随时收发数据，无需组建或加入 SS。此外，IEEE 802.11p 把认证、关联与数据保密机制从 MAC 层移除、移到 IEEE

1609.2 定义的独立更高层 [21]。反过来，IEEE 802.11p 仍保留 BSS 模式，主要用于经 V2I 通信的信息娱乐应用。

MAC 方面，传统 IEEE 802.11 使用载波侦听多路访问（CSMA）/冲突避免（CA）。IEEE 802.11p 仍采用 CSMA 机制，但还采用混合协调功能（HCF）——通过 IEEE 802.11e 定义的增强分布式信道接入（EDCA）保证服务质量（QoS）。不同服务的数据依据重要性有不同优先级。例如，CSMA/CA 与提议的自组织时分多址（STDMA）机制的对比显示 STDMA 的丢包率更低 [25]。但该论文未完整考虑 HCF 机制（未评估基于 QoS 的 EDCA 与基于无竞争期 CFP 的 HCCA）。也有为 WAVE 提出的基于优先级的 TDMA MAC 机制以降低传输丢包率 [26]。

基于 IEEE 802.11p 的 VANET PHY 层衍生自 IEEE 802.11a 标准，有三种信道带宽选项：5 MHz、10 MHz 与 20 MHz，其中推荐 10 MHz。与 IEEE 802.11a 一样，IEEE 802.11p 使用 OFDM：52 个载波（48 个数据载波 + 4 个导频）、8 μs 符号间隔。物理信道支持 BPSK、QPSK、16-QAM 与 64-QAM。除 IEEE 802.11p 外，IEEE 1609.4 定义 MAC 层的多信道行为 [20]：PHY 层由七个信道组成，IEEE 1609.4 定义控制信道（CCH）与服务信道（SCH）间的信道切换机制。IEEE 1609.3 定义 VANET 中两类消息：WAVE 短消息协议（WSMP）与 IPv6 栈 [22]。IPv6 通常用于信息娱乐应用，安全应用则经 WAVE 短消息（WSM）传输。此外，SAE J2945 规定 SAE J2735 DSRC 消息集及相关数据帧、数据元素的最小通信性能要求。为确保车辆间互操作性，SAE J2945 进一步定义 BSM 发送率、发射功率控制与自适应消息率控制。

SAE 标准已被汽车工业广泛用于安全消息实现。特别是 SAE J2735 定义了 15 类安全消息，如基本安全消息（BSM）、信号相位时间（SPT）消息与 MAP 消息 [16]。

BSM 以 10 Hz 频率向周围车辆周期性广播，公布车辆的位置、速度、加速度、航向等状态信息 [27]。使用**选择性广播**：DSRC 发射范围边缘的其他车会转发某辆车发的消息；原始发送者收到转发消息后取消自己的广播。BSM 消息特性在 DSRC 中是强制项。注意 VANET 的选择性广播已在 NS-3 中实现 [28]。SAE J2735 中 BSM 消息由两部分组成：基本部分与可选部分 [29]。基本部分包括车辆的位置、运动、时间与一般状态信息，始终使用 DER 编码与若干八位组二进制大对象编码的组合发送 [27]。可选部分仅在必要时发送，为接收设备的进一步处理提供辅助信息。

DSRC 范围内的车辆可通过 BSM 相互共享态势感知信息，场景包括：

- **变道告警**：车辆在 DSRC 范围内经 V2V 通信周期性共享位置、航向、方向与速度等态势信息。当驾驶员示意变道意图时，OBU 能判断其他车辆是否位于盲区。若盲区确有车辆则向驾驶员告警（盲区告警）；若无，OBU 会依据 BSM 交通信息预测相邻车道的间隙是否足够安全变道。若相邻车道间隙不足，向驾驶员提供变道

告警。

- **碰撞告警**：车辆动态接收 BSM 交通信息，与自身位置、速度、航向、道路信息比对。依据比对算法的结果，车辆判断潜在碰撞是否可能发生，并向驾驶员提供碰撞告警。
- **紧急车辆告警**：紧急车辆发射信号，通知附近车辆有紧急车辆正在接近。

除常规安全消息外，BSM 消息也可用于传输控制消息：协助协作碰撞告警环境 [30]、安全消息路由应用 [17]、或改善功率控制 [31]。对应急信道（即信道 172 与信道 184），BSM 可传达功率控制信息以协调各信道的发射功率。反过来，BSM 也可作为车辆控制算法的输入。控制消息在范围内的车辆间传输。

给定这些 VANET 通信的规范与标准，个人完全有可能实现自己的具备 V2V 与 V2I 通信能力的电台。虽然整个协议栈浩大、电台设计复杂度不小，但信息足以造出符合 IEEE 802.11p 与 IEEE 1609 的电台。在 SDR 平台上实现 IEEE 802.11p 与 IEEE 1609 需要考虑的主要问题包括：电台自身的计算性能、可达成吞吐量意义上的带宽限制、以及协议栈每个功能的实时性。尽管有这些挑战，构建能在路上实时组网的车辆通信 SDR 系统的机会是存在的。

11.3 本章小结

本章简要考察了两个可以大量借助 SDR 技术的现实应用：认知无线电与车联网。认知无线电方面，我们探索了如何在 SDR 平台上搭建认知引擎，并给出了构造决策过程的至少两种方式——熊蜂行为模型或强化学习方法。车联网方面，我们简要介绍了 IEEE 802.11p 与 IEEE 1609 标准——它们使我们能用 SDR 技术从零构建自己的车联网。

在本书中，我们深入探讨了信号、系统与通信的理论基础，继而探索了通信系统相关的现实问题及缓解这些损伤所需的解决方案，最后在介绍认知无线电与车联网等现实应用之前，展示了均衡与 OFDM 实现的若干高级主题。当然，本书只是掀开了整个通信系统领域的冰山一角，但希望它能成为掌握这一重要主题的起点。

11.4 参考文献

[1] Mitola, J., *Cognitive Radio—An Integrated Agent Architecture for Software Defined Radio*, Ph.D. dissertation, Royal Institute of Technology, Stockholm, Sweden, 2000.

[2] Barker, B., A. Agah, and A. M. Wyglinski, “Mission-Oriented Communications Properties for Software Defined Radio Configuration,” in *Cognitive Radio Networks*, Y. Xiao and

F. Hu (eds.), Boca Raton, FL: CRC Press, 2008.

[3] Newman, T., A. M. Wyglinski, and J. B. Evans, "Cognitive Radio Implementation for Efficient Wireless Communication," in *Encyclopedia of Wireless and Mobile Communications*, B. Furht (ed.), Boca Raton, FL: CRC Press, 2007.

[4] Newman, T., R. Rajbanshi, A. M. Wyglinski, J. B. Evans, and G. J. Minden, "Population Adaptation for Genetic Algorithm-Based Cognitive Radios," *Mobile Networks and Applications*, Vol. 13, No. 5, 2008, pp. 442–451.

[5] Newman, T. R., B. A. Barker, A. M. Wyglinski, A. Agah, J. B. Evans, and G. J. Minden, "Cognitive Engine Implementation for Wireless Multicarrier Transceivers," *Journal on Wireless Communications and Mobile Computing*, Vol. 7, No. 9, November 2007, pp. 1129–1142.

[6] Rieser, C. J., *Biologically Inspired Cognitive Radio Engine Model Utilizing Distributed Genetic Algorithms for Secure and Robust Wireless Communications and Networking*, Ph.D. Thesis, Virginia Polytechnic Institute and State University, Blacksburg, VA, 2005.

[7] Troxel, G. D., E. Blossom, and S. Boswell, et al., "Adaptive Dynamic Radio Open-source Intelligent Team (ADROIT): Cognitively Controlled Collaboration among SDR Nodes," in *Proceedings of the IEEE Communications Society Conference on Sensors, Mesh and Ad Hoc Communications and Networks (SECON)—Workshop on Networking Technologies for Software Defined Radio Networks*, Reston, VA, August 2006.

[8] Wyglinski, A. M., M. Nekovee, and T. Hou, *Cognitive Radio Communications and Networks: Principles and Practice*, Burlington, MA: Academic Press, 2009.

[9] Aygun, B., R. J. Gegear, E. F. Ryder, and A. M. Wyglinski, "Adaptive Behavioral Responses for Dynamic Spectrum Access-Based Connected Vehicle Networks," *IEEE Comsoc Technical Committee on Cognitive Networks*, Vol. 1, No. 1, December 2015.

[10] Ferreira, P. V. R., R. Paffenroth, A. M. Wyglinski, T. M. Hackett, S. G. Bilen, R. C. Reinhart, and D. J. Mortensen, "Multi-Objective Reinforcement Learning for Cognitive Radio-Based Satellite Communications," in *34th AIAA International Communications Satellite Systems Conference*, Cleveland, OH, October 2016.

[11] Ferreira, P. V. R., R. Paffenroth, A. M. Wyglinski, T. M. Hackett, S. G. Bilen, R. C. Reinhart, and D. J. Mortensen, "Multi-Objective Reinforcement Learning-Based Deep Neural Networks for Cognitive Space Communications," in *Cognitive Communications for Aerospace Applications Workshop (CCAA)*, Cleveland, OH, June 2017.

[12] Hartenstein, H., and L. Laberteaux, "A Tutorial Survey on Vehicular Ad Hoc Networks," *IEEE Communications Magazine*, Vol. 46, No. 6, 2008, pp. 164–171.

[13] IEEE, *Standard for Information Technology–Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 6: Wireless Access in Vehicular Environments*, Technical Report, July 2010.

[14] Kenney, J. B., "Dedicated Short-Range Communications (DSRC) Standards in the United States," *Proceedings of the IEEE*, Vol. 99, No. 7, 2011, pp. 1162–1182.

[15] Muller, M., "WLAN 802.11p Measurements for Vehicle to Vehicle (V2V) DSRC," *Application Note Rohde & Schwarz*, Vol. 1, 2009, pp. 1–25.

[16] IEEE, *Guide for Wireless Access in Vehicular Environments (WAVE)—Architecture*, IEEE Std 1609.0-2013, March 2014, pp. 1–78.

[17] Tonguz, O., N. Wisitpongphan, F. Bai, P. Mudalige, and V. Sadekar, "Broadcasting in VANET," *2007 Mobile Networking for Vehicular Environments, MOVE*, June 2007, pp. 7–12.

[18] Akhtar, N., S. C. Ergen, and O. Ozkasap, "Vehicle Mobility and Communication Channel Models for Realistic and Efficient Highway VANET Simulation," *IEEE Transactions on Vehicular Technology*, Vol. 64, No. 1, January 2015, pp. 248–262.

[19] Akhtar, N., O. Ozkasap, and S. C. Ergen, "VANET Topology Characteristics under Realistic Mobility and Channel Models," in *2013 IEEE Wireless Communications and Networking Conference (WCNC)*, April 2013, pp. 1774–1779.

[20] IEEE, *Standard for Wireless Access in Vehicular Environments (WAVE)–Multi-Channel Operation Corrigendum 1: Miscellaneous Corrections*, IEEE P1609.4-2010/Cor1/D4, August 2014, pp. 1–24.

[21] IEEE, *Draft Trial-Use Standard for Wireless Access in Vehicular Environments–Security Services for Applications and Management Messages*, IEEE Std P1609.2/D6, 2006.

[22] IEEE, *Approved Draft Standard for Wireless Access in Vehicular Environments (WAVE)–Networking Services*, IEEE P1609.3v3/D6, November 2015, pp. 1–162.

[23] IEEE, *Standard for Information Technology–Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks–Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, Technical Report, March 2012.

[24] Uzcátegui, R. A., A. J. De Sucre, and G. Acosta-Marum, "WAVE: A Tutorial," *IEEE Communications Magazine*, Vol. 47, No. 5, 2009, pp. 126–133.

[25] Bilstrup, K., E. Uhlemann, E. G. Strom, and U. Bilstrup, "Evaluation of the IEEE 802.11p MAC Method for Vehicle-to-Vehicle Communication," in *Vehicular Technology Con-*

ference, VTC2008-Fall, IEEE 68th, 2008, pp. 1–5.

[26] Li, B., M. S. Mirhashemi, X. Laurent, and J. Gao, "Wireless Access for Vehicular Environments," *Journal on Wireless Communications and Networking*, 2009: 576217. https://doi.org/10.1155/2009/576217.

[27] SAE International, *DSRC Implementation Guide: A Guide to Users of SAE J2735 Message Sets over DSRC*, February 2010.

[28] Bür, K., and M. Kihl, "Selective Broadcast for VANET Safety Applications," in *SNOW—the 2nd Nordic Workshop on System and Network Optimization for Wireless*, Salen, Sweden, 2011.

[29] RITA Intelligent Transportation Systems Joint Program Office, *Vehicle Information Exchange Needs for Mobility Applications*, February 2012 [online]. http://www.its.dot.gov/.

[30] ElBatt, T., S. K. Goel, G. Holland, H. Krishnan, and J. Parikh, "Cooperative Collision Warning Using Dedicated Short Range Wireless Communications," *Proceedings of the 3rd International Workshop on Vehicular Ad Hoc Networks—VANET '06*, p. 1, 2006 [online]. http://portal.acm.org/citation.cfm?doid=1161064.1161066.

[31] Yoon, Y., and H. Kim, "Resolving Distributed Power Control Anomaly in IEEE 802.11p WAVE," *IEICE Transactions on Communications*, Vol. E94-B, No. 1, 2011, pp. 290–292.

第 12 章 附录 A　更长的通信史

12.1 A.1　历史总览

本节的目的既不是鼓励死记人名日期、也不是让读者不知所措，而是帮助理解通信系统如何变迁、新概念的采纳可以多么漫长、以及我们每天使用的现代通信基础设施技术从何而来。有人可能怀疑本节是否必要——但正如卡尔·萨根的名言："如果你想做一块从零开始的苹果派，你必须先发明宇宙" [1]。对通信史了解越多，就越能体会当今通信是何等的现代奇迹。仅仅 260 年前，一本杂志上的一封匿名信还只是设想用 26 根导线（每个字母一根；编码方案尚未普及）路由实现即时通信：沿导线发送静电脉冲、在另一端观察木髓球的移动（那时伏打还未发明电池）。这是多个世纪通信革命的早期导火索。

不言而喻，通信系统受限于当时可用的传输介质。澳大利亚西部沙漠的原住民、北美的原住民、驻守长城的古代中国士兵、以及古希腊人都设计了基于火信号或烟信号的复杂消息传输系统。希腊历史学家波利比乌斯在公元前 150 年左右开发了早期的字母烟信号编码系统，把希腊字母转换为数字字符 [2]。这些方法与现代通信系统面临着同样的工程折中——包括传输距离与数据率。

有些通信方案至今仍在使用，例如图 A.1 的旗语（semaphore）——当需要在视线之外的隐身通信时（旗帜不发射射频、不会暴露位置）。

直到 1990 年代初，一切才开始汇聚：RF 能力、软件、算法与计算机硬件的独特组合才真正具备实现条件。事实上，"软件定义无线电" 一词直到 1993 年才被提出。

这段简史将略过约翰内斯·古腾堡约 1440 年发明的活字印刷机（它催生了整个印刷出版经济——到 1700 年，欧洲出版的各类书籍与学术期刊估计达 1.5 至 2 亿册 [3]）。这种与同时代及未来科学家、发明家、工程师共享思想概念的能力，帮助开启了持续的科学革命。

列奥纳多·达·芬奇这样的人不在本列表中。虽然他在解剖学、土木工程、地质学、



图 12.1　2005 年 11 月 29 日波斯湾，军需海员 Ryan Ruona 在尼米兹级航母西奥多·罗斯福号（CVN 71）海上补给期间用旗语发信号（美国海军照片）

光学与流体力学上做出了大量发现，但他没有发表其成果——未来的发明家、科学家与工程师没有正式的文档可供研究或拓展 [4]。

12.2 A.2　1750–1850：工业革命

多方认为 1750 年开启了工业革命（约 1750 至 1850 年）。这一转型给许多人带来变化：

- 手工生产让位于机械化；
- 化学与钢铁生产的新工艺；
- 蒸汽动力的广泛采用；
- 工厂制度的兴起；
- 人们从家庭作坊走向工厂就业。

该时期后期发展出的创新——机车、汽船与蒸汽轮船的采用、热风炼铁以及电报等新技术——持续改变着许多人的日常生活 [5]。

1748：莱昂哈德·欧拉出版《无穷分析引论》[6]。欧拉不仅是极重要、极有影响力的数学家之一，还是最高产的——一生写了 500 多本书籍与论文 [7]。

1752：本杰明·富兰克林在费城著名的风筝实验中证明闪电由电引起 [8]。

1753：《苏格兰杂志》上一位匿名作者提出静电电报：用多根导线，通过观察木髓球的偏转来传输消息 [9]。

1769：詹姆斯·瓦特发明可驱动飞轮的往复式蒸汽机 [10]。

1792：克劳德 · 沙普在法国大革命期间建成横跨法国的旗语网络，提供快速可靠的通信系统。它能在不到一小时内把消息传到 230 公里外——比当时任何通信方式都快 [11]。

1800：亚历山德罗 · 伏打发明伏打电堆（即今天的电池）。有了恒定直流电压源，更实用的实验与新装置的发明成为可能 [12]。

1804：理查德 · 特里维西克改进瓦特蒸汽机，造出第一台全尺寸实用铁路蒸汽机车 [13]。

1820：约翰 · 施魏格造出第一台灵敏检流计——检测与指示电流的机电设备 [14]。

1827：格奥尔格 · 欧姆出版《伽伐尼电路的数学研究》[15]，分享了电路基本定律 $V = I \times R$（后称欧姆定律）。当时欧姆的雇主科隆的三王中学并不欣赏他的工作，欧姆辞去了教职 [16]。

1827：安德烈-玛丽 · 安培出版《基于经验的电动力学现象的数学理论回忆录》[17]。正是这部著作把电动力学创建为一门学科。

1831：迈克尔 · 法拉第用运动的磁铁产生电流（盘形发电机，后来的发电机）[18]。

1833：卡尔 · 弗里德里希 · 高斯在哥廷根的屋顶上架设 1200 米长的导线做电报实验 [19]。

1837：查尔斯 · 巴贝奇设计机械分析引擎——第一台通用计算机 [20]。

1837：塞缪尔 · 摩尔斯获得记录式电报专利。为销售新设备，采用了称为摩尔斯电码的字母编码方案来通信字符串 [21]。电报简单易用，在 19 世纪大部分时间里被广泛使用，数据率取决于报务员的技能与设备能力。

12.3 A.3　1850–1945：技术革命

技术革命（或称第二次工业革命）以制造业的进步为标志，使既有的技术系统——电报与铁路网络、燃气与供水、污水系统——得以大规模普及。1870 年后铁路与电报线路的巨大扩张带来了前所未有的人员与思想流动。同期还引入了新的技术系统，最重要的是电力与电话。

1860：乔瓦尼 · 卡塞利演示其传真电报机（pantelegraph）——早期传真机，通过普通电报线路在 140 公里距离上传输签名图像 [22]。

1864：詹姆斯 · 麦克斯韦发表论文《电磁场的动力学理论》，告诉世界他关于电磁波及其背后数学的发现。这组 20 个方程后来被称为麦克斯韦方程组 [23]。

1865：巴黎国际电报大会上成立了国际电报联盟（ITU），为欧洲日益增长的国际电报业务制定规范并保证互操作性 [24]。

1869：第一条横贯大陆铁路完工。这条 3077 公里的连续铁路把美国东部铁路网与旧金山湾的太平洋海岸连接起来 [25]。

1873：麦克斯韦出版《电磁通论》[26]。许多人读过此书，包括柏林大学的赫尔曼·冯·亥姆霍兹。

1874：卡尔·布劳恩发明猫须检波器（后称点接触整流器）。它由一根轻触半导体矿物晶体（通常为方铅矿）的细线构成，是 20 世纪初到 1945 年早期矿石收音机的检波器 [27]。

1876：亚历山大·格拉汉姆·贝尔成功实现世界第一部电话，开启了基于语音的电子通信新时代 [28]。任何个人都可以与地球上任何其他个人通信——唯一约束是连接二人的有线介质。

1879：托马斯·爱迪生申请美国专利："碳灯丝或碳带卷绕……连接到铂接触导线"的电灯 [29]。

1879：博士生海因里希·赫兹接到导师亥姆霍兹的挑战——证明麦克斯韦方程（几年前才发表）的实际用途。赫兹做了些分析后认为太难，转而研究电磁感应 [30]。

1885：卡尔·本茨造出第一台实用的机动汽车 [31]。

1886：赫兹在卡尔斯鲁厄理工学院获得教授职位，终于能完成导师的挑战，建造了生成与检测无线电波的装置 [30]。这消除了通信对有线介质的需求（不再需要电缆）：发射机产生的无线电波会在区域中传播，直到被接收机截获。

1894：贾格迪什·博斯用猫须检波器在他的微波先驱实验中检测无线电波 [32]，并于 1901 年为方铅矿检波器申请专利 [33]。

1897：布劳恩造出第一支阴极射线管（CRT）与阴极射线示波器 [34]。

1901：伽利尔摩·马可尼演示了史上第一次跨大西洋无线传输——英格兰康沃尔的波尔杜到纽芬兰圣约翰斯的信号山 [35]，消息用摩尔斯电码编码 [36]。超远距离无线传输信息的可行性得到证明。这一实验的成功使无线数据传输成为连接全球人民的选项——不再需要任何有线通信基础设施。

1904：约翰·弗莱明发明振荡阀（后称二电极真空管整流器，即二极管）[37]。弗莱明二极管此后几十年用于收音机与雷达，作为环形混频器的一部分（见图 A.2）。

1905：布劳恩在领取 1909 年诺贝尔物理学奖时描述了他如何精心布置三根天线发射定向信号 [38]。这项发明导致了雷达、智能天线与 MIMO 的发展。

1905：雷金纳德·费森登描述了从新的摩尔斯电码连续波发射机产生可闻信号的方法——他称之为外差原理 [39]。这在当时还不是实用设备，因为振荡真空管尚未发明。

1906：第一届国际无线电报大会在柏林召开，国际无线电报联盟非正式成立 [40]。

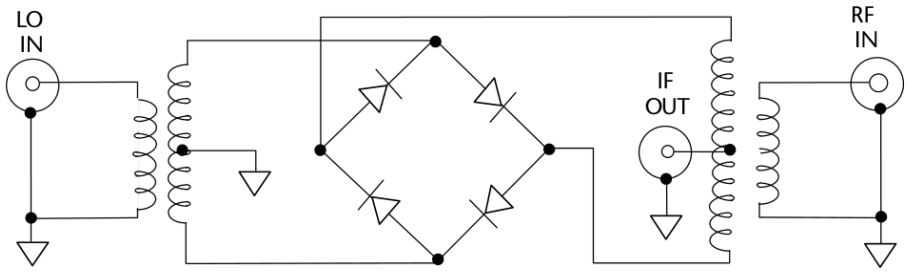


图 12.2　基于弗莱明振荡阀的二极管环形混频器 [45]

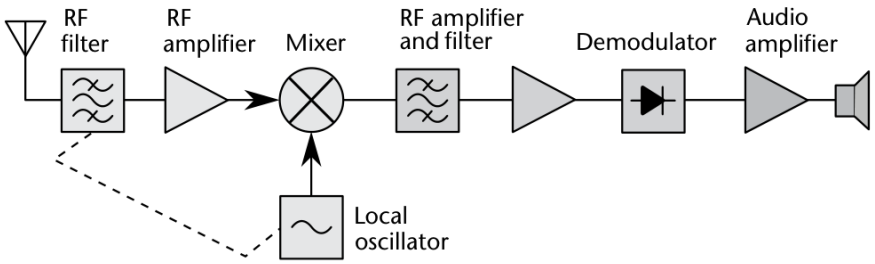


图 12.3　单变频超外差收音机。来自天线的入射信号经射频滤波器衰减部分无用信号、在射频放大器中放大、再与本地振荡器的未调制正弦波混频。结果是输入信号与本振频率之差（更低的频率）——称为中频 IF。中频信号由多级中频（带通滤波 + 放大）选择并增强，然后送到解调器提取已调音频信号，音频放大器进一步放大后由扬声器发声

1907：李 · 德福雷斯特发明栅极 аудion（grid audion）——第一台基于真空管的放大器、三极管的前身 [41]。

1912：亨利 · 福特开发并制造第一辆大众化汽车，使汽车成为实用交通工具 [42]。

1913：哥伦比亚大学本科生埃德温 · 阿姆斯特朗准备了一系列全面演示与论文，仔细记录他对德福雷斯特栅极 аудion 的研究——采用再生（后称正反馈）产生比先前认为可能高数百倍的放大 [43]。

1915：贝尔与托马斯 · 沃森通过纽约到旧金山之间 3400 公里的导线通话 [44]。

1917：埃里克 · 蒂格斯泰特为一部折叠口袋电话申请专利，配有极薄的碳麦克风——是我们今天所知手机的雏形 [46]。

1917：超外差获专利。三个人（吕西安 · 莱维、阿姆斯特朗、瓦尔特 · 肖特基）分别提交了三份不同的专利申请，究竟谁最先已不可知。美国专利局最终承认阿姆斯特朗与莱维为发明人 [47]。超外差是几乎所有现代收音机使用的设计。图 A.3 给出了单变频超外差收音机的框图——许多现代收音机的基础架构。

1924：AT&T 开发与研究部（后来的贝尔电话实验室）的工程师哈里 · 奈奎斯特研究电报信令，目标是找出给定带宽信道可用的最大信令速率，发表论文《影响电报速度的若干因素》[48]。

1927：查尔斯 · 杜卡斯获得电镀电路图案方法的专利——印制电路板制造的基础技术 [49]。

1928：贝尔实验室的哈罗德 · 布莱克获得放大器负反馈专利。他把负反馈定义为降低放大器增益的一类耦合，同时极大地提高了稳定性与带宽 [50]。

1928：奈奎斯特又在论文《电报传输理论中的若干课题》中发表了进一步成果 [51]。这两篇论文为数据传输的定量理解做出了基础性贡献。

1932：国际电报公约与国际无线电报公约在马德里合并为国际电信联盟（ITU）。新公约覆盖电报、电话与无线电三个通信领域 [40]。

1932：由于超外差架构通常多级实现且需要大型元件，一组英国科学家开发了他们称为 homodyne（后改名 synchrodyne，即今天的零中频架构）的收音机 [52]。虽然 homodyne 相比超外差有性能改善，但受限于当时的元件容差（这类电路要求元件偏差极小才能成功工作）以及对 PLL 的需求（超外差只需要一个振荡器），难以实现。

1940：RADAR 一词被美国海军用作 radio detection and ranging（无线电探测与测距）的缩写 [53]。

1945：科幻作家亚瑟 · 克拉克给《无线电世界》编辑的信中描述了对地静止卫星将是理想的电信中继 [54]。

12.4 A.4 1946–1960：喷气时代与太空时代

虽然喷气发动机的研究与专利从 1920 年代就开始了，但直到汉斯 · 冯 · 奥海因与马克斯 · 哈恩设计建造了第一台汽油燃料喷气发动机 HeS 3（推力 5 kN），装上简单紧凑的 He 178 机身，1939 年喷气时代才开始 [55]。商业航空的喷气时代始于 1952 年德 · 哈维兰彗星号（世界首架商用喷气客机）的首个定期航班 [56, 57]。当人们以每小时数百公里的速度飞驰空中时，对复杂通信的需求增长——知道自己的位置与他人的位置（避撞）变得更加关键。

1946：电子数值积分计算机（ENIAC）在宾夕法尼亚大学正式交付，成为最早的电子通用计算机之一 [58]。

1947：贝尔实验室的工程师观察到，当两个金触点接触锗晶体时，产生的信号输出功率大于输入——这个器件很快被称为晶体管。组长威廉 · 肖克利看到了其中的潜力，随后几个月大幅扩展了半导体知识 [59]。

1948：克劳德 · 香农在《贝尔系统技术期刊》发表《通信的数学理论》[60]。香农证明一切通信（语音、无线电、数据等）本质相同，且任何信源都可以用数字数据表示。他量化证明了通信信道存在速度极限（以比特每秒计）——这在当时是革命性的。香农用

数学证明必然存在逼近该极限而不丢失任何信息的编码——该证明至今仍然成立 [61]。

数字通信的概念在 1950 年代后期腾飞：香农证明了无论噪声、失真或信号幅度如何，完美信息传输都是可能的。中继站的概念变得重要——数据可以从源 A 传输到 B（中继）再到 C（目的地）而不损失信息。

1950：美国陆军获得 Auto-Sembly 工艺专利——浸焊电路组装工艺，被军方与工业界广泛采用 [62, 63]。

1950：肖克利出版《半导体中的电子与空穴》[64]，成为其他科学家开发改进晶体管新变种的参考书。

1954：Epsco 公司（由前 UNIVAC 工程师伯尼 · 戈登创立）发布了第一款商用模数转换器（ADC）——采用移位可编程逐次逼近架构并包含采样保持功能。它叫 Datrac：11 位、50 kSPS、150 磅。用真空管实现、耗散 500 瓦、设计为机架安装（19”×15”×26”）、售价 8000 至 9000 美元。它是第一台适合数字化交流波形（如语音）的商用 ADC [65]。受真空管技术所限，这些转换器昂贵、笨重且高功耗——几乎没有商业化用量。

1956：肖克利搬到加州山景城创办肖克利半导体实验室，开启了硅谷革命 [66]。

1957：由于肖克利的管理风格，八位不满的员工离开肖克利半导体实验室创办仙童半导体 [66]。这开启了硅谷延续至今的传统。

1957：苏联把” 基本卫星 1 号”（斯普特尼克 1 号）送入椭圆低地轨道 [67]。

1958：德州仪器（TI）的年轻工程师杰克 · 基尔比得出结论：电路元件的制造可以在一块半导体材料上完成。次年他申请了第一项集成电路专利——微型化电子电路 [68]。

1962：第一颗通信卫星 Telstar 1 发射 [69]。Telstar 1 在缅因州安德沃、英格兰西南部的贡希利与法国西北部的普勒默博杜之间中继数据、单电视频道或多路复用电话电路。由于其非对地静止轨道，跨大西洋信号每 2.5 小时中仅限 20 分钟（卫星经过大西洋上空时）[70]。

1965：两位 MIT 毕业生雷 · 斯塔塔与马修 · 洛伯创立 Analog Devices 公司 [71]。同年公司发布首款产品 101 型运放——曲棍球 puck 大小的模块，用于测试测量设备 [72]。

1965：仙童半导体研发总监戈登 · 摩尔观察到：密集集成电路中的元件数（晶体管、电阻、二极管或电容）大约每年翻一番，并推测这一趋势至少还会持续十年。后来他把预测修正为约每两年 [73]。这就是摩尔定律。

1966：ADC 没有享受到纯数字器件那样的规模缩放。Computer Labs（1978 年被 ADI 收购）当时卖 8 位 10 MSPS 转换器：机架安装、自带线性电源、耗散近 150 瓦、售价约 10000 美元 [65]。

1967：巴里 · 吉尔伯特发明基于晶体管的环形混频器——即吉尔伯特单元（四象限乘法器）[74]。在图 A.2 二极管环形混频器的基础上，把二极管替换为四个晶体管执行

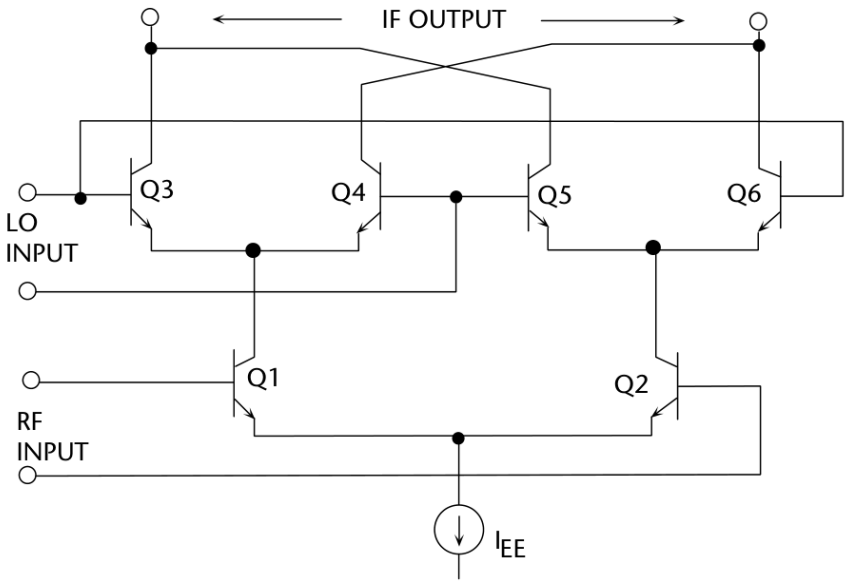


图 12.4　经典集成有源混频器 [45]

相同的开关功能。这构成了图 A.4 所示的经典双极性电路（全平衡版本的最简配置）。数以百万计的这种混频器被制造，如今包括 CMOS、BiCMOS 与 GaAs 变种。

吉尔伯特单元作为有源混频器有如下吸引力：

- 可与其他信号处理电路单片集成；
- 可提供变频增益（二极管环形混频器总是有插入损耗）；
- 驱动 LO 端口所需功率低得多；
- 信号端口间隔离度更好；
- 对负载匹配远不敏感，既不需要双工器也不需要宽带终端。

1967：UCLA 与 UCSD 电气工程教授安德鲁 · 维特比发表卷积编码数据的译码算法 [75]——即维特比算法。它至今仍广泛用于手机纠错码及其他众多应用。维特比没有为该算法申请专利——他不后悔这个决定 [76]。

1969：ARPANET（互联网的基础）上第一条成功消息由 UCLA 的学生程序员发出 [77]。

12.5 A.5　1970–1979：信息时代

信息时代的特征是从工业革命的传统产业转向基于信息计算机化的经济。1970 年代由大量新兴应用驱动：大批量计算器、高分辨率数字电压表、工业过程控制、数字视频、

军用相控阵雷达、医学成像、矢量扫描显示器与光栅扫描显示器。这些系统以前用的是（当时的）传统模拟信号处理技术或 ASIC，但低成本计算技术日益普及，激发了利用数字信号处理带来的更高性能与灵活性的渴望，以及对兼容数据转换器的需求。这个时代的核心是数字逻辑电路及其衍生技术（计算机与手机）的大规模生产与广泛应用。

1970：“数字接收机” 一词由美国国防部某实验室的研究者创造——那里创建了叫 Midas（多用户交互式数字分析系统）的软件基带分析工具，其操作由运行在当时大型机上的软件定义 [78]。

1971：英特尔公司发布 Intel 4004——第一款商用 4 位中央处理器（CPU）[79]。

1971：德州仪器发明并发布第一款微控制器（MCU）TMS1802NC [80]。

1971：ALOHAnet 用 UHF 无线分组网络连接夏威夷各岛。目标是用低成本商用无线设备把瓦胡岛与其他夏威夷岛屿的用户连接到瓦胡岛主校区的中央分时计算机 [81]。

1972：Signetics（后被飞利浦半导体收购，即现在的 NXP，写作本书时正在被高通收购）推出 NE555V 定时器。据 2003 年估计，每年制造 10 亿颗 [82]。

1972：语音频率的脉冲编码调制（PCM）——ITU-T 音频压扩标准 G.711 发布。它是窄带（300–3400 Hz）音频编解码器，提供与铜线上模拟传输（即普通老式电话业务 POTS）相当的语音质量 [83]。

1973：摩托罗拉的马丁 · 库珀在 DynaTAC 原型机上打了第一通公开的手持移动电话——重 4.4 磅（2 千克）。

1973：美国国防部启动国防导航卫星系统（DNSS）规划——很快更名 Navstar，最终成为全球定位系统（GPS）。1978 到 1985 年间发射了十颗原型卫星。

1975：保罗 · 艾伦与比尔 · 盖茨正式创立微软并授权 Altair BASIC [84]。

1975：ADI 发布 AD7570——单片器件上的 10 位 50 kHz CMOS SAR ADC。由于早期 CMOS 工艺难以设计好的比较器放大器与基准源，AD7570 需要外置比较器与外部电压基准 [65]。

1975：巴里 · 吉尔伯特加入 ADI 并发布 AD534 精密 IC 乘法器 [85]。这是第一款使用激光修整与吉尔伯特单元的器件，提供低于 ±0.25% 的四象限误差 [86]。

1975：史蒂文 · 赛森开发第一台数码相机 [87]。重 3.6 千克、用 100×100 像素（0.01 兆像素）传感器，生成的黑白数字图像记录到盒式磁带上，耗时 23 秒 [88]。

1976：史蒂夫 · 乔布斯、史蒂夫 · 沃兹尼亚克与罗纳德 · 韦恩创立苹果电脑公司 [89]。公司首款产品 Apple I 由沃兹尼亚克设计并手工打造 [90]。

1977：《Byte》杂志后来称之为个人计算 1977 三一体的 [91]：

- 苹果发布 Apple II（6502 处理器、1.023 MHz）
- Commodore 发布 PET（6502 处理器、1 MHz）

- Tandy 发布 TRS-80（Zilog Z80 处理器、1.77 MHz）

1978：EEPROM（电可擦除可编程只读存储器）由休斯微电子分部开发。相比 EPROM 有巨大改进：不再需要用紫外光照射存储器来擦除 [92]。

1978：德州仪器生产第一台 Speak & Spell——技术核心是 TMS5100：业界第一款数字信号处理器（DSP）。它还创下其他里程碑：第一颗用线性预测编码做语音合成的芯片 [93]。

1978：ADI 发布 AD574——有史以来最重要的 SAR ADC [94]。它是第一款完整解决方案的器件：内置齐纳基准、定时电路与三态输出缓冲器，可直接接口 8/12/16 位微控制器或微处理器总线 [65]。它成为当时的业界标准 12 位 ADC，但基于同封装两片裸片。

1978：圣地亚哥州立大学电气工程教授弗雷德·哈里斯发表他引用最高的论文《关于离散傅里叶变换谐波分析中窗的使用》[95]——他发表的论文与分享的信息获得超过 10000 次同行引用 [96]。哈里斯博士从 1962 年开始发表论文，至今仍在继续。

12.6 A.6 1980–1989：数字革命

数字革命是从机械与模拟电子技术向数字电子技术的持续变革——依据细分市场不同，始于 1950 年代末到 1970 年代末，伴随数字信号处理与桌面计算的采用，延续至今。

1981：IBM 个人电脑问世——4.77 MHz Intel 8088、16 KB 内存、彩色图形适配器、无磁盘驱动器 [97]。

1982：互联网协议套件（TCP/IP）由美国国防部标准化——允许互联网络在全球扩散 [98]。

1983：摩托罗拉 DynaTAC 8000X 商用便携蜂窝电话发布。充满电约需 10 小时、通话 30 分钟。1984 年定价 3995 美元，使用称为高级移动电话系统（AMPS）的模拟调制方案。

1983：约翰·普罗基斯出版他关于数字通信与信号处理的十本书中的第一本，包括《数字通信》（McGraw Hill）[99, 100]。他继而写了该学科的许多书：《数字信号处理》[101]、《数字信号处理实验室》[101]、《高级数字信号处理》[101]、《语音信号的数字处理》[103] 与《通信系统工程》[104]。他的书教育了一代学生与工程师 SDR 相关的基础与理论。

1983：Altera 创立并于 1984 年交付业界第一款可重编程逻辑器件 EP300——封装上有石英窗，用户需用紫外灯照射裸片擦除保存器件配置的 EPROM 单元 [105]。

1983：罗纳德·里根总统发布指令：GPS 一旦充分发展，将作为公共品向民用免费

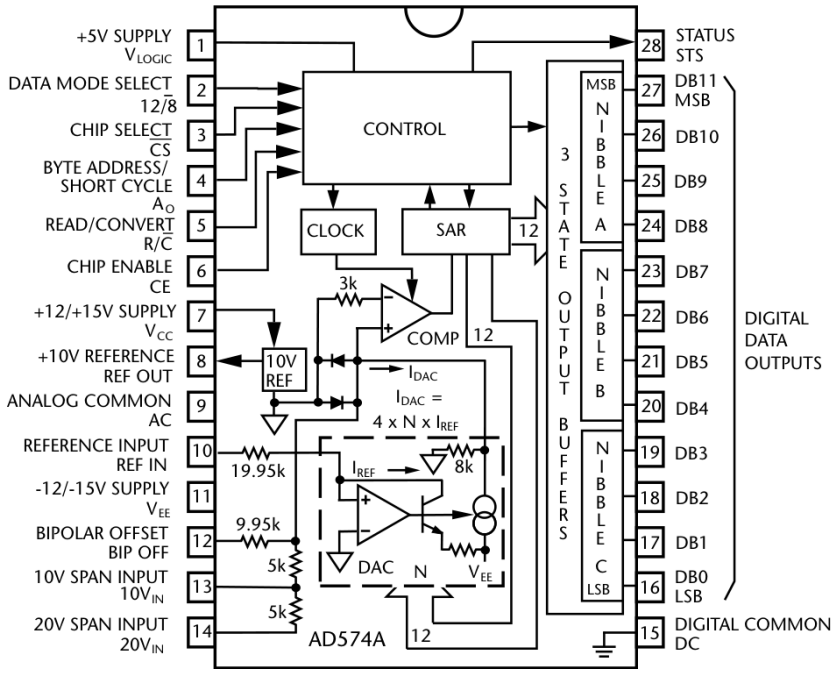


图 12.5　业界标准 AD574 12 位 28.5 kSPS ADC

开放。第一颗生产卫星于 1989 年 2 月 14 日发射，第 24 颗于 1994 年发射。

1983：欧洲开始制定数字蜂窝语音电信标准——最终成为 GSM。

1984：苹果电脑推出第一台 Macintosh。它是第一台商用成功的图形用户界面（GUI）+ 鼠标的个人电脑，这些成为电脑的标准配置 [106]。配置 7.83 MHz 32 位摩托罗拉 68000 CPU、内置 9 英寸单色屏、512×342 图形、400 kB 3.5 英寸索尼软驱、鼠标、128 KB 内存。重 20 磅，尺寸 9.7×10.9×13.5 英寸，售价 1995 至 2495 美元。

1984：“软件无线电”一词由 E-Systems 公司（今雷神）德州加兰分部的一个团队创造，指数字基带接收机——团队在公司 E-Team 通讯上发布 [107]。

1984：MathWorks 创立，MATLAB 在内华达州拉斯维加斯的 IEEE 控制与决策会议上公开亮相 [108]。用于工业与教育（特别是线性代数与数值分析教学），深受信号处理领域的科学家欢迎。

1984：ITU 发布拨号调制解调器 V.32 建议——使用 QAM 调制、符号率 2400 波特、允许 9.6 kbit/s 双向数据传输 [109]。

1985：ADI 以单芯片单片形式重新发布 AD574（见图 A.5）。低价商用塑封转换器首次能够大量供应。

1985：美国联邦通信委员会裁定将 902–928、2400–2483.5 与 5725–5850 MHz 频段开

放为免许可使用 [110]。这创造了工业、科学与医疗（ISM）无线电频段。

1987：来自 13 个欧洲国家的 15 位代表在哥本哈根签署谅解备忘录，开发部署全欧洲统一的蜂窝电话系统，欧盟通过法规使 GSM 成为强制标准。

1987：东芝开发闪速存储器（NOR 与 NAND 型），并在旧金山举行的 IEEE 1984 国际电子器件会议（IEDM）上发布 [111]。

1987：1980 至 1987 年间，超过 150 亿美元的风险投资投入半导体初创企业 [112]。半导体产业持续增长——初创公司如 Linear Technology（1987 年创立、2016 年被 ADI 收购）、Hittite Microwave（被 ADI 收购）、Wolfson Microelectronics（被 Cirrus Logic 收购）、Atmel（1984 年创立、2016 年被 Microchip 收购）、Xilinx（1984 年创立）、Altera（1983 年创立、2015 年被英特尔收购）、Maxim（1983）与 Micron（1978）。

1988：ITU-T 批准 G.722——宽带（50–7000 Hz）音频编解码器，工作在 48/56/64 kbit/s。该编解码器技术基于子带 ADPCM（SB-ADPCM）。凭借比 16 年前 G.711 更宽的带宽，G.722 提供了更好的语音质量 [113]。

1988：Crystal Semiconductor（后被 Cirrus Logic 收购）发布单片 CSZ5316——第一款商用 Σ-Δ ADC。16 位分辨率、20 kSPS 有效吞吐率，适合语音频带数字化 [65]。

1989：爱立信移动的 CTO 尼尔斯 · 吕德贝克提出短距离无线技术连接无线耳机与手机——最终成为蓝牙。

1989：互联网开始公众商用——MCI Mail 与 CompuServe 的电子邮件业务接入 [114]。

12.7 A.7 1990–1999：公共互联网时代（Web 1.0）

数字信号处理与计算技术继续给 1990 年代带来革命。1990 到 1997 年间，美国个人 PC 拥有率从 15% 升至 35%。1990 年代早期及以前的手机非常庞大、缺乏附加功能、只有百分之几的人口使用。1990 年只有几百万人使用在线服务，万维网才刚刚发明。第一个网页浏览器 1993 年上线；到 2001 年，部分西方国家超过 50% 的人有互联网接入、超过 25% 的人有手机。调制解调器、ISDN、线缆调制解调器与 DSL 的进步带来了更快的互联网连接。

无线传输技术随时间逐渐演进：1917 年开发的超外差架构（图 A.3）仍是当时的首选无线架构。它利用当代集成电路，演化为多级接收机架构——通常一至两级混频、后接单片 ADC 与 DAC。多级超外差收发架构见图 A.6。第一变频级把射频（上/下）变换到较低的中频 IF1；IF1 再被搬移到更低的 IF2/基带频率供 ADC 或 DAC 数字化。除混频器外，每级还有滤波器、放大器与步进衰减器。滤波用于抑制带外（OOB）信号——不加抑制的话，OOB 信号会落在期望信号之上产生杂散（spur），使解调困难或不可能。

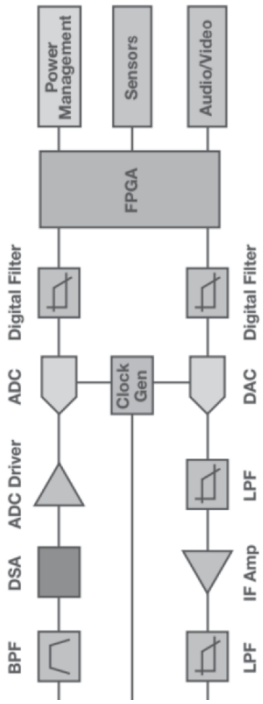


图 12.6　多级超外差接收与发射信号链 [118]

实际的 IF1/IF2 频率取决于频率与杂散规划以及混频器性能和射频前端可用滤波器。这些无线电台多为固定频率（不太可调）——性能（灵敏度）的追求以牺牲灵活性为代价。

1990：Advanced RISC Machines 公司（ARM）作为 Acorn 电脑、苹果与 VLSI Technology 的合资企业成立。公司开发的 ARM 处理器如今用于每年售出的超过 10 亿部手机中的 98% [115]。

1991：MPEG-1 Audio Layer I/II/III 算法（即国际标准 ISO/IEC 11172-3，或称 mp3）1991 年获批、1992 年定稿 [116, 117]。

1991：ITU 发布拨号调制解调器 V.32bis 建议——用维特比算法做网格调制，允许 14.4 kbit/s 双向数据传输 [109]。

1991：计算机专业学生林纳斯 · 托瓦兹开始开发一个操作系统的简单构想——10239 行代码的 Linux 内核 0.01 版发布在 FTP 服务器上 [119]。

1991：芬兰前总理哈里 · 霍尔凯里打出世界第一个 GSM 电话，打给卡里娜 · 苏奥尼奥（坦佩雷市长——诺基亚的发源地）[120]。

1991：美国军方启动软件定义无线电项目 SpeakEasy I [121]。虽然 SDR 技术的不同部分自 1970 年代就已可用、第一个作为 SDR 原型（固定射频实现）的演示在 1988 年展示 [122]，但 SpeakEasy 是第一个大规模软件无线电。

当一台通信设备为唯一目的设计——只用一种波形、在一个频率上——不同人制造的不同设备彼此无法互操作。美军正是有这样的问题：每个军种与部门都有许多不同类型的电台与波形，冲突时期无法实时互通。这不是学术练习——生命因无法通信而逝去。他们需要一个能与十多种不同军用电台通信的系统。

这是 SDR 技术进展的关键里程碑。开发者想测试多频段（不同射频频率）、多模式（不同波形）无线电台在实际环境中的可行性。

1991：欧洲粒子研究组织 CERN 宣传新的万维网项目。

1992：第一条短消息服务（SMS 或短信）在 GSM 网络上发送。

1993：约瑟夫 · 米托拉创造” 软件定义无线电” 一词 [123]。

1994：SpeakEasy 项目成功演示。它不仅是成功，更是 SDR 技术的重大飞跃。它用了四片 40 MHz 的德州仪器 TMS320C40 处理器与一台 SUN Sparc 10 工作站做人机接口，实现了十多种军用通信标准，载波频率 2 至 2000 MHz——这在当时是通信系统工程的一大进展。SpeakEasy 实现允许新功能块（如调制方案与编码方案）的软件升级。注意，以当时的微处理器技术，SpeakEasy 原型体积大到要装在卡车背面、需要相当大的功率。

由于 SpeakEasy 项目花了 3 年完成（两轮摩尔定律），许多人以为缩小到更快、更低功耗的处理器会很容易。但由于一切都是手写汇编（为最大化性能），这做不到。开发陷入停滞。观察结论是：为一个平台写软件花了 3 年，而摩尔定律让该平台在 18 个月内就过时了。

1994：林纳斯 · 托瓦兹发布 Linux 内核 1.0——176,250 行代码 [124]。

1995：索尼在全球发布 PlayStation。它是第一个出货 1 亿台的计算机娱乐平台——在首发 9 年 6 个月后达到 [125]。

1996：约翰 · 奥沙利文把他 1977 年开发的射电天文图像清晰化技术（用傅里叶方法）重新用于减少无线信号的多径干扰——这原本是探测原子粒子大小的爆炸微型黑洞的研究项目的一部分。虽然他的主要实验失败了，但他的信号处理技术获得了专利并应用于后来的 802.11 标准。

1996：林纳斯 · 托瓦兹发布 Linux 内核 2.0——632,062 行代码 [126]。

1997：802.11 协议第一版发布，提供最高 2 Mbit/s 链路速率。

1998：蓝牙特别兴趣组（SIG）成立并发布第一版规范。

1998：ITU 发布拨号调制解调器 V.90 建议——数字下载 56 kbit/s、模拟上传 33.6

kbit/s [127]。

1999：IEEE 批准 802.11b 升级规范，允许 11 Mbit/s 链路速率。

2000：蓝牙产品开始出货，包括鼠标、PC 卡、耳机与手机 [128]。

2001：吉米 · 威尔士与拉里 · 桑格推出维基百科 [129]。

12.8 A.8　2000 之后：万物汇聚

2000 年以来，研发活动量爆炸式增长。新的、更快的或更低功耗的通信方案几乎每年都在宣布。2008 年全球 GSM 用户超过 30 亿；而 2016 年，运营商正在退役网络为 LTE 与其他标准让路。变化的速度已大幅加快。

12.9 参考文献

[1] Sagan, C., *Cosmos*, New York: Random House, 1980.

[2] Polybius, *The Histories of Polybius*, published in Vol. IV of the Loeb Classical Library edition, 1922 through 1927. http://penelope.uchicago.edu/Thayer/E/Roman/Texts/Polybius/10*.html#45.6.

[3] Febvre, L., and H.-J. Martin, *The Coming of the Book: The Impact of Printing 1450–1800*, London: New Left Books, 1976. Quoted in Anderson, B., *Comunidades Imaginadas*, Fondo de Cultura Económica, Mexico, 1993, p. 58f.

[4] Capra, F., *The Science of Leonardo: Inside the Mind of the Genius of the Renaissance*, New York: Doubleday, 2007.

[5] Landes, D. S., *The Unbound Prometheus*, Cambridge, UK: Press Syndicate of the University of Cambridge, 1969.

[6] Euler, L., E101–*Introductio in Analysin Infinitorum*, Volume 1, 1748, The Euler Archive. http://eulerarchive.maa.org/pages/E101.html.

[7] Wilson, R., “Read Euler, Read Euler, He Is the Master of Us All,” March 1, 2007. https://plus.maths.org/content/os/issue42/features/wilson/index.

[8] Franklin, B., “The Kite Experiment,” *The Pennsylvania Gazette*, October 19, 1752, in *The Papers of Benjamin Franklin*, The American Philosophical Society and Yale University, digital edition by The Packard Humanities Institute, Vol. 4, p. 360a.

[9] Marland, E. A., *Early Electrical Communication*, London: Abelard-Schuman Ltd., 1964, pp. 17–19.

[10] Carnegie, A., *James Watt*, The Minerva Group, Inc., p. 215. http://www.jameswatt.info/andrew-carnegie/3-captured-by-steam.html.

[11] Grandstrand, O., "Semaphore Signalling," in R. W. Burns (ed.), *Communications: An International History of the Formative Years*, Herts, UK: The Institution of Engineering and Technology, 2004.

[12] Decker, F., "Volta and the 'Pile,' " *Electrochemistry Encyclopedia*, Case Western Reserve University, January 2005.

[13] Amgueddfa Cymru National Museum Wales, "Richard Trevithick's Steam Locomotive," 2008. https://museum.wales/articles/2008-12-15/Richard-Trevithicks-steam-locomotive/.

[14] Wickens, A. P., *A History of the Brain: From Stone Age Surgery to Modern Neuroscience*, London: Psychology Press, 2014.

[15] Ohm, G., *Die Galvanische Kette, Mathematisch Bearbeitet*. http://www2.ohm-hochschule.de/bib/textarchiv/Ohm.Die_galvanische_Kette.pdf.

[16] Chisholm, H. (ed.), "Ohm, Georg Simon," *Encyclopedia Britannica*, 20 (11th Edition), Cambridge, UK: Cambridge University Press, 1911, p. 34.

[17] Ampère, A.-M., *Mémoire sur la Théorie Mathématique des Phénomènes Électrodynamiques Uniquement Déduite de l'Expérience*. http://www.ampere.cnrs.fr/textes/theoriemathematique/pdf/theorie_mathematique.pdf.

[18] Alglave, M., and J. Boulard, *The Electric Light: Its History, Production, and Applications*, translated by T. O'Conor Sloan, New York: D. Appleton & Co., 1884, p. 224.

[19] Vail, A., *The American Electro Magnetic Telegraph*, 1847, Lea & Blanchard.

[20] Graham-Cumming, J., "The 100-Year Leap" (2010-10-04), O'Reilly Radar. http://radar.oreilly.com/2010/10/the-100-year-leap.html.

[21] Burns, R. W., *Communications: An International History of the Formative Years*, London: The Institution of Engineering and Technology, 2004.

[22] Huurdeman, A. A., *The Worldwide History of Telecommunications*, John Wiley & Sons, 2003.

[23] Maxwell, J., "A Dynamical Theory of the Electromagnetic Field," *Philosophical Transactions of the Royal Society of London*, 1865.

[24] International Telecommunications Union, "ITU History," 1993. http://www.itu.int/itudoc/about/itu/history/history.txt.

[25] Vernon, E. (ed.), *Travelers' Official Railway Guide of the United States and Canada*, Philadelphia: The National General Ticket Agents' Association, June, 1870, Tables 215, 216.

[26] Maxwell, J., *A Treatise on Electricity and Magnetism*, 1873. http://www.aproged.pt/biblioteca/MaxwellI.pdf.

[27] Braun, E., and S. MacDonald, *Revolution in Miniature: The History and Impact of Semiconductor Electronics*, Second Edition, Cambridge, UK: Cambridge University Press, 1982, pp. 11–12.

[28] Bell, A. G., *Improvement in Telegraphy*, U.S. Patent No. 174465, issued March 7, 1876.

[29] Levy, J., *Really Useful: The Origins of Everyday Things*, New York: Firefly Books, 2003, p. 124.

[30] Heilbron, J. L. (ed.), *The Oxford Guide to the History of Physics and Astronomy*, Oxford, UK: Oxford University Press, 2005, p. 148.

[31] Daimler AG, "Company History, Benz Patent Motor Car: The First Automobile (1885–1886)." https://www.daimler.com/company/tradition/company-history/1885-1886.html.

[32] Emerson, D. T., "The Work of Jagadish Chandra Bose: 100 Years of mm Wave Research," *IEEE Transactions on Microwave Theory and Techniques*, Vol. 45, No. 12, December 1997, pp. 2267–2273.

[33] Bose, J. C., *Detector for Electrical Disturbances*, U.S. Patent No. 755840, published September 30, 1901, issued March 29, 1904.

[34] "Karl Ferdinand Braun," *Encyclopædia Britannica*. https://www.britannica.com/biography/Karl-Ferdinand-Braun.

[35] "Guglielmo Marconi," *Encyclopædia Britannica*. https://www.britannica.com/biography/Guglielmo-Marconi.

[36] "Marconi's Transatlantic Signal," *Scientific American*. https://www.scientificamerican.com/article/marconis-transatlantic-signal/.

[37] Fleming, J. A., *Instrument for Converting Alternating Electric Currents into Continuous Currents*, U.S. Patent No. 803,684, issued November 7, 1905.

[38] Braun, K. F., "Electrical Oscillations and Wireless Telegraphy," Nobel Lecture, December 11, 1909. https://www.nobelprize.org/prizes/physics/1909/braun/lecture/.

[39] Fessenden, R. A., *Wireless Telegraphy*, U.S. Patent No. 1050441, issued January 14, 1913 (filed 1905).

[40] International Telecommunications Union, "ITU History: 1947–1992." http://www.itu.int/en/history/Pages/ITU-historical-timeline.aspx.

[41] "Lee de Forest," *Encyclopædia Britannica*. https://www.britannica.com/biography/Lee-de-Forest.

[42] "Henry Ford," *Encyclopædia Britannica*. https://www.britannica.com/biography/Henry-Ford.

[43] "Edwin Howard Armstrong," *Encyclopædia Britannica*. https://www.britannica.com/biography/Edwin-Howard-Armstrong.

[44] "First Telephone Call to San Francisco," AT&T. https://www.att.com/About/company-history/telephones.html.

[45] Analog Devices, "Mixers," in *The Linear Data Conversion Handbook*. http://www.analog.com/media/en/training-seminars/design-handbooks/Linear-Data-Conversion-Handbook/Chapter3-4.pdf.

[46] Eric Tigerstedt, "Kiinnityslaitos kuvien välittämiseen sähkölennätintä pitkin" ("Device for transmitting images over telegraph wires"), Finnish patent #3209, filed October 30, 1917.

[47] "Lev, Lucien," *Radio Hall of Fame*. http://www.radiohof.org/radiohistory/lev.html.

[48] Nyquist, H., "Certain Factors Affecting Telegraph Speed," *Bell System Technical Journal*, Vol. 3, April 1924, p. 324.

[49] Ducas, C., *Process of Electrically Plating Circuit Patterns*, U.S. Patent No. 1,699,147, issued January 15, 1929 (filed 1927).

[50] Black, H. S., *Wave Translation System*, U.S. Patent No. 1,686,792, issued October 9, 1928 (filed 1925).

[51] Nyquist, H., "Certain Topics in Telegraph Transmission Theory," *A.I.E.E. Transactions*, Vol. 47, April 1928, p. 617.

[52] "Synchrodyne," *Radio Museum*. http://www.radiomuseum.org/forum/synchrodyne.html.

[53] "Radar," *Encyclopædia Britannica*. https://www.britannica.com/technology/radar.

[54] Clarke, A. C., "Extra-Terrestrial Relays: Can Rocket Stations Give World-Wide Radio Coverage?" *Wireless World*, October 1945.

[55] “Heinkel He 178,” *Encyclopædia Britannica*. https://www.britannica.com/technology/Heinkel-He-178.

[56] “De Havilland Comet,” *Encyclopædia Britannica*. https://www.britannica.com/technology/de-Havilland-Comet.

[57] “The Jet Age,” *U.S. Centennial of Flight Commission*. https://www.centennialofflight.net/essay/Evolution_of_Flight/jet_age/Ev26.htm.

[58] “ENIAC,” *Encyclopædia Britannica*. https://www.britannica.com/technology/ENIAC.

[59] “Transistor,” *Encyclopædia Britannica*. https://www.britannica.com/technology/transistor.

[60] Shannon, C. E., “A Mathematical Theory of Communication,” *Bell System Technical Journal*, Vol. 27, July and October 1948.

[61] “Claude Shannon,” *Encyclopædia Britannica*. https://www.britannica.com/biography/Claude-Shannon.

[62] U.S. Army, *Auto-Sembly Process*, U.S. Patent No. 2,565,533, issued August 28, 1951 (filed 1950).

[63] “The Auto-Sembly Process,” *Electronic Industries*. http://www.edn.com/Pub/20/03/27.

[64] Shockley, W., *Electrons and Holes in Semiconductors*, D. Van Nostrand Company, 1950.

[65] Kester, W., *Data Conversion Handbook*, Analog Devices, 2005. http://www.analog.com/en/education/education-library/data-conversion-handbook.html.

[66] “Shockley Semiconductor Laboratory,” *The Tech. Silicon Valley Historical Association*. https://www.computerhistory.org/siliconengine/shockley-semiconductor-laboratory/.

[67] “Sputnik,” *NASA*. https://nssdc.gsfc.nasa.gov/nmc/spacecraftDisplay.do?id=1957-001B.

[68] “Jack Kilby,” *Encyclopædia Britannica*. https://www.britannica.com/biography/Jack-Kilby.

[69] “Telstar,” *NASA*. https://nssdc.gsfc.nasa.gov/nmc/spacecraftDisplay.do?id=1962-029A.

[70] “The Story of Telstar,” *Space Daily*. http://www.spacedaily.com/news/telstar-62a.html.

[71] “Analog Devices History,” Analog Devices. http://www.analog.com/en/about-analog-devices/history.html.

[72] “Model 101 Op Amp,” Analog Devices. http://www.analog.com/media/en/news-marketing-collateral/product-highlight/AD101KP-Highlight.pdf.

[73] Moore, G. E., “Cramming More Components onto Integrated Circuits,” *Electronics*, Vol. 38, No. 8, April 19, 1965.

[74] Gilbert, B., “A Precise Four-Quadrant Multiplier with Subnanosecond Response,” *IEEE Journal of Solid-State Circuits*, Vol. SC-3, No. 4, December 1968.

[75] Viterbi, A. J., “Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm,” *IEEE Transactions on Information Theory*, Vol. 13, No. 2, April 1967, pp. 260–269.

[76] “Andrew Viterbi,” *Encyclopædia Britannica*. https://www.britannica.com/biography/Andrew-Viterbi.

[77] “The First ARPANET Message,” *Internet Society*. http://www.internetsociety.org/internet/what-internet/history-internet/brief-history-internet.

[78] “The History of Software Defined Radio,” *Wireless Innovation Forum*. http://www.wirelessinnovation.org/History_of_SDR.

[79] “Intel 4004,” *Intel Corporation*. https://www.intel.com/content/www/us/en/history/museum-story-of-intel-4004.html.

[80] “TMS1802NC,” Texas Instruments. http://www.ti.com/lsds/ti/dsp/processors.page.

[81] “ALOHAnet,” *University of Hawaii*. http://hprc.hawaii.edu/hawnaloha/aloha-history.

[82] “555 Timer IC,” *Electronics Tutorials*. http://www.electronics-tutorials.ws/waveforms/555_timer.html.

[83] ITU-T, *G.711: Pulse Code Modulation (PCM) of Voice Frequencies*, 1972.

[84] “Microsoft Company History,” *Microsoft*. https://www.microsoft.com/about/companyinformation/history.

[85] Gilbert, B., “A New Wide-Band Analogue Technique,” *IEEE Journal of Solid-State Circuits*, Vol. SC-3, No. 4, December 1968.

[86] “AD534 Datasheet,” Analog Devices. http://www.analog.com/media/en/technical-documentation/data-sheets/AD534.pdf.

[87] Sasson, S. A., *Electronic Still Camera*, U.S. Patent No. 4,131,919, issued December 26, 1978 (filed 1977).

[88] “Steven Sasson,” *The Tech*. https://www.computerhistory.org/siliconeng ine/steven-sasson.

[89] “Apple Computer, Inc. History,” *Apple Inc.* https://www.apple.com/about/.

[90] “Apple I,” *Computer History Museum*. https://www.computerhistory.org/co llections/search/object/102702798.

[91] “The 1977 Trinity,” *Byte Magazine*. http://www.byte.com.

[92] “EEPROM,” *Electronic Design*. http://www.electronicdesign.com.

[93] “Speak & Spell,” *The Strong Museum of Play*. http://www.museumofplay.org.

[94] “AD574 Datasheet,” Analog Devices. http://www.analog.com/media/en/tech nical-documentation/data-sheets/AD574.pdf.

[95] Harris, F. J., “On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform,” *Proceedings of the IEEE*, Vol. 66, No. 1, January 1978.

[96] Fred Harris, “Google Scholar Profile.” https://scholar.google.com/citatio ns?user=Mte7H5cAAAAJ.

[97] “IBM Personal Computer,” *IBM Archives*. https://www-03.ibm.com/ibm/hist ory/exhibits/pc25/pc25_intro.html.

[98] “Internet Protocol Suite,” *Wikipedia*. https://en.wikipedia.org/wiki/Intern et_protocol_suite.

[99] “John G. Proakis,” *Northeastern University*. https://www.northeastern.edu.

[100] Proakis, J. G., *Digital Communications*, McGraw Hill, 1983.

[101] Proakis, J. G., and D. G. Manolakis, *Digital Signal Processing*, Prentice Hall, 1996.

[102] Proakis, J. G., *Digital Signal Processing Laboratory*, Prentice Hall.

[103] Proakis, J. G., and D. G. Manolakis, *Digital Processing of Speech Signals*, Prentice Hall.

[104] Proakis, J. G., and M. Salehi, *Communication Systems Engineering*, Prentice Hall.

[105] “Altera Corporation History,” *Altera*. https://www.altera.com.

[106] “Macintosh,” *Computer History Museum*. https://www.computerhistory.org.

[107] “The History of Software Defined Radio,” *Wireless Innovation Forum*. http://ww w.wirelessinnovation.org/History_of_SDR.

[108] “MATLAB’s Origins,” *MathWorks*. https://www.mathworks.com/company/ne wsletters/articles/the-origins-of-matlab.html.

[109] “ITU-T V-Series Recommendations,” *ITU*. http://www.itu.int/ITU-T.

[110] FCC, *Title 47 CFR Part 15: Low Power Communication Devices*, 1985.

[111] “Flash Memory,” *Toshiba*. https://www.toshiba.com.

[112] “Semiconductor Industry in the 1980s,” *Semiconductor Industry Association*. http://www.semiconductors.org.

[113] ITU-T, *G.722: 7 kHz Audio-Coding within 64 kbit/s*, 1988.

[114] “Brief History of the Internet,” *Internet Society*. http://www.internetsociety.org.

[115] “ARM Holdings,” *Encyclopædia Britannica*. https://www.britannica.com/topic/ARM-Holdings.

[116] ISO/IEC 11172-3:1993, *Coding of Moving Pictures and Associated Audio for Digital Storage Media at up to About 1,5 Mbit/s — Part 3: Audio*.

[117] “MPEG-1 Audio Layer III,” *Wikipedia*. https://en.wikipedia.org/wiki/MP3.

[118] Hall, B., and W. Taylor, “X- and Ku-Band Small Form Factor Radio Design.” http://www.analog.com/en/technical-articles/x-and-ku-band-small-form-factor-radio-design.html.

[119] “Linux Kernel Release History,” *The Linux Kernel Archives*. https://www.kernel.org.

[120] “History of GSM,” *GSM Association*. http://www.gsma.com/aboutus/history.

[121] “SpeakEasy,” *Wireless Innovation Forum*. http://www.sdrforum.org.

[122] “The History of Software Defined Radio,” *Wireless Innovation Forum*. http://www.wirelessinnovation.org/History_of_SDR.

[123] Mitola, J., “Software Radios: Survey, Critical Evaluation and Future Directions,” *IEEE Communications Magazine*, Vol. 33, No. 5, May 1995.

[124] “Linux Kernel Release History,” *The Linux Kernel Archives*. https://www.kernel.org.

[125] “PlayStation,” *Sony Computer Entertainment*. https://www.playstation.com.

[126] “Linux Kernel Release History,” *The Linux Kernel Archives*. https://www.kernel.org.

[127] ITU-T, *V.90: Digital Transmission and Functional Characteristics for a 56 kbit/s Modem*, 1998.

[128] “Bluetooth History,” *Bluetooth SIG*. https://www.bluetooth.com.

[129] “Wikipedia,” *Wikipedia*. https://en.wikipedia.org.

第 13 章 附录 B　MATLAB 与 Simulink 入门

本书的实验与开放式设计项目将使用 MATLAB 与 Simulink。本附录作为 MATLAB 的简要复习（你应当用过它）；若你对 Simulink 没有太多经验，本附录将带你入门。请注意：MATLAB 部分主要基于 [1] 的 MATLAB 文档、Simulink 部分基于 [2] 的 Simulink 基础教程——这里提取最重要与最根本的概念，让你读完后快速上手。两款产品的更多信息请查阅 [1] 与 [2]。

13.1 B.1　MATLAB 简介

MATLAB 广泛应用于应用数学的各个领域、大学的教研以及工业界。MATLAB 代表 Matrix Laboratory（矩阵实验室），软件围绕向量与矩阵构建。因此它特别适合解线性代数问题，也适合解代数方程、微分方程与数值积分。MATLAB 拥有一组图形工具，能生成高级 GUI 与 2-D/3-D 数据图。MATLAB 还有若干工具箱，用于通信、信号处理、图像处理、优化及其他专门操作。

MathWorks 创建了优秀的在线教程，复习基础与高级概念，并提供讲师引导的教程展示 MATLAB 的各种能力，见 `https://matlabacademy.mathworks.com`。

13.2 B.2　有用的 MATLAB 工具

本节介绍寻找错误的通用技术，以及用自动代码分析功能检测 MATLAB 代码中可能需要改进的地方。特别是编辑器中的 MATLAB 调试器功能与等效的命令窗口调试函数。

调试是隔离并修复代码问题的过程。调试帮助纠正两类错误：

- **语法错误**：例如函数名拼错或漏掉括号。
- **运行时错误**：这类错误通常是算法性的。例如你可能修改了错误的变量或把计算写错了。运行时错误通常在 M 文件产生意外结果时显现。运行时错误难以追踪，因为出错强制返回 MATLAB 基工作区时，函数的局部工作区会丢失。

13.2.1 B.2.1　代码分析与 M-Lint 消息

MATLAB 可以检查代码问题并通过消息（有时称为 M-Lint 消息）推荐修改以最大化性能与可维护性。lint 一词来自其他编程语言（如 C）中类似工具的名字。MATLAB 的 M-Lint 工具对它判定有改进潜力的 M 文件的每一行显示一条消息。例如常见的 M-Lint 消息是" 某变量已定义但在 M 文件中从未使用"。

你可以用三种方式检查代码问题，三者报告相同的消息：

- 在编辑器中工作时持续检查代码。查看 M-Lint 消息并基于消息修改文件。消息自动持续更新，可以看到你的修改是否解决了消息指出的问题。有些消息提供扩展信息、自动代码修正或两者兼有。
- 为现有 MATLAB 代码文件运行报告：在编辑器中打开文件，选择 Tools > Code Analyzer > Show Code Analyzer Report。
- 为文件夹中所有文件运行报告：在当前文件夹浏览器中点击 Actions 按钮，选择 Reports > Code Analyzer Report。

对每条消息，查看消息及相关代码，按以下流程基于消息修改代码：

- 点击行号在编辑器/调试器中打开 M 文件并定位到该行。
- 查看报告中的 M-Lint 消息，基于消息修改 M 文件中的代码。
- 注意：某些情况下不应基于 M-Lint 消息做任何修改——M-Lint 消息不适用于该特定场景。M-Lint 不能对每种场景提供完美信息。
- 保存 M 文件。若做了可能引入错误的重大修改，考虑保存为不同文件名——这样在解决更新文件的问题时可以参照原文件。
- 若不确定某条消息的含义或该改什么，用帮助浏览器查找相关主题。

还可以用 `mlint` 函数获取 M-Lint 消息。在命令窗口输入 `help mlint` 了解该函数的更多信息。该工具的更多内容见在线文档 [3]。

13.2.2 B.2.2　调试器

MATLAB 编辑器、图形化调试器与 MATLAB 调试函数是纠正运行时问题的利器。它们能访问函数工作区、查看或修改其中的值；还可以设置与清除断点——断点是在文件中临时暂停执行的指示器。在断点处暂停时，可以更改工作区上下文、查看函数调用栈、

逐行执行文件中的代码。

调试有两个重要技术：**断点**与**单步**。设置断点暂停函数的执行，让你检查认为可能出问题的位置的值。调试时还可以单步执行 M 文件，在想检查值的位置暂停。

M 文件中可以设置三种基本断点：

- **标准断点**：在 M 文件的指定行停止。
- **条件断点**：仅在指定条件下在 M 文件的指定行停止。
- **错误断点**：任何 M 文件产生指定类型的警告、错误、NaN 或无穷值时停止。

MATLAB 忙碌时（例如正在运行 M 文件，除非该 M 文件已在断点处暂停）不能设置断点。程序暂停时可以查看工作区中任何变量的当前值——从而在你想看某行代码是否产生期望结果时检查值。若结果符合预期，继续运行或单步到下一行；若不符合预期，则该行或之前的某行含有错误。调试时可以更改当前工作区中变量的值，看新值是否产生期望结果：程序暂停时在命令窗口、工作区浏览器或数组编辑器中给变量赋新值，然后继续运行或单步执行。若新值没有产生期望结果，程序还有别的问题。

除使用编辑器（图形用户界面）外，还可以在命令窗口用调试函数调试 MATLAB 文件，或两种方式交替使用。更多内容见在线文档 [4]。

13.2.3 B.2.3 分析器

性能分析（profiling）是度量程序执行各函数所花时间的方法。用 MATLAB 分析器（Profiler）可以识别代码中哪些函数最耗时间，进而确定为什么调用它们、寻找最小化使用的方法。判断特定函数被调用的次数是否合理往往很有帮助。由于程序通常有多层，你的代码可能并不显式调用最耗时的函数——而是你代码中的函数在调用往下好几层的其他耗时函数。此时确定你的哪些函数应对这些调用负责很重要。

性能分析帮助发现可以通过以下方式解决的性能问题：

- 避免不必要的计算（可能因疏忽而起）；
- 更改算法以避免高代价函数；
- 避免重复计算——存储结果供将来使用。

当大部分时间花在对少量内置函数的调用上时，代码可能已经优化到你所能期望的极限。可以用以下任一方式打开分析器：

- 从 MATLAB 桌面选择 Desktop → Profiler；
- 从 MATLAB 编辑器/调试器的菜单选择 Tools → Open Profiler；
- 在命令历史窗口选中一条或多条语句，右键查看上下文菜单，选择 Profile Code；
- 在命令窗口输入 `profile viewer`。

打开分析器后按以下步骤分析 M 文件或代码行（如图 B.1）：

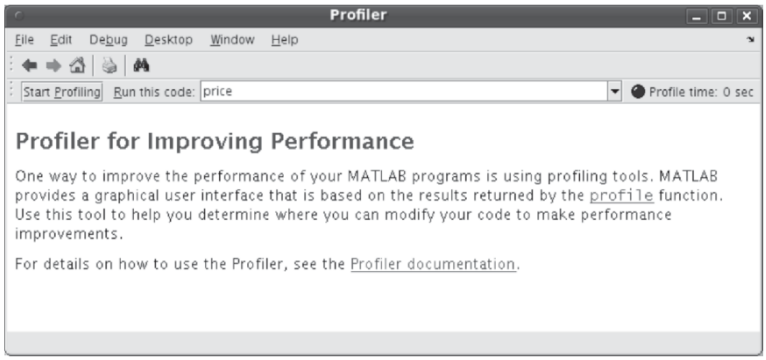


图 13.1　分析器窗口

1. 在分析器的 Run this code 字段输入要运行的语句。
2. 点击 Start Profiling（或输入语句后按 Enter）。
3. 分析完成后，Profile Summary 报告出现在分析器窗口。

更多内容见在线文档 [5]。

13.3 B.3　系统对象

系统对象（System Objects）是 MATLAB 中类的特化，定义了一组让初始化、运行时操作与拆除都变简单的方法。

类本身基本上是一组共享一组变量（称为参数）的函数。这些参数在类内定义并有确定的作用域。虽然系统对象实现了许多方法，但用户应理解的三个主要方法是 `setupImpl`、`stepImpl` 与 `releaseImpl`。写法如下：

```
methods (Access = protected)
    function setupImpl(obj)
        % Set parameter
        obj.x = 1;
    end
end
```

`setupImpl` 用于初始化参数、执行系统对象生命周期内需要的计算。`stepImpl` 是运行时函数（方法），通常被调用多次、消耗或产生输入/输出。最后 `releaseImpl` 用于拆除对象——清除其内存或执行关闭动作。例如对象要向文件写数据时，该方法被调用时会关闭文件。

调用系统对象的 operator 或 step 方法时，首次调用会实际先调用 `setupImpl` 方法。此时系统对象被视为**锁定**（locked）。然后调用 `stepImpl`。后续的 operator 或 step 调用只调用 `stepImpl`。要解锁对象必须调用 `releaseImpl`。解锁后，下一次 operator 或 step 调用会再次调用 `setupImpl`。脚本演示如下：

```
% Instantiate object
ss = dsp.SignalSource;
% setupImpl and stepImpl are called
data = ss();
% stepImpl is only called
data = ss();
% Object is unlocked
ss.release();
% setupImpl and stepImpl are called
data = ss.step();
```

系统对象极其适合封装大量功能、以及需要维护状态的场合。例如滤波器需要状态，是系统对象的显然用例。组成 MATLAB 的各工具箱用与其缩写相关的扩展名标识其系统对象。例如来自 Communication Systems Toolbox 的系统对象有 comm 扩展：`comm.AGC`、`comm.CarrierSynchronizer`、`comm.EVM` 等。DSP Systems Toolbox 的例子有 `dsp.FIRDecimator`、`dsp.SpectrumAnalyzer` 与 `dsp.SignalSource`。系统对象的更多内容（实现与使用的详尽细节）见 MathWorks 文档。

13.4 参考文献

[1] The MathWorks, *MATLAB Documentation*. http://www.mathworks.com/help/techdoc/.

[2] University of Michigan, *Simulink Basics Tutorial*. http://www.engin.umich.edu/group/ctm/working/mac/simulink_basics/.

[3] The MathWorks, “Avoid Mistakes While Editing Code.” http://www.mathworks.com/help/techdoc/matlab_env/brqxeeu-151.html.

[4] The MathWorks, “Debugging Process and Features.” http://www.mathworks.com/help/techdoc/matlab_env/brqxeeu-175.html.

[5] The MathWorks, “Profiling for Improving Performance.” http://www.mathworks.

com/help/techdoc/matlab_env/f9-17018.html.

第 14 章 附录 C　均衡器推导

14.1 C.1　线性均衡器

设收发信机模型中，信息源产生施加于无限冲激串的幅度值 I_n；即

$$s(t) = \sum_{n=-\infty}^{\infty} I_n \delta(t - nT),$$

其中 $\delta(t)$ 是狄拉克冲激函数。对信息信号 $s(t)$ 施加发射脉冲成形滤波器 $h_T(t)$，得到发射机输出信号：

$$v(t) = \sum_{n=-\infty}^{\infty} I_n h_T(t - nT),$$

它随后经过由信道滤波器 $h_C(t)$ 与加性高斯白噪声信号 $z(t)$ 描述的传播信道。信道滤波器的输出为：

$$p(t) = \sum_{n=-\infty}^{\infty} I_n h(t - nT),$$

其中 $h(t) = h_T(t) * h_C(t)$ 是信道冲激响应。接收机截获的信号表示为

$$r(t) = p(t) + z(t) = \sum_{n=-\infty}^{\infty} I_n h(t - nT) + z(t).$$

在接收端，我们想找到均方误差（MSE）的表达式并将其最小化。为此选择接收滤波器 $h_R(t)$ 与 $h(t)$ 匹配，得到最优结果：

$$h_R(t) = h^*(-t),$$

这使得接收滤波器的输出等于

$$y(t) = p(t) * h^*(-t) + z(t) * h^*(-t) = \sum_{n=-\infty}^{\infty} I_n g(t - nT) + z(t) * h^*(-t),$$

其中 $g(t) = h(t) * h_R(t)$，且

$$x(t) = \varphi_h(t) = h(t) * h^*(-t) = \int_{-\infty}^{\infty} h(t+\tau)h^*(\tau)d\tau.$$

设 $v(t) = z(t) * h^*(-t)$，则得到：

$$y(t) = \sum_{n=-\infty}^{\infty} I_n x(t - nT) + v(t).$$

现在对接收滤波器的输出采样，使得

$$y_k = y(kT) = \sum_{n=-\infty}^{\infty} I_n x((k-n)T) + v(kT) = \sum_{n=-\infty}^{\infty} I_n x_{k-n} + v_k.$$

注意 x_k 称为信道自相关。此外，由于经过 $h^*(t)$ 滤波，v_k 采样不是白的——即 $E\{v_k v_{k+n}\} \neq \delta(n)$。处理这种有色噪声有几条路。第一种是求噪声序列 v_k 的 $E\{v_k v_l^*\}$。此法中我们知道 $v_k(t)$ 是高斯的（因为 $z(t)$ 是高斯的）；还知道 $z(t)$ 是白的，即 $S_z(f) = N_0$。于是若 $E\{z^*(s)z(t)\} = N_0\delta(t-s)$，可解 $E\{v_k v_l^*\}$：

$$\begin{aligned} E\{v_k v_l^*\} &= \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} h(t-lT)h^*(s-kT)E\{z^*(s)z(t)\}\,ds\,dt \\ &= N_0 \int_{-\infty}^{\infty} h(t-lT)h^*(t-kT)\,dt \\ &= N_0 \int_{-\infty}^{\infty} h(t+|k-l|T)h^*(t)\,dt. \end{aligned}$$

由于

$$x_k = \int_{-\infty}^{\infty} h^*(t)h(t+kT)\,dt,$$

这给出 $E\{v_k v_l^*\}$ 的表达式：

$$E\{v_k v_l^*\} = N_0 x_{k-l},$$

可以写成 v_k 的自相关函数与功率谱密度；即

$$R_v(k) = E\{v_n v_{k+n}^*\} = N_0 x_k, \qquad S_v(z) = \mathcal{Z}\{R_v(k)\} = N_0 X(z).$$

处理有色噪声的第二种方法是实现所谓的**白化滤波器**。此时我们试图逆转接收滤波器对噪声信号 $z(t)$ 的影响。换言之：既然 v_k 的功率谱密度为 $S_v(z) = N_0 X(z)$，我们最终想让白化后的噪声功率谱密度只等于 N_0。为此假设 x_k 的 z 变换 $X(z)$ 可表示为：

$$X(z) = F(z)F^*(1/z^*).$$

于是，用传递函数为 $1/F^*(1/z^*)$ 的白化滤波器，输出的功率谱密度应为：

$$S_n(z) = \frac{1}{F(z)F^*(1/z^*)} S_v(z) = \frac{1}{X(z)} N_0 X(z) = N_0,$$

产生白色的输出噪声信号。

14.2 C.2 迫零均衡器

设接收机的离散时间模型为：

$$w_k = \sum_{n=-\infty}^{\infty} I_n f_{k-n} + n_k, \tag{C.20}$$

其中 w_k 是白化滤波器的输出信号、f_k 是白化滤波器的冲激响应、n_k 是功率谱密度为 N_0 的白化噪声信号。

在**迫零均衡器**（ZFE）中，我们选择传递函数 $C(z)$ 使每个 ISI 补偿项 $q_k = \delta_k$——意味着不存在 ISI，均衡后的输出 $I_k + n'_k$ 随后被量化，对 $I_k = \pm 1$ 产生差错概率 $P_e = Q(1/\sigma')$。虽然 P_e 常用作通信系统性能度量，但均衡器设计场合我们将用最小均方误差（MMSE）作度量，即 $E\{|I_k - \hat{I}_k|^2\}$。

取 (C.20) 的 z 变换得到：

$$W(z) = I(z)F(z) + N(z)$$

经 ZFE 均衡器滤波后，输出等于

$$W(z)C(z) = I(z)F(z)C(z) + N(z)C(z),$$

其中 $Q(z) = F(z)C(z)$。ZFE 的目标是产生输出

$$\mathcal{Z}^{-1}\{W(z)C(z)\} = I_k * q_k + n'_k,$$

其中 $q_k = \delta_k$（信号无 ISI）。因此该表达式中的 ISI 项是 $F(z)C(z)$，我们希望它等于 $Q(z) = F(z)C(z) = 1$（对应 $q_k = \delta_k$）。整理得 ZFE 应为 $C(z) = 1/F(z)$。

14.3 C.3　判决反馈均衡器

虽然 ZFE 滤波器概念上直接，但几乎总有无限多个抽头——使现实应用中无法实现。此外，白化滤波器输出经 ZFE 均衡后，均衡信号中包含的任何噪声贡献都不再是白的。于是存在更容易在硬件中实现的其他均衡器设计。其中之一是**判决反馈均衡器**（DFE）——由两个滤波器组成：前向滤波器 $a(z)$ 与反馈滤波器 $b(z) - 1$。

前向滤波器 $a(z)$ 是反因果的，形式如下：

$$a(z) = a_{-1}z + a_{-2}z^2 + a_{-3}z^3 + a_{-4}z^4 + \ldots,$$

而反馈滤波器 $b(z)$ 是因果滤波器，形式为：

$$b(z) - 1 = b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3} + b_4 z^{-4} + \ldots$$

于是可以设计 $a(z)$ 与 $b(z)$ 使得

$$a(z) = CF^*(1/z^*)$$

$$b(z) = F(z)$$

$$F(z)F^*(1/z^*) = X(z) + N_0.$$

第 15 章 附录 D　三角恒等式

$$e^{\pm j\theta} = \cos(\theta) \pm j\sin(\theta)$$

$$\cos(\theta) = \frac{1}{2}\left[e^{j\theta} + e^{-j\theta}\right]$$

$$\sin(\theta) = \frac{1}{2j}\left[e^{j\theta} - e^{-j\theta}\right]$$

$$\sin^2(\theta) + \cos^2(\theta) = 1$$

$$\cos^2(\theta) - \sin^2(\theta) = \cos(2\theta)$$

$$\cos^2(\theta) = \frac{1}{2}\left[1 + \cos(2\theta)\right]$$

$$\sin^2(\theta) = \frac{1}{2}\left[1 - \cos(2\theta)\right]$$

$$2\sin(\theta)\cos(\theta) = \sin(2\theta)$$

$$\sin(\alpha \pm \beta) = \sin(\alpha)\cos(\beta) \pm \cos(\alpha)\sin(\beta)$$

$$\cos(\alpha \pm \beta) = \cos(\alpha)\cos(\beta) \mp \sin(\alpha)\sin(\beta)$$

$$\tan(\alpha \pm \beta) = \frac{\tan(\alpha) \pm \tan(\beta)}{1 \mp \tan(\alpha)\tan(\beta)}$$

$$\sin(\alpha)\sin(\beta) = \frac{1}{2}\left[\cos(\alpha - \beta) - \cos(\alpha + \beta)\right]$$

$$\cos(\alpha)\cos(\beta) = \frac{1}{2}\left[\cos(\alpha - \beta) + \cos(\alpha + \beta)\right]$$

$$\sin(\alpha)\cos(\beta) = \frac{1}{2}\left[\sin(\alpha - \beta) + \sin(\alpha + \beta)\right]$$